

# Data Mining (EECS 6412)

Introduction to Data Mining  
Instructor: Aijun An  
EECS, York University

## Outline of Introduction

- ▶ Why data mining?
- ▶ What is data mining?
- ▶ Process of data mining or KDD
- ▶ Course information
- ▶ Overview of data mining
  - ▶ What kind of data to mine from?
  - ▶ What kind of patterns to mine? (Data Mining Tasks)
  - ▶ Data mining R&D issues

2

## Why Data Mining?

- ▶ Data mining started around 1990
- ▶ People were from
  - ▶ Database systems, machine learning (ML), and statistics
- ▶ ML at that time
  - ▶ Worked only on small datasets
  - ▶ Focused on classification, regression and clustering tasks
- ▶ There was need to
  - ▶ Handle large amounts of data
  - ▶ Learn other types of useful patterns

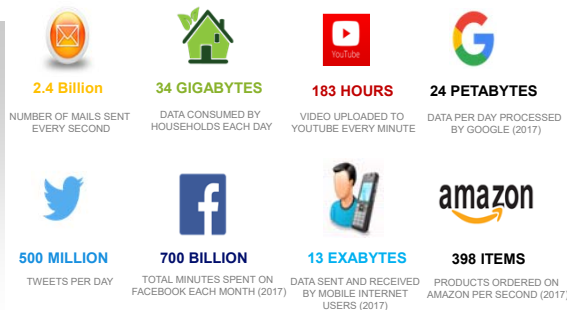
3

## Why Data Mining Became Popular?

- ▶ Data flood
  - ▶ Vast amounts of data are produced every day
- ▶ Data are generated by:
  - ▶ Bank, telecom, stores, other business transactions ...
  - ▶ Scientific fields: astronomy, biology, etc
  - ▶ Web and social media (text, images, audio, video, etc.)
  - ▶ Internet of Things (IoT)
    - ▶ In 2020, 31 billion IoT devices exist (source: Statista).
- ▶ Big Data Era

4

## Big Data Examples



5

## Characteristics of Big Data

- ▶ 5Vs in Big Data:
  - ▶ Volume
  - ▶ Velocity
  - ▶ Variety
  - ▶ Veracity
  - ▶ Value

6

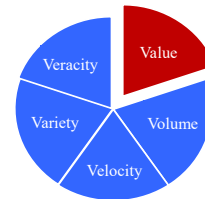
## Value

- ▶ Valuable information hidden in data
- ▶ For example:
  - ▶ What types of credit card transactions are fraud?
  - ▶ What items are often bought together?
  - ▶ What types of customers like a particular product?
  - ▶ What news or ads should be recommended to an online news reader?
  - ▶ Who are the most influential people in a social network?
  - ▶ What topics are people discussing in social media?
  - ▶ What types of patients react positively to a new drug?

7

## Purpose of Data Mining

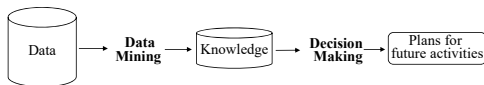
- ▶ Data mining is a tool to get the value out of data.



8

## What is Data Mining?

- ▶ Mining knowledge from data
  - ▶ Data = raw information
  - ▶ Knowledge = patterns or models 'behind' the data



9

## What Is Data Mining?

- ▶ Data mining [Han, 2001]
  - ▶ process of extracting interesting (*non-trivial, implicit, previously unknown* and *potentially useful*) knowledge or patterns from data in *large* databases.
- ▶ Objectives of data mining:
  - ▶ Discover knowledge that characterizes general properties of data
  - ▶ Discover patterns on the previous data in order to make predictions on future data

10

## Example 1

- ▶ Data about investors

| Customer ID | Account Type | Margin Account | Transaction Method | Trades/Month | Sex | Age   | Favorite Recreation | Annual Income |
|-------------|--------------|----------------|--------------------|--------------|-----|-------|---------------------|---------------|
| 1005        | Joint        | No             | Online             | 12.5         | F   | 30-39 | Tennis              | 40-59K        |
| 1013        | Custodial    | No             | Broker             | 0.5          | F   | 50-59 | Skiing              | 80-99K        |
| 1245        | Joint        | No             | Online             | 3.6          | M   | 20-29 | Golf                | 20-39K        |
| 2110        | Individual   | Yes            | Broker             | 22.3         | M   | 40-49 | Fishing             | 40-59K        |
| 1001        | Individual   | Yes            | Online             | 5.0          | M   | 30-39 | Golf                | 60-79K        |

- ▶ Possible business questions
  - ▶ Can we develop a general characterization/profile of different investor types? (characterization/frequent patterns)
  - ▶ What characteristics distinguish between Online and Broker investors? (classification)
  - ▶ Can we develop a model which will predict the average trades/month for a new investor? (regression)

11

## Example 2

### Hypothetical Training Data for Disease Diagnosis

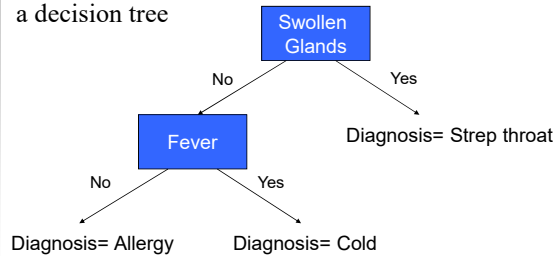
| Patient ID# | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis    |
|-------------|-------------|-------|----------------|------------|----------|--------------|
| 1           | Yes         | Yes   | Yes            | Yes        | Yes      | Strep throat |
| 2           | No          | No    | No             | Yes        | Yes      | Allergy      |
| 3           | Yes         | Yes   | No             | Yes        | No       | Cold         |
| 4           | Yes         | No    | Yes            | No         | No       | Strep throat |
| 5           | No          | Yes   | No             | Yes        | No       | Cold         |
| 6           | No          | No    | No             | Yes        | No       | Allergy      |
| 7           | No          | No    | Yes            | No         | No       | Strep throat |
| 8           | Yes         | No    | No             | Yes        | Yes      | Allergy      |
| 9           | No          | Yes   | No             | Yes        | Yes      | Cold         |
| 10          | Yes         | Yes   | No             | Yes        | Yes      | Cold         |

- In this example dataset, attributes correspond to Symptoms, and Diagnosis.
- The natural question is to predict the Diagnosis (class) [the Output variable] from the symptoms [the input variables].

12

## Example 2 (Cont'd)

Pattern discovered:  
a decision tree



13

## Example 2 (Cont'd)

Using Decision Tree for Prediction

Data Instances with an Unknown Classification

| Patient ID# | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis |
|-------------|-------------|-------|----------------|------------|----------|-----------|
| 11          | No          | No    | Yes            | Yes        | Yes      | ?         |
| 12          | Yes         | Yes   | No             | No         | Yes      | ?         |
| 13          | No          | No    | No             | No         | Yes      | ?         |

What are the predicted diagnoses?

14

## Example 3

Part of training data for Iris plants classification



| Sepal Length | Sepal Width | Petal Length | Petal Width | Class            |
|--------------|-------------|--------------|-------------|------------------|
| 5.1          | 3.5         | 1.4          | 0.2         | Iris Setosa      |
| 4.9          | 3           | 1.4          | 0.2         | Iris Setosa      |
| 4.7          | 3.2         | 1.3          | 0.2         | Iris Setosa      |
| 5.4          | 3.9         | 1.7          | 0.4         | Iris Setosa      |
| 4.8          | 3           | 1.4          | 0.1         | Iris Setosa      |
| 7            | 3.2         | 4.7          | 1.4         | Iris Versicolour |
| 6.4          | 3.2         | 4.5          | 1.5         | Iris Versicolour |
| 5.5          | 2.3         | 4            | 1.3         | Iris Versicolour |
| 5            | 2           | 3.5          | 1           | Iris Versicolour |
| 5.4          | 3           | 4.5          | 1.5         | Iris Versicolour |
| 6            | 3           | 4.8          | 1.8         | Iris Virginica   |
| 6.9          | 3.1         | 5.4          | 2.1         | Iris Virginica   |
| 6.5          | 3           | 5.2          | 2           | Iris Virginica   |
| 6.2          | 3.4         | 5.4          | 2.3         | Iris Virginica   |
| 5.9          | 3           | 5.1          | 1.8         | Iris Virginica   |

15

## Generated Rules for Iris Plants Classification

Rules for Class = Iris Setosa (1 rule)

**Rule 1:** (petal\_length <= 1.9) → (class = Iris Setosa)

Rules for Class = Iris Versicolour (3 rules)

**Rule 1:** (1.9 < petal\_length <= 4.9) ∧ (petal\_width <= 1.6) → (class = Iris Versicolour)

**Rule 2:** (sepal\_length > 5.8) ∧ (petal\_length <= 5.1) ∧ (petal\_width <= 1.7) → (class = Iris Versicolour)

**Rule 3:** (sepal\_length > 5.8) ∧ (sepal\_width > 3.0) ∧ (petal\_length <= 4.8) → (class = Iris Versicolour)

Rules for Class = Iris Virginica (4 rules)

**Rule 1:** (petal\_length > 4.8) ∧ (petal\_width > 1.7) → (class = Iris Virginica)

**Rule 1:** (sepal\_length > 6.2) ∧ (petal\_length > 5.0) → (class = Iris Virginica)

Summary: Total number of rules = 8

Average length of rules = 2.75

Training accuracy = 99.33%

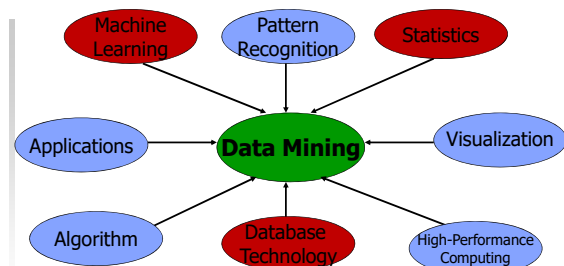
16

## Example 4: Recommendation

- ▶ A person buys a book (product) at amazon.com.
- ▶ Task:
  - ▶ Recommend other books (products) this person is likely to buy
- ▶ One method:
  - ▶ Use frequent pattern mining on transaction histories and may find:
    - ▶ Customers who bought "Advances in Knowledge Discovery and Data Mining", also bought "Data Mining: Concepts and Techniques"
- ▶ Recommendation program is quite successful

17

## Data Mining: Confluence of Multiple Disciplines



18

## Some Related Fields

- ▶ Machine Learning
  - ▶ Subfield of AI
- ▶ Exploratory data analysis
  - ▶ Subfield of Statistics
- ▶ Data warehousing and OLAP (Database)
  - ▶ Data summarization and aggregation
- ▶ Data mining (in comparison to ML)
  - ▶ inherits techniques from ML and statistics
  - ▶ emphasizes processing very large data sets
  - ▶ finds more types of patterns

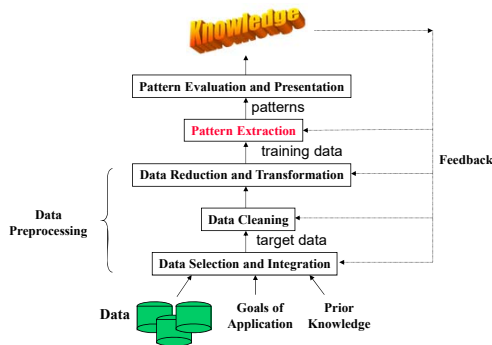
19

## Alternative Name: KDD

- ▶ Knowledge discovery in databases (KDD)
  - ▶ Used by AI, Machine Learning Community since 1989
- ▶ Data mining
  - ▶ Used by DB, business people since 1990
- ▶ Two names are now used interchangeably

20

## Process of Data Mining and KDD



21

## Course Objectives

- ▶ To provide an introduction to data mining
  - ▶ To understand the fundamental concepts, principles, algorithms, and applications of data mining.
- ▶ To gain hands-on experience by
  - ▶ implementing some data mining algorithms.
  - ▶ using data mining software to mine patterns from real world data
- ▶ To study advanced topics in data mining
- ▶ To prepare yourself for
  - ▶ pursuing research in data mining,
  - ▶ using data mining in your research or pursuing data analytics jobs

22

## Course Content

- ▶ Basics of data mining
  - ▶ Pattern mining
    - ▶ Frequent pattern and association rule mining
    - ▶ Sequential pattern mining
  - ▶ Mining classification models
    - ▶ Data preprocessing (feature selection, normalization, missing value handling, etc.)
    - ▶ Decision tree learning
    - ▶ Bayesian classification (naïve Bayes and Bayesian networks)
    - ▶ Neural networks and introduction to deep learning
    - ▶ K-nearest neighbor method
    - ▶ Learning from imbalanced data
  - ▶ Clustering analysis
- ▶ Text classification
- ▶ Graph mining
- ▶ Advanced topics (e.g., recommendation systems, multi-modal sentiment analysis, knowledge graphs, causality discovery, etc.)

23

## Reference Books and Materials

- ▶ Reference books
  - ▶ Jiawei Han, Jian Pei and Hanghang Tong, *Data Mining: Concepts and Techniques*, Morgan Kaufmann, 4th Edition, 2023.
  - ▶ Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar, *Introduction to Data Mining*, Second Edition, Pearson, 2019.
  - ▶ Charu C. Aggarwal, *Data Mining, The Textbook*, Springer, 2015.
  - ▶ Ian H. Witten and Eibe Frank, *Data Mining: Practical Machine Learning Tools and Techniques (Second Edition)*, Morgan Kaufmann, 2005.
- ▶ Some conference/journal papers
  - ▶ (To be) posted on the course web site

24



## Evaluation and Grading

- ▶ Assignments (20%)
  - ▶ 2 assignments
- ▶ Project (45%)
- ▶ Final exam (25%)
- ▶ Participation (10%)

25

## Project

- ▶ Should be done in a group of 2 students.
- ▶ Pick a recent advanced topic in data mining
  - ▶ Topic list will be released in the second week
- ▶ Conduct a survey on the topic
  - ▶ Present the survey to the class in November (10%)
- ▶ Implement and evaluate a (SOTA) method as a baseline
  - ▶ Submit code and a short report (10%)
- ▶ Design your solution to the problem
  - ▶ E.g., a reasonable improvement to the baseline
- ▶ Implement your solution
- ▶ Evaluate your solution by comparing it to baseline(s)
- ▶ Write a final project report (in a research paper format) (20%)
- ▶ Present the project to the class (7-10 minutes) (5%)

26

## Survey Presentation

- ▶ Make a 30-minute presentation to the class
  - ▶ What is the topic area about?
    - ▶ Motivation, objectives, and problem definition
  - ▶ What are the categories of existing techniques?
  - ▶ Briefly present a representative method in at least two categories.
  - ▶ What are the advantages of each category or technique?
  - ▶ What are the limitations of each category or technique?
  - ▶ What are the open (unresolved) issues in the research?
- ▶ Purpose:
  - ▶ Make the class understand the topic areas and techniques for solving the problem
  - ▶ Develop presentation skills

27

## Instructor Information

- ▶ Office hours
  - ▶ Thursdays: 1:00pm-2:00pm
  - ▶ Location: LAS 2048
- ▶ Email: [aan@yorku.ca](mailto:aan@yorku.ca)
- ▶ Course eClass site:
  - ▶ <https://eclass.yorku.ca/course/view.php?id=144958>

28

## Outline of Introduction

- ▶ Why data mining?
- ▶ What is data mining?
- ▶ Process of data mining or KDD
- ▶ Course information
- ▶ Overview of data mining
  - ▶ What kind of data to mine from?
  - ▶ What kind of patterns to mine? (Data Mining Tasks)
  - ▶ Data mining R&D issues

29

## What Kind of Data?

- ▶ Tabular data
- ▶ Transactional data
- ▶ Text data
- ▶ Time-series data
- ▶ Sequence data
- ▶ Data streams
- ▶ Graphs
- ▶ Multimedia databases
- ▶ ...

30

## Tabular Data

### Structured data

- Table
- Examples
- Attributes
- Can be stored in
  - plain text files, or
  - relational databases.

Examples/  
Instances/  
Cases/  
Observations

Attributes/Features/Variables

| OUTLOOK  | TEMPERATURE | HUMIDITY | WINDY | PLAY       |
|----------|-------------|----------|-------|------------|
| sunny    | 85          | 85       | FALSE | Don't Play |
| sunny    | 80          | 90       | TRUE  | Don't Play |
| overcast | 83          | 78       | FALSE | Play       |
| rain     | 70          | 96       | FALSE | Play       |
| rain     | 68          | 80       | FALSE | Play       |
| rain     | 65          | 70       | TRUE  | Don't Play |
| overcast | 84          | 65       | TRUE  | Play       |
| sunny    | 72          | 95       | FALSE | Don't Play |
| sunny    | 69          | 75       | FALSE | Play       |
| rain     | 75          | 80       | FALSE | Play       |
| sunny    | 70          | 70       | TRUE  | Play       |
| overcast | 72          | 90       | TRUE  | Play       |
| overcast | 81          | 75       | FALSE | Play       |
| rain     | 71          | 80       | TRUE  | Don't Play |

An attribute can be:

- Symbolic (discrete, categorical)
- Numeric (continuous, integer)
- Ordinal (categorical with a value order, e.g., temperature with values of low, medium, high)

31

## Transactional Data

| Transaction-id | Itemset                                  |
|----------------|------------------------------------------|
| T100           | Milk, bread, beer, diaper                |
| T200           | Beer, cook, fish, potato, orange, diaper |
| ...            | ...                                      |

Some transactional databases also contain time stamp and the customer id for each transaction

### Patterns

- What kind of product combinations that customers like to buy together?

32

## Text Data

- Documents
  - articles, Web pages, blogs, tweets, emails, product specifications, reports, notes, reviews, etc.
- Structure
  - highly unstructured (news, stories, etc.), or
  - semistructured (HTML/XML documents, etc.)
- What can be discovered from a text database?
  - text classification models
  - keyword or content associations
  - summaries, topics, sentiments, emotions
  - ...

33

## Example of XML Data

```
<breakfast_menu>
  <food>
    <name>Belgian Waffles</name>
    <price>$5.95</price>
    <description>Two of our famous Belgian Waffles with plenty of real maple syrup</description>
    <calories>650</calories>
  </food>
  <food>
    <name>Strawberry Belgian Waffles</name>
    <price>$7.95</price>
    <description>Light Belgian waffles covered with strawberries and whipped cream</description>
    <calories>900</calories>
  </food>
  <food>
    <name>Berry-Berry Belgian Waffles</name>
    <price>$8.95</price>
    <description>Light Belgian waffles covered with an assortment of fresh berries and whipped cream</description>
    <calories>900</calories>
  </food>
  <food>
    <name>French Toast</name>
    <price>$4.50</price>
    <description>Thick slices made from our homemade sourdough bread</description>
    <calories>600</calories>
  </food>
  <food>
    <name>Homestyle Breakfast</name>
    <price>$6.95</price>
    <description>Two eggs, bacon or sausage, toast, and our ever-popular hash browns</description>
    <calories>950</calories>
  </food>
</breakfast_menu>
```

34

## Time Series Data

- A sequence of values that change with time
  - Daily water consumption data in a city
  - Data collected regarding the stock exchange
- Types of analysis
  - Trend analysis
    - To predict future values
  - Similarity search
    - Find similarities in sub-sequences, such as periodic patterns, recurring patterns

35

## Sequence Data

- Sequences of ordered objects or events (with or without concrete notation of time)
  - Bio-sequences
    - DNA, protein
  - Web log data
    - Click stream (web page traversal sequences)
  - Sequences of items bought by a customer
- Patterns
  - Frequent sequences, alignments of two bio-sequences, etc.

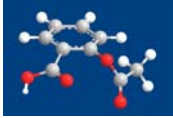
upstream → downstream  
 ... CCTTACTTATAATCTCATCTA ...  
 ... GGAAATGAATATTACGATCATAT ...  
 downstream ← upstream  
 An example DNA sequence and its paired strand  
 (Credit: Gerardo M Nava)

36

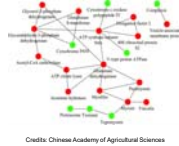
## Graph Data

- ▶ Data entities are connected, forming graphs

Aspirin



Biological Networks

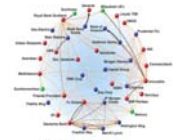


Credit: Chinese Academy of Agricultural Sciences

Social Networks



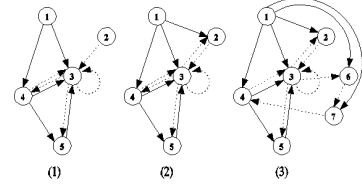
Financial Networks



37

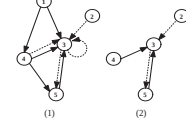
## Example 1 of Graph Dataset

Graph data set: containing one or more graphs (3 in this example)



Patterns can be found:

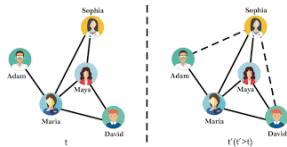
- Frequent subgraphs



38

## Example 2 of Graph Dataset

Social networks



Two graph mining tasks:

- Link prediction
- Node classification

39

## Other Graph Mining Tasks

- ▶ Find communities in social networks
- ▶ Find the most influential people in communities
- ▶ Anomaly detection
- ▶ Etc.

40

## Outline of Introduction

- ▶ Why data mining?
- ▶ What is data mining?
- ▶ Process of data mining or KDD
- ▶ Course information
- ▶ Overview of data mining
  - ▶ What kind of data to mine from?
  - ▶ *What kind of patterns to mine? (Data Mining Tasks)*
  - ▶ Data mining R&D issues

41

## Basic Data Mining Tasks

- ▶ **Predictive:** *Discover patterns on previous & current data in order to make predictions on future data*
  - ▶ Classification
  - ▶ Regression
  - ▶ Structured prediction
- ▶ **Descriptive:** *Discover knowledge that characterizes general properties of data*
  - ▶ Clustering
  - ▶ Concept characterization / Summarization
  - ▶ Association analysis (frequent itemsets, association rules)
  - ▶ Sequential pattern mining
- ▶ **Predictive or descriptive**
  - ▶ Time series analysis
  - ▶ Outlier detection
  - ▶ .....

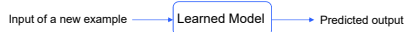
42

## Predictive Tasks

- ▶ A form of **Supervised Learning**
  - Training examples are **labeled** with target outputs

| Patient ID | Input/Independent/Condition attributes/features/variables |       |                |            |          | Target/Output/Decision attribute/variable |
|------------|-----------------------------------------------------------|-------|----------------|------------|----------|-------------------------------------------|
|            | Sore Throat                                               | Fever | Swollen Glands | Congestion | Headache | Diagnosis                                 |
| 1          | Yes                                                       | Yes   | Yes            | Yes        | Yes      | Strep throat                              |
| 2          | No                                                        | No    | No             | Yes        | Yes      | Allergy                                   |
| 3          | Yes                                                       | Yes   | No             | Yes        | No       | Cold                                      |
| 4          | Yes                                                       | No    | Yes            | No         | No       | Strep throat                              |
| 5          | No                                                        | Yes   | No             | Yes        | No       | Cold                                      |
| 6          | No                                                        | No    | No             | Yes        | No       | Allergy                                   |
| 7          | No                                                        | No    | Yes            | No         | No       | Strep throat                              |
| 8          | Yes                                                       | No    | No             | Yes        | Yes      | Allergy                                   |
| 9          | No                                                        | Yes   | No             | Yes        | Yes      | Cold                                      |
| 10         | Yes                                                       | Yes   | No             | Yes        | Yes      | Cold                                      |

- Learn a model from labeled data to make predictions on the future data



43

## Types of Predictive Tasks / Supervised Learning

- ▶ Classification
- ▶ Regression
- ▶ Structured prediction

44

## Classification

- ▶ Target variable is discrete
- ▶ Its values are considered as *classes (labels)*
  - The target attribute is also called *class attribute*
  - Examples:
    - Diagnosis: {strep throat, allergy, cold}
    - Image type: {cat, dog, car, truck, .....}
- ▶ Goal:
  - Learn a **model** from the **labeled** training data
  - Learned model is used to classify new examples

| Patient ID |             |       |                |            |          | Class attribute |
|------------|-------------|-------|----------------|------------|----------|-----------------|
|            | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis       |
| 1          | Yes         | Yes   | Yes            | Yes        | Yes      | Strep throat    |
| 2          | No          | No    | No             | Yes        | Yes      | Allergy         |
| 3          | Yes         | Yes   | No             | Yes        | No       | Cold            |
| 4          | Yes         | No    | Yes            | No         | No       | Strep throat    |
| 5          | No          | Yes   | No             | Yes        | No       | Cold            |
| 6          | No          | No    | No             | Yes        | No       | Allergy         |
| 7          | No          | No    | Yes            | No         | No       | Strep throat    |
| 8          | Yes         | No    | No             | Yes        | Yes      | Allergy         |
| 9          | No          | Yes   | No             | Yes        | Yes      | Cold            |
| 10         | Yes         | Yes   | No             | Yes        | Yes      | Cold            |

45

## Classification (Cont'd)

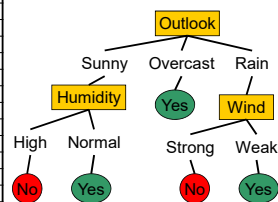
- ▶ Models can be in different types
  - Decision tree
  - A set of rules
  - Bayesian network
  - Support vector machine
  - Neural network
  - .....
- ▶ For different types of models, different types of learning algorithms were designed

46

## Example of Classification Models

- ▶ A decision tree for *PlayTennis*

| Day | Outlook  | Temp | Humid  | Wind   | PlayTennis |
|-----|----------|------|--------|--------|------------|
| D1  | Sunny    | Hot  | High   | Weak   | No         |
| D2  | Sunny    | Hot  | High   | Strong | No         |
| D3  | Overcast | Hot  | High   | Weak   | Yes        |
| D4  | Rain     | Mild | High   | Weak   | Yes        |
| D5  | Rain     | Cool | Normal | Weak   | Yes        |
| D6  | Rain     | Cool | Normal | Strong | No         |
| D7  | Overcast | Cool | Normal | Strong | Yes        |
| D8  | Sunny    | Mild | High   | Weak   | No         |
| D9  | Sunny    | Cool | Normal | Weak   | Yes        |
| D10 | Rain     | Mild | Normal | Weak   | Yes        |
| D11 | Sunny    | Mild | Normal | Strong | Yes        |
| D12 | Overcast | Mild | High   | Strong | Yes        |
| D13 | Overcast | Hot  | Normal | Weak   | Yes        |
| D14 | Rain     | Mild | High   | Strong | No         |



47

## Types of Predictive Tasks / Supervised Learning

- ▶ Classification
- ▶ Regression
- ▶ Structured prediction

48

## Regression

### ► Target variable is numeric, e.g.,

- House price
- Daily water consumption in a city
- Exam score

### ► Goal:

- Learn a model from the *labeled* training data to model the relationship between the input and target variables
- Learned model is used to predict a numeric value for a new example

Input/independent variables/predictors

| Hours | Prep exams | Exam score |
|-------|------------|------------|
| 1     | 1          | 76         |
| 2     | 3          | 78         |
| 2     | 3          | 85         |
| 4     | 5          | 88         |
| 2     | 2          | 72         |
| 1     | 2          | 69         |
| 5     | 1          | 94         |
| 4     | 1          | 94         |
| 2     | 0          | 88         |
| 4     | 3          | 92         |
| 4     | 4          | 90         |
| 3     | 3          | 75         |
| 6     | 2          | 96         |
| 5     | 4          | 90         |
| 3     | 4          | 82         |
| 4     | 4          | 85         |
| 6     | 5          | 99         |
| 2     | 1          | 83         |
| 1     | 0          | 62         |
| 2     | 1          | 76         |

Target attribute/variable (Dependent variable)

49

## Regression

### ► Linear vs. non-linear regression

- Relationship between independent and dependent variables are linear or non-linear

### ► Simple vs. multiple regression

- Simple regression
  - only one independent variable
- Multiple regression
  - multiple independent variables

50

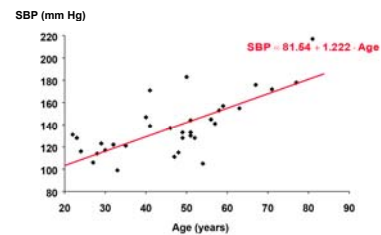
## Simple Linear Regression - Example

Data: Age and systolic blood pressure (SBP) among 33 adult women

| Age | SBP | Age | SBP | Age | SBP |
|-----|-----|-----|-----|-----|-----|
| 22  | 131 | 41  | 139 | 52  | 128 |
| 23  | 128 | 41  | 171 | 54  | 105 |
| 24  | 116 | 46  | 137 | 56  | 145 |
| 27  | 106 | 47  | 111 | 57  | 141 |
| 28  | 114 | 48  | 115 | 58  | 153 |
| 29  | 123 | 49  | 133 | 59  | 157 |
| 30  | 117 | 49  | 128 | 63  | 155 |
| 32  | 122 | 50  | 163 | 67  | 176 |
| 33  | 99  | 51  | 130 | 71  | 172 |
| 35  | 121 | 51  | 133 | 77  | 178 |
| 40  | 147 | 51  | 144 | 81  | 217 |

51

## Simple Linear Regression - Example



adapted from Colton T. Statistics in Medicine. Boston: Little Brown, 1974

52

## Multiple Linear Regression Example

Target attribute/variable

| Hours | Prep exams | Exam score |
|-------|------------|------------|
| 1     | 1          | 76         |
| 2     | 3          | 78         |
| 2     | 3          | 85         |
| 4     | 5          | 88         |
| 2     | 2          | 72         |
| 1     | 2          | 69         |
| 5     | 1          | 94         |
| 4     | 1          | 94         |
| 2     | 0          | 88         |
| 4     | 3          | 92         |
| 4     | 4          | 90         |
| 3     | 3          | 75         |
| 6     | 2          | 96         |
| 5     | 4          | 90         |
| 3     | 4          | 82         |
| 4     | 4          | 85         |
| 6     | 5          | 99         |
| 2     | 1          | 83         |
| 1     | 0          | 62         |
| 2     | 1          | 76         |

### ► Linear regression model learned:

$$\text{Exam score} = 67.67 + 5.56 * (\text{Hours}) - 0.60 * (\text{Prep exams})$$

### ► Prediction:

- A new student with Hours=4 and Prep exams = 2
- Predicted exam score:

$$\text{Exam score} = 67.67 + 5.56 * 4 - 0.60 * 2 = 88.71$$

53

## Types of Predictive Tasks / Supervised Learning

- Classification
- Regression
- Structured prediction

54

## Structured Prediction

- ▶ Outputs are structured objects, e.g.,
  - Sequences, trees, graphs, or other interrelated components
  - There are often relationships among output components
- ▶ Example:
  - Sequence-to-sequence problems
    - Both input and output are a sequence
    - E.g., machine translation (English → French)

55

## Structured Prediction

- Another example: Named entity recognition

Labeled Training data:

John **PERSON** joined **COMPANY** as a data scientist in **Milton Keynes**, after finishing his computer science degree at the **University of Lincoln**.  
  
John works in the **IT** department, which is the internal software development team at **Google** office in **Milton Keynes**. It focuses on developing a next-generation software product called **InsightMaker**, which uses AI to manage information intuitively and discover valuable insights.

Prediction on new data:

When **Sebastian Thrun** **PERSON** started working on self-driving cars at **Google** **COMPANY** in **2007** **DATE**, few people outside of the company took him seriously.

56

## Basic Data Mining Tasks

- ▶ Predictive:
  - ▶ Classification
  - ▶ Regression
  - ▶ Structured prediction
- ▶ **Descriptive:**
  - ▶ Clustering
  - ▶ Concept characterization / Summarization
  - ▶ Association analysis (frequent itemsets, association rules)
  - ▶ Sequential pattern mining
- ▶ Predictive or descriptive
  - ▶ Time series analysis
  - ▶ Outlier detection
  - ▶ .....

57

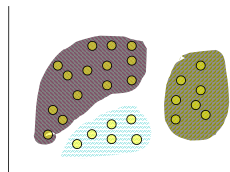
## Descriptive Data Mining Tasks

- ▶ Training data do not have output or target variables
- ▶ Goal is to
  - Extract hidden patterns or relationships in data
  - Discover knowledge that characterizes general properties of data
- ▶ A form of **unsupervised learning**
- ▶ Typical descriptive tasks
  - Clustering
  - Concept characterization / Summarization
  - Association analysis (frequent itemsets, association rules)
  - Sequential pattern mining

58

## Clustering

- ▶ Class label is unknown in the training data.
- ▶ Group data to form classes (clusters)
  - ▶ Unsupervised learning.
- ▶ Principle:
  - ▶ maximizing the intra-class similarity
  - ▶ minimizing the inter-class similarity
- ▶ Applications
  - ▶ Market/customer segmentation
  - ▶ Image segmentation
  - ▶ ....



59

## Concept Characterization

- ▶ Summarization of general properties of objects in a target group
- ▶ Example:
  - ▶ Characterize customers who spend more than \$1000 a year in the AllElectronic store and have excellent credit ratings.
  - ▶ Result: 40-50 years old, employed,

60

## Association Analysis

- ▶ Examples of association rules:
  - ▶  $\text{age}(X, "20..29") \wedge \text{income}(X, "20..29K") \rightarrow \text{buys}(X, "PC")$   
[support = 2%, confidence = 60%]
  - ▶  $\text{contains}(T, "computer") \rightarrow \text{contains}(T, "software")$   
[support=1%, confidence=75%] (T stands for a transaction)
- ▶ Widely used for market basket or transactional data analysis.
  - ▶ What products were often purchased together?

61

## Mining Sequential Patterns

- ▶ Find frequently occurring patterns in a sequence database.
  - ▶ Within 3 months, buy computer  $\rightarrow$  buy USB drive  $\rightarrow$  buy digital camera
- ▶ Applications
  - ▶ Sale campaign analysis
    - ▶ What are the subsequent purchases after buying a PC?
  - ▶ Web log analysis
  - ▶ DNA or protein analysis

62

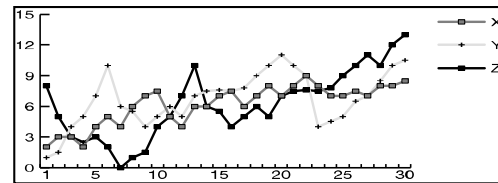
## Basic Data Mining Tasks

- ▶ Predictive:
  - ▶ Classification
  - ▶ Regression
- ▶ Descriptive:
  - ▶ Clustering
  - ▶ Concept characterization / Summarization
  - ▶ Association analysis (frequent itemsets, association rules)
  - ▶ Sequential pattern mining
- ▶ Predictive or descriptive
  - ▶ Time series analysis
  - ▶ Outlier detection

63

## Time Series Analysis

- ▶ Mine from time-series data to
  - ▶ Predict future values (Predictive)
  - ▶ Determine similar patterns over time (Descriptive)
- ▶ Example: Stock Market



64

## Outlier/Anomaly Detection

- ▶ Outlier
  - ▶ A data object that does not comply with the general behavior of the data
- ▶ Noise or exception?
  - ▶ One person's garbage could be another person's treasure
- ▶ Methods:
  - ▶ Clustering
  - ▶ Classification
  - ▶ Regression analysis
  - ▶ ...
- ▶ Useful in fraud detection, rare events analysis

65

## What will be covered in lectures

- ▶ Predictive:
  - ▶ Classification ✓
  - ▶ Regression (will cover regression trees ✓)
  - ▶ Structured prediction
- ▶ Descriptive:
  - ▶ Clustering ✓
  - ▶ Concept characterization / Summarization
  - ▶ Association analysis (frequent itemsets, association rules) ✓
  - ▶ Sequential pattern mining ✓
- ▶ Predictive or descriptive
  - ▶ Time series analysis
  - ▶ Outlier detection

66

## Questions

- ▶ What is the difference between classification and clustering?
- ▶ What is the difference between classification and regression?

67

## Outline of Introduction

- ▶ Why data mining?
- ▶ What is data mining?
- ▶ Process of data mining or KDD
- ▶ Course information
- ▶ Overview of data mining
  - ▶ What kind of data to mine from?
  - ▶ What kind of patterns to mine? (Data Mining Tasks)
- ▶ *Data mining R&D issues*

68

## Major Issues in Data Mining

- ▶ Efficiency
- ▶ Effectiveness
- ▶ User interaction
- ▶ Privacy preserving
- ▶ Fairness and biases
- ▶ Interpretability

69

## Efficiency

- ▶ Develop fast and scalable data mining algorithms (time-efficient)
  - ▶ Effective data structure
  - ▶ Parallel, distributed, and incremental mining
  - ▶ Heuristic and Approximation algorithms
- ▶ Memory-efficient
  - ▶ Develop algorithms that can handle huge amounts of data that cannot be held in RAM.

70

## Effectiveness

- ▶ Accuracy
  - ▶ How to develop models from data to make accurate predictions on future data
- ▶ Interestingness and actionability of discovered patterns
  - ▶ How to identify interesting patterns from a large number of patterns discovered
  - ▶ How to use the discovered patterns

71

## User Interaction

- ▶ Interactive mining
  - ▶ Incorporate background knowledge
  - ▶ Combine objective and subjective measures
- ▶ Visualization helps such an integration
  - ▶ Visualization of data
  - ▶ Presentation of mining results
  - ▶ Visualization of the mining process

72



## Data Mining with Privacy

- ▶ Data may contain private information
  - ▶ Data mining can invade privacy
- ▶ Technical solutions can limit privacy invasion
  - ▶ Replacing sensitive personal data with anon. ID
  - ▶ Alter the data so that real values are obscured
  - ▶ Multi-party computation – distributed data
  - ▶ ...
- ▶ Bayardo & Srikant, Technological Solutions for Protecting Privacy, IEEE Computer, Sep 2003

73

## Fairness and Bias in AI

- ▶ A model is considered *fair* if errors are distributed similarly across different groups of data
- ▶ Unfairness is usually caused by *bias* in
  - ▶ data, or/and
  - ▶ learning algorithm

74

## Interpretability

- ▶ How easy to interpret a learned model/pattern
- ▶ How easy for a human to understand the cause of a decision made by a model

75

## Next Class

- ▶ Frequent pattern and association rule mining
- ▶ Reading: Chapter 4 of Han's book (Pattern mining: basic concepts and methods)

76

# Data Mining (EECS6412)

---

## Frequent Pattern and Association Rule Mining

Aijun An

(Modified from Jiawei Han's slides)

---

# Outline

- ▶ Basic concepts of association rule learning
- ▶ Apriori algorithm
- ▶ FP-Growth algorithm
- ▶ Finding interesting rules.

# Why Mining Association Rules?

- ▶ Objective:

- ▶ Finding interesting co-occurring items (or objects, events) in a given data set.

- ▶ Examples:

- ▶ Given a dataset of transactions, each transaction is a list of items (purchased by a customer during a visit), you may find:
    - ▶  $\{\text{bread, butter}\} \rightarrow \{\text{milk}\}$  [support=2%, confidence=60%]
  - ▶ From a student database, you may find
    - ▶  $\text{major}(x, \text{"CS"}) \wedge \text{gpa}(x, \text{"A"}) \rightarrow \text{has\_taken}(x, \text{"DB"})$  [1%, 75%]

---

# Why Mining Association Rules? (Cont'd)

- ▶ Popular application: *Basket data analysis*
  - ▶ Place items frequently bought together close to each other to increase sales of these items.
  - ▶ ?  $\rightarrow$  *iPad*
    - ▶ What the store should do to boost sales of the particular product, i.e., *iPad*
  - ▶ *iPad*  $\rightarrow$  ?
    - ▶ What other products should the store stock up?
    - ▶ Make recommendations

# What Kind of Datasets?

## Transactional dataset TDS

| TID | Items                  |
|-----|------------------------|
| 100 | f, a, c, d, g, i, m, p |
| 200 | a, b, c, f, l, m, o    |
| 300 | b, f, h, j, o          |
| 400 | b, c, k, s, p          |
| 500 | a, f, c, e, l, p, m, n |

- ▶ Itemset: a set of items
- ▶ A **transaction** is a tuple (tid, X)
  - ▶ Transaction ID tid
  - ▶ Itemset X
- ▶ A **transactional dataset** is a set of transactions
  - ▶ In many cases, a transaction database can be treated as a set of itemsets (ignore TIDs)
- ▶ Association rule from TDS (relates two itemsets):
  - ▶  $\{a, c, m\} \rightarrow \{l\}$  [support=40%, confidence=66.7%]

# What Kind of Datasets? (*Contd*)

## Tabular data

| Day | Outlook  | Temp | Humid  | Wind   | PlayTennis |
|-----|----------|------|--------|--------|------------|
| D1  | Sunny    | Hot  | High   | Weak   | No         |
| D2  | Sunny    | Hot  | High   | Strong | No         |
| D3  | Overcast | Hot  | High   | Weak   | Yes        |
| D4  | Rain     | Mild | High   | Weak   | Yes        |
| D5  | Rain     | Cool | Normal | Weak   | Yes        |
| D6  | Rain     | Cool | Normal | Strong | No         |
| D7  | Overcast | Cool | Normal | Strong | Yes        |
| D8  | Sunny    | Mild | High   | Weak   | No         |
| D9  | Sunny    | Cool | Normal | Weak   | Yes        |
| D10 | Rain     | Mild | Normal | Weak   | Yes        |
| D11 | Sunny    | Mild | Normal | Strong | Yes        |
| D12 | Overcast | Mild | High   | Strong | Yes        |
| D13 | Overcast | Hot  | Normal | Weak   | Yes        |
| D14 | Rain     | Mild | High   | Strong | No         |

- ▶ An attribute-value pair
  - ▶ E.g., Outlook = sunny
- ▶ Record: a set of attribute-value pairs
- ▶ Table: a set of records

- ▶ Association rule from Table (relates two sets of attribute-value pairs):  
(Outlook=sunny) $\wedge$ (Temp=hot)  $\rightarrow$  (PlayTennis=No) [support=14%,  
confidence=100%]

# Definition of Association Rule

- ▶ An association rule is of the form:

$X \rightarrow Y$  [support, confidence]  
where      antecedent      consequent

- ▶  $X \subset I$ ,  $Y \subset I$ ,  $X \cap Y = \emptyset$  and  $I$  is a **set** of items (or attribute-value pairs).
- ▶ **support**: probability that a transaction (or a record) contains  $X$  and  $Y$ , i.e.,

$$\text{support}(X \rightarrow Y) = P(X \cup Y)$$

- ▶ **confidence**: conditional probability that a transaction (or a record) having  $X$  also contains  $Y$ , i.e.,

$$\text{confidence}(X \rightarrow Y) = P(Y|X)$$

- ▶ A rule associates one set of items (or attribute-value pairs) with another set of items (or attribute-value pairs)



# Support and Confidence: Example

$$\text{support}(X \rightarrow Y) = P(X \cup Y)$$

$$\text{confidence}(X \rightarrow Y) = P(Y|X)$$

| Transaction ID | Items Bought |
|----------------|--------------|
| 2000           | A,B,C        |
| 1000           | A,C          |
| 4000           | A,D          |
| 5000           | B,E,F        |

Relative frequency is used to estimate the probability.

- ▶  $\{A\} \rightarrow \{C\}$  (50%, 66.7%)
- ▶  $\{C\} \rightarrow \{A\}$  (50%, 100%)
- ▶  $\{A, C\} \rightarrow \{B\}$  (25%, 50%)
- ▶  $\{A, B\} \rightarrow \{E\}$  (0%, 0%)

# Mining Association Rules

## ► Problem statement

Given

- a *minimum support* ( $min\_sup$ ), also called *support threshold*, and
- a *minimum confidence* ( $min\_conf$ ), also called *confidence threshold*,

find all association rules that satisfy both  $min\_sup$  and  $min\_conf$  from a data set  $D$ .

# Basic Concepts

- ▶ **Strong rules:**

An association rule is *strong* if it satisfies both  $min\_sup$  and  $min\_conf$ .

- ▶  **$k$ -itemset:** an itemset that contains  $k$  items.

- ▶ {computer, financial\_management\_software} is a 2-itemset.

- ▶ **Support count** of an itemset in data set  $D$ : number of transactions in  $D$  that contain the itemset.

- ▶ **Minimum support count** =  $min\_sup \times$  total number of transactions in a data set.

- ▶ **Frequent itemset** in a data set  $D$ : itemset whose support count in  $D$  is at least the minimum support count.

# How to Mine Association Rules

- ▶ A two-step process:
  - ▶ Find all frequent itemsets ---- the key step
  - ▶ Generate strong association rules from frequent itemsets.
- ▶ Example: given min\_sup=50% and min\_conf=50%

| Transaction ID | Items Bought |
|----------------|--------------|
| 2000           | A,B,C        |
| 1000           | A,C          |
| 4000           | A,D          |
| 5000           | B,E,F        |



| Frequent Itemset | Support |
|------------------|---------|
| {A}              | 75%     |
| {B}              | 50%     |
| {C}              | 50%     |
| {A, C}           | 50%     |

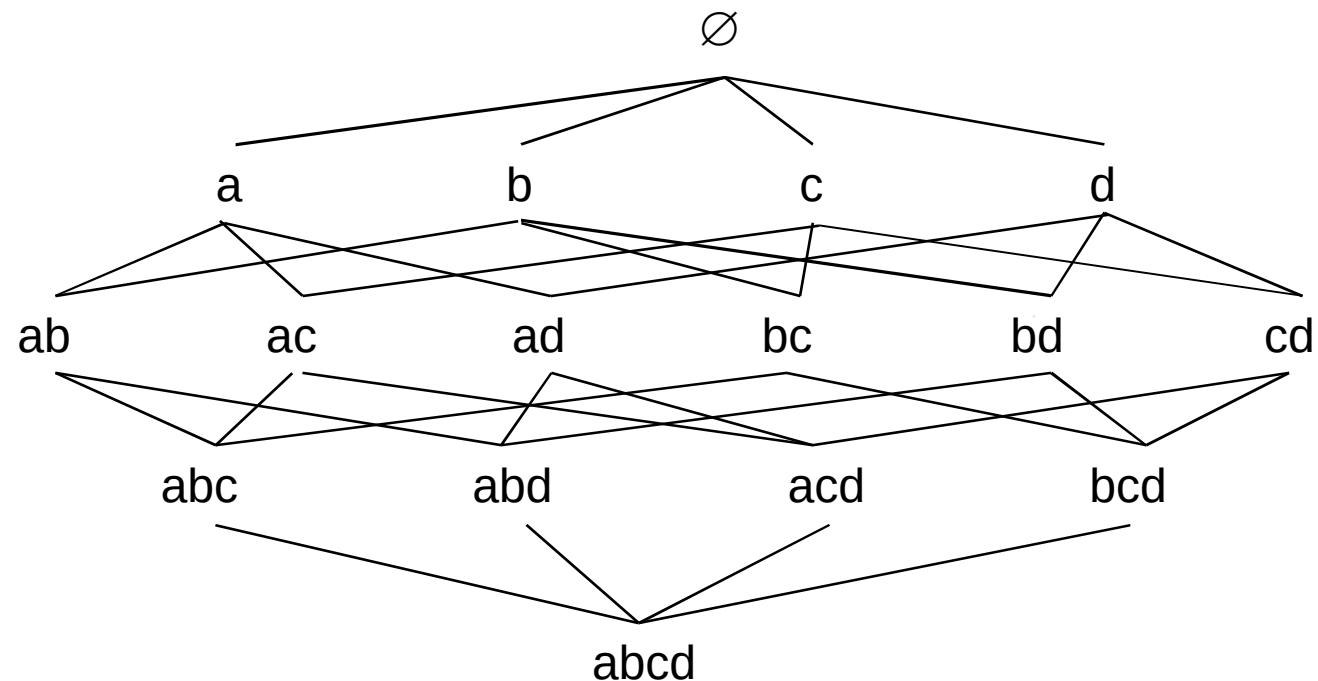
- ▶ Generate strong rules:
  - ▶ {A} → {C} [support=50%, confidence=66.6%]
  - ▶ {C} → {A} [support=50%, confidence=100%]

# Finding Frequent Itemsets

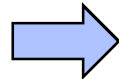
- ▶ Objective: given a dataset, find all the itemsets (or sets of attribute-value pairs) that satisfy the minimum support count.
- ▶ Algorithms
  - ▶ Apriori
  - ▶ FP-Growth
  - ▶ H-Mine
  - ▶ Eclat
  - ▶ Partition
  - ▶ CLOSET
  - ▶ CHARM
  - ▶ etc.

# Search Space for Finding All Frequent Itemsets

- ▶ Search space for D with 4 items:

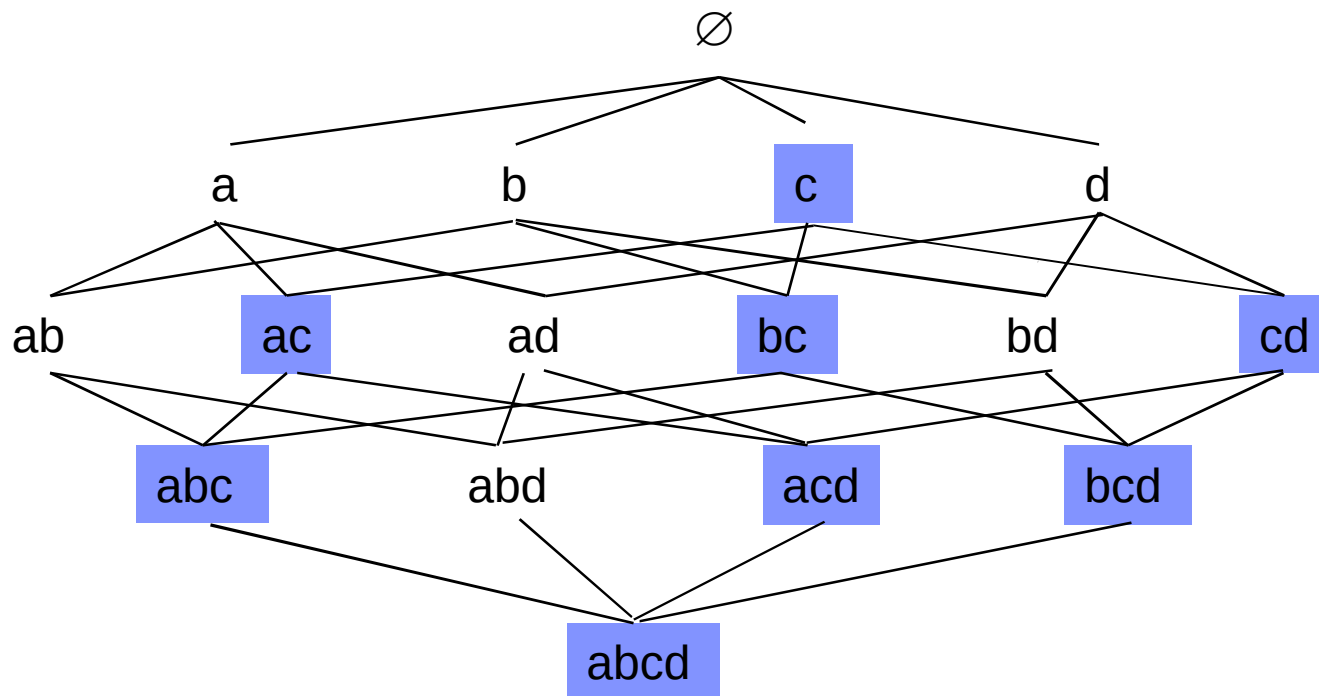


# Apriori Property

- ▶ The **Apriori** property (an *anti-monotone* property), also called *downward closure* property:
  - Any nonempty subset of a frequent itemset must be frequent
  - ▶ i.e., if  $\{A,B,C\}$  is a frequent itemset,  $\{A,B\}$ ,  $\{A,C\}$ ,  $\{B,C\}$ ,  $\{A\}$ ,  $\{B\}$  and  $\{C\}$  must also be frequent.
  - ▶ This is because a transaction containing  $\{A, B, C\}$  also contains  $\{A,B\}$ ,  $\{B,C\}$ , .... Thus, the support count of a subset is not less than that of the superset.
- ▶ Another way to express the property:
  - ▶ If an itemset is infrequent, none of its superset is frequent.
- ▶ No superset of any infrequent itemset should be checked.
  - ▶ Many item combinations can be pruned from the search space. 

# Pruning Search Space Using Apriori Property

- If  $c$  is not frequent, all the sets containing  $c$  are pruned:





# Apriori Algorithm

- ▶ Level-wise iterative generation
  - ▶ Generate length  $(k+1)$  candidate itemsets from frequent  $k$ -itemsets
  - ▶ Test the candidates against dataset  $D$

# The Apriori Algorithm

- ▶ Based on the Apriori property, use *iterative level-wise approach* and *candidate generation-and-test*

- ▶ Pseudo-code:

$C_k$ : a set of candidate itemsets of size  $k$

$L_k$ : the set of frequent itemsets of size  $k$

$L_1 = \{\text{frequent items}\};$     (*Scan dataset to find all frequent 1-itemsets*)

**for** ( $k = 1; L_k \neq \emptyset; k++$ ) **do begin**

$C_{k+1} = \underline{\text{candidates generated from } L_k};$

**if**  $C_{k+1} \neq \emptyset$

**for each** transaction  $t$  in dataset **do**

        increment the count of all candidates in  $C_{k+1}$   
        that are contained in  $t$

} *Scan dataset to  
calculate support  
for each itemset in  
 $C_{k+1}$*

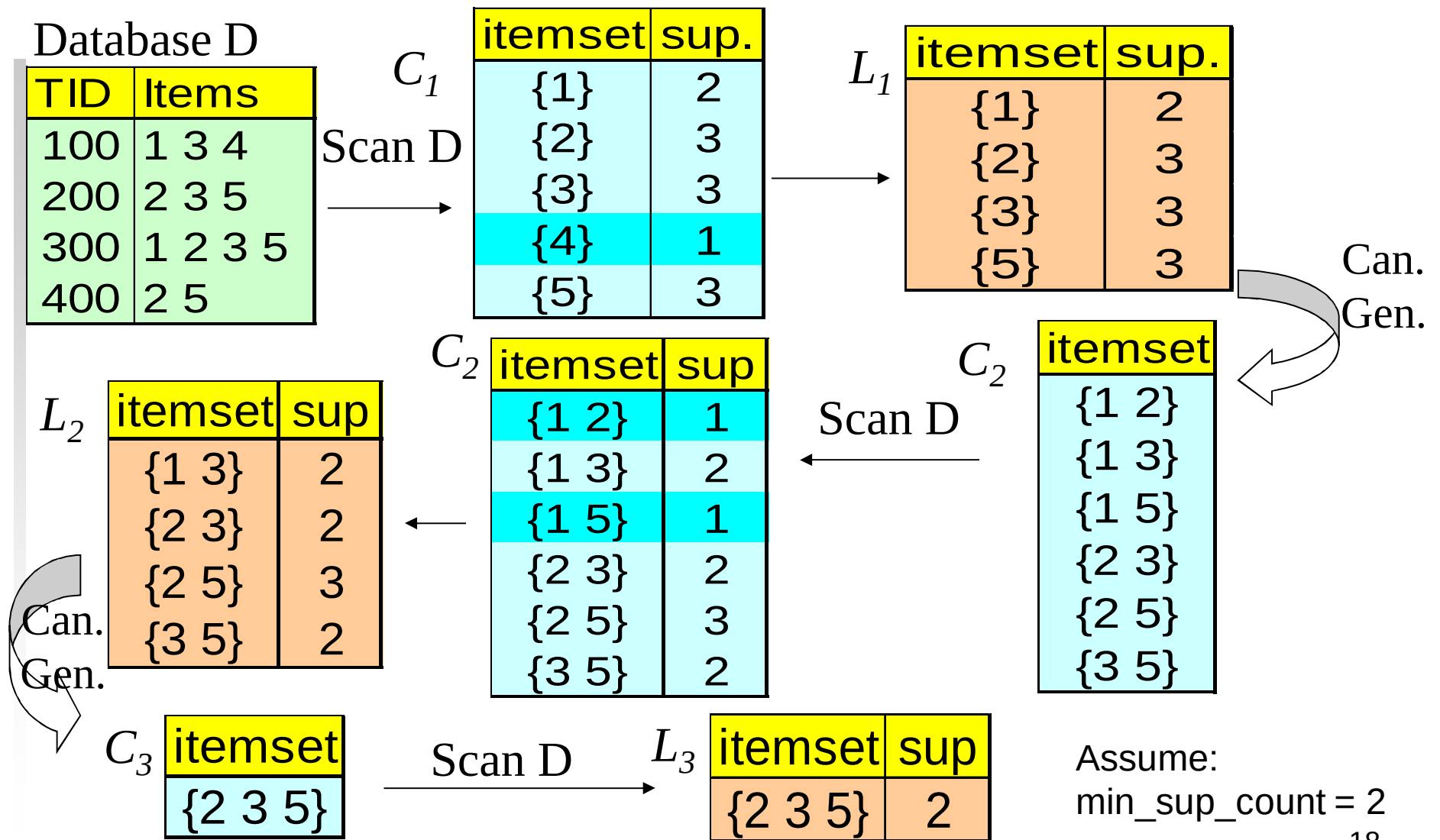
$L_{k+1} = \text{candidates in } C_{k+1} \text{ satisfying min\_support}$

**end**

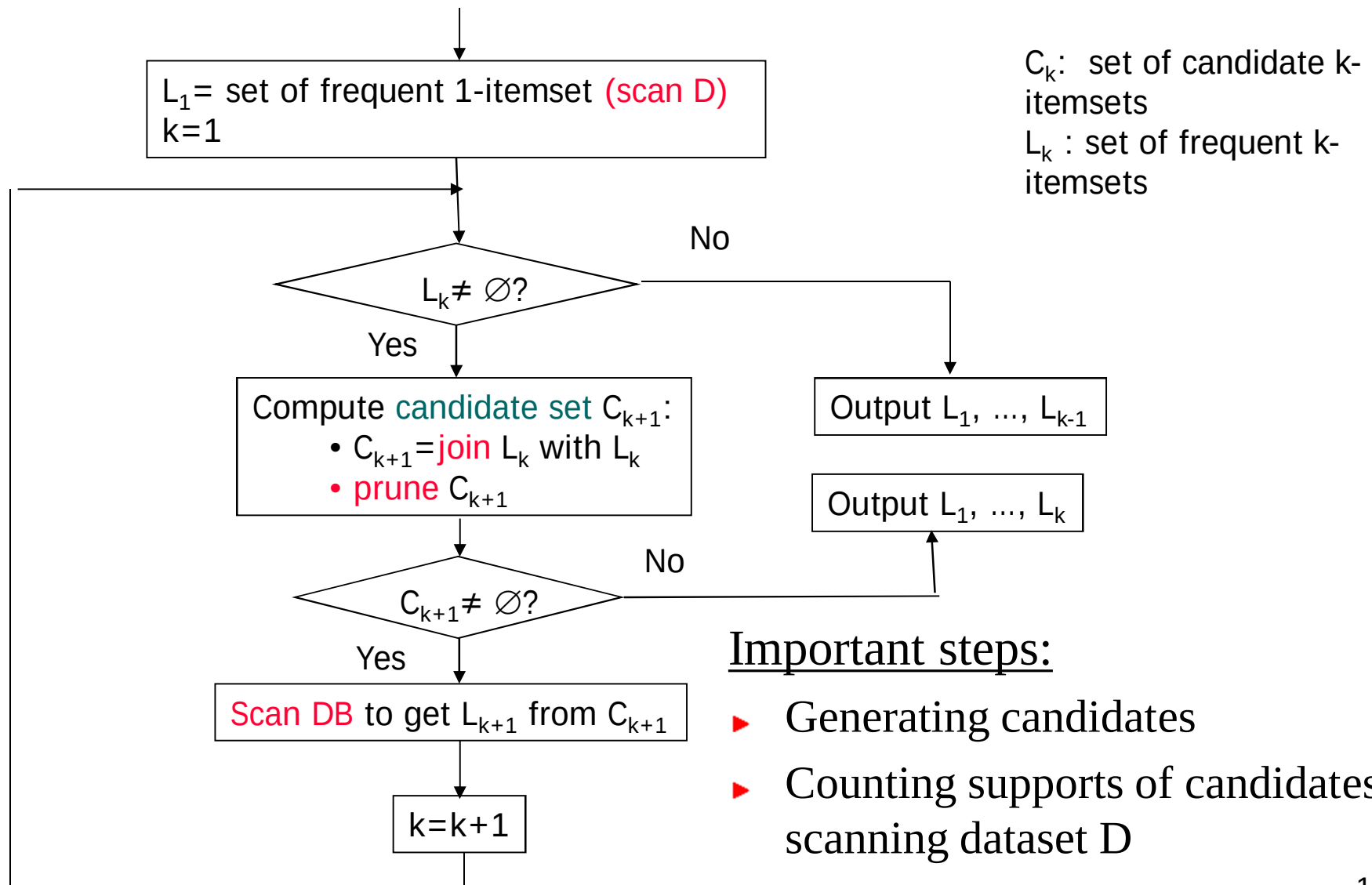
**return**  $\cup_k L_k$ ;

Level-wise Generation process:  $L_k \rightarrow C_{k+1} \rightarrow L_{k+1}$

# The Apriori Algorithm — Example



# Apriori Algorithm (Flow Chart)



# How to Generate Candidates (Generate $C_2$ from $L_1$ )

- ▶ Get all the two-item combinations from  $L_1$
- ▶ Example:
  - ▶  $L_1 = \{1, 2, 3, 5\}$
  - ▶  $C_2 = \{\{1\ 2\}, \{1\ 3\}, \{1\ 5\}, \{2\ 3\}, \{2\ 5\}, \{3\ 5\}\}$

# How to Generate Candidates?

(i.e., How to Generate  $C_{k+1}$  from  $L_k$  when  $k \geq 2$ )

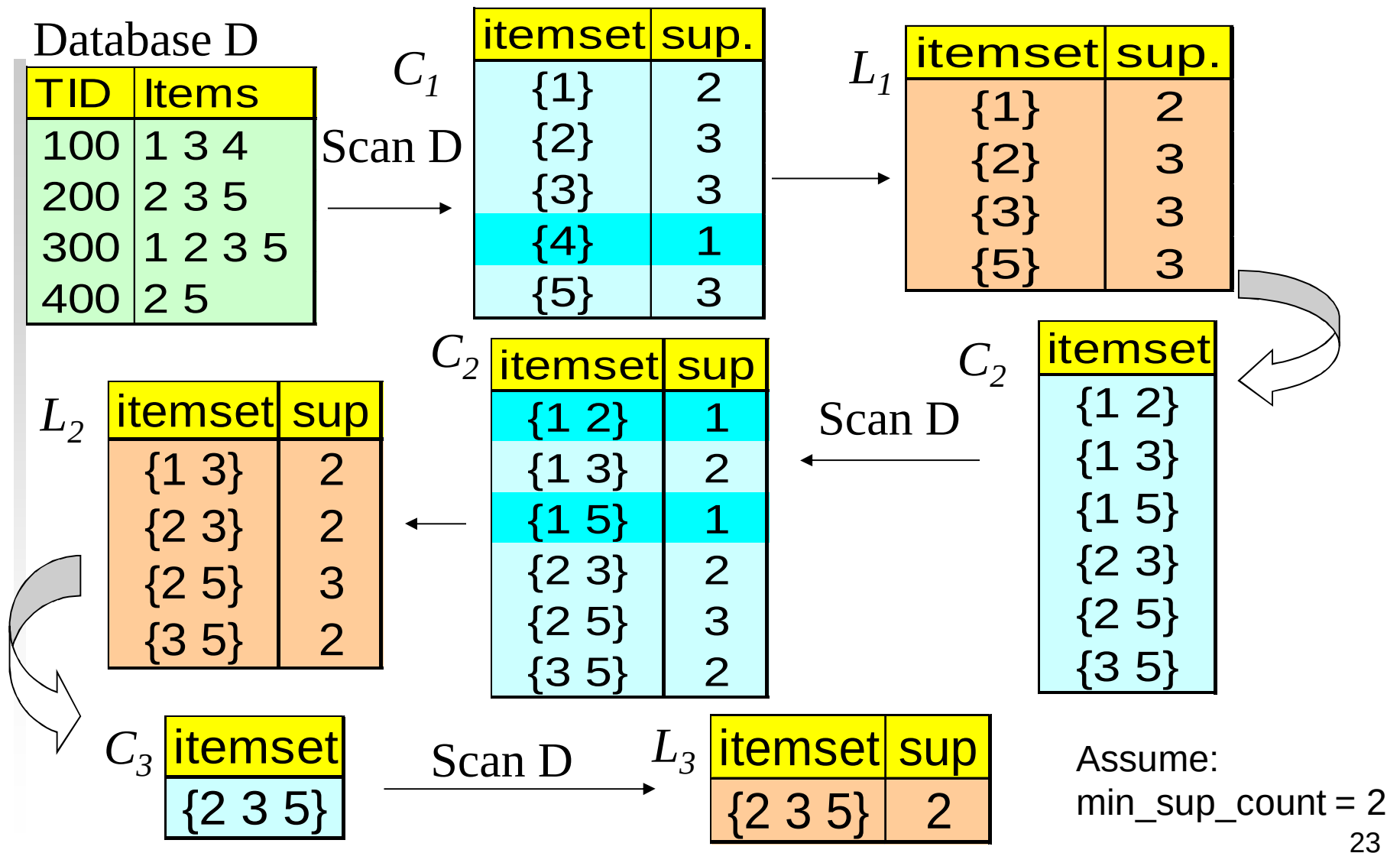
- ▶ Given  $L_k$  = the set of frequent  $k$ -itemsets
- ▶ List the items in each itemset of  $L_k$  in an order  
 $L_3 =$ 

|         |
|---------|
| {1 2 3} |
| {1 2 4} |
| {1 3 4} |
| {1 3 5} |
| {2 3 4} |
- ▶ Given  $L_k$ , generate  $C_{k+1}$  in two steps:
  - ▶ **Join Step**: Join  $L_k$  with  $L_k$  by joining two  $k$ -itemsets in  $L_k$ . Two  $k$ -itemsets are joinable if their first (k-1) items are the same and the last item in the first itemset is smaller than the last item in the second itemset (the condition for joining two members of  $L_k$ ).  
Now,  $C_4 = \{\{1\ 2\ 3\ 4\}, \{1\ 3\ 4\ 5\}\}$
  - ▶ **Prune Step**: Delete all candidates in  $C_{k+1}$  that have a non-frequent subset by checking all length- $k$  subsets of a candidate
    - ▶ Now,  $C_4 = \{\{1\ 2\ 3\ 4\}\}$

# Example of Candidate-generation

- ▶  $L_4 = \{abcd, abcg, abdg, abef, abeh, acdg, bcdg\}$
- ▶ Self-joining:  $L_4 * L_4$ 
  - ▶  $abcdg$  from  $abcd$  and  $abcg$
  - ▶  $abefh$  from  $abef$  and  $abeh$
- ▶ Pruning:
  - ▶  $abefh$  is removed because  $abfh$  or  $aefh$  or  $befh$  is not in  $L_4$
- ▶  $C_5 = \{abcdg\}$

# The Apriori Algorithm — Example



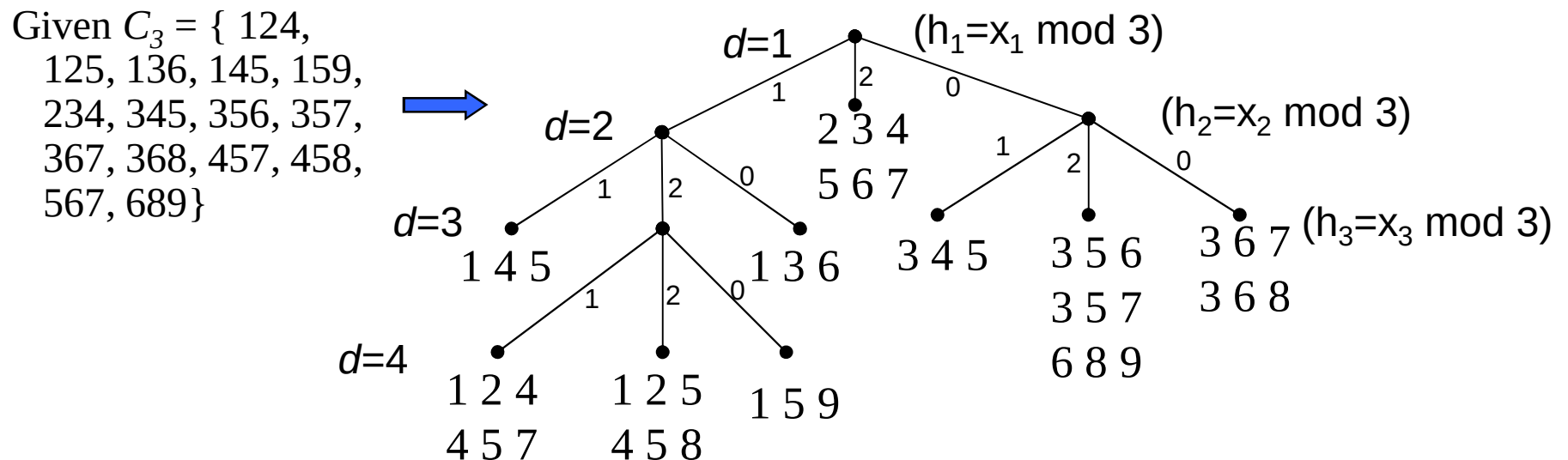


# Support Counting of Candidates in dataset (DS) scan

- ▶ Objective of candidate support counting
  - ▶ Find frequent itemsets  $L_k$  from a set of candidates  $C_k$
- ▶ **Naïve method**: match each candidate with each transaction. Time consuming when
  - ▶ the total number of candidates is very large
  - ▶ one transaction contains many candidates
- ▶ **Hash tree method**:
  - ▶ Store candidate itemsets in  $C_k$  in a hash-tree
    - ▶ Leaf node of hash-tree contains a list of candidates and their counts
    - ▶ Interior node represents a hash function.
  - ▶ Use a subset function to find all the candidates contained in a transaction

# Building a Hash Tree

- Initially treat the root as a leaf node
- Insert itemsets in  $C_k$  into the tree
- When the number of itemsets in a leaf node exceeds a *specified threshold* (such as 3 in the following example), convert the leaf node into an interior node by
  - hashing the itemsets according to the  $d$ -th item of the itemset, where  $d$  is the level of the node.

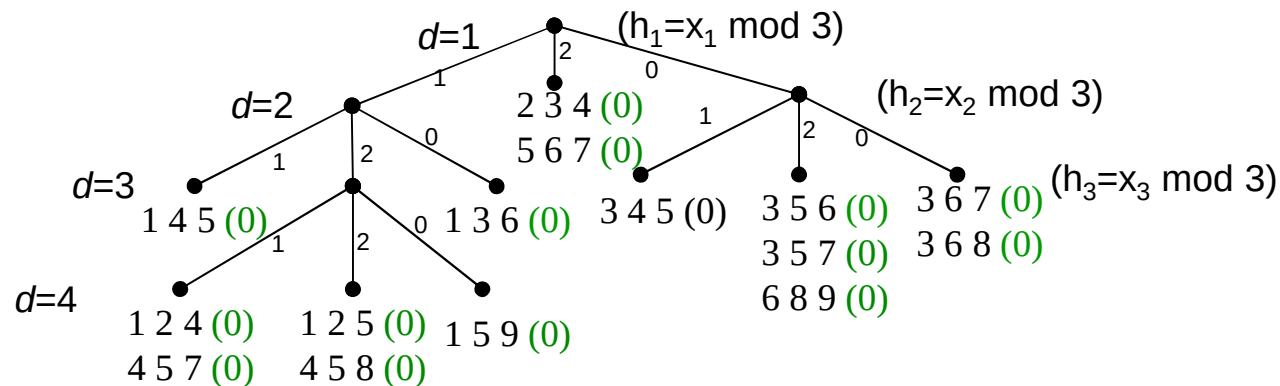


# Scan DS to Obtain the Counts of All Length-3 Candidates (1)

- ▶ Assume a transaction DB:

{1,3,5,6}  
{2,3,5}  
{1,3,6,7,8}  
{3,5,6}  
.....

- ▶ The counts of all the candidates in the hash tree are initialized to zero.



# Scan DB to Obtain the Counts of All Length-3 Candidates (2)

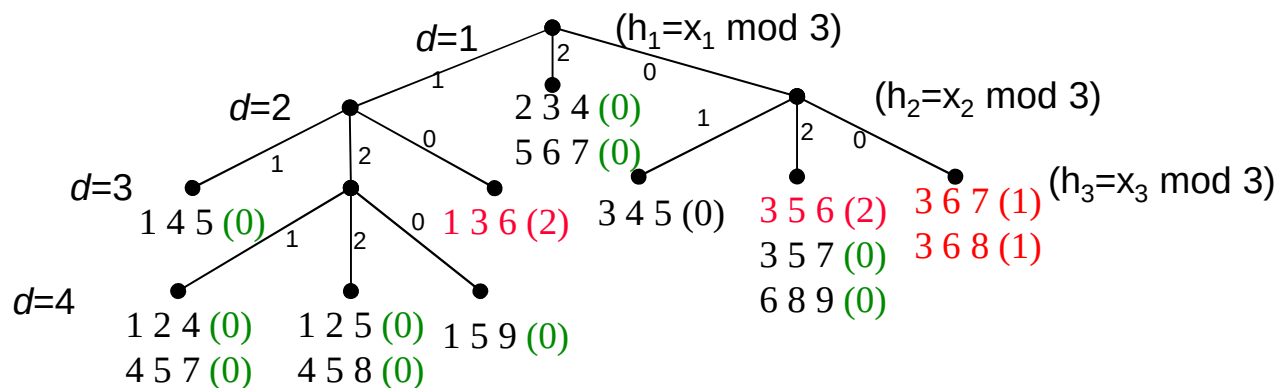
- Assume a transaction DB:

{1,3,5,6}  
 {2,3,5}  
 {1,3,6,7,8}  
 {3,5,6}  
 .....

**General idea:** for each transaction,

- Get all the length-3 subsets
- Check each subset to see if it is in the hash tree
- If yes, increase the count of the subset in the tree.

- Apply the **subset function** on each transaction and increase the count of a candidate if the transaction contains the candidate



# Subset Function

## ▶ Functionality

- ▶ Given  $C_k$  (in a hash tree) and a transaction  $t$ , find all the candidates in  $C_k$  contained in  $t$  and increase the count of these candidates:

$\text{Subset}(C_k, t)$ : candidate itemsets contained in  $t$

## ▶ Method

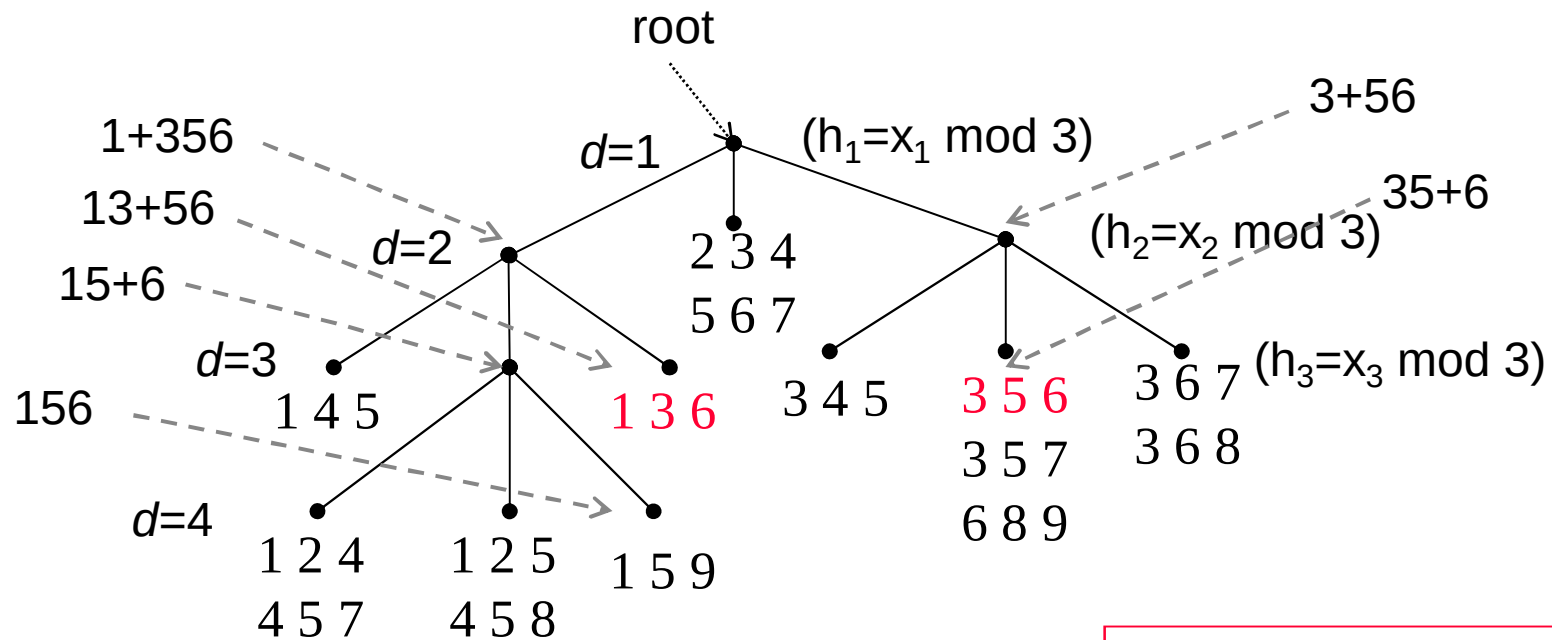
- ▶ Try to check each subset of  $t$  to see if it is contained in the tree as follows:
- ▶ At root, hash on every item in  $t$  (until the  $k$ th item from the end) .
- ▶ At an interior node reached by hashing on item  $i$ , hash on each item that comes after  $i$  in  $t$  (until the  $(k-d+1)$ th item from the end, where  $d$  is the level of the node in the tree), recursively apply to the nodes in the corresponding bucket
- ▶ At a leaf, find itemsets contained in  $t$

## ▶ Benefit

- ▶ Don't have to match each candidate with each transaction.

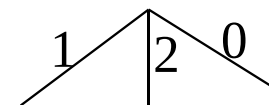
# Example: finding the candidates contained in a transaction.

- ▶ Given a transaction {1 3 5 6} and hash tree:



It only goes to the leaf that may contain subsets of the transaction.

hash function



# How to Mine Association Rules

- ▶ A two-step process:
  - ▶ *Find all frequent itemsets* ---- *done*
  - ▶ *Generate strong association rules from frequent itemsets.*
- ▶ Example: given min\_sup=50% and min\_conf=50%

| Transaction ID | Items Bought |
|----------------|--------------|
| 2000           | A,B,C        |
| 1000           | A,C          |
| 4000           | A,D          |
| 5000           | B,E,F        |



| Frequent Itemset | Support |
|------------------|---------|
| {A}              | 75%     |
| {B}              | 50%     |
| {C}              | 50%     |
| {A, C}           | 50%     |

- ▶ Generate strong rules:
  - ▶ {A} → {C} [support=50%, confidence=66.6%]
  - ▶ {C} → {A} [support=50%, confidence=100%]

# Generate Association Rules from Frequent Itemsets

► Naïve algorithm:

for each frequent itemset  $l$  whose length  $\geq 2$  do  
  for each **nonempty proper** subset  $s$  of  $l$  do  
    if ( $\text{support}(l)/\text{support}(s) \geq \text{min\_conf}$ )  
      output the rule  $s \rightarrow l-s$ , with  
        support =  $\text{support}(l)$  and  
        confidence =  $\text{support}(l)/\text{support}(s)$

Note that we only need to check the confidence.  
Do we need to scan the database? No. Why?



# Generate Association Rules from Frequent Itemsets

## ► Example:

- Given a frequent itemset:  $l = \{A, B, C\}$
- nonempty proper subsets of  $l$  are  $\{A, B\}$ ,  $\{A, C\}$ ,  $\{B, C\}$ ,  $\{A\}$ ,  $\{B\}$ ,  $\{C\}$
- resulting association rules:

$$\{A, B\} \rightarrow \{C\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A, B\})} = 50\%$$

$$\{A, C\} \rightarrow \{B\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A, C\})} = 100\%$$

$$\{B, C\} \rightarrow \{A\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{B, C\})} = 100\%$$

$$\{A\} \rightarrow \{B, C\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A\})} = 33\%$$

$$\{B\} \rightarrow \{A, C\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{B\})} = 29\%$$

$$\{C\} \rightarrow \{A, B\}, \quad \text{confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{C\})} = 100\%$$

*These confidence values are make-up numbers. This example is not continued previous examples.*

If minimum confidence threshold is 70%, only 3 rules are outputted.

# Is Apriori Fast Enough? — Performance Bottlenecks

- ▶ The core of the Apriori algorithm:
  - ▶ Use frequent  $k$ -itemsets to generate candidate  $(k+1)$ -itemsets
  - ▶ Use dataset scan and (hash-tree facilitated) pattern matching to collect counts for the candidate itemsets to generate frequent  $(k+1)$ -itemsets from  $k+1$  candidate set
- ▶ The bottleneck of *Apriori*: Candidate generation and testing
  - ▶ Huge candidate sets:
    - ▶  $10^4$  frequent 1-itemsets will generate more than  $10^7$  candidate 2-itemsets
    - ▶ To discover a frequent pattern of size 100, e.g.,  $\{a_1, a_2, \dots, a_{100}\}$ , one needs to generate at least  $2^{100} \approx 10^{30}$  candidates.
  - ▶ Multiple scans of database:
    - ▶ Needs  $n$  or  $n+1$  scans,  $n$  is the length of the longest frequent pattern

---

# Outline

- ▶ Basic concepts of association rule learning
- ▶ Apriori algorithm
- ▶ *FP-Growth Algorithm*
  - ▶ *Avoid candidate generation*
  - ▶ *Reduce the number of database scans*
- ▶ Finding interesting rules.

# FP-Growth

- ▶ J. Han, J. Pei, and Y. Yin. *Mining Frequent Patterns without Candidate Generation.*, Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- ▶ J. Han, J. Pei, Y. Yin and R. Mao, *Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach*, Data Mining and Knowledge Discovery, 8(1):53-87, 2004. ([http://www-faculty.cs.uiuc.edu/~hanj/pdf/dami04\\_fptree.pdf](http://www-faculty.cs.uiuc.edu/~hanj/pdf/dami04_fptree.pdf))
- ▶ Chapter 6.2.4 (3<sup>rd</sup> edition) or Chapter 5.2.4 (2<sup>nd</sup> Edition)

## Mining Frequent Patterns Without Candidate Generation (FP-growth)

Two major steps:

- ▶ Compress a large database into a compact, Frequent-  
Pattern tree (FP-tree) structure
  - ▶ highly condensed, but complete for frequent pattern mining
  - ▶ **avoid costly database scans**
- ▶ Mine frequent patterns (itemsets) from an FP-tree
  - ▶ A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - ▶ Efficient: **avoid candidate generation** -- generate frequent patterns from the tree directly.

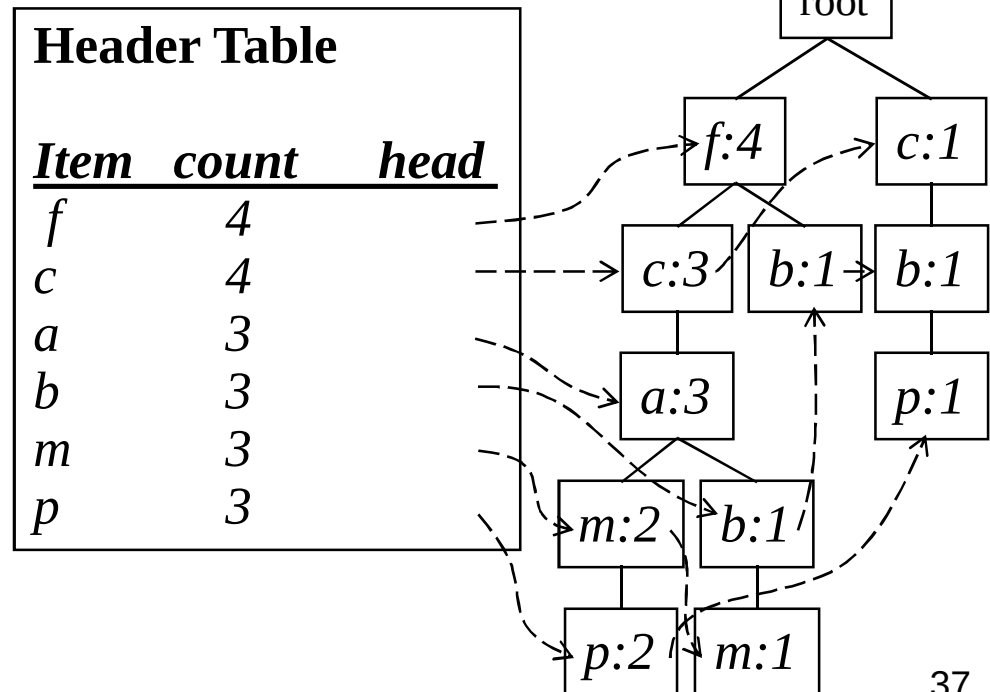
# Construct FP-tree from a Transaction DB

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 0.5*  
*minimum support count = 3*

Steps:

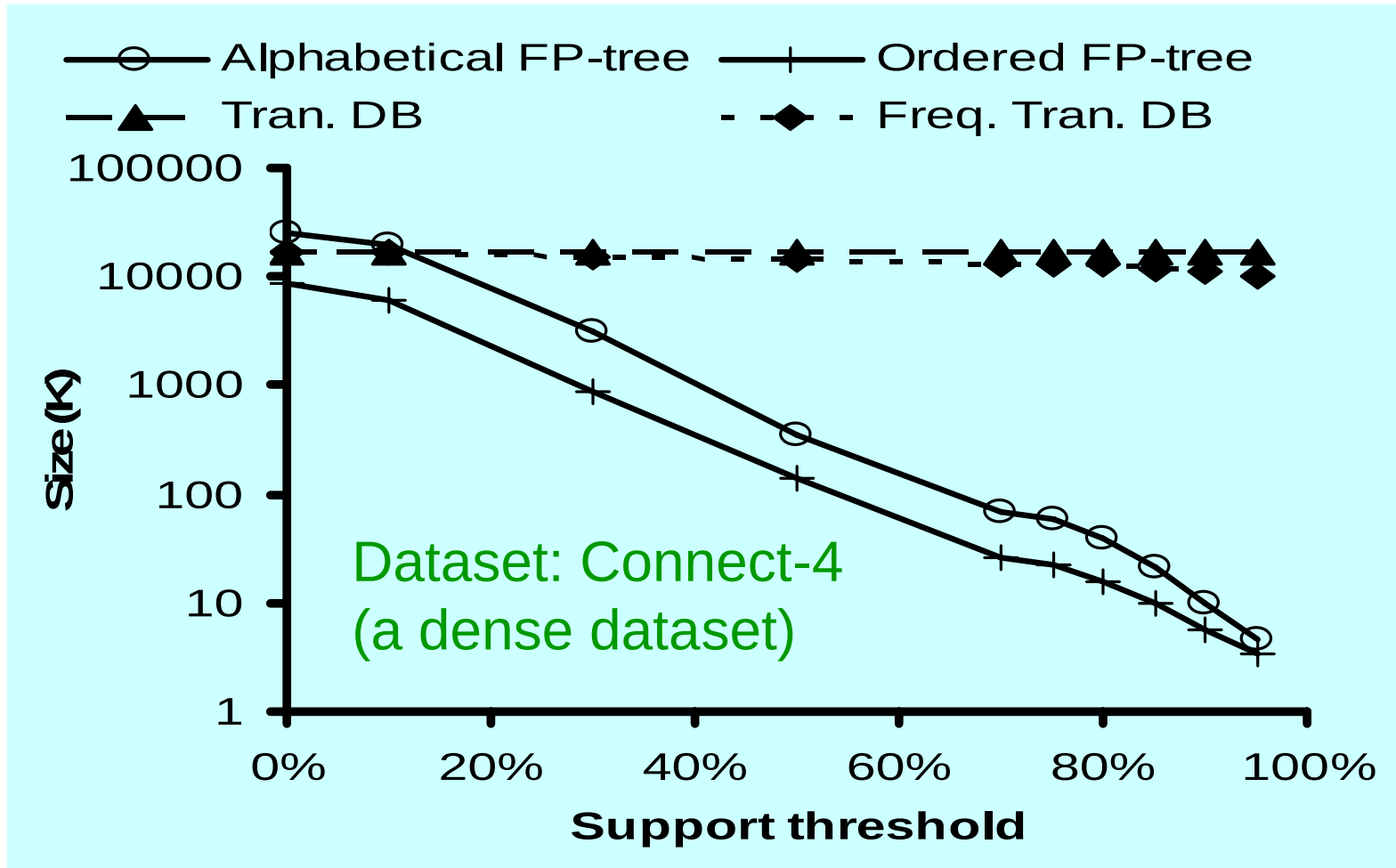
1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Scan DB again, construct FP-tree
  1. Remove infrequent items
  2. Order frequent items in frequency descending order
  3. Put the remaining items in the tree



# Benefits of the FP-tree Structure

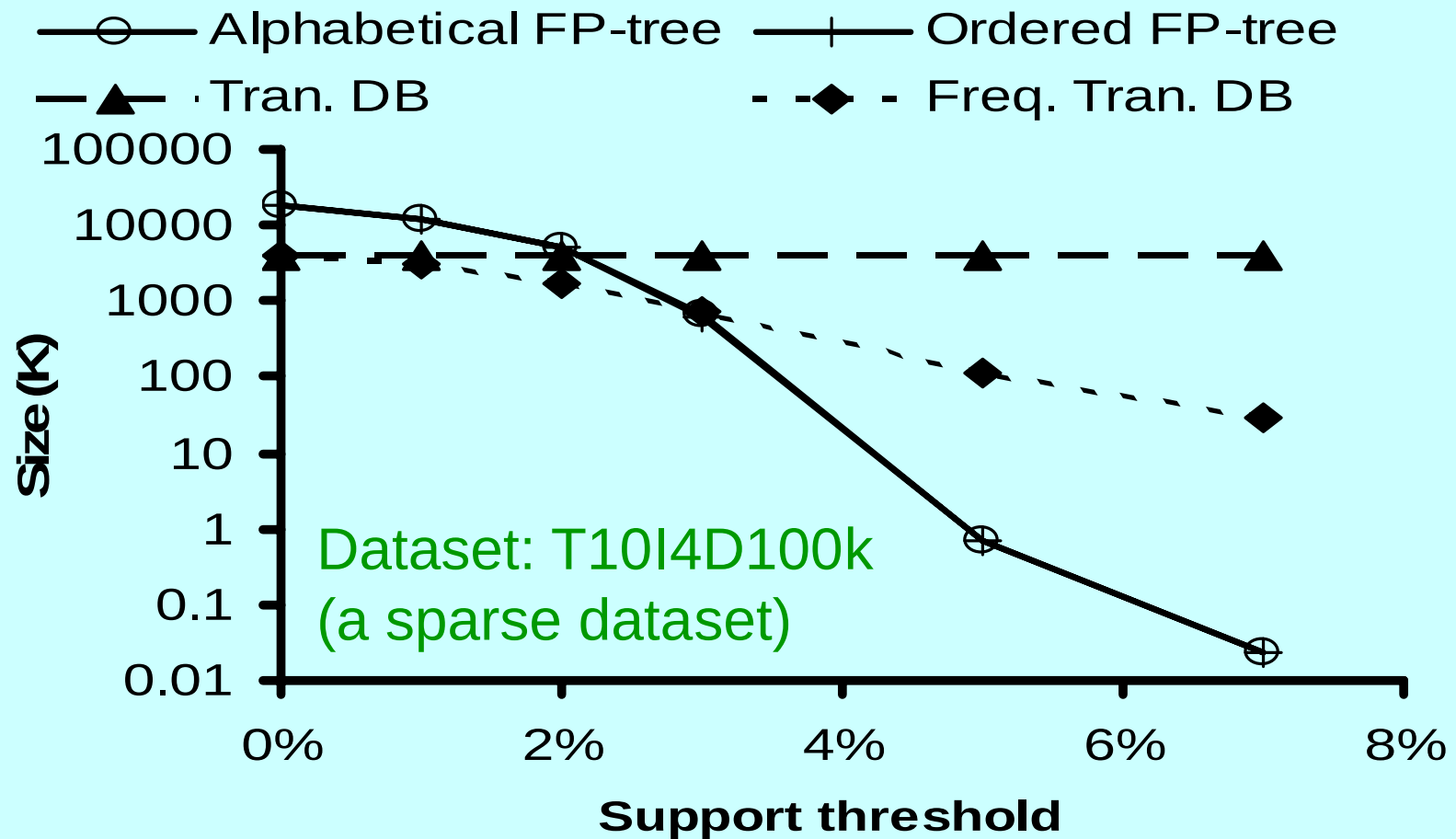
- ▶ Completeness:
  - ▶ map each transaction into a path in the tree
  - ▶ preserves complete information for frequent pattern mining
    - ▶ no need to scan the database any more
- ▶ Compactness
  - ▶ reduce irrelevant information—infrequent items are gone
  - ▶ A path can store one or more transactions
  - ▶ Items in frequency descending order (*f-list*):
    - ▶ more frequent items are more likely to be shared
  - ▶ Not larger than the original database (not counting node-links and the count fields)

# How Effective Is FP-tree?





# Compressing Sparse Dataset



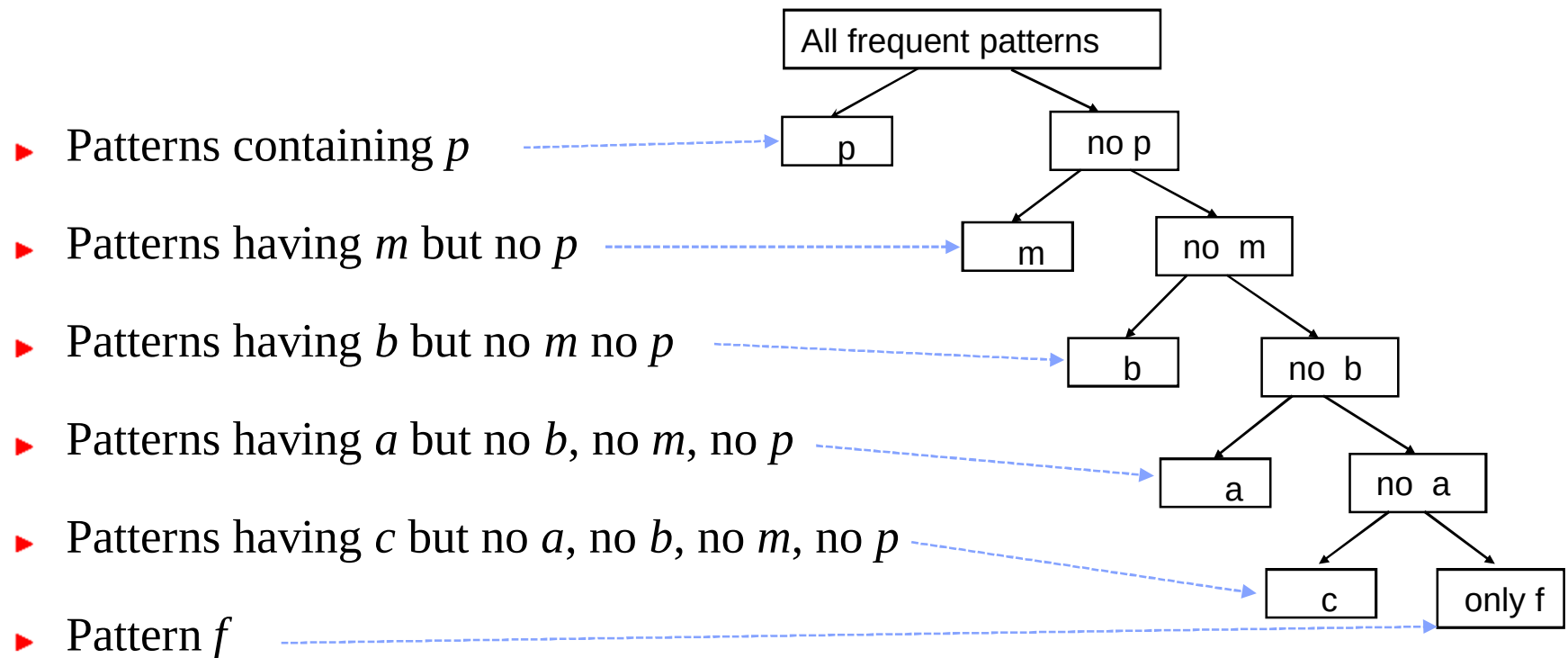
# Mining Frequent Patterns from FP-tree

(Frequent pattern = frequent itemset)

- ▶ General idea:
  - ▶ Divide-and-conquer
  - ▶ Recursively
    - ▶ partition the set of frequent patterns
    - ▶ find frequent patterns in each partition by
      - ▶ building *conditional pattern base* and *conditional FP-tree* for each local frequent item in the partition

# Partition the Set of Frequent Patterns

- ▶ Partition the set of frequent patterns into subsets according to **f-list: f-c-a-b-m-p** (the list of freq. items in frequency- descending order):



- ▶ The partitioning is complete and without any overlap

# Find Frequent Patterns Having Item “p”

- ▶ Only transactions containing  $p$  are needed
- ▶ Form  $p$ -conditional *pattern base* ( $p$ -projected database)  $TDB|p$   $\searrow$  Contains transactions containing  $p$ 
  - ▶ Starting at entry  $p$  of header table
  - ▶ Follow the side-link of frequent item  $p$
  - ▶ Accumulate all transformed prefix paths of  $p$

p-conditional pattern base  $TDB|_p$

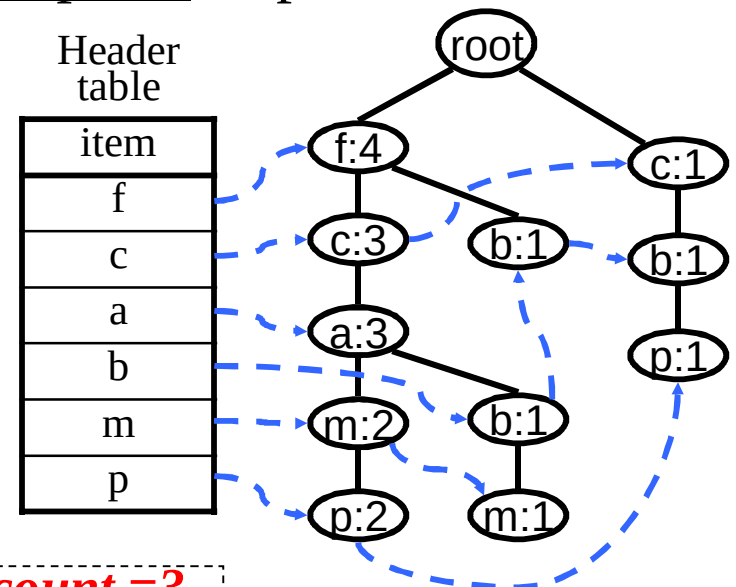
fcam: 2

cb: 1

Local frequent item: c:3

## Frequent patterns containing p

ср: 3



*minimum support count =3*

# Find Frequent Patterns Having Item m But No p

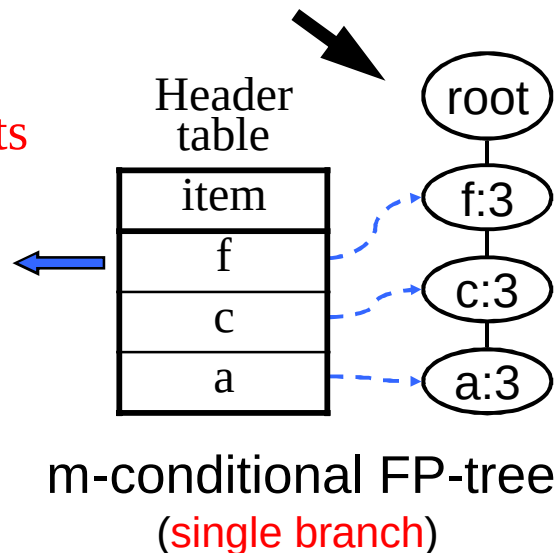
- ▶ Form m-conditional pattern base (m-projected database) TDB|m
  - ▶ Item p is excluded (by looking at only the prefix paths of m)
  - ▶ TDB|m contains **fca:2, fcab:1**
- ▶ Recursively apply FP-growth to find freq. patterns from TDB|m
  - ▶ Local frequent items: f, c, a
  - ▶ After removing local infrequent item: **fca:2, fca:1**
  - ▶ Build **m-conditional FP-tree** from TDB|m

Non-empty subsets  
of {f, c, a}:

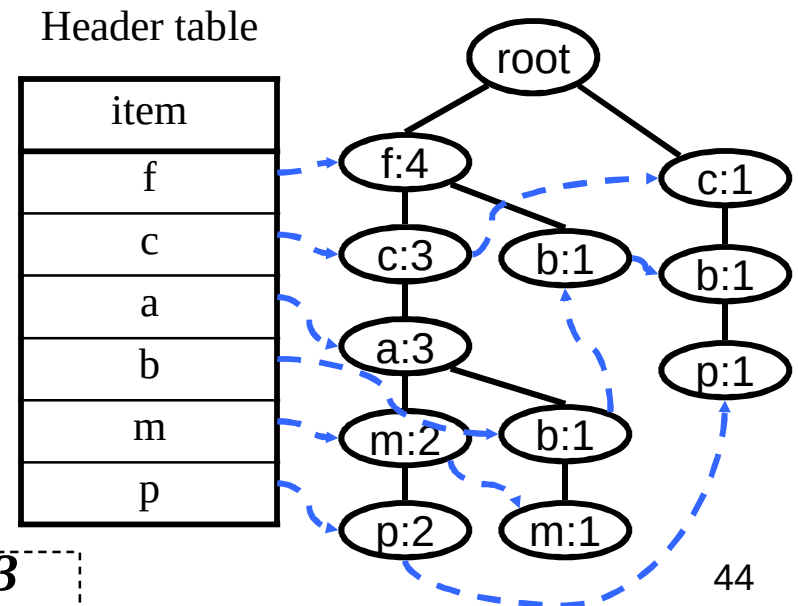
f, c, a  
fc, fa, ca  
fca

Frequent patterns  
having m:

fm, cm, am  
fcm, fam, cam  
fcam



*minimum support count = 3*



# Find Frequent Patterns Having Item m But No p (*more complex situation*)

▶ Assume m-conditional pattern base is: **fca:3, fb:3**

▶ Local frequent items: f:6, c:3, a:3, b:3

▶ Build m-conditional FP-tree

▶ First generate:

▶ fm: 6, cm: 3, am:3, bm:3

▶ To learn longer patterns containing m

▶ Further partition frequent patterns containing m (but no p) into

▶ Patterns containing b

▶ Patterns containing a but no b

▶ Patterns containing c but no b or a

▶ Patterns containing only f (i.e. fm)

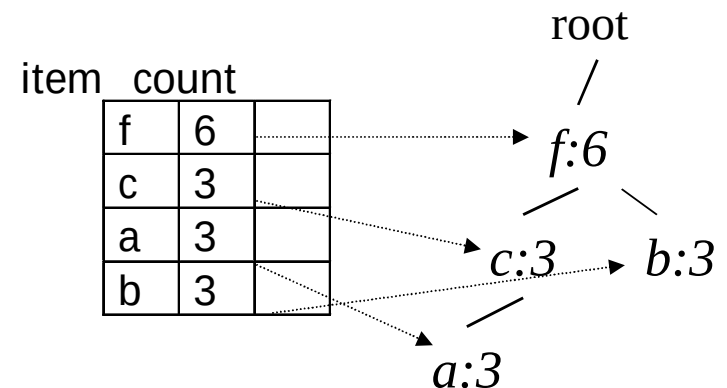
▶ Compute *ym*-conditional pattern bases:

ym    conditional pattern base

*bm*    *f:3*

*am*    *fc:3*

*cm*    *f:3*



# Find Frequent Patterns Having Item m But No p (*more complex situation*)

- ▶ Having *ym*-conditional pattern bases:

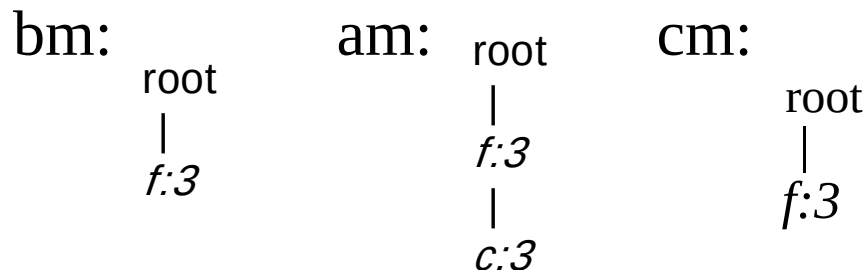
ym    conditional pattern base

*bm*        *f:3*

*am*        *fc:3*

*cm*        *f:3*

- ▶ Built *ym*-conditional FP-trees



- ▶ General frequent patterns with suffix *ym*:

*fbm*,    *fam*, *cam*, *fcam*,    *fc**m*

# Major Steps to Mine FP-tree

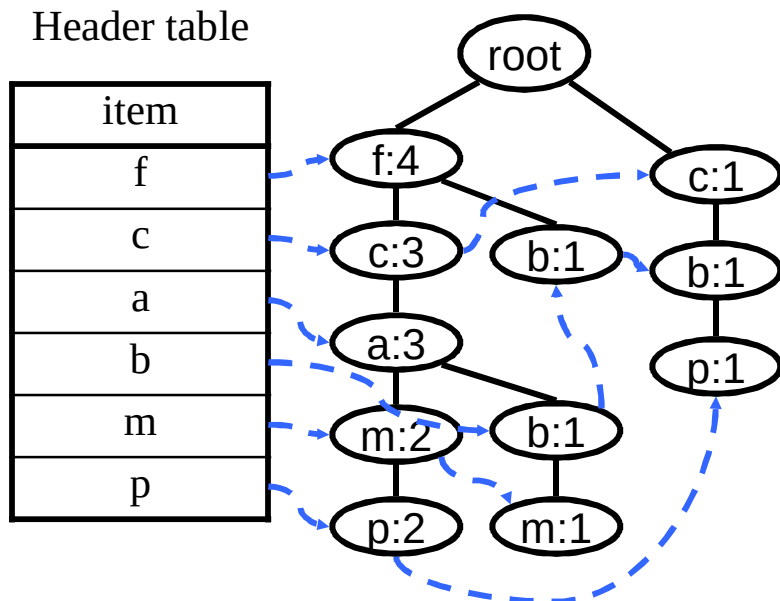
For each item in the FP-tree

1. Construct *conditional pattern base*
2. Construct *conditional FP-tree* from the conditional pattern-base
3. Generate frequent patterns from the conditional FP-tree
  - ▶ If the conditional FP-tree contains a single path, simply enumerate all the subsets of the itemset on the path (*Recursion stopping criterion*)
  - ▶ Otherwise, ***recursively*** mine the conditional FP-tree and grow frequent patterns obtained so far



# Step 1: From FP-tree to Conditional Pattern Base

- ▶ Starting at the frequent header table in the FP-tree
- ▶ Traverse the FP-tree by following the link of each frequent item
- ▶ Accumulate all prefix paths of that item to form a *conditional pattern base*



## Conditional pattern bases

| <u>item</u> | <u>cond. pattern base</u> |
|-------------|---------------------------|
| <i>c</i>    | <i>f:3</i>                |
| <i>a</i>    | <i>fc:3</i>               |
| <i>b</i>    | <i>fca:1, f:1, c:1</i>    |
| <i>m</i>    | <i>fca:2, fcab:1</i>      |
| <i>p</i>    | <i>fcam:2, cb:1</i>       |

## Step 2: Construct Conditional FP-tree

- ▶ For each pattern-base
  - ▶ Accumulate the count for each item in the base to find local frequent items
  - ▶ Remove locally infrequent items
  - ▶ Construct conditional FP-tree for the frequent items of the pattern base

*p*-conditional pattern

base:

*fcam:2, cb:1*



Taken out infrequent items

*c:2, c:1*



root



*c:3*

*p*-conditional FP-tree

*m*-conditional pattern

base:

*fca:2, fcab:1*



Taken out infrequent items

*fca:2, fca:1*



root



*f:3*



*c:3*



*a:3*

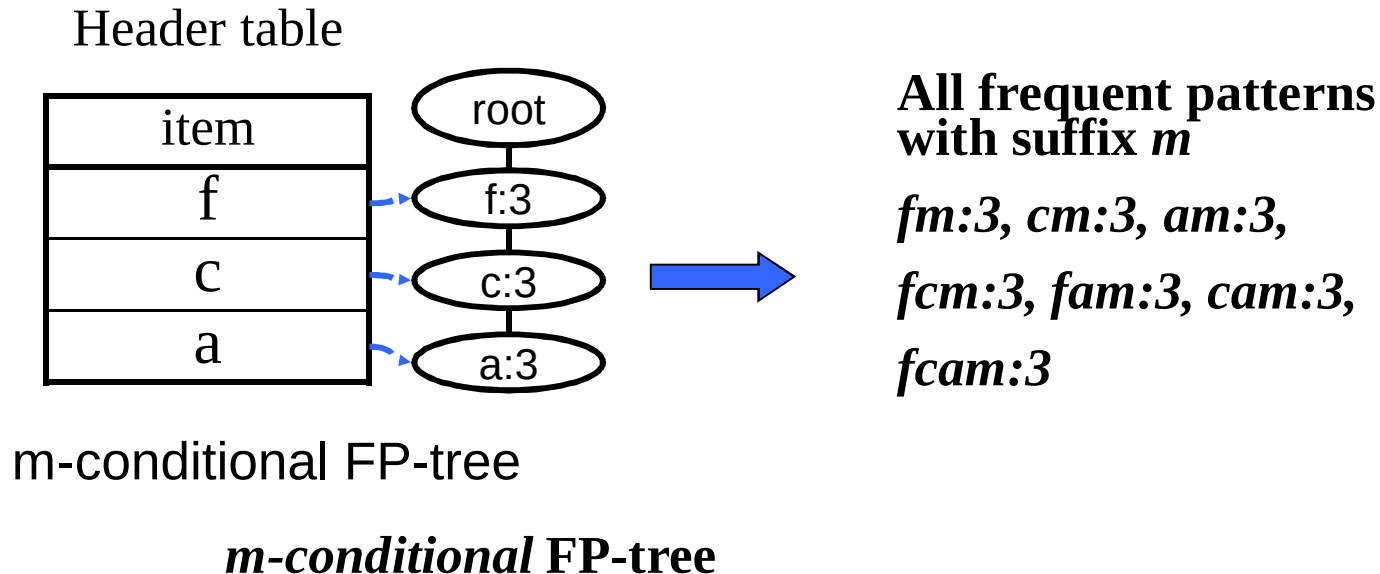
*m*-conditional FP-tree

## Conditional Pattern-Bases and Conditional FP-trees

| Item | Conditional pattern-base    | Conditional FP-tree     |
|------|-----------------------------|-------------------------|
| p    | $\{(fcam:2), (cb:1)\}$      | $\{(c:3)\} p$           |
| m    | $\{(fca:2), (fcab:1)\}$     | $\{(f:3, c:3, a:3)\} m$ |
| b    | $\{(fca:1), (f:1), (c:1)\}$ | Empty                   |
| a    | $\{(fc:3)\}$                | $\{(f:3, c:3)\} a$      |
| c    | $\{(f:3)\}$                 | $\{(f:3)\} c$           |

## Step 3: Generate Frequent Patterns from Conditional FP-tree

- ▶ If an  $x$ -conditional FP-tree has a single path  $P$ 
  - ▶ The complete set of frequent patterns with suffix  $x$  can be generated by enumeration of all the combinations of items in  $P$

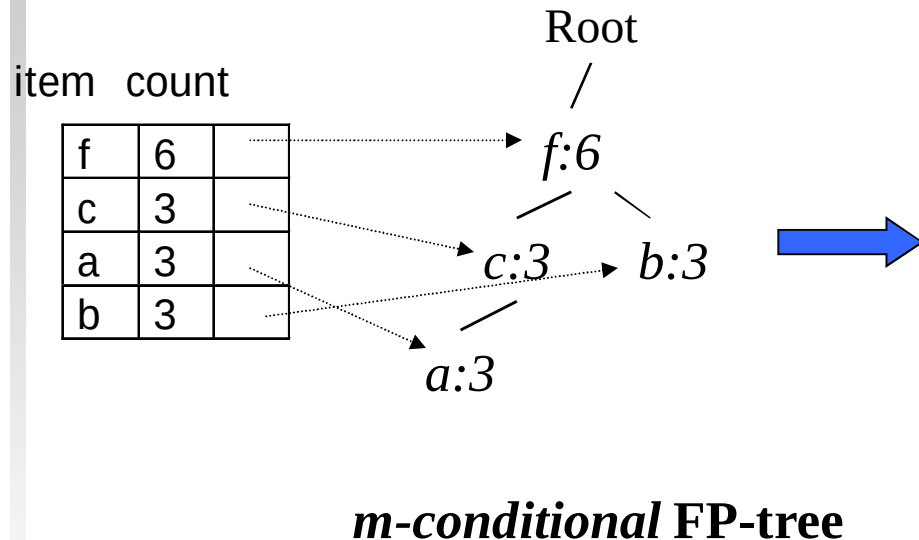


## Step 3: Generate Frequent Patterns from Conditional FP-tree (*Contd.*)

- ▶ **If an  $x$ -conditional FP-tree has more than one path**
  - ▶ For each item  $y$  that appears in  $x$ -conditional FP-tree
    - ▶ Generate pattern  $yx$  with support = the support of  $y$  in  $x$ -conditional FP-tree.
    - ▶ Construct  $yx$ -conditional pattern base and then  $yx$ -conditional FP-tree to generate frequent patterns with suffix  $yx$  (a recursive procedure).

## Step 3: Generate Frequent Patterns from Conditional FP-tree (*Contd.*)

- Suppose  $m$ -conditional FP-tree is



- Generate frequent 2-itemsets having  $m$ :  
 $fm:6, cm:3, am:3, bm:3$

- Compute  $ym$ -conditional pattern bases:

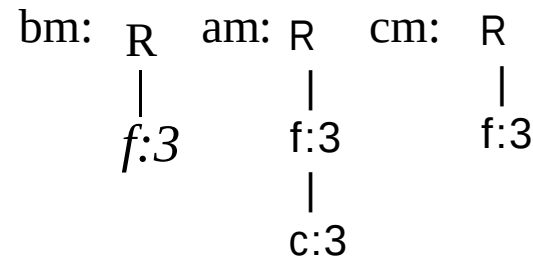
$ym$     conditional pattern base

$bm$      $f:3$

$am$      $fc:3$

$cm$      $f:3$

- Built  $ym$ -conditional FP-trees



- General frequent patterns with suffix  $ym$ :  
 $fbm:3, fam:3, cam:3, fcam:3, fcm:3$

# FP-Growth Algorithm

- ▶ Input: *FP-tree* (a FP-tree built by scanning DB)
- ▶ Output: the complete set of frequent patterns
- ▶ Method: call `FP-growth(FP-tree, null)`
- ▶ Procedure `FP-growth(A_conditional_FP_Tree, A)`
  - ▶ if Tree contains a single path *P*
    - ▶ for each combination (denoted as *B*) of the nodes in the path *P* do
      - ▶ generate pattern *BA* with support = minimum support of nodes in *B*
  - ▶ else for each item  $a_i$  in the header table of *Tree* do
    - ▶ generate pattern  $B=a_iA$  with support = the support of  $a_i$
    - ▶ construct *B*'s conditional pattern base and then *B*'s conditional FP-tree  $Tree_B$ ;
    - ▶ if  $Tree_B$  is not empty,
      - ▶ call **FP-growth**( $Tree_B$ , *B*)

# Exercise

- ▶ A transaction DB:

| TID  | items          |
|------|----------------|
| T100 | I1, I2, I5     |
| T200 | I2, I4         |
| T300 | I2, I3         |
| T400 | I1, I2, I4     |
| T500 | I1, I3         |
| T600 | I2, I3         |
| T700 | I1, I3         |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3     |

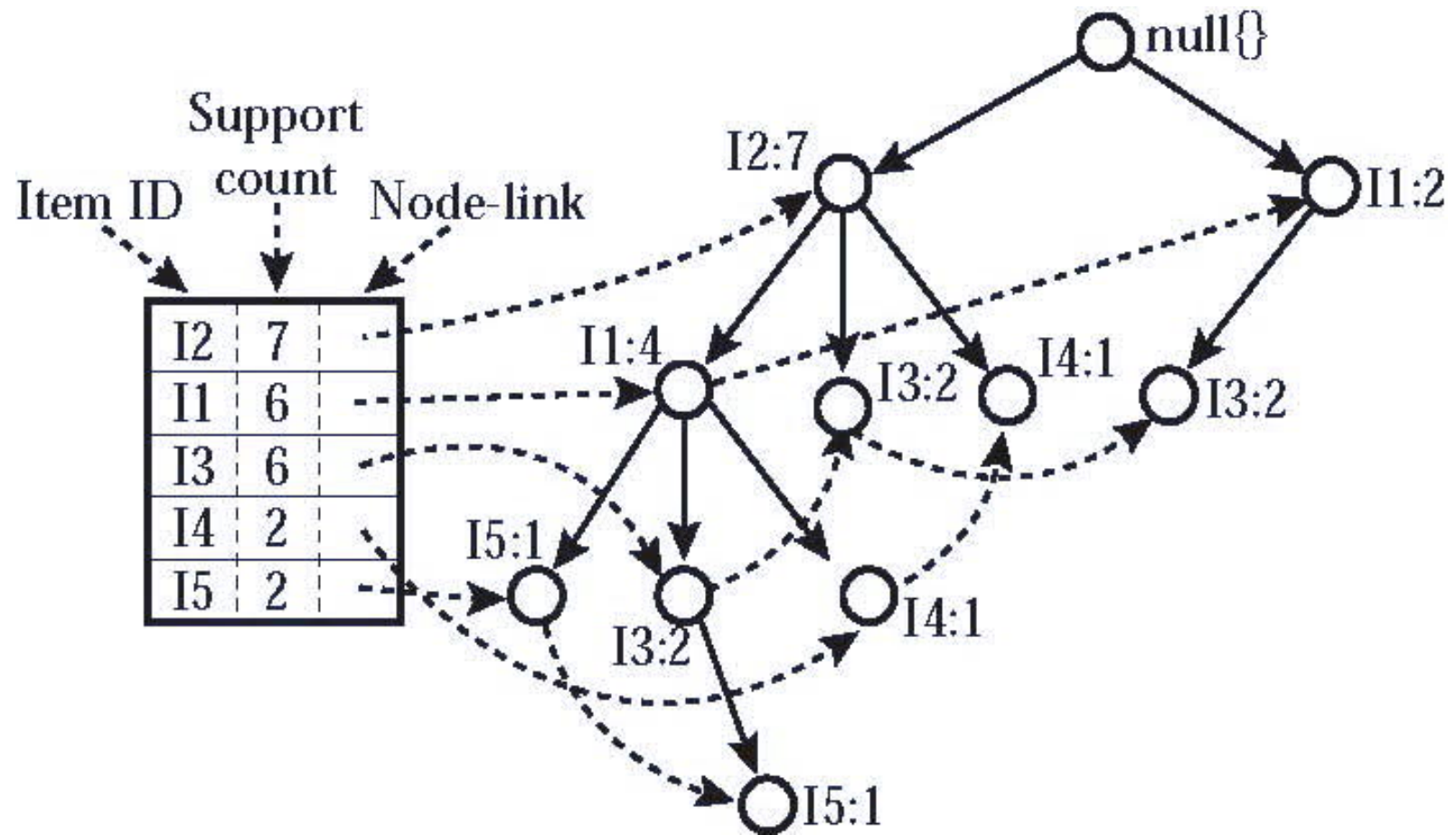
- ▶ Support counts  
for single items:

| Item | Sup. count |
|------|------------|
| {I1} | 6          |
| {I2} | 7          |
| {I3} | 6          |
| {I4} | 2          |
| {I5} | 2          |

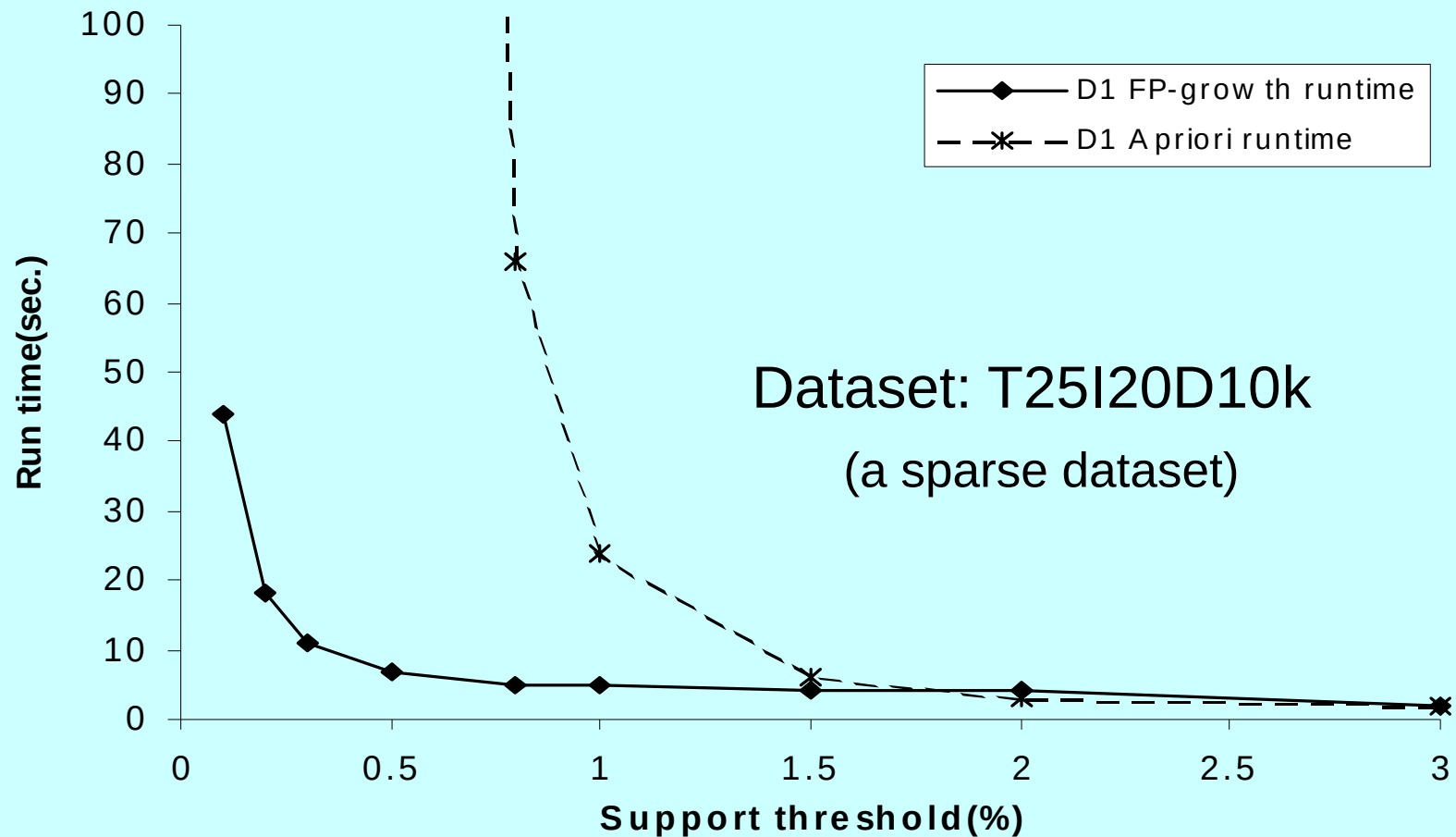
- ▶ Find all frequent patterns with minimum support count =2.



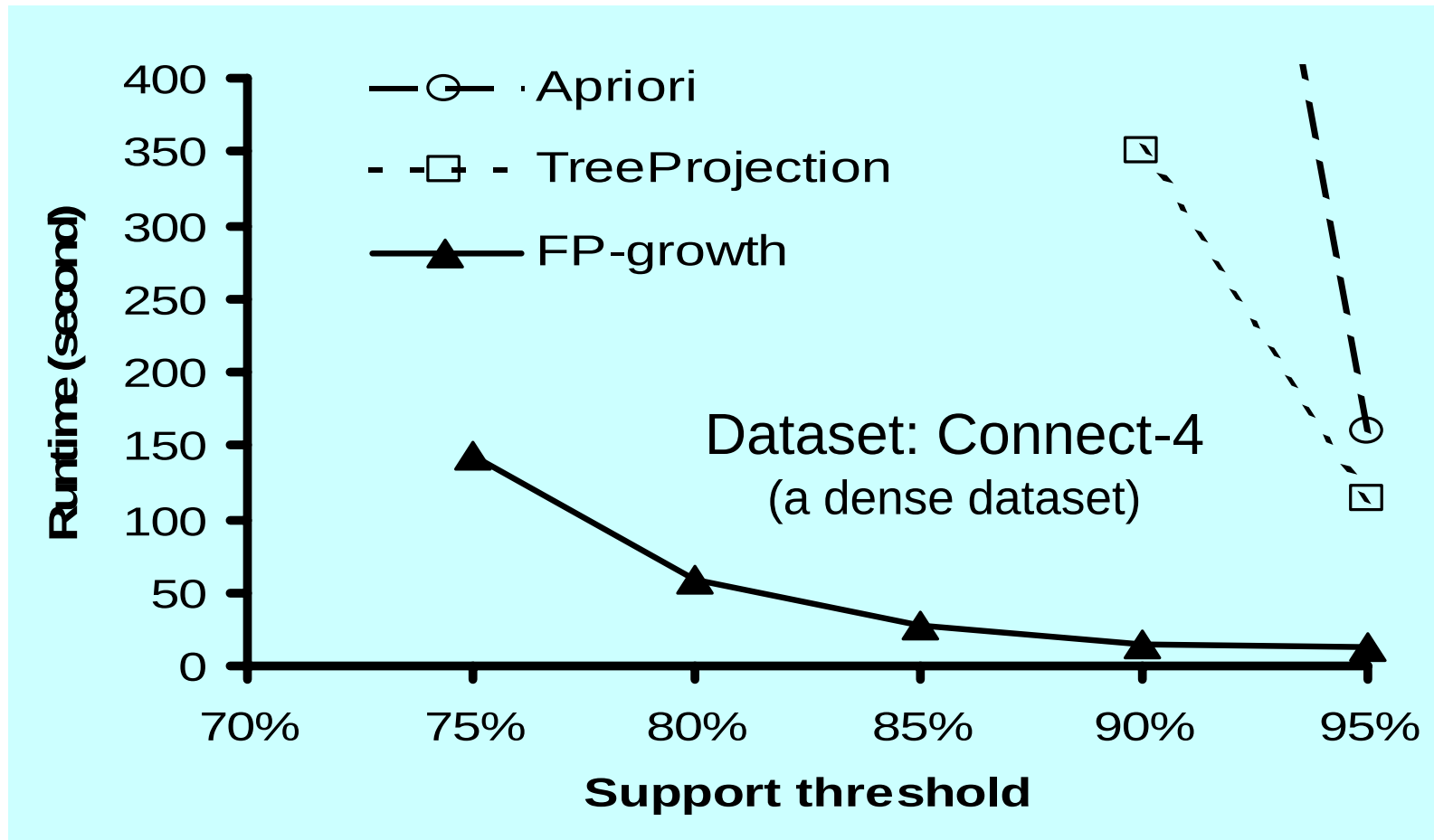
# FP-tree



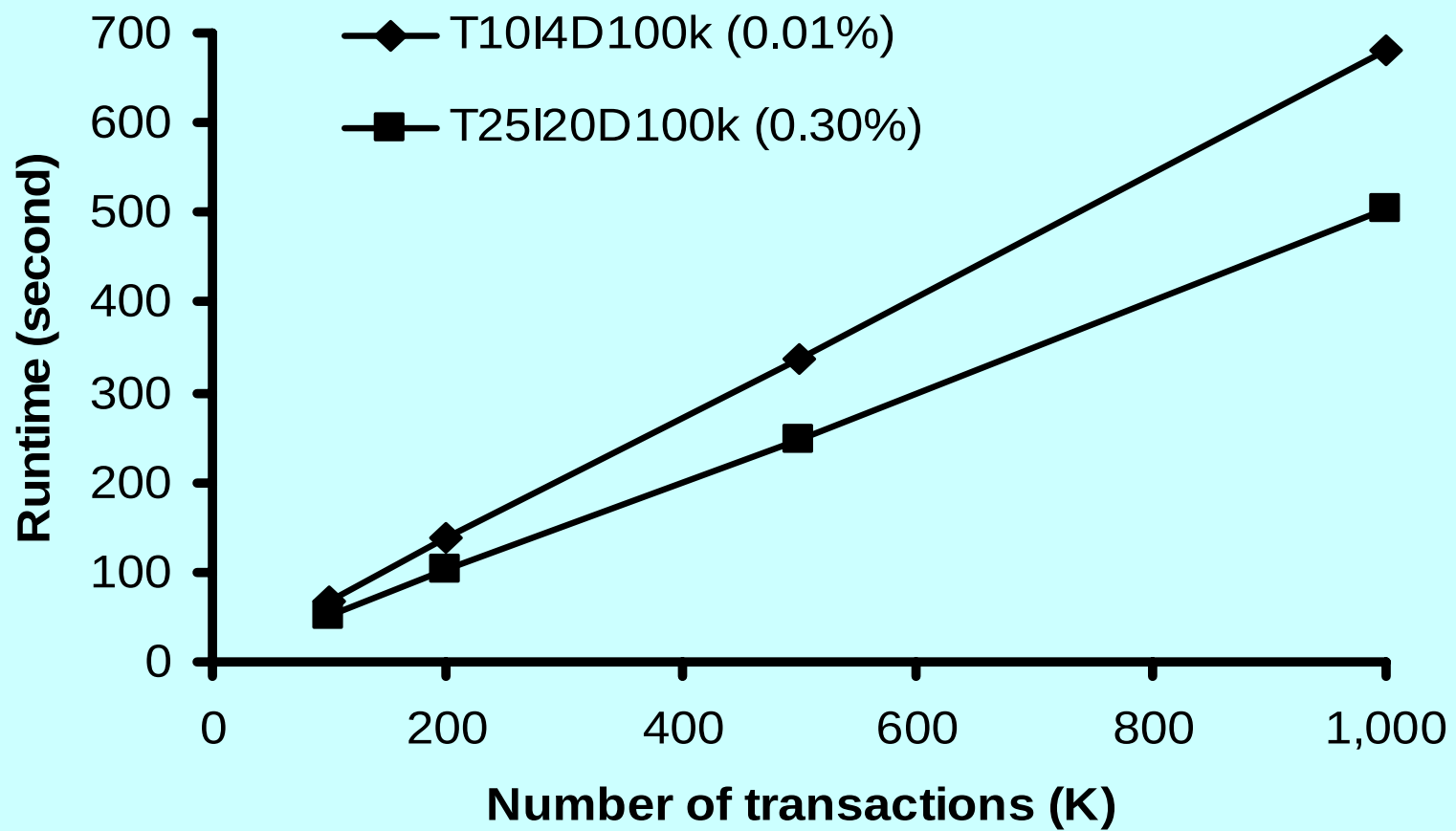
# FP-growth vs. Apriori



# Mining Very Dense Dataset



# Scalability of FP-growth



# Why Is FP-growth Efficient?

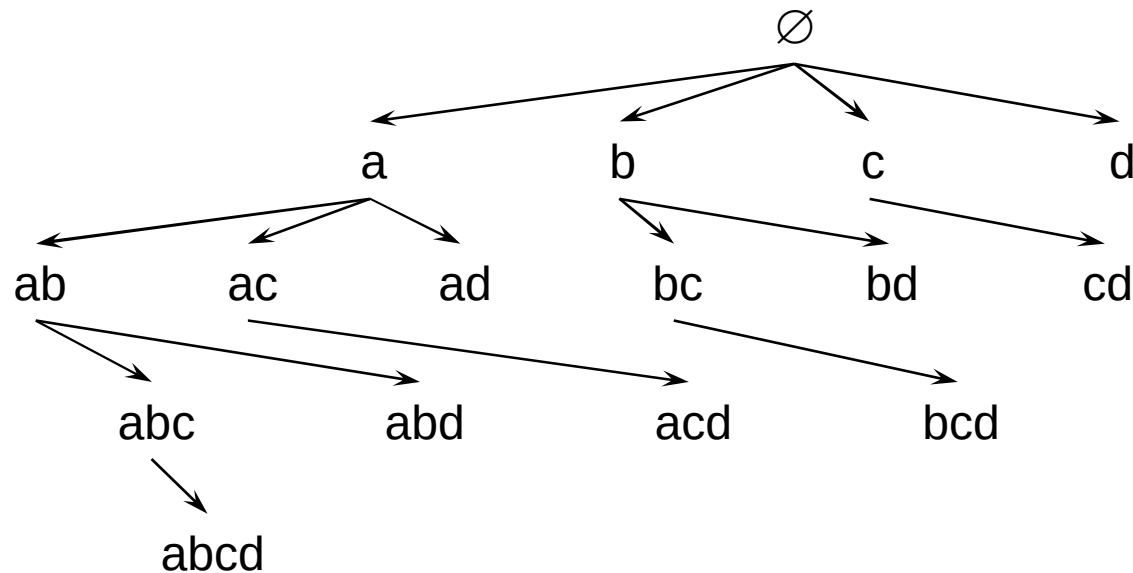
- ▶ Divide-and-conquer strategy
  - ▶ Decompose both the mining task and DB
  - ▶ Lead to focused search of smaller databases
- ▶ No candidate generation nor candidate test
- ▶ Database compression using FP-tree
  - ▶ No repeated scan of entire database
- ▶ Basic operations:
  - ▶ counting local freq items and building FP-tree, no pattern search nor pattern matching

# Major Costs in FP-growth

- ▶ Building FP-trees
  - ▶ A stack of FP-trees
- ▶ Redundant information is stored in a stack of FP-trees.
- ▶ Can we avoid the redundancy?
  - ▶ H-mine (another algorithm by Pei and Han)
    - ▶ Based on a different data structure.
    - ▶ Less memory usage
    - ▶ Better for sparse datasets
  - ▶ There is also other improvement to FP-growth based on the FP-tree structure.

# Compare FP-growth to Apriori

- ▶ Search space for DB with 4 items:



- ▶ Apriori: breadth-first
- ▶ FP-growth: depth-first

# What if FP-tree cannot fit in memory?

*(Partition to reduce # of disk scans)*

- ▶ Partition DB
  - ▶ Each partition is held in main memory
- ▶ Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB (can be proved)
  - ▶ Scan 1: partition database and for each partition find local frequent patterns
  - ▶ Scan 2: consolidate global frequent patterns
- ▶ A. Savasere, E. Omiecinski, and S. Navathe. An efficient algorithm for mining association in large databases. In *VLDB'95*



# Outline

- ▶ Basic concepts of association rule learning
- ▶ Apriori algorithm
- ▶ FP-Growth Algorithm
- ▶ Finding interesting rules

# Two Problems with Association Rule Mining

## ▶ Quantity problem

- ▶ Too many rules can be generated
  - ▶ Given a dataset, the number of rules generated depends on the support and confidence thresholds.
    - ▶ If the support threshold is high, a small number of rules are generated. But some interesting rules are missed.
    - ▶ If the support threshold is low, a huge number of rules are generated.

## ▶ Quality problem

- ▶ Not all the generated rules are interesting

## Number of Generated Patterns versus Support Threshold (An Example)

| Support threshold                   | 0.02 | 0.01 | 0.008 | 0.005 | 0.003 | 0.0028 | 0.0025 | 0.002     | 0.001   |
|-------------------------------------|------|------|-------|-------|-------|--------|--------|-----------|---------|
| Num. of rules<br>(conf. thres.=0.5) | 2    | 14   | 39    | 88    | 723   | 4,556  | 74,565 | 4,800,070 | $>10^9$ |
| Num. of rules<br>(conf. thres.=0.8) | 1    | 7    | 17    | 38    | 591   | 4,172  | 65,615 | 3,584,339 | $>10^9$ |

Number of sessions (transactions): 30586

Number of objects (items): 38679

# Solutions to the Problems

- ▶ Using interestingness measures to rank and remove rules
  - ▶ Remove misleading associations and find correlation rules
  - ▶ Prune patterns using other interestingness measures
- ▶ Constraint-based data mining
  - ▶ Applying constraints in the mining process so the search can be more focused and the results are more useful.
- ▶ Finding only *maximum* or *closed* frequent patterns
  - ▶ Other frequent patterns can be generated from them
- ▶ Using rule structures
  - ▶ Eliminate structurally and semantically redundant rules.
  - ▶ Group or summarize related rules

# Selected Solutions to the Problems

- ▶ *Using interestingness measures to remove or rank rules*
  - ▶ *Remove misleading associations and find correlation rules*
  - ▶ Prune patterns using other interestingness measures
- ▶ Constraint-based data mining
  - ▶ Applying constraints in the mining process so the search can be more focused.

# Misleading Association Rules

|            | Basketball | Not basketball | Sum (row) |
|------------|------------|----------------|-----------|
| Cereal     | 2000       | 1750           | 3750      |
| Not cereal | 1000       | 250            | 1250      |
| Sum(col.)  | 3000       | 2000           | 5000      |

- ▶ *play basketball*  $\Rightarrow$  *eat cereal* [40%, 66.7%] **is misleading**
  - ▶ The overall percentage of students eating cereal is 75% which is higher than 66.7%
- ▶ *play basketball*  $\Rightarrow$  *not eat cereal* [20%, 33.3%] is more positively correlated, although with lower support and confidence

**Association  $\neq$  Positive Correlation**

# Interestingness Measure: Correlation

- ▶ Correlation between two events A and B

- ▶ If  $P(A|B) > P(A)$ , A and B are *positively correlated*.

Note:  $P(A | B) > P(A) \Leftrightarrow P(B | A) > P(B) \Leftrightarrow P(A B) > P(A)P(B)$

- ▶ If  $P(A|B) < P(A)$ , A and B are *negatively correlated*.

Note:  $P(A | B) < P(A) \Leftrightarrow P(B | A) < P(B) \Leftrightarrow P(A B) < P(A)P(B)$

- ▶ If  $P(A|B) = P(A)$ , A and B are *independent*.

Note:  $P(A | B) = P(A) \Leftrightarrow P(B | A) = P(B) \Leftrightarrow P(A B) = P(A)P(B)$

- ▶ A measure of correlation (called **lift**)

$$corr_{A,B} = \frac{P(AB)}{P(A)P(B)}$$

## Pruning Misleading Rules (Keep Correlation Rules)

- ▶ A measure of correlation (**lift**) for rule  $A \rightarrow B$

$$\text{lift}(A \rightarrow B) = \frac{P(AB)}{P(A)P(B)}$$

- ▶ Rules whose lift  $\leq 1$  is **misleading**, which should be removed

- ▶ E.g. the following rule:

*play basketball  $\Rightarrow$  eat cereal [40%, 66.7%]*

should be removed because its lift is 0.89



# Solutions to the Problems

- ▶ *Using interestingness measures to remove or rank rules*
  - ▶ Remove misleading associations and find correlation rules
  - ▶ *Prune patterns using other interestingness measures*
- ▶ Constraint-based data mining
  - ▶ Applying constraints in the mining process so the search can be more focused.

# Many interestingness measures for $A \rightarrow B$

| symbol    | measure             | range            | formula                                                                                                                                                                                                                                                   |
|-----------|---------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\phi$    | $\phi$ -coefficient | -1 ... 1         | $\frac{P(A,B) - P(A)P(B)}{\sqrt{P(A)P(B)(1-P(A))(1-P(B))}}$                                                                                                                                                                                               |
| $Q$       | Yule's Q            | -1 ... 1         | $\frac{P(A,B)P(\bar{A},\bar{B}) - P(A,\bar{B})P(\bar{A},B)}{P(A,B)P(\bar{A},\bar{B}) + P(A,\bar{B})P(\bar{A},B)}$                                                                                                                                         |
| $Y$       | Yule's Y            | -1 ... 1         | $\frac{\sqrt{P(A,B)P(\bar{A},\bar{B})} - \sqrt{P(A,\bar{B})P(\bar{A},B)}}{\sqrt{P(A,B)P(\bar{A},\bar{B})} + \sqrt{P(A,\bar{B})P(\bar{A},B)}}$                                                                                                             |
| $k$       | Cohen's             | -1 ... 1         | $\frac{P(A,B) + P(\bar{A},\bar{B}) - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}$                                                                                                                                               |
| $PS$      | Piatetsky-Shapiro's | -0.25 ... 0.25   | $P(A,B) - P(A)P(B)$                                                                                                                                                                                                                                       |
| $F$       | Certainty factor    | -1 ... 1         | $\max\left(\frac{P(B A) - P(B)}{1 - P(B)}, \frac{P(A B) - P(A)}{1 - P(A)}\right)$                                                                                                                                                                         |
| $AV$      | added value         | -0.5 ... 1       | $\max(P(B A) - P(B), P(A B) - P(A))$                                                                                                                                                                                                                      |
| $K$       | Klogsen's Q         | -0.33 ... 0.38   | $\sqrt{P(A,B)} \max(P(B A) - P(B), P(A B) - P(A))$                                                                                                                                                                                                        |
| $g$       | Goodman-kruskal's   | 0 ... 1          | $\frac{\sum_j \max_k P(A_j, B_k) + \sum_k \max_j P(A_j, B_k) - \max_j P(A_j) - \max_k P(B_k)}{2 - \max_j P(A_j) - \max_k P(B_k)}$                                                                                                                         |
| $M$       | Mutual Information  | 0 ... 1          | $\frac{\sum_i \sum_j P(A_i, B_j) \log \frac{P(A_i, B_j)}{P(A_i)P(B_j)}}{\min(-\sum_i P(A_i) \log P(A_i) \log P(A_i), -\sum_i P(B_i) \log P(B_i) \log P(B_i))}$                                                                                            |
| $J$       | J-Measure           | 0 ... 1          | $\max(P(A,B) \log\left(\frac{P(B A)}{P(B)}\right) + P(\bar{A}\bar{B}) \log\left(\frac{P(\bar{B} \bar{A})}{P(\bar{B})}\right), \\ P(A,B) \log\left(\frac{P(A B)}{P(A)}\right) + P(\bar{A}\bar{B}) \log\left(\frac{P(\bar{A} \bar{B})}{P(\bar{A})}\right))$ |
| $G$       | Gini index          | 0 ... 1          | $\max(P(A)[P(B A)^2 + P(\bar{B} \bar{A})^2] + P(\bar{A})[P(B \bar{A})^2 + P(\bar{B} \bar{A})^2] - P(B)^2 - P(\bar{B})^2, \\ P(B)[P(A B)^2 + P(\bar{A} B)^2] + P(\bar{B})[P(A \bar{B})^2 + P(\bar{A} \bar{B})^2] - P(A)^2 - P(\bar{A})^2)$                 |
| $s$       | support             | 0 ... 1          | $P(A,B)$                                                                                                                                                                                                                                                  |
| $c$       | confidence          | 0 ... 1          | $\max(P(B A), P(A B))$                                                                                                                                                                                                                                    |
| $L$       | Laplace             | 0 ... 1          | $\max\left(\frac{NP(A,B)+1}{NP(A)+2}, \frac{NP(A,B)+1}{NP(B)+2}\right)$                                                                                                                                                                                   |
| $IS$      | Cosine              | 0 ... 1          | $\frac{P(A,B)}{\sqrt{P(A)P(B)}}$                                                                                                                                                                                                                          |
| $\gamma$  | coherence(Jaccard)  | 0 ... 1          | $\frac{P(A,B)}{P(A)+P(\bar{B})-P(A,B)}$                                                                                                                                                                                                                   |
| $\alpha$  | all_confidence      | 0 ... 1          | $\frac{P(A,B)}{\max(P(A), P(B))}$                                                                                                                                                                                                                         |
| $o$       | odds ratio          | 0 ... $\infty$   | $\frac{P(A,B)P(\bar{A},\bar{B})}{P(\bar{A},B)P(A,\bar{B})}$                                                                                                                                                                                               |
| $V$       | Conviction          | 0.5 ... $\infty$ | $\max\left(\frac{P(A)P(\bar{B})}{P(A\bar{B})}, \frac{P(B)P(\bar{A})}{P(B\bar{A})}\right)$                                                                                                                                                                 |
| $\lambda$ | lift                | 0 ... $\infty$   | $\frac{P(A,B)}{P(A)P(B)}$                                                                                                                                                                                                                                 |
| $S$       | Collective strength | 0 ... $\infty$   | $\frac{P(A,B) + P(\bar{A}\bar{B})}{P(A)P(B) + P(\bar{A})P(\bar{B})} \times \frac{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A,B) - P(\bar{A}\bar{B})}$                                                                                                    |
| $\chi^2$  | $\chi^2$            | 0 ... $\infty$   | $\sum_i \frac{(P(A_i) - E_i)^2}{E_i}$                                                                                                                                                                                                                     |

# Pruning rules with interestingness measure

- ▶ Choose a measure in your belief to assess the significance of a rule  $A \rightarrow B$ .
- ▶ Rank the rules according to their interestingness value.
- ▶ Remove rules with small interestingness values

# Solutions to the Problems

- ▶ Using interestingness measures to remove or rank rules
  - ▶ Remove misleading associations and find correlation rules
  - ▶ Prune patterns using other interestingness measures
- ▶ *Constraint-based data mining*
  - ▶ *Applying constraints in the mining process so the search can be more focused.*

# Constrain-based Frequent Pattern and Association Rule Mining

- ▶ Mining frequent patterns or rules with constraint C
  - ▶ find all frequent patterns satisfying not only min\_sup, but also constraint C
  - ▶ find all association rules satisfying not only min\_sup and min\_conf, but also constraint C
- ▶ Examples of Constraints
  - ▶  $? \rightarrow$  *a particular product*
  - ▶ *a particular product*  $\rightarrow$   $?$
  - ▶ small sales (price  $<$  \$10) triggers big sales (sum  $>$  \$200)

# Constrain-based Frequent Pattern Mining (Cont'd)

- ▶ A naïve solution
  - ▶ Testing frequent patterns on C as a post-processing process
- ▶ Some constraints can be incorporated into the mining process to improve the efficiency
- ▶ More efficient approaches
  - ▶ Analyze the properties of constraints comprehensively
  - ▶ Push the constraint as deeply as possible inside the frequent pattern mining
  - ▶ Example: find all frequent itemsets containing item “b”
    - ▶ Can be easily incorporated in FP-growth.
    - ▶ What about Apriori?

# Types of Constraints

- ▶ Anti-monotonic constraints

- ▶ An itemset  $S$  satisfies the constraint, so does any of its subset (That is,  $S$  violates the constraint, so does any of its superset).

- ▶ Monotonic constraints

- ▶ An itemset  $S$  satisfies the constraint, so does any of its superset

- ▶ Examples

- ▶ Sum of the prices of items in  $S \leq 100$  is anti-monotonic
  - ▶ Maximum price in  $S \leq 15$  is anti-monotonic
  - ▶ Sum of the prices of items in  $S \geq 100$  is monotonic
  - ▶ Minimum price in  $S \leq 15$  is monotonic

# How to Use Antimonotonic or Monotonic Constraints in Mining

- ▶ Anti-monotonic constraints
  - ▶ In Apriori:
    - ▶ Use it to prune candidates in each iteration
  - ▶ In FP-growth
    - ▶ Use it to stop growing a pattern
- ▶ Monotonic constraints
  - ▶ If an itemset satisfies a monotonic constraint, no need to check its supersets on the constraint
    - ▶ Only checks their support



# Types of Constraints (Cont'd)

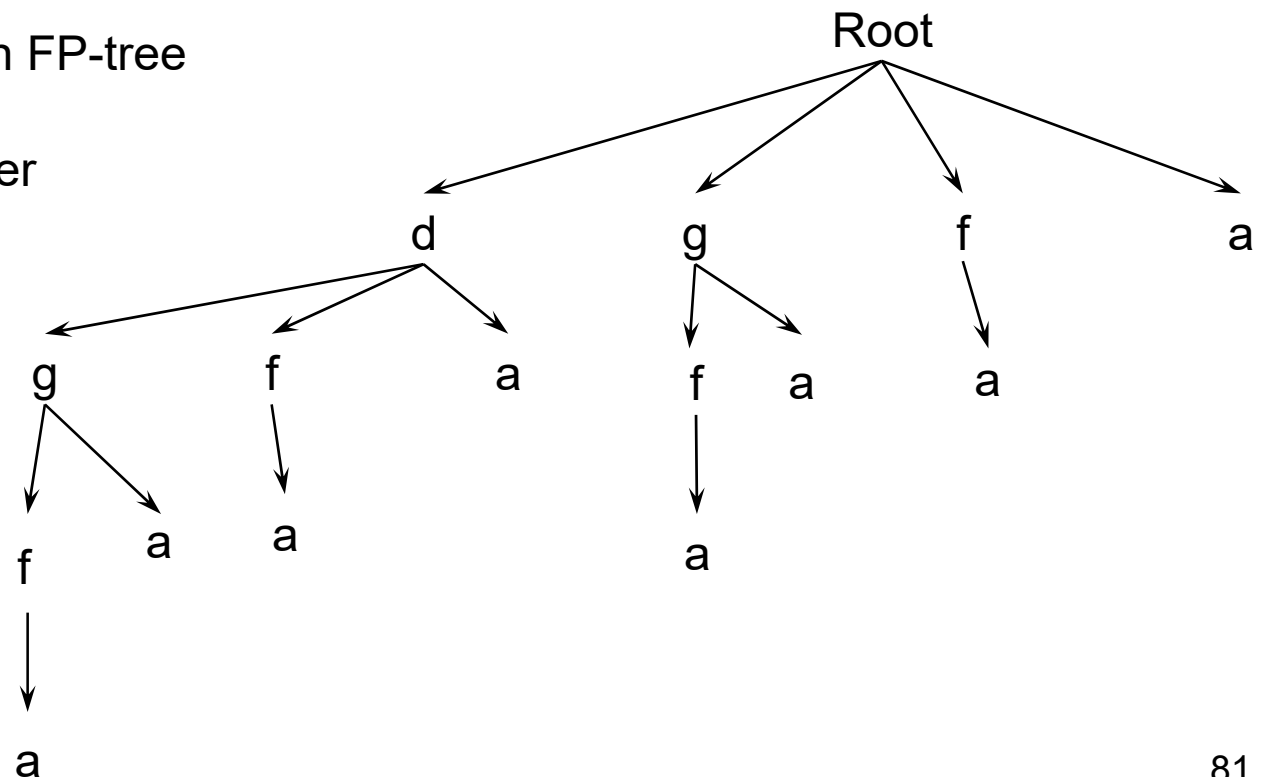
- ▶ Convertible constraints
  - ▶ Some constraints are not anti-monotonic or monotonic
  - ▶ But can be converted to anti-monotonic or monotonic by properly ordering items
- ▶ Example of convertible constraint:
  - ▶ Average price of the items in  $S \geq 25$
  - ▶ Order items in price-descending order
    - ▶  $\langle a, f, g, d, b, h, c, e \rangle$
  - ▶ If an itemset  $afb$  violates  $C$ 
    - ▶ So does  $afbh$ ,  $afb^*$
    - ▶ It becomes anti-monotone!

# Example of Convertible Constraints

- ▶ Convertible constraint:
  - ▶ Average price of the items in  $S \geq 25$
- ▶ Price-descending order of items: a, f, g, d

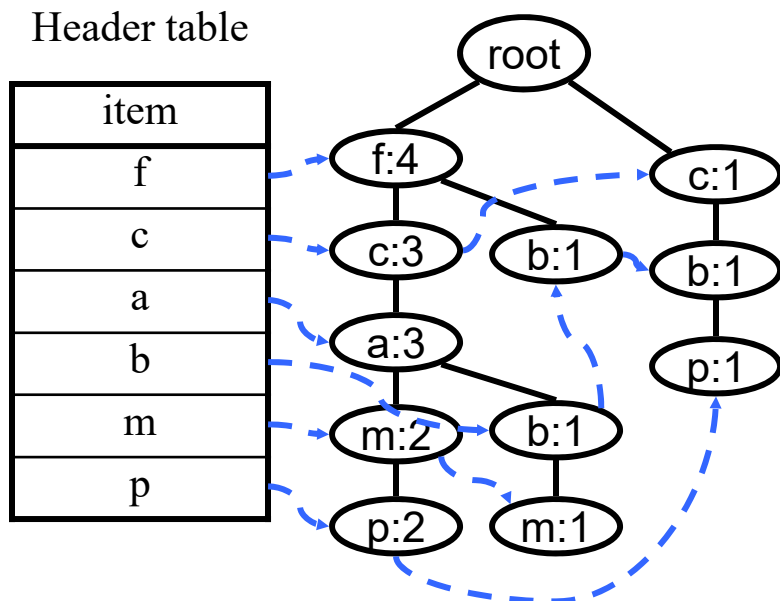
How would you build an FP-tree to do such a search?

- Price-ascending order



# FP-Growth (Recap)

- ▶ Build the tree in item-frequency-descending order (**f-list**: **f-c-a-b-m-p**)
- ▶ Grow frequent patterns in frequency-ascending order:
  - ▶ p->cp
  - ▶ m->am -> cam->fcam



## Conditional pattern bases

| <u>item</u> | <u>cond. pattern base</u> |
|-------------|---------------------------|
| <i>c</i>    | <i>f:3</i>                |
| <i>a</i>    | <i>fc:3</i>               |
| <i>b</i>    | <i>fca:1, f:1, c:1</i>    |
| <i>m</i>    | <i>fca:2, fcab:1</i>      |
| <i>p</i>    | <i>fcam:2, cb:1</i>       |

# Related Topics

- ▶ Contrast pattern mining
  - ▶ Finding patterns and models contrasting two or more classes or conditions
  - ▶ Contrasting groups:
    - ▶ Objects at different time periods
    - ▶ Objects at different spatial locations
    - ▶ Objects across different classes.
  - ▶ Measures for measuring the difference
    - ▶ Frequent/infrequent
    - ▶ Frequency ratio
    - ▶ Odds ratio, etc.
  - ▶ A challenge: need to find the supports of infrequent itemsets in a group.

## Related Topics

- ▶ Mining frequent patterns over data streams
  - ▶ A continuous flow of data generated often at high-speed in a dynamic, time-changing environment
  - ▶ Memory and disk are limited to hold all the data
  - ▶ Processing time may be limited by the rate of arrival of instances
  - ▶ **One scan of data set** is required for online mining
  - ▶ Pattern changes over time
    - ▶ Incremental learning
    - ▶ Change detection
    - ▶ etc

# Related Topics

## ▶ Mining high utility patterns

### ▶ Consider

- ▶ the quantity  $q(i, T_j)$  of an item  $i$  in a transaction  $T_j$
- ▶ the value (e.g., price  $p(i)$ ) of an item  $i$

### ▶ Utility of an item $i$ in a transaction $T_j$ :

$$u(i, T_j) = q(i, T_j) \times p(i)$$

### ▶ Utility of an itemset $X$ in a transaction $T_j$ :

$$u(X, T_j) = \sum_{X \subseteq T_j, i \in X} u(i, T_j)$$

### ▶ Utility of an itemset $X$ in a dataset $D$ :

$$u(X, D) = \sum_{X \subseteq T_j, T_j \in D} u(X, T_j)$$

- ▶ **High utility pattern**: itemsets whose utility in the dataset is no less than a minimum utility threshold

## Related Topics

- ▶ Mining high utility patterns (*cont'd*)
  - ▶ Challenge: utility does not have the downward closure property. That is,  
  
*The utility of a subset /superset of a set  $S$  may be smaller or larger than the utility of  $S$*
  - ▶ This means we cannot use Apriori or FP-growth to find high utility patterns directly since
    - ▶ the two algorithms use the downward closure property of support to cut down the search space

# Related Topics

- ▶ Mining high utility patterns (*cont'd*)
  - ▶ Solution:
    - ▶ Use a utility function  $f$  that is:
      - ▶ an upper bound of the true utility of a pattern  $X$ , i.e.,  $u(X, T_j) \leq f(X, T_j)$
      - ▶ with downward closure property
    - to generate candidates first
  - ▶ Then scan DB to find high utility patterns from the set of candidates



---

# Next Topic

- ▶ Sequential pattern mining (papers on the supplementary reading list)

# Decision Tree Learning

Aijun An

Department of Electrical Engineering and Computer Science  
York University

# Outline

- Basic concepts in decision tree learning
- How to learn a decision/classification tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - Pruning decision tree
  - How to deal with numeric attributes
  - Other issues in decision tree learning
- Regression tree learning
- Bagging and Boosting

# Decision Tree Learning

- Objective of decision tree learning
  - Learn a decision tree from a set of labeled training data
  - The learned tree can be used to make predictions on new examples
- Two types of decision trees
  - Classification tree
    - Purpose is to classify new examples
    - Target attribute is categorical
  - Regression tree
    - Purpose is to predict a numeric value for new examples
    - Target attribute is numeric

# Example of Training Data

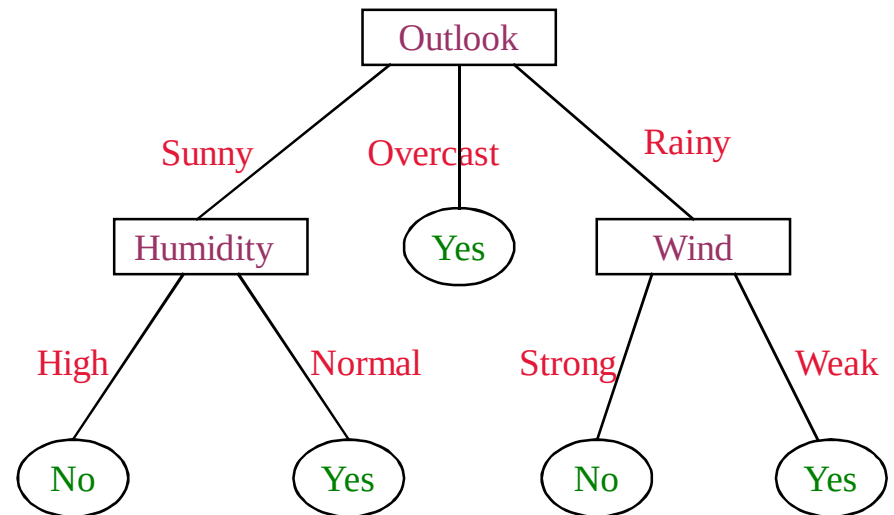
| Input/Condition/Independent attributes |          |             |          |        |            | Target/Output/Decision/Dependent/Class attribute |
|----------------------------------------|----------|-------------|----------|--------|------------|--------------------------------------------------|
| Day                                    | Outlook  | Temperature | Humidity | Wind   | PlayTennis |                                                  |
| D1                                     | Sunny    | Hot         | High     | Weak   | No         |                                                  |
| D2                                     | Sunny    | Hot         | High     | Strong | No         |                                                  |
| D3                                     | Overcast | Hot         | High     | Weak   | Yes        |                                                  |
| D4                                     | Rainy    | Mild        | High     | Weak   | Yes        |                                                  |
| D5                                     | Rainy    | Cool        | Normal   | Weak   | Yes        | Class or Class label                             |
| D6                                     | Rainy    | Cool        | Normal   | Strong | No         |                                                  |
| D7                                     | Overcast | Cool        | Normal   | Strong | Yes        |                                                  |
| D8                                     | Sunny    | Mild        | High     | Weak   | No         |                                                  |
| D9                                     | Sunny    | Cool        | Normal   | Weak   | Yes        |                                                  |
| D10                                    | Rainy    | Mild        | Normal   | Weak   | Yes        |                                                  |
| D11                                    | Sunny    | Mild        | Normal   | Strong | Yes        |                                                  |
| D12                                    | Overcast | Mild        | High     | Strong | Yes        |                                                  |
| D13                                    | Overcast | Hot         | Normal   | Weak   | Yes        |                                                  |
| D14                                    | Rainy    | Mild        | High     | Strong | No         |                                                  |

Training Examples, Instances, Cases, or Observations

for classification problems

# Decision Tree Representation

- *Decision tree*: a tree model for making decisions (e.g., classification)
  - Each non-leaf node tests a condition attribute (Outlook, Humidity, Wind)
  - Each branch corresponds to an attribute value
  - Each leaf node assigns a decision
    - a class for classification
    - a numeric value for regression
- To make a decision on a new example:
  - Test the example against the tree nodes from the root to a leaf node
  - For example:  
⟨Outlook = Sunny, Temperature = Mild, Humidity = high, Wind = Strong⟩ ⇒ No



This tree classifies days according to whether or not they are suitable for playing tennis.

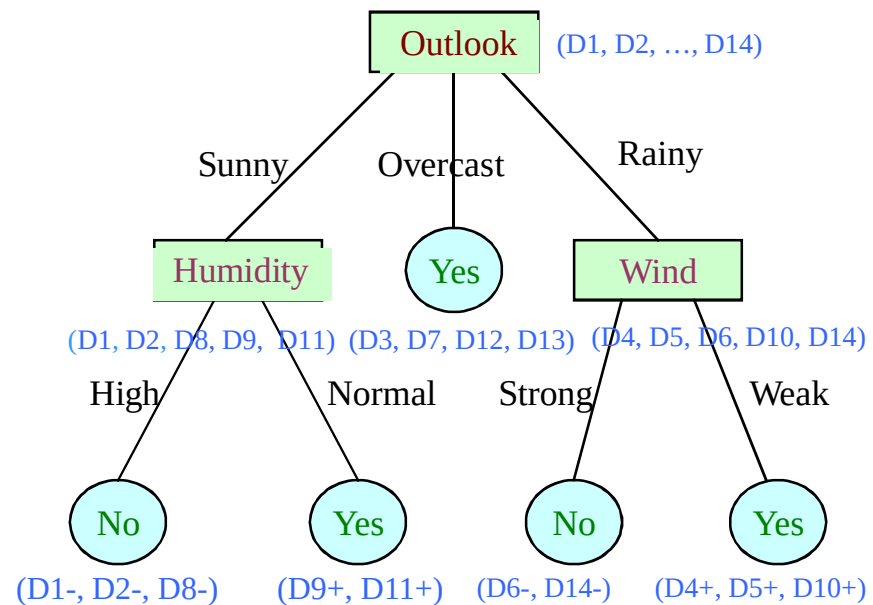
# Decision Tree Learning Algorithms

- Many decision tree learning algorithms
  - ID3 (Quinlan, 1986)
  - C4.5 (Quinlan, 1993)
  - CART (Breiman, Friedman, et. al. 1983)
  - CHAID (Kass, 1980)
  - QUEST (Loh and Shih, 1997)
  - etc.
- They use the same basic tree learning algorithm:
  - Top-down induction of a decision tree

# Top-Down Induction of a Decision Tree

- ▶ Idea: divide and conquer
- ▶ Method: recursive partitioning of training data according to the attribute selected for a tree node.

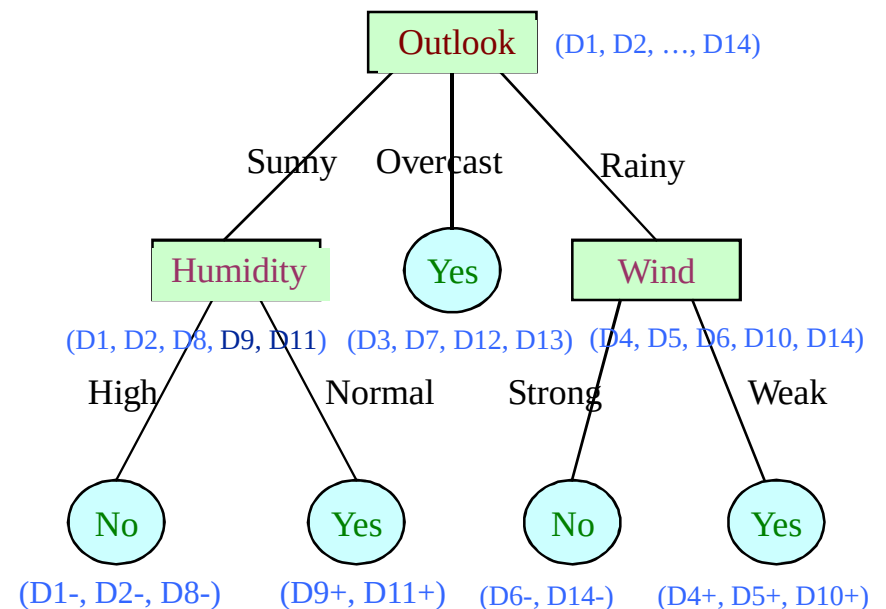
| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rainy    | Mild        | High     | Weak   | Yes        |
| D5  | Rainy    | Cool        | Normal   | Weak   | Yes        |
| D6  | Rainy    | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rainy    | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | Overcast | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rainy    | Mild        | High     | Strong | No         |





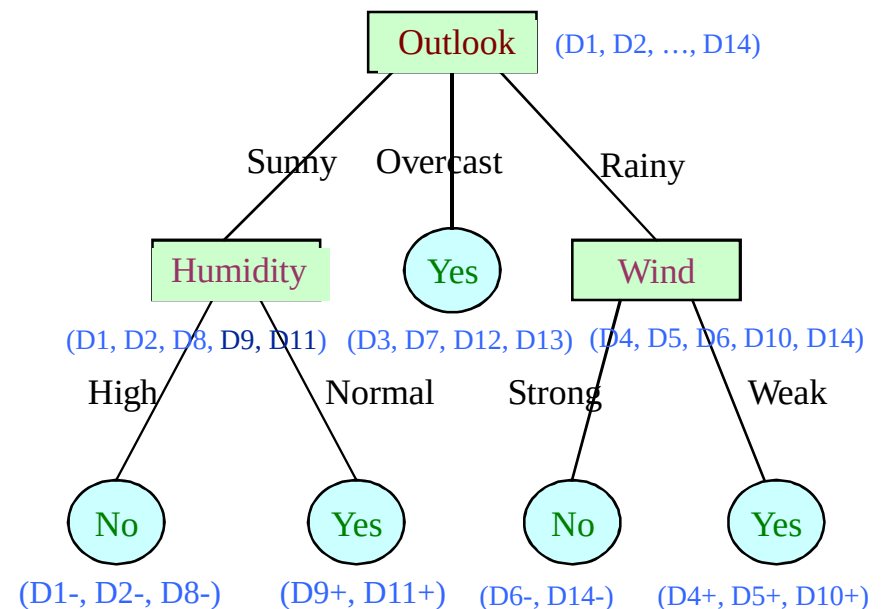
# Issues in Basic Decision Tree Induction

- ▶ Three issues:
  - ▶ How to select an attribute for a node
  - ▶ When to declare a node terminal
  - ▶ How to assign a label to a leaf node
- ▶ Solutions depend on whether the tree is for classification or regression
- ▶ Solutions vary among algorithms



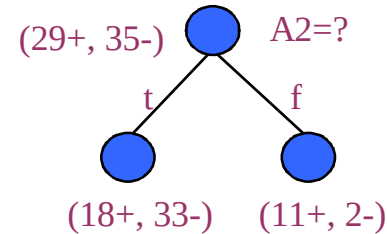
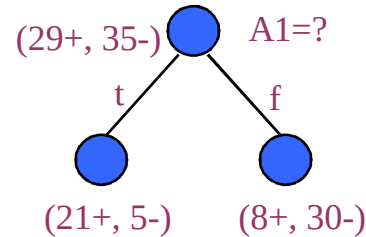
# Solutions to the Issues in **Classification Tree** Induction

- ▶ How to select an attribute for a node
  - ▶ Use a criterion (to be described)
- ▶ When to declare a node terminal
  - ▶ A naïve, but not robust method:  
when node is pure, stop growing.
  - ▶ Use a purity or another threshold
- ▶ How to assign a class to a leaf node
  - ▶ Assign the most common class in the examples in the node to the node
  - ▶ Or output the probability for each class



# How to Select Attribute for Classification

- ▶ Which attribute is the best attribute given a set of attributes and a set of examples?



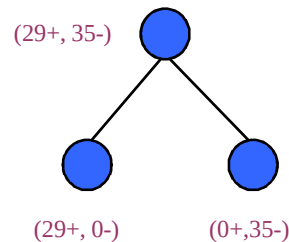
- ▶ Many selection criteria, including:
  - ▶ Information gain (Quinlan, 1983; used in ID3)
  - ▶ Gain ratio (Quinlan, 1986; used in C4.5)
  - ▶ Gini index (Breiman, 1984; used in CART)
  - ▶ Chi-square statistic (Kass, 1980; used in CHAID. Mingers, 1989)
  - ▶ Binarization (Bratko & Kononenko, 86)
  - ▶ Normalized information gain (Lopez de Mantaras, 91)

# Information Gain

- ▶ Objective:

- ▶ Select an attribute so that the data in each of the descendant subsets are the “purest”.

An ideal split:



*However, there may not be an attribute in the data set leading to such a split.*

- ▶ To measure impurity, use *entropy*

- ▶ *Entropy* is a measure, commonly used in information theory, that characterizes the impurity (uncertainty, chaos) of an arbitrary collection of examples.

## Entropy

- ▶ Given a set  $S$  of examples and  $k$  classes ( $C_1, \dots, C_k$ ), the *entropy* of  $S$  with respect to the  $k$  classes is defined as:

$$Entropy(S) = -\sum_{i=1}^k P(C_i) \log_2(P(C_i))$$

where  $P(C_i)$  is the probability of examples in  $S$  that belong to  $C_i$ .

- ▶ The bigger  $Entropy(S)$  is, the more impure  $S$  is.
- ▶ Examples:
  - ▶ If all examples in  $S$  belong to the same class (i.e.,  $S$  is pure),  
 $Entropy(S)=0$
  - ▶ If half of the examples in  $S$  belong to class 1 and the other half belong to class 2,  
 $Entropy(S)=1$
  - ▶ Suppose 9 examples are in class 1 and 5 examples in class 2,  
 $Entropy(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$
  - ▶ If the examples are uniformly distributed in 3 classes,  
 $Entropy(S) = -((1/3)\log_2(1/3)) \times 3 = \log_2 3 = 1.59$

## Information Gain (*Cont'd*)

- ▶ An attribute-selection criterion:
  - ▶ Used to choose an attribute to split a data set
- ▶ Assume that attribute  $A$  has  $m$  values.
  - ▶ Using  $A$ , data set  $S$  is split into  $S_1, S_2, \dots, S_m$ .
- ▶ Information Gain

$Gain(S, A)$  = expected reduction in entropy due to partitioning  $S$  on attribute  $A$

$$Gain(S, A) = Entropy(S) - \sum_{i=1}^m \frac{|S_i|}{|S|} Entropy(S_i)$$

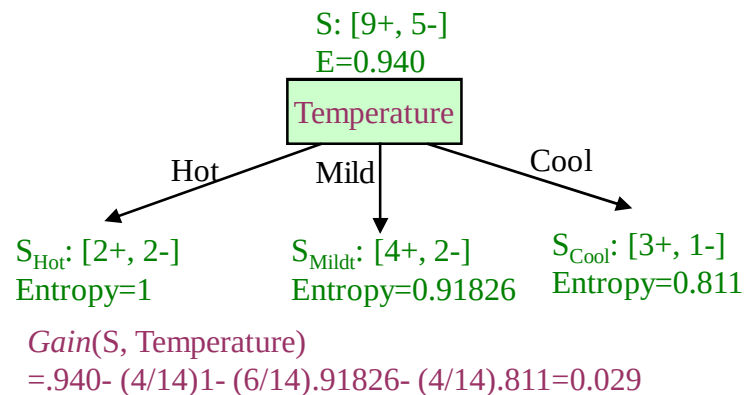
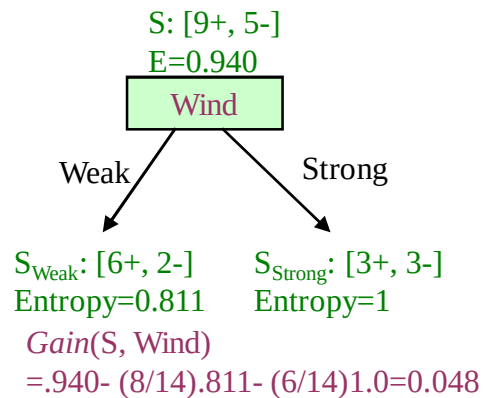
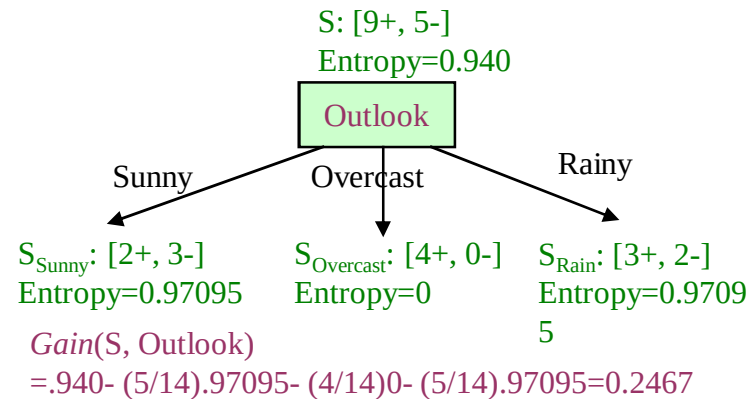
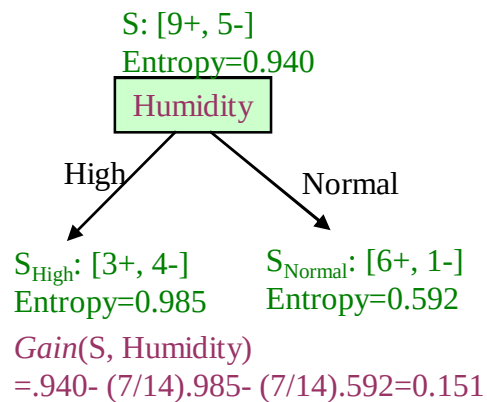
where  $|S|$  is the number of examples in set  $S$ , and  $|S_i|$  is the number of examples in  $S_i$ .

## An illustrative example

### ► Training examples

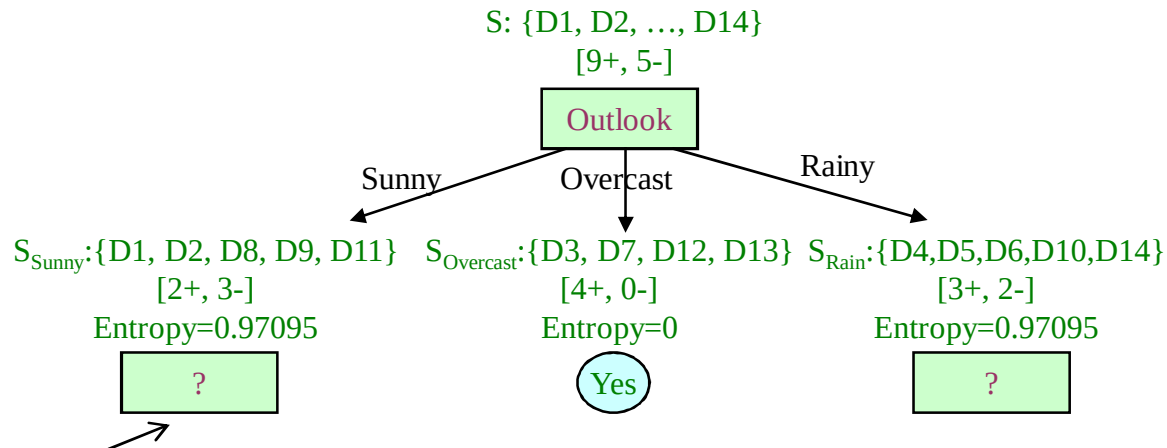
| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rainy    | Mild        | High     | Weak   | Yes        |
| D5  | Rainy    | Cool        | Normal   | Weak   | Yes        |
| D6  | Rainy    | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rainy    | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | Overcast | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rainy    | Mild        | High     | Strong | No         |

## Which attribute is the best for the root?





## An illustrative example (Cont'd.)



Which attribute should be tested here, Humidity, Temperature, or Wind?

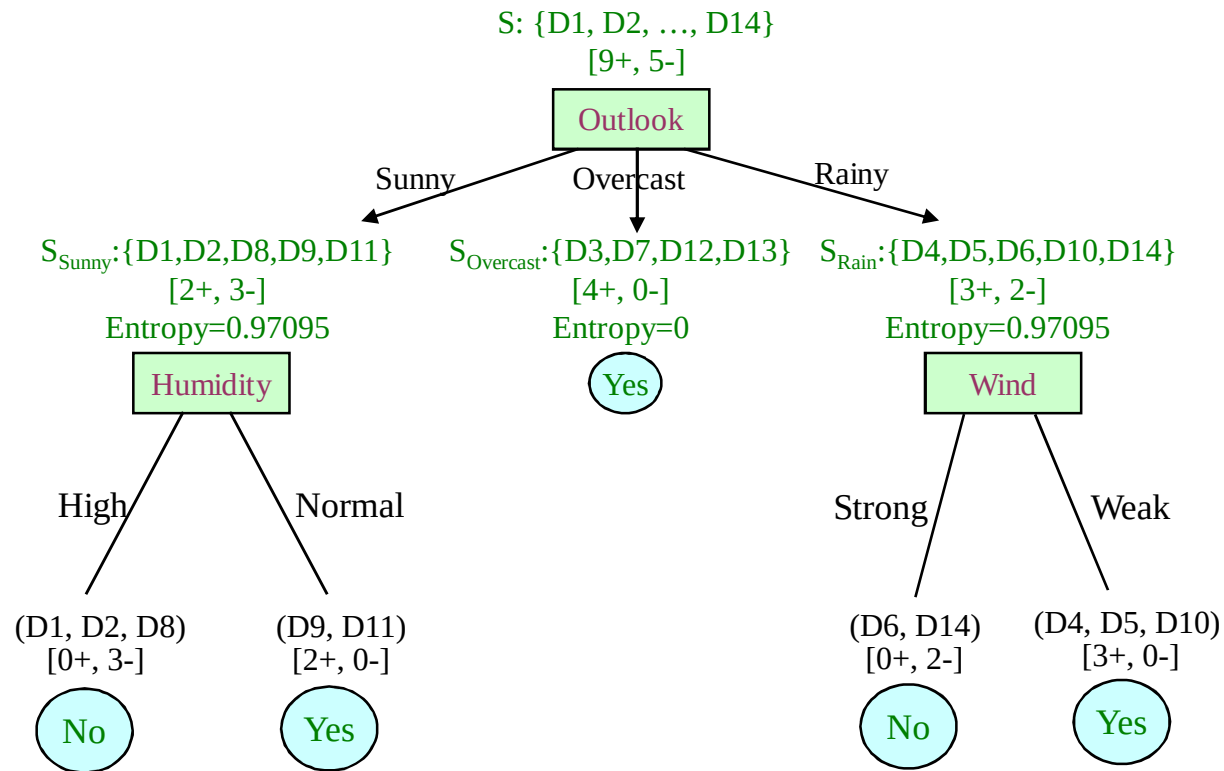
$$Gain(S_{\text{sunny}}, \text{Humidity}) = .97095 - (3/5)0.0 - (2/5)0.0 = 0.97095$$

$$Gain(S_{\text{sunny}}, \text{Temperature}) = .97095 - (2/5)0.0 - (2/5)1.0 - (1/5)0.0 = 0.57095$$

$$Gain(S_{\text{sunny}}, \text{Wind}) = .97095 - (2/5)1.0 - (3/5).918 = 0.02015$$

Therefore, **Humidity** is chosen as the next test attribute for the left branch.

## An illustrative example (Cont'd.)



# Basic Decision Tree Learning Algorithm

1. Select the “best” attribute  $A$  for the root node
2. Create new descendents of the node according to the values of  $A$ :
3. Sort training examples to the descendent nodes.
4. For each descendent node,
  - ▶ if there is no example for the node, the node is marked as a leaf node and labeled with the majority class in its parent node.
  - ▶ else if the training examples associated with the node belong to the same class, the node is marked as a leaf node and labeled with the class
  - ▶ else if there are no remaining attributes on which the examples can be further partitioned, the node is marked as a leaf node and labeled with the most common class among the training cases for classification;
  - ▶ otherwise, recursively apply the process on the new node.

*when to  
terminate  
the  
recursive  
process*

---

# Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - **How to select the best attribute**
    - Information gain ( $\sqrt{\cdot}$ )
    - Gain ratio
    - Gini index
  - Pruning decision tree
  - How to deal with numeric attributes
- Regression tree learning
- Bagging and Boosting

## Bias in the Information Gain Measure

- ▶ Favor unfairly attributes with large numbers of distinct values at the expense of those with few.
  - ▶ E.g., attribute Date: poor predictor, but has the highest gain because it alone perfectly predicts the target attribute over the training data

| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rainy    | Mild        | High     | Weak   | Yes        |
| D5  | Rainy    | Cool        | Normal   | Weak   | Yes        |
| D6  | Rainy    | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rainy    | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | Overcast | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rainy    | Mild        | High     | Strong | No         |

## Gain Ratio

- ▶ Proposed by Quinlan in 1986 (used in C4.5)
- ▶ Idea:
  - ▶ penalizes attributes with many distinct values by dividing information gain by attribute information (**entropy of data with respect to the values of attribute**):

$$SplitInformation(S, A) = - \sum_{v_i \in Values(A)} \frac{|S_{v_i}|}{|S|} \log_2 \frac{|S_{v_i}|}{|S|} = - \sum_{v_i \in Values(A)} P(v_i) \log_2 P(v_i)$$

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInformation(S, A)}$$

## Gini Index

- ▶ Gini diversity index (used by CART)

- ▶ Another measure that measures the impurity of a data set.
- ▶  $S$  is a set of training examples associated with a node
- ▶ Suppose there are  $n$  classes:  $C_i$  ( $i = 1, \dots, n$ )
- ▶  $P(C_i)$  is the probability of examples in  $S$  that belong to  $C_i$ .
- ▶ The Gini impurity of  $S$  with respect to classes can be measured as:

$$i(S) = \sum_{j \neq i} P(C_j)P(C_i) = 1 - \sum_{j=1}^n (P(C_j))^2$$

- ▶ Similar to entropy

- ▶ Minimized if classes for all examples are the same
- ▶ Maximized if equal proportion of classes

- ▶ Selection of attribute using Gini index selects an attribute  $A$  that most reduces the impurity due to partitioning on  $A$ :

$$\Delta i(S, A) = i(S) - \sum_{v_i \in \text{Values}(A)} \frac{|S_{v_i}|}{|S|} i(S_{v_i})$$

# Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - *Pruning decision tree*
  - How to deal with numeric attributes
  - Other issues in decision tree learning
- Regression tree learning
- Bagging and Boosting



# Basic Decision Tree Learning Algorithm

1. Select the “best” attribute  $A$  for the root node
2. Create new descendents of the node according to the values of  $A$ :
3. Sort training examples to the descendent nodes.
4. For each descendent node,
  - ▶ if the training examples associated with the node belong to the same class, the node is marked as a leaf node and labeled with the class
  - ▶ else if there are no remaining attributes on which the examples can be further partitioned, the node is marked as a leaf node and labeled with the most common class among the training cases for classification;
  - ▶ else if there is no example for the node, the node is marked as a leaf node and labeled with the majority class in its parent node.
  - ▶ otherwise, recursively apply the process on the new node.

*when to  
terminate  
the  
recursive  
process*

---

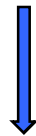
## Overfitting Problem

- ▶ When to declare a node terminal in the basic algorithm:
  - ▶ grow each branch of the tree just deeply enough to classify the training examples as perfectly as possible.
- ▶ This strategy may lead to producing deep branches that cover very few examples.
- ▶ This kind of trees *overfits* the training data in the following two situations:
  - ▶ there is noise in the data → tree fits the noise.
  - ▶ the number of training examples is too small to produce a representative sample of the true target function → tree is too specific to classify future examples well.

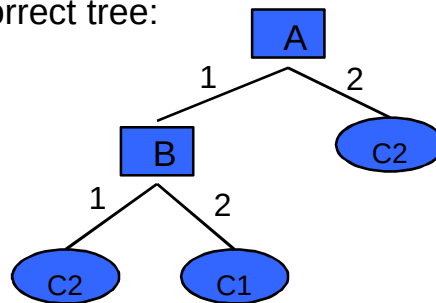
## Example of Overfitting Problem

| A | B | C | Class |
|---|---|---|-------|
| 1 | 1 | 1 | C2    |
| 1 | 2 | 1 | C1    |
| 2 | 1 | 2 | C2    |
| 1 | 2 | 1 | C1    |
| 1 | 2 | 2 | C2    |
| 1 | 2 | 1 | C1    |
| 2 | 1 | 1 | C2    |
| 1 | 2 | 1 | C1    |

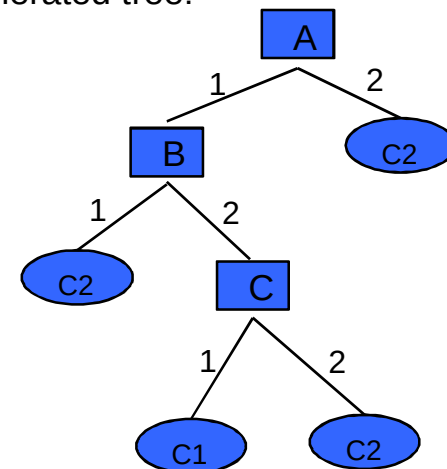
- Assume the model is:  
If  $A=1 \wedge B=2$ , Class=C1;  
Otherwise, Class=C2



Correct tree:



Generated tree:

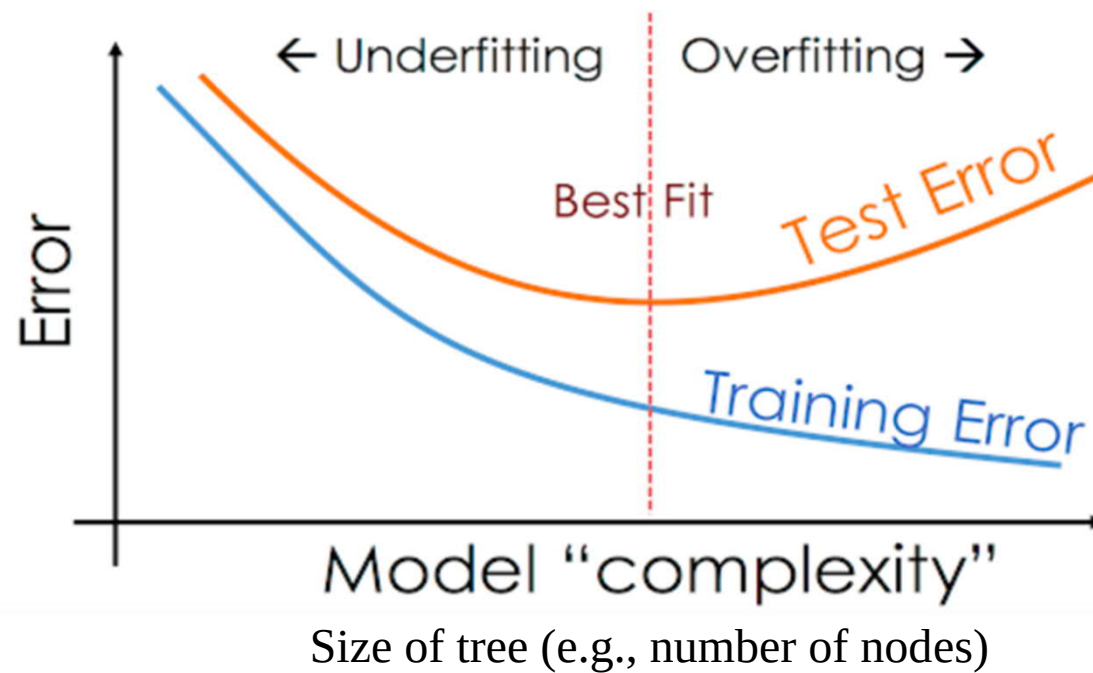


---

## Overfitting Problem (*Cont'd*)

- ▶ A definition of *overfitting*
  - ▶ Consider the error of a model (e.g., a tree)  $h$  over
    - ▶ training data:  $error_{train}(h)$ 
      - ▶ Percentage of examples in training data misclassified by  $h$
    - ▶ entire distribution  $D$  of data:  $error_D(h)$ 
      - ▶ Percentage of examples in entire data misclassified by  $h$
  - ▶ Model  $h$  overfits training data if there is an alternative model  $h'$  such that
$$error_{train}(h) < error_{train}(h') \text{ and } error_D(h) > error_D(h')$$

## Overfitting in Decision Tree Learning (An Example Diagram)



---

## Occam's Razor

- Prefer short hypotheses that fit the observations
- Argument in favor:
  - Fewer short hypotheses than long ones
  - A short hypothesis that fits data unlikely to be a coincidence
  - A long hypothesis that fits data might be a coincidence
- Argument opposed:
  - There are many ways to define short hypotheses
  - What's so special about small sets based on the size of hypothesis
- Solution:
  - Find best/good short hypotheses (not random short ones)
  - Remove unnecessary parts of hypotheses

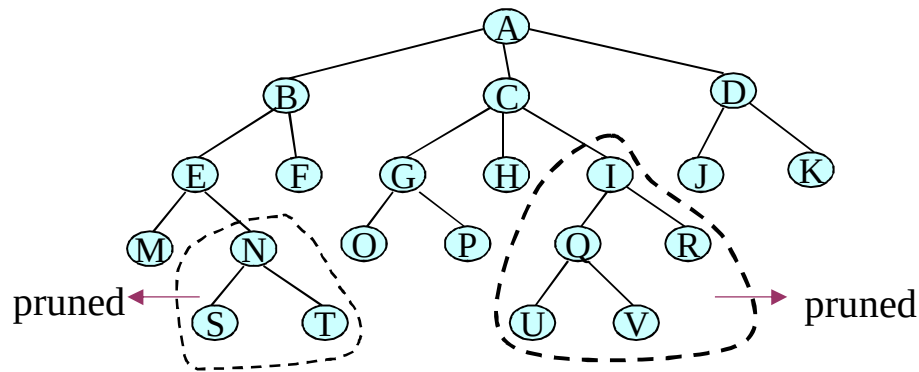
---

## Preventing Overfitting

- ▶ **Pre-pruning:** stop growing the tree when data split is not statistically significant
  - ▶ For example,
    - ▶ set a threshold  $\alpha > 0$  and declare a node terminal if percentage of examples in the most common class  $> \alpha$ , or
    - ▶ set a threshold  $\beta > 0$  and declare a node terminal if highest information gain  $< \beta$
  - ▶ Problems:
    - ▶ Hard to set the threshold value
    - ▶ The splitting is either stopped too soon or continued too far depending on the threshold
  - ▶ Using more complicated stopping rule does not help

## Preventing Overfitting (Cont'd)

- ▶ **Post-pruning:** grow full tree, allowing it to overfit the data, and then remove some subtrees



- ▶ More successful in practice
- ▶ Criterion is needed to determine what to prune



---

## Post-pruning Functions

- ▶ Post-pruning functions are needed to determine which part(s) of the tree should be pruned
- ▶ Several post-pruning functions, including:
  - ▶ Reduced-error (Quinlan, 83)
  - ▶ Error-complexity (CART, 84)
  - ▶ Pessimistic error (Quinlan, 86)
  - ▶ Minimum description length (MDL) (SLIQ by Mehta *et al.* 1996)
  - ▶ Minimum error (Niblett & Bratko, 86)
  - ▶ Critical value (Mingers, 87)
- ▶ There is no single best pruning algorithm

# Reduced-Error Pruning

- ▶ Procedure
  - ▶ Split *training* data into *building* and *validation* sets
  - ▶ Generate an overfitting decision tree using the *building* set
  - ▶ Post-pruning: Do until further pruning is harmful:
    - ▶ Consider each of the internal non-root nodes in the tree to be candidates for pruning
      - ▶ Prune a node by removing subtree rooted at this node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with this node
      - ▶ Evaluate impact of pruning this node on the *validation* set by
        - ▶ calculating the classification error rate of the pruned tree on the *validation* set and comparing it with the error rate of the unpruned tree.
    - ▶ Greedily remove the one whose removal most reduces the error on *validation* set.
- ▶ Aim to produce smallest version of most accurate subtree
- ▶ Limitations:
  - ▶ May be sub-optimal
  - ▶ When data is limited, data used to build the tree is small

## Error-Complexity Pruning

- ▶ Similar procedure to the reduced-error pruning
  - ▶ Split training data into *building* and *validation* sets
  - ▶ Generate the decision tree with the *building* set
  - ▶ Post-pruning: Do until further pruning is harmful:
    - ▶ Evaluate impact of pruning each subtree on the *validation* set
    - ▶ Greedily remove the one whose removal (resulting in tree  $T$ ) minimizes the following expression on the *validation* set:

$$R_{\alpha}(T) = E(T) + \alpha L(T)$$

where  $E(T)$  is the classification error of tree  $T$ ,  $L(T)$  is the number of leaf nodes in  $T$  and  $\alpha$  is the **complexity** cost per leaf node.

- ▶  $R_{\alpha}(T)$  is a linear combination of the classification error rate of the tree and its complexity.
- ▶ Similar to reduced-error pruning,
  - ▶ not suitable if the size of training data is small.
  - ▶ sub-optimal

## Pessimistic-Error Pruning

- ▶ This method does not require a separate validation set.
- ▶ Suppose a subtree  $T_s$  contains  $L$  leaves and  $J$  of training examples associated with  $T_s$  are misclassified by the tree
- ▶ If we replace  $T_s$  with a leaf which misclassifies  $E$  of the associated training examples, the pruned tree will be accepted if

$$E + 1/2 < J + L/2$$

- ▶ Perform top-down traversing over internal nodes
  - ▶ If an internal node is pruned, all its descendants are removed
  - ▶ Relatively fast pruning
- ▶ Advantage
  - ▶ fast: no need to separate growing and validation sets.
  - ▶ Tree building makes use of all the training data.

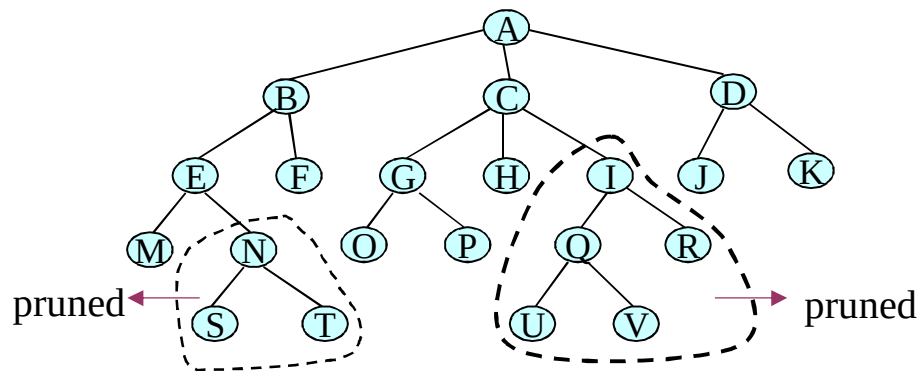
---

## Preventing Overfitting

- ▶ **Pre-pruning:** stop growing the tree when data split is not statistically significant
  - ▶ For example,
    - ▶ set a threshold  $\alpha > 0$  and declare a node terminal if percentage of examples in the most common class  $> \alpha$ , or
    - ▶ set a threshold  $\beta > 0$  and declare a node terminal if highest information gain  $< \beta$
  - ▶ Problems:
    - ▶ Hard to set the threshold value
    - ▶ The splitting is either stopped too soon or continued too far depending on the threshold
  - ▶ Using more complicated stopping rule does not help

## Preventing Overfitting (Cont'd)

- ▶ **Post-pruning:** grow full tree, allowing it to overfit the data, and then remove some subtrees



- ▶ More successful in practice
- ▶ Criterion is needed to determine what to prune

---

## Post-pruning Functions

- ▶ Post-pruning functions are needed to determine which part(s) of the tree should be pruned
- ▶ Several post-pruning functions, including:
  - ▶ Reduced-error (Quinlan, 83)
  - ▶ Error-complexity (CART, 84)
  - ▶ Pessimistic error (Quinlan, 86)
  - ▶ Minimum description length (MDL) (SLIQ by Mehta *et al.* 1996)
  - ▶ Minimum error (Niblett & Bratko, 86)
  - ▶ Critical value (Mingers, 87)
- ▶ There is no single best pruning algorithm

# Reduced-Error Pruning

- ▶ Procedure
  - ▶ Split *training* data into *building* and *validation* sets
  - ▶ Generate an overfitting decision tree using the *building* set
  - ▶ Post-pruning: Do until further pruning is harmful:
    - ▶ Consider each of the internal non-root nodes in the tree to be candidates for pruning
      - ▶ Prune a node by removing subtree rooted at this node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with this node
      - ▶ Evaluate impact of pruning this node on the *validation* set by
        - ▶ calculating the classification error rate of the pruned tree on the *validation* set and comparing it with the error rate of the unpruned tree.
    - ▶ Greedily remove the one whose removal most reduces the error on *validation* set.
- ▶ Aim to produce smallest version of most accurate subtree
- ▶ Limitations:
  - ▶ May be sub-optimal
  - ▶ When data is limited, data used to build the tree is small



## Error-Complexity Pruning

- ▶ Similar procedure to the reduced-error pruning
  - ▶ Split training data into *building* and *validation* sets
  - ▶ Generate the decision tree with the *building* set
  - ▶ Post-pruning: Do until further pruning is harmful:
    - ▶ Evaluate impact of pruning each subtree on the *validation* set
    - ▶ Greedily remove the one whose removal (resulting in tree  $T$ ) minimizes the following expression on the *validation* set:

$$R_{\alpha}(T) = E(T) + \alpha L(T)$$

where  $E(T)$  is the classification error of tree  $T$ ,  $L(T)$  is the number of leaf nodes in  $T$  and  $\alpha$  is the **complexity** cost per leaf node.

- ▶  $R_{\alpha}(T)$  is a linear combination of the classification error rate of the tree and its complexity.
- ▶ Similar to reduced-error pruning,
  - ▶ not suitable if the size of training data is small.
  - ▶ sub-optimal

## Pessimistic-Error Pruning

- ▶ This method does not require a separate validation set.
- ▶ Suppose a subtree  $T_s$  contains  $L$  leaves and  $J$  of training examples associated with  $T_s$  are misclassified by the tree
- ▶ If we replace  $T_s$  with a leaf which misclassifies  $E$  of the associated training examples, the pruned tree will be accepted if

$$E + 1/2 < J + L/2$$

- ▶ Perform top-down traversing over internal nodes
  - ▶ If an internal node is pruned, all its descendants are removed
  - ▶ Relatively fast pruning
- ▶ Advantage
  - ▶ fast: no need to separate growing and validation sets.
  - ▶ Tree building makes use of all the training data.

---

# Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - Pruning decision tree
  - *How to deal with numeric attributes*
- Learning a regression tree
- Bagging and Boosting

---

## How to deal with continuous attributes

Two ways:

- ▶ **Static discretization**

- ▶ Before learning a decision tree, converts a continuous attribute into a discrete attribute by partitioning the range of the continuous attribute into intervals.
- ▶ Interval labels can then be used to replace actual data values.

- ▶ **Dynamic discretization**

- ▶ Dynamically split the value range into two sub-ranges during the tree learning process

## Dynamic Discretization

- ▶ Dynamically split the value range into two sub-ranges and each descendent node corresponds to a sub-range.
  - ▶ Choose to split at the middle value between two examples that are in different categories

| Temperature | PlayTennis? |
|-------------|-------------|
| 40          | No          |
| 48          | No          |
| 60          | Yes         |
| 70          | Yes         |
| 80          | Yes         |
| 90          | No          |

The possible split points are  $\frac{48 + 60}{2} = 54$  and  $\frac{80 + 90}{2} = 85$

- ▶ Evaluate the binary splitting points using the splitting criterion used for selecting attribute. For example, choose the splitting point that leads to the best information gain to compete with other attributes.

# Dynamic Discretization

Example:

| Temperature | PlayTennis? |
|-------------|-------------|
| 40          | No          |
| 48          | No          |
| 60          | Yes         |
| 70          | Yes         |
| 80          | Yes         |
| 90          | No          |

The possible split points are  $\frac{48 + 60}{2} = 54$  and  $\frac{80 + 90}{2} = 85$

Possible binary splits:



Choose the one that gives the better information gain to compete with other attributes

---

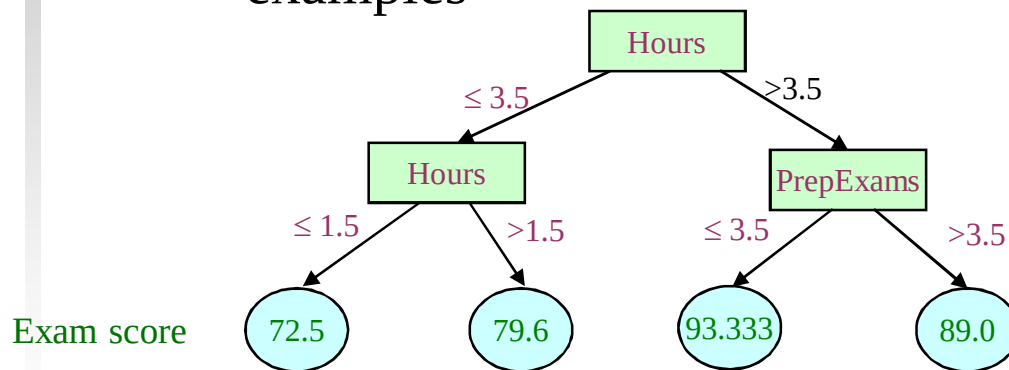
# Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - Pruning decision tree
  - How to deal with numeric attributes
- *Learning a regression tree*
- Bagging and Boosting

# Regression Tree

- When the target attribute is **numeric**,
  - a regression tree can be learned
  - to predict a numeric value for future examples

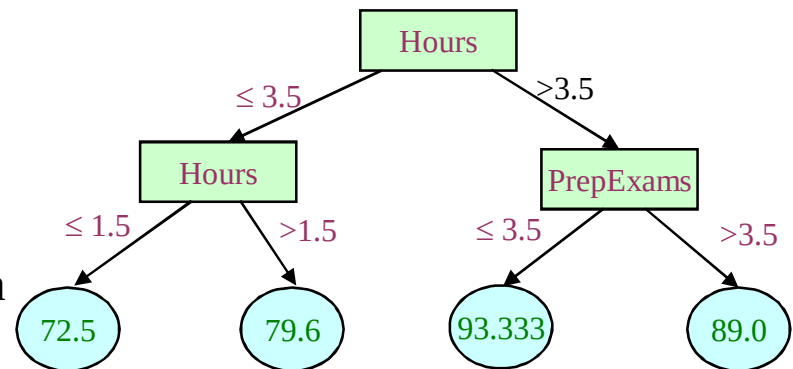
| Hours | Prep exams | Exam score |
|-------|------------|------------|
| 1     | 1          | 76         |
| 2     | 3          | 78         |
| 2     | 3          | 85         |
| 4     | 5          | 88         |
| 2     | 2          | 72         |
| 1     | 2          | 69         |
| 5     | 1          | 94         |
| 4     | 1          | 94         |
| 2     | 1          | 88         |
| 4     | 3          | 92         |
| 4     | 4          | 90         |
| 3     | 3          | 75         |





## Three Issues in Basic **Regression Tree** Induction

- ▶ How to select an attribute for a node
  - ▶ Variance (Mean Squared Error) Reduction
  - ▶ Mean Absolute Error (MAE) Reduction
- ▶ When to declare a node terminal
  - ▶ the variance is zero or less than a threshold,
  - ▶ number of examples is less than a threshold, or
  - ▶ level of the tree reaches a threshold
- ▶ How to assign a value to a leaf node
  - ▶ the mean value of examples on the target attribute

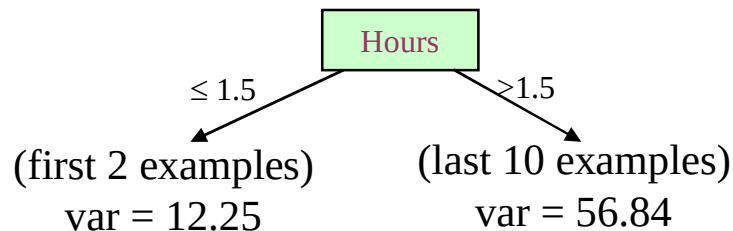


## Attribute Selection Using Variance Reduction for Regression Tree Induction

- Variance of examples in a node:

$$\frac{\sum (X - \bar{X})^2}{n}$$

- Evaluate each possible split:



the weighted average of node variance in a split:

$$\frac{2}{12} \times 12.25 + \frac{10}{12} \times 56.84 = 49.41$$

- Select the split with the lowest variance

| Hours | Prep exams | Exam score |
|-------|------------|------------|
| 1     | 1          | 76         |
| 1     | 2          | 69         |
| 2     | 3          | 78         |
| 2     | 3          | 85         |
| 2     | 2          | 72         |
| 2     | 1          | 88         |
| 3     | 3          | 75         |
| 4     | 5          | 88         |
| 4     | 1          | 94         |
| 4     | 3          | 92         |
| 4     | 4          | 90         |
| 5     | 1          | 94         |



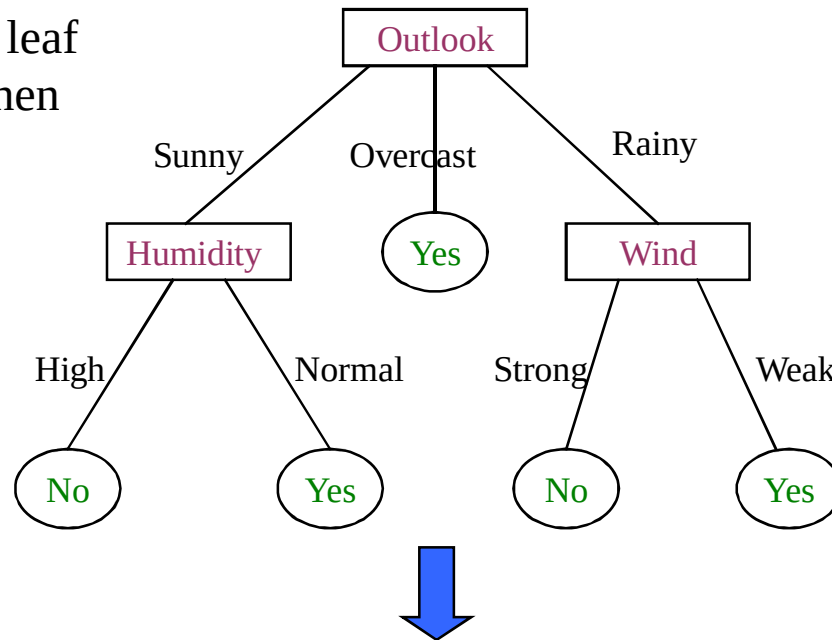
---

## Decision Trees: Strengths

- ▶ Comprehensibility:
  - ▶ Small trees are highly interpretable and intuitive for humans
  - ▶ Can be converted into rules
- ▶ Automatic attribute selection.
- ▶ Fast classification
- ▶ Relatively fast induction
- ▶ Applicable to both regression and classification problems.
- ▶ Mature technology

## Convert Decision Trees to a Set of Rules

- ▶ Each branch from the root to a leaf can be transformed into an if-then rule.



- ▶ If (Outlook is Sunny and Humidity is High), then class is No.
- ▶ If (Outlook is Sunny and Humidity is Normal), then class is Yes.
- ▶ If (Outlook is Overcast), then class is Yes.
- ▶ .....

---

## Decision Trees: Weaknesses

- ▶ Trees can become incomprehensible when their size grows. The large size can be caused by:
  - ▶ Rules converted from a tree
    - ▶ are *mutually exclusive*,
    - ▶ share at least one attribute (the root)
  - ▶ Thus, the size of the tree can grow much larger than the logic needed for overlapping rules (i.e., rules whose coverages overlap).
  - ▶ As an example, in a successful application of ID3 to a chess end game (Mitchie, 86), the tree representation could not be understood at all even by the chess experts.

---

## Decision Trees: Weaknesses (*Cont'd*)

- ▶ When using only one attribute at each internal node, trees are limited to axis-parallel partitions of the instance space
- ▶ Instability
  - ▶ If we change the data a little, the tree can change a lot. So does accuracy.
  - ▶ Can be improved by building a set of trees using *Bagging* and *Boosting*
    - ▶ Random forests (Bagging)
    - ▶ Boosted trees (Boosting)
      - ▶ AdaBoost
      - ▶ Gradient boosting (e.g., XGBoost)

---

# Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - Pruning decision tree
  - How to deal with numeric attributes
- Learning a regression tree
- *Bagging and Boosting*

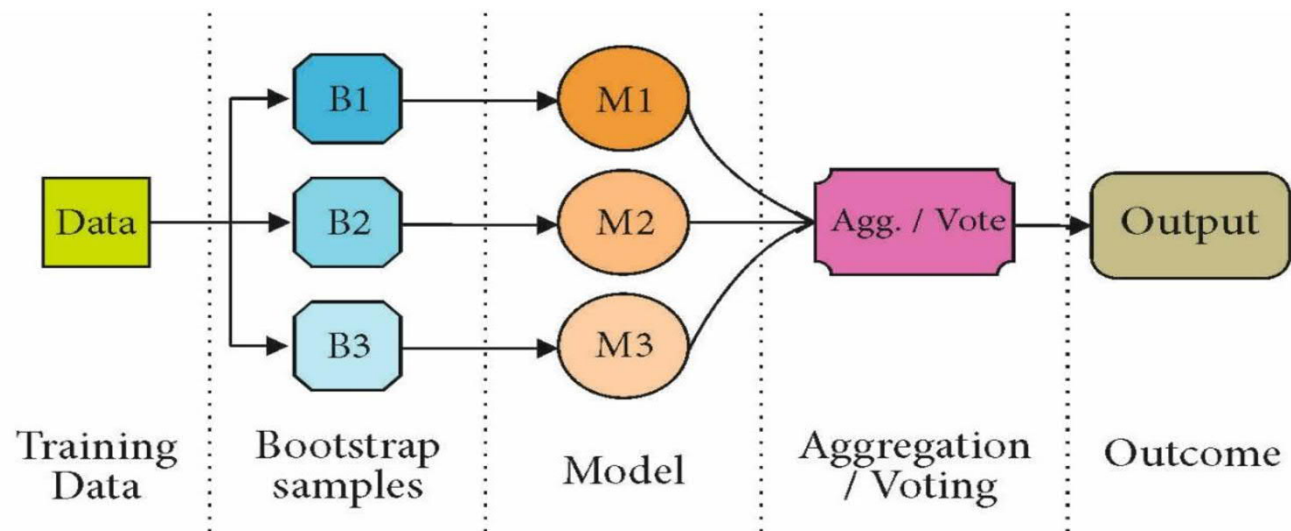
---

## Two General Machine Learning Approaches

- ▶ Single model approach
  - Build one really good model
- ▶ Ensemble learning
  - Build many models and average the results
  - Bagging and Boosting are two ensemble learning techniques



## Bagging (Bootstrap AGGregatING)



A bootstrap sample  $B_i$  is obtained by **random sampling  $N$  examples with replacement** from the data with  $N$  examples

(Picture from <https://analyticsindiamag.com/guide-to-ensemble-methods-bagging-vs-boosting/>)

---

## Random Forests (a Bagging Method)

- ▶ Grow a forest of many trees (e.g., 500)
- ▶ Grow each tree on an independent *bootstrap sample* from the training data with  $N$  examples
  - ▶ Bootstrap sample: Sample  $N$  examples at random with replacement
- ▶ At each node:
  - ▶ Select  $m$  attributes at random out of all  $M$  possible attributes (independently for each node).
  - ▶ Find the best split on the selected  $m$  variables.
- ▶ Grow the trees to maximum depth (i.e., no pruning)
- ▶ Vote/average the trees to get predictions for new data

---

# Random Forests

- ▶ Advantages:

- ▶ Accuracy

- ▶ Random Forests is competitive with the best-known machine learning methods

- ▶ Stability in predictive performance

- ▶ If we change the data a little, the individual trees may change but the predictive performance of the forest is relatively stable because it is a combination of many trees.

- ▶ Disadvantage

- ▶ Longer run time than generating a single tree

- ▶ Interpretability decreases – not interpretable if there are, say, 500 trees.

# Boosting Algorithms

- ▶ Principle:

- ▶ Generate **multiple weak predictors** ( $f_i(x)$ ,  $i=1, \dots, m$ ) and combine their predictions to form one strong prediction rule:

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x)$$

- ▶ **Weak predictors** are **simple predictors that can have an accuracy slightly better than random guessing**
    - ▶ such as a **decision stump**: one-level decision tree (which uses only a single attribute for splitting).
- ▶ Use a **sequential learning strategy** to add a new base model to improve the current ensemble

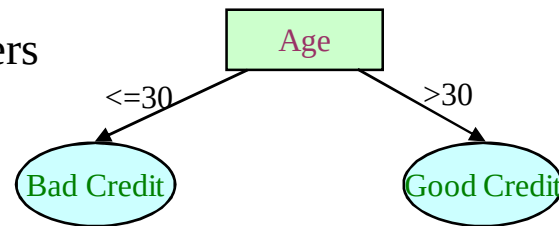
$$F_m(x) = F_{m-1}(x) + w_m f_m(x)$$

That is, learn  $f_1(x)$  first, then  $f_2(x)$ ,  $f_3(x)$ , and so on

# Famous Boosting Algorithms

- ▶ AdaBoost (Adaptive Boosting)

- ▶ Use decision stumps as weak learners



- ▶ Adjust the weights of training examples for subsequent learners to focus on misclassified examples

- ▶ Gradient Boosting

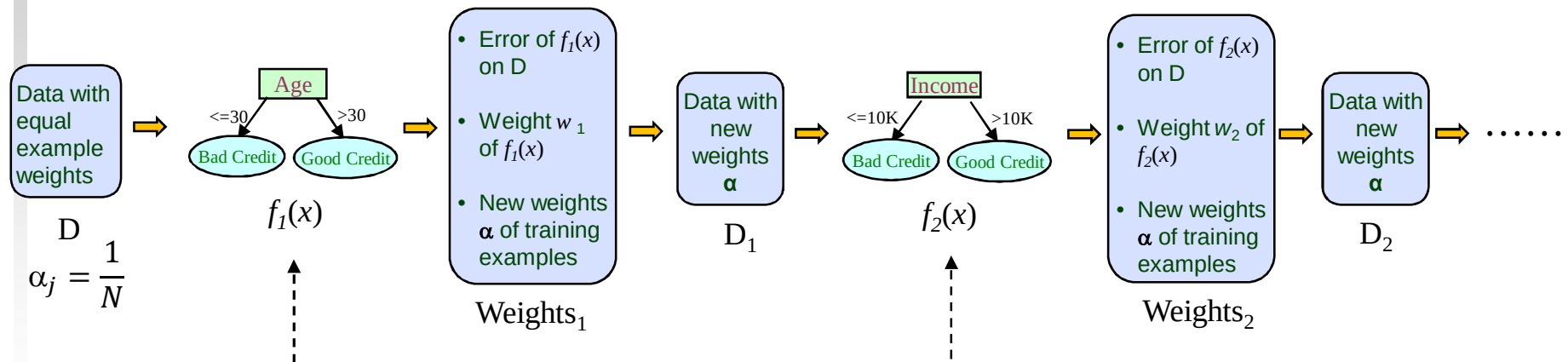
- ▶ Subsequent models learn to predict residuals (to “adjust” the previous ensemble)
  - ▶ XGBoost (eXtreme Gradient Boosting)

# AdaBoost

- Learning an ensemble of weighted **binary classifiers** :

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

Each classifier  $f_i(x)$   
outputs -1 or 1



Choose a decision stump that minimizes the weighted classification error

## How to Compute Error and Weights in AdaBoost

- ▶ Error  $\epsilon_i$  of  $f_i(x)$  is computed as:

$$\epsilon_i = \sum \alpha_j \text{ for all } x_j \text{ misclassified by } f_i(x)$$

which is a *weighted classification error*

- ▶ Weight  $w_i$  of  $f_i(x)$  is computed as:

$$w_i = \frac{1}{2} \ln \left( \frac{1 - \epsilon_i}{\epsilon_i} \right)$$

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x)$$

**Idea:** a less accurate base model has less weight in the final classifier.

- ▶ Weight of example  $x_j$  is adjusted using:

$$\alpha_j^{new} = \begin{cases} \frac{\alpha_j^{old} \cdot e^{w_i}}{Z_i} & \text{if } x_j \text{ is misclassified by } f_i(x) \\ \frac{\alpha_j^{old} \cdot e^{-w_i}}{Z_i} & \text{if } x_j \text{ is correctly classified by } f_i(x) \end{cases}$$

**Idea:** If an example is misclassified, increase its weight; otherwise decrease it.

$Z_i$  is a normalization factor (chosen so that  $\sum \alpha_j^{new} = 1$ ).

# AdaBoost

- ▶ How the weights are derived
  - ▶ Minimizing an exponential loss function
  - ▶ For details, see:
    - ▶ Yoav Freund and Robert E. Schapire, A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting:  
<https://www.sciencedirect.com/science/article/pii/S002200009791504X>
- ▶ Theorem:
  - ▶ If AdaBoost generates  $m$  base models with errors  $\epsilon_1, \epsilon_2, \dots, \epsilon_m$ , the error of the ensemble model  $F_m(x)$  is bounded as:

$$\epsilon \leq 2^m \prod_{i=1}^m \sqrt{\epsilon_i(1 - \epsilon_i)}$$

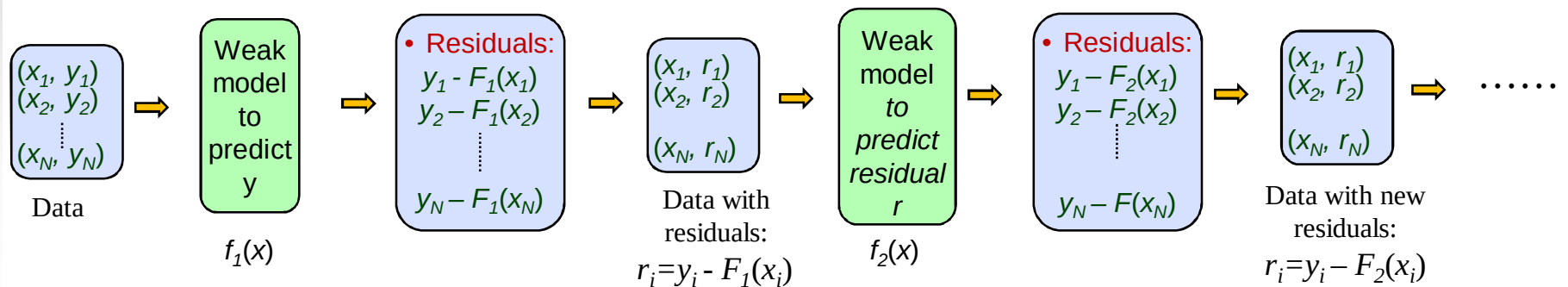


# Gradient Boosting

- ▶ Learning an ensemble of weighted predictors (such as **regressors**):

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

- ▶ The first model  $f_1(x)$  can be very weak: average of target values in the data
- ▶ Subsequent models (e.g., shallow regression trees) are built to predict residuals:



## Gradient Boosting

- ▶ In gradient boosting:

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

$w_i$  are learning rate, often set to a constant such as 0.1 for all the terms

- ▶ It is called “gradient” because training a tree on residuals is the same as training a tree on the partial derivative of the squared error loss function:

$$\frac{1}{2} (y - F_{m-1})^2$$

- ▶ Gradient boosting can be used for both regression and classification.

---

## XGBoost (eXtreme Gradient Boosting)

- ▶ XGBoost (eXtreme Gradient Boosting)
  - ▶ A gradient boosting algorithm
  - ▶ Uses a more regularized model formalization to control over-fitting
  - ▶ Achieve top performance for many tasks
- ▶ For more information:
  - ▶ Video: <https://www.youtube.com/watch?v=OtD8wVaFm6E>

---

## Summary

- Basic concepts in decision tree learning
- How to learn a decision tree
  - Basic decision tree learning algorithm
  - How to select best attribute
  - Pruning decision tree
  - How to deal with numeric attributes
- Learning a regression tree
- Bagging and Boosting
  - Random Forest, AdaBoost, Gradient boosting

# Data Mining (EECS 4412)

---

## Sequential Pattern Mining

Aijun An

Department of Electrical Engineering and Computer Science  
York University

---

# Outline

- ▶ Basic concepts of sequential pattern mining
- ▶ A Simplified Version of GSP Algorithm
- ▶ PrefixSpan

# An Example Sequence Database

| Customer Id | Transaction Time | Items Bought |
|-------------|------------------|--------------|
| 1           | June 25 '93      | 30           |
| 1           | June 30 '93      | 90           |
| 2           | June 10 '93      | 10, 20       |
| 2           | June 15 '93      | 30           |
| 2           | June 20 '93      | 40, 60, 70   |
| 3           | June 25 '93      | 30, 50, 70   |
| 4           | June 25 '93      | 30           |
| 4           | June 30 '93      | 40, 70       |
| 4           | July 25 '93      | 90           |
| 5           | June 12 '93      | 90           |

Figure 1: Database Sorted by Customer Id and Transaction Time

| Customer Id | Customer Sequence           |
|-------------|-----------------------------|
| 1           | { (30) (90) }               |
| 2           | { (10 20) (30) (40 60 70) } |
| 3           | { (30 50 70) }              |
| 4           | { (30) (40 70) (90) }       |
| 5           | { (90) }                    |

Figure 2: Customer-Sequence Version of the Database

- ▶ A *sequence database* consists of a set of sequences
- ▶ A *sequence* is an ordered list of itemsets
- ▶ *Itemset*: a set of items
- ▶ Items within an itemset are unordered.
- ▶ *Element*: an itemset in a sequence

# Example of a Sequential Pattern

- ▶ Database of an online book store
  - ▶ Contains data sequences
  - ▶ Each sequence corresponds to a list of transactions by a given customer.
  - ▶ Each transaction contains the books selected by the customer in one order.
- ▶ A sequential pattern
  - ▶ 5% of customers bought “*Foundation*”, then “*Foundation and Empire*” and “*Ringworld*”, then “*Second Foundation*”.
- ▶ Usefulness
  - ▶ Making recommendations.
  - ▶ Knowing what to stock.



# Mining Sequential Patterns

- ▶ Objective
  - ▶ Finding (interesting) frequent sequences from a sequence database.
- ▶ Applications
  - ▶ Customer shopping sequence analysis
  - ▶ Web log analysis
  - ▶ DNA or protein analysis
  - ▶ Stock market analysis
  - ▶ Medical domain
    - ▶ Predict onset of a disease from a sequence of symptoms, etc.
    - ▶ E.g., <fever → cough → fatigue → diagnosis of pneumonia>
  - ▶ Earthquake prediction
  - ▶ etc.

# Basic Concepts

- ▶ A sequence  $\langle a_1, a_2, \dots, a_n \rangle$  *is contained in* (or *is a subsequence of*) another sequence  $\langle b_1, b_2, \dots, b_m \rangle$  if there exist integers  $i_1 < i_2 < \dots < i_n$  such that  $a_1 \subseteq b_{i_1}, a_2 \subseteq b_{i_2}, \dots, a_n \subseteq b_{i_n}$ .
  - ▶ E.g.,  $\langle (3) (4\ 5) (8) \rangle$  is contained in  $\langle (7) (3\ 8) (9) (4\ 5\ 6) (8) \rangle$
- ▶ *Support of a sequence* is defined as the fraction of total sequences in the database that contains this sequence.
  - ▶ Support of  $\langle (30) (90) \rangle$ ?
  - ▶ Support of  $\langle (20) (60\ 70) \rangle$ ?
  - ▶ Support of  $\langle (30) (70) \rangle$ ?
- ▶ *Frequent sequences* (also called *sequential patterns*)
  - ▶ sequences that satisfy a minimum support (min\_sup).

| Customer Id | Customer Sequence                            |
|-------------|----------------------------------------------|
| 1           | $\langle (30) (90) \rangle$                  |
| 2           | $\langle (10\ 20) (30) (20\ 60\ 70) \rangle$ |
| 3           | $\langle (30\ 50\ 70) \rangle$               |
| 4           | $\langle (30) (40\ 60\ 70) (90) \rangle$     |
| 5           | $\langle (90) \rangle$                       |

# Sequential Pattern Mining

- ▶ What is sequential pattern mining
  - ▶ Given a sequence database, find the set of frequent sequences that satisfy a minimum support ( $\text{min\_sup}$ ).
- ▶ Algorithms
  - ▶ Initial Apriori-like algorithms (Agrawal and Srikant, 95)
    - ▶ AprioriAll, AprioriSome, and DynamicSome
  - ▶ GSP - an Apriori-like, influential mining method (Srikant and Agrawal, 96)
  - ▶ PrefixSpan (Pei, et al, 01)
  - ▶ SPADE (Zaki, 01)
  - ▶ Mining sequential patterns with constraints (Pei, et al, 02)
  - ▶ etc.

# Apriori Property

- ▶ The **Apriori** property in sequential patterns:  
Any nonempty subsequence of a frequent sequence must be frequent
- ▶ If a sequence is infrequent, then none of its super-sequences is frequent.
- ▶ i.e., if  $\langle (3) (4\ 5) \rangle$  is infrequent, so are  $\langle (3) (4\ 5) (8) \rangle$  and  $\langle (3\ 6) (4\ 5) \rangle$

# GSP

- ▶ GSP: Generalized Sequential Pattern Mining
- ▶ Proposed in
  - ▶ R. Srikant and R. Agrawal. Mining Sequential Patterns: Generalizations and Performance Improvements. In Proc. of EDBT'96.
  - ▶ Can be downloaded from the course web site
- ▶ GSP considers some time constraints and item taxonomy
  - ▶ More general than simply mining sequential patterns

# A Simplified Version of GSP

- ▶ GSP without the generalizations
- ▶ Problem statement:
  - ▶ find all frequent sequences from a database of sequences given a *min\_sup*
- ▶ **Length of a sequence** = number of items in the sequence
  - ▶ Length of  $\langle (a)(b) \rangle$  is 2
  - ▶ Length of  $\langle (a\ b) \rangle$  is 2
  - ▶ Length of  $\langle (a\ b)\ (c) \rangle$  is 3
- ▶ A **length- $k$  sequence** is also called *k-sequence*.

# General Description of Simplified GSP

## ▶ Method

- ▶ Take sequences in form of  $\langle x \rangle$  as length-1 candidates
- ▶ Scan database once, find  $L_1$ , the set of length-1 sequential patterns
- ▶ Let  $k=1$ ; while  $L_k$  is not empty do
  - ▶ **Form**  $C_{k+1}$ , the set of length- $(k+1)$  **candidates** from  $L_k$ ;
  - ▶ If  $C_{k+1}$  is not empty, **scan database** once, find  $L_{k+1}$ , the set of length- $(k+1)$  sequential patterns
  - ▶ Let  $k=k+1$ ;

# Finding Length-1 Sequential Patterns

- Initial candidates

- $\langle(a)\rangle, \langle(b)\rangle, \langle(c)\rangle, \langle(d)\rangle, \langle(e)\rangle, \langle(f)\rangle, \langle(g)\rangle, \langle(h)\rangle$

- Scan database once

- count support for candidates

$min\_sup = 40\%$

$min\_sup\_count = 2$

| Seq-id | Sequence                              |
|--------|---------------------------------------|
| 10     | $\langle(bd)(c)(b)(ac)\rangle$        |
| 20     | $\langle(bf)(ce)(b)(fg)\rangle$       |
| 30     | $\langle(ah)(bf)(a)(b)(f)\rangle$     |
| 40     | $\langle(be)(ce)(d)\rangle$           |
| 50     | $\langle(a)(bd)(b)(c)(b)(ade)\rangle$ |

| Cand                                      | Sup |
|-------------------------------------------|-----|
| $\langle(a)\rangle$                       | 3   |
| $\langle(b)\rangle$                       | 5   |
| $\langle(c)\rangle$                       | 4   |
| $\langle(d)\rangle$                       | 3   |
| $\langle(e)\rangle$                       | 3   |
| $\langle(f)\rangle$                       | 2   |
| <del><math>\langle(g)\rangle</math></del> | 1   |
| <del><math>\langle(h)\rangle</math></del> | 1   |



## Length-2 Candidate Generation

- ▶ How to generate  $C_2$  from  $L_1$ 
  - ▶ Merge every pair of frequent length-1 sequences.
  - ▶ Merging two frequent length-1 sequences  $\langle i_1 \rangle$  and  $\langle i_2 \rangle$  will produce three candidate 2-sequences:  
 $\langle i_1 i_2 \rangle$   $\langle i_2 i_1 \rangle$  and  $\langle i_1 i_1 \rangle$   
assuming that  $i_1$  is different from  $i_2$ .
  - ▶ Every frequent length-1 sequence  $\langle i \rangle$  can join with itself to produce  $\langle i i \rangle$ .

# Length-2 Candidates

(Cont'd from the example on Slide 13)

Length of a sequence = the number of items

51 length-2 Candidates

|     | <a>      | <b>      | <c>      | <d>      | <e>      | <f>      |
|-----|----------|----------|----------|----------|----------|----------|
| <a> | <(a)(a)> | <(a)(b)> | <(a)(c)> | <(a)(d)> | <(a)(e)> | <(a)(f)> |
| <b> | <(b)(a)> | <(b)(b)> | <(b)(c)> | <(b)(d)> | <(b)(e)> | <(b)(f)> |
| <c> | <(c)(a)> | <(c)(b)> | <(c)(c)> | <(c)(d)> | <(c)(e)> | <(c)(f)> |
| <d> | <(d)(a)> | <(d)(b)> | <(d)(c)> | <(d)(d)> | <(d)(e)> | <(d)(f)> |
| <e> | <(e)(a)> | <(e)(b)> | <(e)(c)> | <(e)(d)> | <(e)(e)> | <(e)(f)> |
| <f> | <(f)(a)> | <(f)(b)> | <(f)(c)> | <(f)(d)> | <(f)(e)> | <(f)(f)> |

|     | <a> | <b>    | <c>    | <d>    | <e>    | <f>    |
|-----|-----|--------|--------|--------|--------|--------|
| <a> |     | <(ab)> | <(ac)> | <(ad)> | <(ae)> | <(af)> |
| <b> |     |        | <(bc)> | <(bd)> | <(be)> | <(bf)> |
| <c> |     |        |        | <(cd)> | <(ce)> | <(cf)> |
| <d> |     |        |        |        | <(de)> | <(df)> |
| <e> |     |        |        |        |        | <(ef)> |
| <f> |     |        |        |        |        |        |

Without Apriori property,  
 $8*8 + 8*7/2 = 92$   
 candidates

Apriori prunes  
 44.57% candidates

# Finding Length-2 Sequential Patterns

- ▶ Scan database one more time, collect support count for each length-2 candidate
- ▶ There are 19 length-2 candidates which pass the minimum support threshold in our example DB
  - ▶ They are length-2 sequential patterns:  
<(a)(a)>, <(a)(b)>  
<(b)(a)>, <(b)(b)>, <(b)(c)>, <(b)(d)>, <(b)(e)>, <(b)(f)>  
<(c)(a)>, <(c)(b)>, <(c)(d)>  
<(d)(a)>, <(d)(b)>, <(d)(c)>  
<(f)(b)>, <(f)(f)>  
<(bd)>, <(bf)>, <(ce)>

## Generating $C_k$ from $L_{k-1}$ ( $k > 2$ )

- ▶ Join step: generate  $C_k$  by joining  $L_{k-1}$  with  $L_{k-1}$ 
  - ▶ All items in an itemset are ranked in an order.
  - ▶ A sequence  $s_1$  joins with  $s_2$  *if the subsequence obtained by dropping the first item of  $s_1$  is the same as the subsequence obtained by dropping the last item of  $s_2$ .*
  - ▶ The joined sequence is  $s_1$  plus the last item of  $s_2$ .
    - ▶ The added item becomes a separate element if it was a separate element in  $s_2$ , or part of the last element of  $s_1$  otherwise.
  - ▶ Examples:
    - ▶ Joining  $\langle(1)(2\ 3)(4)\rangle$  and  $\langle(2\ 3)(4)(5)\rangle$  produces  $\langle(1)\ (2\ 3)(4)(5)\rangle$
    - ▶ Joining  $\langle(1)(2\ 3)(4)\rangle$  and  $\langle(2\ 3)(4\ 5)\rangle$  produces  $\langle(1)(2\ 3)(4\ 5)\rangle$

## Generating $C_k$ from $L_{k-1}$ ( $k > 2$ ) (Cont'd)

- ▶ Prune step: delete candidates in  $C_k$  that have infrequent  $(k-1)$ -subsequence
  - ▶ A  $(k-1)$ -subsequence of sequence  $s$  is derived by dropping an item from  $s$
  - ▶ Check each  $(k-1)$ -subsequence against  $L_{k-1}$
  - ▶ Example:
    - ▶ If  $\langle (ab)(d) \rangle$ ,  $\langle (b)(ad) \rangle$ ,  $\langle (b)(de) \rangle$  are all length-3 frequent sequences, then
    - ▶ Join result:  $\langle (ab)(de) \rangle$
    - ▶ Its length-3 subsequences:
      - ▶  $\langle (b)(de) \rangle$ ,  $\langle (a)(de) \rangle$ ,  $\langle (ab)(e) \rangle$ ,  $\langle (ab)(d) \rangle$
    - ▶  $\langle (a)(de) \rangle$  and  $\langle (ab)(e) \rangle$  are infrequent,  $\langle (ab)(de) \rangle$  is pruned
      - ▶ As long as one of them is infrequent, can stop checking others.
      - ▶ No need to check  $\langle (ab)(d) \rangle$  and  $\langle (b)(de) \rangle$  (Why?)
  - ▶  $C_4$  becomes empty

## Generating $C_k$ from $L_{k-1}$ ( $k > 2$ ) (Cont'd)

### ▶ More Examples:

- ▶ If  $\langle(a)(b)\rangle$ ,  $\langle(a)(a)\rangle$  and  $\langle(b)(a)\rangle$  are all length-2 sequential patterns, then length-3 candidates are:
  - ▶ Join result:
    - ▶  $\langle(a)(b)(a)\rangle$ ,  $\langle(a)(a)(b)\rangle$ ,  $\langle(a)(a)(a)\rangle$ ,  $\langle(b)(a)(b)\rangle$ , and  $\langle(b)(a)(a)\rangle$ .
  - ▶ After pruning:
    - ▶  $\langle(a)(b)(a)\rangle$ ,  $\langle(a)(a)(b)\rangle$ ,  $\langle(a)(a)(a)\rangle$ ,  $\langle(b)(a)(a)\rangle$ .
- ▶ If  $\langle(bd)\rangle$ ,  $\langle(b)(b)\rangle$  and  $\langle(d)(b)\rangle$  are all length-2 sequential patterns, what are the length-3 candidates?
  - ▶ Join result:
    - $\langle(bd)b\rangle$ ,  $\langle(b)(bd)\rangle$ ,  $\langle(b)(b)(b)\rangle$ ,  $\langle(d)(bd)\rangle$ ,  $\langle(d)(b)(b)\rangle$
  - ▶ After pruning:
    - ▶  $\langle(bd)b\rangle$ ,  $\langle(b)(b)(b)\rangle$ ,  $\langle(d)(b)(b)\rangle$

# Example Continued

- ▶  $L_4$ : Length-4 sequential patterns:  $min\_sup\_count=2$

- ▶  $\langle (b)(c)(b)(a) \rangle$
- ▶  $\langle (bd)(b)(a) \rangle$
- ▶  $\langle (bd)(b)(c) \rangle$
- ▶  $\langle (bd)(c)(a) \rangle$
- ▶  $\langle (bd)(c)(b) \rangle$
- ▶  $\langle (bf)(b)(f) \rangle$
- ▶  $\langle (d)(c)(b)(a) \rangle$

| Seq-id | Sequence                                |
|--------|-----------------------------------------|
| 10     | $\langle (bd)(c)(b)(ac) \rangle$        |
| 20     | $\langle (bf)(ce)(b)(fg) \rangle$       |
| 30     | $\langle (ah)(bf)(a)(b)(f) \rangle$     |
| 40     | $\langle (be)(ce)(d) \rangle$           |
| 50     | $\langle (a)(bd)(b)(c)(b)(ade) \rangle$ |

- ▶  $C_5$ : Length-5 candidates (after join and prune):
  - ▶  $\langle (bd)(c)(b)(a) \rangle$
- ▶  $L_5$ : Length-5 sequential pattern (found by scanning DB):
  - ▶  $\langle (bd)(c)(b)(a) \rangle$ : 2

# Summary of Simplified GSP

- ▶ Make the first pass over the sequence database  $D$  to find all the 1-element (length-1) frequent sequences
- ▶ Repeat until no new candidate or frequent sequences are found
  - ▶ **Candidate Generation:**
    - ▶ **Merge** joinable pairs of frequent sequences of length  $(k-1)$  to generate candidate sequences that contain  $k$  items
    - ▶ **Prune** candidate  $k$ -sequences that contain infrequent  $(k-1)$ -subsequences
  - ▶ **Support Counting and Candidate Elimination:**
    - ▶ Make a new pass over the sequence database  $D$  to find the support count for each candidate sequence
    - ▶ Eliminate candidate  $k$ -sequences whose actual support is less than  $min\_sup$



## Better Method: PrefixSpan

- ▶ Generate all frequent sequences without candidate generation and testing.
- ▶ J. Pei et al. PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In Proc. of ICDE'01.
- ▶ Will talk about it next.

# PrefixSpan

- ▶ Generate all frequent sequences without candidate generation and testing.
- ▶ Strategy
  - ▶ Divide and conquer
    - ▶ Divide the patterns to be mined into subsets and find patterns in each subset recursively.
  - ▶ Projection-based
    - ▶ Recursively project a sequence database into a set of smaller databases based on the frequent “prefix” mined so far
    - ▶ Mine each projected database to find frequent “suffixes”

# Prefix and Suffix

## ► Suppose

- all the items in an element (i.e. an itemset) of a sequence are listed alphabetically.
- ( ) is used to enclose an itemset
- ( ) is omitted for itemsets with a single item

► *Prefixes* of sequence  $\langle a(abc)(ac)d(cf) \rangle$  are  $\langle a \rangle$ ,  $\langle aa \rangle$ ,  $\langle a(ab) \rangle$ ,  $\langle a(abc) \rangle$ ,  $\langle a(abc)a \rangle$ , ... .

► *Suffixes* of sequence  $\langle a(abc)(ac)d(cf) \rangle$  :

- $\langle (abc)(ac)d(cf) \rangle$  is the suffix wrt prefix  $\langle a \rangle$
- $\langle (_bc)(ac)d(cf) \rangle$  is the suffix wrt prefix  $\langle aa \rangle$
- $\langle (_c)(ac)d(cf) \rangle$  is the suffix wrt prefix  $\langle a(ab) \rangle$
- ...

# Mining Sequential Patterns by Prefix Projections

- ▶ Step 1: find length-1 sequential patterns
  - ▶  $\langle a \rangle$ ,  $\langle b \rangle$ ,  $\langle c \rangle$ ,  $\langle d \rangle$ ,  $\langle e \rangle$ ,  $\langle f \rangle$
- ▶ Step 2: divide search space. The complete set of freq. seq. can be partitioned into 6 subsets:
  - ▶ The ones having prefix  $\langle a \rangle$ ;
  - ▶ The ones having prefix  $\langle b \rangle$ ;
  - ▶ ...
  - ▶ The ones having prefix  $\langle f \rangle$

| SID | sequence                          |
|-----|-----------------------------------|
| 10  | $\langle a(abc)(ac)d(cf) \rangle$ |
| 20  | $\langle (ad)c(bc)(ae) \rangle$   |
| 30  | $\langle (ef)(ab)(df)cb \rangle$  |
| 40  | $\langle eg(af)cbc \rangle$       |

min\_sup count=2

## Finding Freq. Seq. with Prefix <a>

- ▶ Only need to consider sequences in D containing <a>
- ▶ In a sequence containing <a>, only the subsequence (suffixes) prefixed with the first occurrence of <a> should be considered.
- ▶ The collection of such subsequences is called *<a>-projected database*:

<(abc)(ac)d(cf)>,  
<(\_d)c(bc)(ae)>,  
<(\_b)(df)cb>,  
<(\_f)cbc>.

D

| SID | sequence          |
|-----|-------------------|
| 10  | <a(abc)(ac)d(cf)> |
| 20  | <(ad)c(bc)(ae)>   |
| 30  | <(ef)(ab)(df)cb>  |
| 40  | <eg(af)cbc>       |

min\_sup count=2

Next: recursively mine *<a>-projected database* to find frequent sequences in the projected database.

## Finding Freq. Seq. with Prefix <a> (Cont'd)

- ▶ Find local frequent length-1 sequences by scanning *<a>-projected database*
  - ▶ *<a>-projected database*:  $\langle(abc)(ac)d(cf)\rangle$ ,  $\langle(_d)c(bc)(ae)\rangle$ ,  $\langle(_b)(df)cb\rangle$ ,  $\langle(_f)c bc\rangle$  min\_sup count=2
  - ▶ *Local frequent length-1 sequences*:  
 $\langle a \rangle:2$ ,  $\langle b \rangle:4$ ,  $\langle _b \rangle:2$ ,  $\langle c \rangle:4$ ,  $\langle d \rangle:2$ ,  $\langle f \rangle:2$
- ▶ Generate all the length-2 freq. seq. having prefix <a>:  
 $\langle aa \rangle:2$ ,  $\langle ab \rangle:4$ ,  $\langle (ab) \rangle:2$ ,  $\langle ac \rangle:4$ ,  $\langle ad \rangle:2$ ,  $\langle af \rangle:2$
- ▶ Further partition frequent sequences with prefix <a> into 6 subsets
  - ▶ Having prefix <aa>;
  - ▶ Having prefix <ab>;
  - ▶ ...
  - ▶ Having prefix <af>

## Finding Freq. Seq. with Prefix <aa>

- ▶ From <a>-projected database:  
<(abc)(ac)d(cf)>, <(\_d)c(bc)(ae)>, <(\_b)(df)cb>, <(\_f)cbc>
- ▶ Generate <aa>-projected database:  
<(\_bc)(ac)d(cf)>, <(\_e)>
- ▶ No local frequent items, stop growing prefix <aa>.

min\_sup count=2

| SID | sequence          |
|-----|-------------------|
| 10  | <a(abc)(ac)d(cf)> |
| 20  | <(ad)c(bc)(ae)>   |
| 30  | <(ef)(ab)(df)cb>  |
| 40  | <eg(af)cbc>       |

# Finding Freq. Seq. with Prefix <ab>

- ▶ From <a>-projected database:  
<(abc)(ac)d(cf)>, <(\_d)c(bc)(ae)>, <(\_b)(df)cb>, <(\_f)cbc>
- ▶ Generate <ab>-projected database:  
<(\_c)(ac)d(cf)>, <(\_c)(ae)>, <c>
- ▶ Local frequent length-1 sequences
  - ▶ <a>:2, <c>:2, <(\_c)>:2
- ▶ Generate length-3 freq. seq. with prefix <ab>:
  - ▶ <aba>:2, <abc>:2, <a(bc)>:2
- ▶ Further partition the set of freq. Seq. prefixed with <ab>:
  - ▶ Sequences with prefix <aba>
  - ▶ Sequences with prefix <abc>
  - ▶ Sequences with prefix <a(bc)>



# Finding Freq. Seq. with Prefix <aba>

- ▶ From <ab>-projected database:  
    <(\_c) (ac)d(cf)>, <(\_c)(ae)>, <c>
- ▶ Generate <aba>-projected database:
  - ▶ <(\_c)d(cf)>, <(\_e)>
- ▶ No local frequent length-1 sequence, stop growing <aba>.

# Finding Freq. Seq. with Prefix <abc>

- ▶ From <ab>-projected database:  
    <(\_c) (ac)d(cf)>, <(\_c)(ae)>, <c>
- ▶ Generate <abc>-projected database:
  - ▶ <d(cf)>
- ▶ No local frequent length-1 sequence, stop growing <abc>.

# Finding Freq. Seq. with Prefix $\langle a(bc) \rangle$

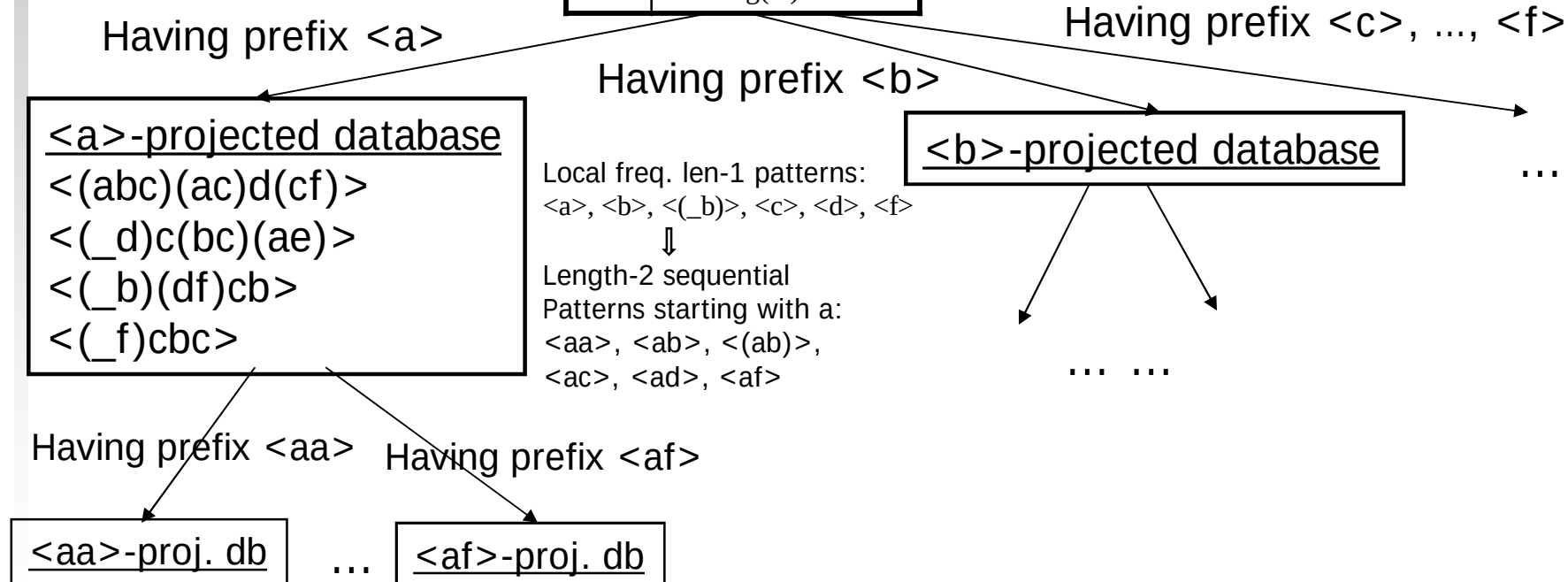
- ▶ From  $\langle ab \rangle$ -projected database:
  - $\langle (\_c) (ac)d(cf) \rangle$ ,  $\langle (\_c)(ae) \rangle$ ,  $\langle c \rangle$
- ▶ Generate  $\langle a(bc) \rangle$ -projected database:
  - ▶  $\langle (ac)d(cf) \rangle$ ,  $\langle (ae) \rangle$
- ▶ Local frequent length-1 sequences:
  - ▶  $\langle a \rangle:2$
- ▶ Generate length-4 freq. seq. with prefix  $\langle a(bc) \rangle$ :
  - ▶  $\langle a(bc)a \rangle:2$
- ▶ Further mining  $\langle a(bc)a \rangle$ -projected database returns no frequent sequence prefixed with  $\langle a(bc)a \rangle$ .
- ▶ This finishes generating all the patterns prefixed with  $\langle ab \rangle$ .

# Completeness of PrefixSpan

SDB

| SID | sequence          |
|-----|-------------------|
| 10  | <a(abc)(ac)d(cf)> |
| 20  | <(ad)c(bc)(ae)>   |
| 30  | <(ef)(ab)(df)cb>  |
| 40  | <eg(af)cbc>       |

Length-1 sequential patterns  
<a>, <b>, <c>, <d>, <e>, <f>



---

# Efficiency of PrefixSpan

- ▶ No candidate sequence needs to be generated
- ▶ Projected databases keep shrinking
- ▶ Major cost of PrefixSpan:
  - ▶ constructing projected databases.
  - ▶ overlapping suffixes in recursively projected databases
    - ▶ can be improved by *Pseudo-projections*

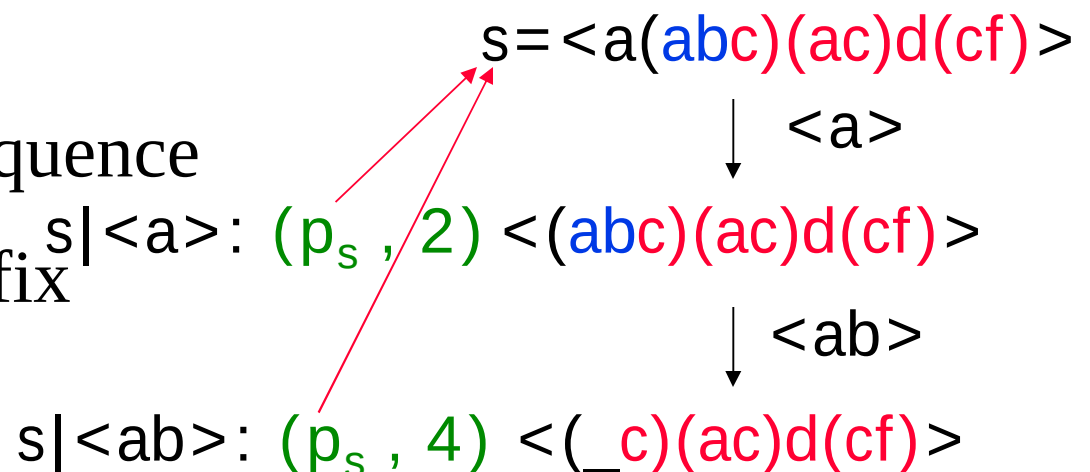
# Speed-up by Pseudo-projection

- ▶ Major cost of PrefixSpan: projection
  - ▶ Suffixes of a sequence often appear repeatedly in recursively projected databases
    - ▶  $\langle(abc)(ac)d(cf)\rangle$  appears in  $\langle a \rangle$ -projected database
    - ▶  $\langle(_c)(ac)d(cf)\rangle$  appears in  $\langle ab \rangle$ -projected database

- ▶ When (projected) database can be held in main memory, **use pointers (p, o) to form projections**

- ▶ **p**: Pointer to the sequence

- ▶ **o**: Offset of the suffix



# Pseudo-Projection vs. Physical Projection

- ▶ Pseudo-projection avoids physically copying suffixes
  - ▶ Efficient in running time and space when database can be held in main memory
- ▶ However, it is not efficient when database cannot fit in main memory
  - ▶ Disk-based random accessing is very costly
- ▶ Suggested Approach:
  - ▶ Integration of physical and pseudo-projection
  - ▶ Swapping to pseudo-projection when the projected database fits in memory

# Experiments and Performance Analysis

- ▶ Comparing PrefixSpan with GSP, FreeSpan and SPADE in large databases
  - ▶ GSP (IBM Almaden, Srikant & Agrawal EDBT'96)
  - ▶ FreeSpan (J. Han J. Pei, B. Mortazavi-Asi, Q. Chen, U. Dayal, M.C. Hsu, KDD'00)
  - ▶ SPADE (Zaki, 01)
- ▶ PrefixSpan is fastest and scalable.



# Problem of Sequential Pattern Mining

- ▶ GSP and PrefixSpan finds all the sequences that satisfy the support threshold.
- ▶ A long frequent sequence contains a combinatorial number of frequent subsequences:
  - ▶ For a length-100 sequential pattern, there exist  $2^{100} - 1$  nonempty subsequences.
- ▶ Problem: too many patterns are generated.

# Solutions

- ▶ Mining maximal or closed sequential patterns
  - ▶ Maximal sequential pattern:
    - ▶ A frequent sequence  $s$  is *maximal* if there exists no frequent super-sequence of  $s$ .
  - ▶ Closed sequential pattern:
    - ▶ A sequence  $s$  is *closed* if there exists no super-sequence of  $s$  with the same support as  $s$ .
    - ▶ *CloSpan* (Yan, Han and Afshar, 2003): an efficient closed sequential pattern mining method.

# Solutions (Cont'd)

- ▶ Constraint-based mining of sequential patterns
  - ▶ Constraints
    - ▶ Item constraint
      - ▶ Find patterns containing a, b, c, but no d.
    - ▶ Length constraint
      - ▶ Find patterns having at least (or at most) 10 items
    - ▶ Super-pattern constraint
      - ▶ Find patterns that contain <(PC)(Digital-camera)>
    - ▶ Aggregate constraint
      - ▶ Find patterns the average price of whose items is over \$100.

# Solutions (Cont'd)

- ▶ More constraints
  - ▶ Regular expression constraint
    - ▶  $\langle a^*\{bb|(bc)d|dd\} \rangle$
  - ▶ Duration constraint
    - ▶ Find patterns of events about +/- 24 hours of a shooting
  - ▶ Gap constraint
    - ▶ Find purchasing patterns such that the gap between each consecutive purchases is less than 1 month.

# Solutions (Cont'd)

## ▶ Properties of constraints

### ▶ Anti-monotonic constraint

- ▶ If a sequence  $s$  satisfies constraint  $C$ , so does every non-empty subsequence of  $s$
- ▶ Examples:  $\text{support}(s) \geq 5\%$ ,  $\text{length}(s) < 10$

### ▶ Monotonic constraint

- ▶ If a sequence  $s$  satisfies constraint  $C$ , so does every super sequence of  $s$ .
- ▶ Examples:  $\text{length}(s) \geq 10$ , super pattern constraints.

## ▶ A systematic study on constraint-based sequential pattern mining:

- ▶ J. Pei, J. Han, and W. Wang. "Mining Sequential Patterns with Constraints in Large Databases". *In Proceedings of CIKM'02.*

# Other Research on Frequent Pattern Mining

- ▶ Mining other kinds of frequent patterns
  - ▶ Frequent tree/graph mining
    - ▶ Find common structural components
  - ▶ Examples of applications
    - ▶ XML documents can be modeled as trees
    - ▶ Molecule or biochemical structures can be modeled as graphs
    - ▶ Web connection structures can be modeled as graphs
- ▶ Finding frequent patterns from data streams
  - ▶ Only one scan of DB is allowed.

## Other Research on Frequent Pattern Mining (Cont'd)

- ▶ Finding high-utility patterns (either itemsets or sequences)
  - ▶ Consider the quantity of the item inside a transaction.
  - ▶ Consider the importance (e.g., price) of an item
  - ▶ Find patterns whose revenue is at least, e.g., \$1000

# Frequent Pattern Mining Resources

- ▶ Web site: <http://fimi.cs.helsinki.fi/> contains some frequent itemset mining implementations, datasets and papers.
- ▶ SPMF: open-source data mining library containing frequent sequence and itemset mining:  
<http://www.philippe-fournier-viger.com/spmf/>



# Performance Evaluation of Machine Learning Algorithms

---

Aijun An

Department of Electrical Engineering and Computer Science  
York University

---

# Outline

- ▶ Introduction
- ▶ Predictive performance Measures
  - ▶ Classification
  - ▶ Regression
- ▶ Performance Evaluation Methods
  - ▶ Holdout
  - ▶ Repeated holdout
  - ▶ Cross-validation
  - ▶ Bootstrap
- ▶ Methods for model comparisons
  - ▶ Significance test

# Performance Evaluation

- ▶ Performance of a machine learning algorithm can be evaluated in the following aspects
  - ▶ Predictive performance
    - ▶ How accurate is the learned model in prediction?
  - ▶ Interpretability
    - ▶ Complexity of the learned model
  - ▶ Efficiency
    - ▶ Time complexity (speed)
      - ▶ Time to build the model
      - ▶ Time to classify examples using the model
      - ▶ Scalability
        - ▶ How run time changes with the increase of size of data.
    - ▶ Memory complexity
- ▶ Here we focus on the **predictive performance**

# Predictive Performance Evaluation

- ▶ Objective:
  - ▶ Find out how good is a learned *model* (i.e., classifier or regressor) in making predictions on *a test set of examples*?
  - ▶ The test set should be different from the training data from which the model is built from
- ▶ Performance measures depend on the task
  - ▶ Classification
  - ▶ Regression

# Performance Measures for Classification

- ▶ Performance measures
  - ▶ Classification accuracy
  - ▶ Classification error rate
  - ▶ Classification cost
  - ▶ Precision
  - ▶ Recall
  - ▶ F-measure
  - ▶ Area Under ROC Curve (AUC)
  - ▶ etc.

# Accuracy and Error Rate

- ▶ **Accuracy** of a classifier on a data set:

$$\frac{\text{number of examples classified correctly}}{\text{total number of examples in the data set}}$$

- ▶ **Error rate** of a classifier ( = 1 – accuracy)

$$\frac{\text{number of examples classified incorrectly}}{\text{total number of examples in the data set}}$$

- ▶ Accuracy or error on the training data is not a good indicator of the classifier's performance on future data. Should use accuracy or error on a test data set.

# Confusion Matrix

- ▶ Confusion matrix
  - ▶ Records the numbers of correct and incorrect classifications of a classifier on a test data set.
  - ▶ Is often used to calculate all the metrics
- ▶ For problems with 2 classes:

|              |             | Predicted class                       |                                      |                        |
|--------------|-------------|---------------------------------------|--------------------------------------|------------------------|
|              |             | Class = yes                           | Class = no                           | Total                  |
| Actual class | Class = yes | # of yes examples classified into yes | # of yes examples classified into no | # of true yes examples |
|              | Class = no  | # of no examples classified into yes  | # of no examples classified into no  | # of true no examples  |
|              | Total       | # of predicted yes examples           | # of predicted no examples           | Total # of examples    |

# Confusion Matrix

- Expressed using TP, FP, FN and TN:

- ▶ TP: true positive
- ▶ FP: false positive
- ▶ FN: false negative
- ▶ TN: true negative

**Predicted class**

| <b>Actual class</b> |             | Class = yes | Class = no | Total       |
|---------------------|-------------|-------------|------------|-------------|
|                     | Class = yes | TP          | FN         | TP+FN       |
|                     | Class = no  | FP          | TN         | FP+TN       |
|                     | Total       | TP+FP       | FN+TN      | TP+TN+FP+FN |

- ▶ 
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ▶ 
$$\text{Error rate} = \frac{FP + FN}{TP + TN + FP + FN}$$



# Example

- ▶ Consider the following **confusion matrix**:

|              |              | Predicted class |             |       |
|--------------|--------------|-----------------|-------------|-------|
|              |              | cancer = yes    | cancer = no | Total |
| Actual class | cancer = yes | 90              | 210         | 300   |
|              | cancer = no  | 140             | 9560        | 9700  |
|              | Total        | 230             | 9770        | 10000 |

- ▶ What is the accuracy?
  - ▶  $\text{Accuracy} = (90+9560)/10000 = 96.50\%$
- ▶ What is the error rate?
  - ▶  $\text{Error rate} = (140+210)/10000 = 3.5\%$

# Limitation of Accuracy or Error Rate

- ▶ Consider a 2-class data set:
  - ▶ Number of Class 0 examples = 9,990
  - ▶ Number of Class 1 examples = 10 (e.g., fraud cases in fraud detection)
- ▶ If a model predicts everything to be class 0, what is the accuracy?
  - ▶  $9990/10000 = 99.9 \%$
- ▶ Accuracy is misleading because model does not detect any class 1 example (which is often important)
  - ▶ Do not consider the cost of different types of misclassification

# Misclassification Cost

- ▶ Misclassification cost

- ▶ In practice, different types of misclassifications often incur different costs.

- ▶ E.g., in making loan decisions, the cost of lending to a defaulter is far greater than the lost-business cost of refusing a loan to a non-defaulter.

- ▶ Cost matrix:

|              |         | Predicted class                                     |                                                     |
|--------------|---------|-----------------------------------------------------|-----------------------------------------------------|
|              |         | Class 1                                             | Class 2                                             |
| Actual class | Class 1 | 0                                                   | Cost of misclassifying a Class 1 example to Class 2 |
|              | Class 2 | Cost of misclassifying a Class 2 example to Class 1 | 0                                                   |

## Misclassification Cost (*Cont'd*)

- ▶ Calculating misclassification cost of a model on a test set

- ▶ Confusion matrix: **Predicted class**

| <b>Actual<br/>class</b> | <b>Predicted class</b>                           |                                                  |
|-------------------------|--------------------------------------------------|--------------------------------------------------|
|                         | Class 1                                          | Class 2                                          |
|                         | Class 1                                          | Class 2                                          |
| Class 1                 | # of Class 1 examples<br>classified into Class 1 | # of Class 1 examples<br>classified into Class 2 |
| Class 2                 | # of Class 2 examples<br>classified into Class 1 | # of Class 2 examples<br>classified into Class 2 |

- ▶ Misclassification cost:  $\sum_i \text{cost}_i \times \text{num}_i$

where  $\text{cost}_i$  is the cost in the  $i$ th cell of the cost matrix and  $\text{num}_i$  is the value in the  $i$ th cell of the confusion matrix.

# Classification Cost (*Cont'd*)

## ► Example

### ► Cost matrix:

|              |         | Predicted class |         |
|--------------|---------|-----------------|---------|
|              |         | Class 1         | Class 2 |
| Actual class | Class 1 | 0               | 6       |
|              | Class 2 | 1               | 0       |

### ► Confusion matrix of models on a test data set:

from model 1:

| Actual class | Predicted class |         |
|--------------|-----------------|---------|
|              | Class 1         | Class 2 |
| Class 1      | 65              | 10      |
| Class 2      | 20              | 40      |

from model 2:

| Actual class | Predicted class |         |
|--------------|-----------------|---------|
|              | Class 1         | Class 2 |
| Class 1      | 70              | 5       |
| Class 2      | 30              | 30      |

### ► Misclassification cost:

80

What about accuracy?

60

---

# Performance Measures for Classification

- ▶ Performance measures
  - ▶ Classification accuracy
  - ▶ Classification error rate
  - ▶ Classification cost
  - ▶ Precision
  - ▶ Recall
  - ▶ F-measures
  - ▶ Area Under ROC Curve (AUC)
  - ▶ etc.

# Precision, Recall and F-measure

- ▶ Usually for measuring the performance on predicting examples of one class (the interesting, usually small class)
- ▶ **Precision:** (Consider that examples of the class in question are positive)
  - ▶ Exactness: what % of examples that the classifier labeled as positive are actually positive

$$precision = \frac{TP}{TP + FP}$$

- ▶ **Recall**

- ▶ Completeness - what % of actual positive examples are classified as positive

$$recall = \frac{TP}{TP + FN}$$

|              |             | Predicted class |            |             |
|--------------|-------------|-----------------|------------|-------------|
| Actual class |             | Class = yes     | Class = no | Total       |
|              | Class = yes | TP              | FN         | TP+FN       |
|              | Class = no  | FP              | TN         | FP+TN       |
|              | Total       | TP+FP           | FN+TN      | TP+TN+FP+FN |

# Precision, Recall and F-measure

- ▶ F measure (F1 or F-score): harmonic mean of precision and recall

$$F\_score = \frac{2 \times \textit{precision} \times \textit{recall}}{\textit{precision} + \textit{recall}}$$



# Precision, Recall and F-measure

- ▶ For measuring performance on all the classes in the data set, take an average among all classes
  - ▶ Macro-average
    - ▶ Take the average of precisions or recalls on all classes
    - ▶ Treat each class equally
    - ▶ Used when classes are imbalanced
  - ▶ Micro-average
    - ▶ Sum up the individual true positives, false positives, and false negatives of the system for different sets and
    - ▶ Use them to compute overall precision and recall.
    - ▶ Cares the model's overall performance instead of per-class performance
    - ▶ Used when classes are balanced
- ▶ F-score is computed the same way

# Performance Measures for Classification

- ▶ Performance measures
  - ▶ Classification accuracy
  - ▶ Classification error rate
  - ▶ Classification cost
  - ▶ Precision
  - ▶ Recall
  - ▶ F-measures
  - ▶ Area Under ROC Curve (AUC)
  - ▶ etc.

# ROC Curve and AUC

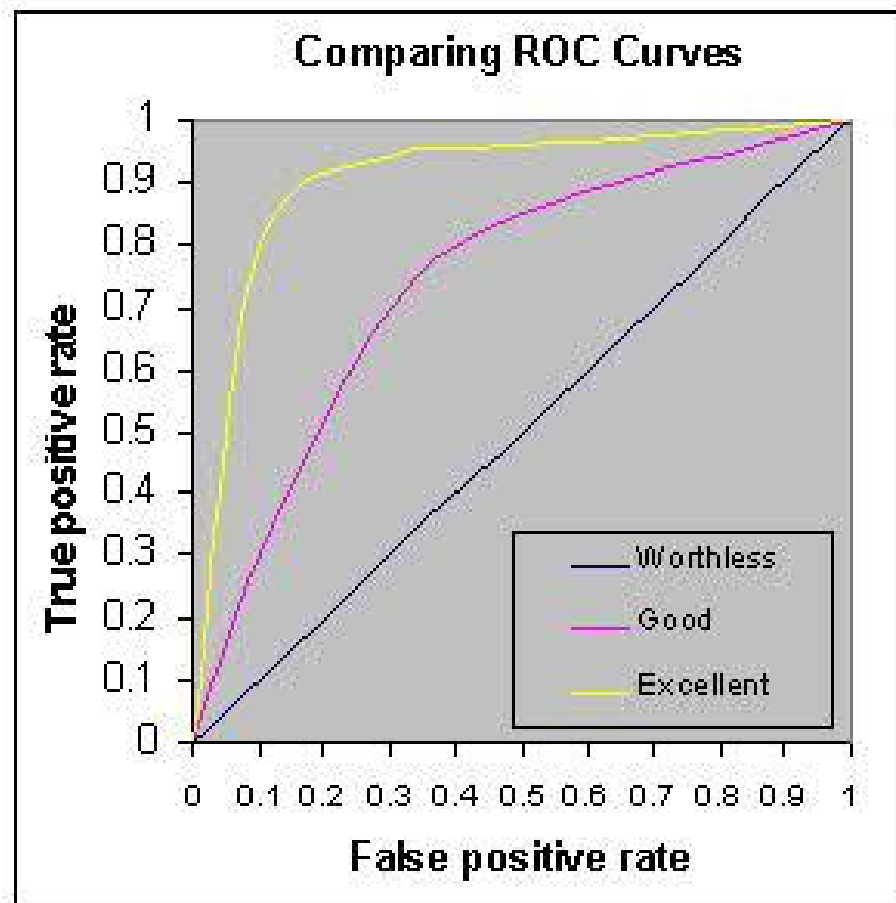
- ▶ ROC = Receiver Operating Characteristic
  - ▶ Started in electronic signal detection theory (1940s - 1950s)
  - ▶ Has become very popular in machine learning applications to assess classifiers
- ▶ ROC curve: a plot of true positive rate (TPR) against false positive rate (FPR)

|              |             | Predicted class |            |             |
|--------------|-------------|-----------------|------------|-------------|
|              |             | Class = yes     | Class = no | Total       |
| Actual class | Class = yes | TP              | FN         | TP+FN       |
|              | Class = no  | FP              | TN         | FP+TN       |
|              | Total       | TP+FP           | FN+TN      | TP+TN+FP+FN |

$$TPR = \frac{TP}{TP+FN} = Recall \qquad FPR = \frac{FP}{FP + TN}$$

# ROC Curve

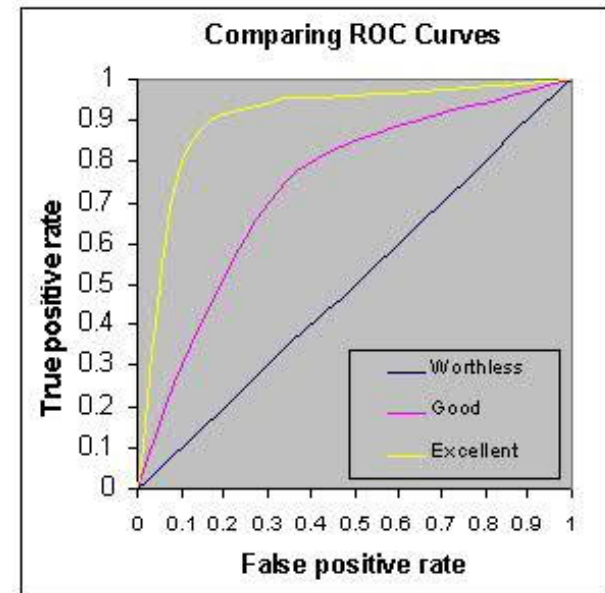
- Plot of TPR against FPR



# How to Obtain ROC Curve for a Classifier

- ▶ Given a test data set, there is usually one point (FPR, TPR) in the ROC space - How to create a curve?
- ▶ ROC is often used for a binary classifier that outputs the probability that an example belongs to the positive class.

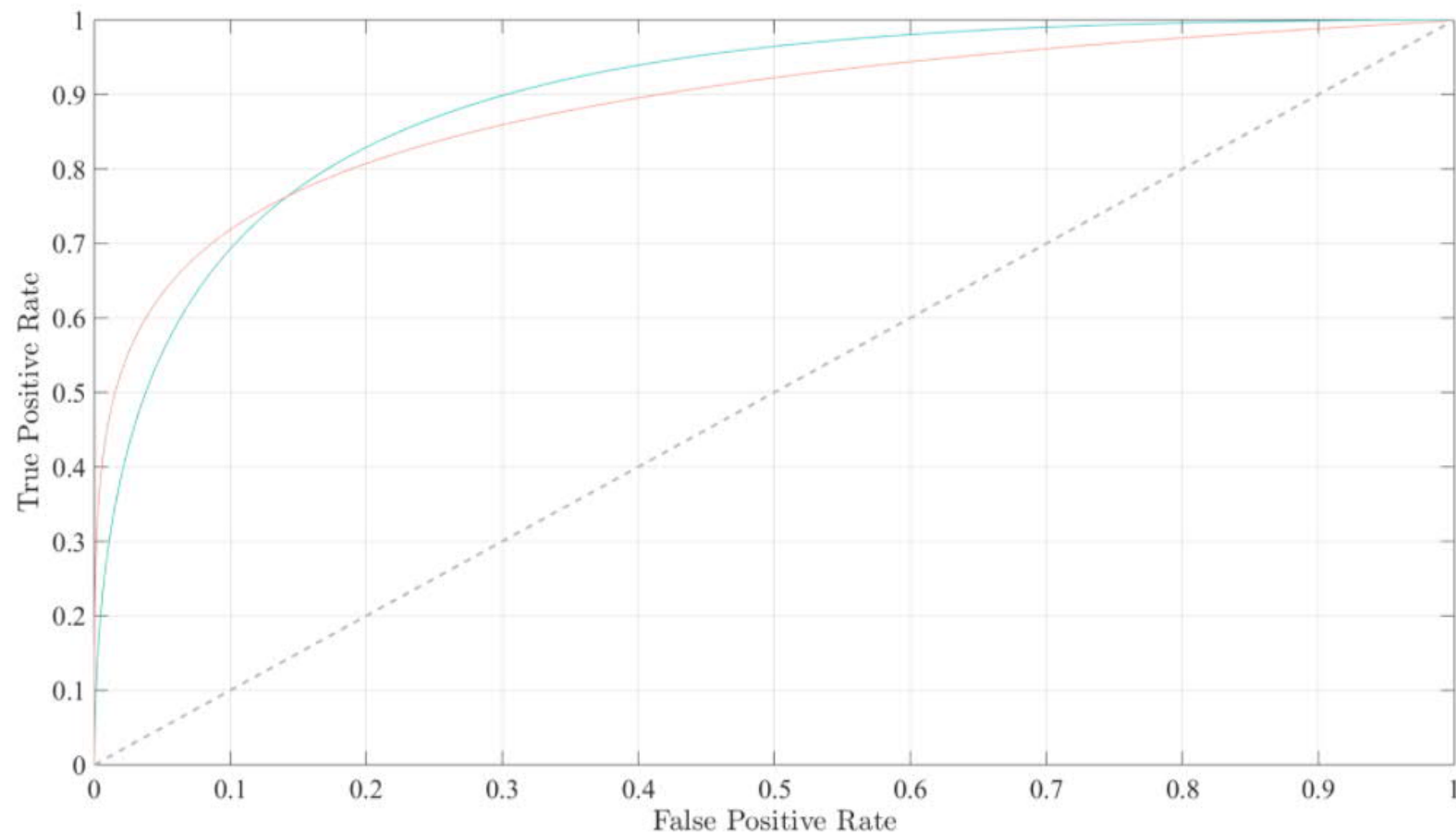
| Test cases | Probability belong to positive class |
|------------|--------------------------------------|
| e1         | 0.92                                 |
| e2         | 0.90                                 |
| e3         | 0.85                                 |
| e4         | 0.85                                 |
| e5         | 0.78                                 |
| ...        | ...                                  |



- ▶ Different probability thresholds lead to different classifiers
  - ▶ Each classifier has a point in the ROC space.

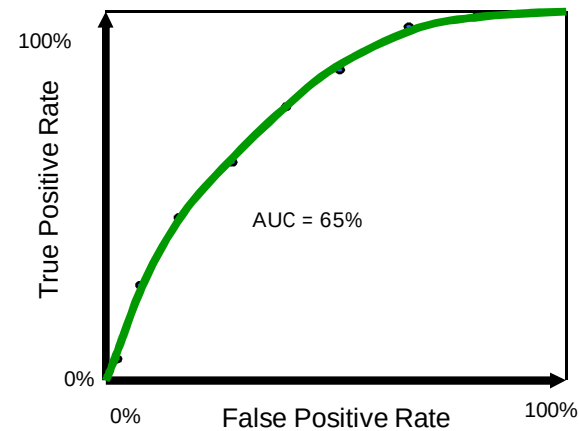
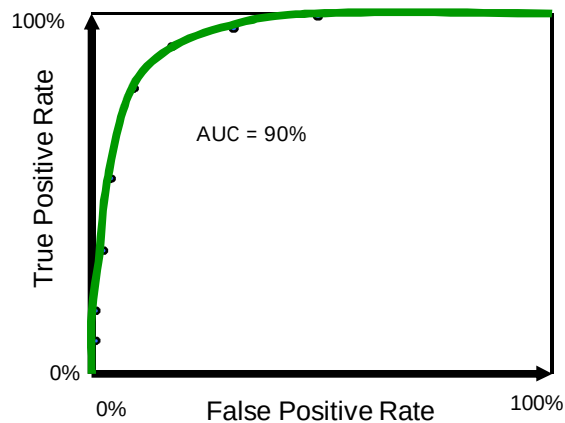
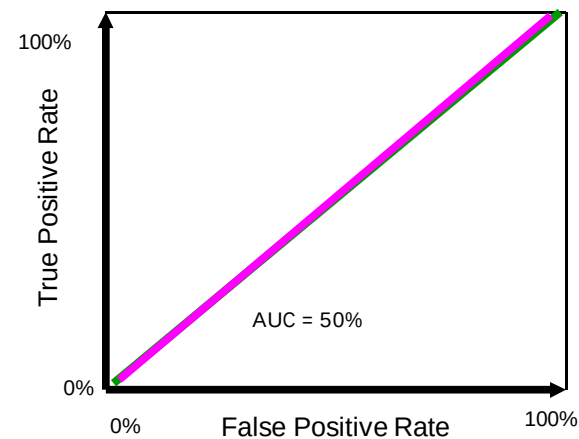
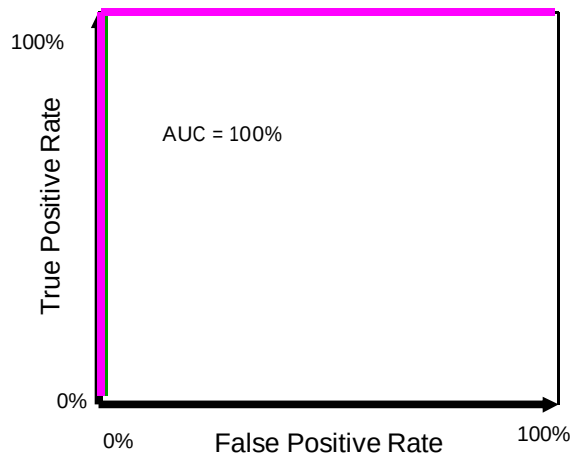
# Comparing ROC Curves

- ▶ Which of the following two ROC curves is better?



# AUC

- ▶ AUC = Area Under ROC Curve
- ▶ Overall measure of test performance



---

# Outline

- ▶ Introduction
- ▶ Predictive performance Measures
  - ▶ Classification
  - ▶ *Regression*
- ▶ Performance Evaluation Methods
  - ▶ Holdout
  - ▶ Repeated holdout
  - ▶ Cross-validation
  - ▶ Bootstrap
- ▶ Methods for model comparisons
  - ▶ Significance test



# Performance Metrics for Regression

- ▶ **Mean Absolute Error (MAE)** computes the average deviation between predicted value  $y_i$  and the actual value  $\hat{y}_i$  on a set of  $N$  examples:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- ▶ **Mean Squared Error (MSE)** places more emphasis on larger deviations:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- ▶ **Root Mean Squared Error (RMSE)** has similar scale as MAE and places more emphasis on larger deviations:

$$RMSE = \frac{1}{N} \sqrt{\sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

## Performance Metrics for Regression (Cont'd)

- ▶ **Relative Absolute Error (RAE)** divides MAE by the MAE of a simple model that predicts the mean value:

$$RAE = \frac{\sum_{i=1}^N |y_i - \hat{y}_i|}{\sum_{i=1}^N |y_i - \bar{y}|}$$

- ▶ **Relative Squared Error (RSE)** divides MSE by MSE of a simple model that predicts the mean value:

$$RSE = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

# Outline

- ▶ Introduction
- ▶ Predictive performance Measures
  - ▶ Classification
  - ▶ Regression
- ▶ *Performance Evaluation Methods*
  - ▶ Holdout
  - ▶ Repeated holdout
  - ▶ Cross-validation
  - ▶ Bootstrap
- ▶ Methods for model comparisons
  - ▶ Significance Test

# Performance Evaluation Methods

## ▶ Problem:

- ▶ Given a set  $S$  of data and a machine learning algorithm  $A$ , how do we evaluate the predictive performance of  $A$  on  $S$ ?
- ▶ We cannot learn a model from  $S$  and evaluate the model on  $S$  again because
  - ▶ Error on the training data is not a good indicator of a model's performance on future data.

## ▶ Evaluation methods

- ▶ Hold-out estimation
- ▶ Repeated hold-out estimation
- ▶ Cross validation
- ▶ Bootstrap

# Performance Evaluation Methods

- ▶ Hold-out estimation
  - ▶ Randomly split the data set into a training set and a test set, e.g., training set ( $2/3$ ), test set ( $1/3$ )
  - ▶ Build a model from the training set and estimate the error (or another measure, e.g., cost) on the test set
- ▶ Repeated hold-out estimation
  - ▶ Holdout estimate can be made more reliable by repeating the process with different random splits
  - ▶ For each split, an error rate is collected.
  - ▶ An overall error rate is obtained by averaging the error rates on the different splits

## Performance Evaluation Methods (*Cont'd*)

- ▶ Cross-validation
  - ▶ Randomly divide the data set into  $k$  subsets of equal size
  - ▶ use  $k-1$  subsets as training data and one subset as test data – do this  $k$  times using each subset in turn for testing
  - ▶ The error rates are averaged to yield an overall error estimate
  - ▶ This is called  $k$ -fold cross-validation

## Performance Evaluation Methods (*Cont'd*)

- ▶ Stratified cross-validation
  - ▶ Ensures that classes in a subset have approximately the same distribution as in the original data set.
  - ▶ Stratification reduces the estimate's variance.
- ▶ Standard method for evaluation: stratified 10-fold cross-validation.
  - ▶ Why 10? Extensive experiments have shown that this is the best choice to get an accurate estimate.
  - ▶ There is also some theoretical evidence for this.

# Performance Evaluation Methods (*Cont'd*)

## ▶ Leave-one-out

- ▶ A special case of  $k$ -fold cross-validation method when  $k$  equals to the number  $N$  of examples in the data set
- ▶ In each iteration, the test data set contains only one example and the training data set contains the rest  $N-1$  examples.
- ▶ Advantages:
  - ▶ The greatest possible amount of data is used for training in each iteration.
  - ▶ Deterministic: no random sampling is involved
- ▶ Disadvantages:
  - ▶ Long running time
  - ▶ Non-stratified partitions of the data set



# Evaluating Classifier Accuracy: Bootstrap

- ▶ Bootstrap

- ▶ Works well with small data sets
- ▶ Samples the given training examples randomly *with replacement*
  - ▶ i.e., each time an example is selected, it is equally likely to be selected again and re-added to the training set – resulting in a *bootstrap* sample

- ▶ Several bootstrap methods, and a common one is **.632 bootstrap**

- ▶ A data set with  $N$  examples is sampled  $N$  times, with replacement, resulting in a training set of  $N$  examples. The examples that did not make it into the training set end up forming the test set.
  - ▶ About 63.2% of the original data end up in the bootstrap, and the remaining 36.8% form the test set (since  $(1 - 1/N)^N \approx e^{-1} = 0.368$ )
- ▶ Repeat the sampling procedure  $k$  times, each leading to a model  $M_i$ . The overall accuracy of the model  $M$ :

$$Acc(M) = \frac{1}{k} \sum_{i=1}^k (0.632 \times Acc(M_i)_{test\_set} + 0.368 \times Acc(M_i)_{train\_set})$$

---

# Outline

- ▶ Introduction
- ▶ Predictive performance Measures
- ▶ Performance Evaluation Methods
  - ▶ Holdout
  - ▶ Repeated holdout
  - ▶ Cross-validation
  - ▶ Bootstrap
- ▶ Methods for model comparisons
  - ▶ *Significance Test*

# Is the Difference between Two Models Significant?

- ▶ Are the cross-validation results of  $Model_1$  and  $Model_2$  on a data set  $S$  significantly different?
- ▶ Are the performances of  $Model_1$  and  $Model_2$  on a number of data sets significantly different?
- ▶ Paired t-test or other statistical tests can be used.

# Paired t-test

- ▶ The *paired t-test* is a statistical hypothesis test that
  - ▶ tests the difference between the means for a pair of random samples. The observations are paired or matched in some way.
  - ▶ **Null hypothesis:** the two means are not significantly different

▶ Example:

| Tree    | Number of rusted leaves:<br>year 1 | Number of rusted leaves:<br>year 2 (after treatment) |
|---------|------------------------------------|------------------------------------------------------|
| 1       | 38                                 | 32                                                   |
| 2       | 10                                 | 16                                                   |
| 3       | 84                                 | 57                                                   |
| 4       | 36                                 | 28                                                   |
| 5       | 50                                 | 55                                                   |
| 6       | 35                                 | 12                                                   |
| 7       | 73                                 | 61                                                   |
| 8       | 48                                 | 29                                                   |
| Average | 46.8                               | 36.2                                                 |

## Paired t-test (*Cont'd*)

- ▶ If you run a paired t-test on the example in the last slide, using a t-test program, say, at
  - ▶ <https://www.graphpad.com/quickcalcs/ttest2/>
- ▶ Result:
  - ▶  $t = 2.43$ , degrees of freedom = 7
  - ▶ **p-value: 0.045**
    - ▶ The probability that the null hypothesis is true is 0.045
- ▶ If the p-value  $\leq 0.05$ , the null hypothesis can be rejected
  - ▶ Meaning the means of two samples are significantly different

# How Paired t-test is Done Internally

► Calculate t-Statistic:  $t = \frac{\bar{d}}{\frac{\sigma}{\sqrt{n}}}$

- $\bar{d}$  is the mean of the differences between the paired observations
- $\sigma$  is the standard deviation of the differences
- $n$  is the number of pairs

► Calculate the degree of freedom:

$$df = n-1$$

► Find the p-value:

- Find the critical t-value from the t-distribution table:
  - The value on the  $df$  row that is largest but smaller than the t-Statistic
- Get the corresponding p-value

► If the p-value  $\leq 0.05$ , the null hypothesis can be rejected

- Meaning the means of two samples are significantly different

|           | P     |       |        |        |        |         |         |
|-----------|-------|-------|--------|--------|--------|---------|---------|
| one-tail  | 0.1   | 0.05  | 0.025  | 0.01   | 0.005  | 0.001   | 0.0005  |
| two-tails | 0.2   | 0.1   | 0.05   | 0.02   | 0.01   | 0.002   | 0.001   |
| DF        |       |       |        |        |        |         |         |
| 1         | 3.078 | 6.314 | 12.706 | 31.821 | 63.656 | 318.289 | 636.578 |
| 2         | 1.886 | 2.92  | 4.303  | 6.965  | 9.925  | 22.328  | 31.6    |
| 3         | 1.638 | 2.353 | 3.182  | 4.541  | 5.841  | 10.214  | 12.924  |
| 4         | 1.533 | 2.132 | 2.776  | 3.747  | 4.604  | 7.173   | 8.61    |
| 5         | 1.476 | 2.015 | 2.571  | 3.365  | 4.032  | 5.894   | 6.869   |
| 6         | 1.44  | 1.943 | 2.447  | 3.143  | 3.707  | 5.208   | 5.959   |
| 7         | 1.415 | 1.895 | 2.365  | 2.998  | 3.499  | 4.785   | 5.408   |
| 8         | 1.397 | 1.86  | 2.306  | 2.896  | 3.355  | 4.501   | 5.041   |
| 9         | 1.383 | 1.833 | 2.262  | 2.821  | 3.25   | 4.297   | 4.781   |
| 10        | 1.372 | 1.812 | 2.228  | 2.764  | 3.169  | 4.144   | 4.587   |
| 11        | 1.363 | 1.796 | 2.201  | 2.718  | 3.106  | 4.025   | 4.437   |
| 12        | 1.356 | 1.782 | 2.179  | 2.681  | 3.055  | 3.93    | 4.318   |
| 13        | 1.35  | 1.771 | 2.16   | 2.65   | 3.012  | 3.852   | 4.221   |
| 14        | 1.345 | 1.761 | 2.145  | 2.624  | 2.977  | 3.787   | 4.14    |
| 15        | 1.341 | 1.753 | 2.131  | 2.602  | 2.947  | 3.733   | 4.073   |

t-distribution table

# Comparing Two Learning Algorithms on One Data Set

- ▶ Given two learning algorithms ( $L_1$  and  $L_2$ ) and a data set  $S$ , run  $k$ -fold cross-validation.

- ▶ Result of 10-fold:

| Fold    | Error rate (%) of $L_1$ | Error rate (%) of $L_2$ |
|---------|-------------------------|-------------------------|
| 1       | 5                       | 7                       |
| 2       | 2                       | 1.9                     |
| 3       | 7.8                     | 5.7                     |
| 4       | 4.9                     | 5.1                     |
| 5       | 12                      | 15                      |
| 6       | 8                       | 9                       |
| 7       | 7.6                     | 9.8                     |
| 8       | 10                      | 11                      |
| 9       | 6                       | 5.9                     |
| 10      | 8                       | 9.8                     |
| Average | 7.13                    | 8.02                    |

- ▶ Paired t-test result:

- ▶ p-value = 0.089

# Comparing Two Learning Algorithms on a Number of Data Sets

- ▶ Given two learning algorithms ( $L_1$  and  $L_2$ ) and a few data sets  $S_1, S_2, \dots, S_m$ , run  $k$ -fold cross-validation on each algorithm and each data set.
- ▶ Results of 10-fold cross-validation of  $L_1$  and  $L_2$  on each data set:

| Dataset | Average error rate (%) of 10-fold CV of $L_1$ | Average error rate (%) of 10-fold CV of $L_2$ |
|---------|-----------------------------------------------|-----------------------------------------------|
| $S_1$   | 6.7                                           | 5.0                                           |
| $S_2$   | 2                                             | 0.5                                           |
| $S_3$   | 20.6                                          | 15.7                                          |
| $S_4$   | 10.2                                          | 6.8                                           |
| $S_5$   | 1.8                                           | 1.8                                           |
| $S_6$   | 9                                             | 6.5                                           |
| $S_7$   | 7.6                                           | 9.8                                           |
| $S_8$   | 17                                            | 11                                            |
| Average | 9.36                                          | 7.14                                          |

- ▶ Paired t-test result:
  - ▶ p-value = 0.048



## Note on Paired t-test

- ▶ When the sample size is less than 30, it is required that
  - ▶ Differences between a pair of samples are approximately normally distributed.
- ▶ If not normally distributed,
  - ▶ Wilcoxon signed-rank test can be used

# Summary

- ▶ Predictive performance Measures
  - ▶ Classification
    - ▶ Accuracy, error, misclassification cost, precision, recall, F-score, AUC
  - ▶ Regression
    - ▶ MAE, MSE, RMSE, RAE, RSE
- ▶ Performance Evaluation Methods
  - ▶ Holdout
  - ▶ Repeated holdout
  - ▶ Cross-validation
  - ▶ Bootstrap
- ▶ Methods for model comparisons
  - ▶ Significance Test

# Data Mining

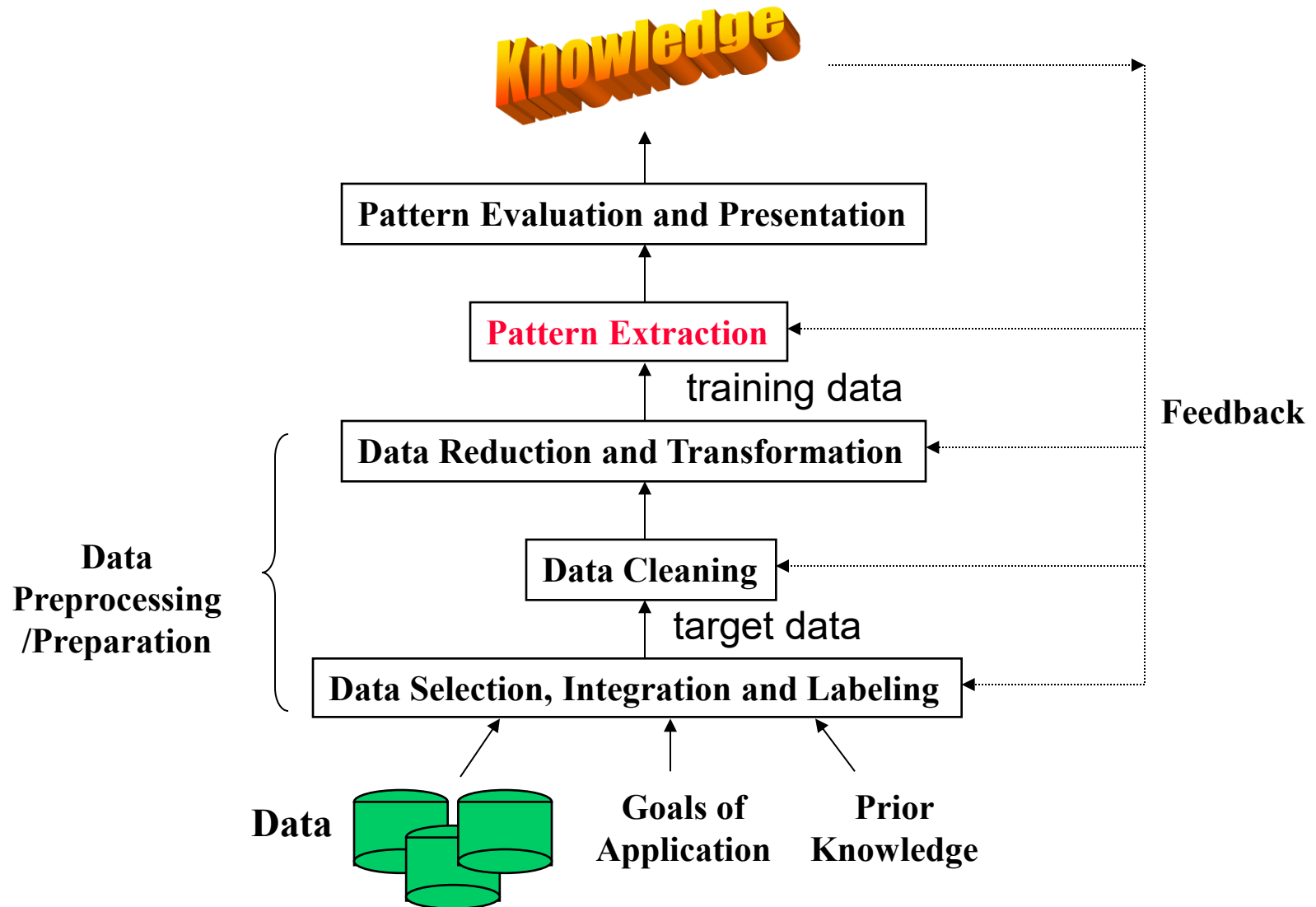
---

## Data Preprocessing/Preparation

Aijun An

Department of Electrical Engineering and Computer Science  
York University

# Process of Data Mining and KDD



# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Why Data Preprocessing?

- ▶ Heterogeneous data – data integration
  - ▶ From various departments, in various forms
- ▶ Data not labeled – Data Labeling
  - ▶ None or a small portion of examples have true outputs.
- ▶ Dirty data – data cleaning
  - ▶ Incomplete data: missing attribute values
    - ▶ e.g., occupation=""
  - ▶ Noisy data: containing errors
    - ▶ e.g., Salary="-10"
  - ▶ Discrepancies in codes or names
    - ▶ e.g., US=USA

# Why Data Preprocessing? (Cont'd)

- ▶ Data not in the right format – data transformation
  - ▶ Normalization, discretization, etc.
- ▶ A huge amount of data – data reduction
  - ▶ Speed up mining
- ▶ Data set is too small or not diverse – data augmentation
  - ▶ To learn a better model

No quality data, no quality mining results!

# Major Tasks in Data Preprocessing

- ▶ Data integration
  - ▶ Integration of multiple databases or files
- ▶ Data labeling
  - ▶ Provide true outputs (labels) for training examples
- ▶ Data cleaning
  - ▶ Fill in missing values, identify outliers and smooth out noisy data, and resolve discrepancies
- ▶ Data transformation
  - ▶ Feed the right data to the mining algorithm
- ▶ Data reduction and augmentation
  - ▶ Obtains reduced representation in volume but produces the same or similar analytical results, or
  - ▶ Creates more examples if the data set is too small
- ▶ Data discretization
  - ▶ Part of data reduction and data transformation but with particular importance, transform numerical data into symbolic (discrete) data



# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Integration

- ▶ Data integration:
  - ▶ combines data from multiple sources into a coherent store
- ▶ Schema integration
  - ▶ integrate metadata from different sources
  - ▶ *Entity identification problem*: identify real world entities from multiple data sources, e.g.,  $A.cust-id \equiv B.cust-\#$
- ▶ Detecting and resolving data value conflicts
  - ▶ for the same real world entity, attribute values from different sources are different
  - ▶ possible reasons: different representations, different scales,
    - ▶ e.g., hotel price in different currencies, metric vs. British units
    - ▶ e.g., Age="42", Birthday="03/07/1997"
    - ▶ e.g., Was rating "1,2,3", now rating "A, B, C"

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Cleaning

## ▶ Why is data dirty?

### ▶ Incomplete data come from

- ▶ human/hardware/software problems (e.g. equipment malfunction)
- ▶ different consideration between the time when the data was collected and when it is analyzed.
  - ▶ certain data may not be considered important at the time of entry

### ▶ Noisy data come from the process of

- ▶ data collection
- ▶ data entry
- ▶ data transmission

## ▶ Data cleaning tasks

- ▶ Fill in missing values
- ▶ Identify outliers and smooth out noisy data

# How to Handle Missing Values in Pre-processing?

- ▶ Fill in the missing value manually: tedious + infeasible?
- ▶ Ignore the tuple containing missing values:

| Cust-id | Age | Gender | Income | Credit |
|---------|-----|--------|--------|--------|
| 1       | 36  | M      | \$54K  | good   |
| 2       | 24  | M      | \$20K  | bad    |
| 3       | 37  | M      | \$50K  | ?      |
| 4       | 23  | F      | \$30K  | good   |
| 5       | 55  | F      | \$25K  | good   |
| 6       | 35  | ?      | \$16K  | bad    |
| 7       | 33  | F      | \$10K  | bad    |

- ▶ usually done when class label is missing (assuming the task is classification)
  - ▶ not effective when missing values in attributes spread in many different tuples.
- ▶ Fill it in with a value “unknown”
  - ▶ patterns containing “unknown” is ugly

# How to Handle Missing Values in Pre-processing ? (*Contd.*)

- ▶ Global estimation
  - ▶ the attribute mean/median for numeric attributes
  - ▶ the most probable value for symbolic (i.e. categorical) attributes
- ▶ Local estimation: smarter
  - ▶ the attribute mean/median for all the tuples belonging to the same class (for numeric attributes)
  - ▶ the most probable value within the same class (for symbolic attributes)

| Cust-id | Age | Gender | Income | Credit |
|---------|-----|--------|--------|--------|
| 1       | 36  | F      | \$55K  | good   |
| 2       | 24  | ?      | \$20K  | bad    |
| 3       | 37  | F      | \$50K  | good   |
| 4       | 23  | F      | \$30K  | good   |
| 5       | 55  | F      | \$25K  | good   |
| 6       | 35  | M      | ?      | bad    |
| 7       | 33  | M      | \$10K  | bad    |

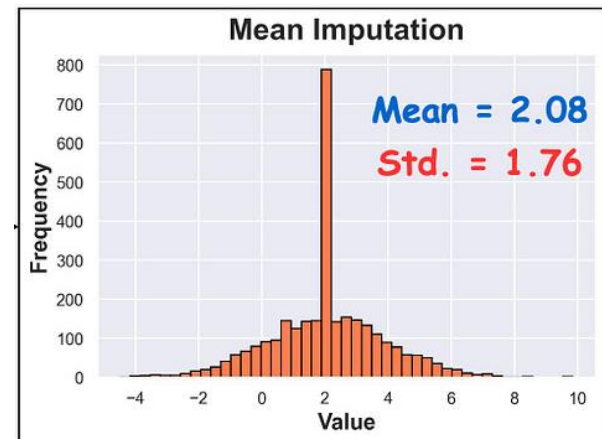
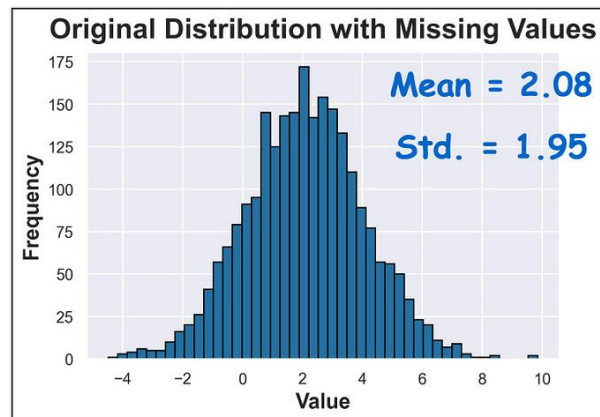
# Disadvantage:

Replacing (imputing) missing values with mean or any other fixed value:

- alters summary statistics
- changes the distribution
- inflates the presence of a specific value

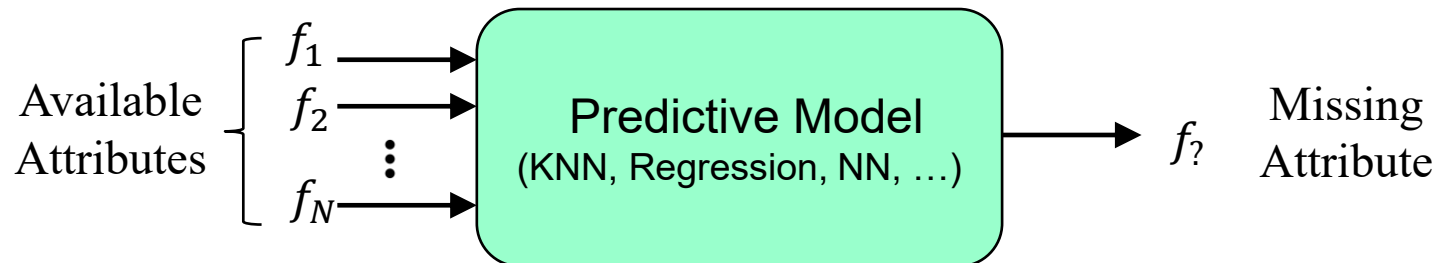
This can lead to:

- inaccurate modeling
- incorrect conclusions



# How to Handle Missing Values in Pre-processing ? (*Contd.*)

- ▶ Use inference-based prediction techniques, such as
  - ▶ Nearest-neighbor estimator, decision tree, regression, neural network, etc.
  - ▶ good method with overhead



- ▶ A predictive model like a decision tree or KNN can estimate missing feature values by learning from other available features and predicting the most likely value for the missing one.



# Noisy Data

- ▶ Noise: random error or variance in a measured variable
- ▶ Incorrect attribute values may be due to
  - ▶ faulty data collection instruments
  - ▶ data entry problems
  - ▶ data transmission problems

# How to Handle Noisy Data?

## ▶ Clustering

- ▶ detect and remove outliers (An outlier is a value that does not follow the general pattern of the rest)

## ▶ Regression

- ▶ smooth by fitting the data into regression functions

## ▶ Using Z-score

## ▶ Moving average

- ▶ Use the arithmetic mean of neighborhood examples

## ▶ Data generalization or discretization:

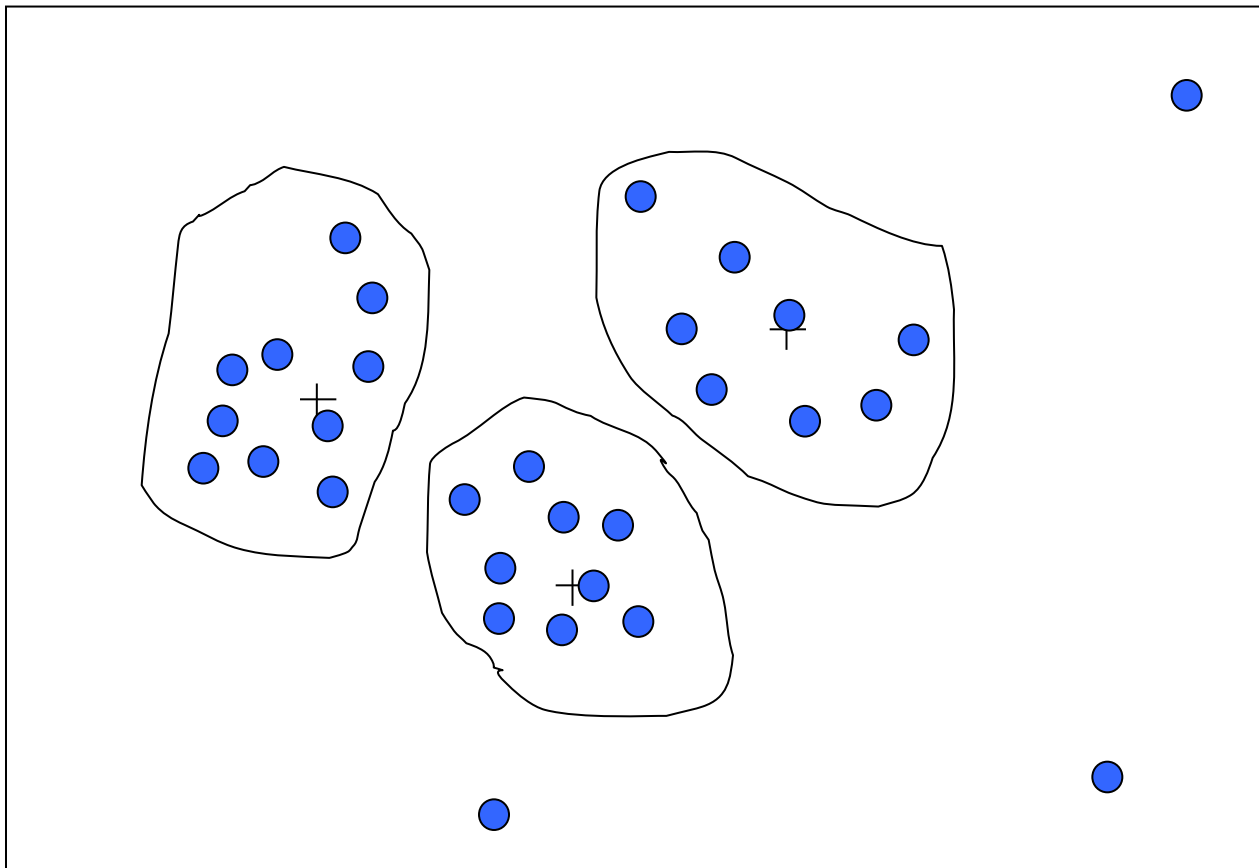
- ▶ first sort data and partition it into bins
- ▶ then one can smooth by bin labels, etc.

## ▶ Combined computer and human inspection

- ▶ detect suspicious values and check by human (e.g., deal with possible outliers)

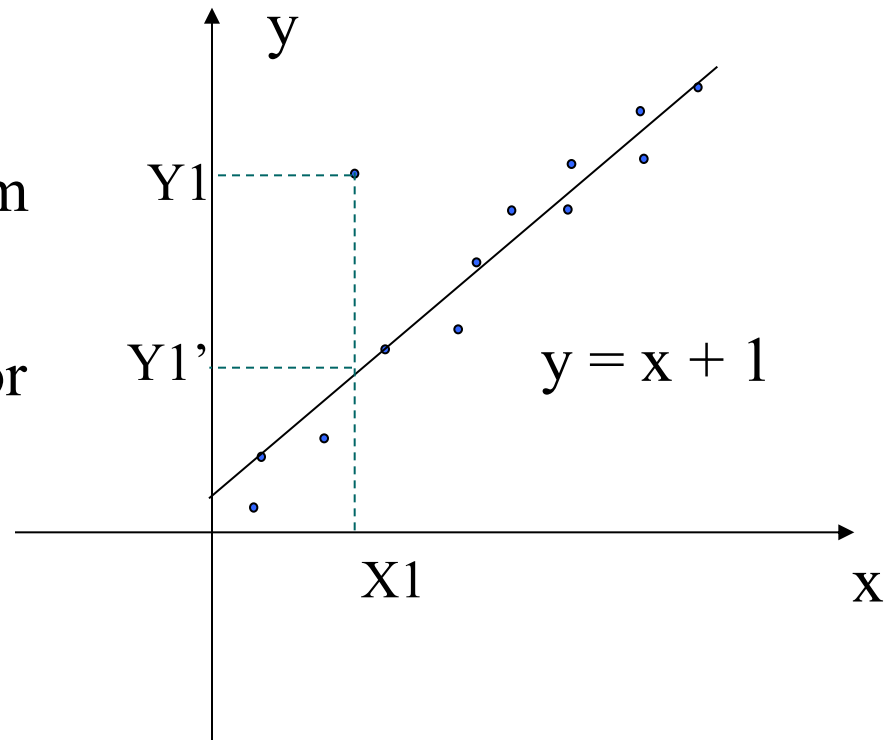
# Cluster Analysis

- Clustering detects outliers as points far from any cluster center, which can then be removed or corrected to handle noisy data.



# Regression

- ▶ Fit the data to a function.
- ▶ Data points too far away from the function are outliers.
- ▶ A *single linear regression*, for instance, finds the line to fit data with 2 variables so that one variable can predict the other.
- ▶ More variables can be involved in *multiple linear regression*.

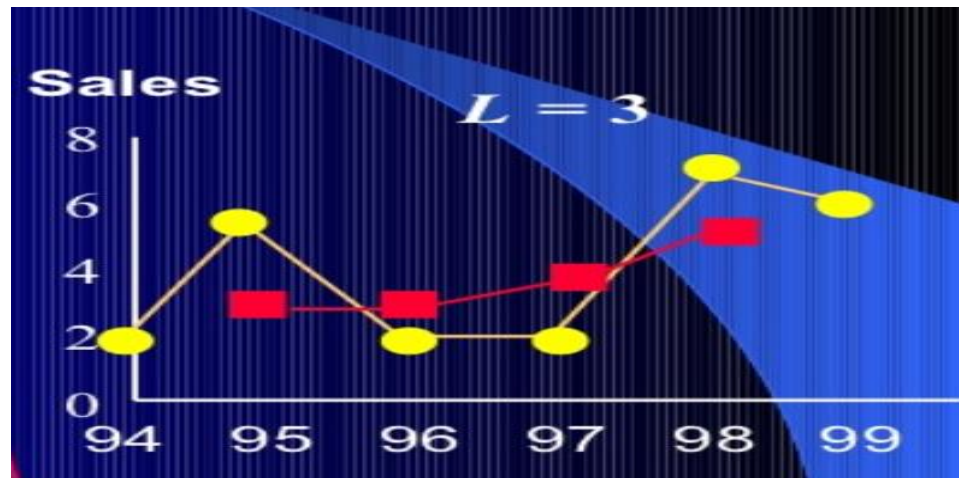


# Moving Average

- ▶ Use neighborhood values to smooth out noise
- ▶ Typically used for time-series data
  - ▶ Use series of arithmetic means over time
  - ▶ Result depends on choice of length  $L$  for computing mean.
- ▶ Can also be used on spatial data such as images

# Moving Average Example

| Year | Sales | Moving Average |
|------|-------|----------------|
| 1994 | 2     | NA             |
| 1995 | 5     | 3              |
| 1996 | 2     | 3              |
| 1997 | 2     | 3.67           |
| 1998 | 7     | 5              |
| 1999 | 6     | NA             |



# Using Z-scores to Detect Outliers

- ▶ Given an attribute  $X$  (e.g., Height) with a set of values, Z-score of value  $x_i$  is:

$$Z_i = \frac{x_i - \mu}{\sigma}$$

where  $\mu$  is the mean of  $X$ , and  $\sigma$  is the standard deviation of  $X$ :

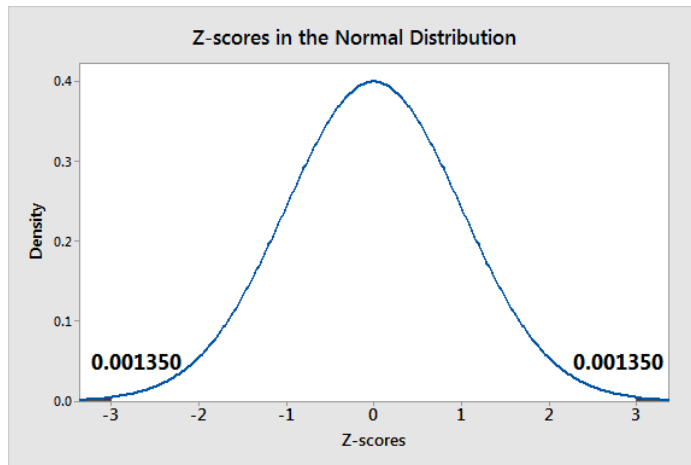
$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N}} \text{ or } \sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N-1}}$$

- ▶  $z_i$  measures how many  $\sigma$ 's away  $x_i$  is

| Height  | Z-score  |
|---------|----------|
| 1.5895  | -0.34603 |
| 1.6508  | -0.31975 |
| 1.7131  | -0.29301 |
| 1.7136  | -0.29283 |
| 1.7212  | -0.28954 |
| 1.7296  | -0.28595 |
| 1.7343  | -0.28394 |
| 1.7663  | -0.27020 |
| 1.8018  | -0.25501 |
| 1.8394  | -0.23888 |
| 1.8869  | -0.21852 |
| 1.9357  | -0.19757 |
| 1.9482  | -0.19223 |
| 2.1038  | -0.12551 |
| 10.8135 | 3.60910  |

# Using Z-scores to Detect Outliers

- ▶ The further away  $x_i$ 's Z-score is from zero, the more unusual it is.
- ▶ A **standard cut-off value** for finding outliers is
  - ▶ Z-scores of **+/-3** or further from zero.
- ▶ Distribution of Z-scores in a standard normal distribution



*Z-scores more extreme than +/-3 have a probability of 0.0027 ( $2 \times 0.00135$ )*

- ▶ Limitation:
  - ▶ If data don't follow the normal distribution, this approach might not be accurate.



# Some Other Outlier Detection Methods

- ▶ The following methods consider multiple attributes together to determine whether an example is an outlier:
  - ▶  $N$  = the number of examples
  - ▶  $k$  = the number of attributes,

| Outlier Detection Methods | Outlier Threshold                                     |
|---------------------------|-------------------------------------------------------|
| Mahalanobis Distance      | $> \chi^2$ critical value with $k$ degrees of freedom |
| Cook's Distance           | $> 4/(N-k-1)$                                         |
| Leverage                  | $> 2(k+1)/N$                                          |
| Standardized DFFIT        | $> 2\sqrt{(k+1)/N}$                                   |
| Standardized DFBETA       | $> 2/\sqrt{N}$                                        |

# Data Generalization or Discretization

- ▶ Objective

- ▶ Smooth out local noise by generalization and discretization

- ▶ Methods for data generalization

- ▶ Nominal attributes

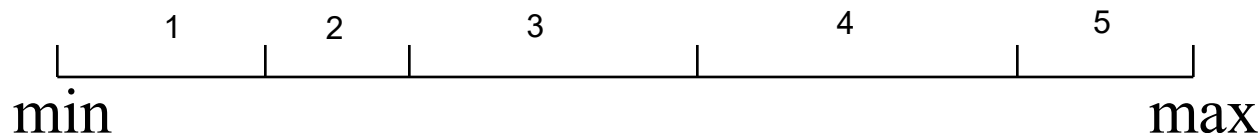
- ▶ Generalization

Toronto → Ontario → Central Canada → Canada

- ▶ Real-valued attributes

- ▶ Discretization:

- ▶ Partition the value range of an attribute into bins/intervals
      - ▶ label each bin with discrete values



# Data Generalization or Discretization

- ▶ Other benefits
  - ▶ Reduce the number of distinct values of a feature so that the size of the search space for patterns is reduced.
    - ▶ Will see this in rule induction techniques
  - ▶ Model learned (e.g. decision tree) is smaller

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Reduction

- ▶ What is data reduction?
  - ▶ Reduce the size of data while retaining its essential information
- ▶ Why data reduction?
  - ▶ Efficiency
    - ▶ A data set may be too large to fit in main memory.
    - ▶ The expected time for inducing a solution may be too long. Trade off accuracy for speed-up.
  - ▶ Accuracy
    - ▶ Better models may be found by using a reduced subset. E.g.,
      - ▶ Removing irrelevant redundant features mitigates the risk of overfitting
      - ▶ Too large data set may cause the model to fit too many exceptions if data are noisy
  - ▶ Interpretability
    - ▶ A larger dataset can often lead to larger (or deeper) decision trees
    - ▶ A smaller set of features makes the model more interpretable.

# Data Reduction Operations

## ▶ Standard data form

| <i>Case/Example</i> | <i>feature<sub>1</sub></i> | <i>...</i> | <i>feature<sub>k</sub></i> | <i>Class</i> |
|---------------------|----------------------------|------------|----------------------------|--------------|
| $e_1$               | $V_{1,1}$                  | $\dots$    | $V_{1,k}$                  | $c_1$        |
| $\dots$             | $\dots$                    | $\dots$    | $\dots$                    | $\dots$      |
| $e_i$               | $V_{i,1}$                  | $\dots$    | $V_{i,k}$                  | $c_i$        |
| $\dots$             | $\dots$                    | $\dots$    | $\dots$                    | $\dots$      |
| $e_n$               | $V_{n,1}$                  | $\dots$    | $V_{n,k}$                  | $c_n$        |

## ▶ Data reduction operations

- ▶ Feature reduction (reduce the number of columns)
- ▶ Case reduction (reduce the number of rows)

# Types of Attributes (Features)

- ▶ Three types of attributes:
  - ▶ Nominal (symbolic, categorical) — values from an unordered set
    - ▶ Eg: {red, yellow, blue, ....}
  - ▶ Ordinal — values from an ordered set
    - ▶ Eg: {low, medium, high}
  - ▶ Continuous — real numbers
    - ▶ Eg: {-9.8, 3.9, 8.7, 19.1}
- ▶ Next: feature reduction for classification tasks.

# Feature Reduction Techniques

- ▶ Feature Selection
  - ▶ Selecting a subset of features with predictive performance comparable to the full set of features
- ▶ Feature Projection (also called Feature Extraction)
  - ▶ Mapping the original set of features into a new set of fewer features with new values.

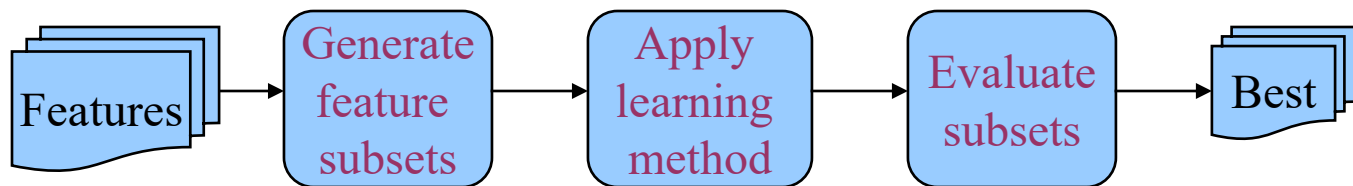


# Feature Selection

- ▶ Objective

- ▶ Find an optimal subset of features according to an objective function.

- ▶ An optimal subset selection



- ▶ A *practical objective* is to remove clearly extraneous features

- ▶ leaving the table reduced to manageable dimensions
  - ▶ not necessarily to select the optimal subset.

# Feature Selection Methods

- ▶ Filter Methods: select a subset of original features.
  - ▶ Feature Selection from Means and Variances (✓)
  - ▶ Feature Selection by Mutual Information (✓)
  - ▶ Feature Selection by Chi-Square Test (✓)
  - ▶ Feature Selection by Decision Trees (✓)
  - ▶ Feature Selection by Rough Sets, etc.
- ▶ Wrapper Methods: feature selection is being “wrapper around” a learning algorithm.
  - ▶ This is the optimal method in the last slide.
  - ▶ Running time is long; infeasible in practice if there are many features.

# Feature Selection from Means and Variances

## ▶ Principle

- ▶ Compute the means of a feature for each class, normalized by the variances;
- ▶ If the means are far apart, interest in a feature increases (the feature has potential in terms of distinguishing between classes);
- ▶ If the means are indistinguishable, interest wanes in that feature.

## ▶ Two intuitive methods

- ▶ Independent feature analysis ( $\sqrt{\phantom{x}}$ )
  - ▶ Assuming the features are independent. Features are examined individually.
- ▶ Distance-based feature selection
  - ▶ Features are examined collectively.

## ▶ Limitation: only applied to continuous features.

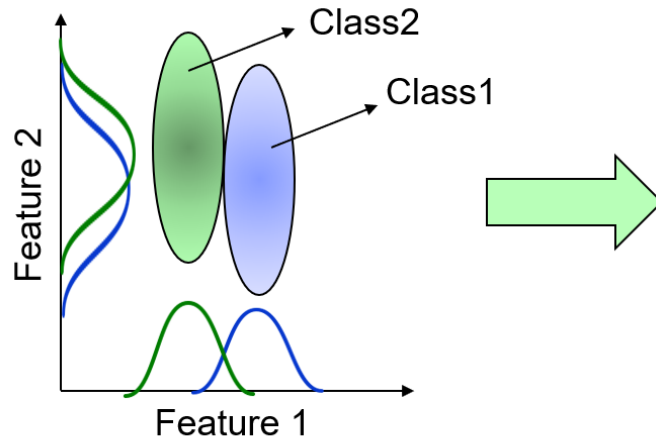
# Independent Feature Analysis

- ▶ For a problem with two classes:  $C_1$  and  $C_2$  :
  - ▶ Compute  $mean_1(f)$  and  $mean_2(f)$ : the means of feature  $f$  measured for  $C_1$  and  $C_2$
  - ▶ Compute  $var_1(f)$  and  $var_2(f)$ : the variances of feature  $f$  measured for  $C_1$  and  $C_2$
  - ▶ Significance test (t-test):

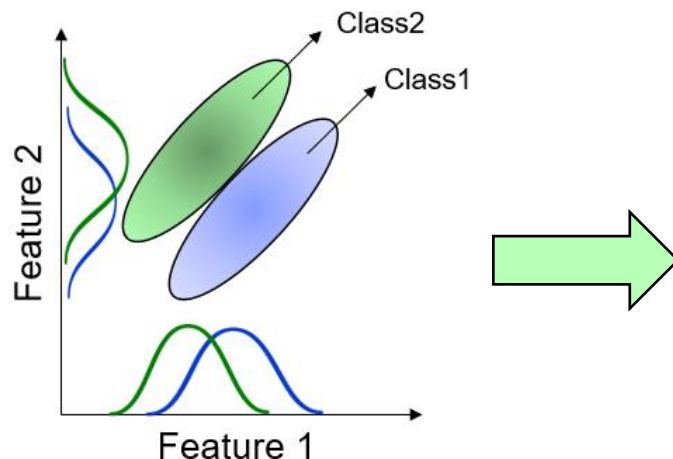
$$|mean_1(f) - mean_2(f)| > sig \times \sqrt{\frac{var_1(f)}{n_1} + \frac{var_2(f)}{n_2}}$$

- ▶  $n_1$  and  $n_2$  are the numbers of cases in  $C_1$  and  $C_2$
    - ▶  $sig = 2$  for the 95% confidence level.
  - ▶ If the comparison fails the test, the feature can be deleted.
- ▶ For  $k$  classes,  $k$  pairwise comparisons are conducted for  $f$ .
  - ▶ Each pairwise comparison compares feature means for class  $C_i$  and  $\neg C_i$  ( $i=1, \dots, k$ ).
  - ▶ A feature is retained if it is significant for at least one of the pairwise comparisons.
- ▶ Limitations:
  - ▶ Only applies to numeric attributes
  - ▶ Treat each feature independently

## Example (Independent Feature Analysis):



- Features are independent:
- Feature 1 provides better class separability.
- so Feature 2 is removed during selection.



- Features are not independent:
- They are collaboratively separable.
- so independent feature analysis cannot be used for feature reduction.

# Feature Selection by Mutual Information

- ▶ Objective:
  - ▶ Select features according to the mutual information between a feature and the class variable.
- ▶ The mutual information (same as information gain) between the class variable  $y$  and a discrete feature  $x$  :

$$MI_x = \sum_j \sum_i [P(y = c_i, x = v_j) \times \log_2 \frac{P(y = c_i, x = v_j)}{P(y = c_i)P(x = v_j)}]$$

- ▶  $P(y=c_i)$  is the probability of examples in class  $c$ .
  - ▶  $P(x=v_j)$  is the probability that feature  $x$  takes on value  $v$ .
  - ▶  $P(y=c_i, x=v_j)$  is the probability of an example in class  $c$  and having  $v$  for feature  $x$ .
- ▶ MI measures the mutual dependence between the two variables  $x$  and  $y$ .
  - ▶ The higher the value, the more dependent one variable is on the other.

# MI (example):

| X (Feature) | Y (Prediction) |
|-------------|----------------|
| A           | 0              |
| A           | 0              |
| B           | 1              |
| B           | 1              |
| B           | 0              |

$$P(X = A) = \frac{2}{5}, \quad P(X = B) = \frac{3}{5}$$
$$P(Y = 0) = \frac{3}{5}, \quad P(Y = 1) = \frac{2}{5}$$

$$P(X = A, Y = 0) = \frac{2}{5}, \quad P(X = A, Y = 1) = 0, \quad P(X = B, Y = 0) = \frac{1}{5}, \quad P(X = B, Y = 1) = \frac{2}{5}$$

$$M_X = \sum_{x,y} P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}$$

$$M_X = (0.4) \log_2 \frac{0.4}{0.4 \times 0.6} + (0.2) \log_2 \frac{0.2}{0.6 \times 0.4} + (0.4) \log_2 \frac{0.4}{0.6 \times 0.4}$$
$$M_X \approx 0.278 \text{ bits}$$

# Feature Selection by Mutual Information

## *(Cont'd)*

- ▶ *MI* is used to weight and rank features.
  - ▶ You can select the top  $k$  features with the highest weights.
- ▶ Suitable for nominal or discrete attributes.
  - ▶ For continuous features, a discretization algorithm can be applied first to convert a real-valued feature to a discrete-valued feature.
- ▶ Limitations:
  - ▶ Treat each feature independently.
  - ▶ No natural threshold for mutual information



# Feature Selection Methods

- ▶ Filter Methods: select a subset of original features.
  - ▶ Feature Selection from Means and Variances (✓)
  - ▶ Feature Selection by Mutual Information (✓)
  - ▶ Feature Selection by Chi-Square Test (✓)
  - ▶ Feature Selection by Decision Trees (✓)
  - ▶ Feature Selection by Rough Sets, etc.
- ▶ Wrapper Methods: feature selection is being “wrapper around” a learning algorithm.
  - ▶ This is the optimal method in the last slide.
  - ▶ Running time is long; infeasible in practice if there are many features.

# Feature Selection by Chi-Square Test

- ▶ Remove the features that are not co-related to the class attribute according to the  $\chi^2$  test
- ▶ The purpose of a  $\chi^2$  test is to determine whether two variables are co-related.
  - ▶ E.g., we want know if there is any relationship between the gender of undergraduate students in a university and their footwear preferences.
- ▶ Observations about the two variables in a sample are usually expressed in a contingency table:

|        | Sandals | Sneakers | Leather shoes | Boots | Other | Total |
|--------|---------|----------|---------------|-------|-------|-------|
| Male   | 6       | 17       | 13            | 9     | 5     | 50    |
| Female | 13      | 5        | 7             | 16    | 9     | 50    |
| Total  | 19      | 22       | 20            | 25    | 14    | 100   |

# Chi-Square Significance Test

- ▶ The **null hypothesis** is that *the two variables are unrelated* (that is, only randomly related).
- ▶  $\chi^2$  test determines:
  - ▶ whether we should reject the null hypothesis and
  - ▶ *at what significance level (**p-value**)* we should reject the null hypothesis.
    - ▶ The **p-value** is the probability of obtaining the observed sample results (or a more extreme result) when the null hypothesis is actually true
    - ▶ Usually, p-value should be **at most 0.05** in order to reject the null hypothesis.

# Chi-Square Significance Test

Footwear preference

|        | Sandals | Sneakers | Leather shoes | Boots | Other | Total |
|--------|---------|----------|---------------|-------|-------|-------|
| Male   | 6       | 17       | 13            | 9     | 5     | 50    |
| Female | 13      | 5        | 7             | 16    | 9     | 50    |
| Total  | 19      | 22       | 20            | 25    | 14    | 100   |

- ▶ For the example above,
  - ▶ The null hypothesis is that *Gender* is unrelated with *Footwear preference*
  - ▶ But the  $\chi^2$  test shows that we should reject this hypothesis at the significance level of 0.01(**p-value**)

# How to Calculate $\chi^2$

- ▶ Given the contingency table:

|        | Sandals | Sneakers | Leather shoes | Boots | Other | Total |
|--------|---------|----------|---------------|-------|-------|-------|
| Male   | 6       | 17       | 13            | 9     | 5     | 50    |
| Female | 13      | 5        | 7             | 16    | 9     | 50    |
| Total  | 19      | 22       | 20            | 25    | 14    | 100   |

- ▶ Compute the expected frequency for each cell  
assuming null hypothesis

- ▶ The expected frequency of cell<sub>i,j</sub> is

$$E_{ij} = \frac{\text{the total of row } i \times \text{the total of column } j}{\text{sample size}}$$

- ▶ For example, the expected frequency of the upper left cell is  $\frac{50 \times 19}{100}$

## How to Calculate $\chi^2$ (Cont'd)

- ▶ Compute the chi-square value for the table

|        | Sandals | Sneakers | Leather shoes | Boots | Other | Total |
|--------|---------|----------|---------------|-------|-------|-------|
| Male   | 6       | 17       | 13            | 9     | 5     | 50    |
| Female | 13      | 5        | 7             | 16    | 9     | 50    |
| Total  | 19      | 22       | 20            | 25    | 14    | 100   |

- ▶ Let  $O_{ij}$  denote the observed value in cell <sub>$i,j$</sub>

$$\chi^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

- ▶ For example, the chi-square value of the above table is 14.026

# How to Calculate $\chi^2$ (Cont'd)

- ▶ Calculate the degrees of freedom for the table

|        | Sandals | Sneakers | Leather shoes | Boots | Other | Total |
|--------|---------|----------|---------------|-------|-------|-------|
| Male   | 6       | 17       | 13            | 9     | 5     | 50    |
| Female | 13      | 5        | 7             | 16    | 9     | 50    |
| Total  | 19      | 22       | 20            | 25    | 14    | 100   |

$$df = (r-1)(c-1)$$

- ▶ where  $r$  is the number of rows and  $c$  is the number of columns (without the row or column for totals)
- ▶ For example, the degrees of freedom for the above table is 4.
- ▶ This is because, given row or column totals, all but one of the values in a given row or column are free to vary.

# How to Calculate $\chi^2$ (Cont'd)

- ▶ Using the chi-square table to determine the *p-value* for rejecting the null hypothesis

| df  | P = 0.05 | P = 0.01 | P = 0.001 |
|-----|----------|----------|-----------|
| 1   | 3.84     | 6.64     | 10.83     |
| 2   | 5.99     | 9.21     | 13.82     |
| 3   | 7.82     | 11.35    | 16.27     |
| 4   | 9.49     | 13.28    | 18.47     |
| 5   | 11.07    | 15.09    | 20.52     |
| ... | ...      | ...      | ...       |

- ▶ The table lists the critical values (i.e., thresholds)
- ▶ The calculated chi-square value for a contingency table must be greater than the critical value corresponding to the *df* of the table and a *p-value* (e.g., 0.05) in order to reject the null hypothesis at the significance level (p-value).



# Feature Selection Methods

- ▶ Filter Methods: select a subset of original features.
  - ▶ Feature Selection from Means and Variances (✓)
  - ▶ Feature Selection by Mutual Information (✓)
  - ▶ Feature Selection by Chi-Square Test (✓)
  - ▶ Feature Selection by Decision Trees (✓)
  - ▶ Feature Selection by Rough Sets, etc.
- ▶ Wrapper Methods: feature selection is being “wrapper around” a learning algorithm.
  - ▶ This is the optimal method in the last slide.
  - ▶ Running time is long; infeasible in practice if there are many features.

# Feature Selection by Decision Trees

## ▶ Objective

- ▶ Decision tree learning methods integrates feature selection to their algorithms and decision tree is a fast learning method.
- ▶ Make use of the decision tree learning technique to select features from a data set for other learning methods, such as neural networks, that take substantially more time to search their solution space.
  - ▶ Decision tree learning is a relatively fast learning method.

## ▶ Method

- ▶ Apply a decision tree learning algorithm to the data set to generate a decision tree.
- ▶ Select features that appear in the tree.

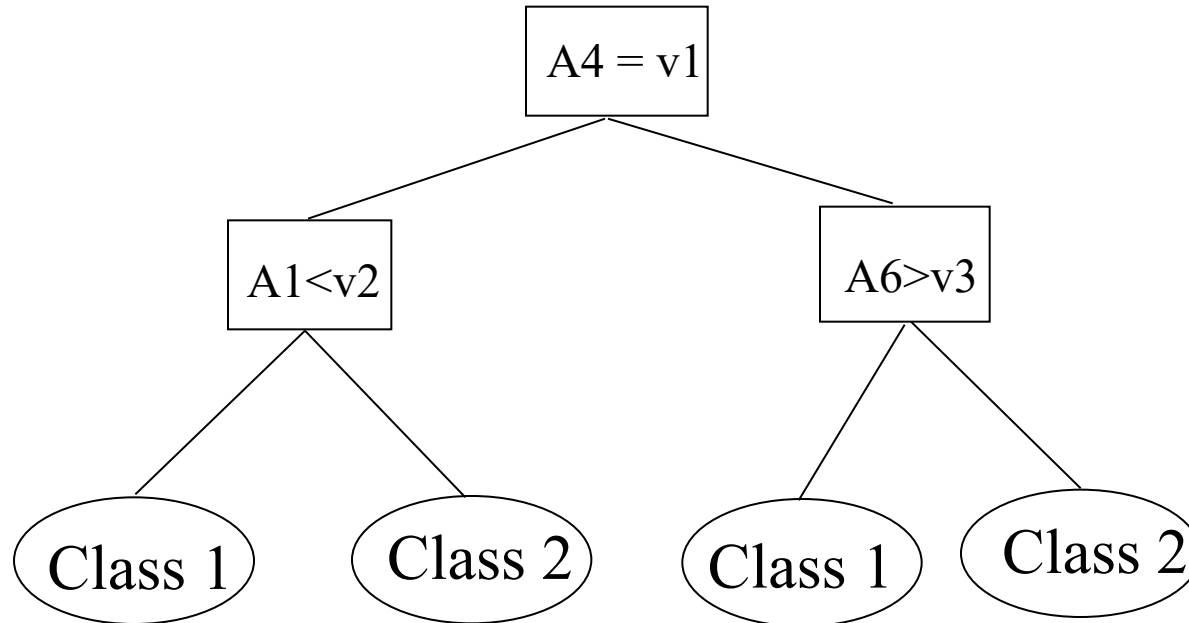
## ▶ Advantage

- ▶ *Context sensitive*. Tree methods evaluate candidate features in the context of related features that have already been selected.

# Example of Feature Selection Using Decision Tree

Initial attribute set:

$\{A1, A2, A3, A4, A5, A6\}$



-----> Reduced attribute set:  $\{A1, A4, A6\}$

# Feature Reduction Techniques

- ▶ Feature Selection
  - ▶ Selecting a subset of features with predictive performance comparable to the full set of features
- ▶ Feature Projection (also called Feature Extraction)
  - ▶ Mapping the original set of features into a new set of fewer features with new values.

# Feature Projection

## ▶ Objective

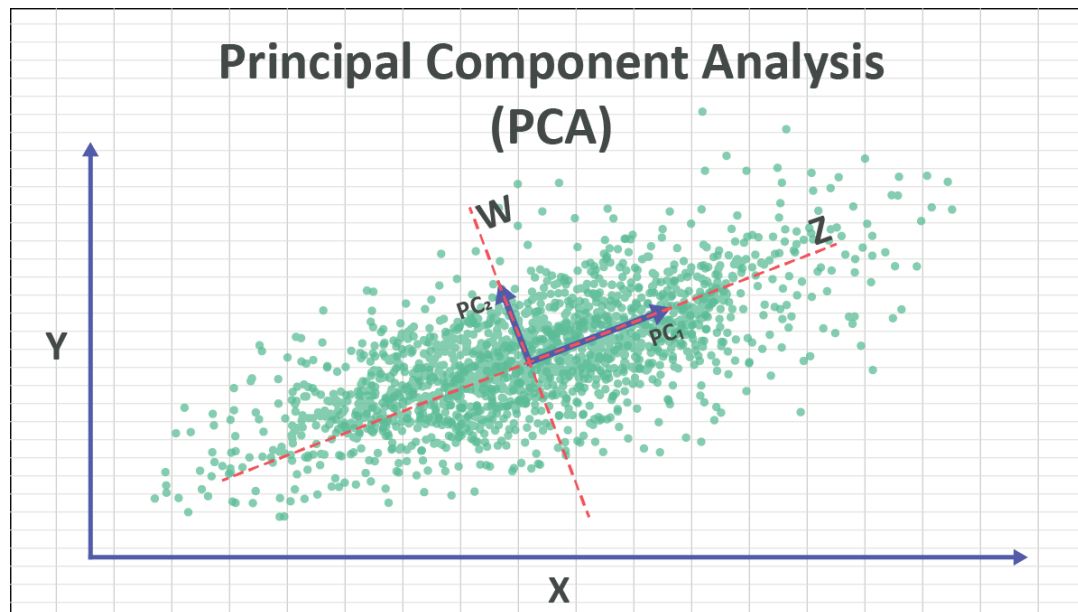
- ▶ Map the original features into a smaller set of newly created features which hopefully retains most of the relevant information from *all* the original features.

## ▶ Methods

- ▶ Principal component analysis (PCA)
- ▶ Neural autoencoder
- ▶ t-distributed stochastic neighbor embedding (t-SNE)
- ▶ .....

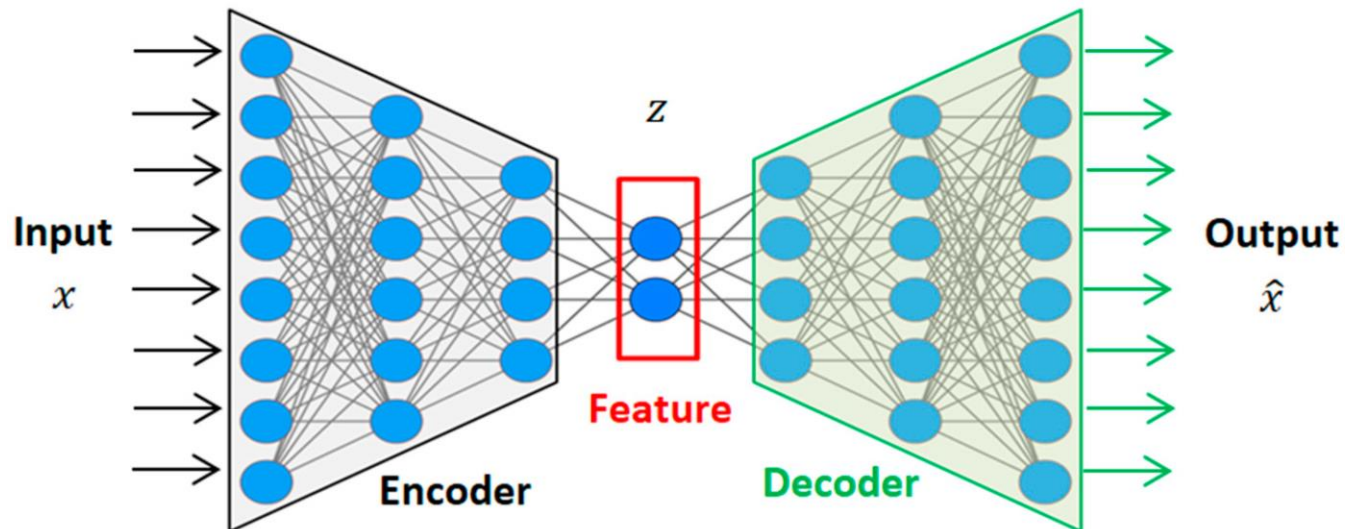
# Feature Projection

- ▶ Principal Component Analysis:
  - ▶ PCA transforms correlated features into a set of uncorrelated components.
  - ▶ It captures the directions (principal components) with the highest variance in the data.
  - ▶ It replaces original features with top  $k$  principal components that explain the most variance.



# Feature Projection

- ▶ Neural Autoencoder:
  - ▶ It learns to compress input data into a smaller representation and then reconstruct it.
  - ▶ The goal is to capture the most important information in a low-dimensional form.
  - ▶ PCA is linear, while autoencoders learn nonlinear feature projections.



# Data Mining

---

## Data Preprocessing/Preparation-2

Aijun An

Department of Electrical Engineering and Computer Science  
York University



# Data Reduction Outline

- ▶ Feature Reduction
- ▶ Case Reduction

# Case Reduction

- ▶ Objective: reduce the number of examples, the largest dimension in the data set
- ▶ How many examples are enough?
  - ▶ Application dependent - depends on the complexity of the patterns to be extracted from the data.
    - ▶ If the pattern is simple, the results are unlikely to change even with additional cases. For example,  $x > 1$  completely separates two classes.
    - ▶ For complex patterns, large volumes of data can supply more evidence for the correctness of the induced patterns.
- ▶ Some types of problems requiring more data than others:
  - ▶ Multiclass classification
  - ▶ Regression
  - ▶ Imbalanced data sets
  - ▶ Deep neural network training

# Case Reduction Methods

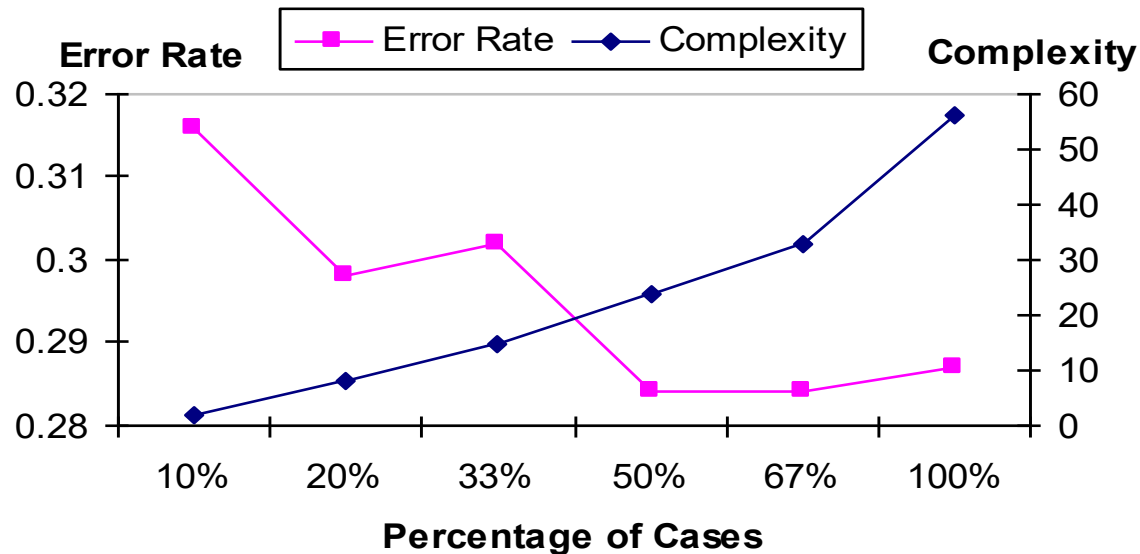
- ▶ Simple Random Sampling
  - ▶ A single sample
  - ▶ Incremental samples
  - ▶ Average samples
- ▶ Sampling by Boosting Class Balance
- ▶ Stratified Sampling

# Single Simple Random Sample

- ▶ Choose  $n$  objects randomly from a set  $D$  of  $N$  objects ( $n < N$ ) so that each object has the same probability of being chosen.
- ▶ Two methods
  - ▶ Simple random sampling without replacement (SRSWOR)
    - ▶ Each object cannot be chosen more than once
  - ▶ Simple random sampling with replacement (SRSWR)
    - ▶ Each time an object is drawn, it is recorded and placed back to  $D$  so that it can be drawn again.

# Problem with Single Sampling

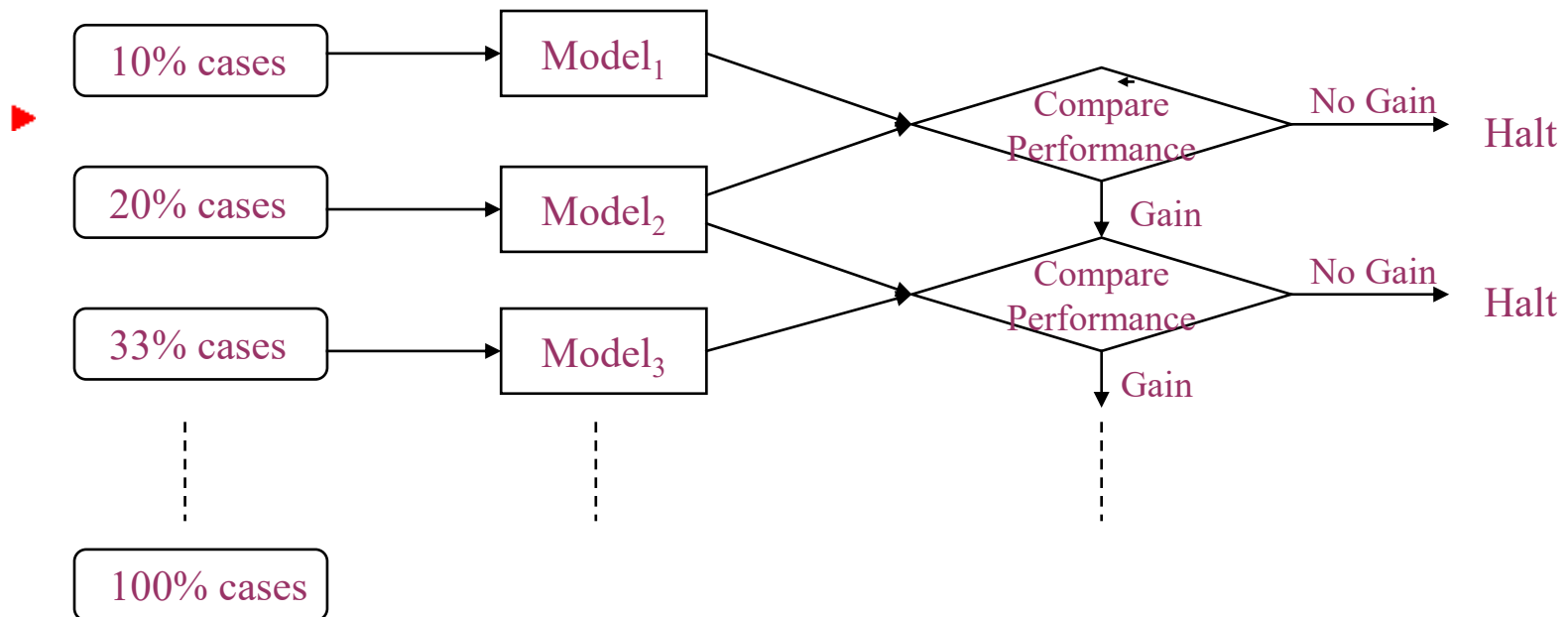
- ▶ We don't know the suitable sample size.
- ▶ Too small, the model may not be accurate enough; too big, the model may be more complex
  - ▶ An example:



# Incremental Sampling

- ▶ Objective: Spot trends in error and complexity by *learning with incrementally larger random subsets of the data* to help produce a ***single solution***.
- ▶ A typical pattern of incremental subsets is:

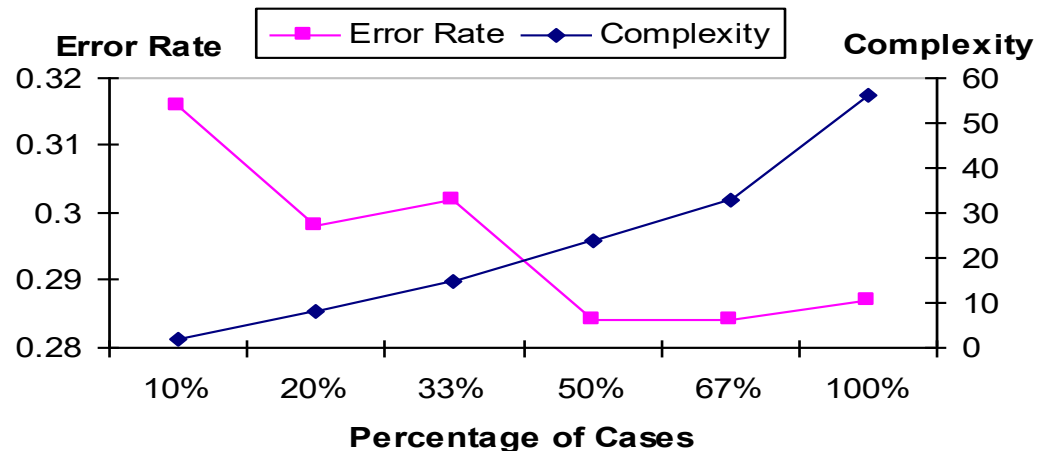
10%, 20%, 33%, 50%, 67%, 100%



# Incremental Sampling (Cont.)

- ▶ Performance measures:
  - ▶ Error rate (test error, that is error on a test data set)
  - ▶ Complexity of the solution (e.g. number of nodes in a tree)
- ▶ Plot error and complexity relative to increasing sample size.

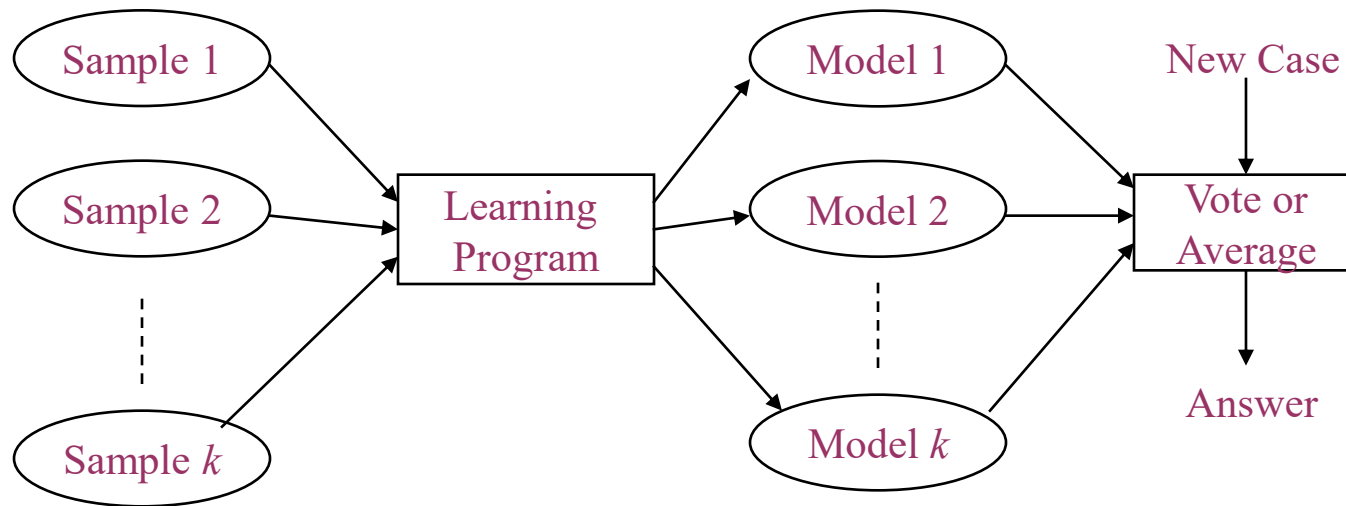
Example:



- ▶ Make decision on whether to do further sampling
  - ▶ Net changes in error and complexity are examined:
    - ▶ Is the error smaller?
    - ▶ Is the complexity acceptable?
    - ▶ Is complexity increasing much more than error is decreasing?

# Average Sampling

- ▶ For a dataset containing a huge number of cases that exceed the maximum capacity of a learning program.
- ▶ Average sampling:
  - ▶ Select  $k$  random samples of  $n$  cases, where  $n < N$ .
  - ▶ Solutions from different samples are combined in the prediction phase.



- ▶ Averaged or voted solutions usually have less error than the single solution found on all cases in the database.



# Sampling by Boosting Class Balance

- ▶ Suitable for **imbalanced data sets** where
  - ▶ almost all examples belong to the larger class, and
  - ▶ far fewer cases to the smaller, usually more interesting, class.
- ▶ Many real-world datasets are extremely imbalanced.
  - ▶ E.g., in a bio-chemistry data set for predicting biological potency of chemical compounds,
    - ▶ Only 0.16% of the compounds belong to the class of highly active compounds, which is the most interesting class that can lead to discovery of new drugs.
    - ▶ Remaining 99.84% of compounds are inactive.
- ▶ **Challenge:**
  - ▶ A model that classifies all examples into the larger class is very accurate
  - ▶ Needs to avoid learning such a model.

# Sampling by Boosting Class Balance

Two ways to boost class balance:

- ▶ ***Up-sampling***: Increase the number of examples in the minority class
  - ▶ **Repeat** (or give higher weights to) the examples in the minority class in the sample - increase the sample size.
  - ▶ **Interpolation-based methods**
    - ▶ SMOTE: Generate synthetic data points for the minority class by linearly interpolating between minority instances and their neighbors.
    - ▶ ADSYN: an extension of SMOTE that generates additional synthetic samples for data points more difficult to learn based on the ratio of majority class samples in their nearest neighbors.

# Sampling by Boosting Class Balance

- ▶ ***Down-sampling:***

- ▶ Keep the minority cases intact or randomly sample a high percentage of them, while including a low percentage random subset of a larger class.

- ▶ **Hybrid:** use both up-sampling and down-sampling

- ▶ Note:

- ▶ the predictive performance on the interesting minority cases may increase
  - ▶ the overall predictive performance on data of all classes may decrease.
    - ▶ If consider the small class as positive, false positive rate increases

# Stratified Sampling

- ▶ The data set  $D$  is partitioned into mutually disjoint subsets, called *strata*.
- ▶ Then randomly sample data from each stratum
- ▶ Objective: ensure a representative sample, especially when the data are skewed.

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Augmentation

- ▶ Increase the size and diversity of data by artificially creating variations of the existing data
- ▶ Need for data augmentation
  - ▶ Deep learning often requires large amounts of data
  - ▶ Many applications have limited data available
  - ▶ Collecting large datasets (especially labeled data) is labor-intensive
  - ▶ More data can alleviate the overfitting problem, increasing predictive performance
  - ▶ Can be used to alleviate the class imbalance issue by augmenting data from the under-represented class

# General Data Augmentation Techniques

## ▶ Basic Interpolation

- ▶ Interpolates existing examples (of the same class) to create new examples of the class.
- ▶ Local manipulation

## ▶ Augmentation data synthesis (Advanced)

- ▶ Learns data patterns from a global view and generates new examples with a learned model
- ▶ GANs (Generative Adversarial Networks)
  - ▶ Train a discriminator in conjunction with a generator, making the latter generate synthetic data that closely resembles the existing data.
- ▶ Variational Autoencoder
- ▶ Diffusion models

# Data Augmentation Techniques for Images

- ▶ Special techniques for images
  - ▶ Geometric transformations
    - ▶ Randomly flip, crop, rotate, stretch, and zoom images
  - ▶ Color space transformations
    - ▶ Randomly change RGB color channels, contrast, and brightness
  - ▶ Kernel filters
    - ▶ Randomly change the sharpness or blurring of the images
  - ▶ Random erasing
    - ▶ Delete some parts of the initial image.
  - ▶ Mixing images
    - ▶ Blending and mixing multiple images



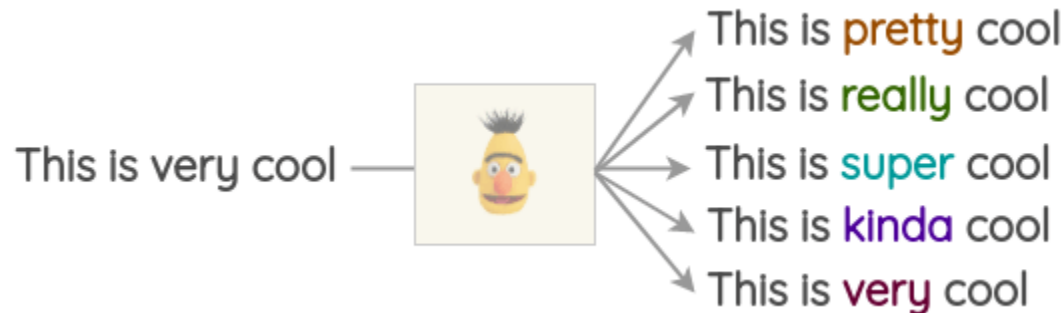
# Data Augmentation Techniques for Texts

- ▶ Special basic techniques for texts

- ▶ Synonym replacement

- ▶ Lexical-based replacement

- ▶ Use a lexical database (e.g., WordNet) to find synonyms to replace words/tokens in the original sentence.



(Picture source: Amit Chaudhary "[A Visual Survey of Data Augmentation in NLP](#)")

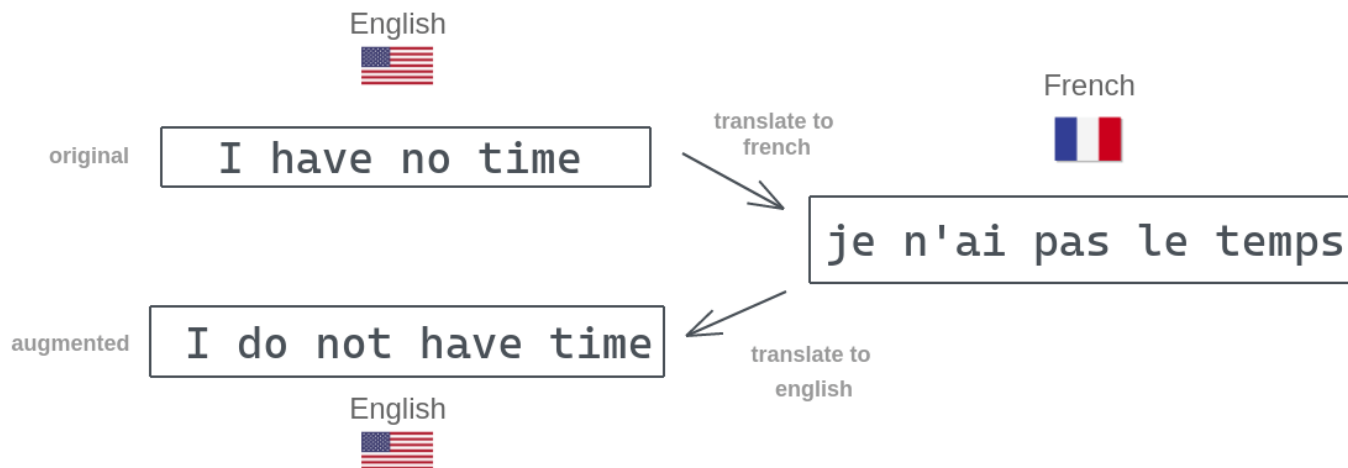
- ▶ Word embedding based replacement

- ▶ Use pre-trained word embeddings (e.g, GloVe, Word2Vec, fastText, ELMo, BERT, etc.) to find the nearest word vector as the replacement in the original sentence

# Data Augmentation Techniques for Texts

## ► Back translation

- Translate a text into another language and then translate it back to the original language.
- Different sentences of the same or similar meaning are created.



(Picture source: Amit Chaudhary ["Back Translation for Text Augmentation with Google Sheets"](#))

# Data Augmentation for Imbalanced Data

- ▶ Imbalanced data sets:
  - ▶ almost all examples belong to the larger class, and
  - ▶ far fewer cases to the smaller, usually more interesting, class.
- ▶ Many real-world datasets are extremely imbalanced.
  - ▶ E.g., in a bio-chemistry data set for predicting biological potency of chemical compounds,
    - ▶ Only 0.16% of the compounds belong to the class of highly active compounds, which is the most interesting class that can lead to discovery of new drugs.
    - ▶ Remaining 99.84% of compounds are inactive.
- ▶ Challenge:
  - ▶ A model that classifies all examples into the larger class is very accurate
  - ▶ Needs to avoid learning such a model.

# Data Augmentation for Imbalanced Data

## ▶ Up-sampling

- ▶ Repeat (or give higher weights to) the examples in the minority class in the sample - increase the sample size.
- ▶ Interpolation-based methods
  - ▶ SMOTE: Generate synthetic data points for the minority class by linearly interpolating between minority instances and their neighbors.
  - ▶ ADSYN: an extension of SMOTE that generates additional synthetic samples for data points more difficult to learn based on the ratio of majority class samples in their nearest neighbors.

## ▶ Note:

- ▶ the predictive performance on the interesting minority cases may increase, but
- ▶ the overall predictive performance on data of all classes may decrease.

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Labeling

- ▶ Provide correct outputs (e.g., class values) for training examples
- ▶ Essential for supervised training
- ▶ Methods
  - ▶ Crowdsourced labeling
  - ▶ Semi-supervised labeling
  - ▶ Active learning
  - ▶ Weakly-supervised labeling
  - ▶ Distant supervision

# Crowdsourced Labeling

## ▶ Idea:

- ▶ Break down a labeling task into smaller and more manageable parts
- ▶ Outsource and distribute them to a large number of non-expert annotators

## ▶ Full human participation

## ▶ Issues

- ▶ Need clear guidelines
- ▶ Quality varies
  - ▶ Need to prune low-quality ones
- ▶ Inconsistency occurs
  - ▶ Resolve inconsistency or keep the consensus labels

# Semi-Supervised Labeling

- ▶ Idea:
  - ▶ Leverage a small amount of labeled data to infer the labels of unlabeled data.
- ▶ Some methods
  - ▶ Multi-classifier based
    - ▶ Train multiple classifiers using the labeled data and use them to generate pseudo labels for unlabeled data.
  - ▶ Graph-based
    - ▶ Construct a graph where each node is an example and each edge represents the distance between 2 nodes
    - ▶ Infer labels through label propagation in the graph
  - ▶ Using reinforcement learning (RL) (e.g., ChatGPT)
    - ▶ Train initial models with labeled data
    - ▶ Train a reward function for RL from human feedback
    - ▶ Train a model using RL with the reward function



# Active Learning

- ▶ An iterative procedure involving humans in the loop
- ▶ In each iteration:
  - ▶ Learn a model from the labeled data
  - ▶ Select an (or a batch of) unlabeled samples for humans to annotate
- ▶ Key in how to select the unlabeled samples in each iteration
  - ▶ Select the most uncertain ones (e.g., based on the predicted probability of an example belonging to a class)
- ▶ Needs continuous involvement from human

# Weakly-Supervised Labeling

- ▶ Infer labels based on human-designed labeling functions.
- ▶ The function is often some heuristic rules and varies for different data types
  - ▶ For example, using seed words for text classification
    - ▶ Texts containing happy, joy, delighted, etc. are labeled as happy in emotion detection.
- ▶ Minimum or partial human participation
  - ▶ Provide the heuristics
- ▶ Can be used to quickly generate a large number of labels.

# Distant Supervision

- ▶ A weakly-supervised approach
- ▶ Assign labels by leveraging external sources
  - ▶ For example, for relation extraction from two entities in text documents:
    - ▶ Semantic relationships between entities are labeled based on external data, such as Freebase.
- ▶ Often does not require human participation (automatic)
- ▶ Generated data can be noisy

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# Data Transformation

- ▶ Transform the data into an appropriate form for mining
- ▶ Attribute/feature construction
  - ▶ New attributes constructed from the given ones
    - ▶ e.g., compute average sale amount using total sale amount divided by units sold.
- ▶ Normalization: scale attribute values to fall within a small, specified range
  - ▶ min-max normalization
  - ▶ z-score normalization
  - ▶ normalization by decimal scaling
- ▶ Converting discrete attributes to numeric ones
- ▶ Discretization
  - ▶ Transform numeric attributes into discrete attributes

# Data Transformation: Normalization

## ▶ min-max normalization

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new\_max}_A - \text{new\_min}_A) + \text{new\_min}_A$$

where  $\min_A$  and  $\max_A$  are the minimum and maximum values of attribute  $A$ , and  $[\text{new\_min}_A, \text{new\_max}_A]$  is the new range

## ▶ Example: Attribute *income* has values

- ▶ \$12,000, \$20,000, \$25,000, \$30,000, \$45,000, \$60,000, \$73,600, \$98,000

- ▶ normalized into values in range  $[0, 1]$ :

0, 0.093, 0.151, 0.209, 0.384, 0.558, 0.716, 1

## ▶ Problems:

- ▶ “Out of bounds” error occurs if a future input case falls outside the original range for  $A$
- ▶ A too big or too small value could be noise. If they are used as min or max value for normalization, the results are not reliable.

# Data Transformation: Normalization (Contd.)

- ▶ z-score normalization

$$v' = \frac{v - mean_A}{s_A}$$

where  $mean_A$  is the mean of attribute  $A$  and  $s_A$  is the standard deviation of  $A$  (suppose values are :  $v_1, v_2, \dots, v_n$ ):

$$s_A = \sqrt{\frac{1}{n} \sum_{i=1}^n (v_i - mean_A)^2} \quad \text{or} \quad s_A = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (v_i - mean_A)^2}$$

- ▶ Example:

- ▶ Attribute income has values:

- ▶ \$12,000, \$20,000, \$25,000, \$30,000, \$45,000, \$60,000, \$73,600, \$98,000

- ▶ The mean and standard deviation of the attribute *income* are

- ▶ \$45,450 and \$29,735, respectively

- ▶ With z-score normalization, the values are transformed into:

- ▶ -1.12, -0.86, -0.69, -0.52, -0.02, 0.49, 0.95, 1.77

- ▶ Advantages:

- ▶ useful when the actual min and max are unknown

- ▶ better deal with outliers than min-max normalization

# Data Transformation: Normalization (Contd.)

- ▶ Normalization by decimal scaling

$$v_i' = \frac{v_i}{10^k}$$

where  $k$  is the smallest integer such that  $\text{Max}(|v_i'|) \leq 1$

- ▶ Example:

- ▶ Suppose the recorded values of A range from -986 to 97
- ▶ The maximum absolute value of A is 986.
- ▶ Then  $k = 3$
- ▶ -986 is normalized to -0.986 and 97 is normalized to 0.097



# Data Transformation

- ▶ Transform the data into an appropriate form for mining
- ▶ Attribute/feature construction
  - ▶ New attributes constructed from the given ones
    - ▶ e.g., compute average sale amount using total sale amount divided by units sold.
- ▶ Normalization: scale attribute values to fall within a small, specified range
  - ▶ min-max normalization
  - ▶ z-score normalization
  - ▶ normalization by decimal scaling
- ▶ Converting discrete attributes to numeric ones
- ▶ Discretization
  - ▶ Transform numeric attributes into discrete attributes

# Converting Discrete Attributes to Numeric Ones

- ▶ Some popular machine learning algorithms (such as neural networks, SVM, etc.) cannot deal with categorical attributes
- ▶ Method 1:
  - ▶ Convert a nominal attribute to a numerical attribute, for example:
    - ▶  $\text{yes} \Rightarrow 1$  and  $\text{no} \Rightarrow 0$
    - ▶  $\text{blue} \Rightarrow 1$ ,  $\text{yellow} \Rightarrow 2$ ,  $\text{red} \Rightarrow 3$ , etc.
  - ▶ Problem?
- ▶ Method 2:
  - ▶ Convert a nominal attribute to  $n$  binary attributes, where  $n$  is the number of values of the nominal attribute
    - ▶ For example, if the domain of attribute A is  $\{a_1, a_2, a_3\}$ , we convert it into 3 attributes to represent A.
      - ▶ If  $A=a_1$ , the values of the three attribute are 1, 0, 0.
      - ▶ If  $A=a_2$ , the values of the three attribute are 0, 1, 0.
      - ▶ If  $A=a_3$ , the values of the three attribute are 0, 0, 1.

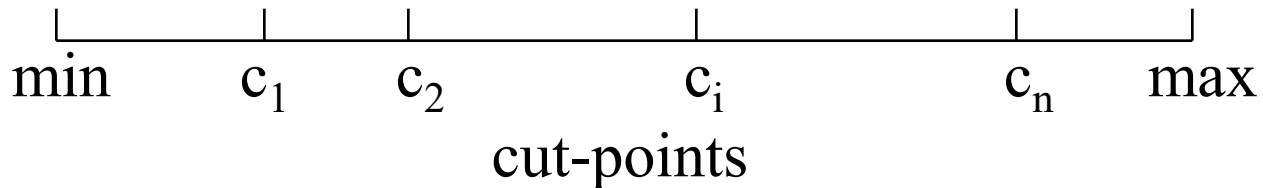
*One-hot  
encoding*

# Outline

- ▶ Why preprocess the data?
- ▶ Data integration
- ▶ Data cleaning
- ▶ Data reduction
- ▶ Data augmentation
- ▶ Data labeling
- ▶ Data transformation
- ▶ Discretization

# What is Discretization?

- ▶ A discretization algorithm
  - ▶ converts continuous attributes into discrete attributes by partitioning the range of a continuous attribute into intervals.
  - ▶ Interval labels can then be used to replace actual data values.

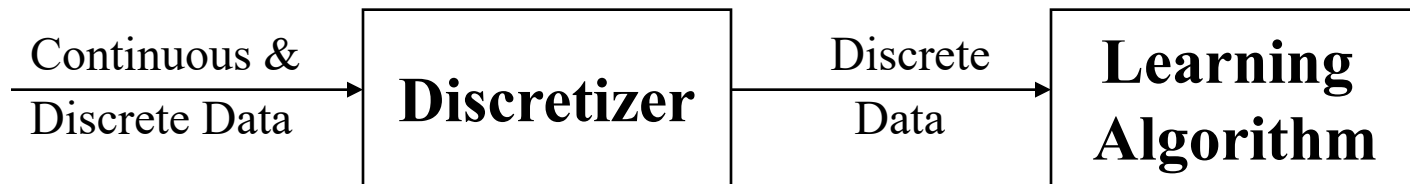


# Why Need Discretization?

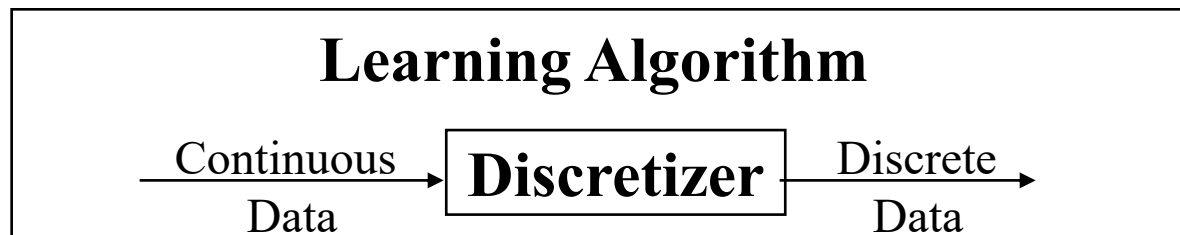
- ▶ Some learning algorithms are limited to discrete inputs.
- ▶ Efficiency: handling (lots of) continuous values tends to slow down learning considerably. (*Unique Value reduction*)
- ▶ Accuracy: in the presence of noise good discretization can sometimes improve predictive accuracy. (*Smoothing out noise*)
- ▶ Intelligibility: discretization may lead to smaller sizes of induced trees or rule sets.

# Two Architectures

- ▶ Discretization before learning starts (Static discretization)

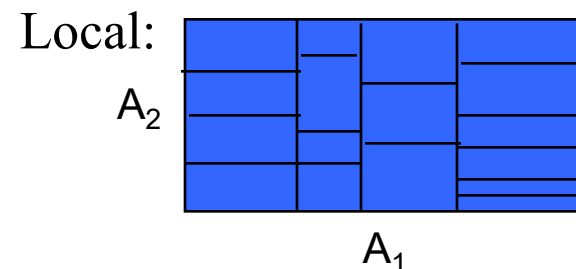
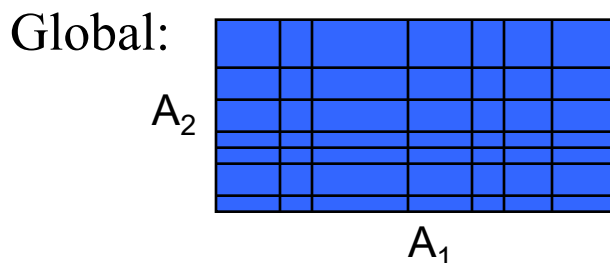


- ▶ Discretization during the learning process (Dynamic discretization)



# Classification of Discretization Methods

- ▶ Supervised vs. unsupervised.
  - ▶ Supervised discretization uses class information.
  - ▶ Unsupervised does not use class labels.
- ▶ Bottom-up vs. top-down
  - ▶ Bottom-up: start from intervals with one value each and repeatedly merge intervals until some stopping criterion is satisfied.
  - ▶ Top-down: start from one interval with all values and repeatedly split intervals until some stopping criterion is satisfied.
- ▶ Global vs. local
  - ▶ Global: an attribute is partitioned over the entire continuous range, using global information and independent of other attributes.
  - ▶ Local: partition is applied to local regions of an attribute range.



# Unsupervised Discretization

## ▶ Equal-width binning

- ▶ It divides the range of an attribute into  $k$  intervals of equal size: uniform grid
- ▶ If  $A$  and  $B$  are the lowest and highest values of the attribute, the width of intervals will be:  $W = (B - A)/k$ .

## ▶ Equal-depth/frequency binning

- ▶ It divides the range into  $k$  intervals, each containing approximately the same number of values

## ▶ k-means clustering

- ▶ Given  $k$  bins, distribute the values in the bins to minimize the average distance of a value from its bin mean.



# Equal-width Binning

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- Partition into (equi-width) bins: 3 intervals of equal size
  - Bin 1: 4, 8, 9
  - Bin 2: 15, 21, 21, 24
  - Bin 3: 25, 26, 28, 29, 34
- Pros and Cons
  - ▶ The most straightforward
  - ▶ But outliers may dominate presentation
  - ▶ Skewed data is not handled well.

# Equal-depth Binning

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34 (12 points in total)
- Partition into 3 (equi-depth) bins:
  - Bin 1: 4, 8, 9, 15
  - Bin 2: 21, 21, 24, 25
  - Bin 3: 26, 28, 29, 34
- ⑩ Good data scaling; better handle skewed data

# K-mean Clustering

- ▶ Input: (1) a set of values for an attribute  
(2)  $k$  = number of bins
- ▶ Sort the input values and keep the unique values
- ▶ Create  $k$  bins using equal-depth binning
- ▶ Compute bin means ( $mean_1, mean_2, \dots, mean_k$ )
- ▶ Compute global distance:  $D_{new} = \sum_i \sum_j (v_{ij} - mean_i)^2$   
where  $mean_i$  is the mean in  $bin_i$  and  $v_{ij}$  is the  $j$ th value in  $bin_i$ .
- ▶ Repeat
  - ▶  $D_{old} = D_{new}$
  - ▶ for each  $bin_i$ 
    - ▶ for each  $v_{ij}$  in  $bin_i$ 
      - ▶ If  $(|v_{ij} - mean_{i-1}|) < (|v_{ij} - mean_i|)$ , move  $v_{ij}$  to  $bin_{i-1}$ .
      - ▶ If  $(|v_{ij} - mean_{i+1}|) < (|v_{ij} - mean_i|)$ , move  $v_{ij}$  to  $bin_{i+1}$ .
    - ▶ Compute new bin means and  $D_{new}$
- ▶ Until  $D_{new}$  is not less than  $D_{old}$ .

# Supervised Discretization

## ▶ ChiMerge

- ▶ Based on chi-square test
- ▶ Entropy-based discretization method
  - ▶ Based on an entropy minimization heuristic

# Entropy-Based Discretization

- ▶ Supervised, top-down discretization
- ▶ Employs an entropy minimization heuristic for splitting the range of a continuous attribute.
- ▶ Given a set  $S$  of examples and  $k$  classes, the *entropy* of  $S$  with respect to the  $k$  classes is defined as:

$$Ent(S) = -\sum_{i=1}^k P(C_i) \log_2(P(C_i))$$

where  $P(C_i)$  is the probability of examples in  $S$  that belong to  $C_i$ .

- ▶ The bigger  $Ent(S)$  is, the more impure  $S$  is.

# Entropy-Based Discretization

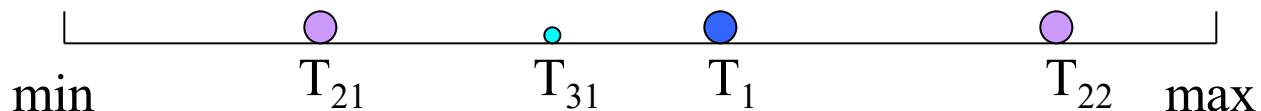
Given an attribute  $A$  and a set  $S$  of training examples:

- ▶ Sort the examples in a set  $S$  by increasing values of the attribute  $A$ :  $\{v_1, v_2, \dots, v_n\}$ .
- ▶ A potential cut-point  $T$ : midpoint between  $v_i$  and  $v_{i+1}$  dividing  $S$  into  $S_1$ :  $\{v_1, v_2, \dots, v_i\}$  and  $S_2$ :  $\{v_{i+1}, \dots, v_n\}$ .
- ▶ A total of  $n-1$  potential cut-points.
- ▶ Suppose a cut-point  $T$  partitions  $S$  into  $S_1$  and  $S_2$ . Entropy (with respect to the class attribute) after the partition induced by cutpoint  $T$ :

$$Ent(T, S) = \frac{|S_1|}{|S|} Ent(S_1) + \frac{|S_2|}{|S|} Ent(S_2)$$

where  $|S|$ ,  $|S_1|$  and  $|S_2|$  = # of examples in  $S$ ,  $S_1$  and  $S_2$

- ▶ Select  $T_A$  for which  $E(T_A, S)$  is **minimal** to split the range into two subranges
- ▶ The process is recursively applied to partitions obtained until some stopping criterion is met.



# Stopping Criteria for Entropy-Based Discretization

## ▶ Stopping criteria in D-2 (Catlett, 1991):

Recursive partitioning stops if any of the following is satisfied:

- ▶ all the examples in the interval belong to the same class.
- ▶ number of examples in an interval is below a given level;
- ▶ maximum number of cut-points for an attribute is reached;
- ▶ the entropy reduction on all possible cut-points is equal;

## ▶ Stopping criterion based on Minimum Description Length Principle (MDLP) (Fayyad and Irani, 1993):

Recursive partitioning stops iff

$$Ent(S) - Ent(T, S) \leq \frac{\log_2(N-1)}{N} + \frac{\Delta(T; S)}{N}$$

$$\Delta(T; S) = \log_2(3^k - 2) - [kEnt(S) - k_1Ent(S_1) - k_2Ent(S_2)]$$

where  $T$  is the cut point leading to the smallest  $Ent(T, S)$ ,  $k$ ,  $k_1$  and  $k_2$  are the number of classes in  $S$ ,  $S_1$  and  $S_2$ , respectively, and  $N$  is the number of examples in  $S$ .

# Summary

- ▶ Data preparation is a big issue for data mining
- ▶ Data preparation includes
  - ▶ Data integration
  - ▶ Data cleaning
    - ▶ Handle missing values
    - ▶ Detect and remove noise
  - ▶ Data reduction
  - ▶ Data augmentation
  - ▶ Data labeling
  - ▶ Data transformation
  - ▶ Discretization
- ▶ A lot of methods have been developed but still an active area of research



# Bayesian Classification

---

Aijun An

Department of Electrical Engineering and Computer Science  
York University

---

# Outline

- ▶ Introduction
- ▶ Bayes Theorem
- ▶ Finding MAP hypothesis
- ▶ Naive Bayes Classifier
- ▶ Bayesian belief networks

# Introduction

- ▶ Goal:
  - ▶ Determine the most probable hypothesis (class)
  - ▶ E.g, Given new instance  $x$ , what is its most probable classification?
- ▶ Probabilistic learning and prediction:
  - ▶ Estimate explicit probabilities for all hypotheses (classes)
  - ▶ Predict multiple hypotheses, weighted by their probabilities
  - ▶ Can combine prior knowledge (such as prior probabilities, probability distributions, causal relationships between variables in belief networks) with observed data

---

## Introduction (*Cont'd*)

- ▶ Incremental learning:
  - ▶ Each training example can incrementally increase/decrease the probability that a hypothesis is correct.
- ▶ Flexible in handling inconsistency
- ▶ Standard:
  - ▶ provides a standard of optimal decision making against which other methods can be measured

# Bayes Theorem

$$P(h | x) = \frac{P(x | h)P(h)}{P(x)}$$

- ▶  $P(h)$  = prior probability of hypothesis  $h$
- ▶  $P(x)$  = probability that example  $x$  is observed
- ▶  $P(h | x)$  = posterior probability of  $h$  given  $x$
- ▶  $P(x | h)$  = conditional probability of  $x$  given  $h$  (often called the likelihood of  $h$  given  $x$ )

# Finding Maximum a Posteriori Hypothesis

$$P(h | x) = \frac{P(x | h)P(h)}{P(x)}$$

- ▶ Goal: Find the most probable hypothesis  $h$  from a set  $H$  of candidate hypotheses given an example  $x$ .
- ▶ The most probable hypothesis is called *maximum a posteriori (MAP)* hypothesis  $h_{MAP}$ :

$$h_{MAP}(x) = \arg \max_{h \in H} P(h | x)$$

$$= \arg \max_{h \in H} \frac{P(x | h)P(h)}{P(x)} \quad (P(x) \text{ is constant for all hypotheses})$$

$$= \arg \max_{h \in H} P(x | h)P(h)$$

- ▶ If assume  $P(h_i) = P(h_j)$  (classes are equally likely), then can further simplify, and choose the *Maximum likelihood* (ML) hypothesis

$$h_{ML}(x) = \arg \max_{h \in H} P(x | h)$$

# Example

- ▶ Does patient have cancer or not?
  - ▶ A patient takes a lab test and the result comes back positive.
  - ▶ The test returns a correct positive result in only 98% of the cases in which the disease is actually present,
  - ▶ The test returns a correct negative result in only 97% of the cases in which the disease is not present.
  - ▶ Furthermore, .008 of the entire population have this cancer.

$$P(cancer) = \quad P(\neg cancer) =$$

$$P(+ | cancer) = \quad P(- | cancer) =$$

$$P(+ | \neg cancer) = \quad P(- | \neg cancer) =$$

Our goal is to find the maximum between:

$$P(cancer | +) \text{ and } P(\neg cancer | +)$$

# Learning Probabilities from Data

- Suppose we do not know the probabilities used in the example in the last slide.
- But we are given a set of data.
- In order to conduct the reasoning, i.e., to find the MAP hypothesis  $h_{MAP}$ , we can estimate the probabilities used in the reasoning from the data.



# Learning Probabilities from Data

- Suppose there are  $k$  possible hypotheses (i.e., classes):  
$$h_1, h_2, \dots, h_k$$

- In order to find:

$$h_{MAP}(x) = \arg \max_{h_i \in H} P(x | h_i) P(h_i)$$

- We need to estimate:
  - ▶  $P(h_1), P(h_2), \dots, P(h_k),$
  - ▶  $P(x|h_1), P(x|h_2), \dots, P(x|h_k)$  for each possible instance  $x$ ,

# Practical Problem with Finding MAP Hypothesis

- ▶ Suppose instance  $x$  is described by attributes values  $\langle x_1, x_2, \dots, x_n \rangle$  and there is a set  $C$  of classes:  $c_1, c_2, \dots, c_k$ .

$$\begin{aligned} c_{MAP}(x) &= \arg \max_{c_j \in C} P(c_j | x_1, x_2, \dots, x_n) \\ &= \arg \max_{c_j \in C} \frac{P(x_1, x_2, \dots, x_n | c_j) P(c_j)}{P(x_1, x_2, \dots, x_n)} \\ &= \arg \max_{c_j \in C} P(x_1, x_2, \dots, x_n | c_j) P(c_j) \end{aligned}$$

- ▶ Given data set with many attributes, it is infeasible to estimate  $P(x_1, x_2, \dots, x_n | c_j)$  for all possible  $x$  values unless we have a very, very large set of training data. It is also computationally expensive.

# Naive Bayes Classifier

- ▶ Naive assumption: values of attributes are conditionally independent given a class

$$P(x_1, x_2, \dots, x_n | c_j) = \prod_i P(x_i | c_j)$$

which gives:

$$\begin{aligned} c_{NB}(x) &= \arg \max_{c_j \in C} P(x_1, x_2, \dots, x_n | c_j) P(c_j) \\ &= \arg \max_{c_j \in C} P(c_j) \prod_i P(x_i | c_j) \end{aligned}$$

- ▶ Probabilities can be estimated from the training data.

# Estimating Probabilities

- ▶ Estimate  $P(c_j)$ :

$$P(c_j) = \frac{\text{\# of training examples of class } c_j}{\text{\# of training examples}}$$

- ▶ Estimate  $P(x_i|c_j)$  for each attribute value  $x_i$  of attribute  $A_i$  and each class  $c_j$ 
  - ▶ If attribute  $A_i$  is categorical,

$$P(x_i | c_j) = \frac{\text{\# of training examples of class } c_j \text{ with } x_i \text{ for } A_i}{\text{\# of training examples of class } c_j}$$

# Estimating Probabilities

- ▶ If attribute  $A_i$  is continuous, can assume a distribution, such as normal distribution,

$$P(x_i | c_j) = \frac{1}{\sqrt{2\pi}\sigma_{c_j}} e^{-\frac{(x_i - \mu_{c_j})^2}{2\sigma_{c_j}^2}}$$

where  $\mu_{c_j}$  and  $\sigma_{c_j}$  are the mean and standard deviation of the values of  $A_i$  for training examples of class  $c_j$

$$\sigma_{c_j} = \sqrt{\frac{1}{n-1} \sum_{x_i \in c_j} (x_i - \mu_{c_j})^2}$$

# Naive Bayes Algorithm

- ▶ Naive Bayes Learning (*from examples*)

- ▶ For each class  $c_j$

$$\hat{P}(c_j) \leftarrow \text{estimate } P(c_j)$$

- ▶ For each attribute for which  $x_i$  is a value

$$\hat{P}(x_i | c_j) \leftarrow \text{estimate } P(x_i | c_j)$$

- ▶ Classifying new instance ( $x$ )

$$c_{NB}(x) = \arg \max_{c_j \in C} \hat{P}(c_j) \prod_{x_i \in x} \hat{P}(x_i | c_j)$$

# Example

## Training dataset

Classes:

c1:buys\_computer='yes'

c2:buys\_computer='no'

Classify new example:

X = (age ≤ 30,  
Income = medium,  
Student = yes  
Credit\_rating = Fair)

| age     | income | student | credit_rating | buys_computer |
|---------|--------|---------|---------------|---------------|
| ≤30     | high   | no      | fair          | no            |
| ≤30     | high   | no      | excellent     | no            |
| 30...40 | high   | no      | fair          | yes           |
| >40     | medium | no      | fair          | yes           |
| >40     | low    | yes     | fair          | yes           |
| >40     | low    | yes     | excellent     | no            |
| 31...40 | low    | yes     | excellent     | yes           |
| ≤30     | medium | no      | fair          | no            |
| ≤30     | low    | yes     | fair          | yes           |
| >40     | medium | yes     | fair          | yes           |
| ≤30     | medium | yes     | excellent     | yes           |
| 31...40 | medium | no      | excellent     | yes           |
| 31...40 | high   | yes     | fair          | yes           |
| >40     | medium | no      | excellent     | no            |

## Example (*cont'd*)

### ► Learning:

#### ► Compute $P(c_i)$

$$P(\text{buy\_computer}=\text{"yes"})=9/14$$

$$P(\text{buy\_computer}=\text{"no"})=5/14$$

#### ► Compute $P(x_j|c_i)$ for each class and each attribute value pair:

$$P(\text{age} \leq 30 \mid \text{buys\_computer}=\text{"yes"}) = 2/9 = 0.222$$

$$P(\text{age} \leq 30 \mid \text{buys\_computer}=\text{"no"}) = 3/5 = 0.6$$

.....

$$P(\text{income}=\text{"medium"} \mid \text{buys\_computer}=\text{"yes"}) = 4/9 = 0.444$$

$$P(\text{income}=\text{"medium"} \mid \text{buys\_computer}=\text{"no"}) = 2/5 = 0.4$$

.....

$$P(\text{student}=\text{"yes"} \mid \text{buys\_computer}=\text{"yes"}) = 6/9 = 0.667$$

$$P(\text{student}=\text{"yes"} \mid \text{buys\_computer}=\text{"no"}) = 1/5 = 0.2$$

.....

$$P(\text{credit\_rating}=\text{"fair"} \mid \text{buys\_computer}=\text{"yes"}) = 6/9 = 0.667$$

$$P(\text{credit\_rating}=\text{"fair"} \mid \text{buys\_computer}=\text{"no"}) = 2/5 = 0.4$$

.....



## Example (*cont'd*)

- Classification: to classify:

**$x=(age \leq 30, income = medium, student=yes, credit\_rating=fair)$**

**$P(x|c_i)$  :**

$$\begin{aligned} & P(x|buys\_computer="yes") \\ &= P(age \leq 30|buys\_computer=yes) \times P(income=medium|buys\_computer=yes) \times \\ & \quad P(student=yes|buys\_computer=yes) \times P(credit=fair|buys\_computer=yes) \\ &= 0.222 \times 0.444 \times 0.667 \times 0.0667 \\ &= 0.044 \end{aligned}$$

$$P(x|buys\_computer="no") = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.019$$

**$P(c_i|x) \propto P(x|c_i) * P(c_i)$  :**

$$P(buys\_computer="yes"|x) \propto$$

$$P(x|buys\_computer="yes") * P(buys\_computer="yes") = 0.028$$

$$P(buys\_computer="no"|x) \propto$$

$$P(x|buys\_computer="no") * P(buys\_computer="no") = 0.007$$

**$x$  belongs to class " $buys\_computer=yes$ "**

# Naïve Bayesian Classifier: Comments

- ▶ Advantages :

- ▶ Easy to implement
- ▶ Good results were obtained in many applications

- ▶ Disadvantage

- ▶ Assumption: class conditional independence of attributes, therefore loss of accuracy
- ▶ Practically, dependencies exist among attributes
  - ▶ For example, *headache* and *body temperature* are dependent attributes for *flu* dataset.
- ▶ Dependencies among these cannot be modeled by Naïve Bayesian Classifier

- ▶ How to deal with these dependencies?

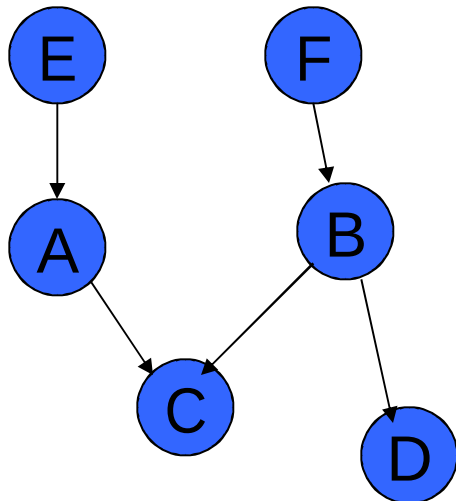
- ▶ Bayesian Belief Networks

# Bayesian Belief Networks

- ▶ Naive Bayes assumption of conditional independence is too restrictive.
- ▶ But it's intractable without such assumptions...
- ▶ Bayesian Belief networks provide an intermediate approach which
  - ▶ allows dependencies among attributes
  - ▶ but assumes conditional independence among subsets of attributes.

# Bayesian Belief Networks

- ▶ A graphical model of causal relationships. Two components:
  - ▶ A *directed acyclic graph (DAG)*: represents dependency among variables (attributes)

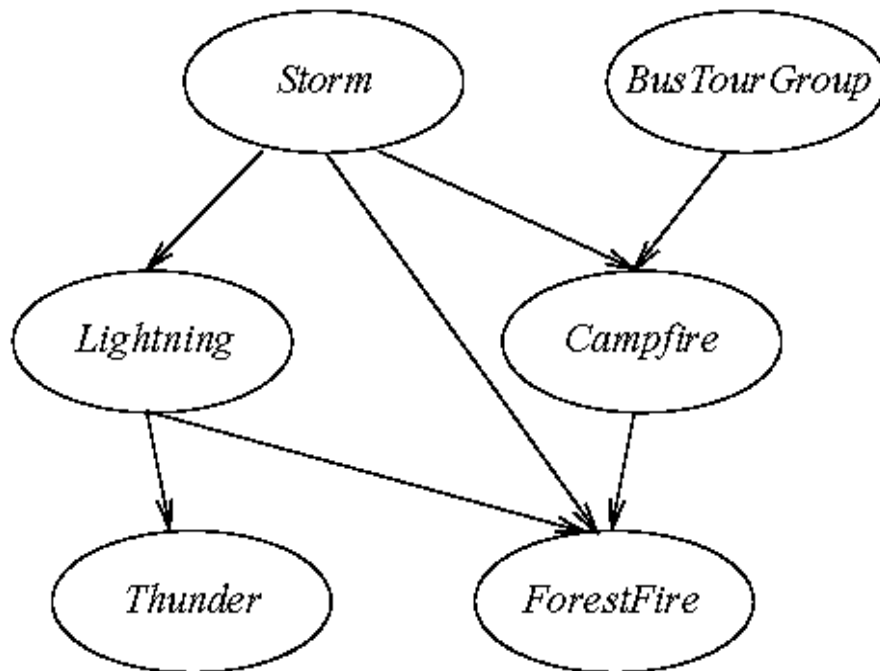


- **Nodes:** variables (including class attribute)
- **Links:** dependencies (e.g., A depends on E)
- **Parents:** immediate predecessors. E.g., A,B are the parents of C. B is the parent of D
- **Descendant:** X is a descendant of Y if there is a direct path from Y to X.
- **Conditional Independency:**
  - Assume: each variable is conditionally independent of its nondescendants given its parents.
  - Definition: X is conditionally independent of Y given Z iff  $P(X|Y,Z)=P(X|Z)$
  - E.g.: C is conditional independent of D given A and B. Thus,  $P(C|A, B, D)=P(C|A, B)$
- **Acyclic:** has no loops or cycles

- ▶ A *conditional probability table (CPT)* for each variable X: specifies the conditional probability distribution  $P(X|\text{Parents}(X))$ .

# Example of CPT

- Suppose each variable is binary (contain two values:  $X$  and  $\neg X$ )



CPT table for *Campfire*

|          | $S, B$ | $S, \neg B$ | $\neg S, B$ | $\neg S, \neg B$ |
|----------|--------|-------------|-------------|------------------|
| $C$      | 0.4    | 0.1         | 0.8         | 0.2              |
| $\neg C$ | 0.6    | 0.9         | 0.2         | 0.8              |

$P(C|S, B)$

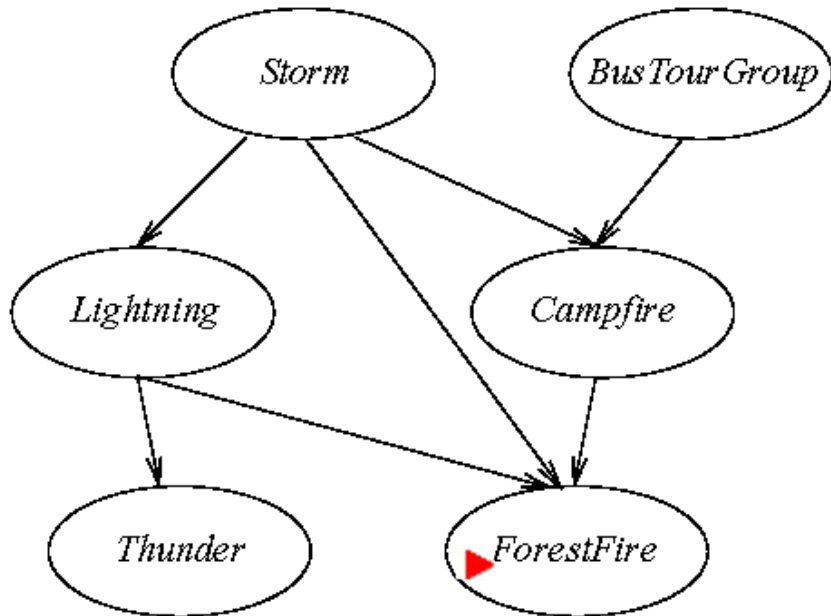


- There is a conditional probability table (CPT) for each variable

# Inference Rule in Bayesian Networks

- ▶ The joint probability of any tuple  $(x_1, \dots, x_n)$  corresponding to the variables or attributes  $(X_1, \dots, X_n)$  is computed by

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{Parents}(X_i))$$

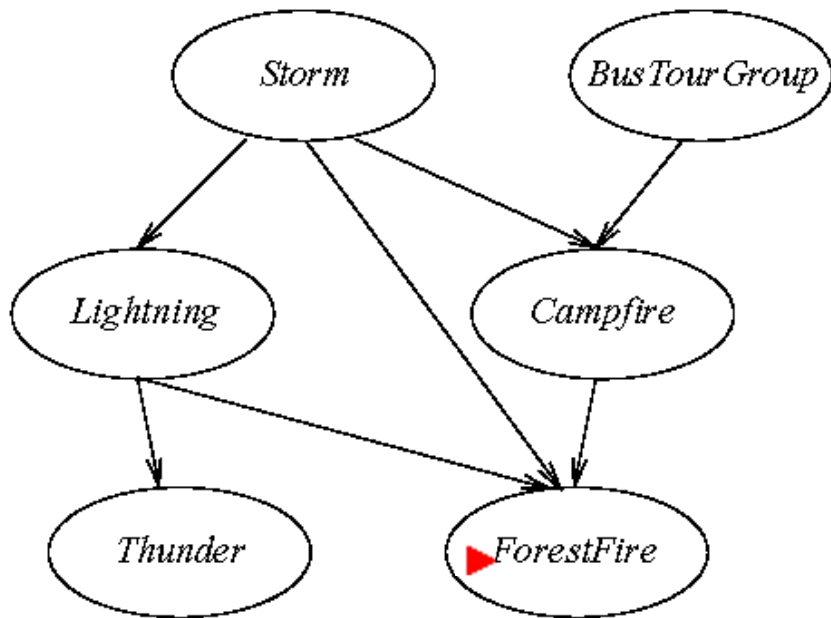


- ▶ Example:

$$P(\neg S, B, \neg L, C, \neg T, F) = P(\neg S) \times P(B) \times P(\neg L \mid \neg S) \times P(C \mid \neg S, B) \times P(\neg T \mid \neg L) \times P(F \mid \neg L, \neg S, C)$$

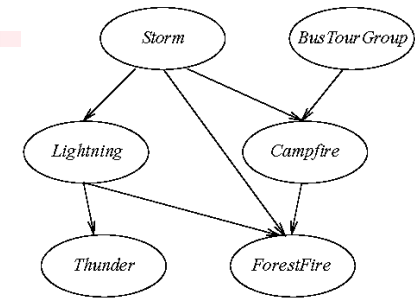
# Inference in Bayesian Networks

- ▶ A Bayesian network can be used to infer the (probabilities of) values of one or more network variables, given observed values of others.
- ▶ Example:
  - ▶ Given Storm=0, BusTourGroup=1, Lightning=0, Campfire=1, Thunder=0, we want to know ForestFire=?



- ▶ Compute two probabilities:
  - (1)  $P(F | \neg S, B, \neg L, C, \neg T) = P(F | \neg L, \neg S, C)$
  - (2)  $P(\neg F | \neg S, B, \neg L, C, \neg T) = P(\neg F | \neg L, \neg S, C)$
- ▶ ForestFire = True if (1) > (2)

# Inference in Bayesian Networks



## ▶ Another example:

- ▶ Given Storm=1, Campfire=0, ForestFire=1, what is the probability distribution of Thunder?
- ▶ Compute two probabilities:

$$(1) P(T | S, \neg C, F) = P(T, L | S, \neg C, F) + P(T, \neg L | S, \neg C, F)$$

$$= P(T | L, S, \neg C, F)P(L | S, \neg C, F) + P(T | \neg L, S, \neg C, F)P(\neg L | S, \neg C, F)$$

$$= P(T | L)P(L | S, \neg C, F) + P(T | \neg L)P(\neg L | S, \neg C, F)$$

$$\text{where } P(L | S, \neg C, F) = \frac{P(L, F | S, \neg C)}{P(F | S, \neg C)} = \frac{P(F | L, S, \neg C)P(L | S, \neg C)}{P(F, L | S, \neg C) + P(F, \neg L | S, \neg C)}$$

$$= \frac{P(F | L, S, \neg C)P(L | S)}{P(F, L | S, \neg C) + P(F, \neg L | S, \neg C)} = \frac{P(F | L, S, \neg C)P(L | S)}{P(F | L, S, \neg C)P(L | S, \neg C) + P(F | \neg L, S, \neg C)P(\neg L | S, \neg C)}$$

$$= \frac{P(F | L, S, \neg C)P(L | S)}{P(F | L, S, \neg C)P(L | S) + P(F | \neg L, S, \neg C)P(\neg L | S)}$$

$$\text{and similarly } P(\neg L | S, \neg C, F) = \frac{P(F | \neg L, S, \neg C)P(\neg L | S)}{P(F | L, S, \neg C)P(L | S) + P(F | \neg L, S, \neg C)P(\neg L | S)}$$

(2)  $P(\neg T | S, \neg C, F)$  can be calculated similarly.

- ▶ Thunder = True if (1) > (2)



# Learning of Bayesian Networks

- ▶ Several scenarios of this learning task
  - ▶ Network structure might be *known* or *unknown*.
  - ▶ Training examples might provide values of all network variables, or just *some*.
- ▶ Scenario 1: If structure known and observe all variables:
  - ▶ Then it's easy as training a Naive Bayes classifier.
  - ▶ Learn only CPTs (estimate the conditional probabilities from training data)

# Learning of Bayesian Networks

- ▶ Scenario 2: Suppose structure known, variables partially observable
  - ▶ For example, observe *ForestFire*, *Storm*, *BusTourGroup*, *Thunder*, but not *Lightning*, *Campfire*...
  - ▶ Similar to training neural network with hidden units. In fact, can learn network conditional probability tables using *gradient ascent* method!
- ▶ Scenario 3: When structure unknown
  - ▶ Use heuristic search or constraint-based technique to search through potential structures.
  - ▶ K2 algorithm

# Summary: Bayesian Belief Networks

- ▶ Combine prior knowledge with observed data
- ▶ Intermediate approach that allows both dependencies and conditional independencies
- ▶ Other issues
  - ▶ Extend from categorical to real-valued variables
  - ▶ Parameterized distributions instead of tables
  - ▶ More effective inference and learning methods
  - ▶ ...

# Introduction to Neural Networks



Aijun An

Department of Electrical Engineering and Computer Science

York University



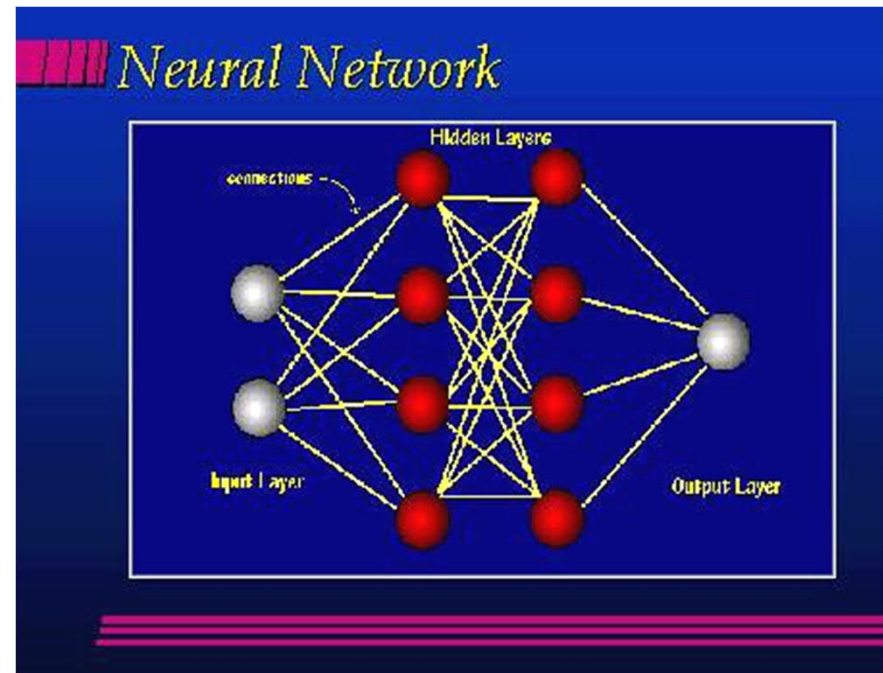
---

# Outline

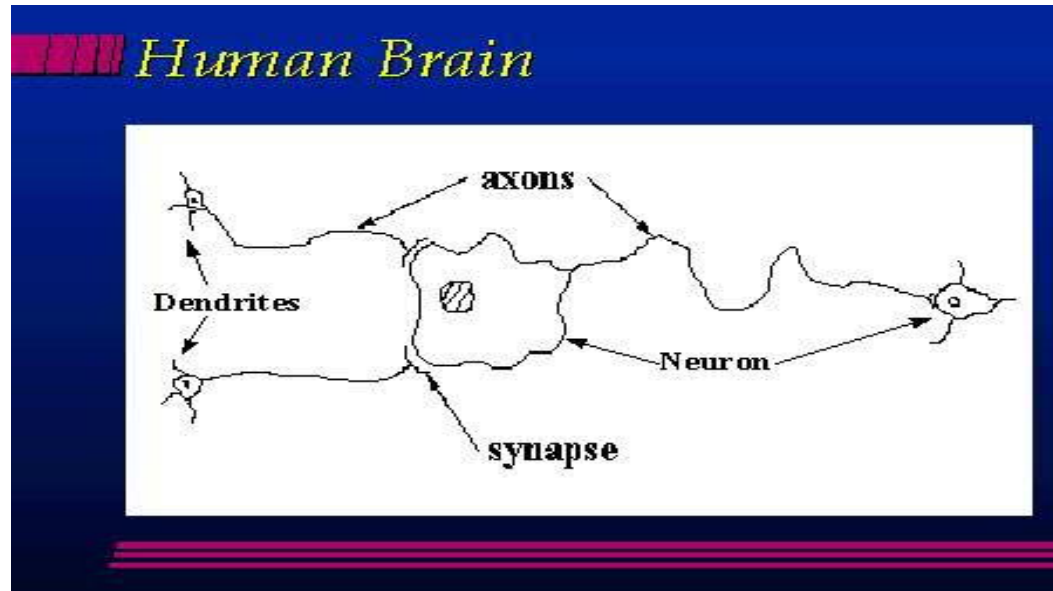
- ▶ Introduction
- ▶ Types of general neural network structures
- ▶ What happens inside a neuron?
- ▶ Activation Functions
- ▶ Output and Input Design
- ▶ How to train a neural network
  - ▶ Backpropagation algorithm
- ▶ How to prevent overfitting
- ▶ Applications and history of neural networks
- ▶ Brief introduction to deep learning

# What is an Artificial Neural Network?

- ▶ It is a formalism for representing functions inspired from biological systems
- ▶ It consists of layers of interconnected computing units
  - ▶ Each unit computes a simple function.
  - ▶ The inputs of the units in one layer are typically the outputs of the units in the previous layer.
  - ▶ Each connection is associated with a weight.
  - ▶ Parallel computing can be performed among the units in each layer

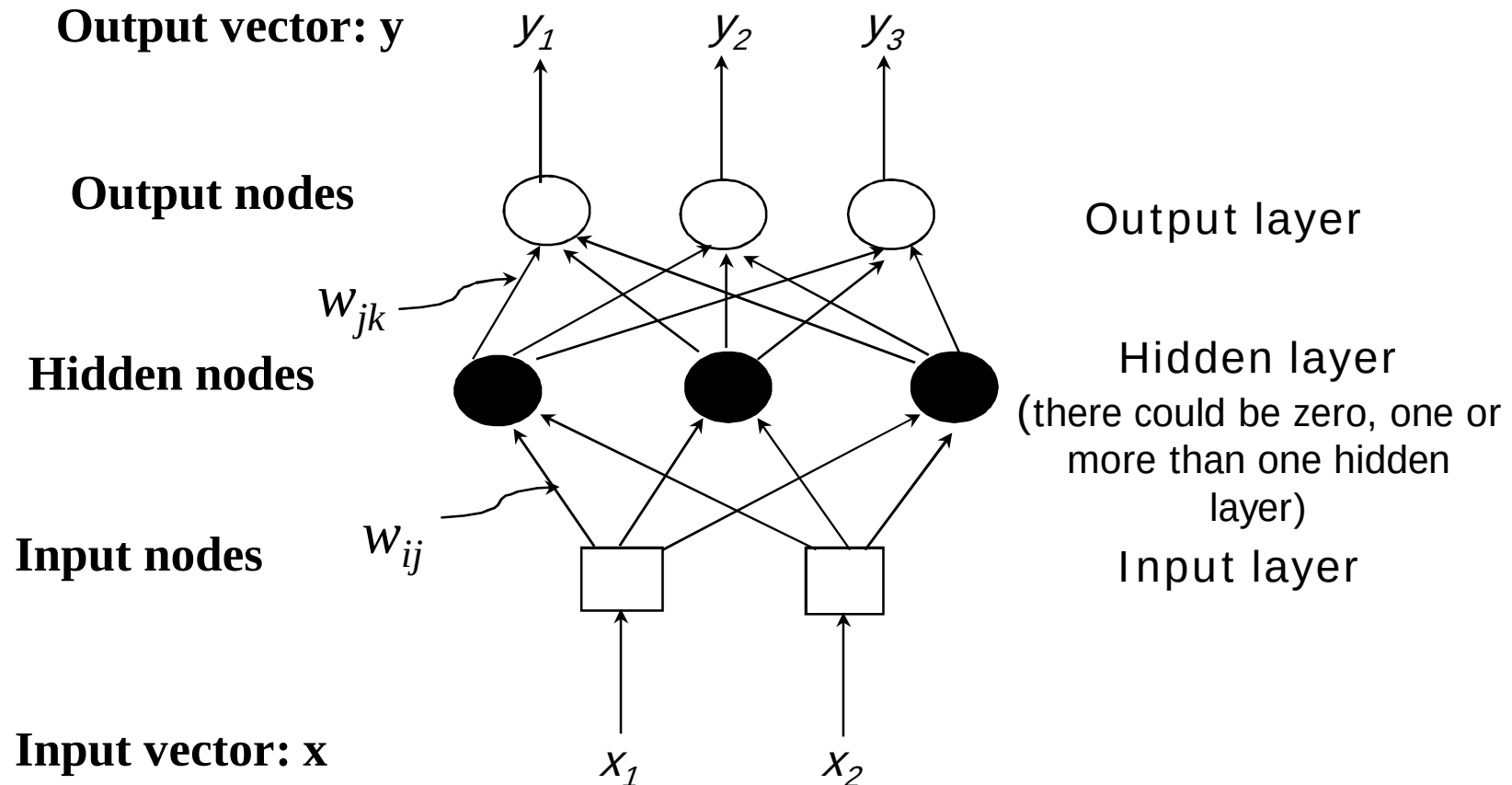


# Biological Motivation



- ▶ Biological Learning Systems are built of very complex webs of interconnected neurons.
  - ▶ Number of neurons:  $10^{10}$
  - ▶ Connections per neuron:  $10^4$  --  $10^5$
- ▶ Information-Processing abilities of biological neural systems must follow from highly *parallel processes* operating on representations that are distributed over many neurons
  - ▶ Scene recognition time: 0.1 second
- ▶ ANNs attempt to capture this mode of computation

# An Example of Neural Networks



- ▶ Each hidden or output node or unit is called a neuron
- ▶ Each connection is associated with a weight
- ▶ The network is usually **fully connected**, unless otherwise specified.
  - ▶ Each unit provides an input to each unit in the next layer

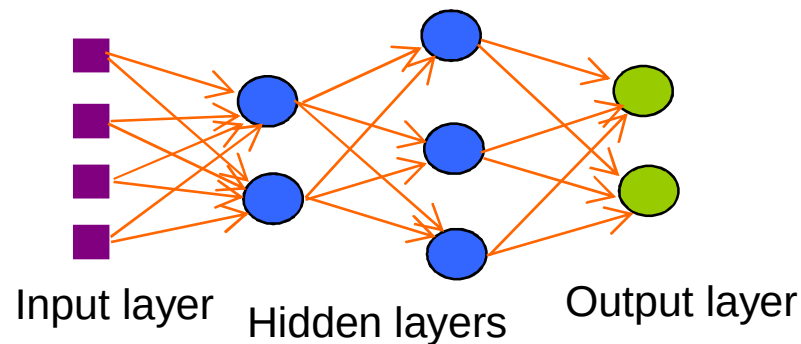


# Types of Neural Networks

- ▶ *Feed-forward* neural networks

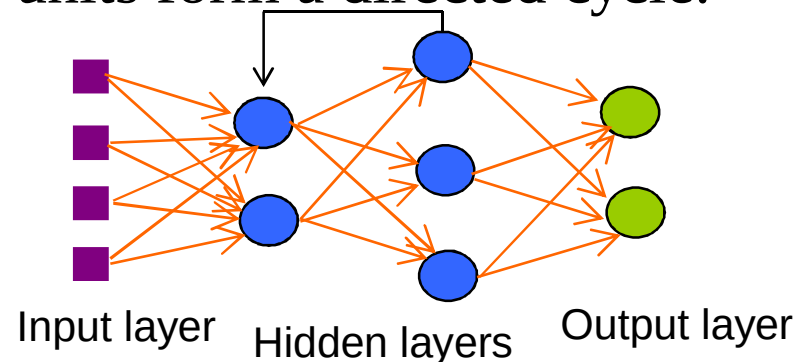
- ▶ None of the connections cycles back to an input unit or a unit of a previous layer.

- ▶ An example:



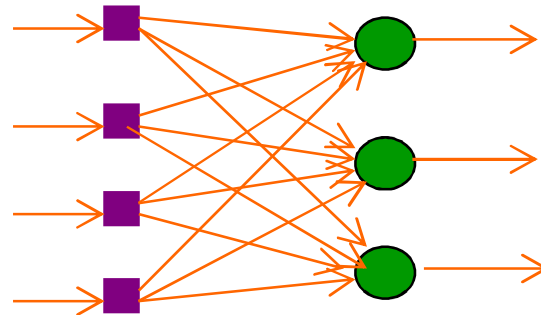
- ▶ *Recurrent* neural networks (*Feedback* neural network)

- ▶ Connections between units form a directed cycle.



# Single-layer Feed-Forward Neural Network

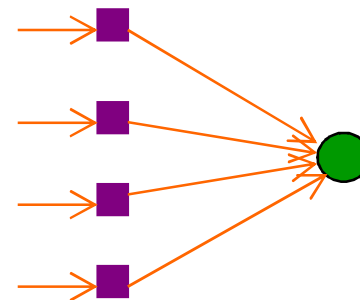
- ▶ A single-layer network does not have hidden layers



Input layer    Output layer

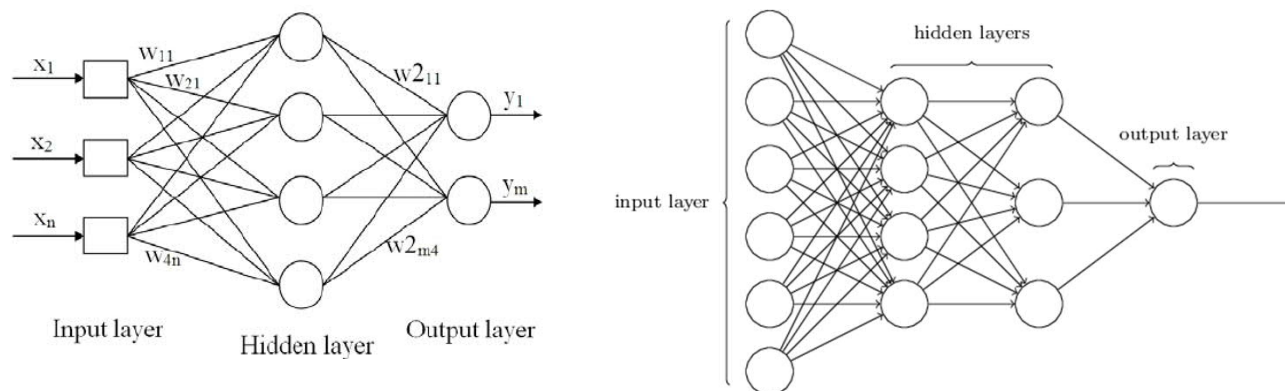
- ▶ The input layer is not counted as a “layer” because no computation is performed there.
- ▶ Perceptron
  - ▶ Contain only one computing node

If the *sigmoid* activation function is used in the perceptron, it is the **logistic regression** model



# Multi-layer Feed-Forward Neural Network

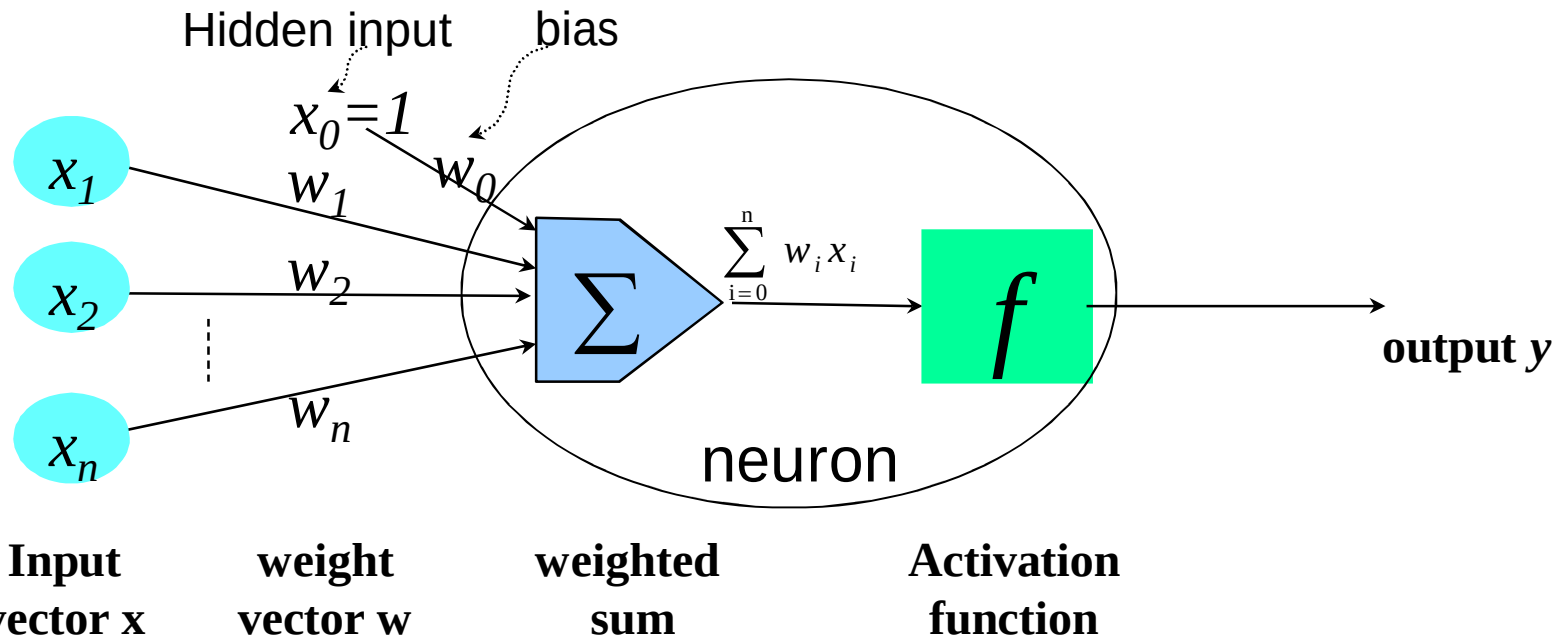
- ▶ A multilayer network contains one or more hidden layers.
  - ▶ It overcomes the limitation of single-layer NN: single-layer NN cannot handle non-linearly separable learning tasks.



- ▶ If the network is *fully connected*, it is called **Multilayer Perceptron (MLP)**
- ▶ When the number of hidden layers  $> 1$ , it is considered as a **deep neural network (DNN)**.
  - ▶ A DNN may or may not be fully connected

# What Happens inside a Neuron?

- ▶ A neuron is the basic information processing unit of an NN



- ▶ A neuron first sums the weighted inputs and then apply an activation function
- ▶ *Activation function*
  - ▶ Often maps a large input domain onto a smaller range (if so, it is called *squashing function*)
  - ▶ Often nonlinear (if so, referred to as *nonlinearity*)

---

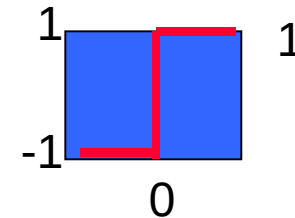
# Examples of Activation Functions

- ▶ Threshold logic function
- ▶ Sigmoid function
- ▶ Tanh function
- ▶ Relu function
- ▶ Leaky Relu
- ▶ ...

# Examples of Activation Functions

- ▶ Threshold logic function, such as,

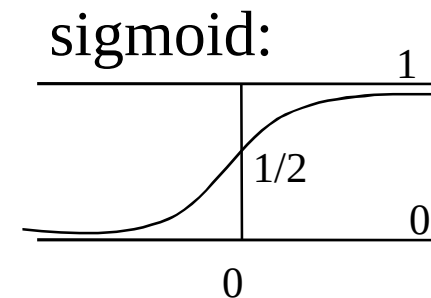
$$f(x) = \text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & \text{otherwise} \end{cases}$$



- ▶ not a smooth function
- ▶ not usually used in modern neural networks

- ▶ Sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$



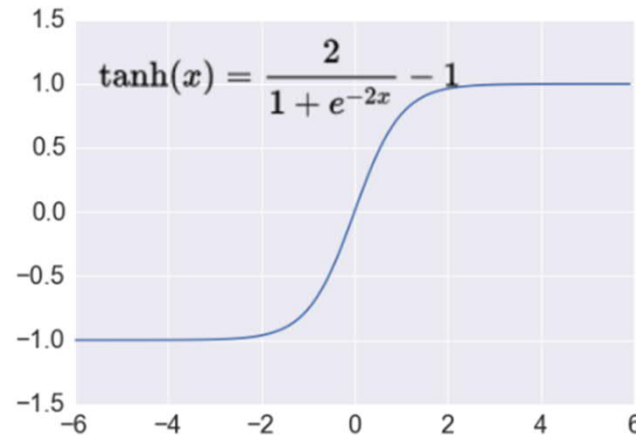
- ▶ differentiable:

$$f'(x) = f(x)(1 - f(x))$$

a good feature for the purpose of training neural networks

# Examples of Activation Functions

- ▶ Tanh function:

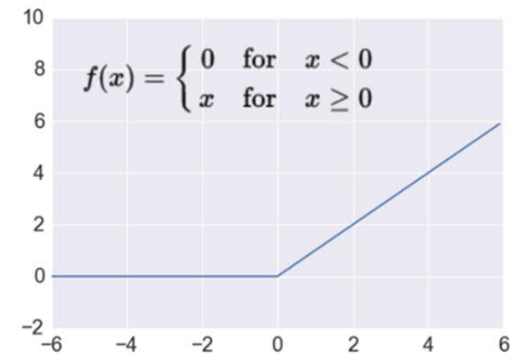


- ▶ Tanh is a scaled sigmoid:  $\tanh(x) = 2\text{sigmoid}(2x) - 1$
- ▶ Range:  $(-1, 1)$
- ▶ Differentiable.
- ▶ Advantage over sigmoid: larger derivative
  - ▶ For input range  $[-1, 1]$ , the derivative of Tanh ranges  $[0.42, 1]$ , while the derivative of Sigmoid ranges  $[0.20, 0.25]$ .
  - ▶ Zero-centered, making the optimization process more flexible and faster to converge

# Examples of Activation Functions

- ▶ Rectified Linear Units (ReLU),

$$f(x) = \max(0, x)$$



- ▶ Commonly used in deep neural networks

- ▶ Differentiable:

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

**Cavet: Dying Relu problem:**

- Could produce **dead neurons** that outputs zero for all training examples

- ▶ Advantages:

- ▶ Efficient computation
- ▶ No vanishing gradient problem in network training
- ▶ Sparse activation (many neurons output zero), which helps prevent overfitting.

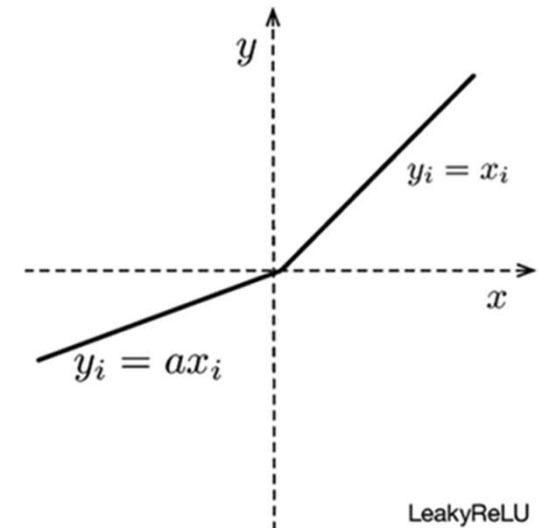


# Examples of Activation Functions

- ▶ Leaky ReLU:

$$f(x) = \max(\alpha x, x)$$

where  $\alpha$  is a small value, e.g., 0.01



- ▶ Commonly used in deep neural networks

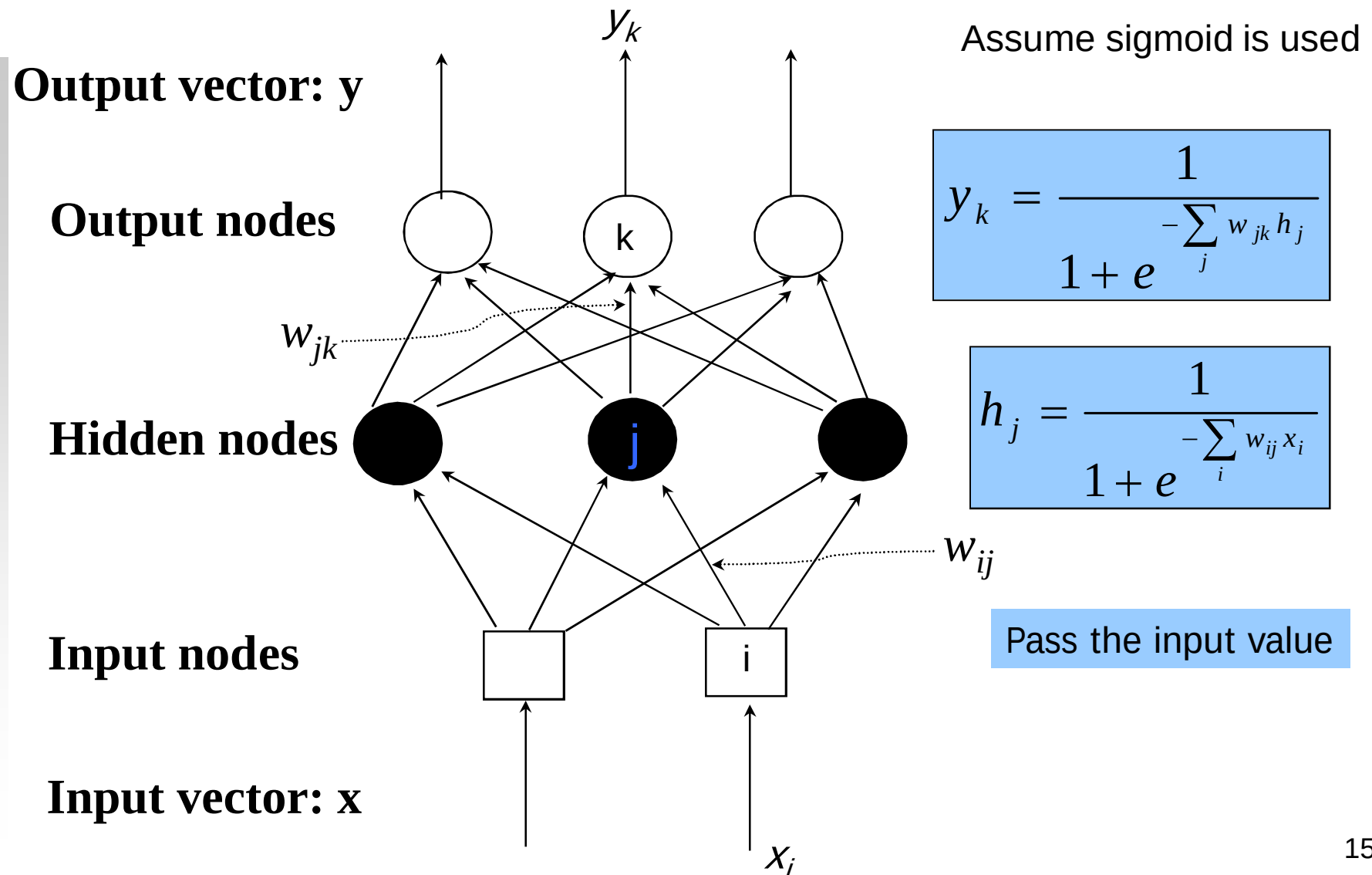
- ▶ Differentiable:

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ \alpha, & \text{otherwise} \end{cases}$$

- ▶ Advantages:

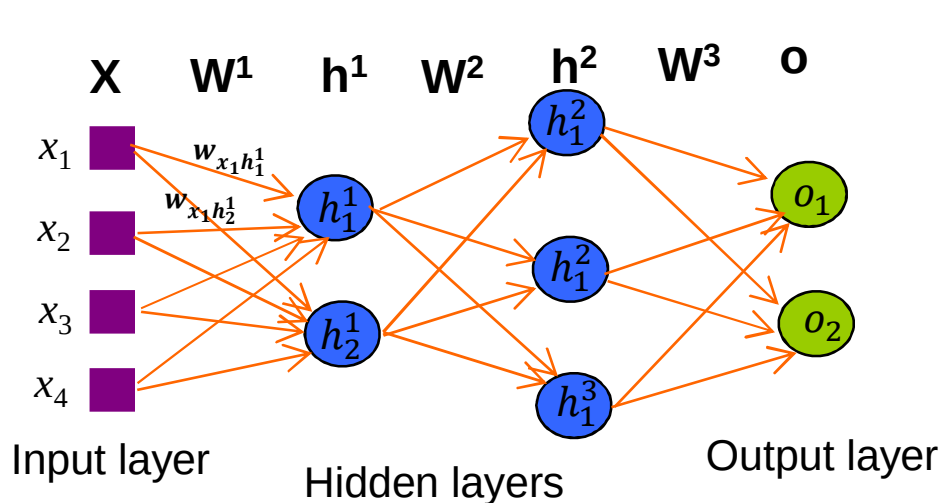
- ▶ Similar to ReLU but with less sparsity
- ▶ Solving the dying Relu problem

# How Output is Computed?



# How Output is Computed?

- Computation can be expressed using matrix multiplications:



| W <sup>1</sup>              | x <sub>1</sub> | x <sub>2</sub> | x <sub>3</sub> | x <sub>4</sub> |
|-----------------------------|----------------|----------------|----------------|----------------|
| h <sub>1</sub> <sup>1</sup> | 0.4            | 0.3            | -0.1           | 0.5            |
| h <sub>2</sub> <sup>1</sup> | 0.1            | -0.8           | 0.2            | -0.1           |

Diagram illustrating the weight matrix W<sup>1</sup> and the input vector x. The weight matrix W<sup>1</sup> is shown as a table with rows h<sub>1</sub><sup>1</sup> and h<sub>2</sub><sup>1</sup>, and columns x<sub>1</sub>, x<sub>2</sub>, x<sub>3</sub>, x<sub>4</sub>. The input vector x is shown as a column vector with elements x<sub>1</sub>, x<sub>2</sub>, x<sub>3</sub>, x<sub>4</sub>. The weight w<sub>x<sub>1</sub>h<sub>1</sub><sup>1</sup> is highlighted, and the weight w<sub>x<sub>1</sub>h<sub>2</sub><sup>1</sup> is also highlighted.</sub></sub>

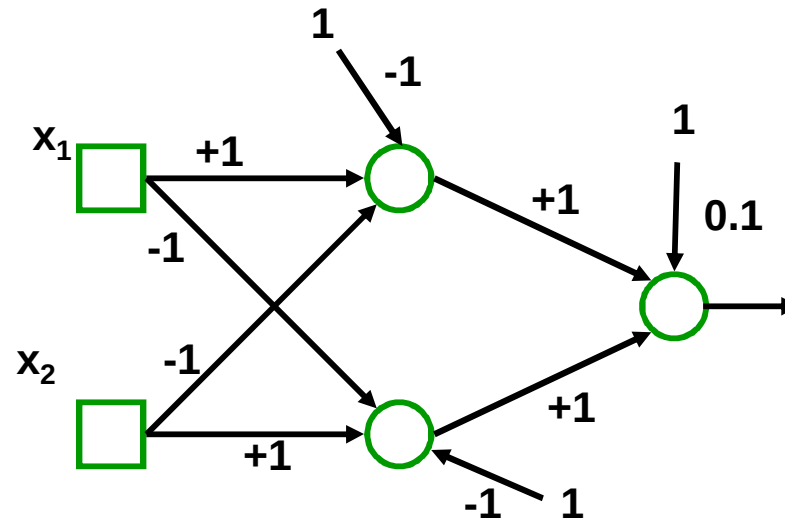
$$\mathbf{h}^1 = f(\mathbf{W}^1 \mathbf{x}) \quad \mathbf{h}^2 = f(\mathbf{W}^2 \mathbf{h}^1) \quad \mathbf{o} = f(\mathbf{W}^3 \mathbf{h}^2)$$

where  $f$  is the activation function.

# Function Modeled by a Neural Network

- ▶ A neural network can be used to model a function:
  - ▶  $y=f(\mathbf{x})$  where  $\mathbf{x}$  is the input vector and  $\mathbf{y}$  is the output vector
- ▶ The function it models is determined by
  - ▶ Topology of the neural network:
    - ▶ the number of input units in the input layer
    - ▶ the number of hidden layers
    - ▶ the number of units in each hidden layer
    - ▶ the number of units in the output layer
  - ▶ Values of weights
  - ▶ Activation function chosen.

# What function does this NN model?



- ▶  $x_1$  and  $x_2$  takes binary values of  $\{-1, 1\}$
- ▶ Activation function is:

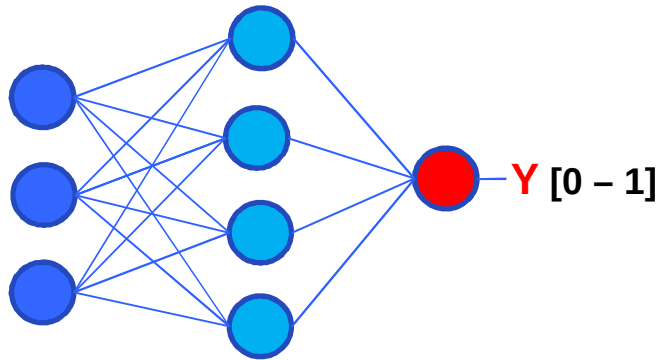
$$f(x) = \text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & \text{otherwise} \end{cases}$$

# Choice of Activation Function

- ▶ Hidden layer nodes:
  - ▶ Usually use the same activation function for all hidden nodes.
  - ▶ Common choices:
    - ▶ Multilayer perceptron (MLP): ReLU
    - ▶ Convolutional neural network (CNN): ReLU
    - ▶ Recurrent neural network (RNN): Sigmoid or Tanh
- ▶ Output layer nodes:
  - ▶ Depending on the problem type
  - ▶ Binary classification: Sigmoid
  - ▶ Multiclass classification: Softmax
  - ▶ Regression: linear (“identify” function or no activation)

# Output Layer Design

- For binary classification:



|                        |           |
|------------------------|-----------|
| Number of output nodes | 1         |
| Activation Function    | Sigmoid   |
| Class 1                | $Y > 0.5$ |
| Class 2                | $Y < 0.5$ |

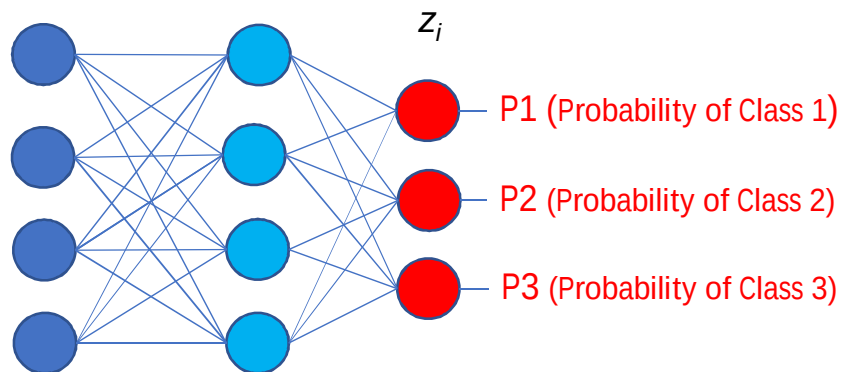
**Activation function:**  $f(z) = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$

# Output Layer Design

## ► Multi-class classification

| Age | Spectacle-prescrip | astigmatism | Tear-prod-rate | Contact-lenses |
|-----|--------------------|-------------|----------------|----------------|
| 18  | myope              | no          | reduced        | none           |
| 20  | hypermetrope       | no          | normal         | soft           |
| 39  | myope              | yes         | normal         | hard           |

|                        |                      |
|------------------------|----------------------|
| Number of output nodes | = number of classes  |
| Activation Function    | Softmax              |
| Class                  | = argmax(P1, P2, P3) |



$z_i$  is the weighted sum of inputs to the  $i$ th output node (before activation)

Desired output for each output node:

| none | soft | hard |
|------|------|------|
| 1    | 0    | 0    |
| 0    | 1    | 0    |
| 0    | 0    | 1    |

$$\text{Softmax: } P_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \sum_{j=1}^K P_j = 1$$

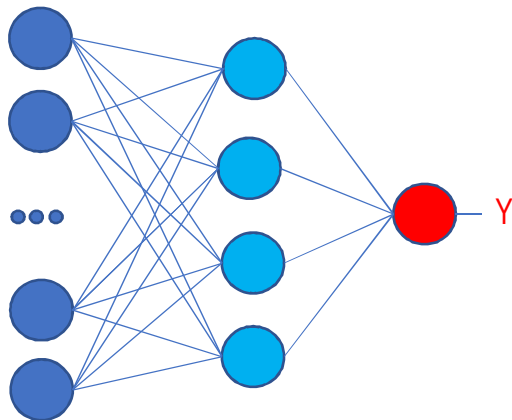


# Output Layer Design

## ► Regression

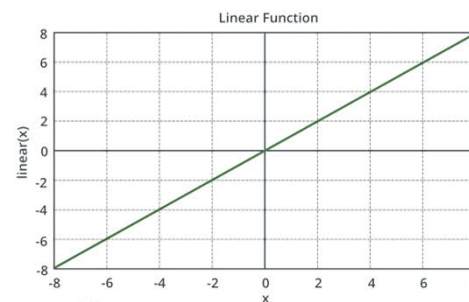


| Features (independent variables) |         |      |     |       |      | Output (dependent variable) |
|----------------------------------|---------|------|-----|-------|------|-----------------------------|
| abandon                          | ability | able | ... | youth | zone | Number of Likes             |
| 5                                | 0       | 10   |     | 0     | 0    | 1000                        |
| 0                                | 17      | 0    |     | 0     | 2    | 45                          |
| 4                                | 0       | 0    |     | 22    | 1    | 530                         |



$$f(z) = z$$

|                        |               |
|------------------------|---------------|
| Number of output nodes | = one         |
| Activation Function    | Linear (None) |



# Inputs to Neural Networks

- ▶ Neural net can take continuous input variables naturally.
- ▶ A discrete attribute can be encoded by:
  - ▶ using one input unit per value of the attribute
  - ▶ For example, if the domain of attribute A is {a1, a2, a3}
    - ▶ We assign 3 input units to represent A.
    - ▶ If A=a1, the input to the 3 units is 1, 0, 0.
    - ▶ If A =a2, the input to the 3 units is 0, 1, 0.
    - ▶ If A=a3, the input to the 3 units is 0, 0, 1.
- ▶ The input values to a neural net are usually normalized to speed up the learning phase.

*One-hot  
encoding*

# Training a Neural Network

- ▶ Before training can begin, the user must decide on
  - ▶ Network topology
    - ▶ # of hidden layers and # of units in each hidden layer
  - ▶ Type of the activity function
  - ▶ no clear rules on the best topology
  - ▶ network design is often a trial-and-error process
- ▶ The objective of training
  - ▶ search in the space of sets of weights to obtain a set of weights that makes almost all the examples in the training data classified correctly.
- ▶ Also called *connectionist* learning
- ▶ Most popular neural network learning algorithm is *backpropagation*

# Training: Backpropagation algorithm

- ▶ Objective:

- ▶ searches for weight values that minimize a loss function over a set of training examples

- ▶ Loss function:

- ▶ Classification problem: **cross-entropy loss**

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (y_k^i \log \hat{y}_k^i)$$

- ▶  $\theta$  denotes the weights
- ▶  $K$  is the number of classes and  $N$  is the number of examples
- ▶  $y^i$  is the true target value (1 or 0) of the  $i$ th example
- ▶  $\hat{y}^i$  is the predicted value for the  $i$ th example
- ▶ Regression problem: **mean squared error loss**

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (y^i - \hat{y}^i)^2$$

# Cross-Entropy

- Cross-entropy measures the difference between two probability distributions  $p$  and  $q$  over a random variable  $V$ :

$$H(p, q) = - \sum_{v \in V} p(v) \log q(v)$$

- The higher the cross entropy, the more dissimilar between  $p$  and  $q$ .
- For example,
  - If  $p=[0, 1, 0]$  and  $q=[0.15, 0.6, 0.25]$ , then

$$H(p, q) = -[0 \cdot \log(0.15) + 1 \cdot \log(0.6) + 0 \cdot \log(0.25)] = 0.736$$

- If  $p=[0, 1, 0]$  and  $q=[0.1, 0.8, 0.1]$ , then

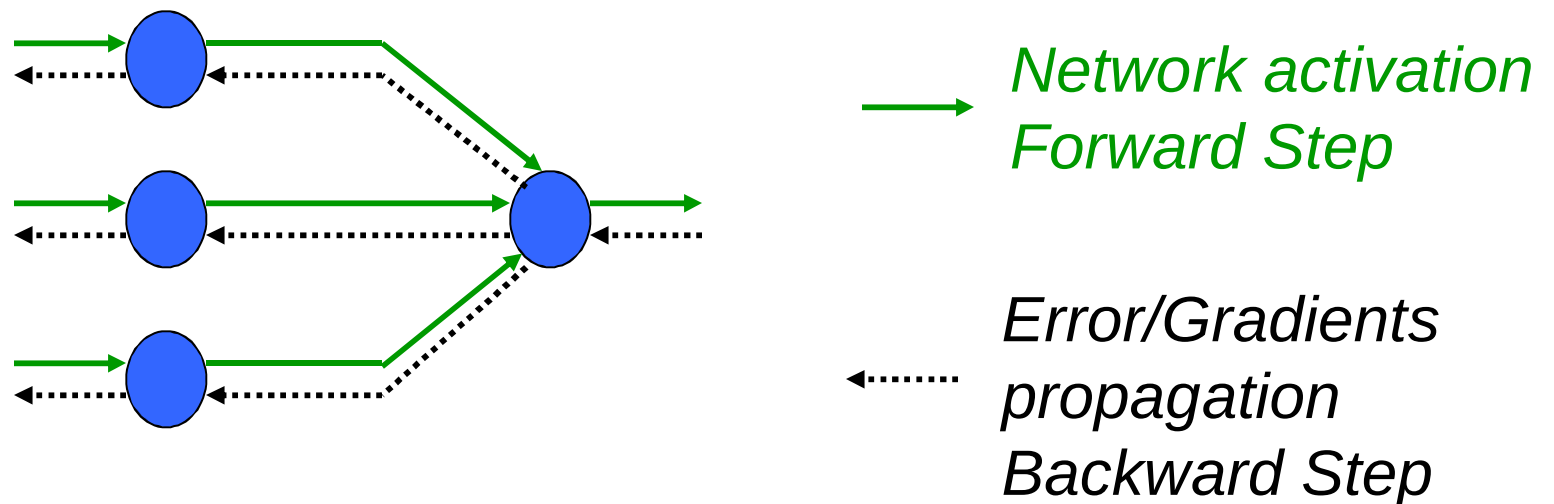
$$H(p, q) = -[0 \cdot \log(0.1) + 1 \cdot \log(0.8) + 0 \cdot \log(0.1)] = 0.2231$$

# Training: Backpropagation algorithm

- ▶ The algorithm first initializes the weights with small random values
- ▶ Then backpropagation consists of the repeated application of the following two passes:
  - ▶ **Forward pass**: in this step the network is activated on one example and the output of the network is computed.
  - ▶ **Backward pass**: in this step, starting at the output layer,
    - ▶ the error of the output layer is computed
    - ▶ its gradients are propagated backwards through the network, layer by layer, and weights are updated.

# Backpropagation

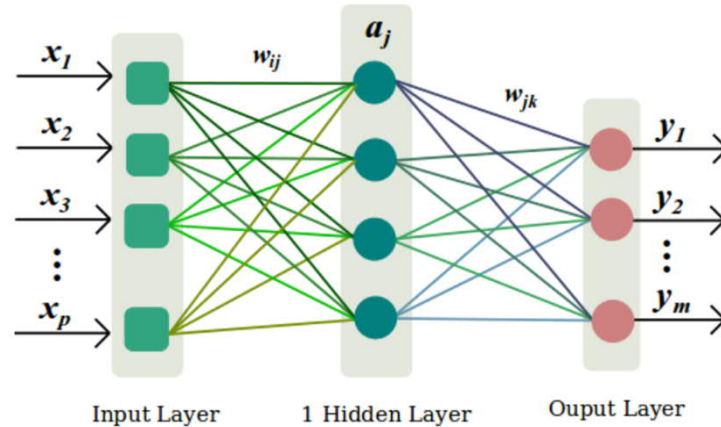
- ▶ Back-propagation training algorithm



- ▶ Backprop adjusts the weights of the NN in order to minimize the loss function.
- ▶ *Gradient descent* or *stochastic gradient descent* algorithm can be used

# How to Update the Weights with Gradient Descent?

- $L(\theta)$  is a function of the weights  $\theta = \{w_1, w_2, \dots\}$



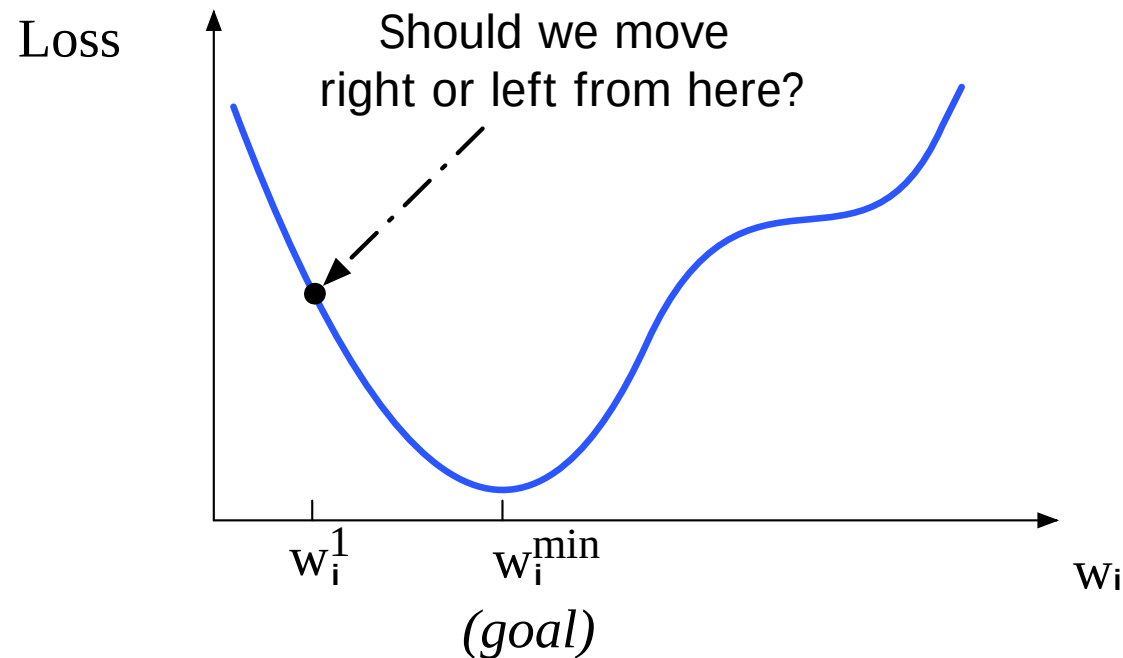
- Each weight  $w_i$  is updated by the value of the partial gradient  $\frac{d}{dw_i} L(\theta)$  weighted by a **learning rate**  $\eta$  :

$$w_i = w_i - \eta \frac{d}{dw_i} L(\theta)$$

Higher learning rate  $\eta$  means move  $w_i$  faster, but if too big may keep oscillating



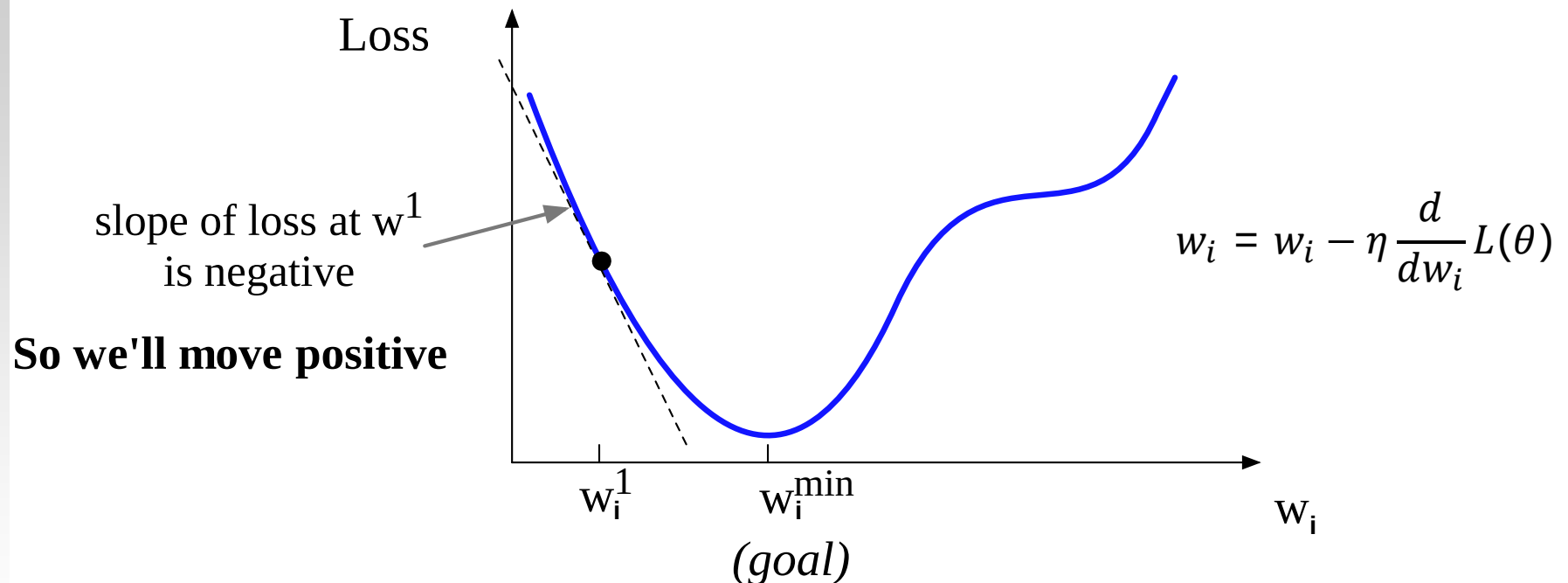
# Why update weight this way?



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

# Why update weight this way?

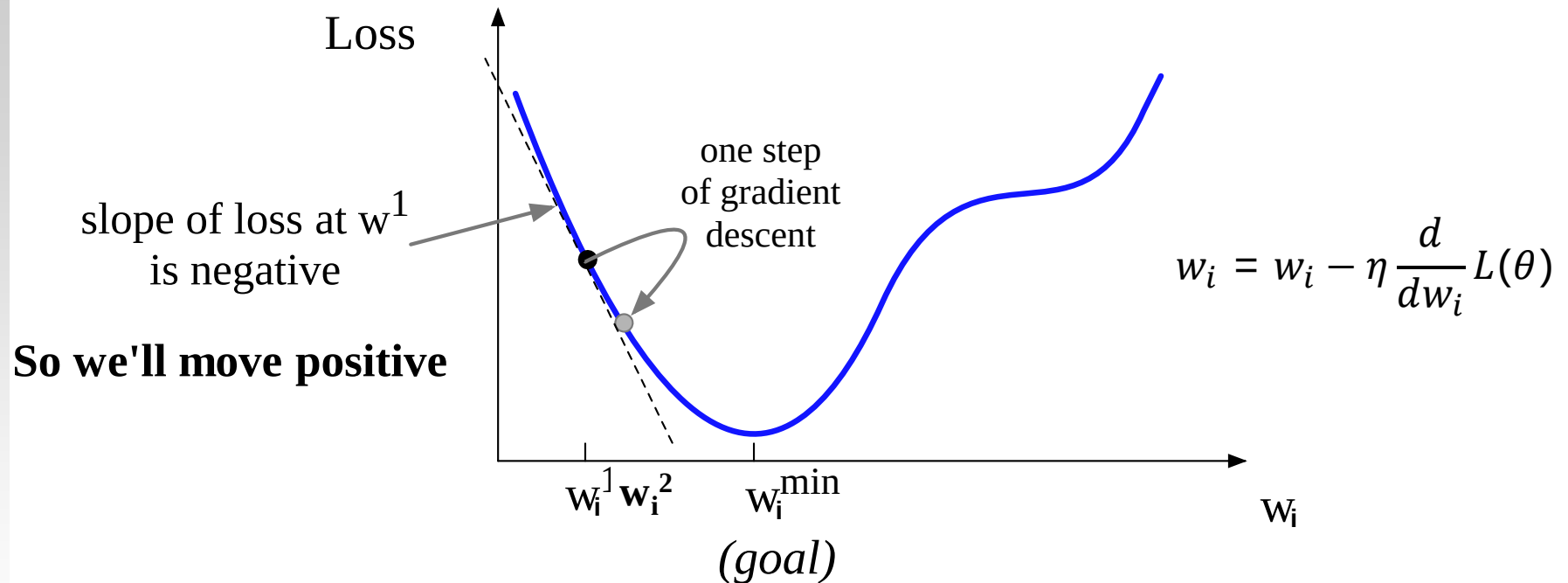
Find the gradient of the loss function at the current point and move in the opposite direction.



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

# Why update weight this way?

Move  $w$  in the reverse direction from the slope of the function



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

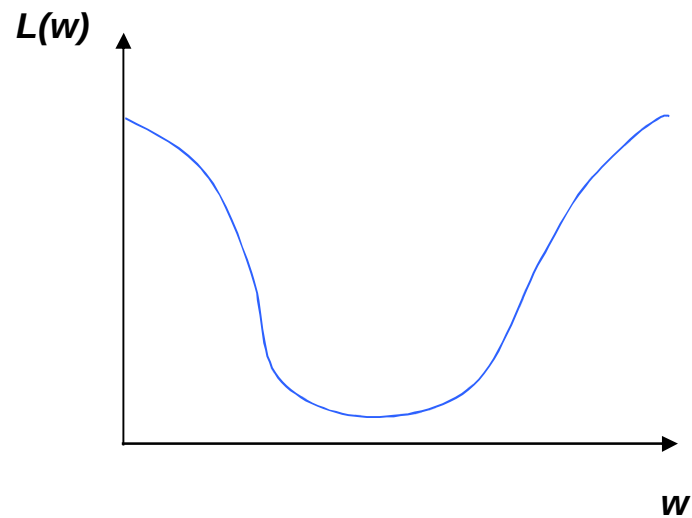
## How much do we move in that direction ?

- The value of the gradient  $\frac{d}{dw} L(\theta)$  weighted by a **learning rate  $\eta$**

$$w_i = w_i - \eta \frac{d}{dw_i} L(\theta)$$

- ▶ Smaller  $\eta$  means moving  $w_i$  slower
- ▶ Higher  $\eta$  means moving  $w$  faster, but if too big may keep oscillating

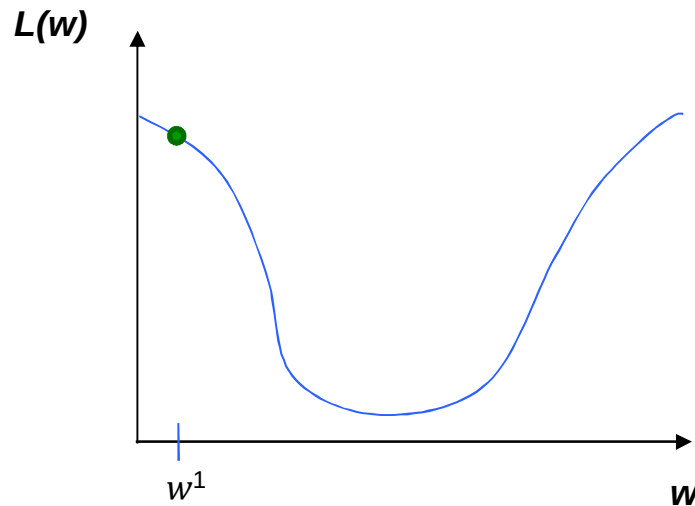
# Illustration of Gradient Descent



(Adapted from James Tompkin (Brown University))

# Illustration of Gradient Descent

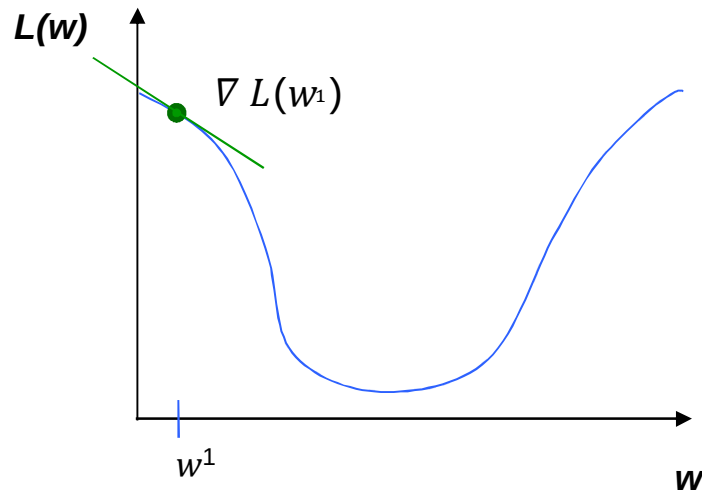
Pick random starting point.



(Picture credit: James Tompkin (Brown University))

# Illustration of Gradient Descent

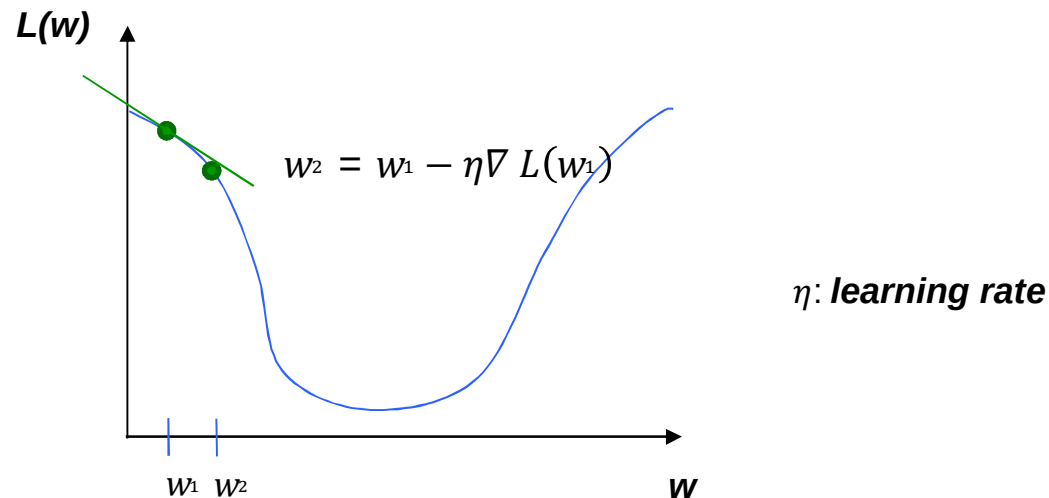
Compute gradient at point (analytically or by finite differences)



(Adapted from slides of James Tompkin, Brown University)

# Illustration of Gradient Descent

Move along parameter space in direction of negative gradient

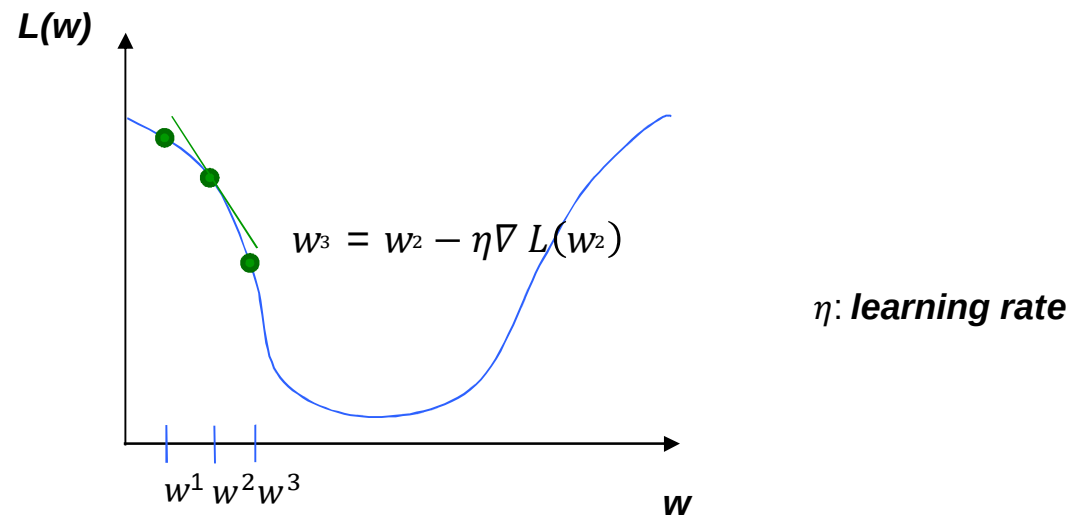


(Adapted from slides of James Tompkin, Brown University)



# Illustration of Gradient Descent

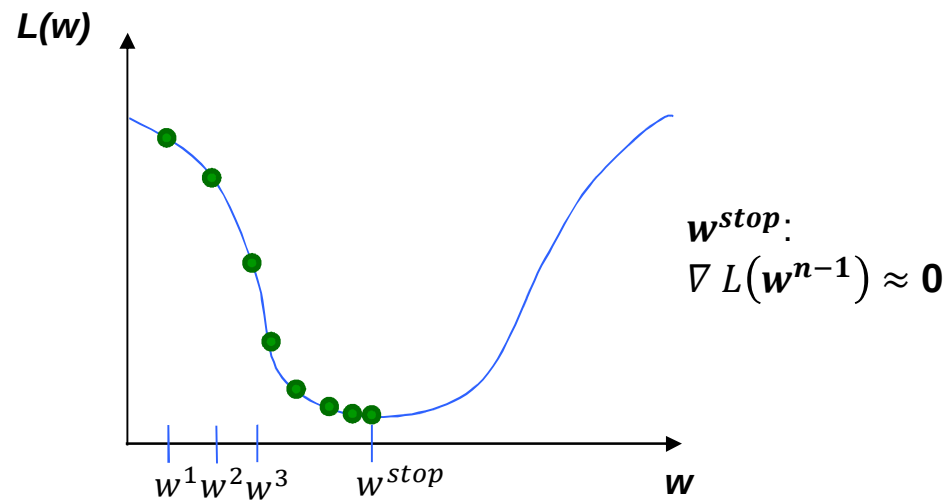
Move along parameter space in direction of negative gradient.



(Adapted from slides of James Tompkin, Brown University)

# Illustration of Gradient Descent

Stop when  $w$  does not change or the change is very small.



# How to Compute the Gradients

- ▶ Final layer weight update:

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

- ▶ Gradient computation by chain rule:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

where  $z = \sum_i w_i h_i$  and  $y(z)$  is the activation function

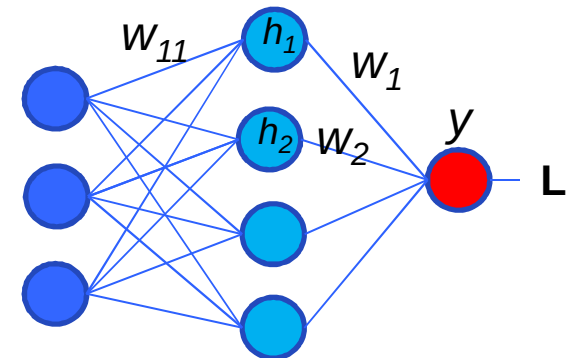
- ▶ Example: assume:

- ▶  $L = \frac{1}{2}(t - y)^2$ , where  $t$  is the true output (label)

- ▶  $y(z)$  is sigmoid:  $y(z) = \sigma(z) = \frac{1}{1+e^{-z}}$

$$\frac{\partial L}{\partial y} = -(t - y), \quad \frac{\partial y}{\partial z} = \sigma(z)(1 - \sigma(z)), \quad \frac{\partial z}{\partial w_i} = h_i$$

$$\frac{\partial L}{\partial w_i} = (y - t)\sigma(z)(1 - \sigma(z)) h_i$$



# How to Compute the Gradients

- ▶ Hidden layer weight update:  $w_{11} = w_{11} - \eta \frac{\partial L}{\partial w_{11}}$ 
  - ▶ Gradient **backpropagation** by chain rule:

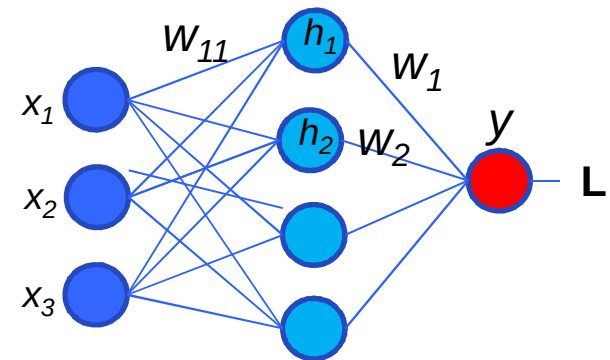
$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial w_{11}}$$

- ▶ Assume sigmoid activation:

$$\frac{\partial y}{\partial h_1} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial h_1} = \sigma(z)(1 - \sigma(z))w_1$$

$$\frac{\partial h_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}} = \sigma(z_1)(1 - \sigma(z_1))x_1$$

- ▶  $\frac{\partial L}{\partial y}$  and  $\frac{\partial y}{\partial h_1}$  are **backpropagated** from the computation in the final layer



# Backpropagation (*Cont'd*)

- ▶ Terminating condition:  
Training stops when
  - ▶ all  $\Delta w_{ij}$  are smaller than a specified threshold, or
  - ▶ the percentage of examples misclassified in the previous epoch is below some threshold, or
  - ▶ a pre-specified number of **epochs** has expired.
    - ▶ An **epoch** is the training of the neural network with **all the training data in one iteration**

## Backpropagation (*Cont'd*)

- ▶ Weight updates can be either case-based, epoch-based or mini-batch based
  - ▶ *Case updating*: weights are updated after presentation of each training example (a version of Stochastic Gradient Decent or SGD)
  - ▶ *Epoch updating*: weight increments are accumulated and are updated after all the training examples have been presented (Gradient Descent)
  - ▶ *Mini-batch updating*: present a randomly-selected subset of examples; weight increments are accumulated, and weights are updated after all the selected examples have been presented (min-batch SGD)

# Min-batch Stochastic Gradient Descent

1. Initializes the model parameters  $\mathbf{w}$  with small random values (e.g., ranging from -1.0 to 1.0, or -0.5 to 0.5)
2. Randomly shuffle the training examples into mini-batches
3. For each mini-batch  $B$ :
  - For each  $(x^i, y^i)$  in  $B$ ,
    - Propagate the inputs forward to obtain the output  $\hat{y}^i$  using the current parameters  $\mathbf{w}$ :
    - Compute loss/error  $L(\mathbf{w}, x^i)$  using  $y^i$  and  $\hat{y}^i$  :
    - Compute the partial gradient  $\frac{\partial}{\partial w_j} L(\mathbf{w}, x^i)$  by **backpropagation**
  - Update each parameter  $w_j$  using:
$$w_j = w_j - \frac{\eta}{|B|} \sum_{x \in B} \frac{\partial}{\partial w_j} L(\mathbf{w}, x)$$
4. Go to Step 2 until  $\mathbf{w}$  converges (or the number of epochs exceeds a threshold)

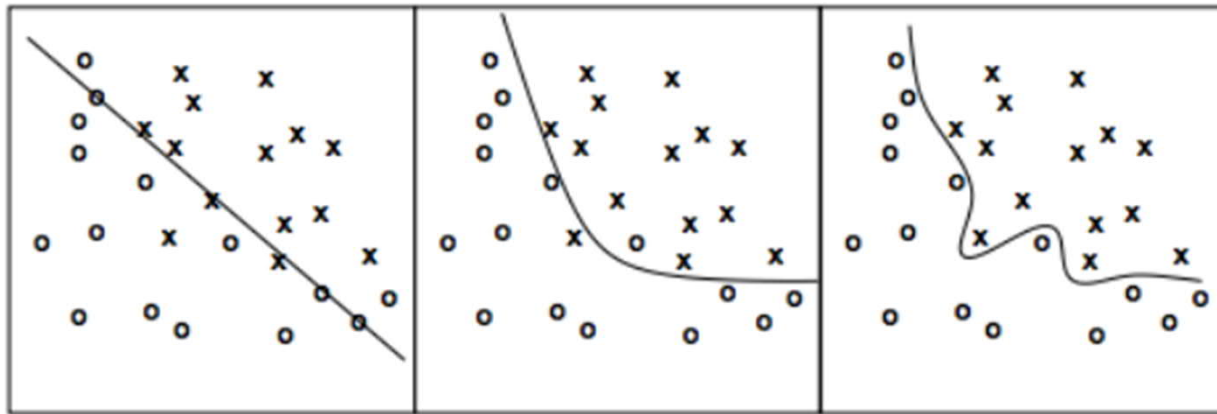
---

# Outline

- ▶ *Introduction*
- ▶ *Types of neural network structures*
- ▶ *What happens inside a neuron?*
- ▶ *How to use a neural network for classification*
- ▶ *How to train a neural network*
  - ▶ *Backpropagation algorithm*
- ▶ **How to prevent overfitting**
- ▶ *Applications and history of neural networks*
- ▶ *Brief introduction to deep learning*



# Overfitting

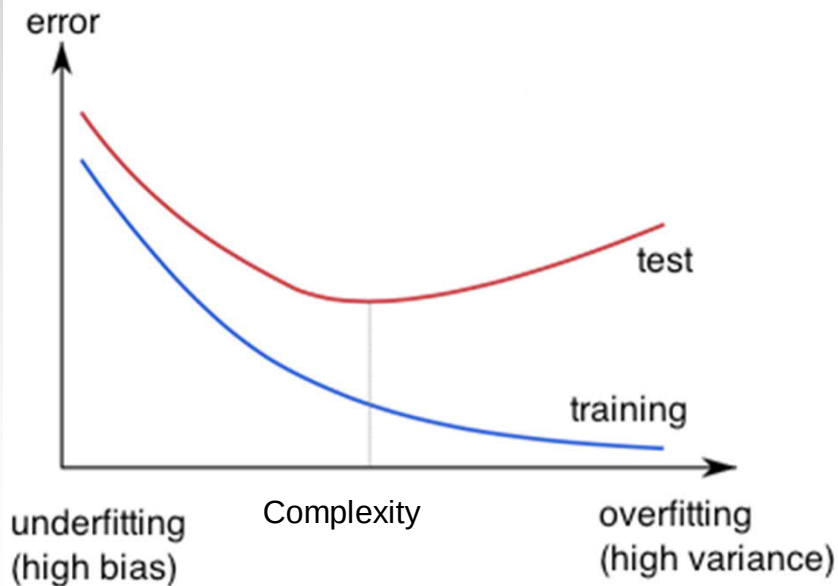


inadequate

good compromise

over-fitting

<http://wiki.bethanycrane.com/overfitting-of-data>



Learned hypothesis may **fit** the training data very well, even outliers (**noise**) but fail to **generalize** to new examples (test data)

[https://www.neuraldesigner.com/images/learning/selection\\_error.svg](https://www.neuraldesigner.com/images/learning/selection_error.svg)

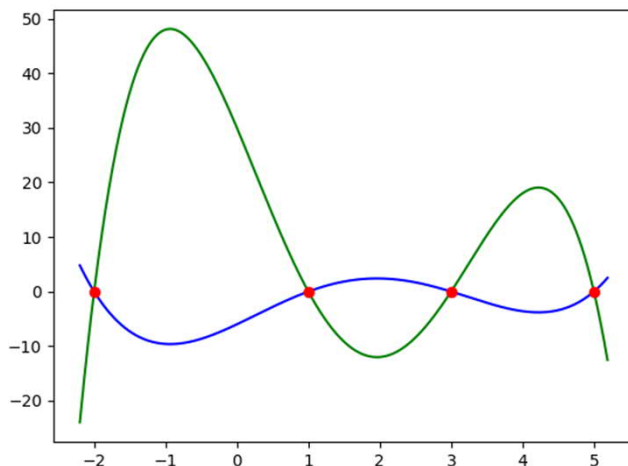
# Prevent Overfitting

## ► Regularization

- Add regularization term to the objective function to penalize big weights ( $\theta$ ). For example,

$$L_{reg}(\theta) = L(\theta) + \lambda \sum_k \theta_k^2 \quad \leftarrow \begin{array}{l} \text{L2 regularization} \\ \text{(Squared L2 norm of } \theta) \end{array}$$

- Why regularizing the parameters helps to prevent overfitting?



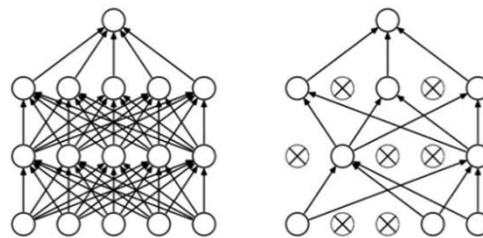
$$y_{\text{green}} = -x^4 + 7x^3 - 5x^2 - 31x + 30$$

$$y_{\text{blue}} = \frac{1}{5}x^4 - \frac{7}{5}x^3 + x^2 + \frac{31}{5}x - 6$$

# Prevent Overfitting

## ▶ Dropout

- ▶ Randomly drop units (along with their connections) during training



## ▶ Benefits:

- ▶ Simplifies the network
  - ▶ A simpler network has less capacity to memorize the training data and is therefore less likely to overfit.
- ▶ Ensemble effect
  - ▶ Each time a different subset of neurons is dropped, similar to training a different “thinned” version of the network.
- ▶ Noise injection
  - ▶ Introduce a form of noise into the training process, preventing the model from fitting the data too closely

---

# Prevent Overfitting

- ▶ Early stopping
  - ▶ Split the training data into building and validation sets.
  - ▶ Use validation error to decide when to stop training
  - ▶ Stop when monitored quantity has not improved after n subsequent epochs

---

# Advantages of Neural Networks

- ▶ Solve problems that are too complicated for conventional techniques
  - ▶ can model both linear or non-linear functions
  - ▶ can do both classification and regression
- ▶ Able to learn from complicated or noisy data
- ▶ Problem solving (e.g., classification) with a neural network is fast

# Limitations

- ▶ Training is slow
  - ▶ could take hours, days and even longer
- ▶ In general, few rules for setting up the parameters regarding the topology of the network
- ▶ Difficult for human to understand the learned target function
  - ▶ Effective “Black Boxes” whose rules of operation are completely unknown
- ▶ Important to have many known examples for training (inputs and related outputs)

# Applications

- ▶ Computer Vision
- ▶ Speech Recognition
- ▶ Pattern Recognition
- ▶ Natural Language Processing (NLP)
- ▶ Financial prediction
- ▶ Screening Mortgage & Credit Applications
- ▶ Oil & Gas Exploration
- ▶ Medical Diagnosis
- ▶ Targeted Marketing

# History of Neural Networks

- ▶ 1943: McCulloch and Pitts proposed a model of a neuron --> Perceptron.
- ▶ 1960s: Widrow and Hoff explored Perceptron networks (which they called “Adelines”) and the delta rule.
- ▶ 1962: Rosenblatt proved the convergence of the perceptron training rule.
- ▶ 1969: Minsky and Papert showed that the Perceptron cannot deal with nonlinearly-separable data sets---even those that represent simple function such as X-OR.
- ▶ 1970-1985: Very little research on Neural Nets
- ▶ 1986: Invention of Backpropagation [Rumelhart and McClelland, but also Parker and earlier on: Werbos] which can learn from nonlinearly-separable data sets.
- ▶ Since 1986: A lot of research in Neural Nets!
- ▶ In 2006, Geoff Hinton, et al., showed an effective way to train a **deep neural network (DNN)** by
  - ▶ effectively pre-training one layer at a time in an unsupervised way and
  - ▶ then fine-tuning it using supervised backpropagation
- ▶ Since then, many DNN architectures have been proposed and deep learning is the state-of-the-art machine learning technique.



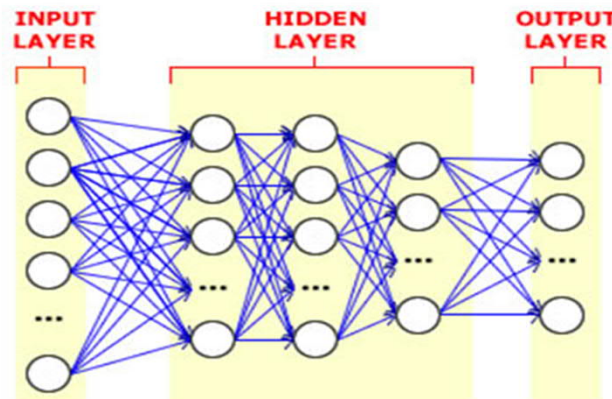
---

# Outline

- ▶ Introduction
- ▶ Types of neural network structures
- ▶ What happens inside a neuron?
- ▶ Activation functions
- ▶ Input and output design
- ▶ How to train a neural network
  - ▶ Backpropagation algorithm
- ▶ How to prevent overfitting
- ▶ Applications and history of neural networks
- ▶ **Brief introduction to deep learning**

# Deep Neural Networks

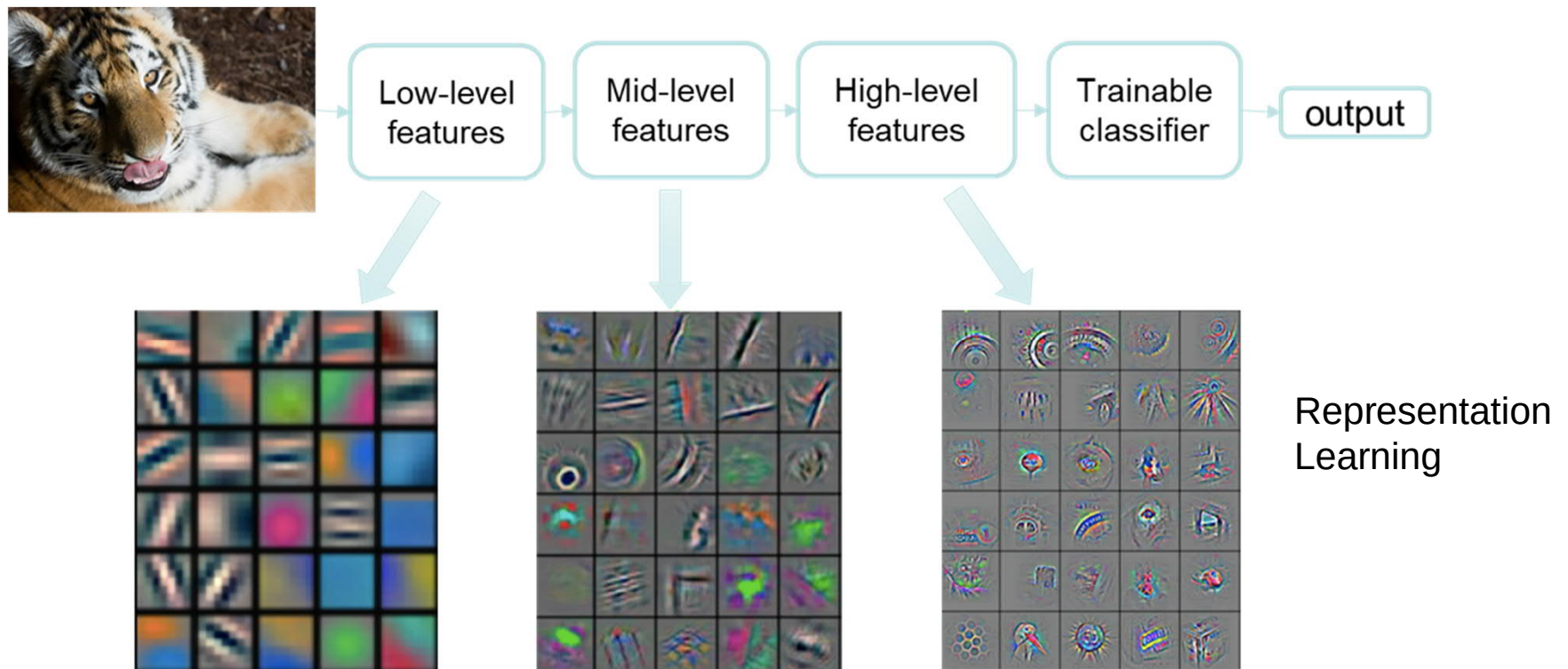
- ▶ Neural network with more than one hidden layer of nodes between input and output



- ▶ Providing breakthrough results in
  - ▶ Image classification
  - ▶ Speech recognition
  - ▶ Natural language processing
  - ▶ .....

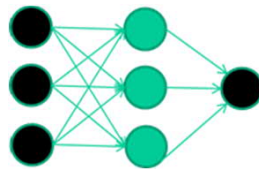
# Deep Neural Networks

- ▶ Why is it generally better than other methods?
  - ▶ Seek to learn rich hierarchical representations (i.e. features) automatically through multiple stage of feature learning process.

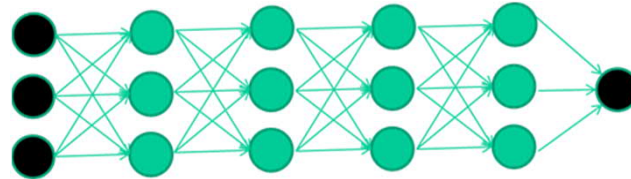


# Deep Neural Networks

- ▶ The previous NN training algorithm is
  - ▶ good for training networks with 1 or 2 hidden layers



- ▶ Not good at learning networks with more hidden layers.

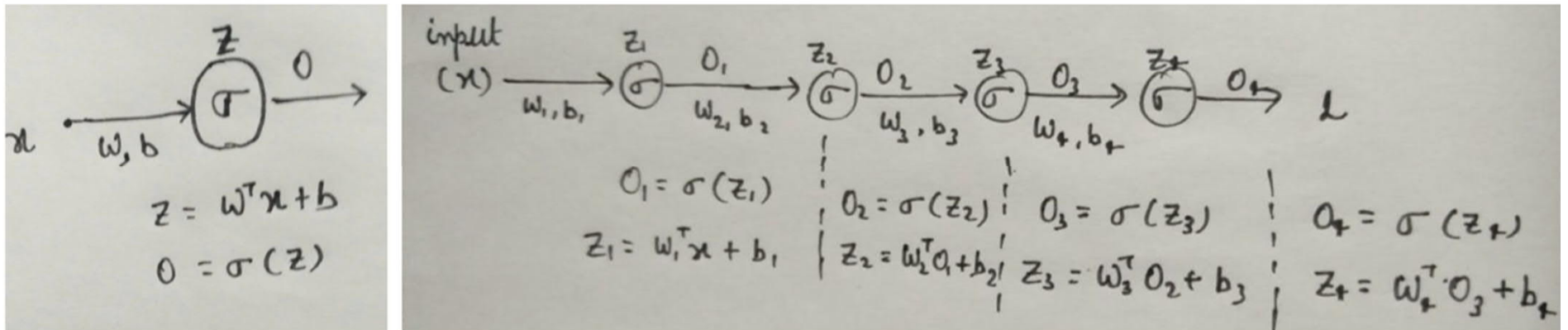


Perform worse than NN  
with 1 or 2 layers

- ▶ Reasons:
  - ▶ Vanishing and exploding gradients problems
  - ▶ Many basins in loss function exist and easily get stuck in poor local minima

# Gradient Vanishing Problem

- ▶ Gradients of the loss function with respect to the weights in the earlier layers of the network become **extremely small** or **vanish**



- ▶ Recall  $w_i = w_i + \Delta w_i$  and  $\Delta w_i = -\eta \frac{\partial L}{\partial w_i}$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial o_4} \frac{\partial o_4}{\partial o_3} \frac{\partial o_3}{\partial o_2} \frac{\partial o_2}{\partial o_1} \frac{\partial o_1}{\partial w_1} ; \text{where, } O_i = \sigma(z_i) \quad z_i = w_i o_{i-1} + b_i$$

- ▶ Derivatives of Sigmoid range (0,0.25) and derivatives of Tahn range (0,1)
- ▶ Problem:
  - ▶ Weights in the earlier layers are very hard or slow to train, leading to a stagnation in the learning process.

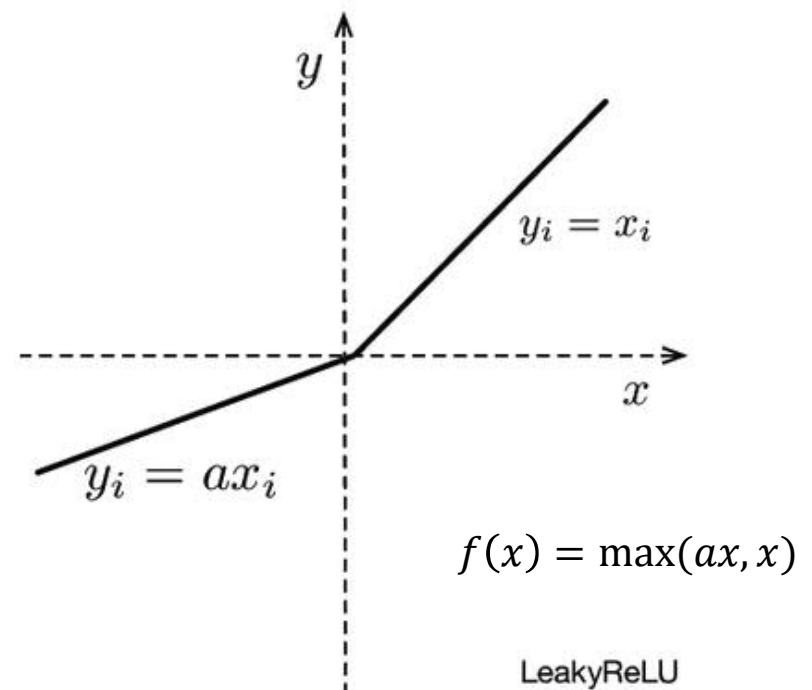
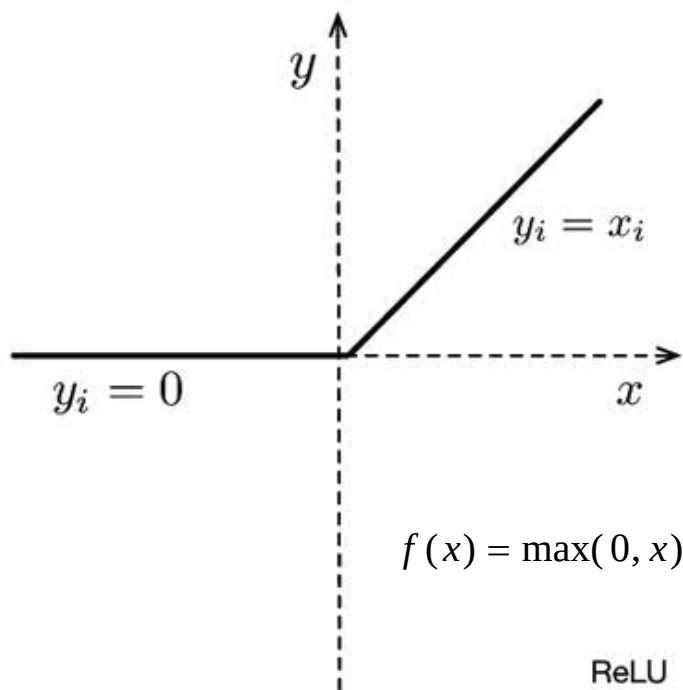
# Gradient Vanishing Problem

- ▶ Causes of the problem:
  - ▶ Activation Functions:
    - ▶ Sigmoid and tanh functions have gradients in the range  $(0, 0.25)$  and  $(0, 1)$ , respectively. When these gradients are multiplied during backpropagation, they can diminish exponentially with each layer.
  - ▶ Deep Architectures:
    - ▶ In very deep networks, the repeated multiplication of small gradients can lead to an exponential decrease in the gradient magnitude as it is propagated back through the layers.
  - ▶ Poor Weight Initialization:
    - ▶ If the weights are initialized poorly, it can exacerbate the vanishing gradient problem. For instance, weights that are too small can lead to small gradients during backpropagation.

# Solutions to Gradient Vanishing Problem

- ▶ Activation functions:

- ▶ Use ReLu or its variants (e.g., Leaky ReLu, Parametric ReLu, etc.)



- ▶ Leaky ReLu:

- ▶ avoid the "dying ReLU" problem, where some neurons can become permanently inactive during training.

# Solutions to Gradient Vanishing Problem

- ▶ Gradient Clipping:
  - ▶ Force the gradient values (element-wise) to a specific minimum or maximum value if the gradient exceeds an expected range.
- ▶ Network Architectures:
  - ▶ Skip Connections/Residual Networks (ResNets):
    - ▶ Architectures like ResNets use skip connections to allow gradients to flow through the network more directly, mitigating the vanishing gradient problem.
  - ▶ Use of Gated Units in RNNs:
    - ▶ In recurrent neural networks (RNNs), gated units like Long Short-Term Memory (LSTM) cells and Gated Recurrent Units (GRUs) are designed to address the vanishing gradient problem by incorporating mechanisms that allow gradients to flow unchanged.



# Gradient Exploding Problem

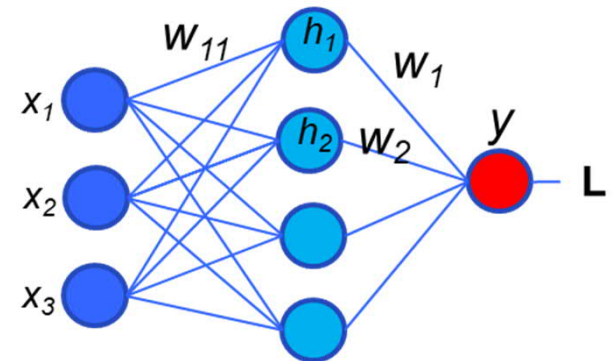
- ▶ Gradients are too large leading to very large updates to the network's weights, causing
  - ▶ the model to become unstable
  - ▶ the loss function to diverge (i.e., loss increases)
- ▶ Gradient backpropagation by chain rule:

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial w_{11}}$$

where

$$\frac{\partial y}{\partial h_1} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial h_1} = \frac{\partial y}{\partial z} w_1$$

$$\frac{\partial h_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} x_1$$



- If  $w_i$  or  $x_i$  is large, the partial derivative values can be greater than 1.
- If all weights are large values and the network is deep, gradients can explode.

---

# Gradient Exploding Problem

- ▶ Symptoms:
  - ▶ NaN (Not a Number) values in the gradients or weights.
  - ▶ A very high loss at the beginning of training, or a loss that becomes very high suddenly.
  - ▶ An unstable model where the output for a given input changes dramatically over training iterations.

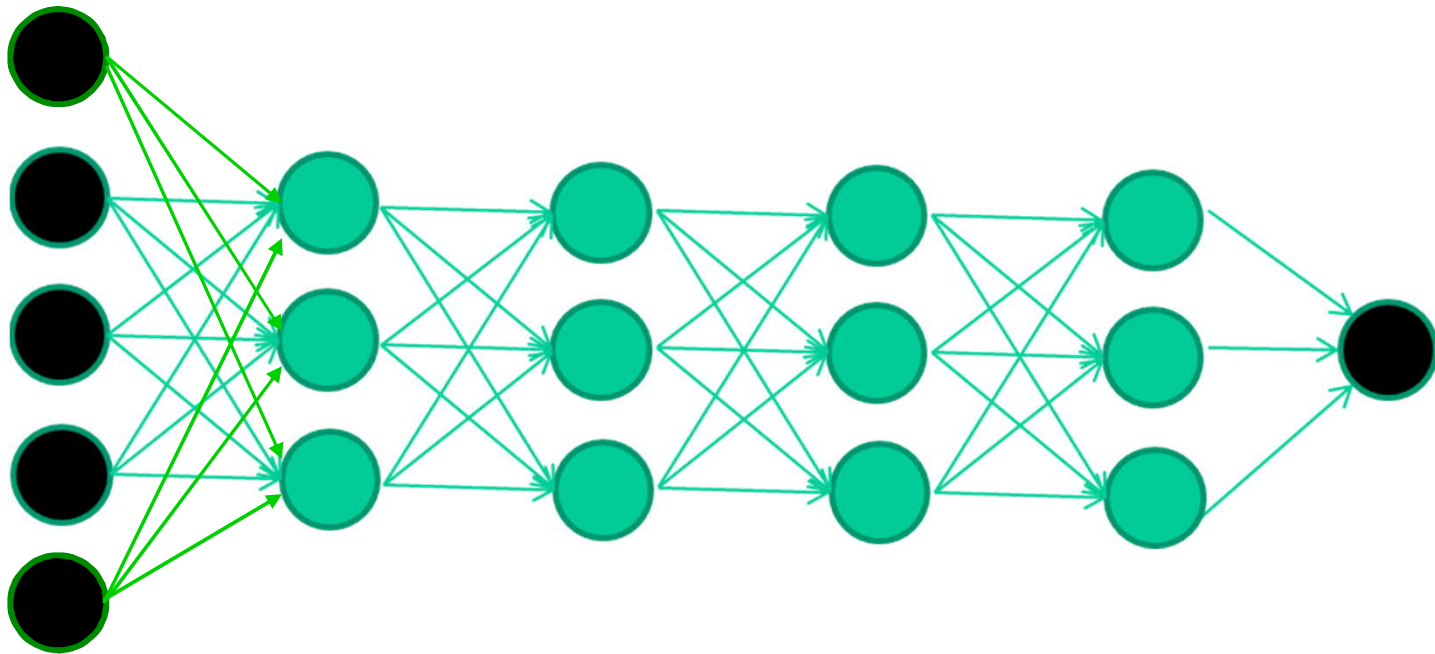
# Solutions to Gradient Exploding Problem

- ▶ Gradient Clipping:
  - ▶ Clipping the gradients during backpropagation to ensure they do not exceed a certain threshold.
- ▶ Gradient Scaling:
  - ▶ Normalizing the error gradient vector so that the vector norm (magnitude) equals a predefined value, such as 1.0
- ▶ Weight Initialization:
  - ▶ Proper initialization of network weights to small values can help prevent gradients from becoming too large.
- ▶ Batch Normalization:
  - ▶ Normalize the outputs of an intermediate layer over a batch of training data.

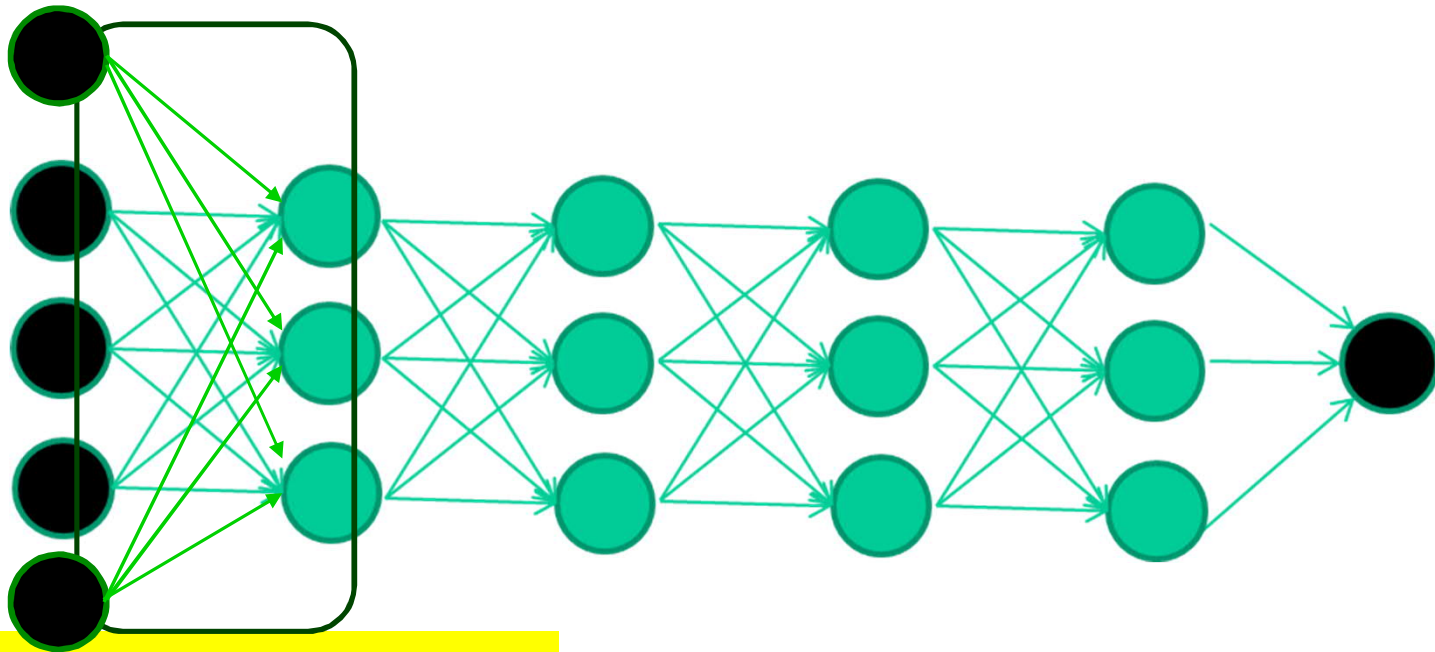
# A Solution for Stuck on Poor Local Minimum

- ▶ A trick to avoid **getting stuck on a poor local minimum**:
  - ▶ Use unsupervised learning to initialize (pre-train) weights
    - ▶ Weights are initialized layer by layer
    - ▶ Each layer is trained using an auto-encoder
  - ▶ Two such early tricks (used popularly in 2006 – early 2010s)
    - ▶ Stacked Autoassociator or autoencoder Network to pre-train a deep NN.
    - ▶ Stacked Restricted Boltzmann machines (RBM) to pre-train a deep believe network

# Pre-train Multi-layer NN Layer-by-layer

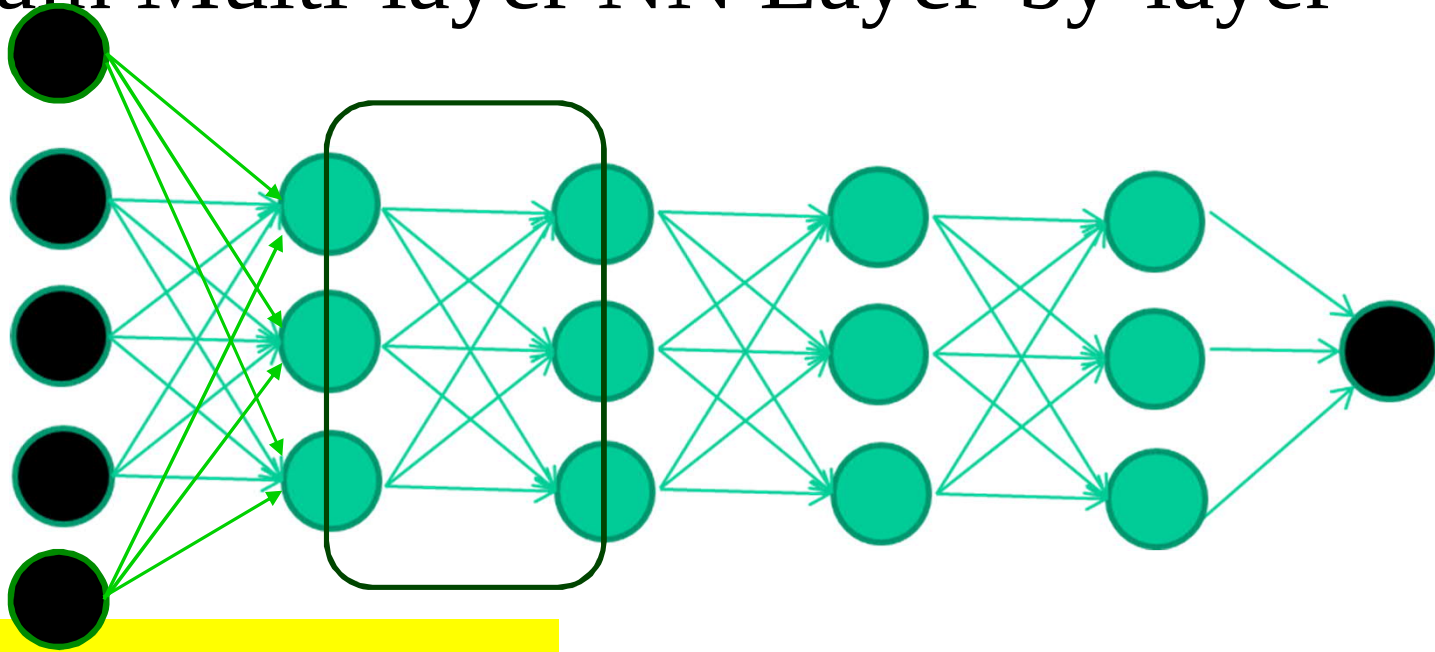


# Pre-train Multi-layer NN Layer-by-layer



**Train this layer first**

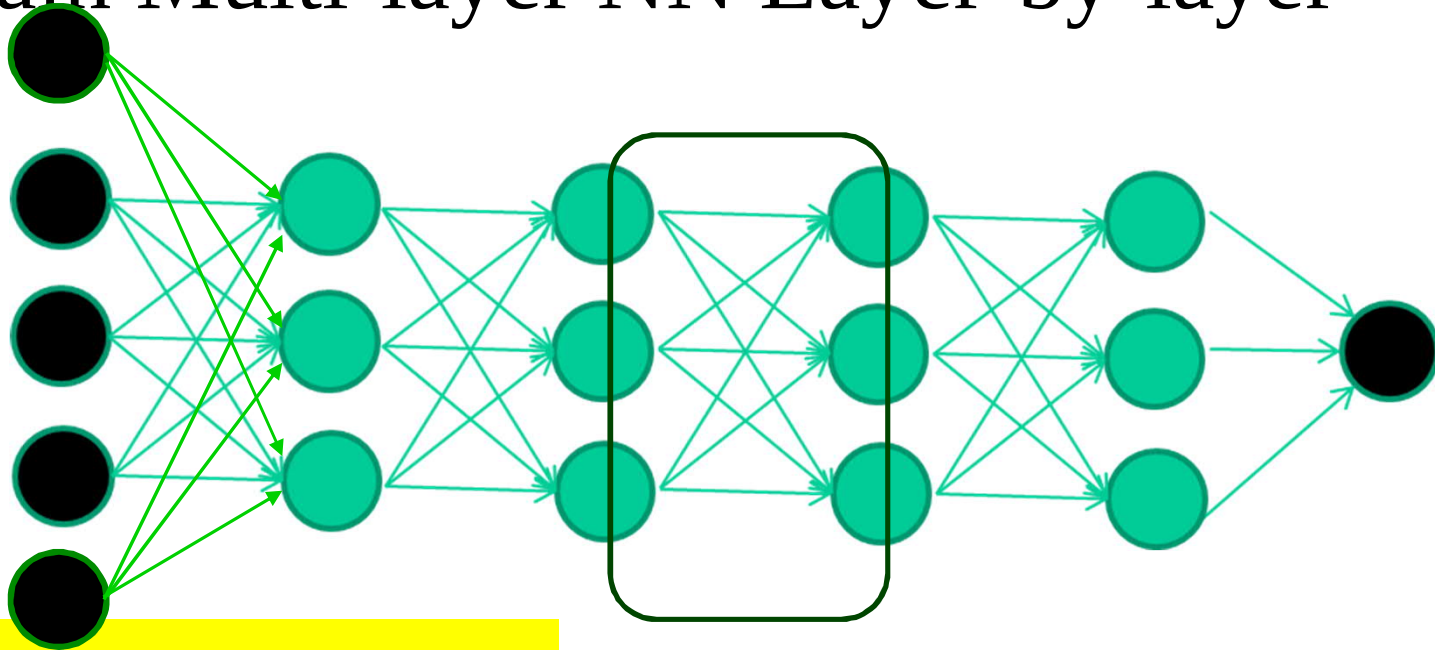
# Pre-train Multi-layer NN Layer-by-layer



**Train this layer first**

**then this layer**

# Pre-train Multi-layer NN Layer-by-layer



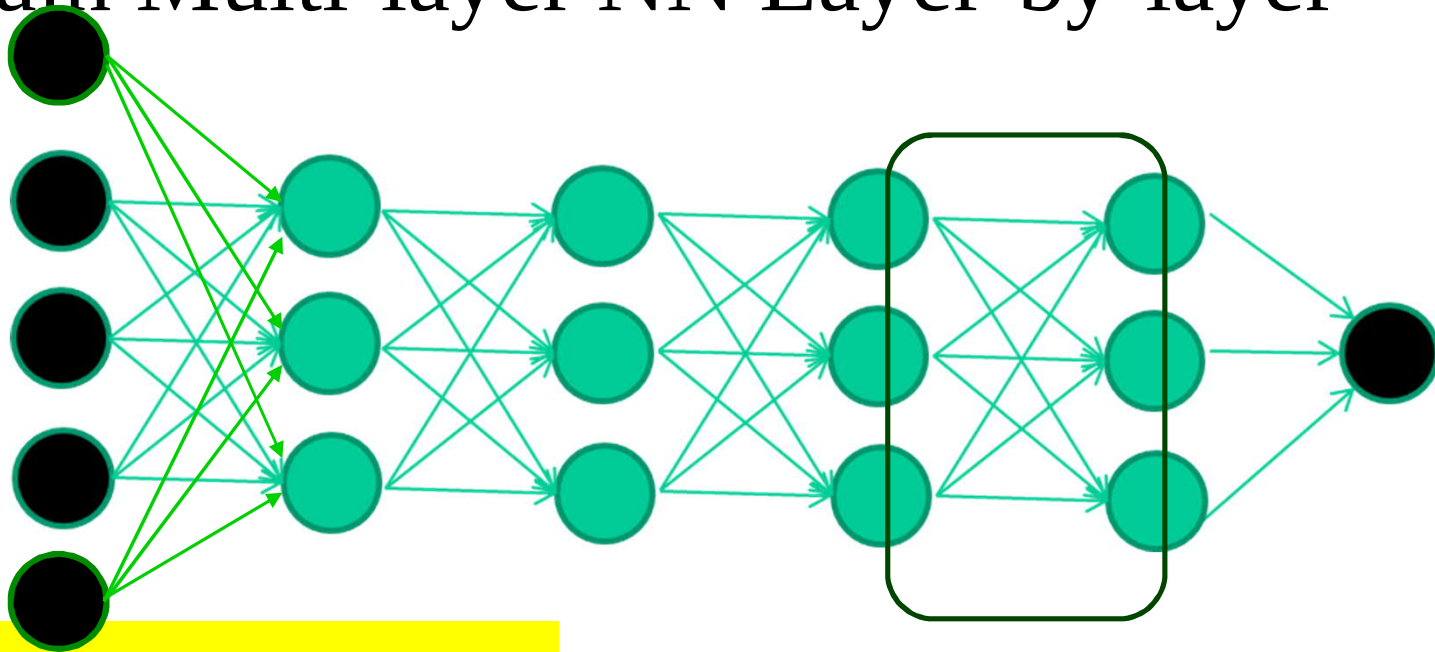
**Train this layer first**

**then this layer**

**then this layer**



# Pre-train Multi-layer NN Layer-by-layer



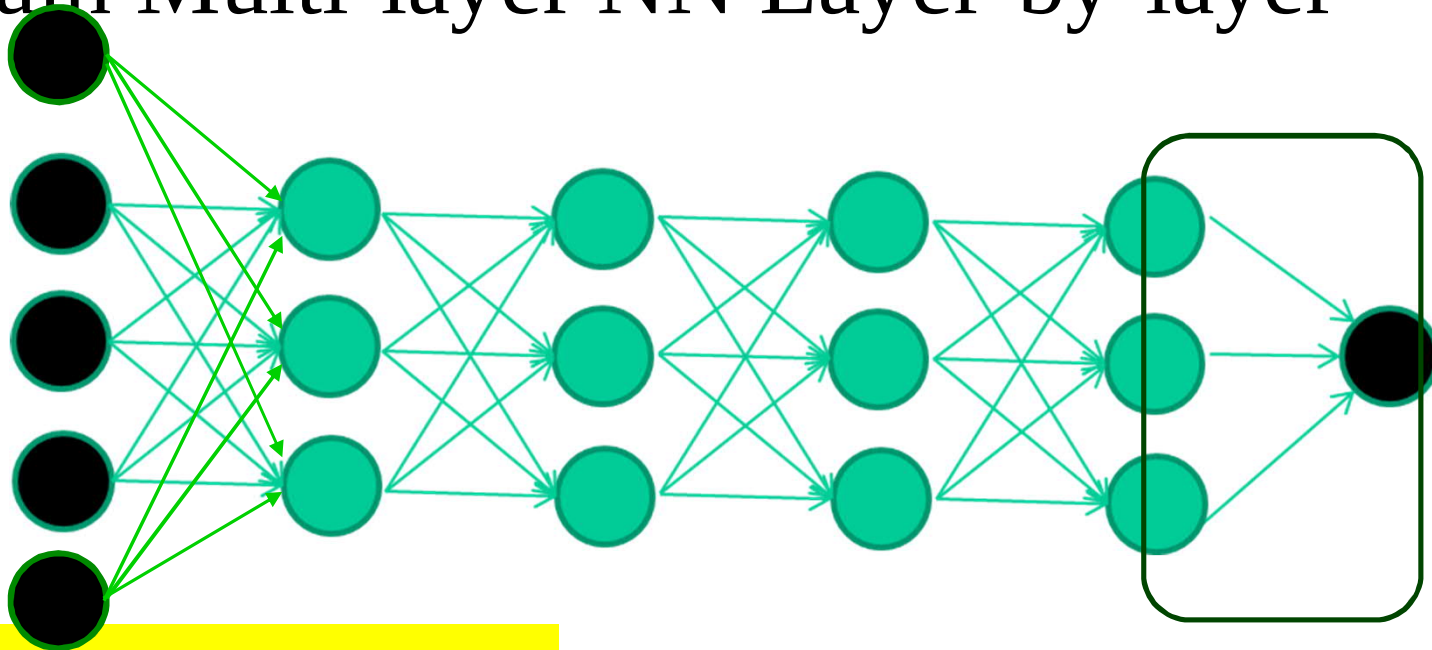
**Train this layer first**

**then this layer**

**then this layer**

**then this layer**

# Pre-train Multi-layer NN Layer-by-layer



**Train this layer first**

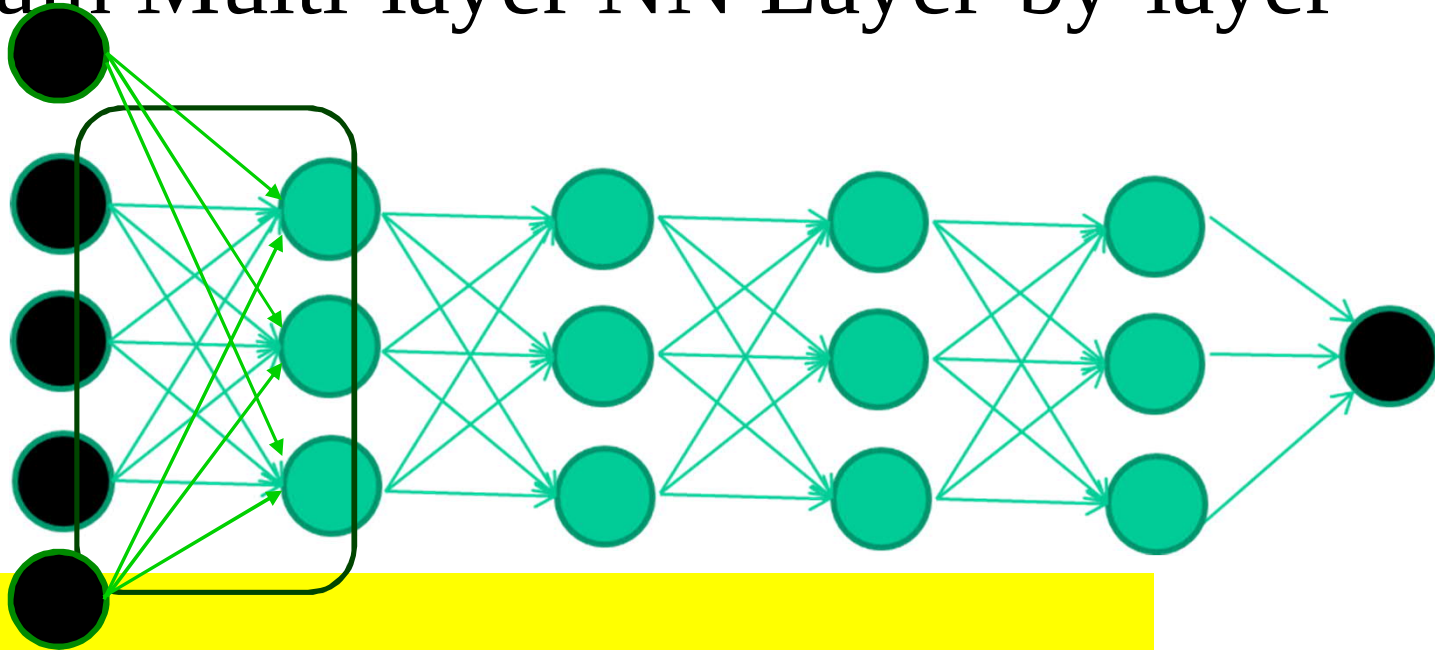
**then this layer**

**then this layer**

**then this layer**

**finally this layer**

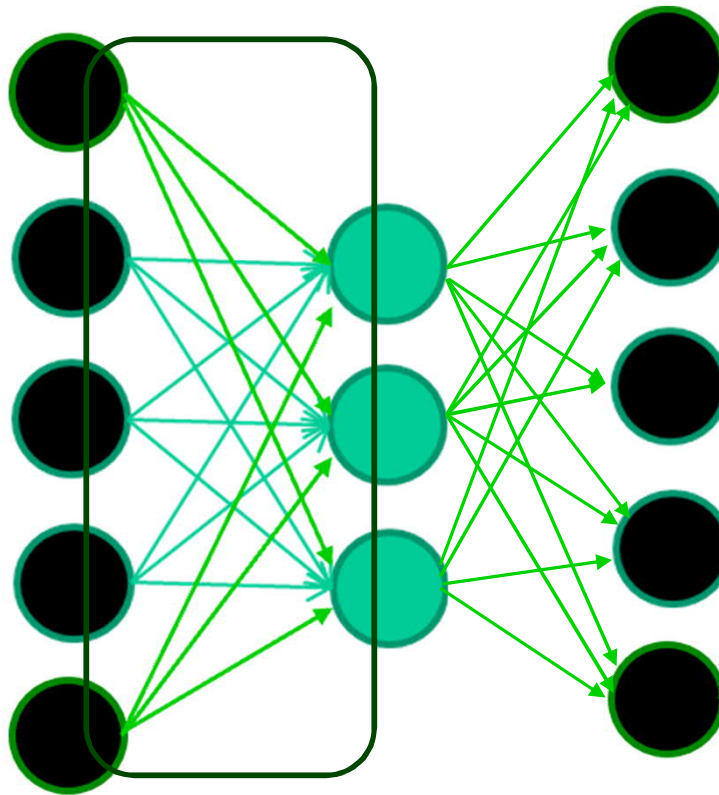
## Pre-train Multi-layer NN Layer-by-layer



***EACH of the (non-output) layers is trained to be, e.g., an **auto-encoder*****

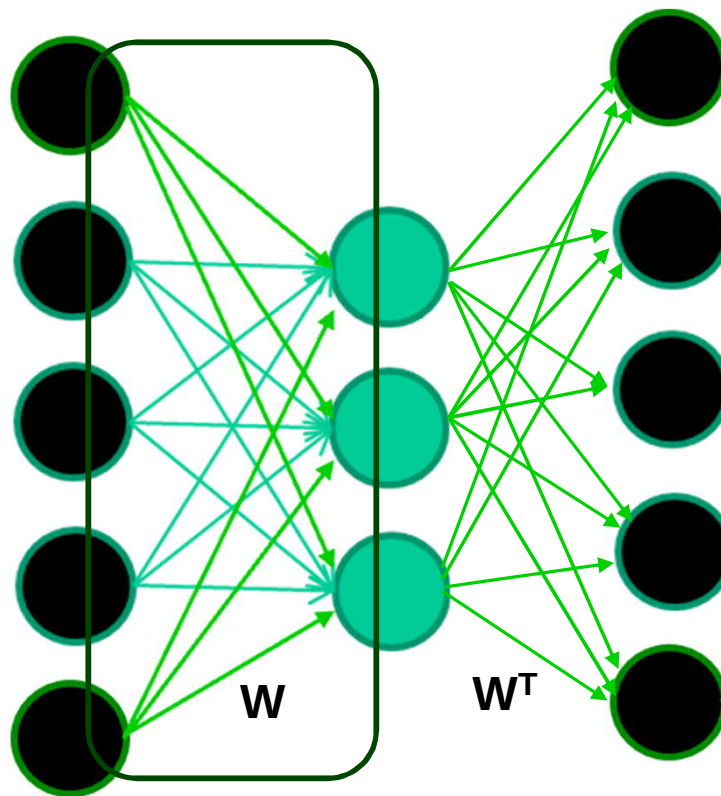
***Basically, it is forced to learn good features that describe what comes from the previous layer***

# Auto-encoder



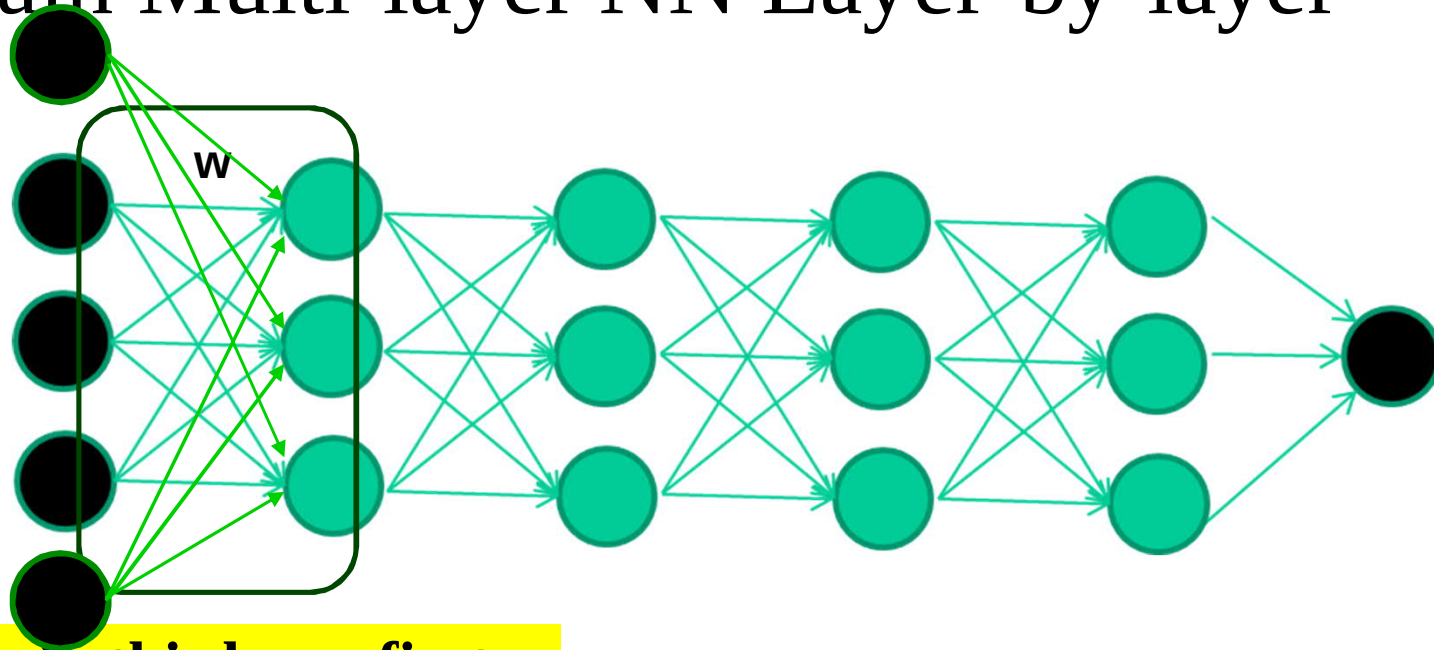
**An auto-encoder is trained, with an absolutely standard weight-adjustment algorithm to reproduce the input**

# Auto-encoder



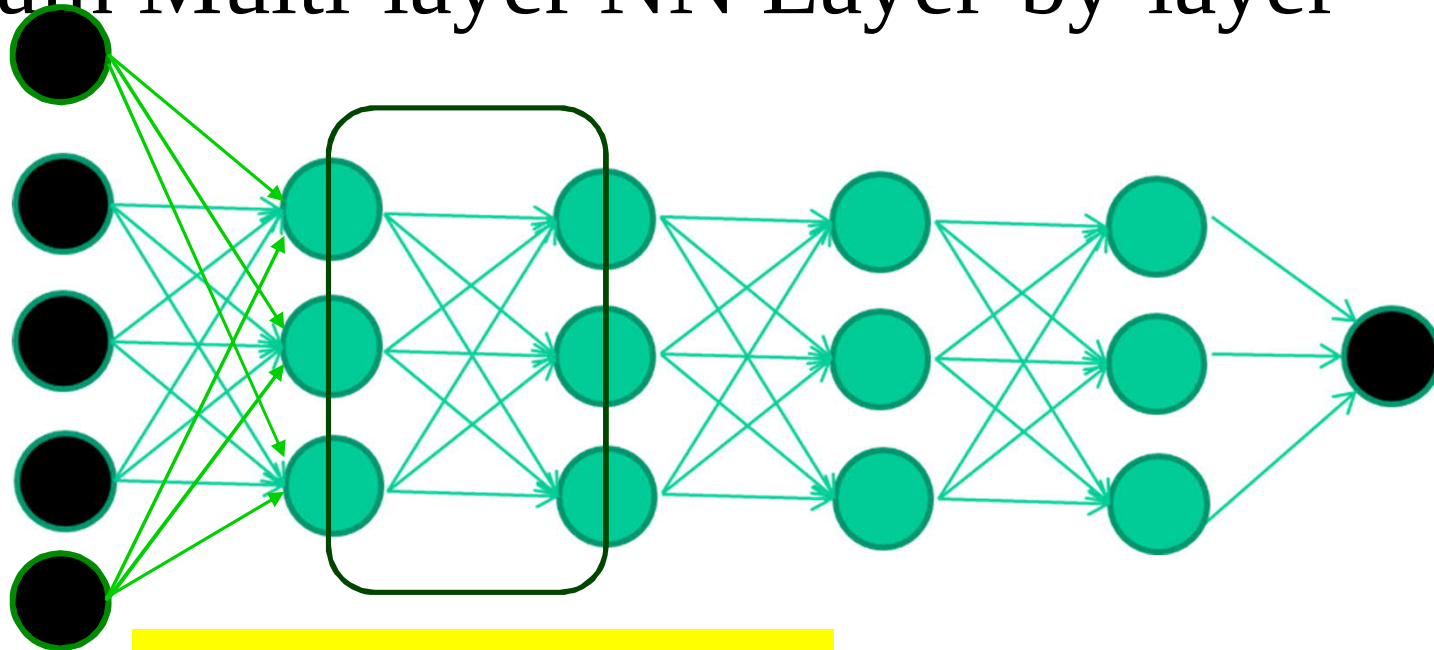
By making this happen with (many) fewer units than the inputs, this forces the ‘hidden layer’ units to become good feature detectors

# Pre-train Multi-layer NN Layer-by-layer



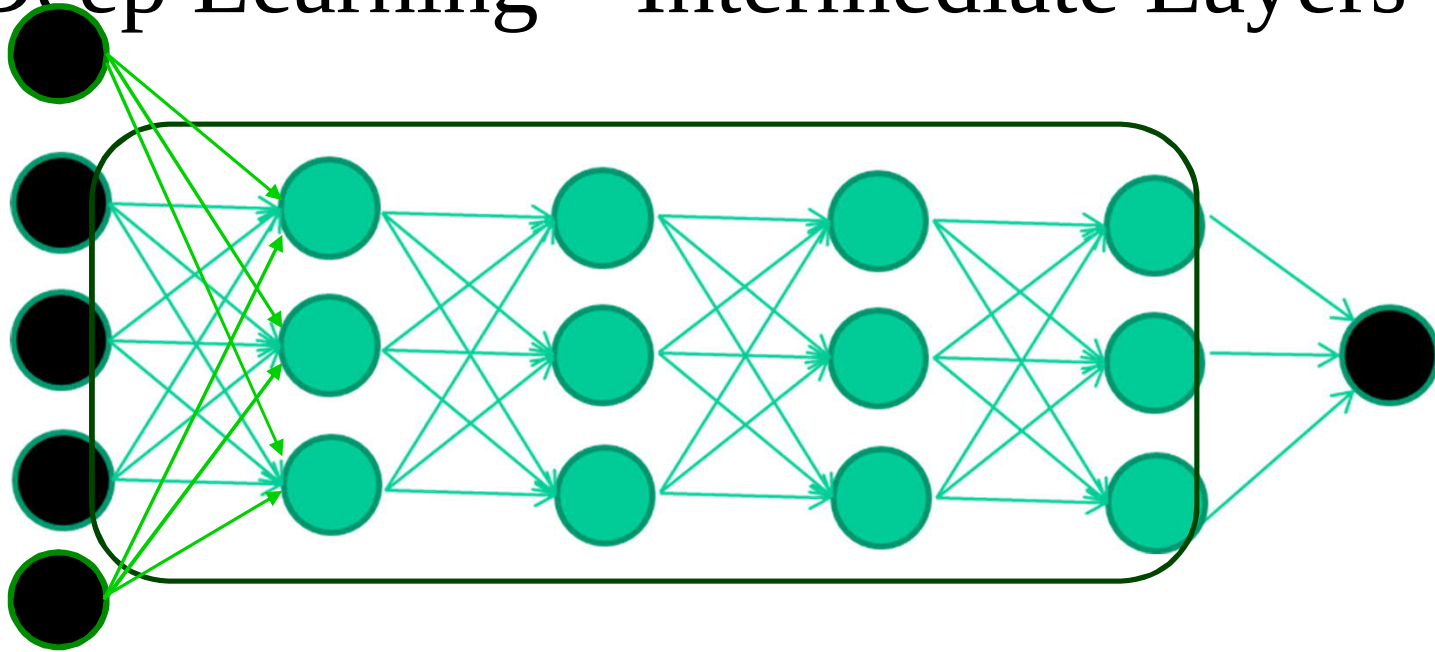
**Train this layer first  
using auto-encoder**

# Pre-train Multi-layer NN Layer-by-layer



**then training this layer  
using auto-encoder**

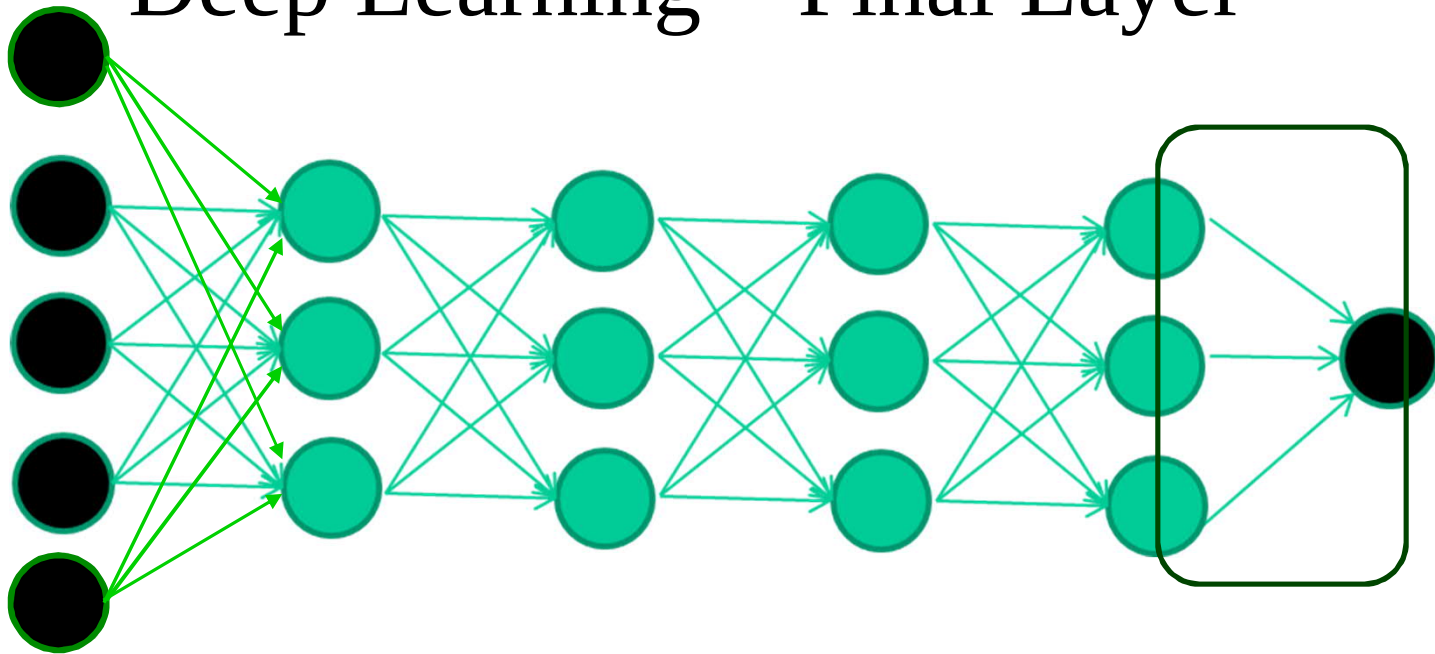
## Deep Learning – Intermediate Layers



Intermediate layers are each trained to be auto encoders (or similar)



## Deep Learning – Final Layer



Final layer is trained to predict class labels

- The weights for the previous layers are initialized with the weights trained by auto-encoders
- The final layer weights are initialized randomly
- The final training (with labeled data) updates all the weights using back-propagation.

# Deep Learning

- ▶ That's an early idea (2006)
- ▶ There have been
  - ▶ Different kinds of autoencoders, variations on architectures
  - ▶ Different weight initialization techniques, such as transfer learning, self-supervised learning, etc.
  - ▶ Many types of deep neural network structures
  - ▶ ...
- ▶ Very fast growing area ...

# Deep Learning Architectures

- ▶ Convolutional neural networks
- ▶ Residual neural networks
- ▶ Recurrent neural networks
  - ▶ LSTM, GRU
- ▶ Sequence-to-sequence model
- ▶ Transformer
- ▶ Big language models
  - ▶ BERT, GPT-3, SBERT, T5, BART, MT-NLG, ChatGPT, GPT-4 .....

# K-Nearest Neighbor Algorithm

---

Aijun An

Department of Electrical Engineering and Computer Science  
York University

---

# Outline

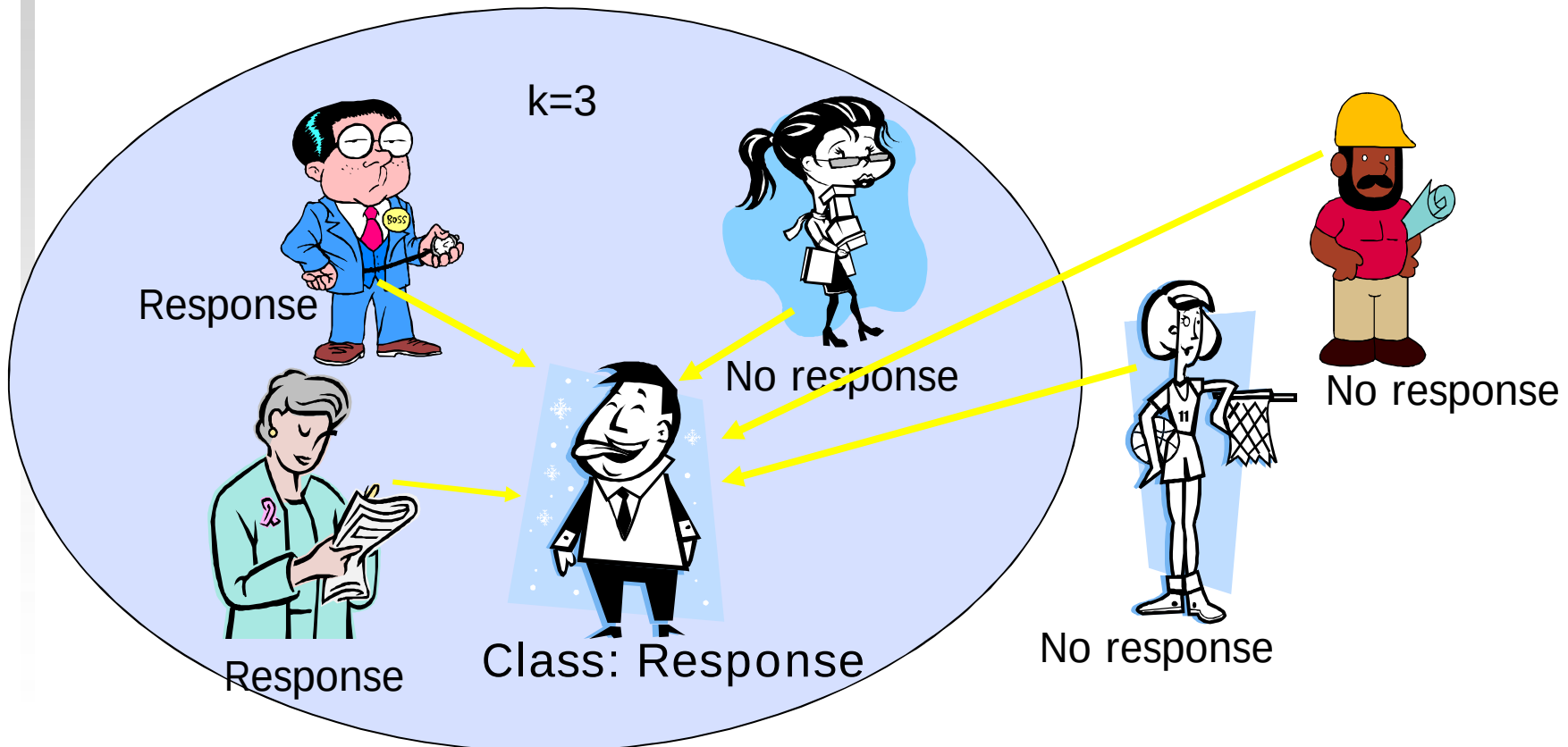
- ▶ KNN description with examples
- ▶ Distance measures
- ▶ Three issues and their solutions
- ▶ Pros and cons

# K-Nearest Neighbor Algorithm

- ▶ A supervised learning method
  - ▶ A set of *labeled* training examples is given
  - ▶ Can do both classification and regression
- ▶ It does not learn a model (no learning or induction)
- ▶ To make a prediction on a new example, it
  - ▶ finds  $k$  most similar examples in the training data
  - ▶ use these examples' target values to figure out the prediction for the new example

# K-Nearest Neighbor Classifier

- ▶ To determine the class of a new example E:
  - ▶ Calculate the distance between E and all examples in the training set
  - ▶ Select  $k$  nearest examples to E in the training set
  - ▶ Assign E to the most common class among its  $k$ -nearest neighbors



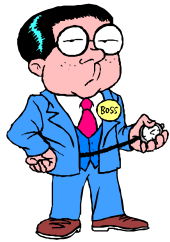
# K-Nearest Neighbor Learning

- ▶ **Instance/Memory-Based/Lazy Learning**
  - ▶ No model is built: Store all training examples
  - ▶ Any processing is delayed until a new instance must be classified.
- ▶ **Lazy vs. Eager Learning**
  - ▶ Lazy:
    - ▶ No training phase or model. Use training data during prediction
  - ▶ Eager:
    - ▶ Construct a generalization model from training data and use it for prediction
- ▶ **Inductive Assumption of KNN**
  - ▶ Similar inputs map to similar outputs
    - ▶ If not true, learning is impossible
    - ▶ If true, learning reduces to computing similarity/distance



# Similarity or Distance Measures

- ▶ Assume that each example is represented with a set of numerical attributes



John:  
Age=35  
Income=95K  
No. of credit cards=3



Rachel:  
Age=41  
Income=215K  
No. of credit cards=2

- ▶ “Closeness” can be measured by the ***Euclidean distance*** between two examples  $X = (x_1, x_2, x_3, \dots, x_n)$  and  $Y = (y_1, y_2, y_3, \dots, y_n)$ :

$$D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- ▶ Distance (John, Rachel):

$$\sqrt{(35-41)^2 + (95K-215K)^2 + (3-2)^2}$$

# Example : 3-Nearest Neighbors

| Customer                                                                                   | Age | Income | No. credit cards | Response |
|--------------------------------------------------------------------------------------------|-----|--------|------------------|----------|
| John      | 35  | 35K    | 3                | No       |
| Rachel    | 22  | 50K    | 2                | Yes      |
| Hannah    | 63  | 200K   | 1                | No       |
| Tom      | 59  | 170K   | 1                | No       |
| Nellie  | 25  | 40K    | 4                | Yes      |
| David   | 37  | 50K    | 2                | ?        |

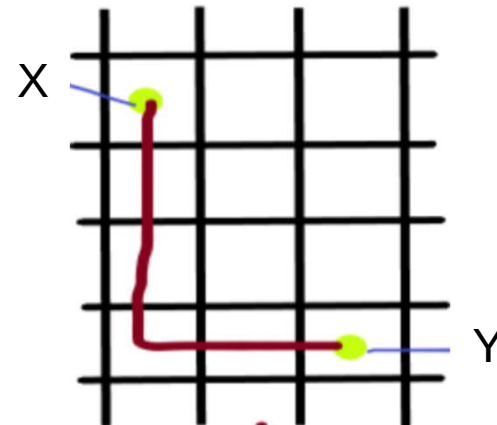
# Example (3-NN)

| Customer                                                                                   | Age | Income (K) | No. cards | Response | Distance from David                                       |
|--------------------------------------------------------------------------------------------|-----|------------|-----------|----------|-----------------------------------------------------------|
| John      | 35  | 35         | 3         | No       | $\text{sqrt} [(35-37)^2 + (35-50)^2 + (3-2)^2] = 15.16$   |
| Rachel    | 22  | 50         | 2         | Yes      | $\text{sqrt} [(22-37)^2 + (50-50)^2 + (2-2)^2] = 15$      |
| Hannah    | 63  | 200        | 1         | No       | $\text{sqrt} [(63-37)^2 + (200-50)^2 + (1-2)^2] = 152.23$ |
| Tom      | 59  | 170        | 1         | No       | $\text{sqrt} [(59-37)^2 + (170-50)^2 + (1-2)^2] = 122$    |
| Nellie  | 25  | 40         | 4         | Yes      | $\text{sqrt} [(25-37)^2 + (40-50)^2 + (4-2)^2] = 15.74$   |
| David   | 37  | 50         | 2         | Yes      |                                                           |

## Other Distance Measures (Numeric Attributes)

- ▶ **Manhattan distance:** Given two examples  $X = (x_1, x_2, x_3, \dots, x_n)$  and  $Y = (y_1, y_2, y_3, \dots, y_n)$ :

$$D(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^n |x_i - y_i|$$

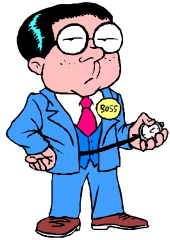


- ▶ **Cosine similarity:**

$$\text{Similarity}(\mathbf{X}, \mathbf{Y}) = \cos(\theta) = \frac{\mathbf{X} \cdot \mathbf{Y}}{\|\mathbf{X}\| \|\mathbf{Y}\|}$$

where  $\theta$  is the angle between  $\mathbf{X}$  and  $\mathbf{Y}$

# A Problem and its Solution



**John:**

Age=35

Income=35K

No. of credit cards=3



**Rachel:**

Age=22

Income=50K

No. of credit cards=2

$$\text{Distance (John, Rachel)} = \sqrt{(35-22)^2 + (35000-50000)^2 + (3-2)^2}$$

- ▶ Distance between examples could be dominated by some attributes with relatively large numbers (e.g., income in our example).
- ▶ Important to normalize features (e.g., map numbers to numbers between 0-1)
  - ▶ If Highest income = 200K and Lowest income=0, use Min-Max normalization:
    - John's income is normalized to 35/200
    - Rachel's income is normalized to 50/200

# KNN with Normalization of Variables

| Customer                                                                                   | Age            | Income (K)       | No. cards            | Response |
|--------------------------------------------------------------------------------------------|----------------|------------------|----------------------|----------|
| John      | $55/63 = 0.55$ | $35/200 = 0.175$ | $\frac{3}{4} = 0.75$ | No       |
| Rachel    | $22/63 = 0.34$ | $50/200 = 0.25$  | $2/4 = 0.5$          | Yes      |
| Hannah    | $63/63 = 1$    | $200/200 = 1$    | $\frac{1}{4} = 0.25$ | No       |
| Tom      | $59/63 = 0.93$ | $170/200 = 0.85$ | $\frac{1}{4} = 0.25$ | No       |
| Nellie  | $25/63 = 0.39$ | $40/200 = 0.2$   | $4/4 = 1$            | Yes      |
| David   | $37/63 = 0.58$ | $50/200 = 0.25$  | $2/4 = 0.5$          | Yes      |

# Another Problem and its Solution

- ▶ Distance works naturally with numerical attributes
- ▶ What if we have nominal attributes?

Example: Married

| Customer | Married | Income (K) | No. cards | Response |
|----------|---------|------------|-----------|----------|
| John     | Yes     | 35         | 3         | No       |
| Rachel   | No      | 50         | 2         | Yes      |
| Hannah   | No      | 200        | 1         | No       |
| Tom      | Yes     | 170        | 1         | No       |
| Nellie   | No      | 40         | 4         | Yes      |
| David    | Yes     | 50         | 2         |          |

# K-NN with Nominal Attributes

- ▶ Method 1:

- ▶ Convert a nominal attribute to a numerical attribute, for example:
  - ▶  $\text{yes} \Rightarrow 1$  and  $\text{no} \Rightarrow 0$
  - ▶  $\text{blue} \Rightarrow 1$ ,  $\text{yellow} \Rightarrow 2$ ,  $\text{red} \Rightarrow 3$ , etc.
- ▶ Problem?

- ▶ Method 2:

- ▶ Convert a nominal attribute to  $n$  binary attributes, where  $n$  is the number of values of the nominal attribute
  - ▶ For example, if the domain of attribute  $A$  is  $\{a_1, a_2, a_3\}$ , we convert it into 3 attributes to represent  $A$ .
    - ▶ If  $A=a_1$ , the values of the three attribute are 1, 0, 0.
    - ▶ If  $A=a_2$ , the values of the three attribute are 0, 1, 0.
    - ▶ If  $A=a_3$ , the values of the three attribute are 0, 0, 1.

*One-hot  
encoding*



# K-NN with Nominal Attributes

- ▶ Method 3:

- ▶ Do not convert, but change the distance calculation:

$$\text{Distance } (x, y) = \sum_{i=1}^m \text{dist}(x_i, y_i)$$

where  $m$  is the number of attributes, and

$$\text{dist}(x_i, y_i) = \begin{cases} 0 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i = y_i \\ 1 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i \neq y_i \\ | \text{norm}(x_i) - \text{norm}(y_i) | & \text{if } x_i \text{ and } y_i \text{ are continuous} \end{cases}$$

where  $|\text{norm}(x)|$  is a normalized value of  $x$ .

# Third Problem and its Solution

- ▶ Curse of dimensionality

- ▶ As the number of attributes increases, distance between examples becomes larger and more uniform.
- ▶ If the number of relevant attributes is fixed, increasing the number of less relevant attributes may make distance less reliable

$$D(X, Y) = \sqrt{\sum_{i=1}^{relevant} (x_i - y_i)^2 + \sum_{j=1}^{irrelevant} (x_j - y_j)^2}$$

- ▶ Solutions

- ▶ Feature selection
- ▶ Weighted distance with feature weights

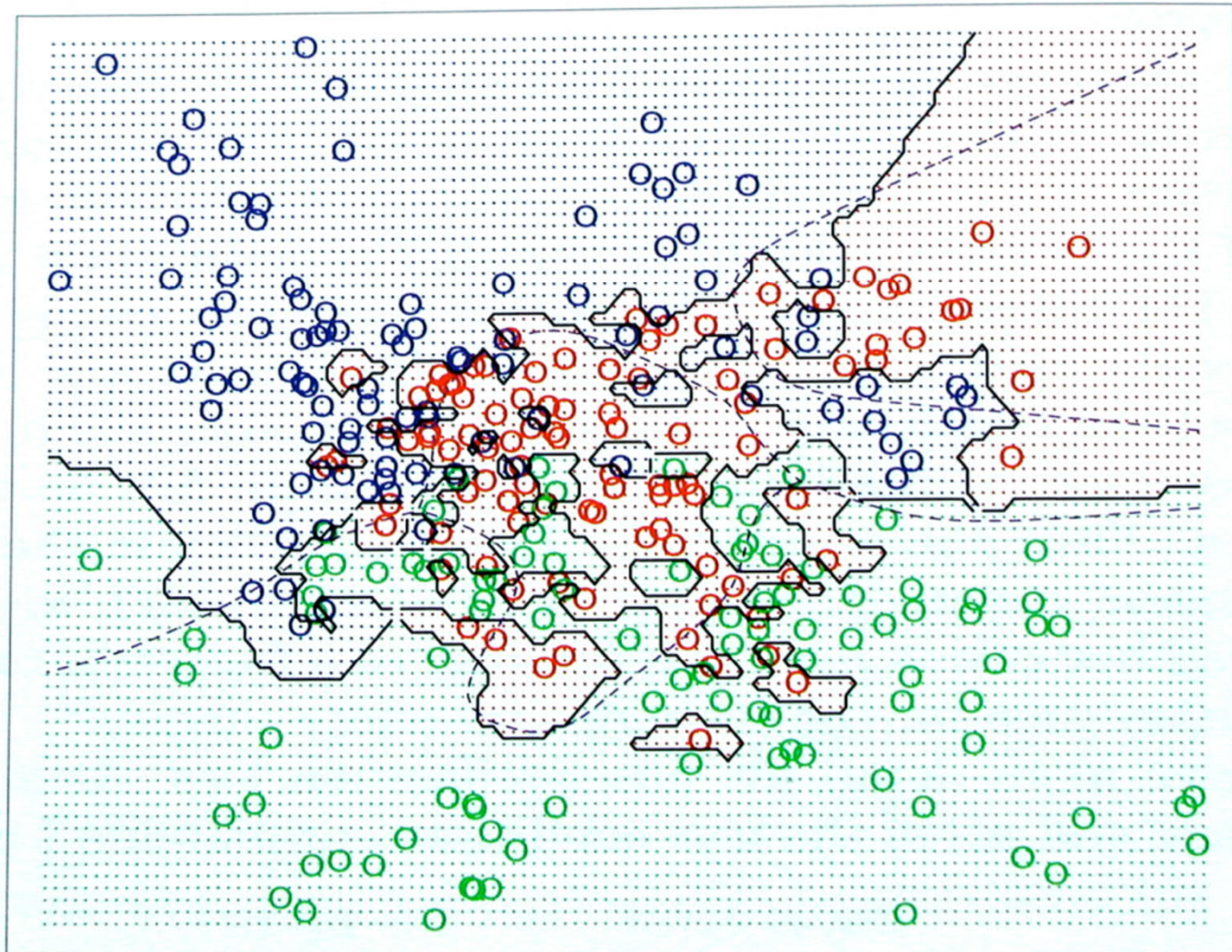
$$D(X, Y) = \sqrt{\sum_{i=1}^m w_i (x_i - y_i)^2}$$

# How to Choose $k$

- ▶ Small  $k$ :
  - ▶ Sensitive to noise (particularly class noise)
  - ▶ Captures fine structure of space better
  - ▶ May be necessary with small training sets
- ▶ Large  $k$ :
  - ▶ Less sensitive to noise
  - ▶ Better class probability estimates for the example to be classified
  - ▶ Large training sets allow larger values of  $k$
- ▶ Can use cross-validation on the training data to choose  $k$

# Class Boundaries

1-Nearest Neighbor

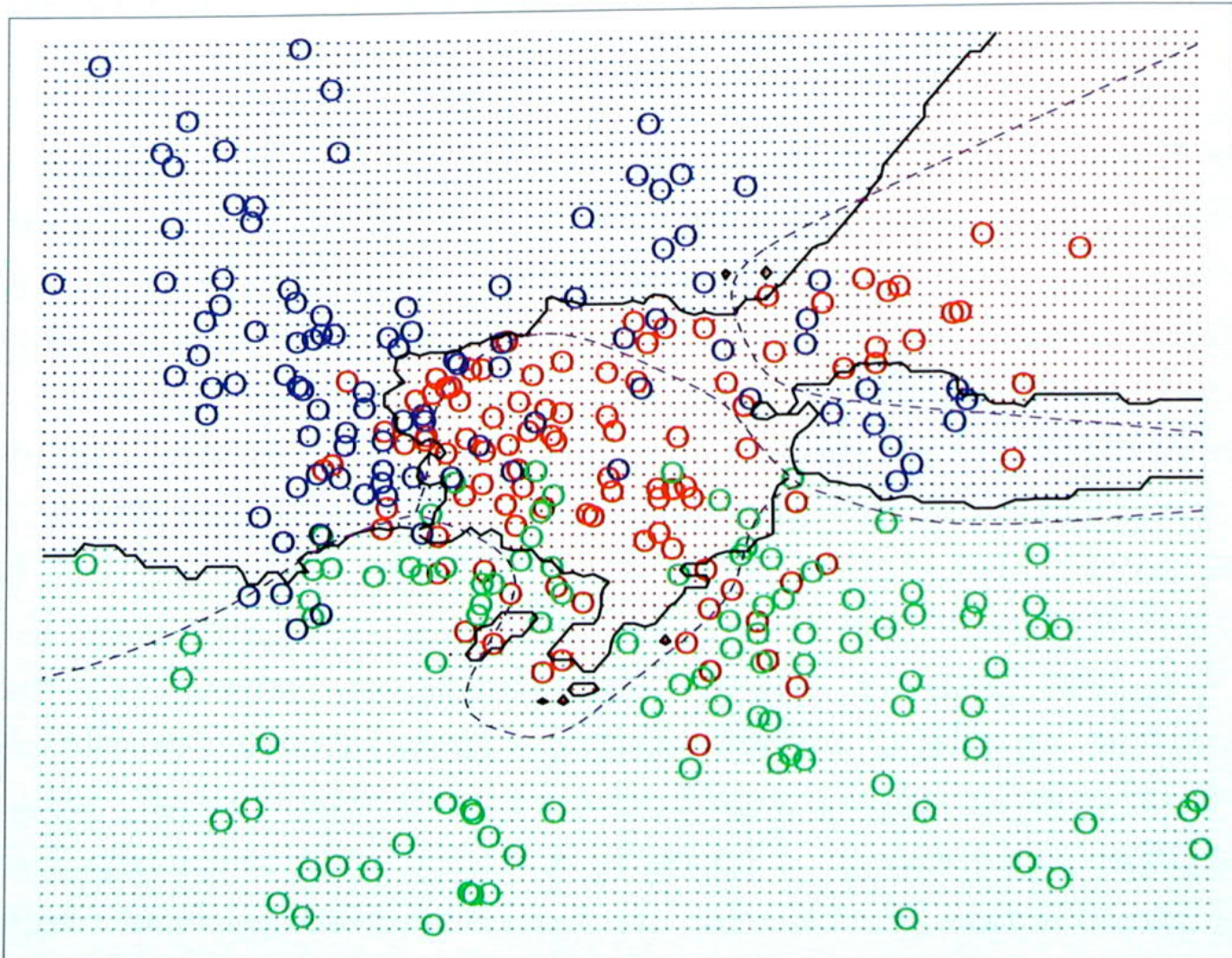


*From Hastie, Tibshirani, Friedman 2001 p418*



# Class Boundaries

15-Nearest Neighbors



*From Hastie, Tibshirani, Friedman 2001 p418*

# Strengths and Weaknesses

## ▶ Strengths:

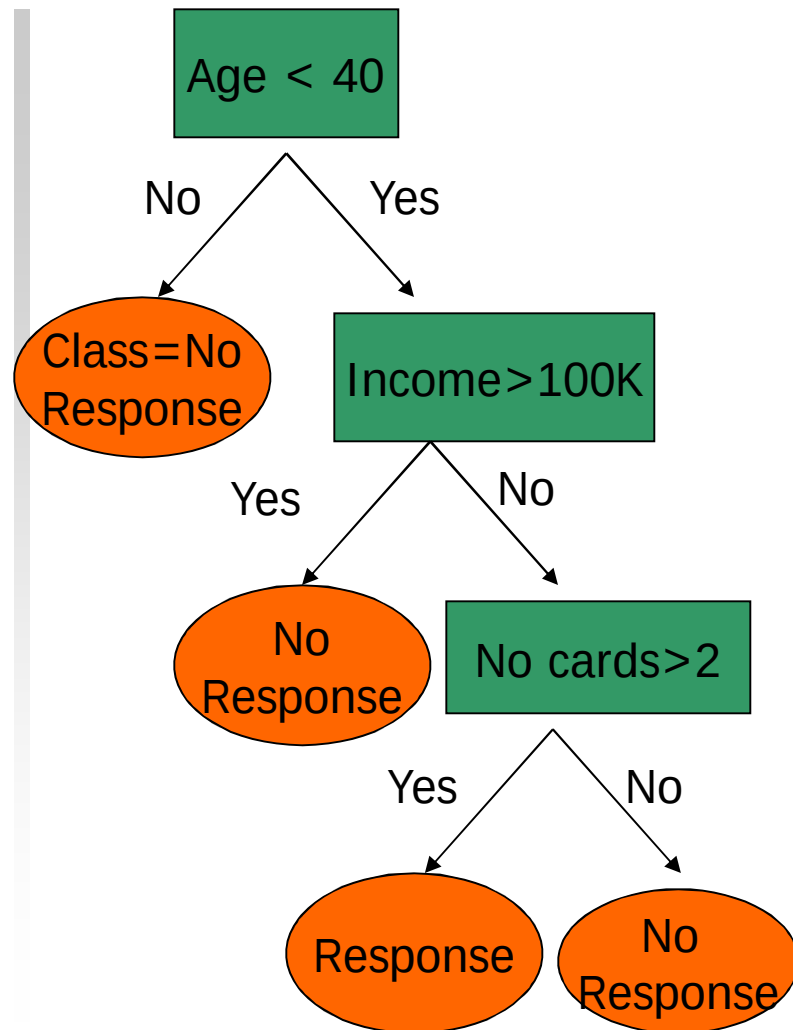
- ▶ Simple to implement and use
- ▶ Comprehensible – easy to explain prediction
- ▶ Robust to noisy data by averaging k-nearest neighbors.
- ▶ Can learn complex target functions
- ▶ Can be used for both classification and regression

## ▶ Weaknesses:

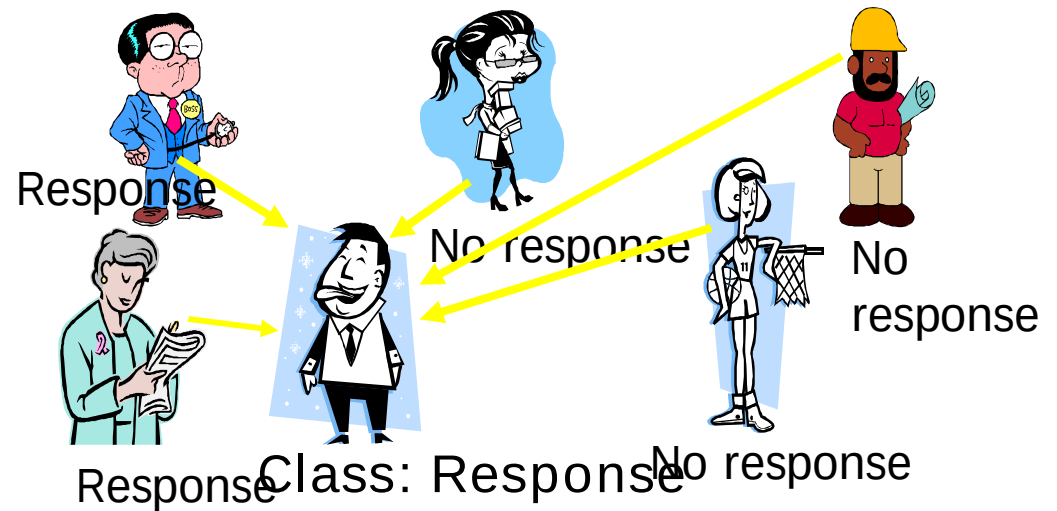
- ▶ Need a lot of space to store all examples.
- ▶ Takes more time to classify a new example than with a model
  - ▶ need to calculate and compare distance from new example to all other examples.

# Decision Tree vs K-Nearest Neighbor Classifier

## Classification Tree Model



## K-Nearest Neighbors



---

# Summary

- ▶ KNN description with examples
- ▶ Distance measures
- ▶ Three issues and their solutions
- ▶ Pros and cons



---

# Summary

- ▶ KNN description with examples
- ▶ A commonly used distance measure
- ▶ Three issues and their solutions
- ▶ Pros and cons

# Text Classification

---

Aijun An

Department of Electrical Engineering and Computer Science

York University

# Outline

- ▶ Introduction and applications
- ▶ Traditional methods
  - ▶ Text Representation
  - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)

# Text Mining

- ▶ It refers to data mining using text documents as data.
  - ▶ A text document could be an article, a web page, a product review, an xml file, an email message, a blog and so on.
- ▶ Tasks of text mining
  - ▶ Text classification
  - ▶ Text clustering
  - ▶ Text summarization
  - ▶ Topic detection
  - ▶ ...

# Text Classification

- ▶ Learn a classification model from a set of pre-classified documents
- ▶ Classify new text documents using the learned model
- ▶ Applications
  - ▶ Classify articles into categories
  - ▶ Classify web pages into different categories
  - ▶ Classify emails into different categories
  - ▶ Spam email filtering
  - ▶ ...

# Example Applications

- ▶ News topic classification (e.g., Google News)  
 $C = \{\text{politics, sports, business, health, tech, ...}\}$
- ▶ Sentiment classification  
 $C = \{\text{positive review, negative review, neutral review}\}$
- ▶ “SafeSearch” filtering  
 $C = \{\text{pornography, not pornography}\}$
- ▶ Language classification  
 $C = \{\text{English, Spanish, Chinese, ...}\}$
- ▶ Email sorting  
 $C = \{\text{spam, meeting reminders, invitations, ...}\}$  – user-defined!

# Text Representation

- ▶ Most classification learning programs require the examples to be represented as a tuple, which is a vector of attribute values.
- ▶ How to represent a document using a vector of attribute values?
- ▶ Text Representation methods
  - ▶ Traditional methods
    - ▶ Bag of words
  - ▶ Neural-network based embedding methods

# Traditional Methods

## ▶ Attributes

- ▶ “Bag of words” method: Use a set of words as attributes

## ▶ Attribute values

- ▶ Method 1: use 0 or 1 as attribute value to indicate whether the word appears in the document.
- ▶ Method 2: use the absolute or relative frequency of each word in the document as the attribute value.
- ▶ Method 3: assign a weight to a word in a document using TF-IDF and use the weight as the attribute value



# Text Representation (*Cont'd*)

Training data sets:

► Method 1:

|                             | <b>word<sub>1</sub></b> | <b>word<sub>2</sub></b> | <b>...</b> | <b>word<sub>m</sub></b> | <b>Class</b> |
|-----------------------------|-------------------------|-------------------------|------------|-------------------------|--------------|
| <b>document<sub>1</sub></b> | 0                       | 1                       | ...        | 1                       | C1           |
| <b>document<sub>2</sub></b> | 1                       | 0                       | ...        | 1                       | C2           |
| ...                         | ...                     | ...                     | ...        | ...                     | ...          |
| <b>document<sub>n</sub></b> | 1                       | 0                       |            | 0                       | C2           |

► Method 2 with absolute term frequency:

|                             | <b>word<sub>1</sub></b> | <b>word<sub>2</sub></b> | <b>...</b> | <b>word<sub>m</sub></b> | <b>Class</b> |
|-----------------------------|-------------------------|-------------------------|------------|-------------------------|--------------|
| <b>document<sub>1</sub></b> | 0                       | 3                       | ...        | 1                       | C1           |
| <b>document<sub>2</sub></b> | 2                       | 0                       | ...        | 3                       | C2           |
| ...                         | ...                     | ...                     | ...        | ...                     | ...          |
| <b>document<sub>n</sub></b> | 5                       | 0                       |            | 0                       | C2           |

# Method 3: TF-IDF Term Weighting

## ▶ TF: term frequency

▶ Definition:  $TF = t_{ij}$

▶ frequency of term  $i$  in document  $j$

▶ Purpose: makes the frequent words *for the document* more important

## ▶ IDF: inverted document frequency

▶ Definition:  $IDF = \log(N/n_i)$

▶  $n_i$  : number of documents containing term  $i$

▶  $N$  : total number of documents

▶ Purpose: makes rare words *across documents* more important

## ▶ TF-IDF value of a term $i$ in document $j$

▶ Definition:  $TF \times IDF = t_{ij} \times \log(N/n_i)$

# Example: TF-IDF Weighted Vectors

Assume there are three documents in the training set:

Document D1: “yes we got no bananas”

Document D2: “what you got”

Document D3: “yes I like what you got”

|     | yes  | we   | got | no   | bananas | what | you  | I    | like |
|-----|------|------|-----|------|---------|------|------|------|------|
| D1: | .18  | 0.48 | 0   | 0.48 | 0.48    | 0    | 0    | 0    | 0    |
| D2: | 0    | 0    | 0   | 0    | 0       | 0.18 | 0.18 | 0    | 0    |
| D3: | 0.18 | 0    | 0   | 0    | 0       | 0.18 | 0.18 | 0.48 | .48  |

## Other versions of TF-IDF

- ▶ **Using Relative Document Frequency:**

$$W_{td} = (tf_{td} / |d|) \times \log(N/df_i)$$

where  $|d|$  is the length of the document  $d$  (i.e., the number of terms the document contains)

- ▶ **Logarithmically Scaled Frequency:**

$$W_{t,d} = \log(1 + tf_{t,d}) \times \log(N/df_t)$$

- ▶ 1 is added in the first term to prevent  $\log(0)$ .

- ▶ Both prevent bias towards longer documents:

# Text Processing for Selecting the Bag of Words (BOW)

- ▶ Word (token) extraction
  - ▶ Extract all the words in a document
  - ▶ Convert them into lower cases
- ▶ Stop words removal
- ▶ Stemming or Lemmatization
- ▶ Selecting words

---

# Stop Words

- ▶ Many of the most frequently used words in English are worthless in text mining – these words are called *stop words*.
- ▶ Examples of stop words  
*the, of, and, to, a, ...*
- ▶ Typically about 400 to 500 such words
- ▶ For an application, there may be additional domain-specific stop words
- ▶ These stop words are often removed from the set of words when representing a document with BOW

# Stemming

- ▶ A technique used to find the root/stem of a word.

- ▶ For example:

- ▶ discussed
- ▶ discusses
- ▶ discussing
- ▶ discuss

Stem: discuss

- ▶ Usefulness

- ▶ Reduce the number of words
- ▶ Improve effectiveness of text classification

# Example Stemming Rules

- ▶ Remove ending
  - ▶ If a word ends with *s*, preceded by a consonant other than an *s*, then delete the *s*.
  - ▶ If a word ends with *ed*, preceded by a consonant, delete the *ed* unless this leaves only a single letter.
- ▶ Transform words
  - ▶ If a word ends with “ies” but not “eies” or “aies”, then “ies” is replaced with “y”.



# Stemming Algorithms

- ▶ Porter stemming algorithm
  - ▶ The most widely used stemming algorithm
  - ▶ Developed by Martin Porter at the University of Cambridge in 1980
  - ▶ <http://www.tartarus.org/~martin/PorterStemmer/> contains source codes in a few languages
- ▶ Other stemming algorithms
  - ▶ Lancaster Stemmer
    - ▶ more aggressive than the Porter Stemmer. It can sometimes over-stem
  - ▶ Snowball Stemmer:
    - ▶ an improvement of the Porter Stemmer

# Lemmatization

- ▶ Similar to Stemming
  - ▶ Reducing words to their base or root form,
- ▶ Different from Stemming
  - ▶ Takes into account the vocabulary and morphological analysis of words to remove inflectional endings only
  - ▶ Return **the base or dictionary form** of a word, known as “**lemma**”.
  - ▶ Thus, a lemma is a valid word in the vocabulary, while stemming may result in a stem that is not a valid word
- ▶ More sophisticated than stemming
  - ▶ requires a comprehensive vocabulary and morphological analysis to accurately determine the lemma

# Lemmatization

## ► Examples:

1. Running:
  1. Stemming (e.g., using Porter stemmer): "runn"
  2. Lemmatization: "run"
2. Better:
  1. Stemming: "better"
  2. Lemmatization: "good"
3. Mice:
  1. Stemming: "mice"
  2. Lemmatization: "mouse"
4. Feet:
  1. Stemming: "feet"
  2. Lemmatization: "foot"
5. Flying:
  1. Stemming: "fly"
  2. Lemmatization: "fly"
6. Quickly:
  1. Stemming: "quickli"
  2. Lemmatization: "quickly"

---

# Text Processing for Selecting the Bag of Words

- ▶ Word (token) extraction
  - ▶ Extract all the words in a document
  - ▶ Convert them into lower cases
- ▶ Stop words removal
- ▶ Stemming or lemmatization
- ▶ *Selecting words*

# Feature Selection

- ▶ Selecting the “bag of words” to represent documents
- ▶ Why do we need to select?
  - ▶ The number of unique words in a set of documents can be too many.
  - ▶ Learning program may not be able to handle all possible features
  - ▶ Good features can result in higher accuracy

# What are Good and Bad Features?

- ▶ Good features: (should be kept)
  - ▶ Co-occur with a particular category
  - ▶ Do not co-occur with other categories
- ▶ Bad features: (best to remove)
  - ▶ Uniform across all categories
  - ▶ Very infrequent (appear 1 or 2 times in the whole training set of documents)
    - ▶ unlikely to be met again
    - ▶ can be noise
    - ▶ co-occurrence with a class can be due to chance

# Feature Selection Methods

- ▶ Class independent methods (Unsupervised)
  - ▶ Document Frequency (DF)
  - ▶ Term Strength (TS)
- ▶ Class-dependent methods (Supervised)
  - ▶ Information Gain (IG)
  - ▶ Mutual Information (MI)
  - ▶  $\chi^2$  statistic (CHI)

# Document Frequency (DF)

- ▶ Document frequency of a word  $w$ :

$DF(w)$  = number of documents containing  $w$

- ▶ Rank the words according to their document frequency
- ▶ Select the first  $m$  words with high DF values



---

# Document Frequency (*Cont'd*)

- ▶ Advantages

- ▶ Easy to compute
- ▶ Can remove rare words (hence noise)

- ▶ Disadvantages

- ▶ **Class independent:**

- ▶ If the word appears frequently in many classes, it cannot distinguish the classes well
- ▶ Some infrequent terms can be good discriminators, which cannot be selected by this method.

# Information Gain/Mutual Information

- ▶ A measure of importance of the feature for predicting the classes of documents
- ▶ Defined as:
  - ▶ The number of “bits of information” gained by knowing the word is present or absent

$$\begin{aligned} \text{Gain}(w) = & -\sum_{i=1}^k P(C_i) \log P(C_i) \\ & + P(w) \sum_{i=1}^k P(C_i | w) \log P(C_i | w) + P(\bar{w}) \sum_{i=1}^k P(C_i | \bar{w}) \log P(C_i | \bar{w}) \end{aligned}$$

where  $w$  is a word and  $C_1, C_2, \dots, C_k$  are classes.

- ▶ Rank the words according to their information gain value
- ▶ Select the first  $m$  words with high gain values

# Information Gain (*Cont'd*)

- ▶ Advantage:
  - ▶ Consider the classes
- ▶ Disadvantage:
  - ▶ Computationally expensive (compared to using DF)
  - ▶ Noisy words occurring only once in the document collection have high IG
- ▶ Solution
  - ▶ Remove rare words (appears 1 or 2 times) first. This can
    - ▶ reduce the amount of computation, and
    - ▶ remove noisy words that have by-chance correlations with the classes.

# What Do People Do In Practice with the Bag of Words Representation Method?

- ▶ Rare term removal
  - ▶ rare across the whole collection (i.e. DF is very low)
  - ▶ met in a single document
- ▶ Most frequent term removal (i.e. removing stop words) (*often*)
- ▶ Stemming. (*often*)
- ▶ Use a class-dependent method (e.g., the information gain, chi-square method) to select features.
- ▶ After the BOW presentation, any classification method can be used to learn a classification model.

---

# Limitations of Bag of Words

- ▶ Cannot model phrases
  - ▶ “New York” is split into “New” and “York”
- ▶ Does not consider semantic relationships between words
  - ▶ “happy” and “delighted” have similar meanings
  - ▶ “old bike” vs “used bike”
- ▶ Does not consider the position or order of words in a document
  - ▶ “this is interesting” vs “is this interesting”
- ▶ Does not consider the context a word is in
  - ▶ “Bank of Montreal” vs “river bank”

# Beyond Bag of Words

- ▶ N-gram document representation
  - ▶ Bigram
    - a pair of consecutive words in the document
  - ▶ N-gram
    - a sequence of  $n$  consecutive words in the document
  - ▶ Document representation
    - Use a set of bi/n-grams to represent documents, or
    - Add them to the bag of words
  - ▶ **Pros**: can represent phrases
  - ▶ **Cons**: WAY too many features to deal with, and many are very sparse.
- ▶ Document representation using *neural embeddings*

---

# Outline

- ▶ Introduction and applications
- ▶ Traditional methods
  - ▶ Text Representation
  - ▶ Text Preprocessing Steps
- ▶ *Advanced techniques for text representation*
  - ▶ *Neural embeddings*
  - ▶ *Text classification with embeddings*

# Word Embedding

- ▶ A type of word representation using vectors
  - ▶ Embed words into a vector space
  - ▶ The vector of a word is called its embedding

A 4-dimensional embedding

|        |     |      |      |     |
|--------|-----|------|------|-----|
| cat => | 1.2 | -0.1 | 4.3  | 3.2 |
| mat => | 0.4 | 2.5  | -0.9 | 0.5 |
| on =>  | 2.1 | 0.3  | 0.1  | 0.4 |
| ...    |     |      |      |     |

- ▶ Given a vocabulary of words
  - ▶ The embedding of the  $i^{th}$  word in the vocabulary is a  *$d$ -dimensional vector*:

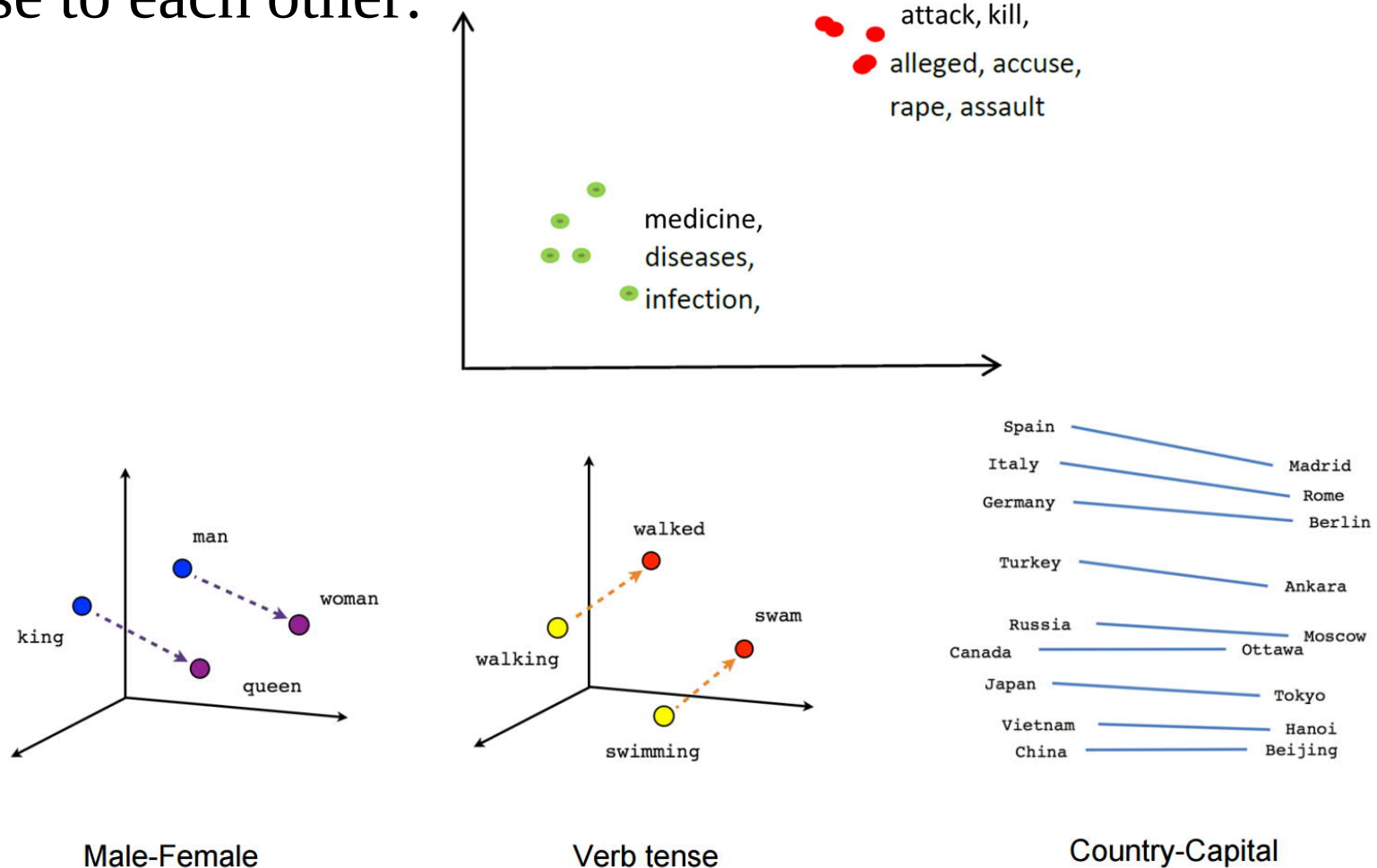
$$w_i \in \mathbb{R}^d$$

where  $d$  is typically in the range 50 to 1000.



# Important Feature of Word Embedding

- ▶ In an embedding space, **semantically similar** words are close to each other:



# Benefits of Word Embedding

- ▶ Word embeddings is better than one-hot encoding
- ▶ One-hot encoding:
  - ▶ Large dimension: size of vocabulary (could be tens of thousands)
  - ▶ Sparse
  - ▶ Words are considered independent (no semantics is encoded)

```
Rome = [1, 0, 0, 0, 0, 0, ..., 0]
Paris = [0, 1, 0, 0, 0, 0, ..., 0]
Italy = [0, 0, 1, 0, 0, 0, ..., 0]
France = [0, 0, 0, 1, 0, 0, ..., 0]
...
```

- ▶ Word embedding

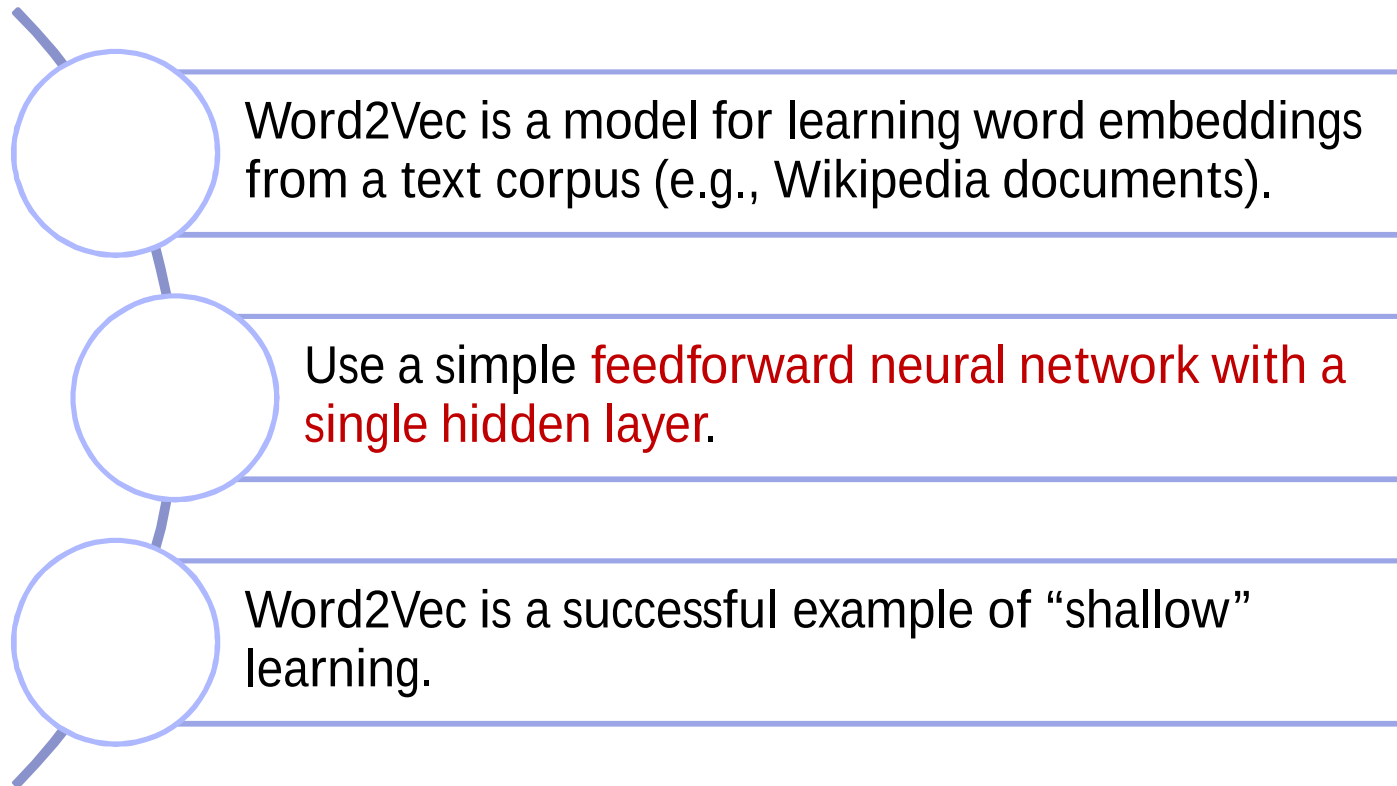
- ▶ Low dimensionality: typically 50 – 1000
- ▶ Dense and distributed
- ▶ Semantically related words are close to each other.

```
Rome = [1.73  0.73 -0.90 -0.62 0.12 -1.35 ...]
Paris = [0.77  1.18 -1.12 -0.75 -0.60 -1.05 ...]
Italy = [1.77 -0.78 -0.95 0.33 0.04 -0.09 ...]
France = [0.67 0.30 -1.05 -0.70 -0.78 0.00 ...]
```

# Main Methods for Word Embedding

- ▶ Latent Semantic Indexing (Deerwester et al., '88).
- ▶ Neural Net Language Models (NN-LMs) (Bengio et al., '06)
- ▶ Convolutional Nets for tagging (SENNA) (Collobert & Weston, '08).
- ▶ Supervised Semantic Indexing (Bai et al, '09).
- ▶ Wsabee (Weston et al., '10).
- ▶ Recurrent NN-LMs (Mikolov et al., '10).
- ▶ Recursive NNs (Socher et al., '11).
- ▶ **Word2Vec** (Mikolov et al., '13).
- ▶ GloVe (Pennington et al., '14)
- ▶ BERT (Devlin et al., '18)
- ▶ ROBERTa (Meta AI' 19)
- ▶ .....

# Characteristics of Word2Vec



# Two Learning Architectures of Word2Vec

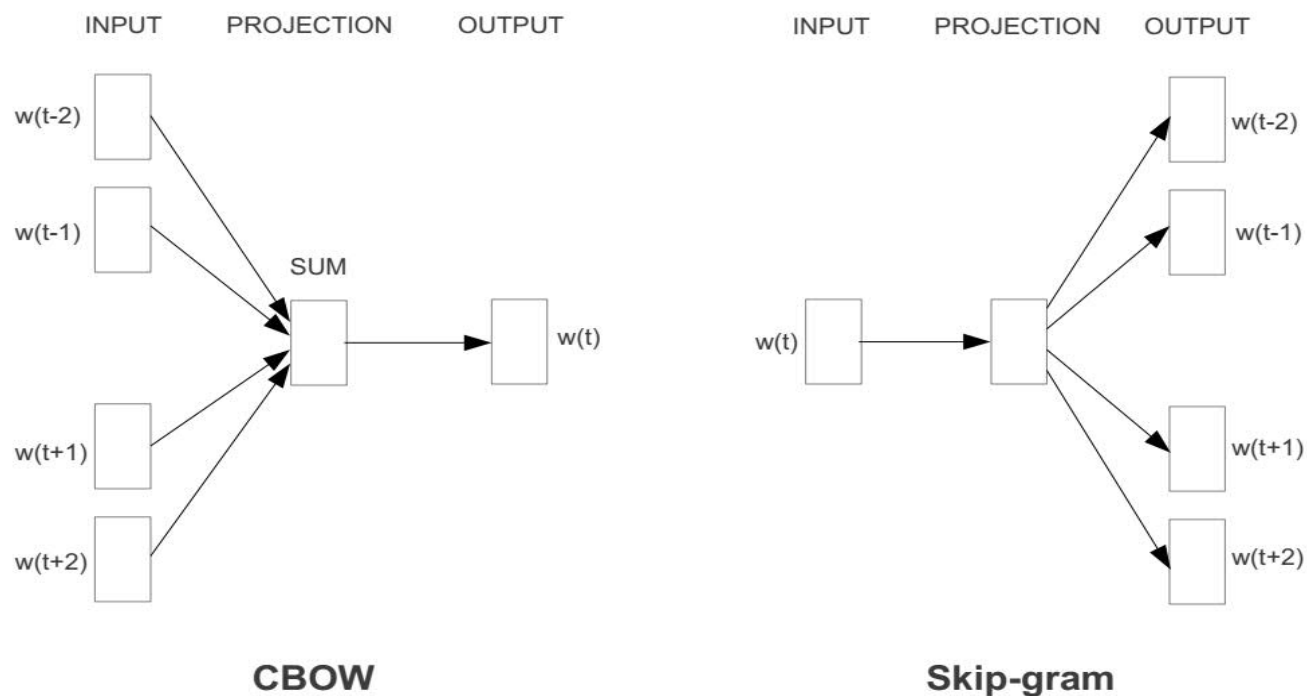
- ▶ Continuous Bag-of-Words (CBOW)
  - ▶ Model is trained to predict **target** words from **context** words
- ▶ Skip-gram
  - ▶ Model is trained to predict **context** words from **target** words

Snapshot of the **training text corpus** (for *context window size* = 2):

... The **cute** **cat** **jumps** **over** **the** lazy dog ...

$w(t-2)$   $w(t-1)$   $w(t)$   $w(t+1)$   $w(t+2)$   
context words target word context words

# Word2Vec: Two Learning Architectures



Snapshot of the training text corpus (for context window size = 2):

... The cute cat jumps over the lazy dog ...

$w(t-2)$   $w(t-1)$   $w(t)$   $w(t+1)$   $w(t+2)$   
context words target word context words

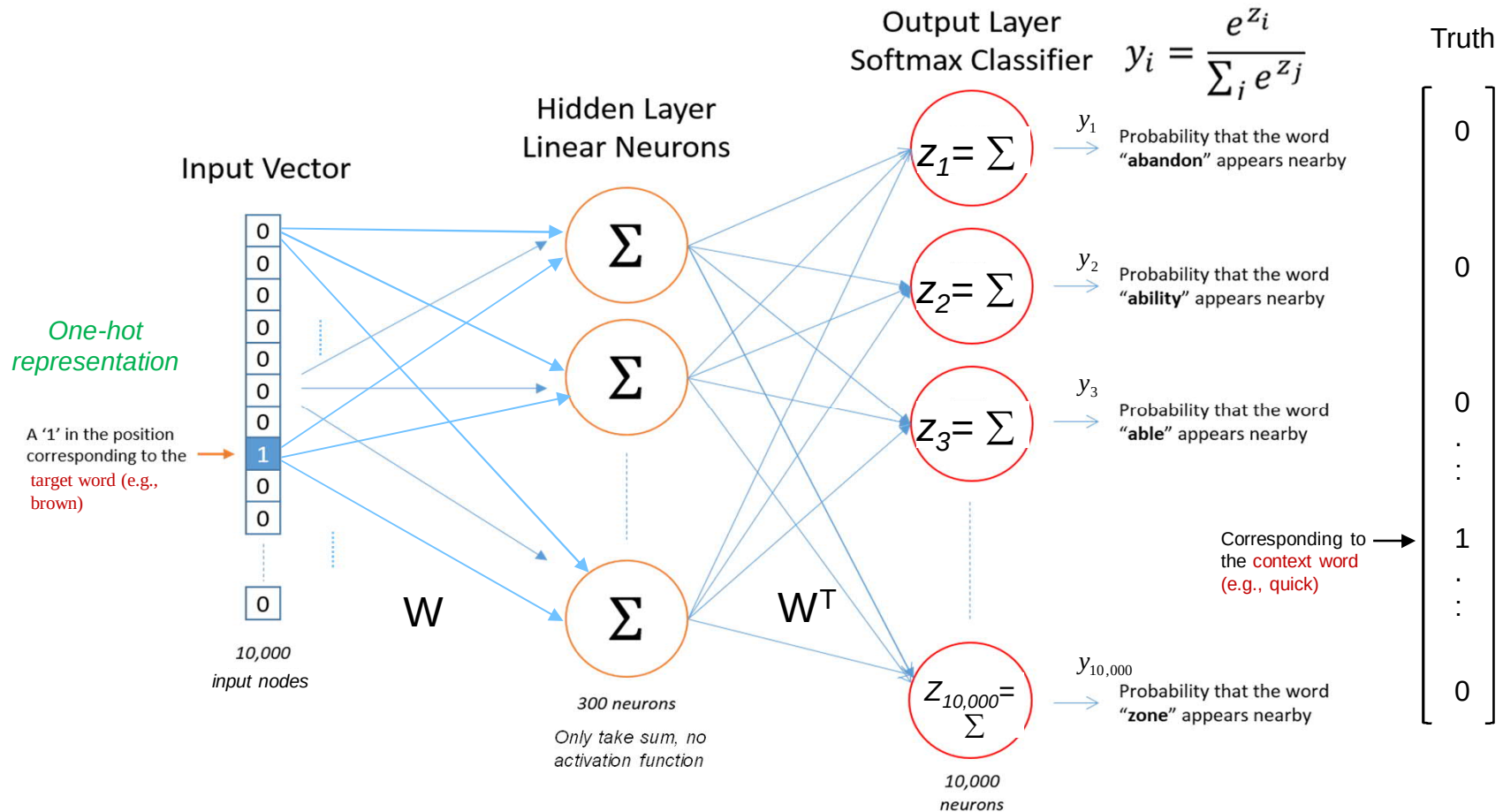
# Skip-gram

- ▶ Learn to predict the context words from a target word based on a training corpus
  - ▶ Target word moves from the beginning to the end of the corpus
  - ▶ Context words are the  $k$  words before or after the target word ( $k$  is the context window size)
  - ▶ Training data contains (target word, context word) pairs:

| Source Text                                                                                                     | Training Samples |       |       |                                                                  |      |                                                             |
|-----------------------------------------------------------------------------------------------------------------|------------------|-------|-------|------------------------------------------------------------------|------|-------------------------------------------------------------|
| <table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡                 | The              | quick | brown | (the, quick)<br>(the, brown)                                     |      |                                                             |
| The                                                                                                             | quick            | brown |       |                                                                  |      |                                                             |
| The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡                 | quick            | brown | fox   | (quick, the)<br>(quick, brown)<br>(quick, fox)                   |      |                                                             |
| quick                                                                                                           | brown            | fox   |       |                                                                  |      |                                                             |
| The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡                 | brown            | fox   | jumps | (brown, the)<br>(brown, quick)<br>(brown, fox)<br>(brown, jumps) |      |                                                             |
| brown                                                                                                           | fox              | jumps |       |                                                                  |      |                                                             |
| The <table><tr><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡ | quick            | brown | fox   | jumps                                                            | over | (fox, quick)<br>(fox, brown)<br>(fox, jumps)<br>(fox, over) |
| quick                                                                                                           | brown            | fox   | jumps | over                                                             |      |                                                             |

# Skip-gram Model

Assume there are 10,000 words in the vocabulary:

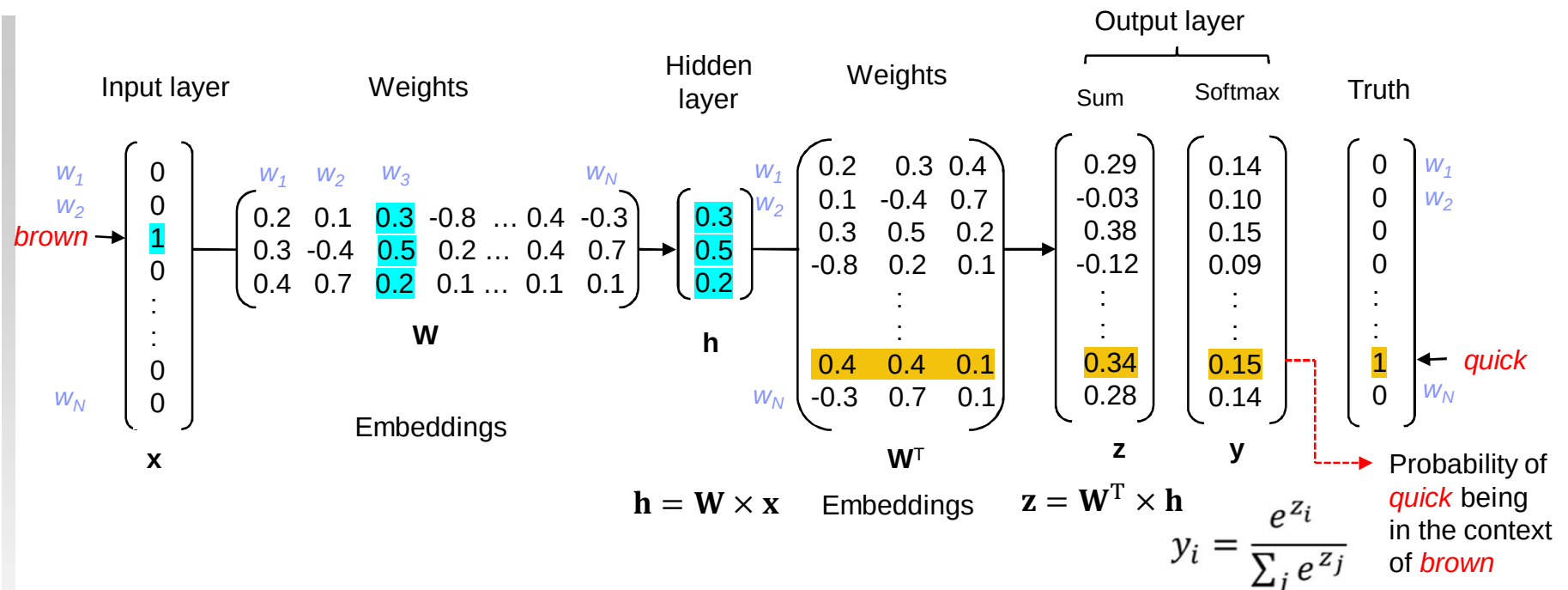


- After training, the weights are the word embeddings
- Dimension of the word vector is the number of neurons in the hidden layer



# Skip-gram Model Example

Assume training example is (*brown*, *quick*) and embedding dimension is 3:

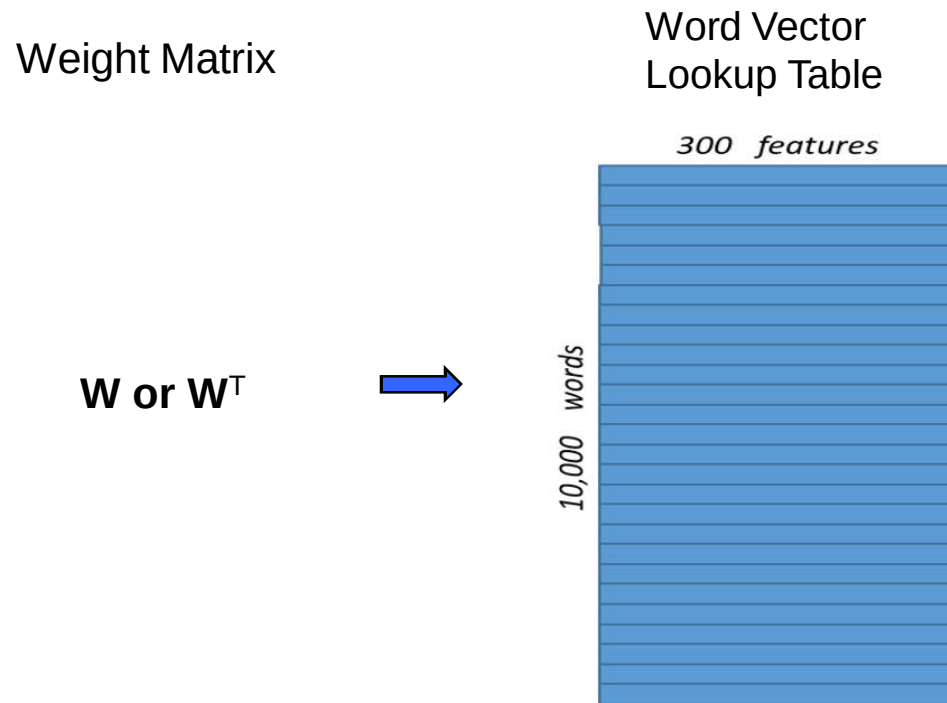


$w_i$  represents  $i$ th word in the vocabulary

- $z_i$  is the dot product of the target and context word embeddings, representing the similarity between the two words
- Such training tries to make words close to each other in the text corpus have similar embeddings

# Word Vectors: weights

Assume there are 10,000 words in the vocabulary and 300 hidden neurons:



Dimension of the word vector is the number of neurons in the hidden layer

# Skip-gram Model with Negative Sampling

- ▶ Since # of output nodes is huge, computation cost is high
- ▶ Negative sampling
  - ▶ Randomly select a small set of non-context words as “negative examples”
  - ▶ Training data contains:
    - ▶ Positive examples:
      - ▶ (target word, context word) pairs
    - ▶ Negative examples:
      - ▶ (target word, random non-context word) pairs

# Skip-gram Model with Negative Sampling

## ▶ Example

The quick brown fox jumps over the lazy dog.

### ▶ Positive examples for target word “brown”:

(brown, the)

(brown, quick)

(brown, fox)

(brown, jumps)

### ▶ Negative examples might be:

(brown, by)

(brown, target)

(brown, forever)

(brown, well)

.....

# Skip-gram Model with Negative Sampling

- ▶ Use Sigmoid as the activation function at the output layer
  - ▶ Each node is trained to output the probability of the corresponding word in the context of the target word.
- ▶ Give a positive example and a few negative examples (5-20) with the same target word

Positive: (t, c)

Negative: (t, r<sub>1</sub>), (t, r<sub>2</sub>), ..., (t, r<sub>k</sub>)

- ▶ Only the output nodes corresponding to the context word c and random words r<sub>1</sub>, r<sub>2</sub>, ... r<sub>k</sub> are activated and used in backpropagation training
- ▶ Loss function:

$$L(\theta) = -P(\mathbf{c}|\mathbf{t}) + \frac{1}{k} \sum_{i=1}^k P(\mathbf{r}_i|\mathbf{t})$$

$$P(\mathbf{c}|\mathbf{t}) = \text{sigmoid}(\mathbf{c} \cdot \mathbf{t})$$

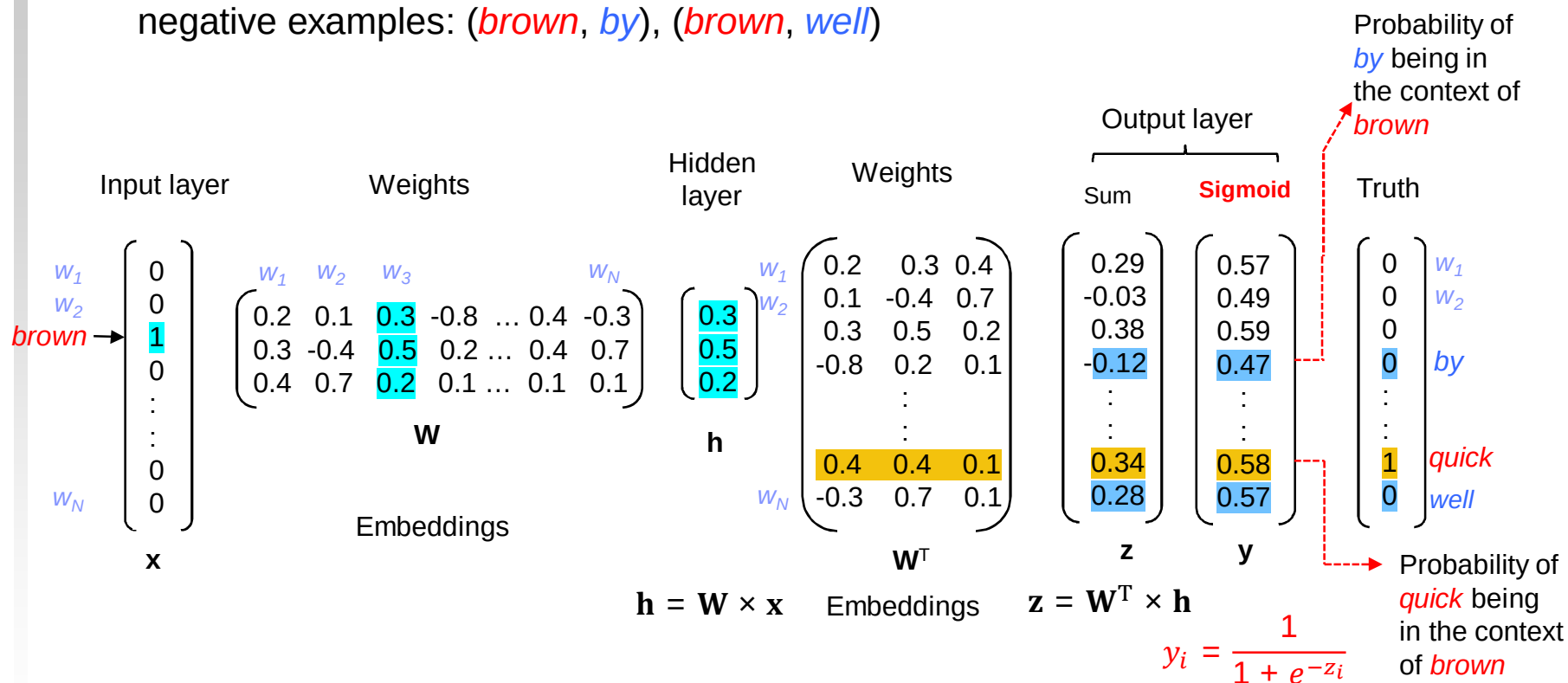
$$P(\mathbf{r}_i|\mathbf{t}) = \text{sigmoid}(\mathbf{r}_i \cdot \mathbf{t})$$

# Skip-gram Model with Negative Sampling

Assume:

positive example: (*brown*, *quick*)

negative examples: (*brown*, *by*), (*brown*, *well*)



$$L(\theta) = -P(\mathbf{c}|\mathbf{t}) + \frac{1}{k} \sum_{i=1}^k P(\mathbf{r}_i|\mathbf{t}) = -0.58 + 0.47 + 0.57$$

# Some Other Word Embedding Models

- ▶ **GloVe**
  - ▶ does not rely just on local statistics (local context information of words)
  - ▶ incorporates global statistics (word co-occurrence)
- ▶ **FastText**
  - ▶ consider subword information, making it effective for understanding morphologically rich languages or misspelled words.
- ▶ **ELMo**
  - ▶ Context-dependent

# Some Other Word Embedding Models

- ▶ BERT (Bidirectional Encoder Representations from Transformers)
  - ▶ Based on transformers
  - ▶ Context-dependent
    - ▶ “financial bank” vs “river bank”
  - ▶ Trained with unlabeled text on two NLP tasks:
    - ▶ Masked language model
    - ▶ Next sentence prediction
  - ▶ Two models
    - ▶ BERT Base: 12 layers and 110 million parameters
    - ▶ BERT Large: 24 layers and 340 million parameters
- ▶ RoBERTa, ALBERT, DistilBERT, etc.
  - ▶ Variations and optimizations of BERT that aim to improve aspects like training speed, memory efficiency, and accuracy.



# Some Other Word Embedding Models

- ▶ GPT (Generative Pretrained Transformer)
  - ▶ primarily designed for generation tasks
  - ▶ can be adapted for classification by using the representations from the last layer.

# Data Mining (EECS 6412)

---

Graph Neural Networks

Instructor: Aijun An

EECS, York University

---

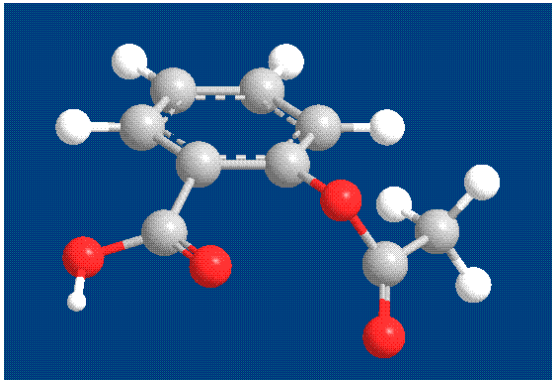
# Outline of Introduction

- ▶ Why graph neural networks (GNNs)?
- ▶ Graph fundamentals
- ▶ Graph mining tasks
- ▶ Graph representation methods
- ▶ Graph embeddings
- ▶ GNN basics
- ▶ How to train a GNN
- ▶ Advantages of GNNs

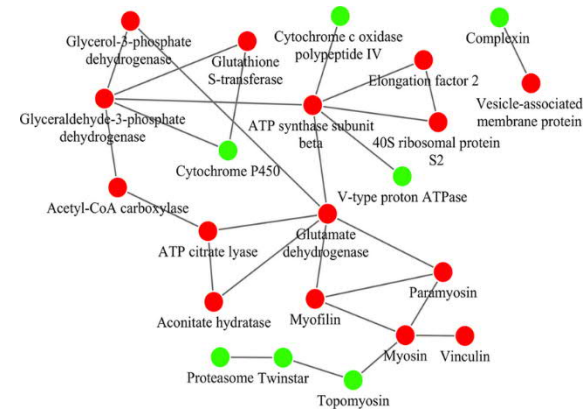
# Graph Data

- ▶ Data entities are connected, forming graphs

## Aspirin



## Biological Networks

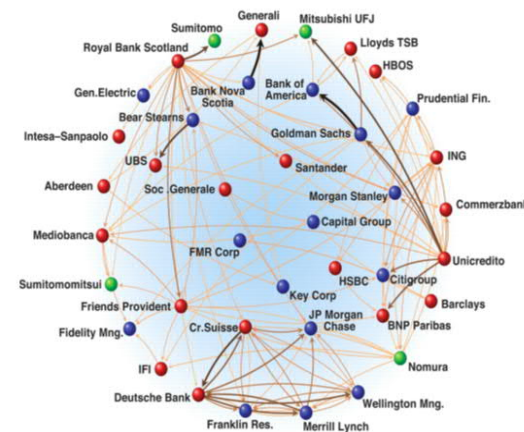


Credits: Chinese Academy of Agricultural Sciences

## Social Networks



## Financial Networks



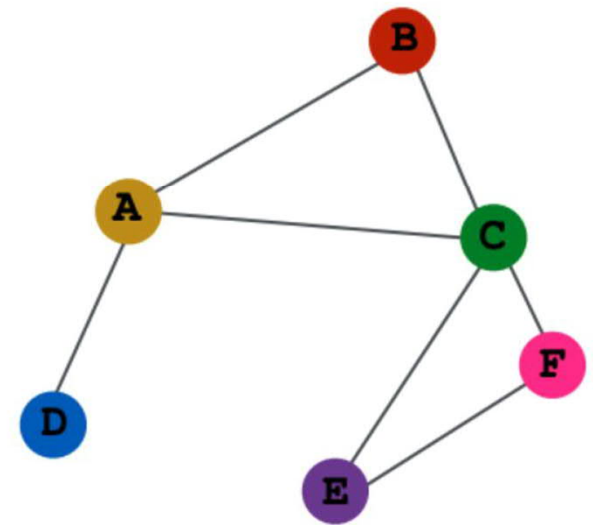
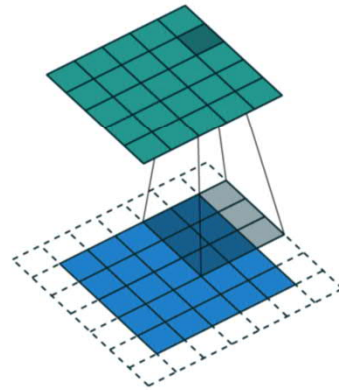
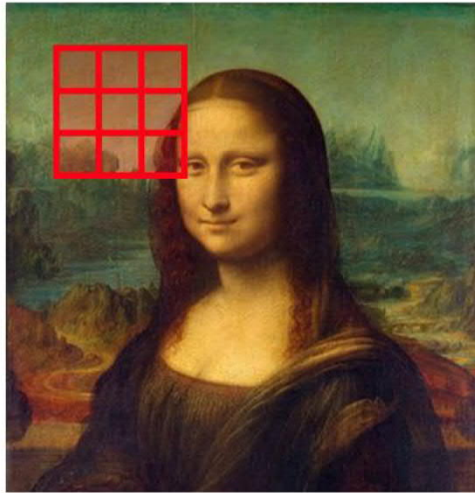
Sample of the international financial network  
Credits: ETH Zurich

# Motivation for Graph Neural Networks (GNNs)

- ▶ Neural networks do not usually consider the relationships between entities, except
  - ▶ CNNs assumes Euclidean structures (grids) of images
  - ▶ RNNs assume sequential relationships between elements of an input sequence
- ▶ Graphs are non-Euclidean and irregular
  - ▶ Need models that respect relationships.
- ▶ GNNs generalize deep learning to graphs via message passing.

# Images vs Graphs

## From image to graph



- Constant number of neighbors
- Fixed ordering of neighbors

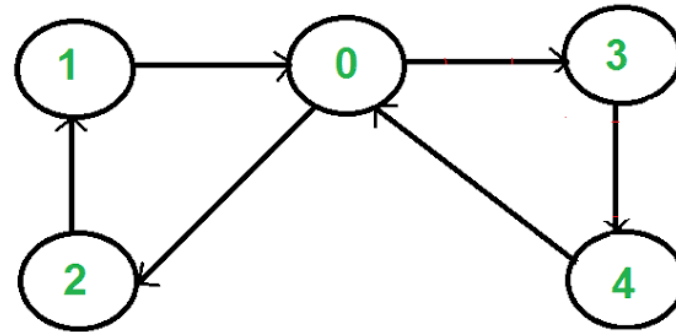
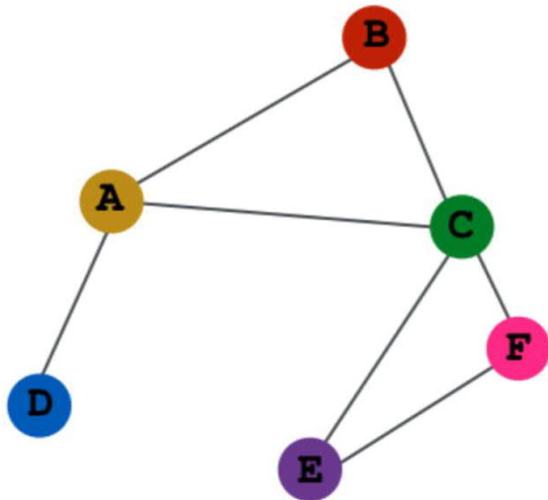
- Different number of neighbors
- No ordering of neighbors

# Basic Concepts of Graphs

- ▶ A graph is defined as  $G = (V, E)$ 
  - ▶  $V$  is a set of nodes (representing entities)
  - ▶  $E$  is a set of edges (representing relationships)
- ▶ Node can have attributes:
  - ▶ Features that describe the node (e.g., user age, paper topic)
- ▶ Edge can have attributes:
  - ▶ Features that describe the edge (e.g., weights, types, directions, timestamps).

# Basic Concepts of Graphs

- ▶ Graphs can be directed or undirected

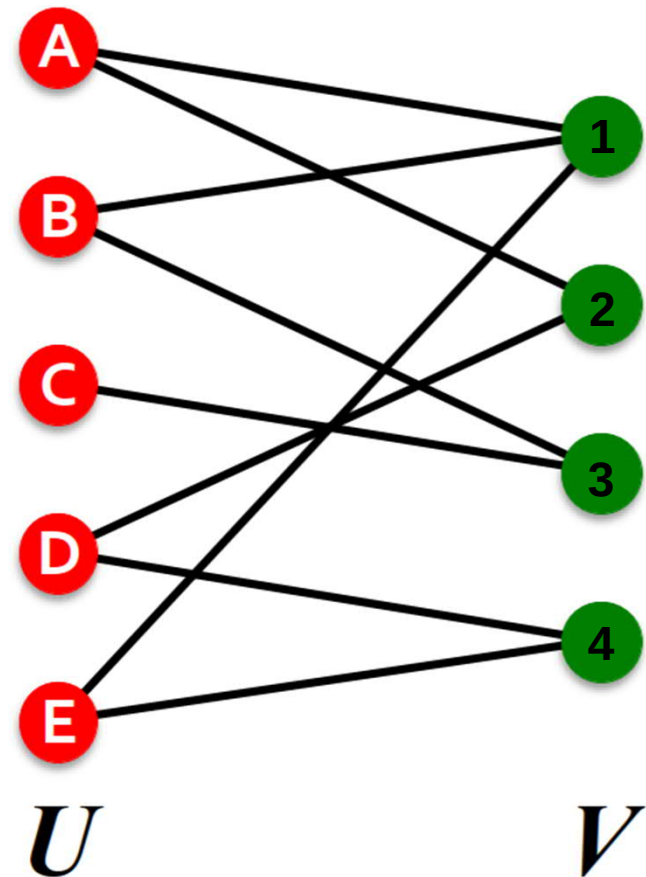


- ▶ Node degree:
  - ▶ Number of edges connecting the node
- ▶ In-degree of a node (for directed graphs):
  - ▶ Number of edges coming into the node
- ▶ Out-degree of a node (for directed graphs):
  - ▶ Number of edges going out of the node.

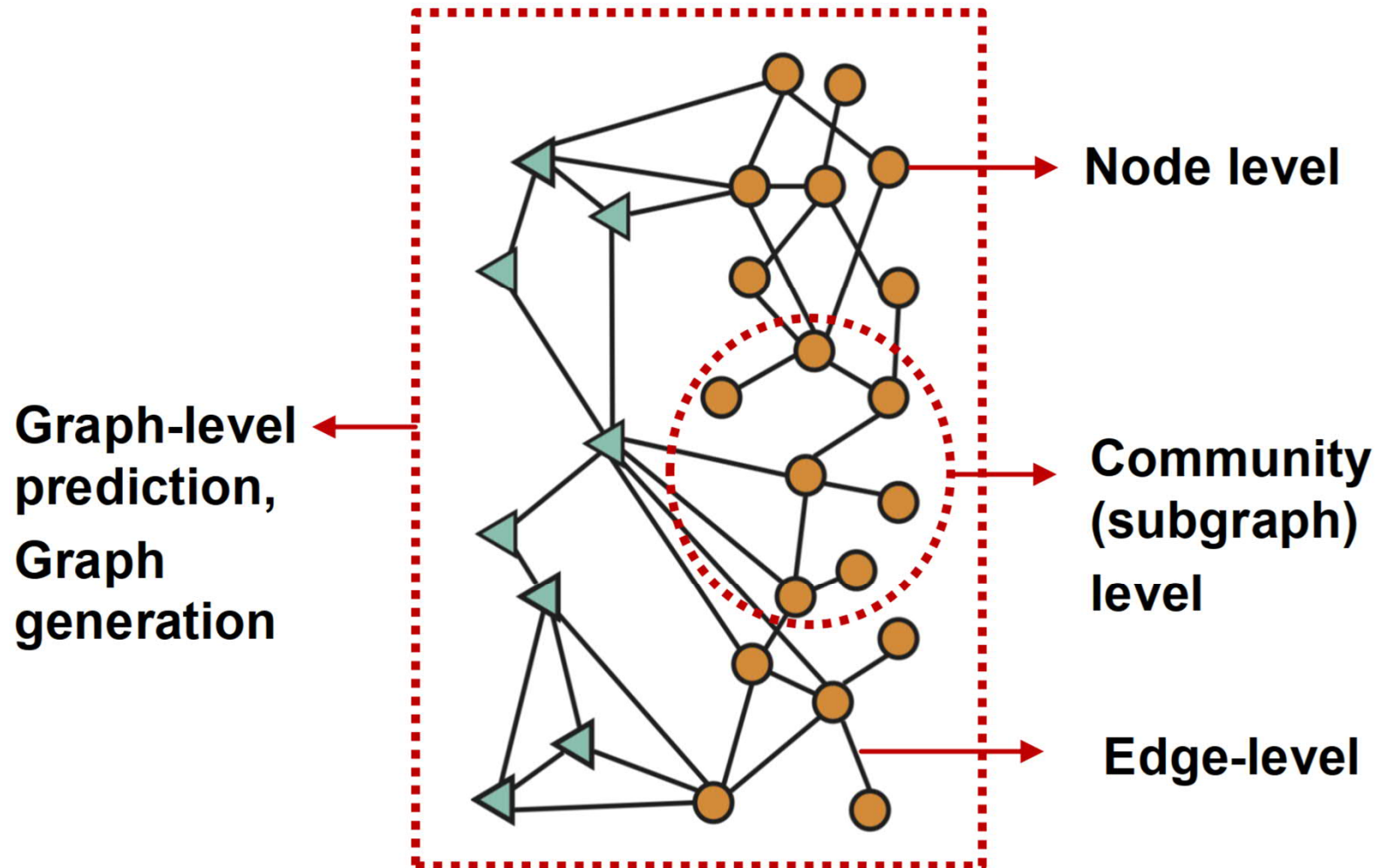


# Bipartite Graph

- ▶ Nodes can be divided into two disjoint sets  $U$  and  $V$  such as every link connects a node in  $U$  to a node in  $V$
- ▶ Examples:
  - ▶ Users-to-Items in a recommender system
  - ▶ Users-to-Movies in movie rating
  - ▶ Recipes-to-Ingredients

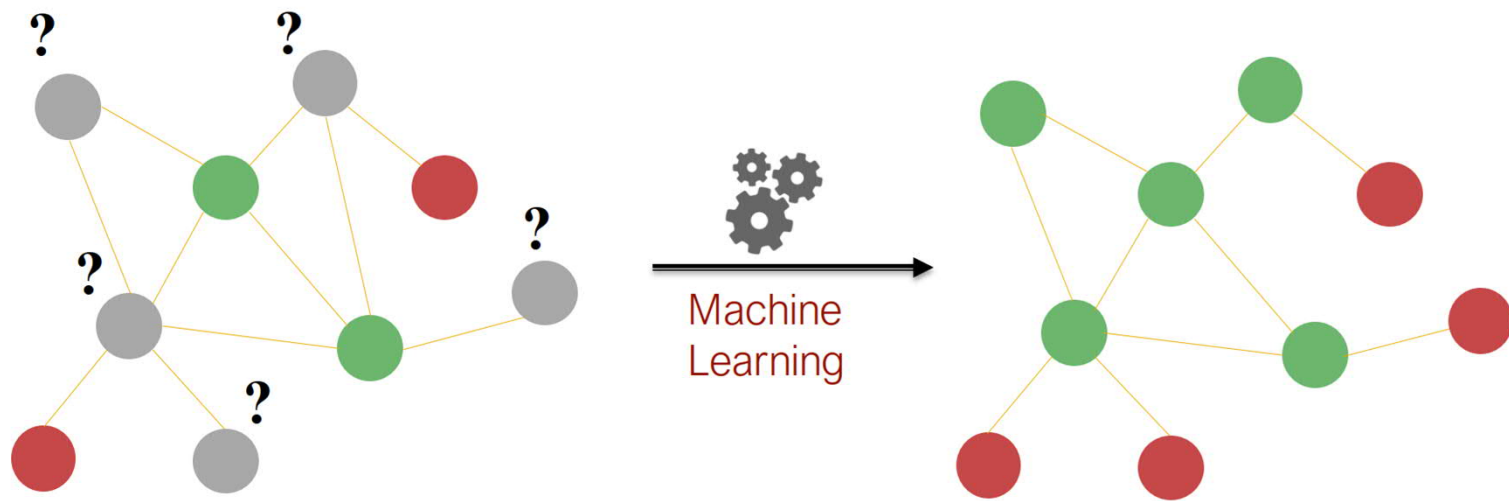


# Different Types of Graph Tasks



# Node-Level Tasks

## ▶ Node classification

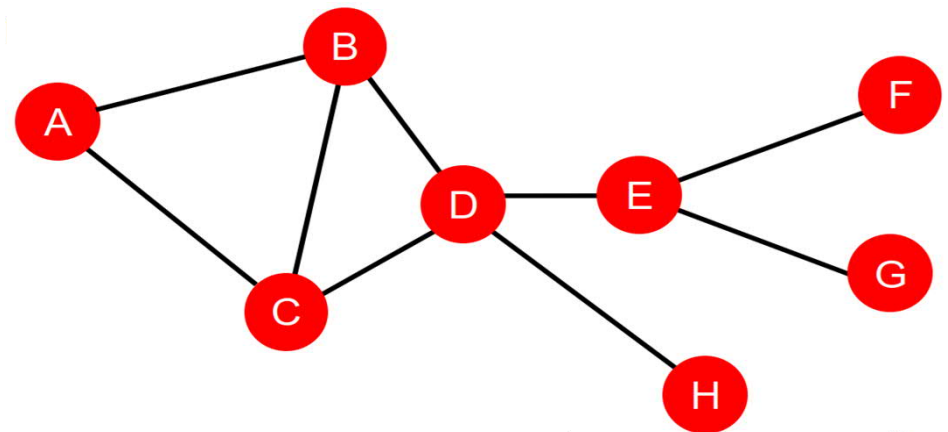


## ▶ Examples:

- ▶ Detecting fraudulent users in a financial network.
- ▶ Determining the political leaning of a social media user
- ▶ Identifying the function of a protein in a biological network

# Node-Level Tasks

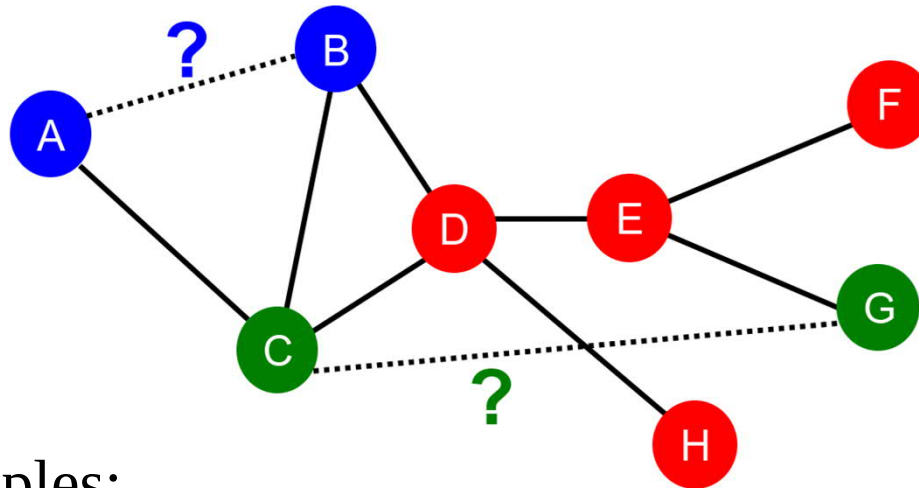
- ▶ Finding node importance by, e.g.,
  - ▶ Node degree
  - ▶ Number of shortest paths passing through the node
  - ▶ Average shortest path length to other nodes
  - ▶ Substructures around the node



# Edge-Level Tasks

- ▶ Link prediction

- ▶ Predict new/missing/unknown links based on existing links

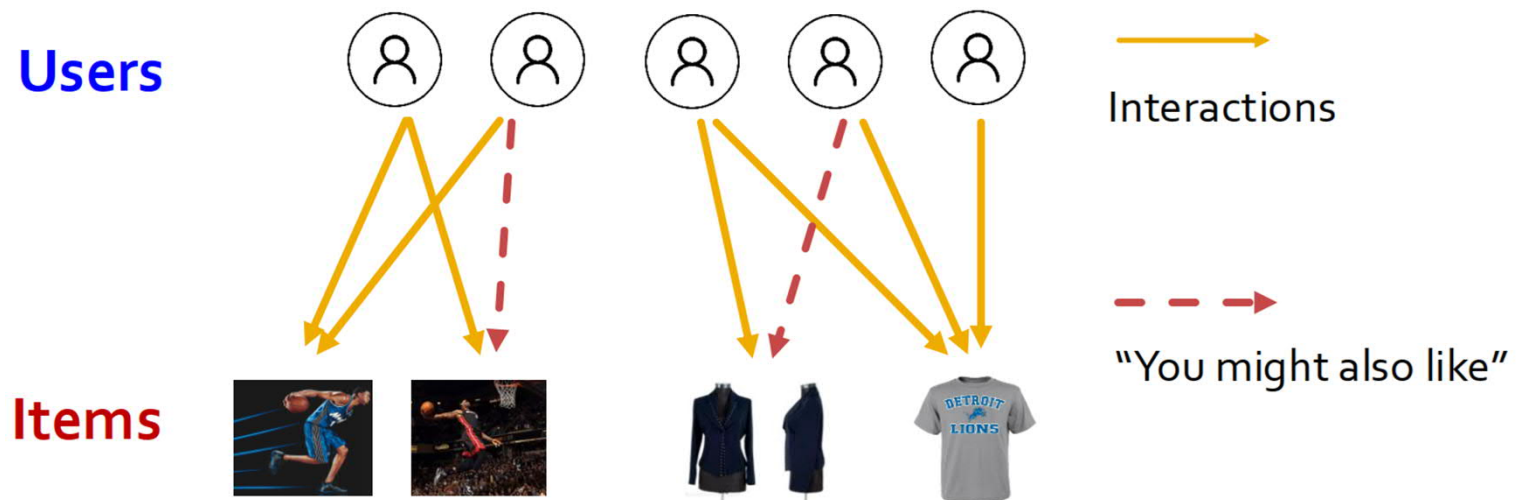


- ▶ Examples:

- ▶ Recommending friends in social networks
    - ▶ Predicting protein-protein interactions in biology
    - ▶ Recommending items to users in e-commerce systems
    - ▶ Identifying relationships in knowledge graphs

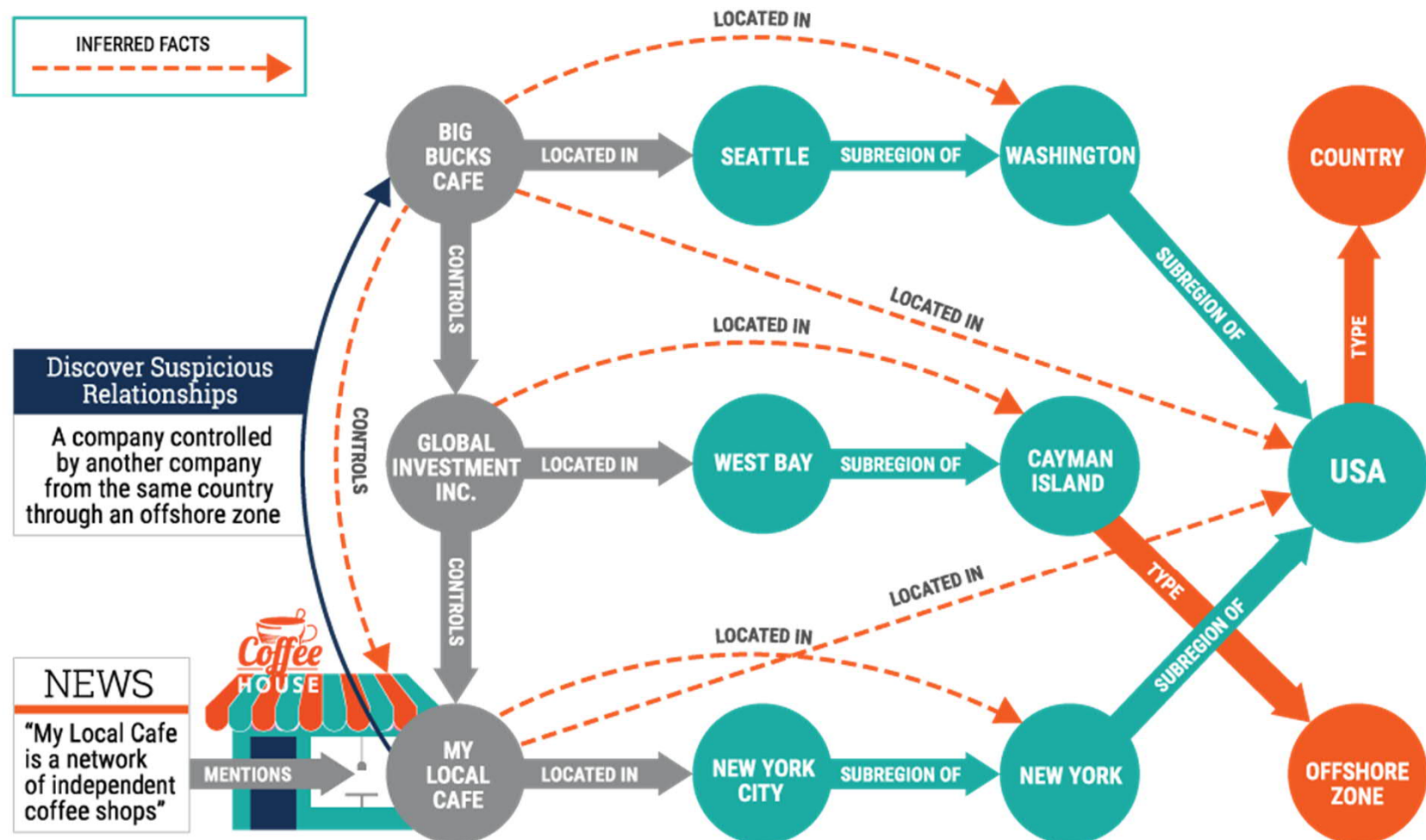
# Link Prediction Examples

- ▶ Recommender systems
  - ▶ Nodes: users and items
  - ▶ Edges: a user bought an item
  - ▶ Goal: recommend items a user might like



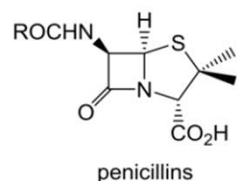
# Link Prediction Examples

- Identifying relationships in knowledge graphs

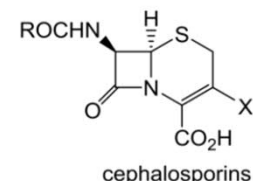


# Graph-Level Tasks

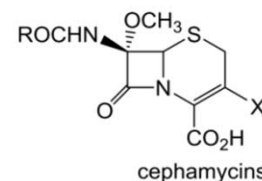
- ▶ Goal: make a prediction for an entire graph or a subgraph
- ▶ Examples:
  - ▶ Antibiotics prediction from promising molecules
    - ▶ Antibiotics are small molecular graphs
      - ▶ Nodes: Atoms
      - ▶ Edges: Chemical bonds
  - ▶ Predicting whether a molecule is toxic
  - ▶ Predicting the smell of a molecule
  - ▶ ...



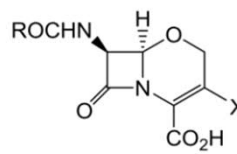
penicillins



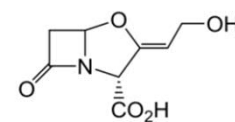
cephalosporins



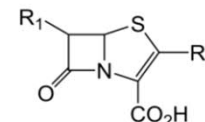
cephamycins



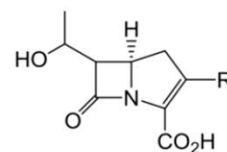
oxacephem



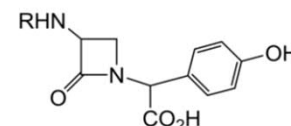
clavulanic acid  
(an oxapenem)



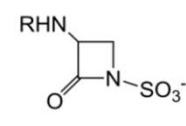
penems



carbapenems



nocardicin



monobactams

Konaklieva, Monika I. "Molecular targets of  $\beta$ -lactam-based antimicrobials: beyond the usual suspects." *Antibiotics* 3.2 (2014): 128-142.



---

## Other Graph-Level Tasks

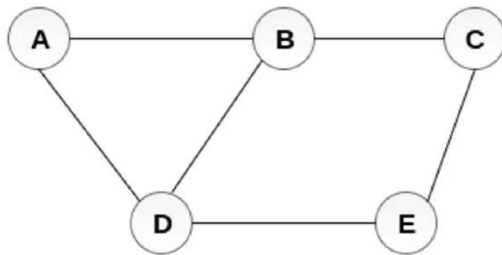
- ▶ Community Detection
  - ▶ Identify densely linked clusters of nodes
- ▶ Network similarity
  - ▶ How similar two (sub)networks are

# Graph Representation Methods

- ▶ Explicit representation
  - ▶ Encode the exact connectivity between nodes
  - ▶ Types:
    - ▶ Adjacency matrix
    - ▶ Adjacency list
    - ▶ Edge list
- ▶ Implicit representation
  - ▶ Represent nodes, edges, or entire graphs as low-dimensional numerical vectors, called *graph embeddings*
  - ▶ These are learned representations
  - ▶ Preserve structural and semantic similarity

# Adjacency matrix

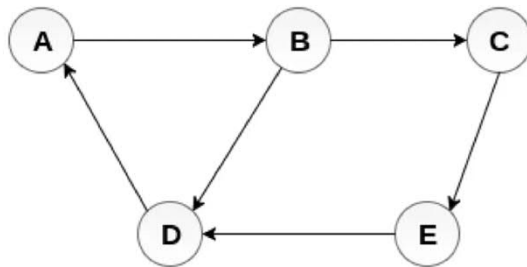
- ▶  $N \times N$  matrix ( $N$  is the number of nodes)



Undirected Graph

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 | 0 |
| B | 1 | 0 | 1 | 1 | 0 |
| C | 0 | 1 | 0 | 0 | 1 |
| D | 1 | 1 | 0 | 0 | 1 |
| E | 0 | 0 | 1 | 1 | 0 |

Adjacency Matrix



Directed Graph

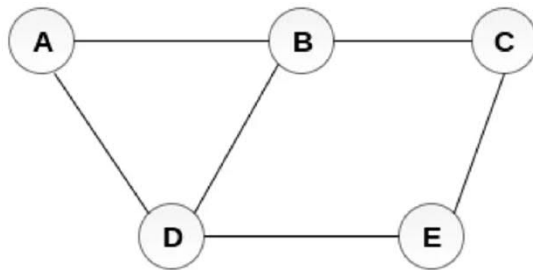
|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 1 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

Adjacency Matrix

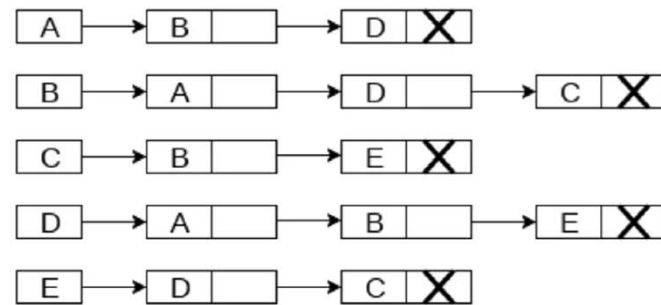
If there are weights on edges, weights can be used instead of binary numbers.

# Adjacency list

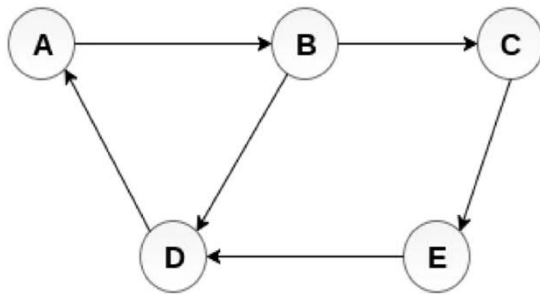
- ▶ Space efficient for sparse matrix



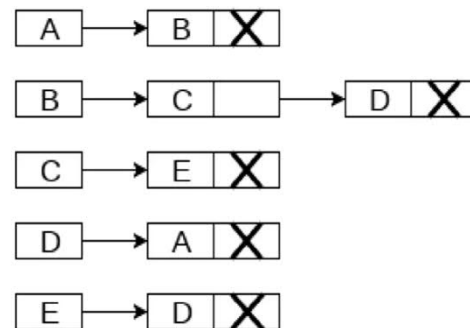
Undirected Graph



Adjacency List



Directed Graph



Adjacency List

---

# Graph Embeddings

- ▶ Represent nodes/edges/graphs using low-dimensional *numerical vectors*
- ▶ Capture *structural* patterns and *semantic* relationships,
- ▶ Enable machine-learning models to operate on graphs efficiently while preserving important topological information.
- ▶ Embeddings are *learned* representations

# Types of Graph Embedding Learning Methods

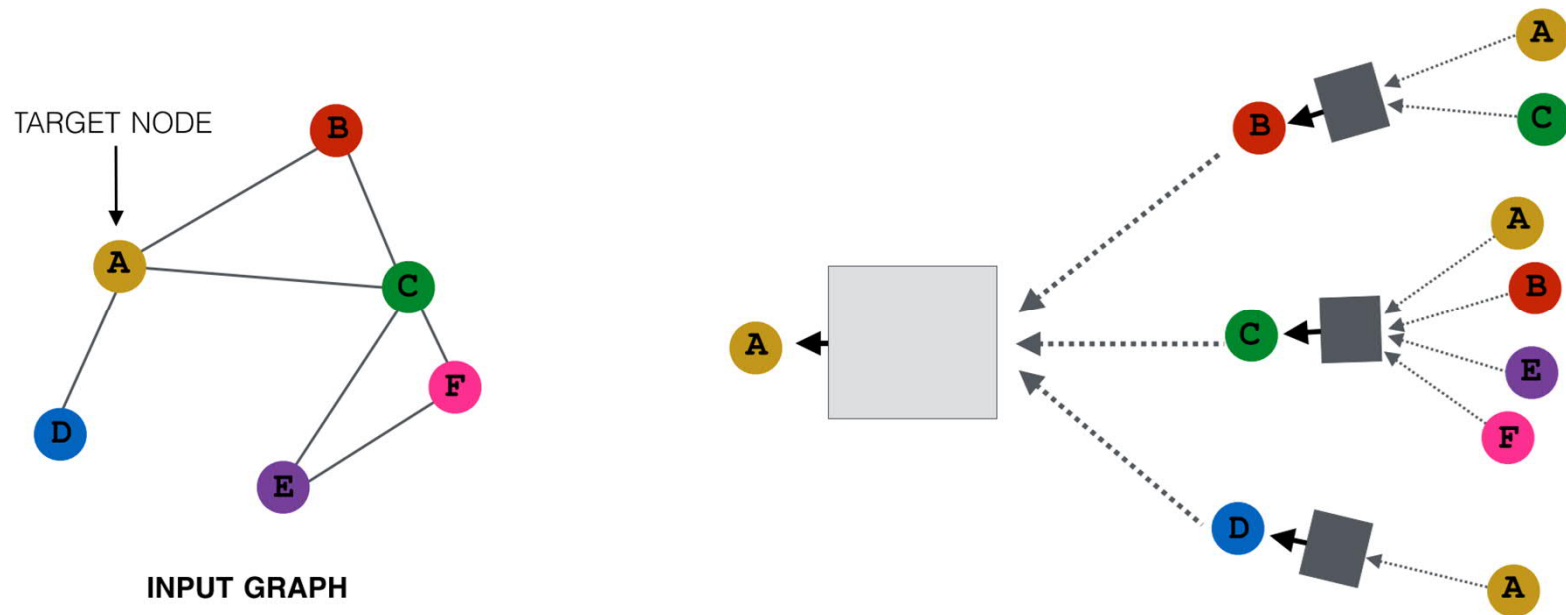
- ▶ Traditional
  - ▶ Node2Vec
  - ▶ DeepWalk
  - ▶ ...
- ▶ Graph neural network (GNN) based
  - ▶ Graph Convolutional Network (GCN)
  - ▶ GraphSAGE
  - ▶ ...

# Graph Neural Networks

- ▶ Deep learning model that generates a node embedding by aggregating the node's neighbors' embeddings.
- ▶ Intuition:
  - ▶ A node's state is influenced by its interactions with its neighbors in the graph.

# Graph Neural Networks

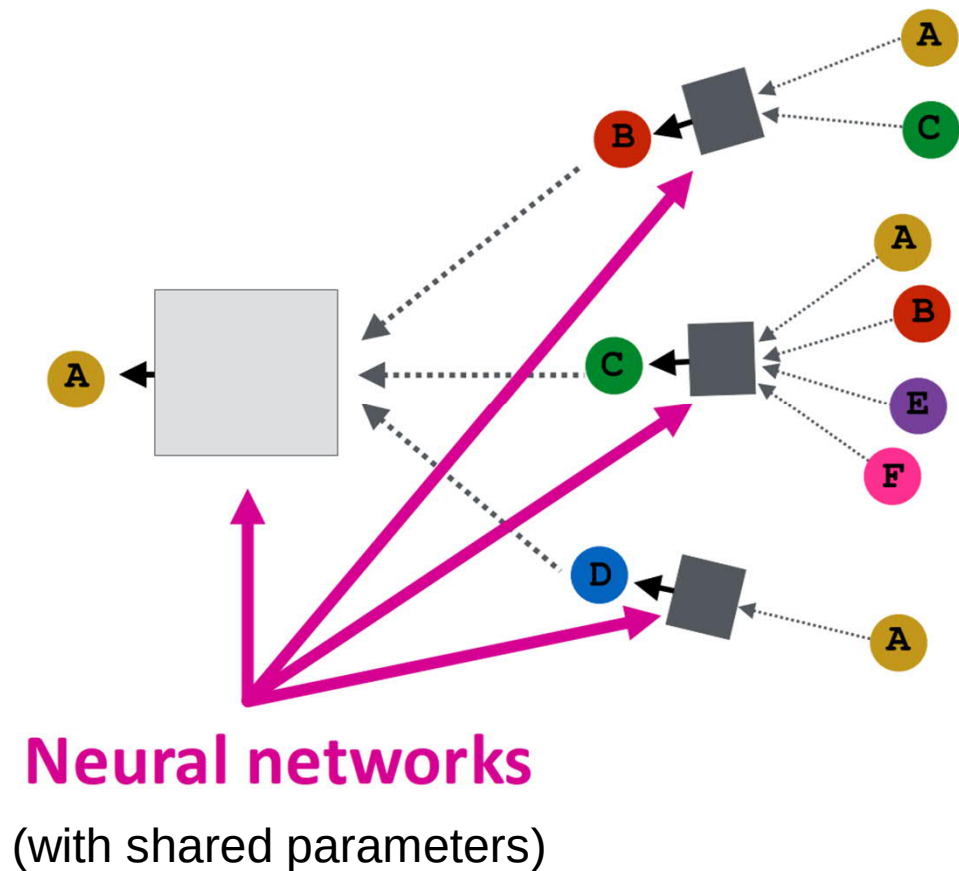
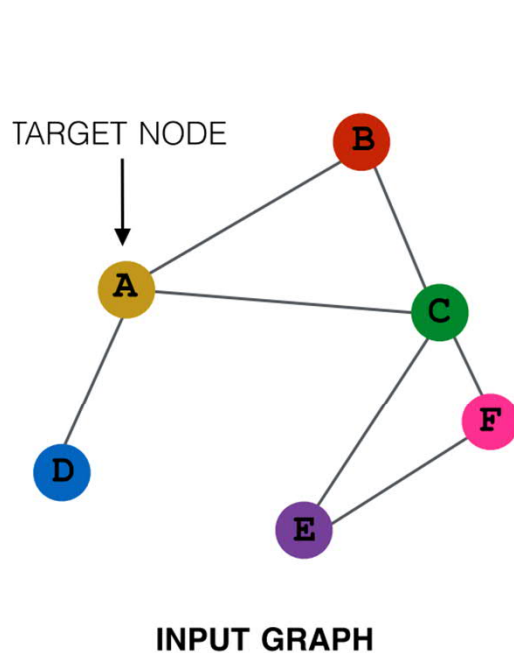
- ▶ Key idea:
  - ▶ Generate node embeddings by aggregating neighbors





# Graph Neural Networks

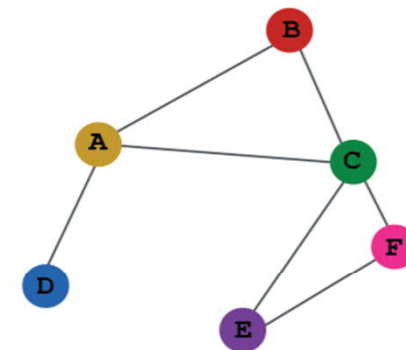
- ▶ Nodes aggregate information from their neighbors using neural networks



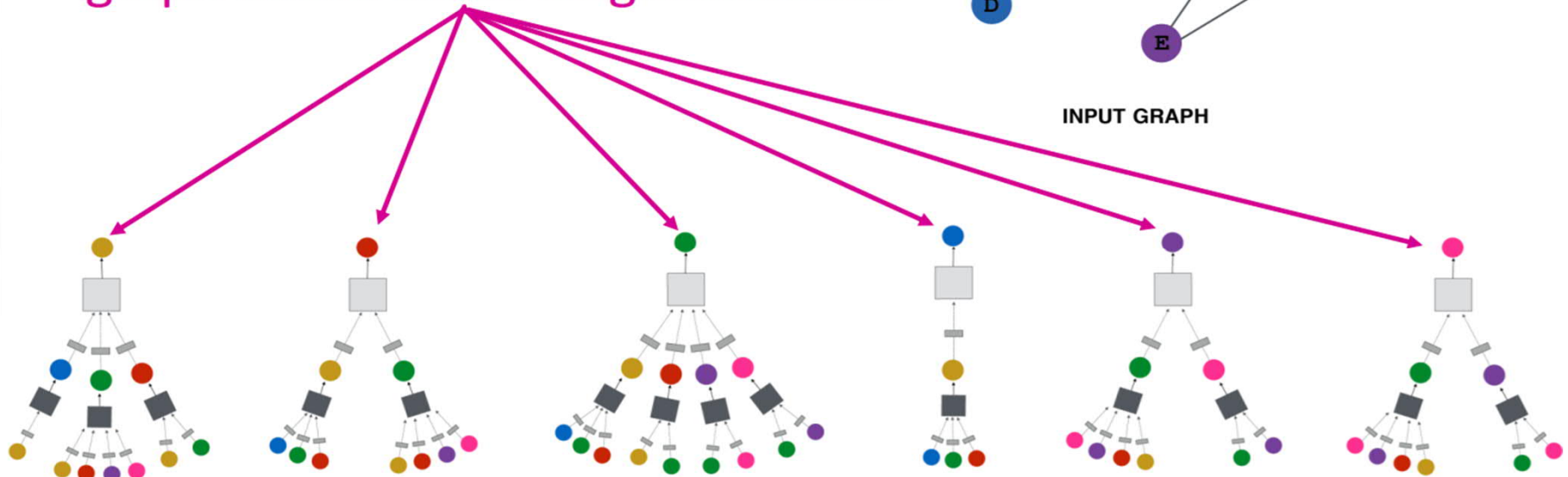
# Graph Neural Networks

- ▶ Network neighborhood of a node defines its computation graph

Every node defines a computation graph based on its neighborhood!

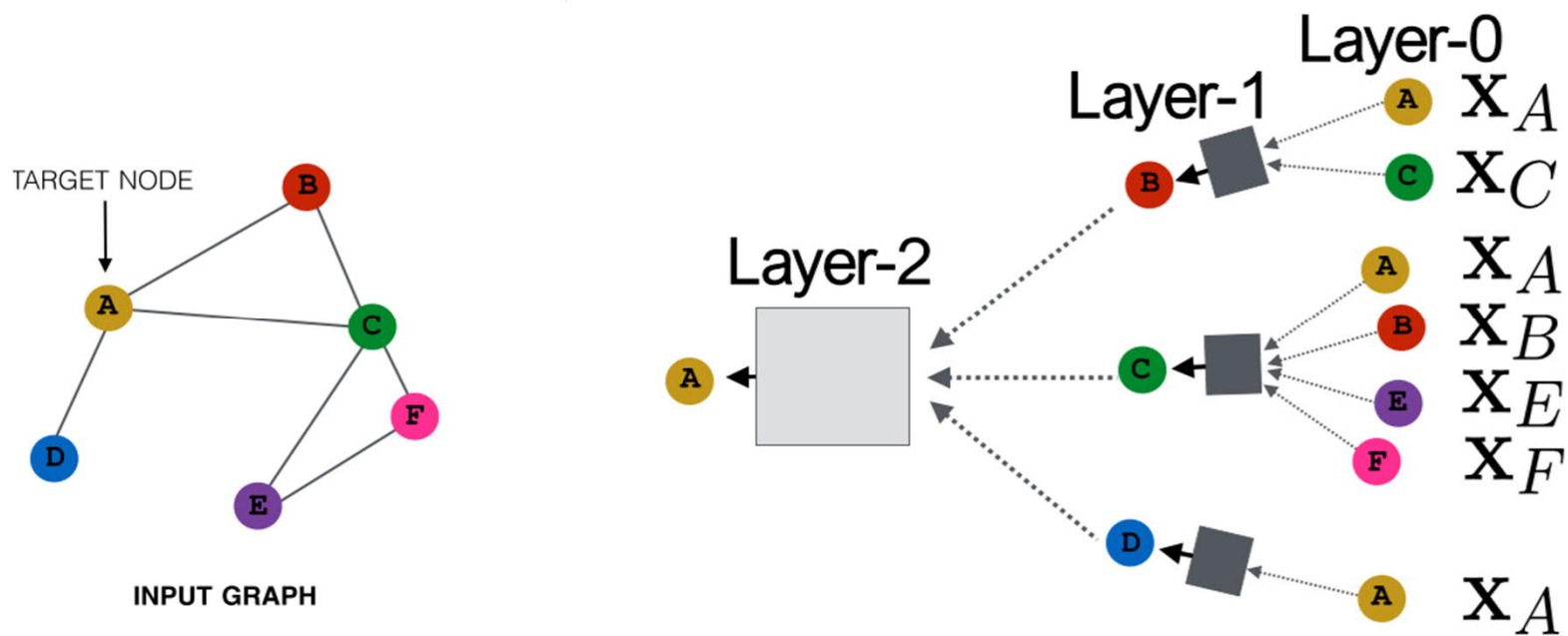


INPUT GRAPH



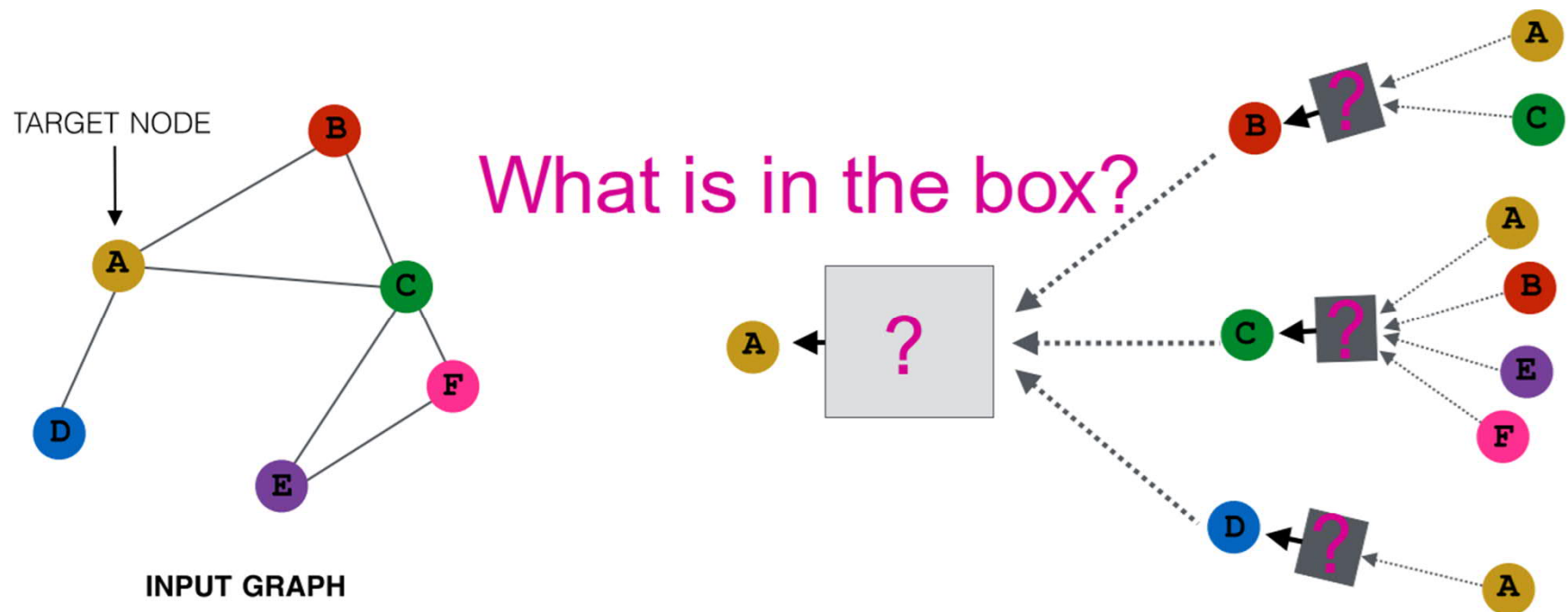
# Graph Neural Networks

- ▶ Graph can be of arbitrary depth:
  - ▶ Nodes have embeddings at each layer
  - ▶ Layer-0 embedding of node  $v$  is its input feature  $x_v$
  - ▶ Layer- $k$  embedding gets information from nodes that are  $k$  hops ways



# Neighborhood Aggregation

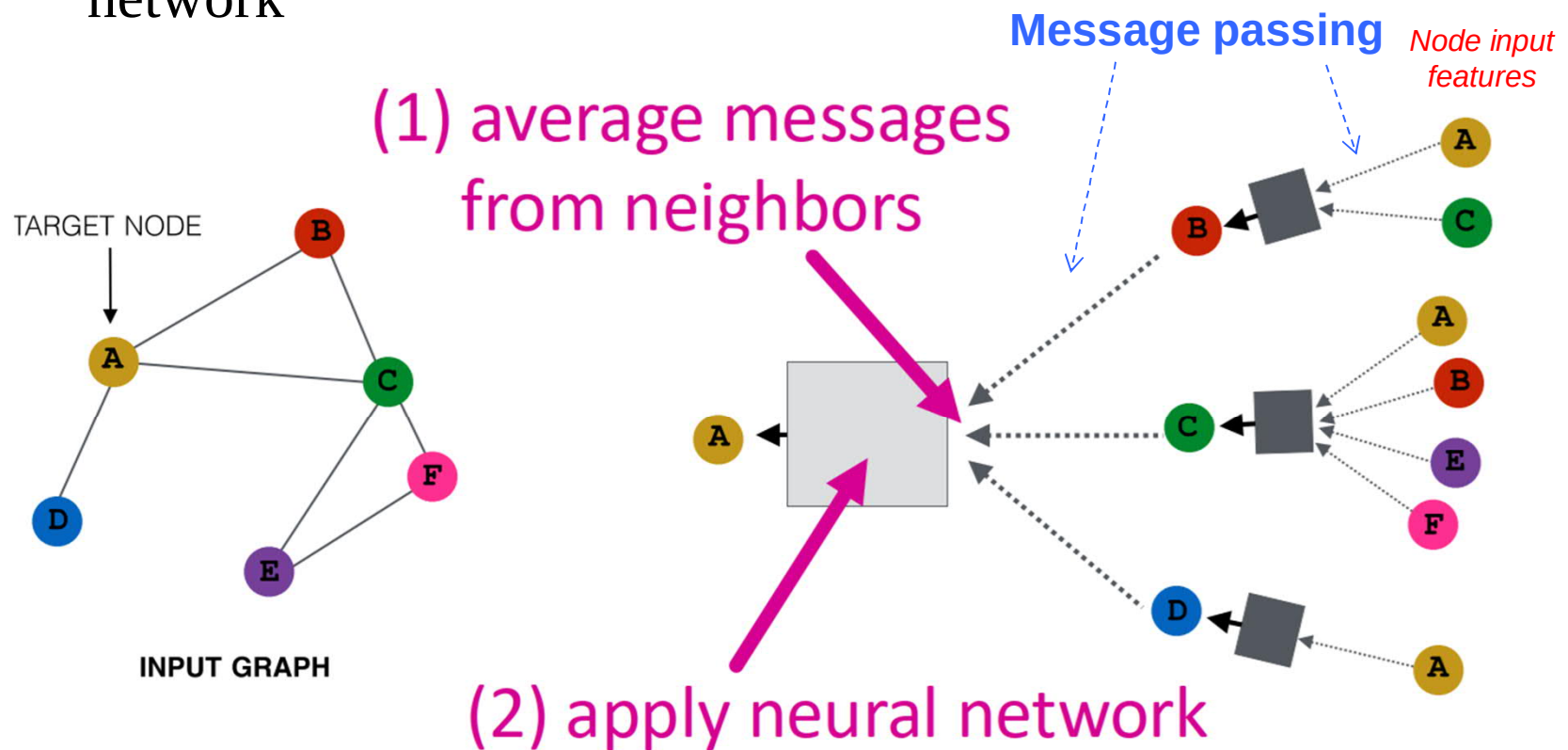
- ▶ Different approaches may aggregate information across the layers differently



# How to Compute the Embedding of a Node

## ► Basic approach:

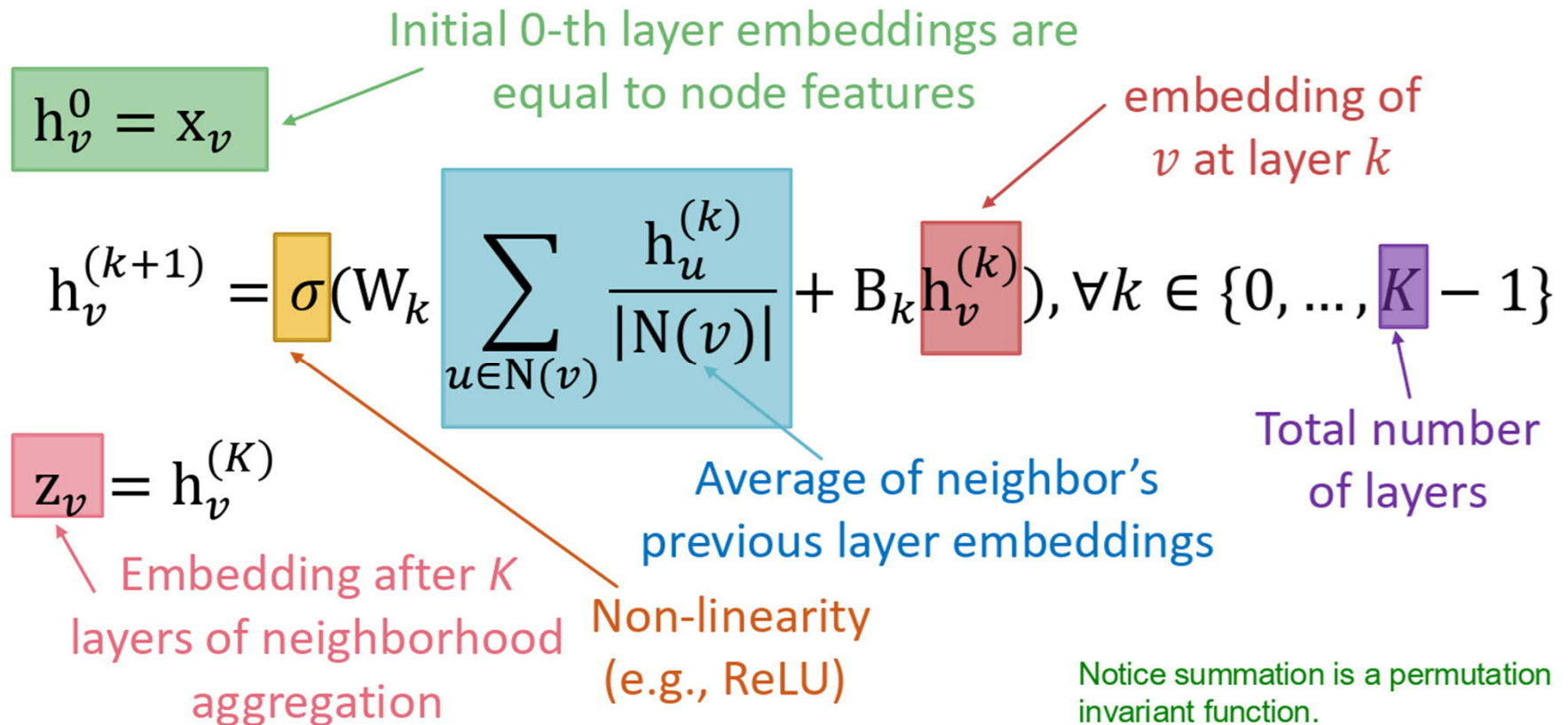
- Average information from neighbors and apply a neural network



# How to Compute the Embedding of a Node

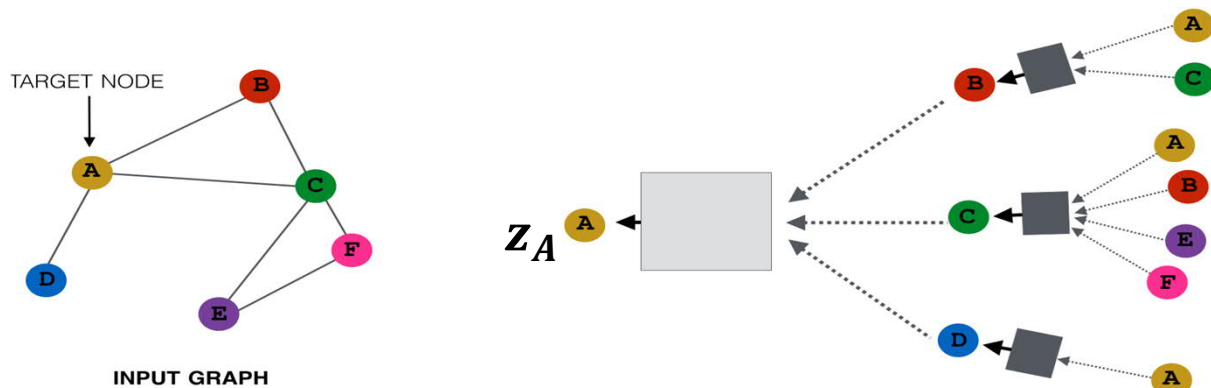
## ► Math:

- Average information from neighbors and apply a neural network



# How to Train the Model

- ▶ What to train to generate node embeddings



Trainable weight matrices  
(i.e., what we learn)

$$h_v^{(0)} = x_v$$

$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

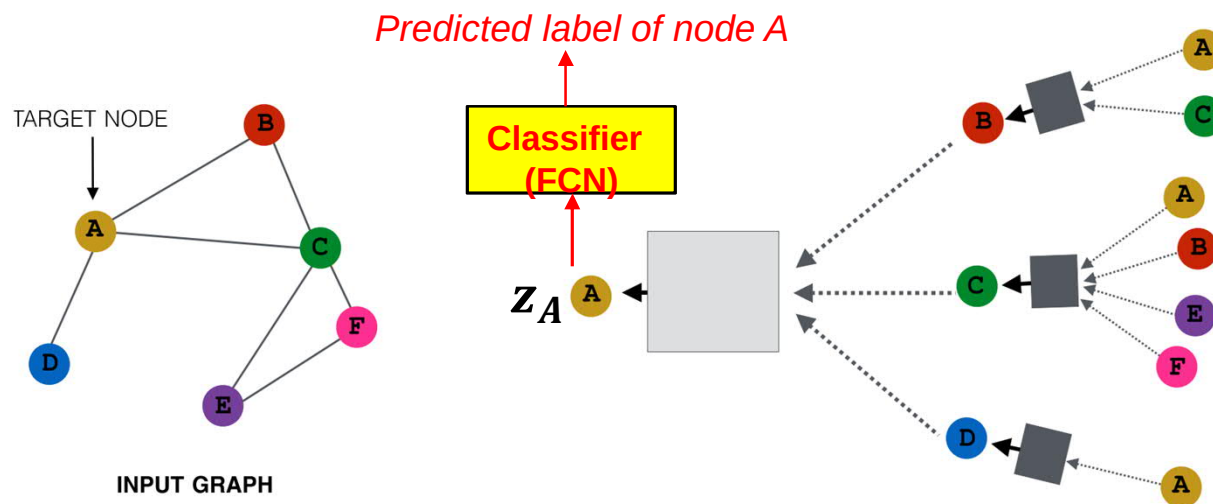
$$z_v = h_v^{(K)}$$

Final node embedding

$W_k$  is the matrix for converting the aggregated previous-layer embeddings from neighbors  
 $B_k$  is the matrix for transforming its own vector from the previous layer

# How to Train the Model

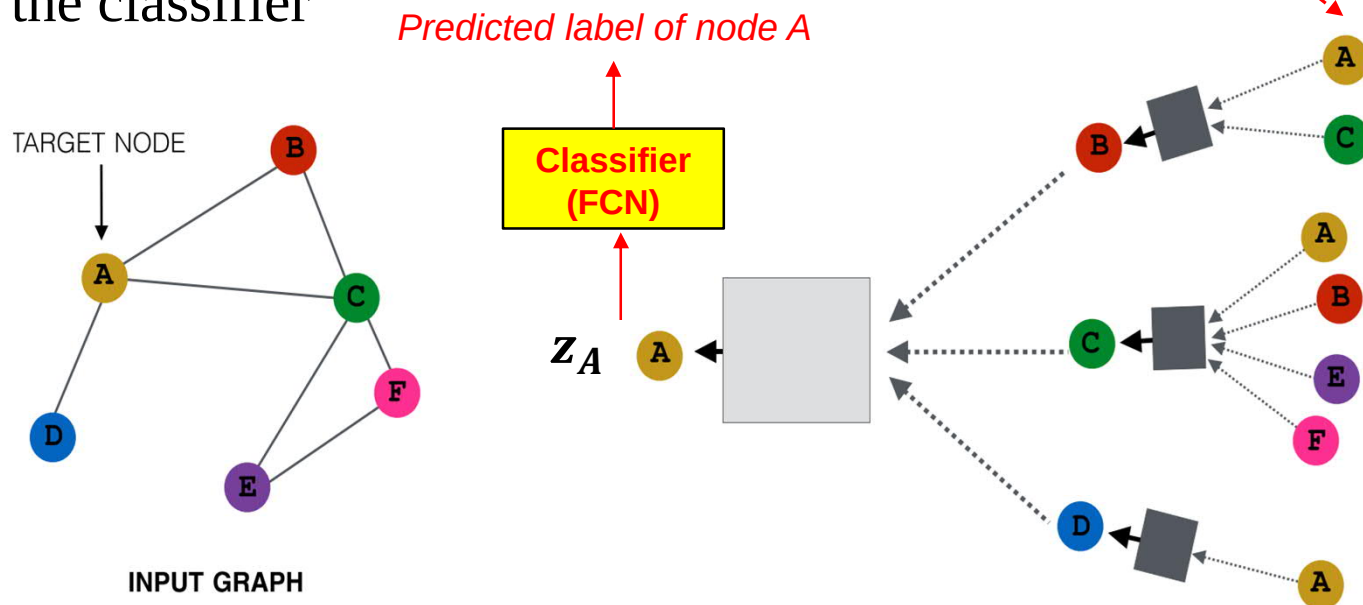
- ▶ One way to train the model is to use the node classification task
  - ▶ If some nodes have a class label (e.g., if the node is a fraudulent user, or the functional role of a protein, etc.)
  - ▶ These labels serve as the ground truth
- ▶ Add a **classifier** (one or two fully-connected layers) on top of the final embedding layer





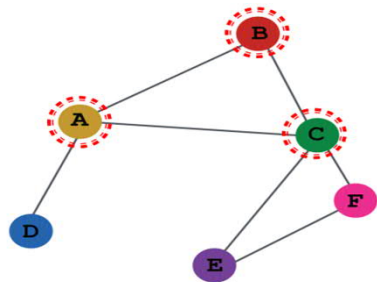
# How to Train the Model

- ▶ Given a node with class label,
  - ▶ Use the input vectors of its K-hop neighbors as input
  - ▶ Go through the GCN encoder (i.e., K layers transformation) with the current weights to get the embedding  $\mathbf{z}_A$
  - ▶ Feed  $\mathbf{z}_A$  into the classifier to obtain the predicted label for A
  - ▶ Compute the loss and backpropagate to update all weights in the GNN and the classifier

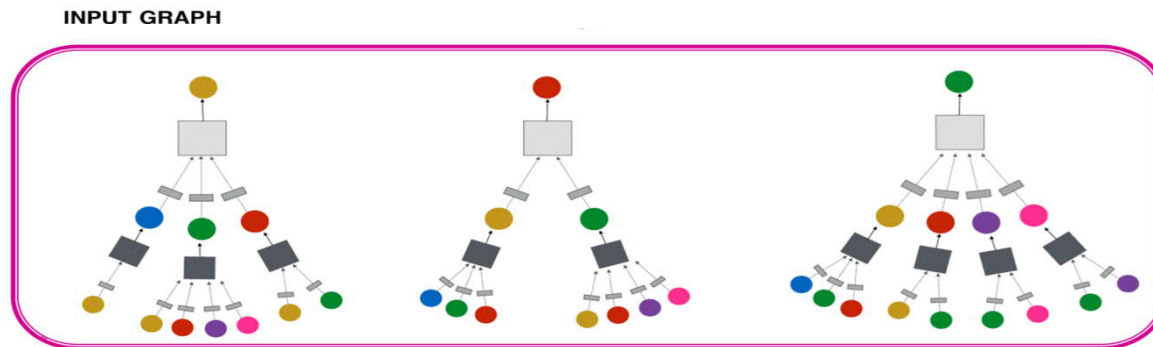


# How to Train the Model

- ▶ Training goes through all (or a subset of) the nodes that have class labels
  - ▶ Each node is like a training example
  - ▶ Each node has its own *computation graph*
- ▶ Can go with multiple epochs (i.e., go through all the labeled nodes multiple times).



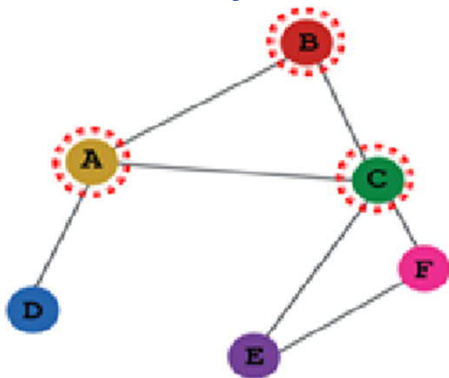
Computation graphs for nodes A, B and C:



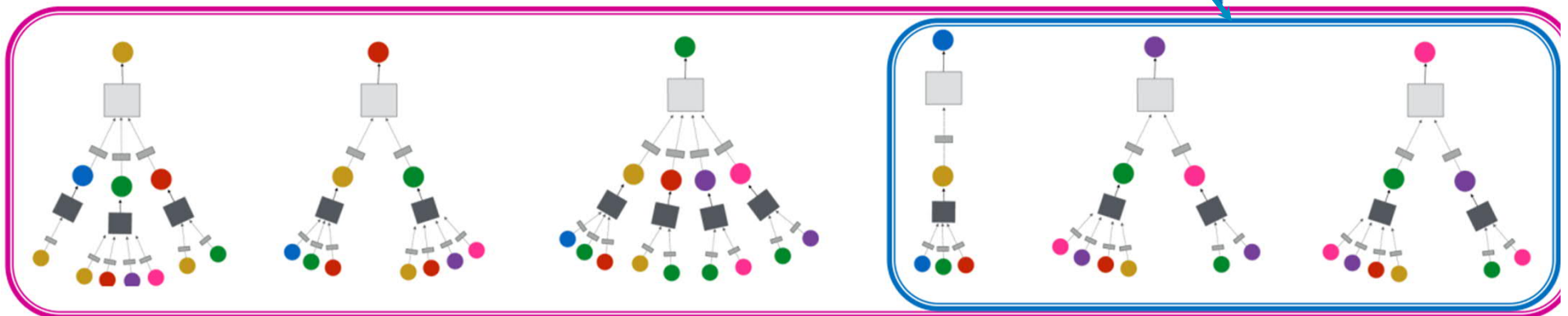
# Inductive Capability of GNN

- ▶ After training, we
  - ▶ not only obtain the embeddings of the nodes with class labels
  - ▶ but also embeddings of the nodes without class labels

Assume that only A, B and C are labeled:

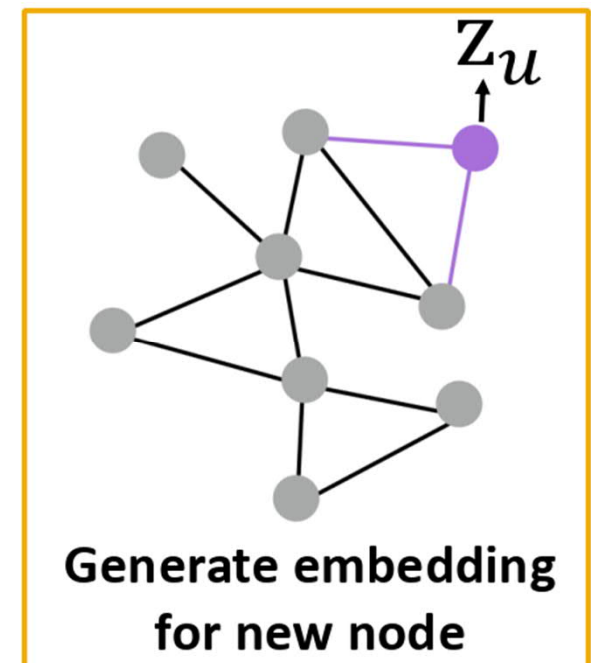
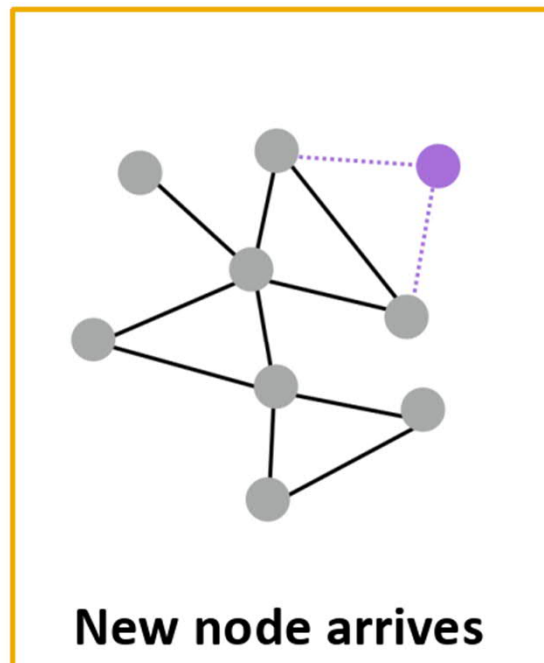
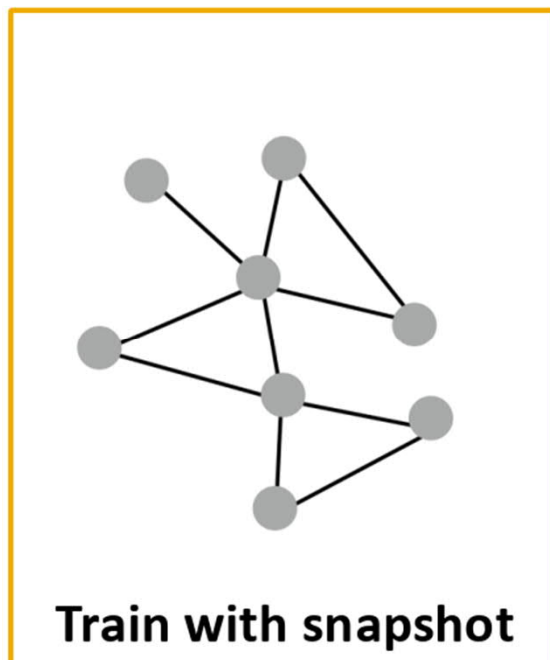


Embeddings of nodes D, E and F are also obtained through the training:



# Inductive Capability of GNN: New Nodes

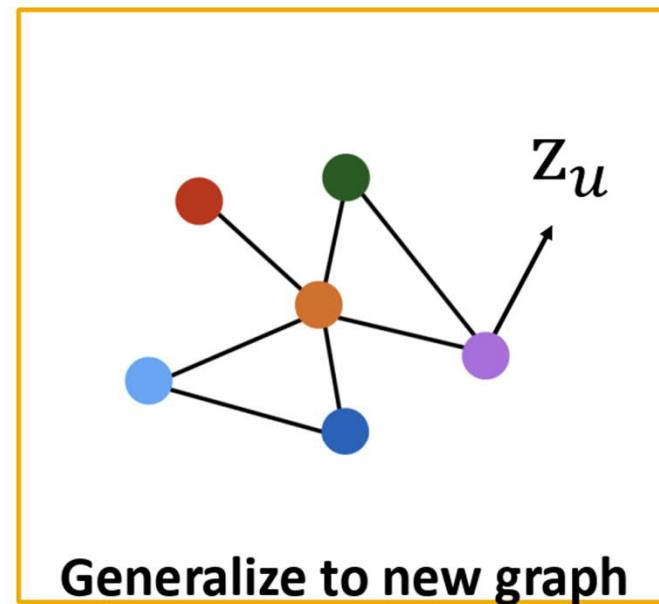
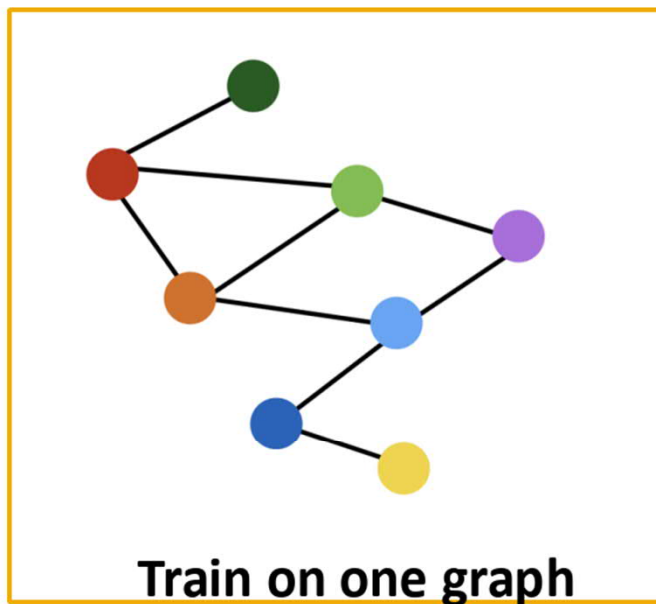
- ▶ Can generate embeddings on **new/unseen nodes**



- Many application settings constantly encounter previously unseen nodes:
  - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

# Inductive Capability of GNN: New Graphs

- ▶ Can generate embeddings on **new graphs**



Inductive node embedding → Generalize to entirely unseen graphs

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

# Usefulness of Node Embeddings

- ▶ With the trained node embeddings, we can
  - ▶ Compute edge embeddings (e.g., by averaging the nodes embeddings)
  - ▶ Sub-graph embeddings
- ▶ Perform graph tasks:
  - ▶ Node classification
  - ▶ Node clustering (Community detection)
  - ▶ Link prediction
  - ▶ Graph similarity computation
  - ▶ ...

# Advantages of GNN for Graph Embeddings over Traditional Graph Embedding Methods

- ▶ Can consider node features
  - ▶ Previous methods (e.g., Node2Vec, based on random walk) cannot incorporate node features. They only consider the graph structure
- ▶ Can incorporate edge weights.
- ▶ Inductive:
  - ▶ Can generate embeddings for new nodes and node graphs

---

# Summary

- ▶ Graph fundamentals
- ▶ Graph mining tasks
- ▶ Graph representation methods
- ▶ Graph embeddings
- ▶ GNN basics
- ▶ How to train a GNN
- ▶ Advantages of GNNs

## Acknowledgements

Some slides are adapted from slides by Jure Leskovec of Stanford University.



# Outline

- ▶ Introduction and applications
- ▶ Traditional methods
  - ▶ Text Representation
  - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)
  - ▶ Neural network-based embeddings
    - ▶ Static Embeddings
    - ▶ Contextual Embeddings with Transformers
  - ▶ Text classification with embeddings

# Motivation for Contextual Word Embedding

- ▶ Word2Vec and its variants are static word embedding models
  - ▶ The embedding of a word is fixed after training and does not change with its context.
- ▶ However, a word can have different meaning, e.g.,
  - ▶ Bank of Montreal vs. Bank of St. Lawrence River.
  - ▶ The chicken did not cross the road because it was too tired.  
vs.  
The chicken did not cross the road because it was too wide,

# Contextual Word Embeddings

- ▶ A representation of meaning of a word should be different in different contexts!
- ▶ Contextual Embedding:
  - ▶ A word has different vectors that expresses different meanings depending on the surrounding words
  - ▶ Consider other words in the sentence or document when computing the embedding of a word
- ▶ How to compute contextual embeddings?
  - ▶ Self-Attention.

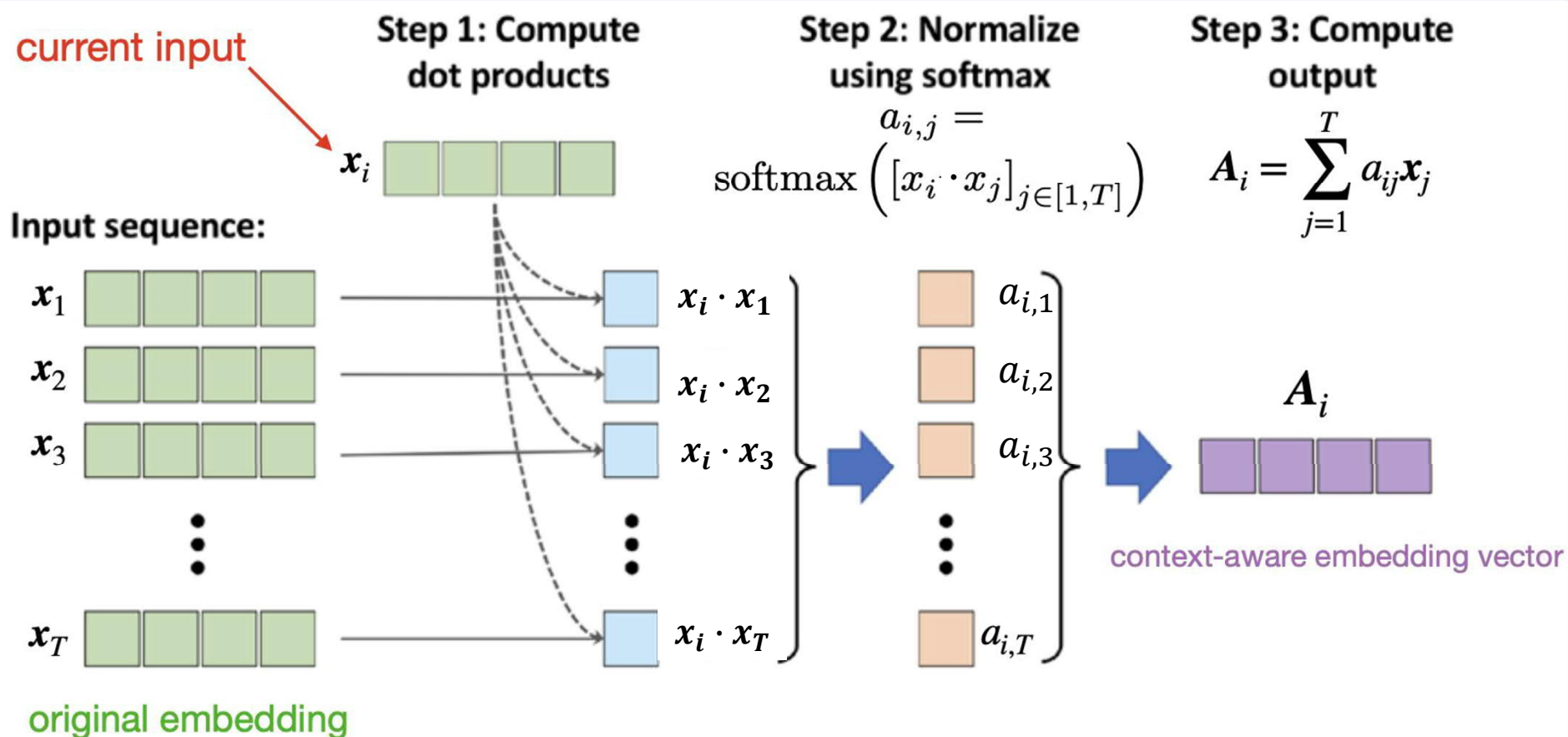
# Self-Attention

- ▶ Attention is a mechanism in neural networks that
  - ▶ allows the model to selectively focus on specific parts of the input data when producing an output,
  - ▶ Assigns dynamic importance weights to different elements.
  - ▶ Mimics human cognitive attention
  - ▶ Crucial for handling long sequences and capturing long-range dependencies in data, such as the relationships between words in a sentence.
- ▶ Self-attention is a mechanism that
  - ▶ Computes a contextualized representation for each word in the input sequence by allowing it to "look at" (i.e., attend to) and weigh the importance of all other words in that same sequence.
  - ▶ The embedding of a word is a weighted sum of the embeddings of all other words in the sequence.

# Self-Attention – Basic, No-Trainable Weights

- Given an input sentence  $(x_1, x_2, \dots, x_T)$ , where  $x_i$  is a word/token (represented by their embeddings), convert them into context-aware embeddings  $(A_1, A_2, \dots, A_T)$

## Self-Attention (Basic-No weights)



# Self-Attention with Trainable Weight Matrices

- ▶ Train three weight matrices ( $W^q$ ,  $W^k$ ,  $W^v$ ) to convert each word (or token) embedding  $x_i$  into three vectors  $q_i$ ,  $k_i$ ,  $v_i$ , representing 3 separate roles of  $x_i$

- ▶ **Query vector:**  $q_i = x_i W^q$

- ▶ Represents the token's "question" presented to other tokens.

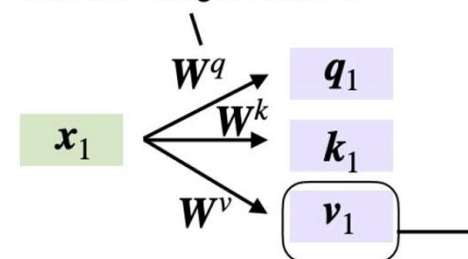
- ▶ **Key vector:**  $k_i = x_i W^k$

- ▶ Represents the token's "answer" to other tokens' questions.

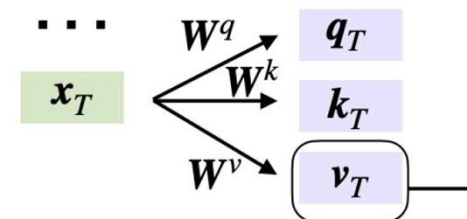
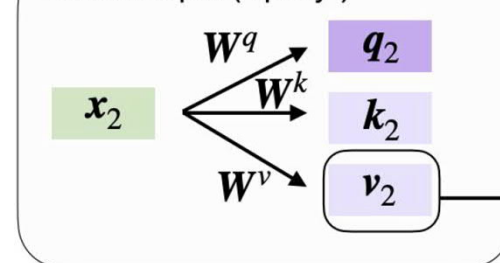
- ▶ **Value vector:**  $v_i = x_i W^v$

- ▶ Represents the actual information the token carries.

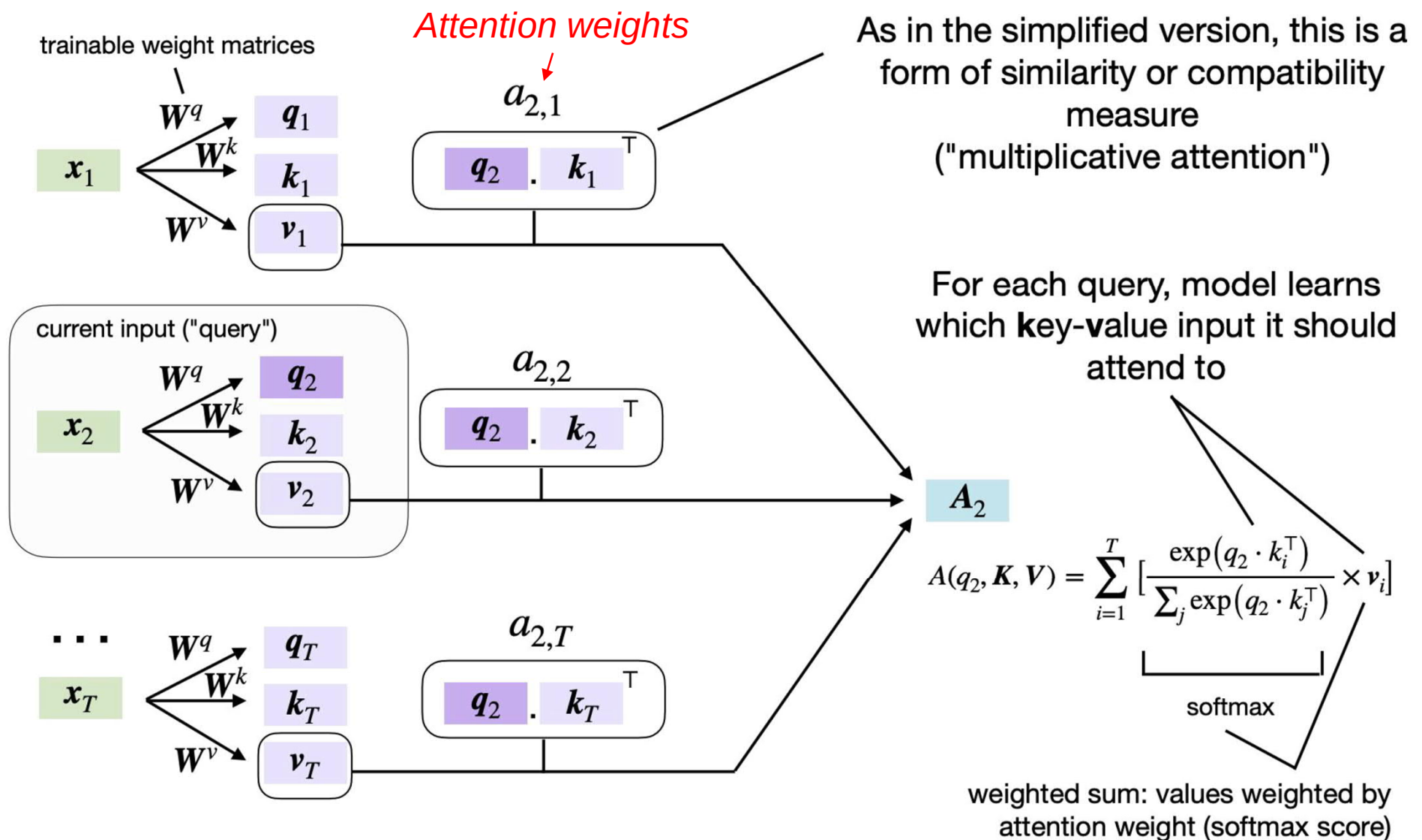
trainable weight matrices



current input ("query")



# Self-Attention with Trainable Weights

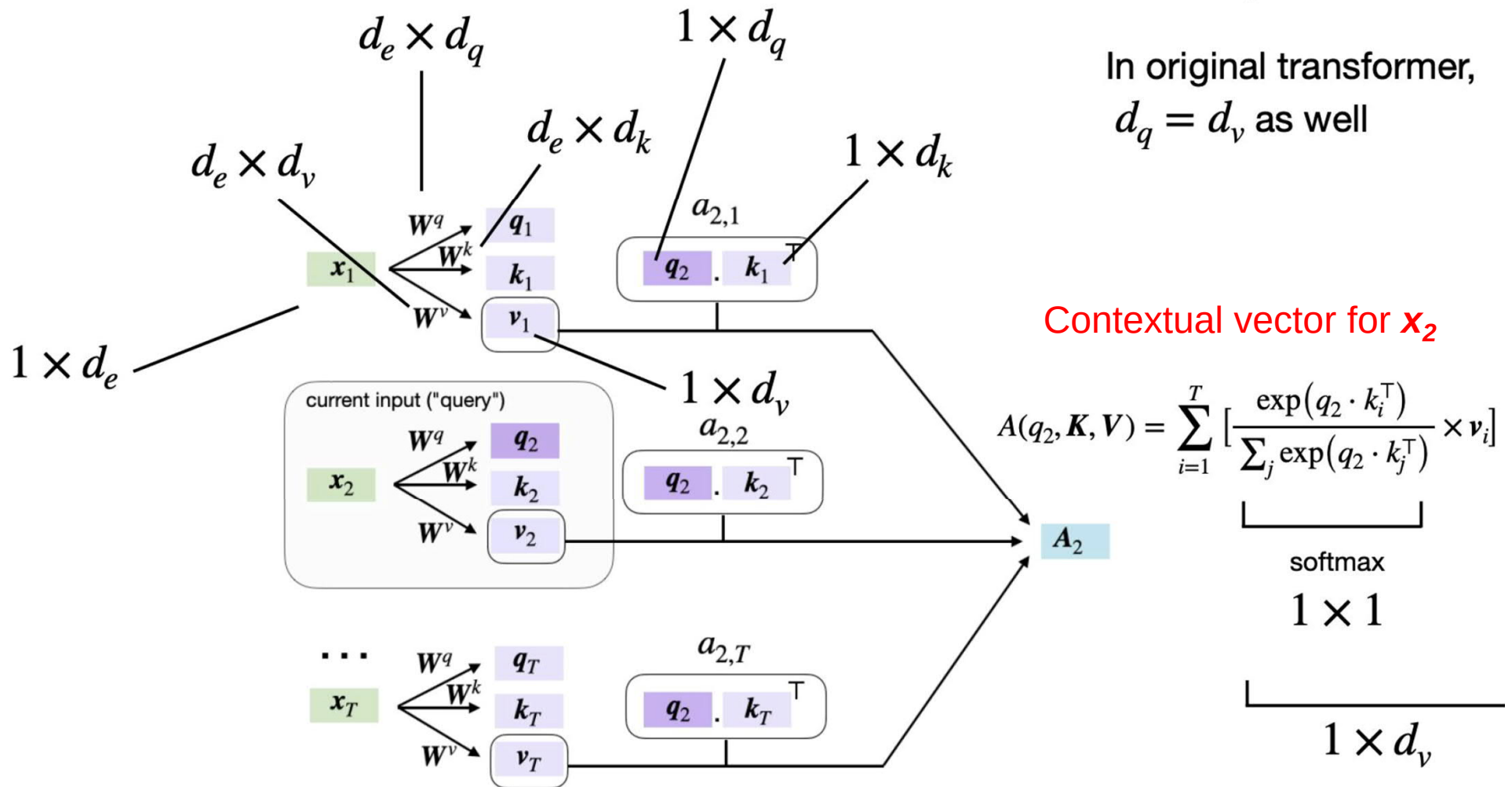


# Dimensions of Matrices and Vectors

$d_e$  = embedding size (original transformer = 512)

where  $d_q = d_k$

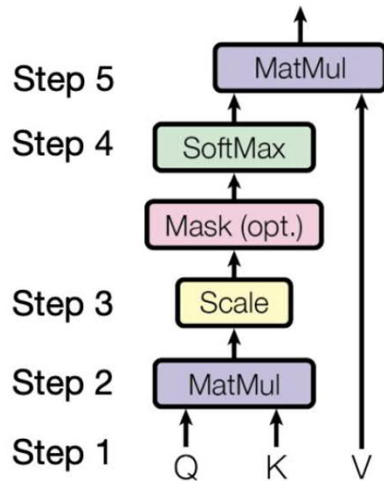
In original transformer,  
 $d_q = d_v$  as well



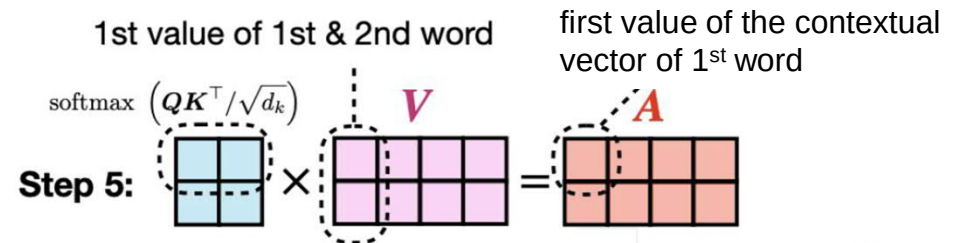
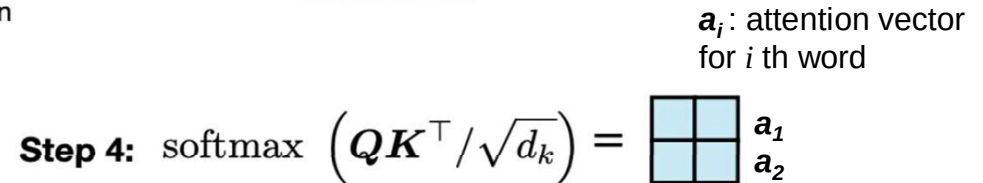
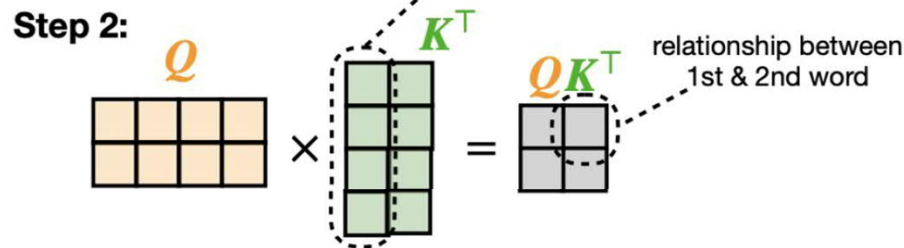
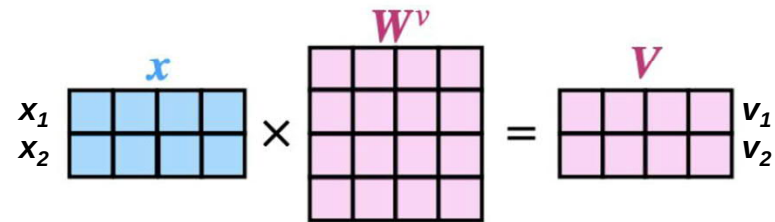
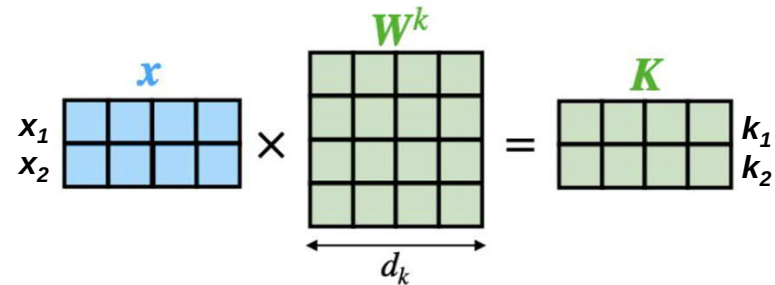
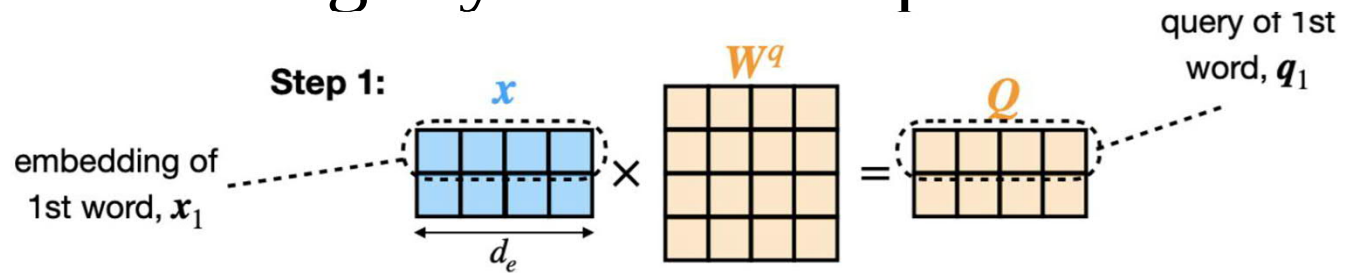


# Actual Attention: Slightly More Complicated

## Scaled Dot-Product Attention

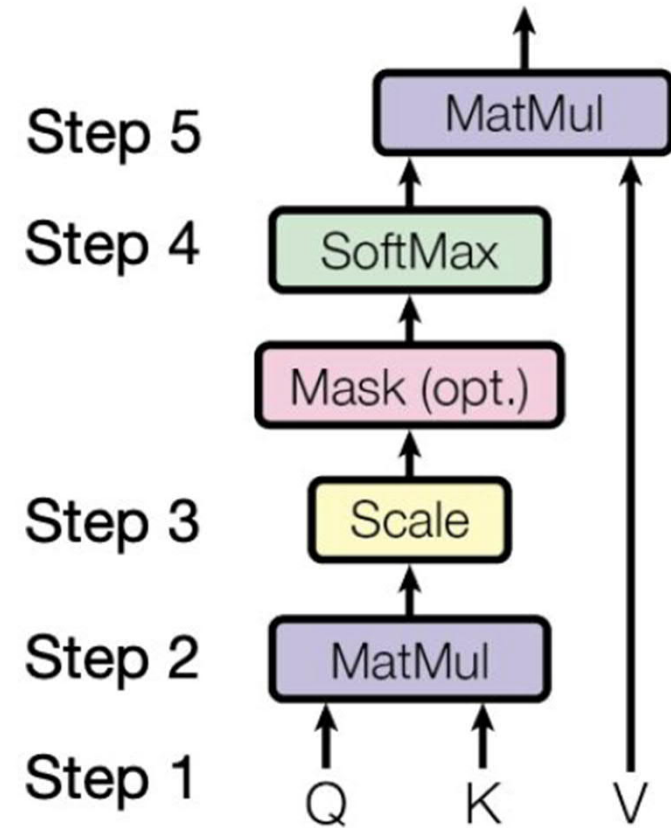


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., 2017. Attention Is All You Need.



# Attention Formula (Single Head)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



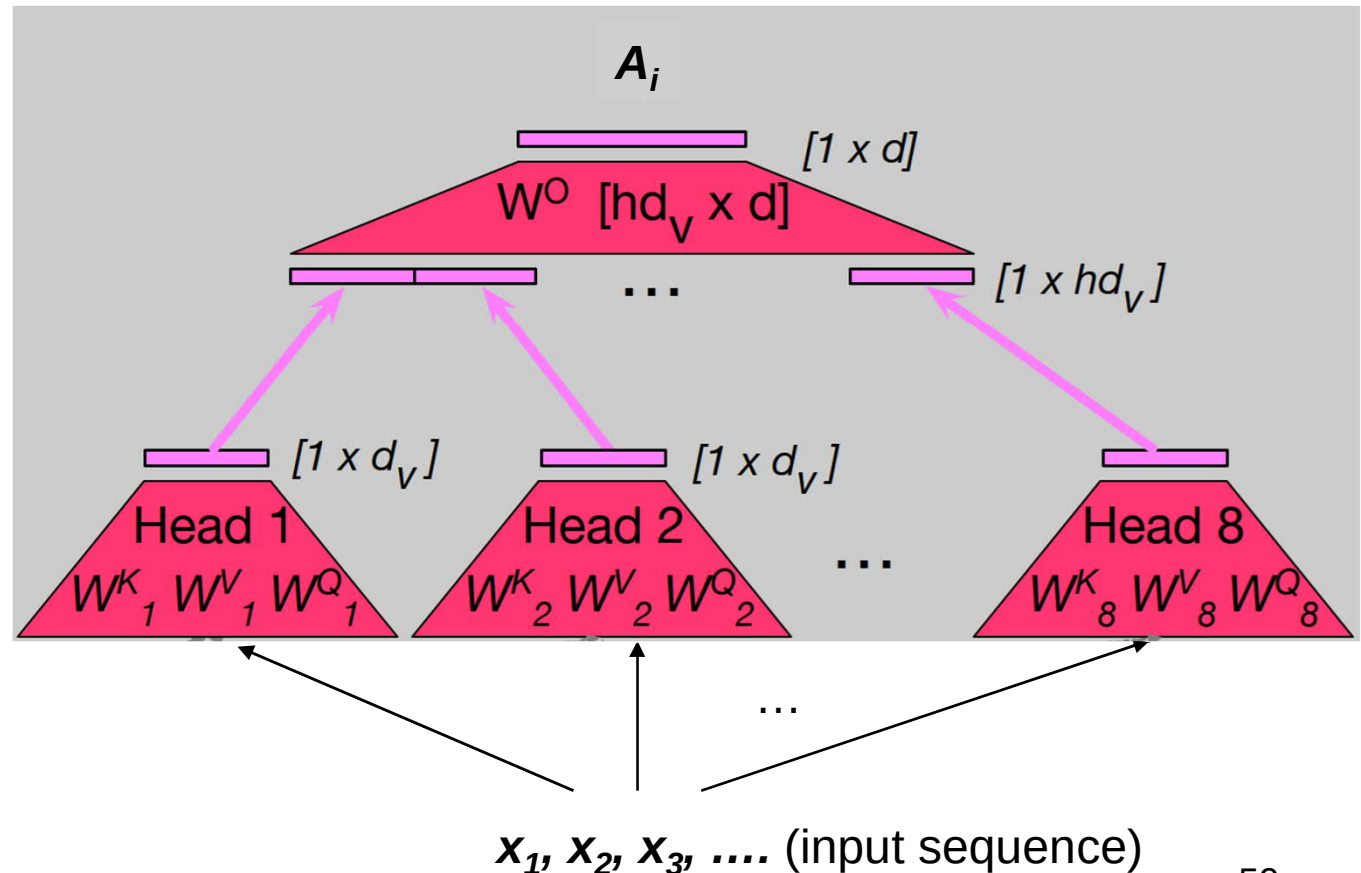
# Multi-Head Attention

- ▶ Project the embeddings of input words multiple times, each with a set of  $W^K$ ,  $W^V$  and  $W^Q$ .
- ▶ Results are concatenated and project to a smaller dimension

Project down to  $d$  dimension

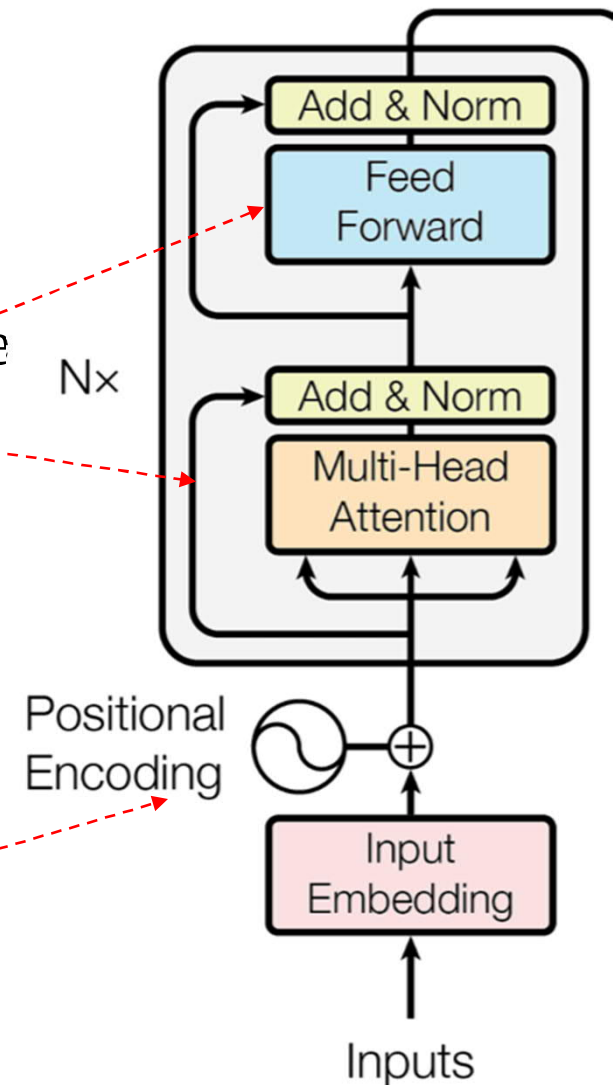
Concatenate outputs

Each head attend differently to context (considering different aspects)



# Transformer Encoder

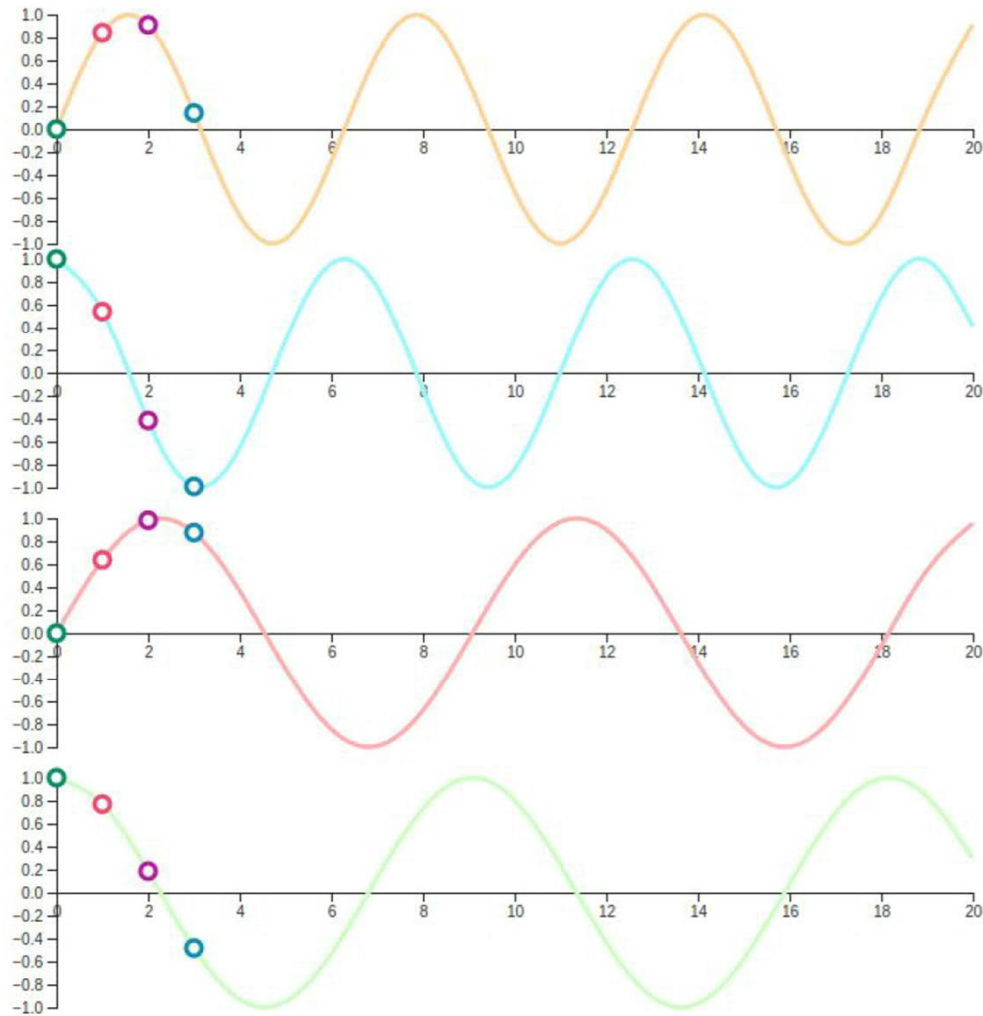
- ▶ Multi-head attention is just the main part of the Transform Encoder block
- ▶ A residual connection is added to mitigate gradient vanishing
- ▶ A feed-forward layer is added after a normalization layer
  - ▶ Converts the contextual embedding of each word using a non-linear activation function
- ▶ At the input part, positional encoding is added to the input embedding



# Positional Encoding

- ▶ Provides information about the order of tokens in a sequence, ensuring the Transformer understands word order despite processing words/tokens in parallel.
- ▶ Use sine and cosine functions to create unique positional embeddings that are **added** to the input token embeddings
- ▶ Without positional encoding, the model would treat the input as a bag of unordered words/tokens

# Positional Encoding



| p0    | p1    | p2     | p3     |     |
|-------|-------|--------|--------|-----|
| 0.000 | 0.841 | 0.909  | 0.141  | i=0 |
| 1.000 | 0.540 | -0.416 | -0.990 | i=1 |
| 0.000 | 0.638 | 0.983  | 0.875  | i=2 |
| 1.000 | 0.770 | 0.186  | -0.484 | i=3 |

## Positional Encoding

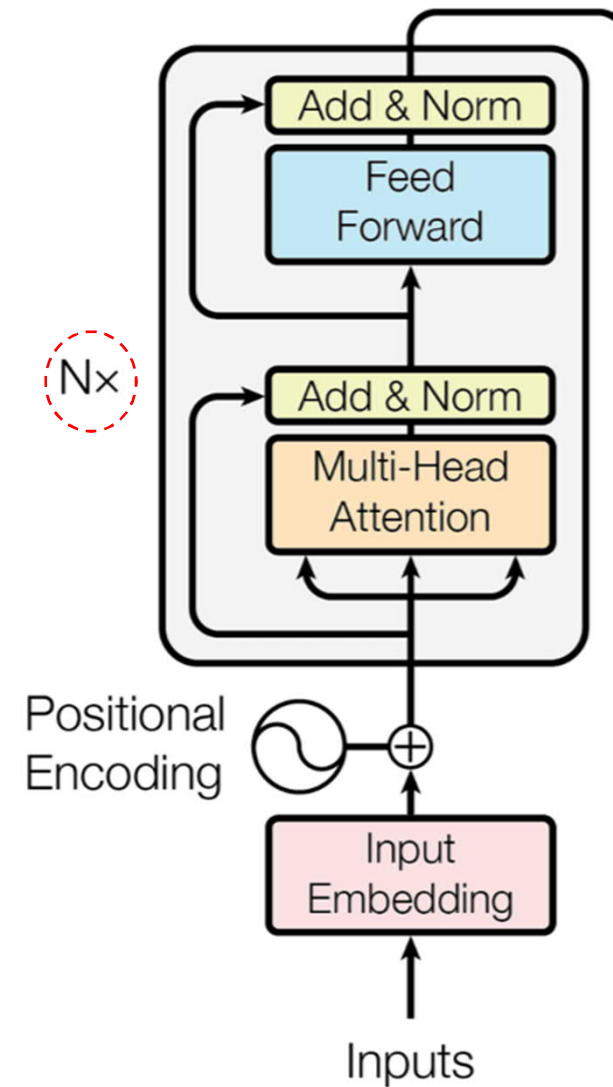
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- $pos$  = token position (0, 1, 2, ...)
- $i$  = dimension index (0, 1, 2, ...)
- $d_{model}$  = embedding dimension (e.g., 512)

# Transformer Encoder (N Blocks)

- ▶ Can have N blocks
- ▶ BERT-Small: 12 blocks
- ▶ BERT-Large: 24 blocks



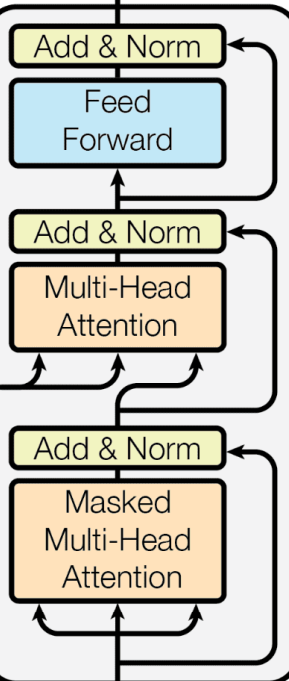
# Transformer Architecture

## Decoder

Output  
Probabilities

Softmax

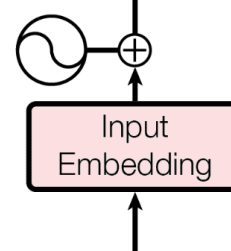
Linear



## Encoder

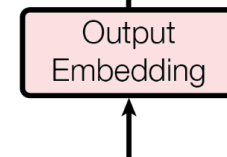
$N \times$

Positional  
Encoding



Inputs

Positional  
Encoding



Outputs  
(shifted right)

## Encoder

- Processes the input sequence
- Creates a contextualized embedding for each input token
- Captures the meaning of the input sequence and relationships between words.

## Decoder

- Generates the output sequence step-by-step, using
  - Encoder's contextualized representations
  - Its own previously generated tokens



# How to Train $W^q$ , $W^k$ , $W^v$

- ▶ Depends on the model
- ▶ For the BERT model:
  - ▶ Use a huge set of corpora
    - ▶ Books Corpus: This dataset contains
      - ▶ approximately 800 million words from over 11,000 unpublished books.
      - ▶ a variety of genres, which helps the model learn diverse language patterns.
    - ▶ English Wikipedia: This dataset consists of English Wikipedia articles,
      - ▶ totaling around 2.5 billion words.
      - ▶ a broad range of topics and formal writing styles, contributing to the model's general language understanding.
  - ▶ Trained with unlabeled text on two NLP tasks:
    - ▶ Masked language model
    - ▶ Next sentence prediction

# Outline

- ▶ Introduction and applications
- ▶ Traditional methods
  - ▶ Text Representation
  - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)
  - ▶ Neural network-based embeddings
    - ▶ Static Embeddings
    - ▶ Contextual Embeddings with Transformers
  - ▶ Text classification with embeddings

# Usefulness of Word Embeddings

- ▶ Can be used to compute similarity between words

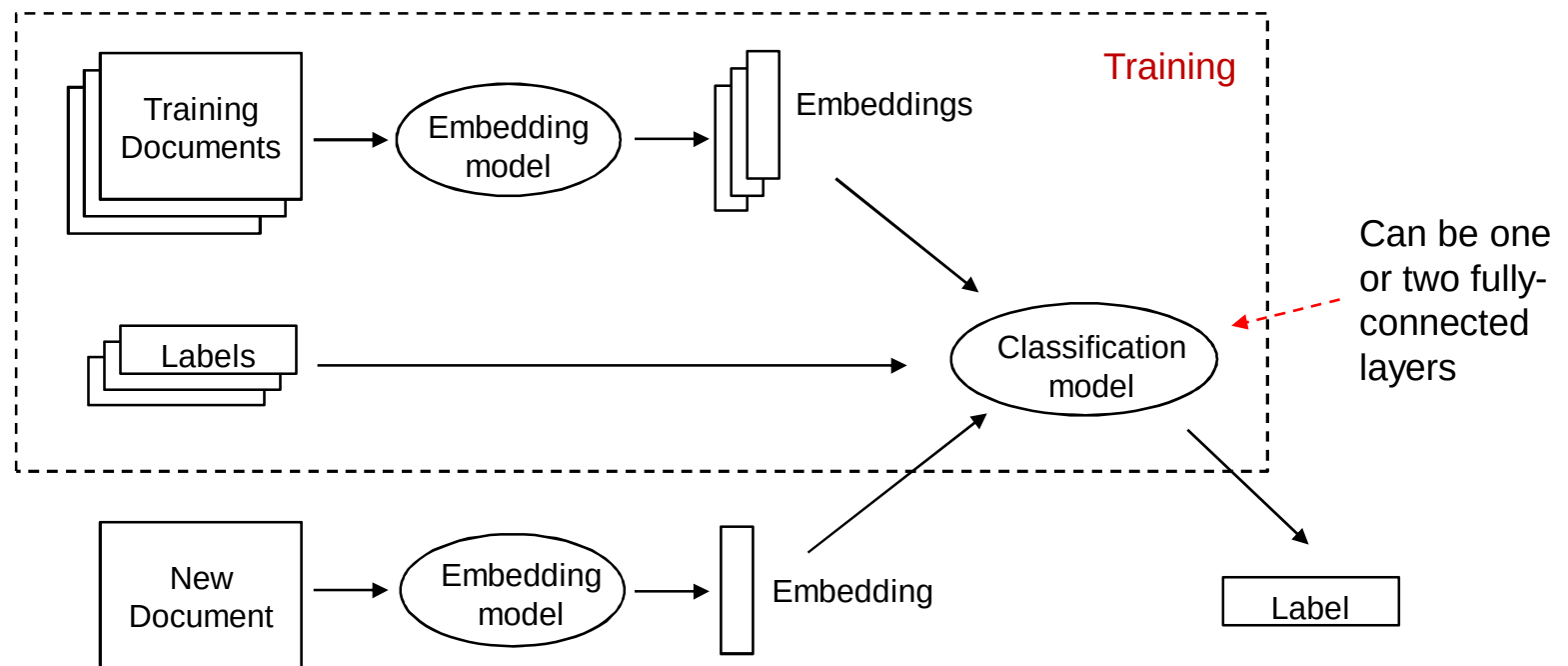
$$\text{cosine\_similarity}(w_i, w_j) = \frac{\vec{w}_i \cdot \vec{w}_j}{\|\vec{w}_i\| \|\vec{w}_j\|}$$

where  $w_i$  and  $w_j$  are words and  $\vec{w}_i$  and  $\vec{w}_j$  are word vectors

- ▶ Useful for tasks, such as **text matching**, **information retrieval**, etc.
- ▶ Building block for longer text embedding
  - ▶ **Sentence embedding** (Sent2Vec, Sentence-BERT, etc.)
  - ▶ **Paragraph/Document** Embedding (doc2vec)
- ▶ Used in many other NLP tasks, such as
  - ▶ Text classification
  - ▶ Machine translation
  - ▶ Summarization
  - ▶ Question answering

# Text Classification with Embeddings

- ▶ Objective is to learn to classify a document
- ▶ Two main steps:
  - ▶ Compute an embedding for the document
  - ▶ Classify the embedding into a class



# How to Get Document/Text Embeddings

- ▶ There are a few ways to do so, e.g.,
  - 1) Compute a *document vector* with an average of word/token embeddings (classic)
  - 2) **Fine-tune** a pre-trained model (e.g., BERT, RoBERTa, Longformer) to get a *sentence/text embedding*.
  - 2) **Use a pre-trained model for** *sentence/text embeddings* (e.g., SBERT, SimCSE, etc.)

## (1) Classic Methods for Computing Document Vector with Word/Token Embeddings

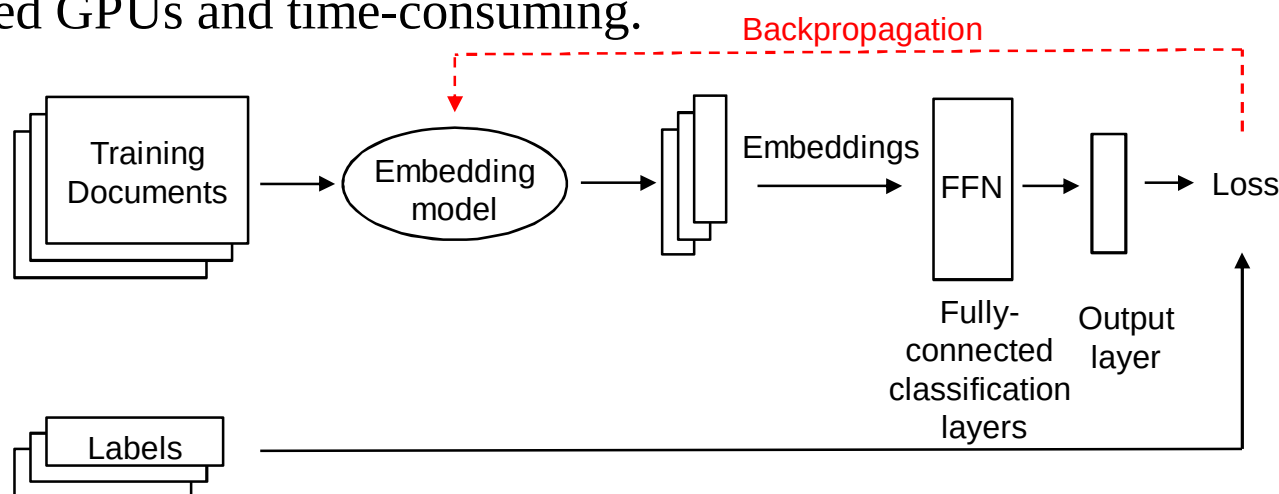
- ▶ Simple averaging on word embeddings
  - ▶ Average the embeddings of the words occurring in the document
- ▶ TF-IDF weighted averaging on word embeddings
  - ▶ Using the TF-IDF value of a word in the document as the weight for the word when averaging word embeddings
- ▶ Word embeddings can be
  - ▶ Pre-trained with a large text corpus (e.g., Word2Vec, GloVe, FastText)
  - ▶ Fine-tuned with the training data for the classification task
  - ▶ Freshly trained with the training data (seldom-used)

## (2) Fine-tune Pre-trained Sentence/Text Embedding Model

- ▶ Some pre-trained transformer-based embedding models, like BERT, RoBERTa, Longformer, etc.
  - ▶ Put a special token [CLS] at the beginning of an input sentence/text
  - ▶ Can use the [CLS] token embedding as the sentence/text embedding
- ▶ Their input sizes ( $m$  tokens) vary:
  - ▶ BERT, RoBERTa: 512 tokens
  - ▶ Longformer: 4096 tokens
- ▶ If document fits the input size, the [CLS] token embedding is the document embedding
- ▶ If not,
  - ▶ take the first or middle  $m$  tokens of the document to classify
  - ▶ or chunk the document into chunks and average the predictions over the chunks.

## (2) Fine-tune Pre-trained Sentence/Text Embedding Model (*Cont'd*)

- ▶ These models (BERT, RoBERTa, Longformer) are trained
  - ▶ for producing contextual word/token embeddings with self-supervised tasks masked token prediction and next sentence prediction.
- ▶ For classification, it is better to **fine-tune** the model with the classification training data to obtain good accuracy.
  - ▶ Need GPUs and time-consuming.





### (3) Use a Pre-trained Sentence/Text Embedding without Fine-tuning

- ▶ Some transformer models are trained to produce sentence/text embeddings that better capture the meaning
  - ▶ Sentence-BERT (SBERT) – most commonly used
    - ▶ A framework or method that fine-tunes BERT or its variants (e.g., RoBERTa, MPNet, etc.) to produce fixed-size sentence/text embeddings.
    - ▶ Uses two (or three) identical BERT-like models with shared weights to process sentence pairs (or triplets) independently
    - ▶ Trained on multiple loss functions such as contrastive loss, ensuring that positive pairs (similar sentences) are closer in the vector space than negative pairs (dissimilar sentences) by a certain margin.
  - ▶ Other methods: SimCSE, USE

### (3) Use a Pre-trained Sentence/Text Embedding without Fine-tuning (*Cont'd*)

- ▶ A popular library for pre-trained sentence/text embedding models are Hugging Face's
  - ▶ *sentence-transformers*
- ▶ The *sentence-transformers* library
  - ▶ Can efficiently produce sentence, text, and word embeddings using pre-trained transformer models
  - ▶ Contain many base models:
    - ▶ [https://www.sbert.net/docs/pretrained\\_models.html](https://www.sbert.net/docs/pretrained_models.html)

### (3) Use a Pre-trained Document/Text Embedding without Fine-tuning (*Cont'd*)

#### ▶ Sample code using sentence\_transformers

```
!pip install -U sentence_transformers

from sentence_transformers import SentenceTransformer

# Initialize the SBERT model
model = SentenceTransformer('bert-base-nli-mean-tokens') # Example base model based on BERT-Base

# List of reviews, each review can be a single or multiple sentences
reviews = [
    "Great product, fast shipping!",
    "Not happy with the purchase. The item arrived late and was damaged.",
    "Excellent quality, would buy again."
]

# Generate embeddings for each review
review_embeddings = model.encode(reviews)
```

Each element in `review_embeddings` is the vector representation of the corresponding review

# Summary

- ▶ Text classification has many applications
- ▶ Traditional methods
  - ▶ Text Representation
  - ▶ Text Preprocessing Steps
  - ▶ Can use any classification models
- ▶ Advanced techniques for text representation and classification
  - ▶ Neural network-based embeddings
    - ▶ Static Embeddings
    - ▶ Contextual Embeddings with Transformers
  - ▶ Text classification with embeddings

# Clustering

---

Aijun An

Department of Electrical Engineering and Computer Science  
York University

# Outline

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

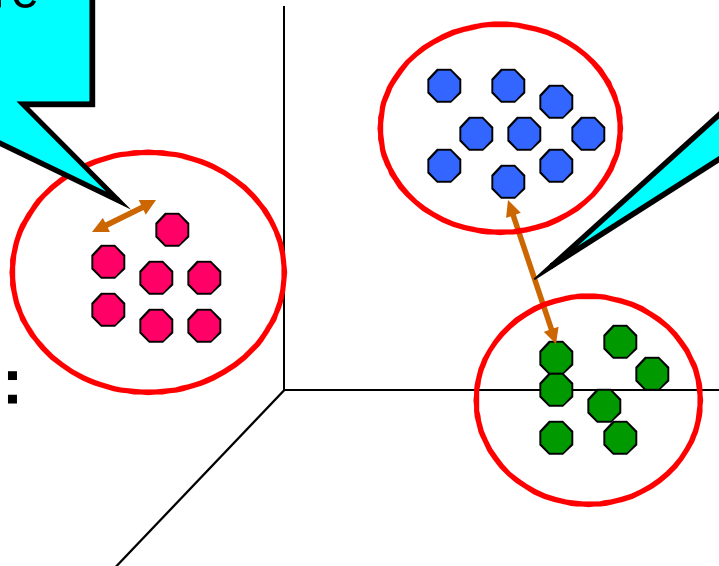
# What Is Clustering?

- ▶ Group data into clusters so that the points in one group are **similar** to each other and are as **different** as possible from the points in other groups

Intra-cluster distances are minimized

Inter-cluster distances are maximized

**Unsupervised learning:  
no predefined classes**



# Examples of Clustering Applications (1)

- ▶ **Customer Segmentation (Marketing):**
  - ▶ **Grouping customers** based on purchasing behavior, preferences, and demographics to tailor marketing strategies and product offerings.
- ▶ **Document Clustering (Text Mining):**
  - ▶ **Grouping similar documents** together, which is useful for organizing large document corpora, such as news articles, research papers, or social media posts.
- ▶ **Image Segmentation (Computer Vision):**
  - ▶ **Dividing an image into segments** for easier analysis, such as object recognition or medical image analysis.
  - ▶ E.g., Soil scientists filter trees from background



# Example of Image Segmentation

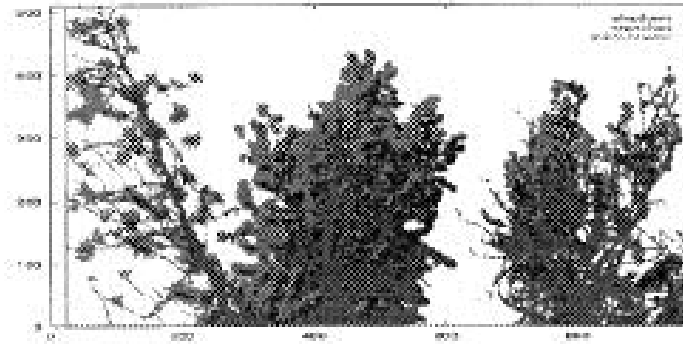
- Filtering real images
  - Images of trees taken in near-infrared band (NIR) and visible wavelength (VIS)
  - 512x1024 pixels and each of them contains a pair of brightness values (NIR,VIS)



The images taken in NIR and VIS

1<sup>st</sup> step: 284 seconds

2<sup>nd</sup> step: 71 seconds



The sunlit leaves, branches and shadows

# Examples of Clustering Applications (2)

## ▶ **Bioinformatics & Healthcare**

- ▶ Gene expression analysis (**grouping similar genes** to predict possible functions of genes with unknown function )
- ▶ Disease outbreak detection (**clustering similar symptoms or patients**)
- ▶ Drug discovery (**clustering similar chemical compounds**)

## ▶ **Social Network Analysis:**

- ▶ **Grouping users based on their interactions**, such as identifying communities or clusters of individuals with similar interests in social networks.

## ▶ **Recommendation Systems:**

- ▶ **Grouping users or items based on preferences** to make personalized recommendations, for instance in movie recommendations or e-commerce platforms.

# Examples of Clustering Applications (3)

## ▶ **City-planning:**

- ▶ Identifying groups of houses according to their house type, value, and geographical location

## ▶ **WWW**

- ▶ Cluster web documents
- ▶ Cluster web log data to discover groups of users

## ▶ **Anomaly Detection:**

- ▶ Identifying outliers or abnormal behavior in data, such as fraud detection in financial transactions or network intrusion detection.

# Types of Data to Be Clustered (1)

- ▶ Tabular data (Data matrix)
  - ▶ Each row is an data object (called example)
    - ▶ E.g., a customer, a document, an image, a data point in Euclidean space, etc.
  - ▶ Each column represents an attribute or feature

| Customer ID | Age | Income | Gender | Membersh<br>ip Type |
|-------------|-----|--------|--------|---------------------|
| 101         | 25  | 50000  | Male   | Gold                |
| 102         | 32  | 62000  | Female | Silver              |
| 103         | 45  | 75000  | Male   | Platinum            |
| 104         | 29  | 48000  | Female | Gold                |
| 105         | 36  | 70000  | Male   | Silver              |

- ▶ No class labels. Learning is *unsupervised*.

# Types of Data to Be Clustered (2)

## ▶ Sets

- ▶ Documents, each represented by e.g., a set of n-grams,
- ▶ Movies:

| Movie ID | Genre Set                    |
|----------|------------------------------|
| 1        | {Action, Adventure, Sci-Fi}  |
| 2        | {Drama, Romance}             |
| 3        | {Action, Thriller, Crime}    |
| 4        | {Comedy, Family}             |
| 5        | {Adventure, Fantasy, Action} |
| 6        | {Drama, Crime, Mystery}      |

## ▶ Sequences

- ▶ genes, click streams on a web site, time-series, etc.

## ▶ Graphs

- ▶ Protein interaction networks, chemical compounds, etc.

# Attribute Types of Tabular Data

## ▶ Example of data matrix

| Customer ID | Age | Income | Gender | Membership Type |
|-------------|-----|--------|--------|-----------------|
| 101         | 25  | 50000  | Male   | Gold            |
| 102         | 32  | 62000  | Female | Silver          |
| 103         | 45  | 75000  | Male   | Platinum        |
| 104         | 29  | 48000  | Female | Gold            |
| 105         | 36  | 70000  | Male   | Silver          |

## ▶ Attributes can be

- ▶ Numeric (real values, integers)
- ▶ Categorical (Symbolic)
  - ▶ Nominal (values without meaningful order)
    - ▶ Color, gender, product types
  - ▶ Ordinal (values with a meaningful order)
    - ▶ Temperature (low, medium, high)

# Outline

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
- ▶ Disk-based large-scale clustering algorithms
  - ▶ BFR
  - ▶ CURE
- ▶ Cluster Validity Measures

# Commonly Used Distance Measures

- ▶ Minkowski Distance (General Form)
  - ▶ For numeric data
- ▶ Cosine Distance
  - ▶ For numeric data
- ▶ Jaccard Distance
  - ▶ For sets
- ▶ Hamming Distance
  - ▶ For strings of the same lengths



# Distance Metric

- ▶ A distance measure is a ***distance metric*** if it satisfies the following 4 conditions:
  - ▶ **Non-negativity:**  
$$d(x,y) \geq 0$$
  - ▶ **Identity of Indiscernibles:**  
$$d(x,y) = 0 \text{ if and only if } x=y$$
  - ▶ **Symmetry:**  
$$d(x,y) = d(y,x)$$
  - ▶ **Triangle Inequality:**  
$$d(x,z) \leq d(x,y) + d(y,z)$$
- ▶ For robust clustering, always prefer metrics

# Distance (or Similarity) Measures

- ▶ For data containing only *numeric attributes*, a popular measure of distance between two data objects,  $i$  and  $j$ , is *Minkowski distance*:

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)}$$

where  $i = (x_{i1}, x_{i2}, \dots, x_{ip})$  and  $j = (x_{j1}, x_{j2}, \dots, x_{jp})$  are two  $p$ -dimensional data objects, and  $q$  is a positive integer.

- ▶ If  $q = 1$ ,  $d$  is *Manhattan distance*

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

# Distance (or Similarity) Measures

- ▶ If  $q = 2$ ,  $d$  is Euclidean distance:

$$d(i, j) = \sqrt{(x_{i_1} - x_{j_1})^2 + (x_{i_2} - x_{j_2})^2 + \dots + (x_{i_p} - x_{j_p})^2}$$

- ▶ **Minkowski distance is a distance metric**

- ▶  $d(i, j) \geq 0$  (nonnegative)
- ▶  $d(i, i) = 0$  (distance to itself is 0, and iff the two objects are same)
- ▶  $d(i, j) = d(j, i)$  (symmetric)
- ▶  $d(i, j) \leq d(i, k) + d(k, j)$  (Triangular inequality)

- ▶ Also, one can use weighted distance (or other dissimilarity measures)

$$d(i, j) = \sqrt[q]{w_1 |x_{i_1} - x_{j_1}|^q + w_2 |x_{i_2} - x_{j_2}|^q + \dots + w_p |x_{i_p} - x_{j_p}|^q} \quad (q > 0)$$

# Distance (or Similarity) Measures

- ▶ Another similarity measure for data with only numeric attributes is cosine similarity:

$$\text{Similarity}(\mathbf{x}, \mathbf{y}) = \cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

where  $\theta$  is the angle between two examples  $\mathbf{x}$  and  $\mathbf{y}$ .

- ▶ **Cosine Distance** = 1- Cosine\_Similarity
- ▶ Cosine distance is not a distance metric. It does not satisfy
  - ▶ Identity of indiscernibles
  - ▶ Triangle inequality

# Commonly Used Distance Measures

## ▶ Jaccard Distance

- ▶ Jaccard similarity measures the similarity between two sets:  $S_1$  and  $S_2$ :

$$Sim(S_1, S_2) = |S_1 \cap S_2| / |S_1 \cup S_2|$$

- ▶ Jaccard distance =  $1 - Sim(S_1, S_2)$
- ▶ It is a distance metric

## ▶ Hamming Distance

- ▶ Measures how different two strings of equal length are.
- ▶ Counts the number of positions at which the corresponding symbols (usually bits or characters) are different.

- ▶ Example:

- ▶ String A: 1011101

- ▶ String B: 1001001

Distance(A, B) = 2

- ▶ It is a distance metric

# Distance Measures

- ▶ We focus on tabular data with Minkowski distance measure
- ▶ It is good for numerical data.
- ▶ What if data contains only *nominal attributes*?
- ▶ Method 1:
  - ▶ Convert a nominal attribute to a numerical attribute, for example:
    - ▶ yes  $\Rightarrow$  1 and no  $\Rightarrow$  0
    - ▶ blue  $\Rightarrow$  1, yellow  $\Rightarrow$  2, red  $\Rightarrow$  3, etc.
  - ▶ Problem?

# Distance Measures

- ▶ Method 2:

- ▶ One-hot encoding for each attribute

- ▶ For example, if the domain of attribute A is {a1, a2, a3}, we convert it into 3 attributes to represent A.
      - ▶ If A=a1, the values of the three attribute are 1, 0, 0.
      - ▶ If A=a2, the values of the three attribute are 0, 1, 0.
      - ▶ If A=a3, the values of the three attribute are 0, 0, 1.

# Distance (or Similarity) Measures

## ► Method 3:

- Distance between two instances  $\mathbf{x}$  and  $\mathbf{y}$ :

$$\text{Distance}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \text{dist}(x_i, y_i)$$

where  $\mathbf{x}=(x_1, x_2, \dots, x_m)$  and  $\mathbf{y}=(y_1, y_2, \dots, y_m)$

$$\text{dist}(x_i, y_i) = \begin{cases} 0 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i = y_i \\ 1 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i \neq y_i \end{cases}$$



# Distance Measures

- ▶ What if data contain *both numeric and nominal attributes*?
- ▶ Method 1:
  - ▶ One-hot encoding for each nominal attribute

- ▶ Method 2:

$$\text{Distance}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \text{dist}(x_i, y_i)$$

where  $\mathbf{x} = (x_1, x_2, \dots, x_m)$  and  $\mathbf{y} = (y_1, y_2, \dots, y_m)$

$$\text{dist}(x_i, y_i) = \begin{cases} 0 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i = y_i \\ 1 & \text{if } x_i \text{ and } y_i \text{ are nominal and } x_i \neq y_i \\ | \text{norm}(x_i) - \text{norm}(y_i) | & \text{if } x_i \text{ and } y_i \text{ are continuous} \end{cases}$$

# Outline

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

# Major Clustering Approaches

- ▶ Partitioning algorithms: Construct various partitions and then evaluate them by some criterion
- ▶ Hierarchical algorithms: Create a hierarchical decomposition of the set of data (or objects) using some criterion
- ▶ Density-based: based on connectivity and density functions
- ▶ Model-based: A model is hypothesized for each of the clusters and the idea is to find the best fit of that model to each other

# Partitioning Algorithms: Basic Concept

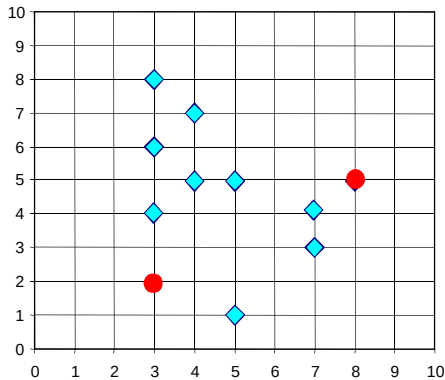
- ▶ Partition  $n$  objects into  $k$  clusters
  - ▶ Optimize the chosen partitioning criterion
- ▶ Global optimal: exhaustively enumerate all partitions
  - ▶ Infeasible in practice
- ▶ Heuristic methods: *k-means* and *k-medoids* algorithms
  - ▶ *k-means* (MacQueen'67): Each cluster is represented by the mean values of the objects in the cluster (as the cluster center, or centroid)
  - ▶ *k-medoids* or PAM (Partition around medoids) (Kaufman & Rousseeuw'87): Each cluster is represented by one of the objects in the cluster

# K-means

Group objects into  $k$  clusters:

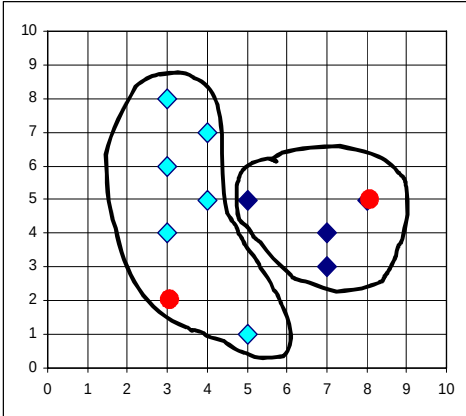
- ▶ Arbitrarily choose  $k$  objects as the initial cluster centers
- ▶ Until no change, do
  - ▶ (Re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster
  - ▶ Update the cluster means, i.e., calculate the mean value of the objects for each cluster

| Age Group | Percentage |
|-----------|------------|
| 18-24     | 15%        |
| 25-34     | 20%        |
| 35-44     | 25%        |
| 45-54     | 20%        |
| 55-64     | 15%        |
| 65-74     | 10%        |
| 75-84     | 5%         |
| 85+       | 5%         |

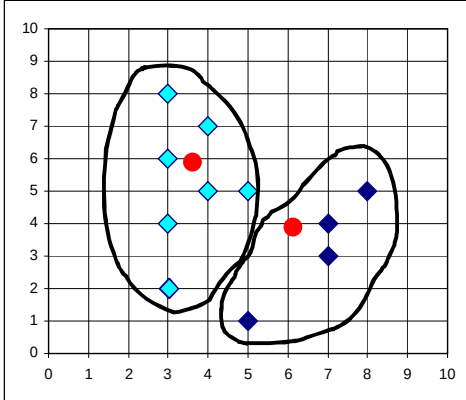


Arbitrarily choose K objects as initial cluster center

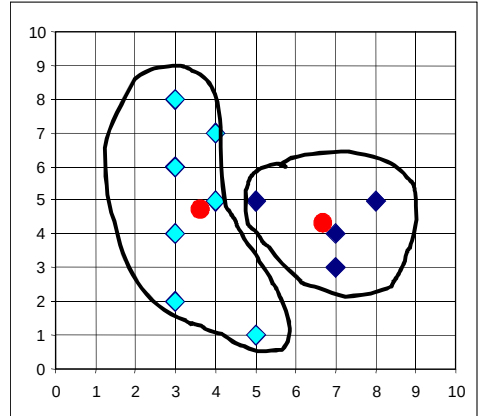
Assign each object to most similar center



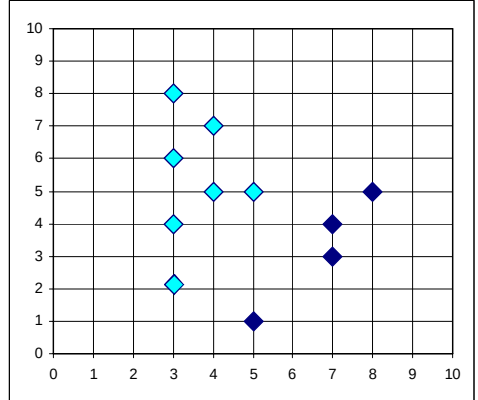
↑ reassign



Update the cluster means

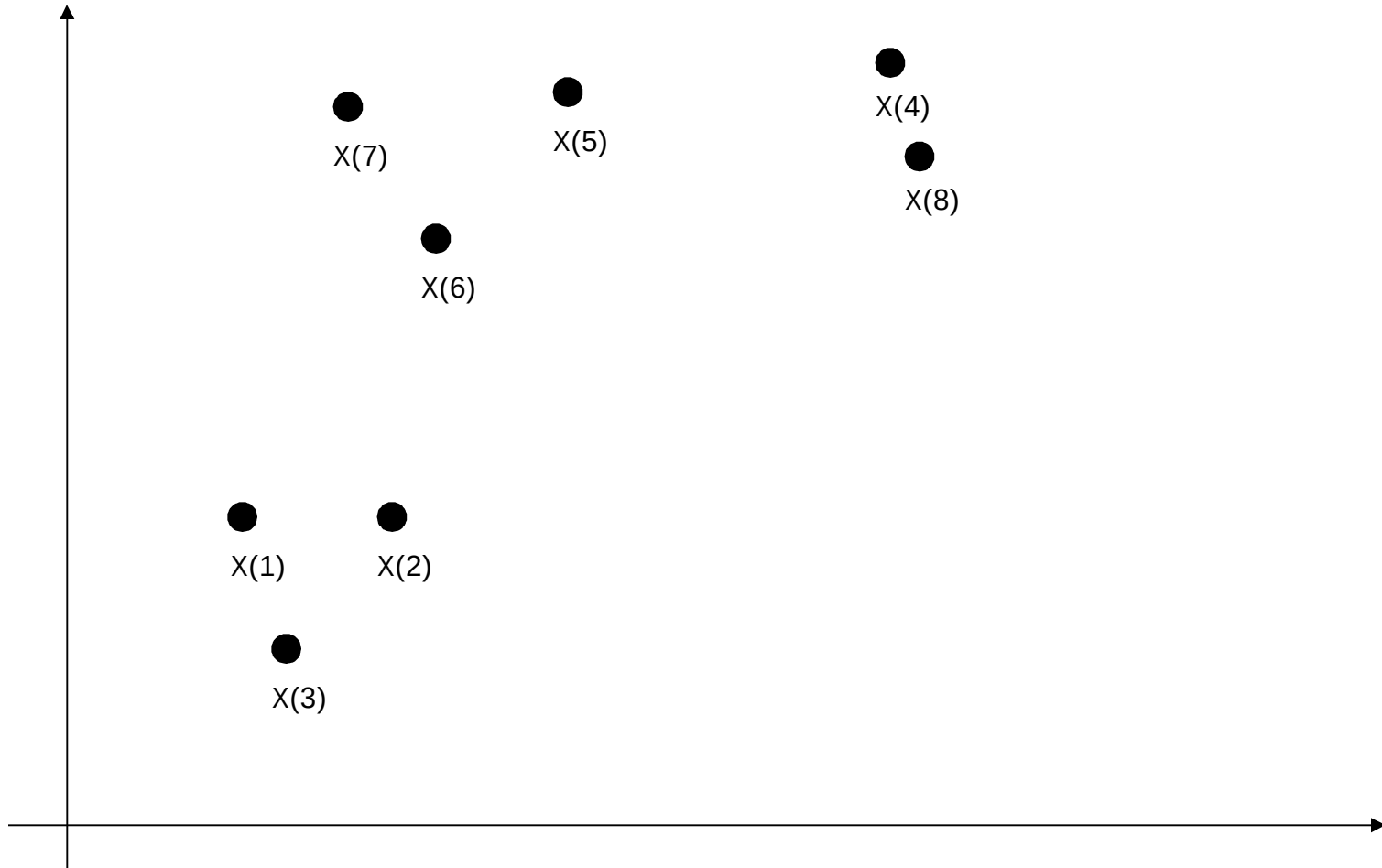


↓ reassign

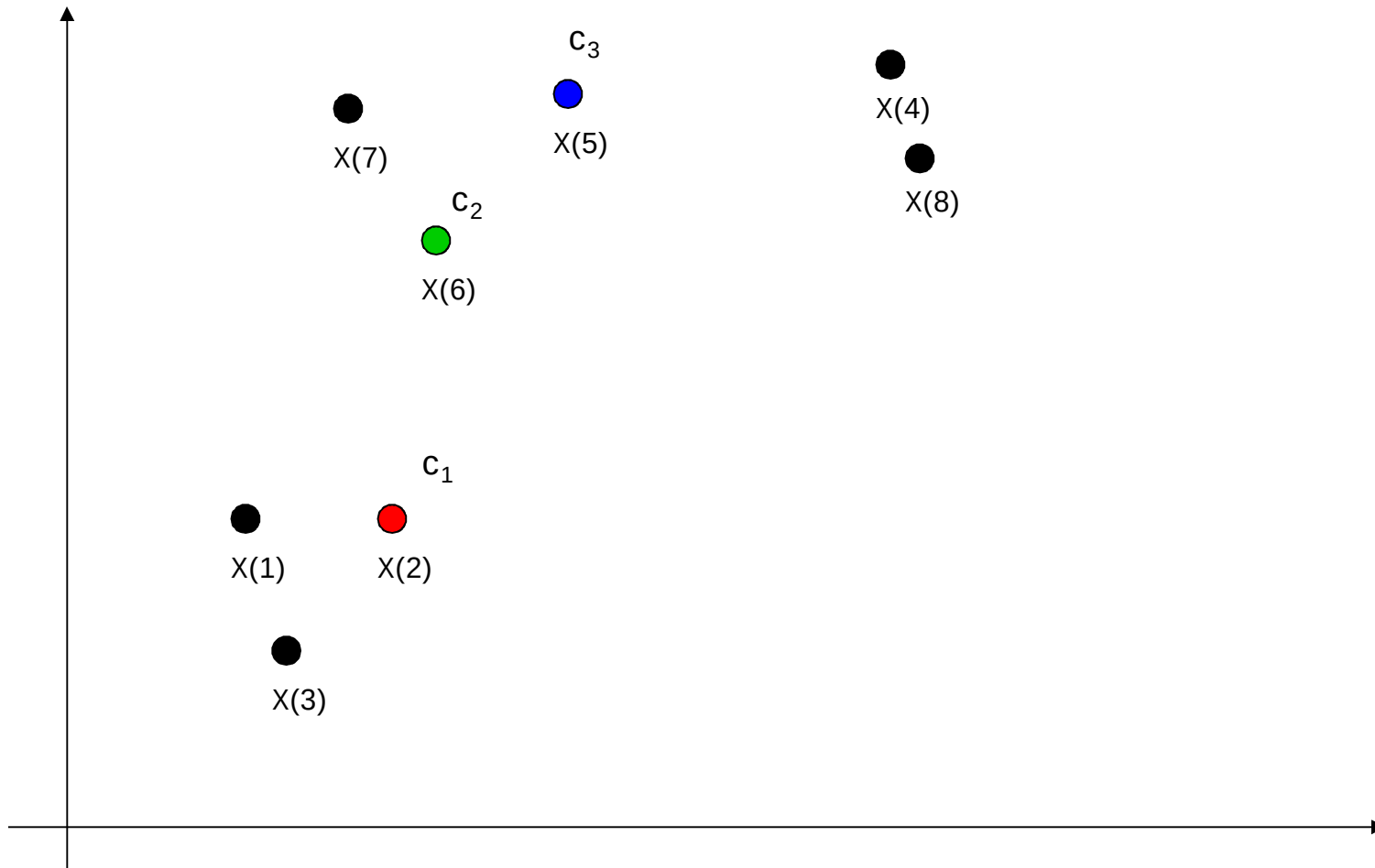


Update  
the  
cluster  
means

# K-means example

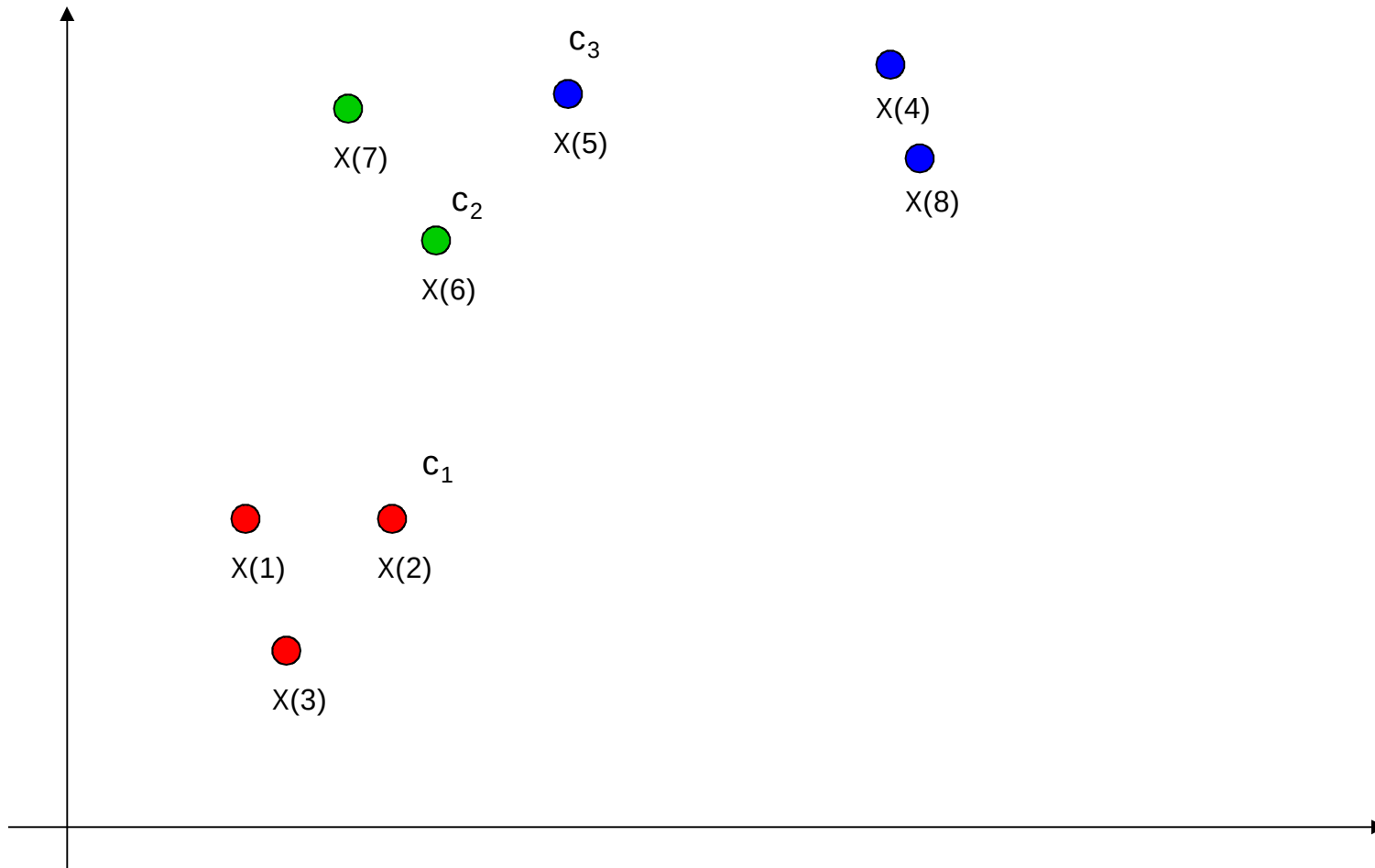


# K-means example

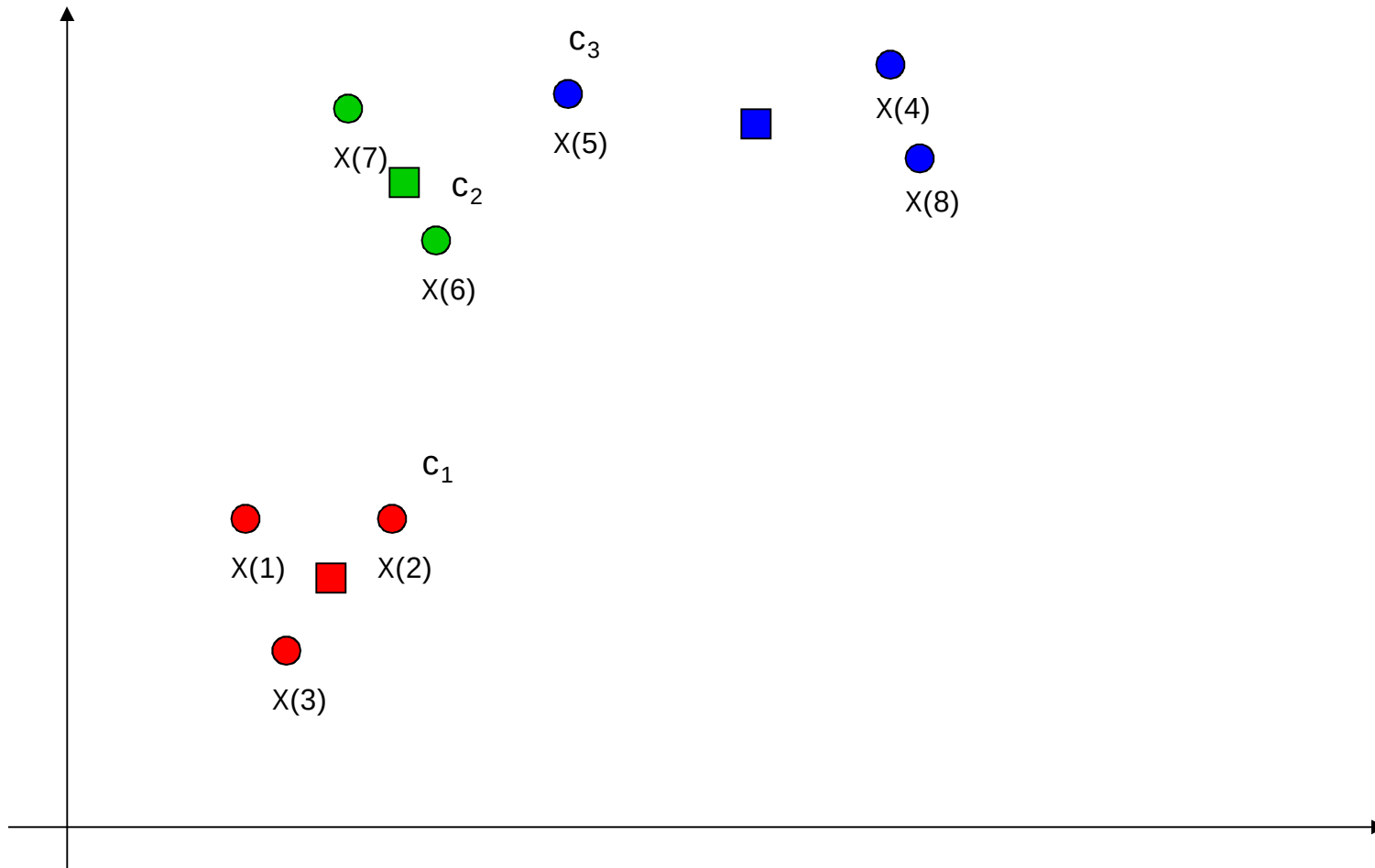




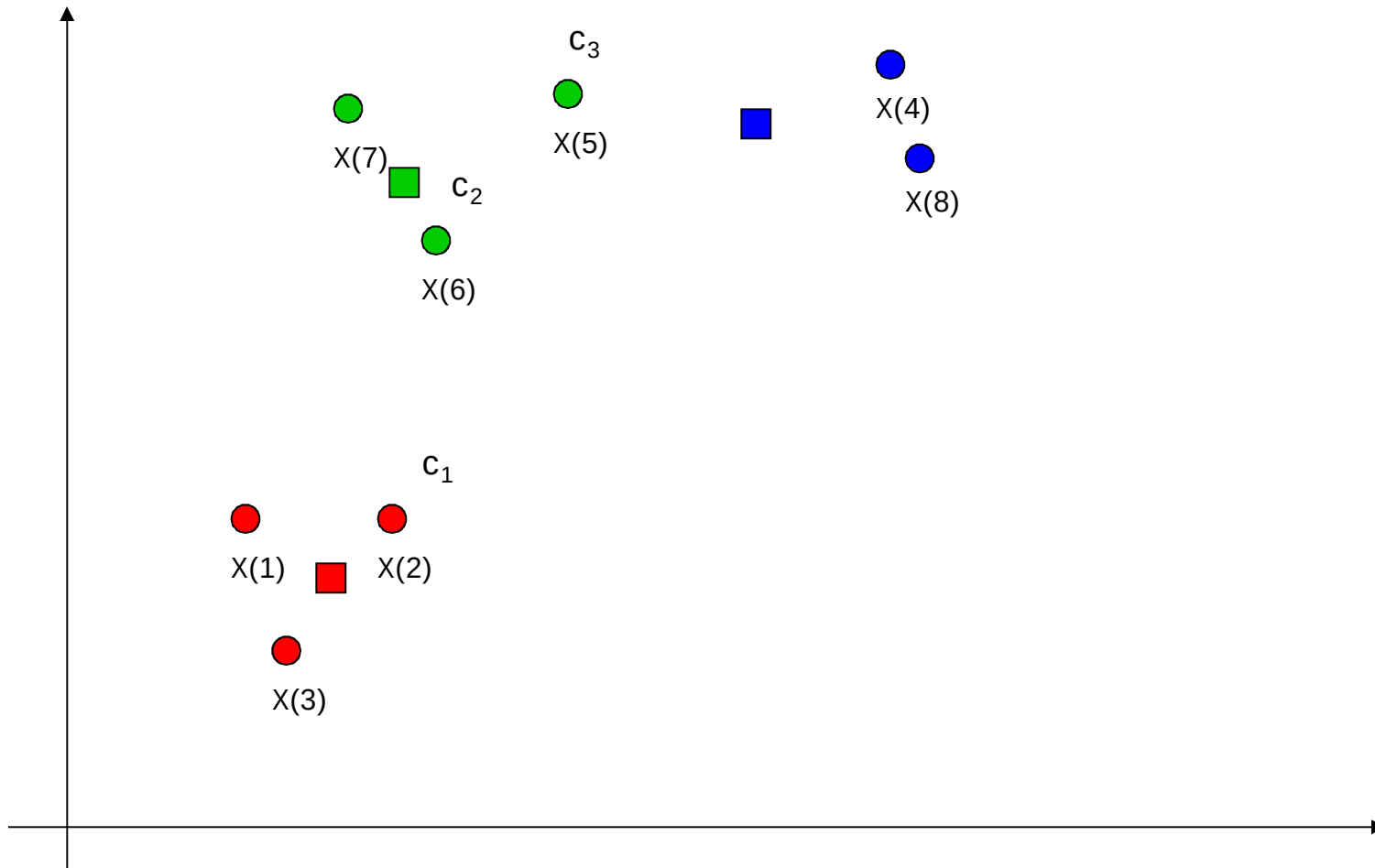
# K-means example



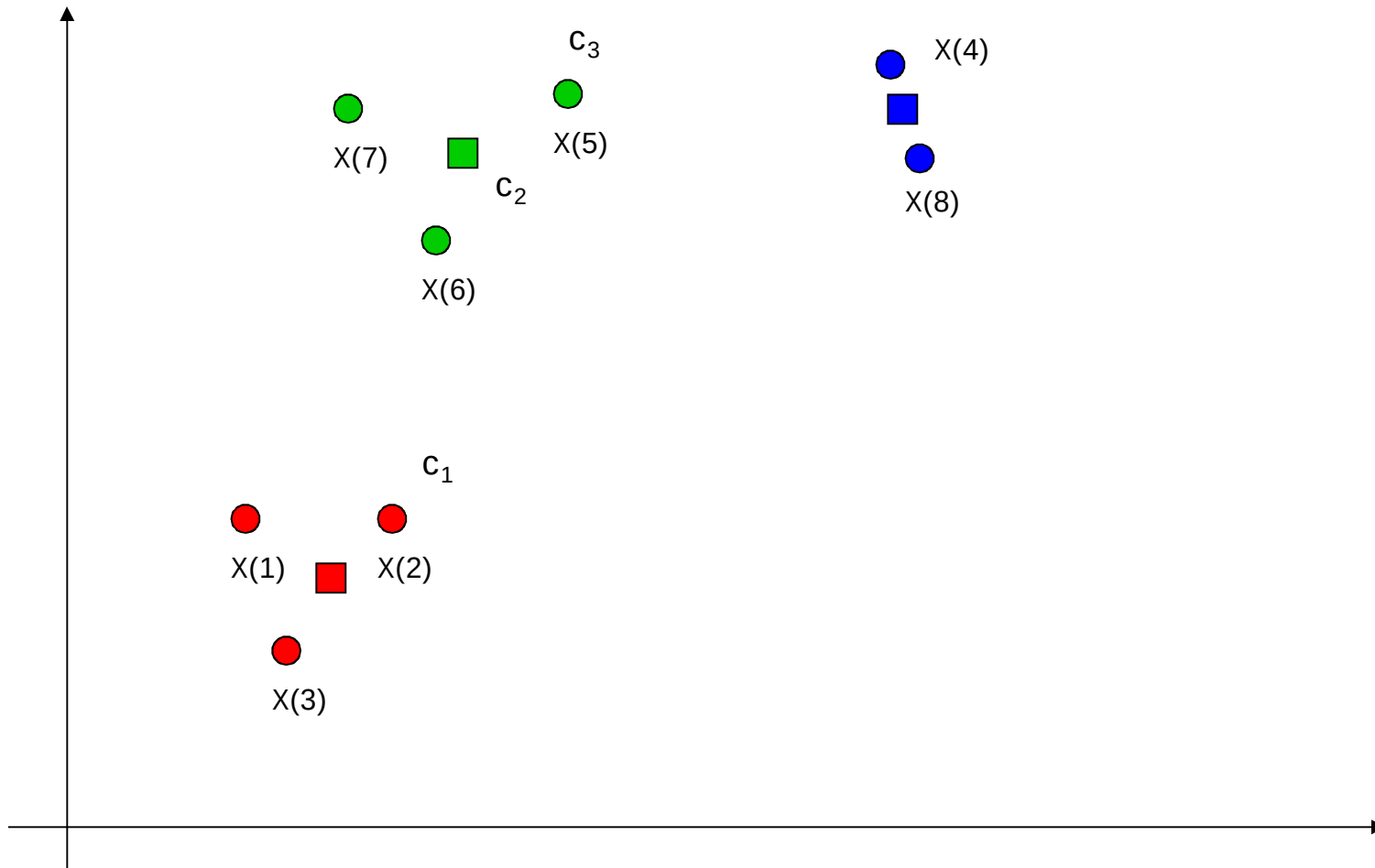
# K-means example



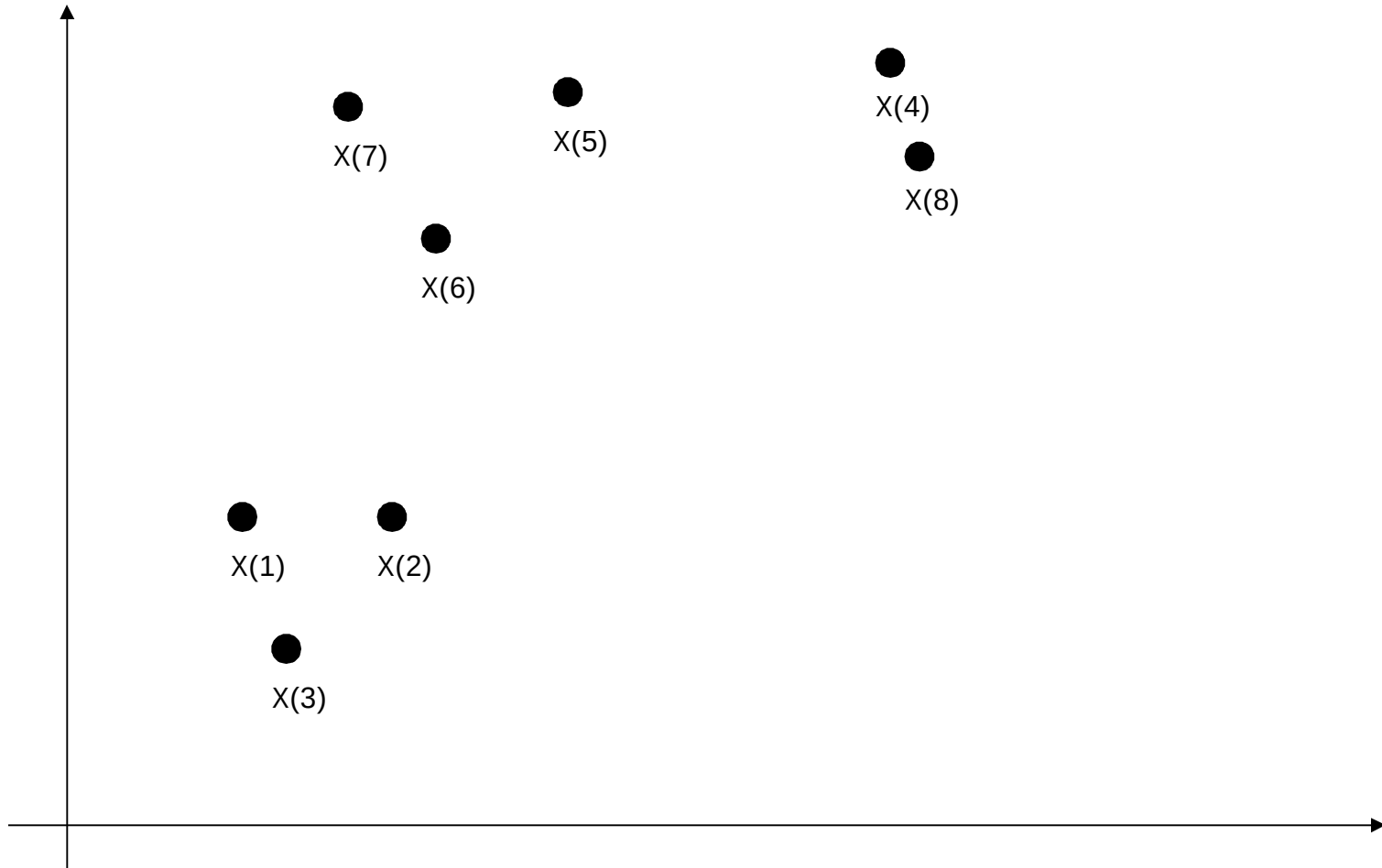
# K-means example



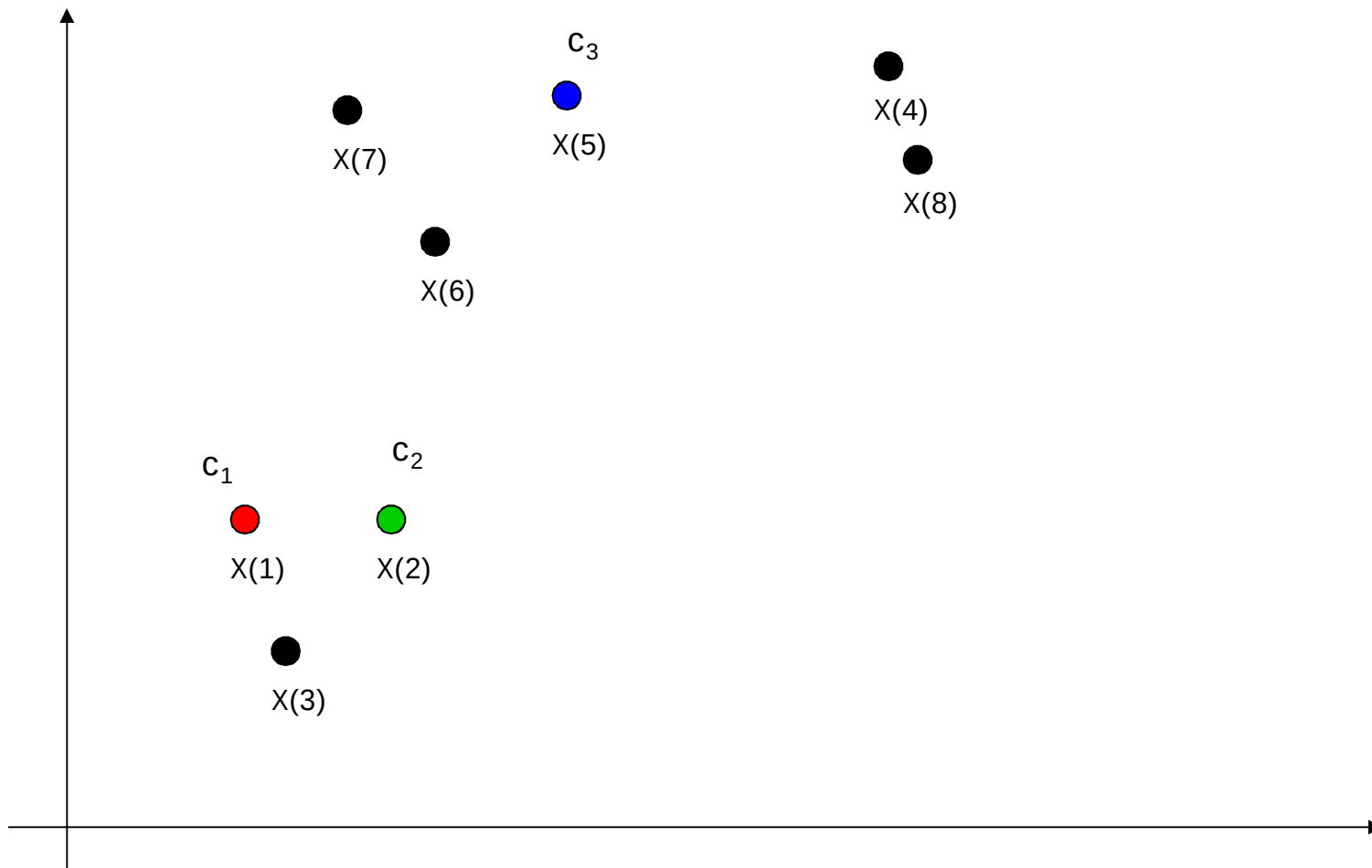
# K-means example



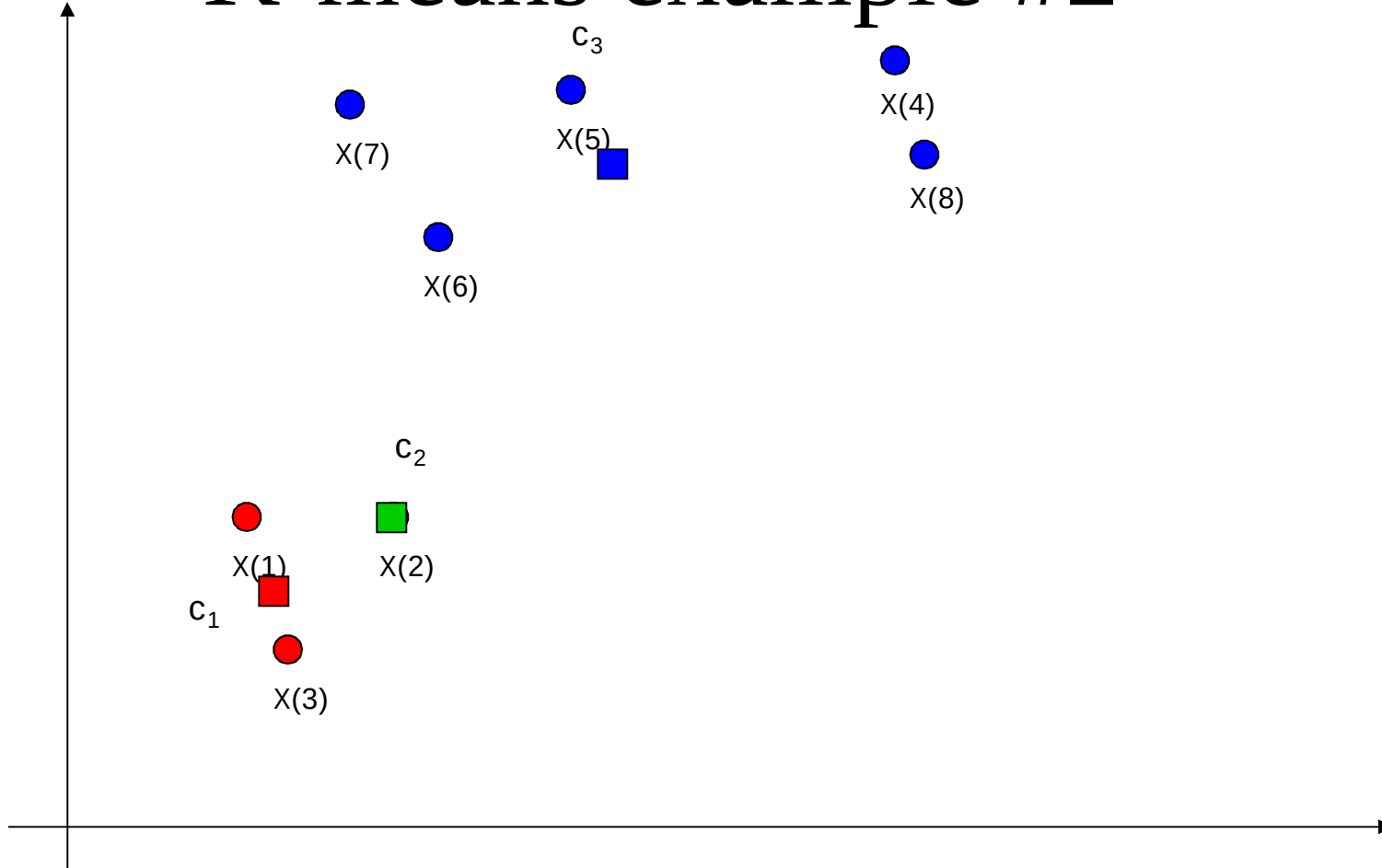
## K-means example #2



## K-means example #2



# K-means example #2



# How to Determine $k$

Several techniques. One of them is:

## ► Elbow Method

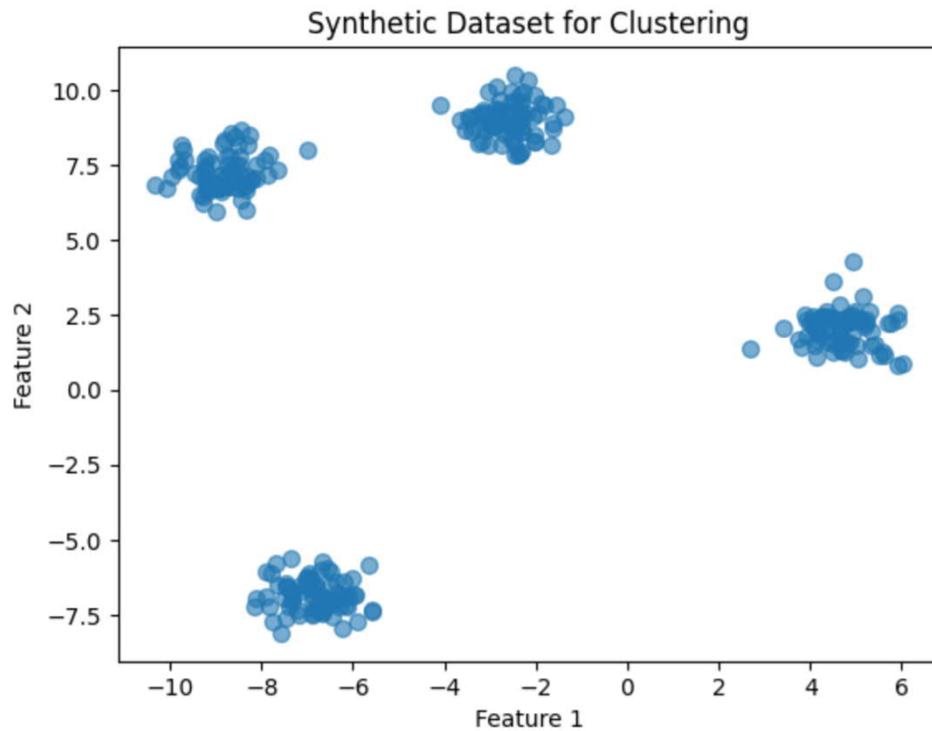
- Compute the *sum of squared distances (SSD)* between points and their assigned cluster centroids:

$$SSD = \sum_{i=1}^k \sum_{x \in C_i} dist(x, v_i)^2$$

- $v_i$  is the center of the cluster  $C_i$
- $k$  is the number of clusters
- Plot SSD against  $k$ 
  - Look for an "elbow point" where the rate of decrease sharply slows, indicating diminishing benefits from adding more clusters.

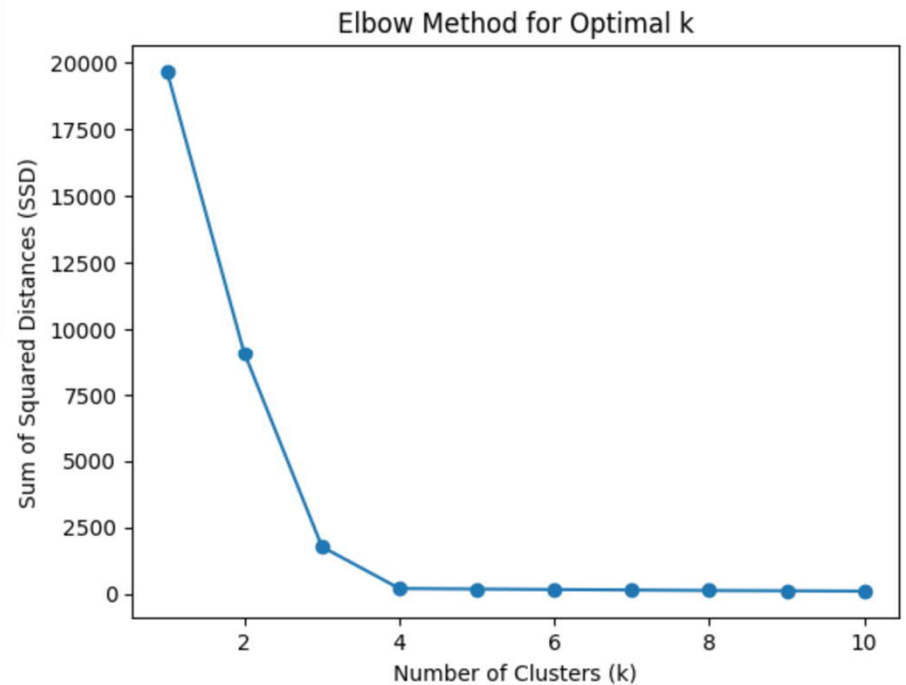


# Illustration of Elbow Method



The elbow is around **k=4**, matching the actual number of clusters in the synthetic dataset.

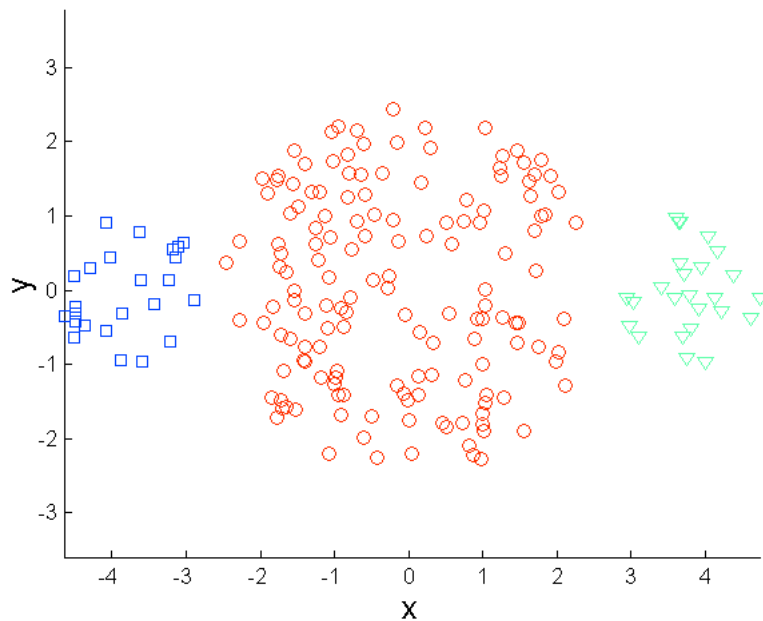
$$SSD = \sum_{i=1}^k \sum_{x \in C_i} dist(x, v_i)^2$$



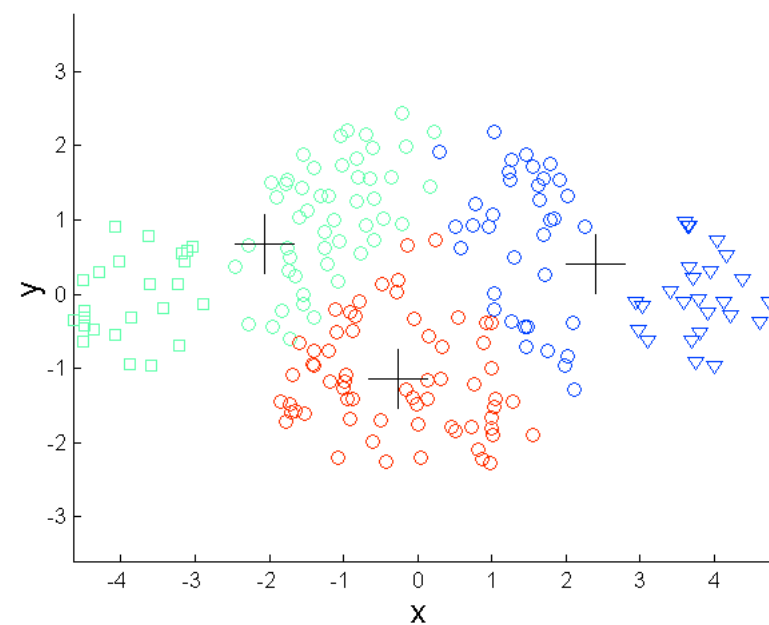
# Comments on the *K-Means* Method

- ▶ Strength: *Relatively efficient*:  $O(tkn)$ , where  $n$  is # objects,  $k$  is # clusters, and  $t$  is # iterations. Normally,  $k, t \ll n$ .
  - ▶ Comparing: PAM:  $O(k(n-k)^2)$ , CLARA:  $O(ks^2 + k(n-k))$
- ▶ Comment: Often terminates at a *local optimum*.
- ▶ Weaknesses
  - ▶ Sensitive to the initial clusters
  - ▶ Applicable only when *mean* is defined, then what about categorical data?
  - ▶ Need to specify  $k$ , the *number* of clusters, in advance
  - ▶ Very sensitive to noise and *outliers*
  - ▶ May have a problem when clusters have different sizes and are close to each other.  example
  - ▶ Not suitable to discover clusters with *non-convex shapes*  example

# A Limitation of K-means: Differing Sizes

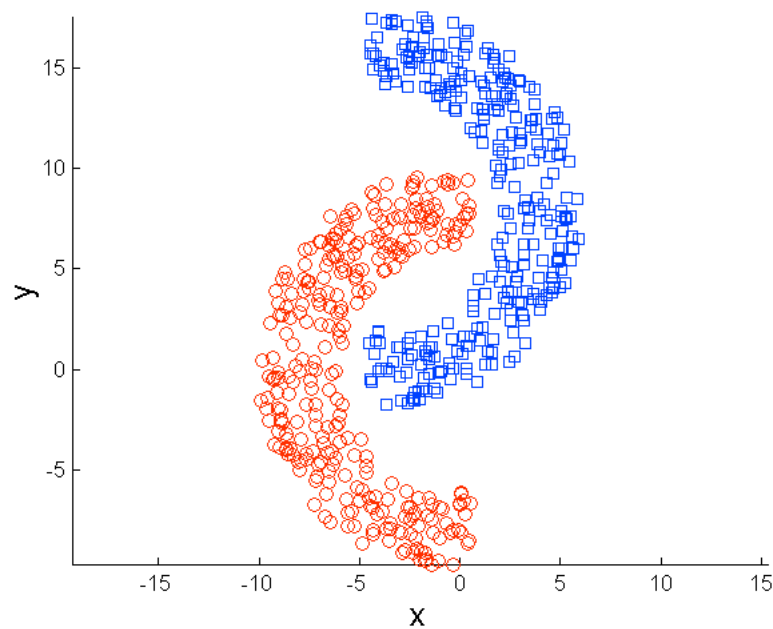


**Original Points**

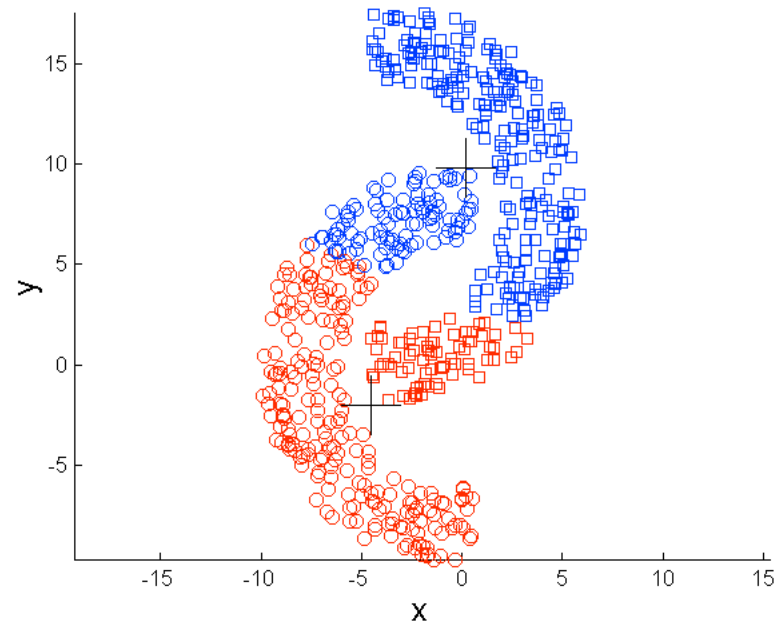


**K-means (3 Clusters)**

# A Limitation of K-means: Non-globular (Non-Convex) Shapes



**Original Points**



**K-means (2 Clusters)**

# Outline

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

# Better Initialization: k-Means++

- ▶ The k-means algorithm is sensitive to initial clusters
  - ▶ Not stable
  - ▶ Bad initialization leads to **bad-quality clusters** and typically **more iterations to converge**.
- ▶ K-Means++:
  - ▶ Uses a smarter initialization strategy
    - ▶ More stable
    - ▶ Improve clustering quality
    - ▶ Faster

---

# k-Means++

1. Pick the first centroid randomly.
2. Select subsequent centroids probabilistically, favoring points farther from existing centroids.
3. Run k-means after initializing the centroids.

## Selecting Subsequent Centroids Probabilistically (Step 2)

- ▶ Compute distance  $D(x_i)$  for each data point

- ▶ For each point  $x_i$  in the data set  $X$ , compute:

$$D(x_i)^2 = \min_{c \in C} \|x_i - c\|^2$$

- $C$  is the set of already chosen centroids
    - $\|x_i - c\|^2$  is the squared Euclidean distance between  $x_i$  and  $c$

- ▶ Compute probability of choosing  $x_i$  as the next centroid:

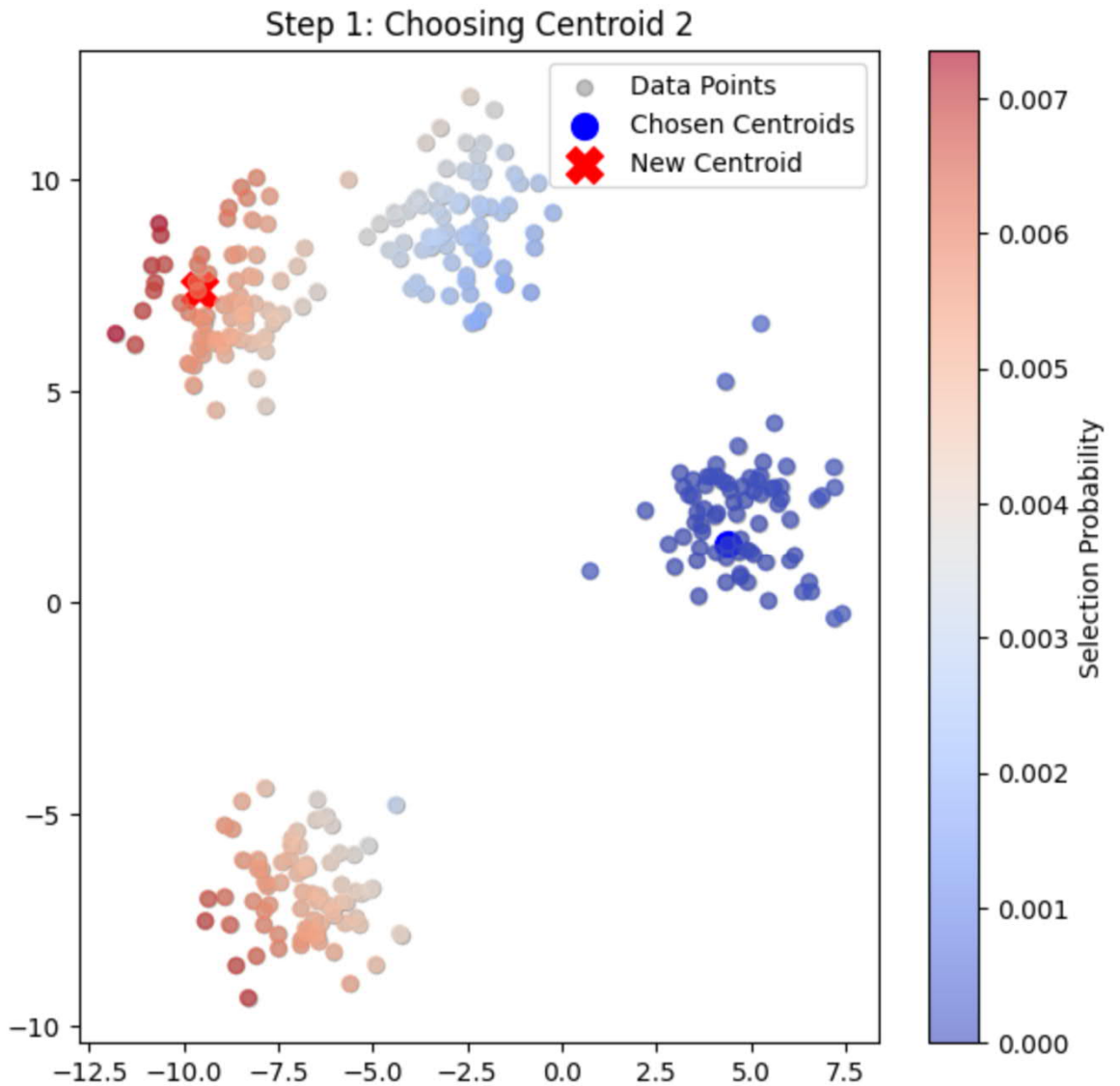
$$P(x_i) = \frac{D(x_i)^2}{\sum_{x_j \in X} D(x_j)^2}$$

- ▶ Use **weighted random sampling** to pick the next centroid
  - ▶ Points with higher distance  $D(x_i)$  have higher chance to be selected.



# Example

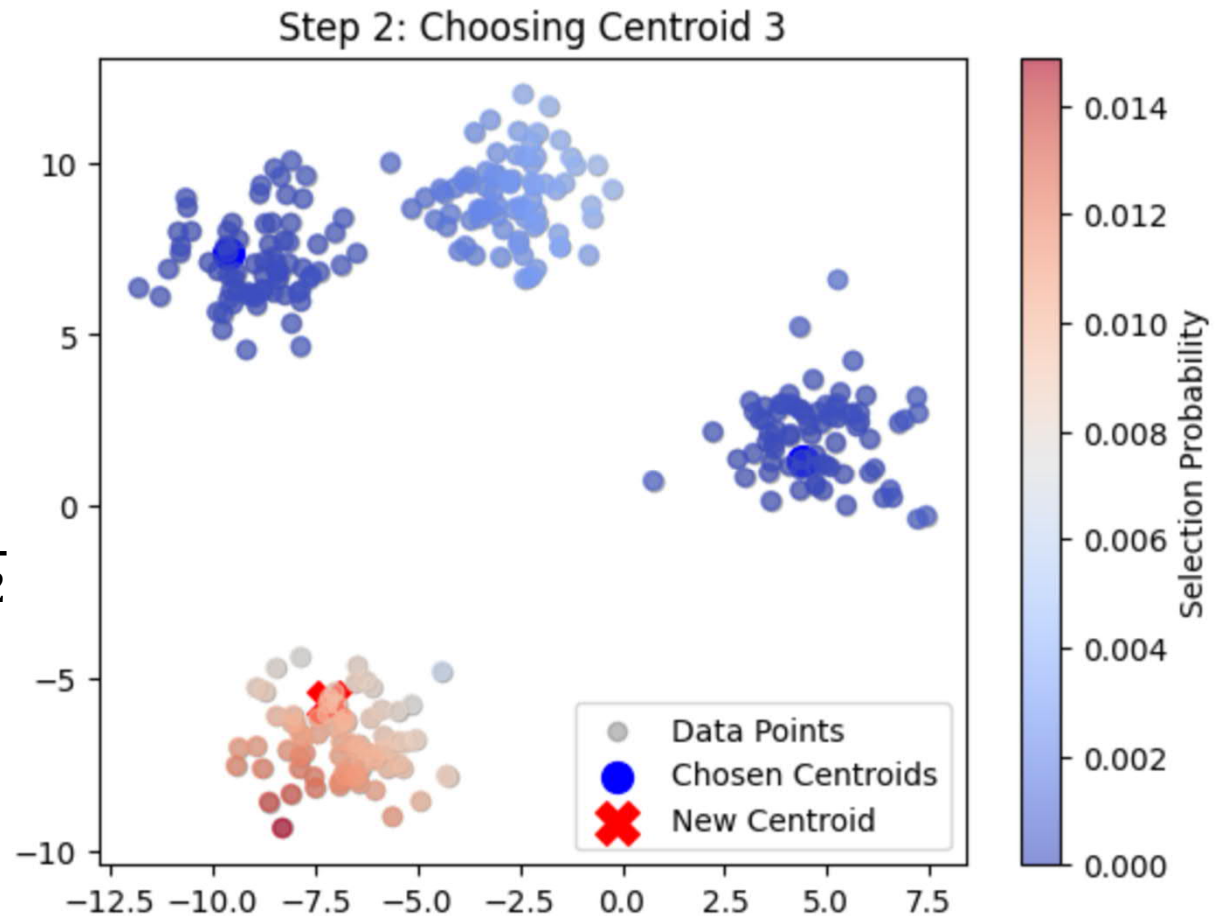
$$D(x_i)^2 = \min_{c \in C} \|x_i - c\|^2$$
$$P(x_i) = \frac{D(x_i)^2}{\sum_{x_j \in X} D(x_j)^2}$$



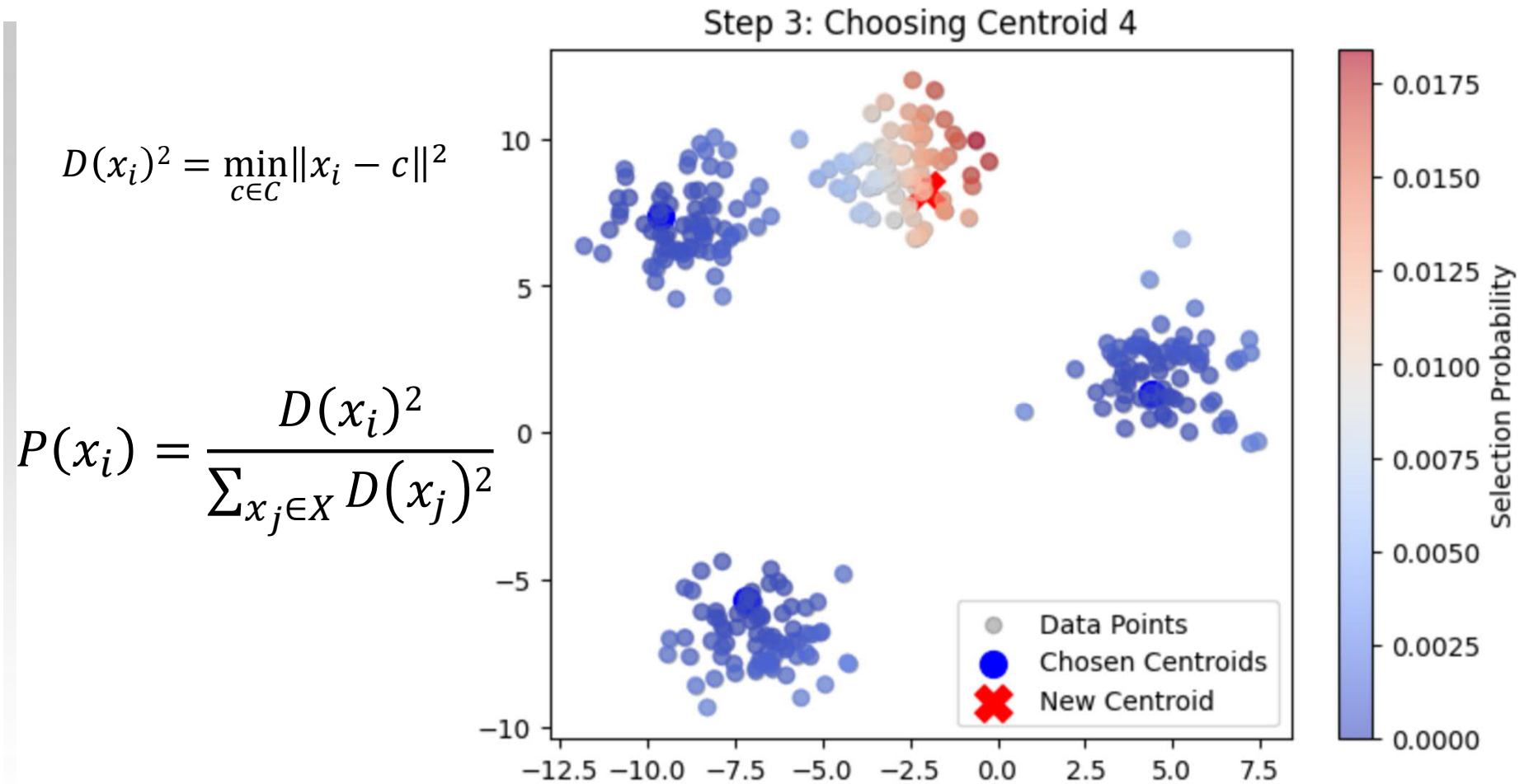
# Example

$$D(x_i)^2 = \min_{c \in C} \|x_i - c\|^2$$

$$P(x_i) = \frac{D(x_i)^2}{\sum_{x_j \in X} D(x_j)^2}$$



# Example



# k-Means++

Why choose subsequent centroids **probabilistically**, why not choosing the  $x_i$  with largest  $D(x_i)$ ?

- **Avoid sensitivity to outliers**
- **Better Representation of Data Structure:**
  - Centroids are more likely to be placed in regions of the data that are dense and representative of the underlying clusters.
  - This leads to **better clustering results**.

# k-Means vs. k-Means++

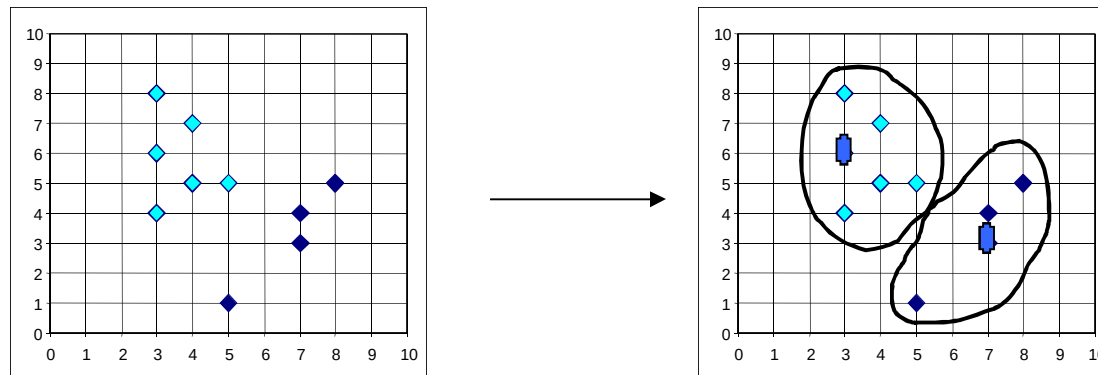
| Feature                 | k-means                                  | k-means++                                                                                                 |
|-------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Centroid Initialization | Random                                   | Distance-based stochastic                                                                                 |
| Convergence Speed       | Can be slow if centroids are bad         | Generally faster                                                                                          |
| Cluster Quality         | More likely to get stuck in local minima | More likely to find a better solution                                                                     |
| Time complexity         | $O(tkn)$                                 | Initialization: $O(kn)$<br>Subsequent steps: $O(tkn)$<br>Overall: $O(tkn)$ where $t$ is typically smaller |

# Outline

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and k-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

# Another Problem of k-Means

- ▶ The k-means algorithm is sensitive to outliers !
  - ▶ Since an object with an extremely large value may substantially distort the distribution of the data.
- ▶ K-Medoids: Instead of taking the **mean** value of the objects in a cluster as a reference point, **medoids** (also called **clustroids**) can be used, which is the **most centrally located** object in a cluster.

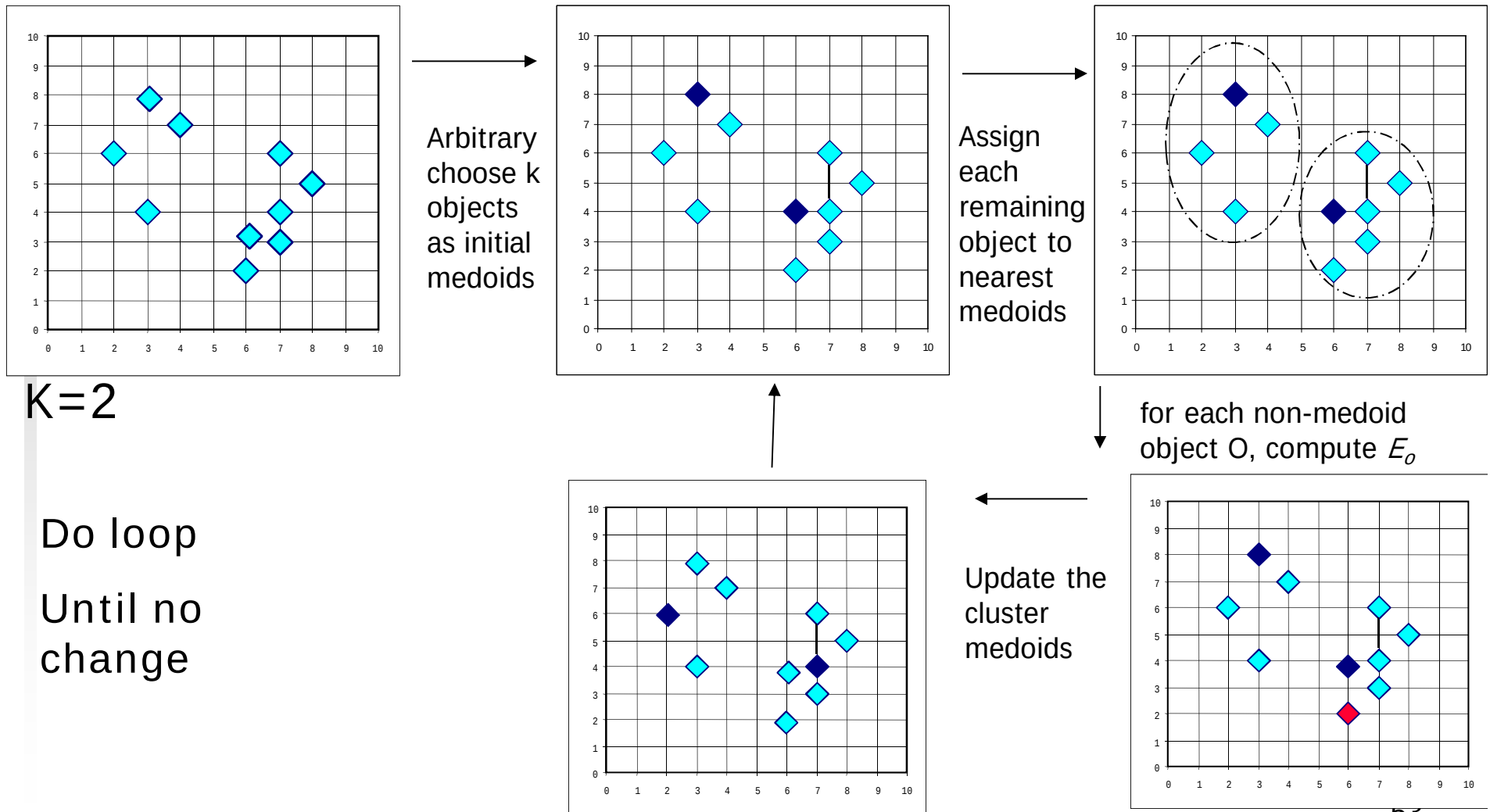


# PAM: A K-medoids Method

- ▶ PAM: Partitioning Around Medoids (1987)
- ▶ Arbitrarily choose  $k$  objects as the initial medoids
- ▶ Until no change, do
  - ▶ (Re)assign each object to the cluster whose medoid is the nearest
  - ▶ For each pair of a non-medoid object  $o'$  and a medoid  $o$ , compute the total cost,  $S$ , of swapping medoid  $o$  with  $o'$
  - ▶ If the lowest cost  $S_{lowest} < 0$  then swap  $o$  with  $o_{lowest}'$  to form the new set of  $k$  medoids



# Typical k-medoids algorithm (PAM)



# How to Choose the new Medoid for a Cluster

- ▶ Use the sum of distances criterion

$$E_o = \sum_{p \in C_i} d(p, o)$$

- $o$  is a candidate medoid for the  $i$ th cluster  $C_i$
  - $d(p, o)$  is the distance between  $o$  and point  $p$  in  $C_i$ .
- ▶ Choose point  $o$  in  $C_i$  that minimizes  $E_o$

# Pros and Cons of PAM

- ▶ PAM is more robust than k-means in the presence of noise and outliers
  - ▶ Medoids are less influenced by outliers
- ▶ PAM is efficient for small data sets but does not scale well for large data sets
  - ▶  $O(k(n-k)^2)$  for each iteration
- ▶ Sampling based method: CLARA
  - ▶ Randomly select a subset of data
  - ▶ Using k-medoids to cluster the subset
  - ▶ Remaining examples are assigned to the closest cluster

# Outline

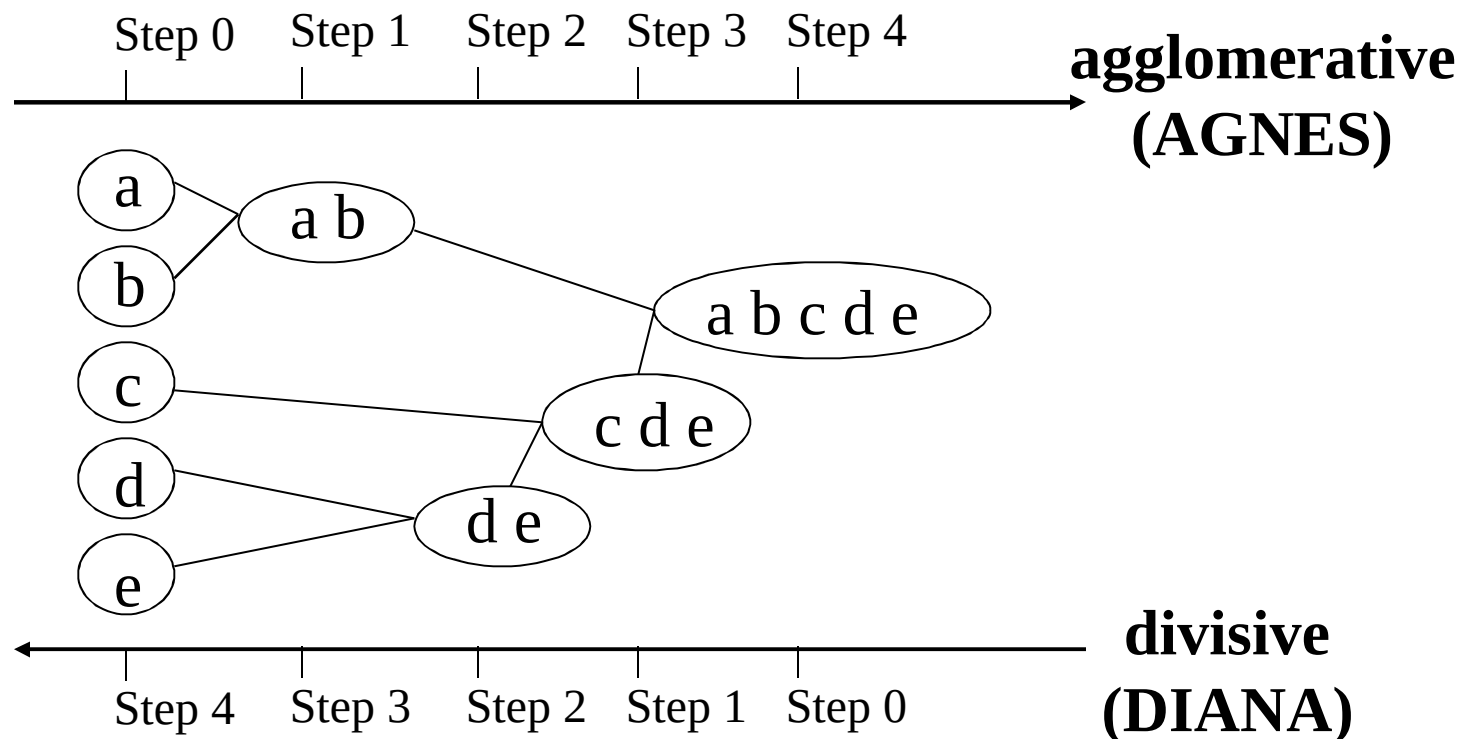
- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and k-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

# Hierarchical Clustering

- ▶ Iteratively merge or split clusters to form a tree of clusters
- ▶ Two types
  - ▶ Agglomerative (bottom-up): merge clusters iteratively
    - ▶ Start by placing each object in its own cluster
    - ▶ Merge these small clusters into larger and larger clusters
    - ▶ until all objects are in a single cluster
    - ▶ Most hierarchical methods belong to this category. They differ only in their definition of between-cluster similarity
  - ▶ Divisive (top-down): split a cluster iteratively
    - ▶ Start with all objects in one cluster and subdivide them into smaller pieces

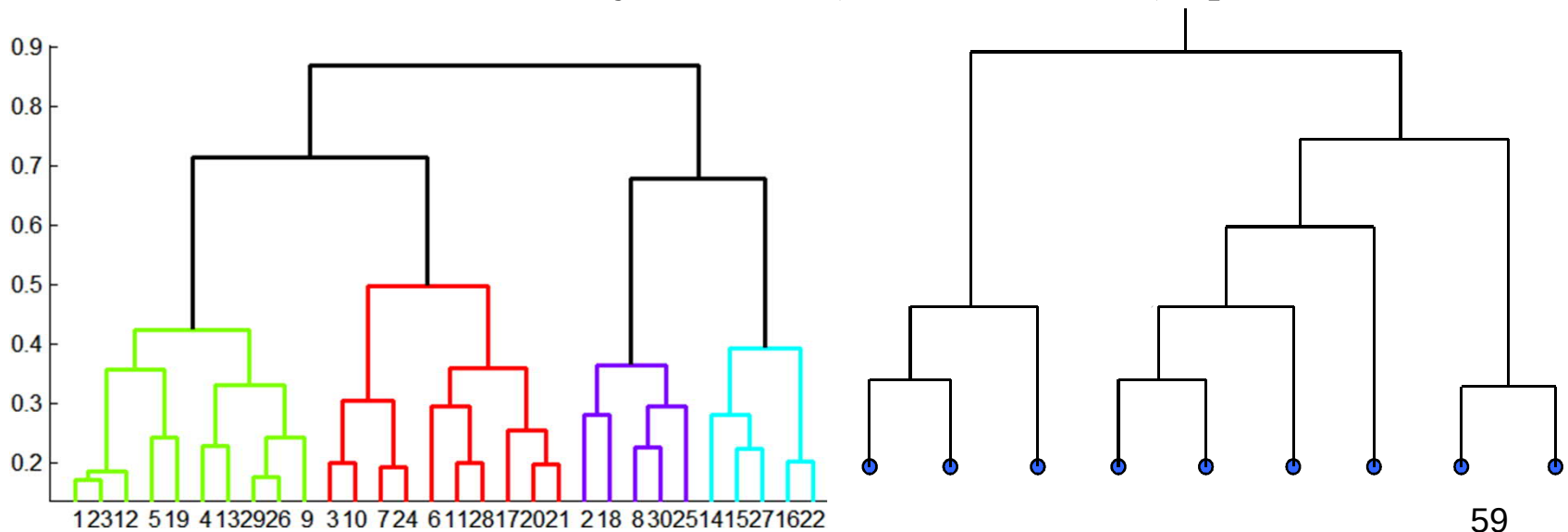
# Hierarchical Clustering

- This method **does not require the number of clusters  $k$  as an input**, but needs a termination condition

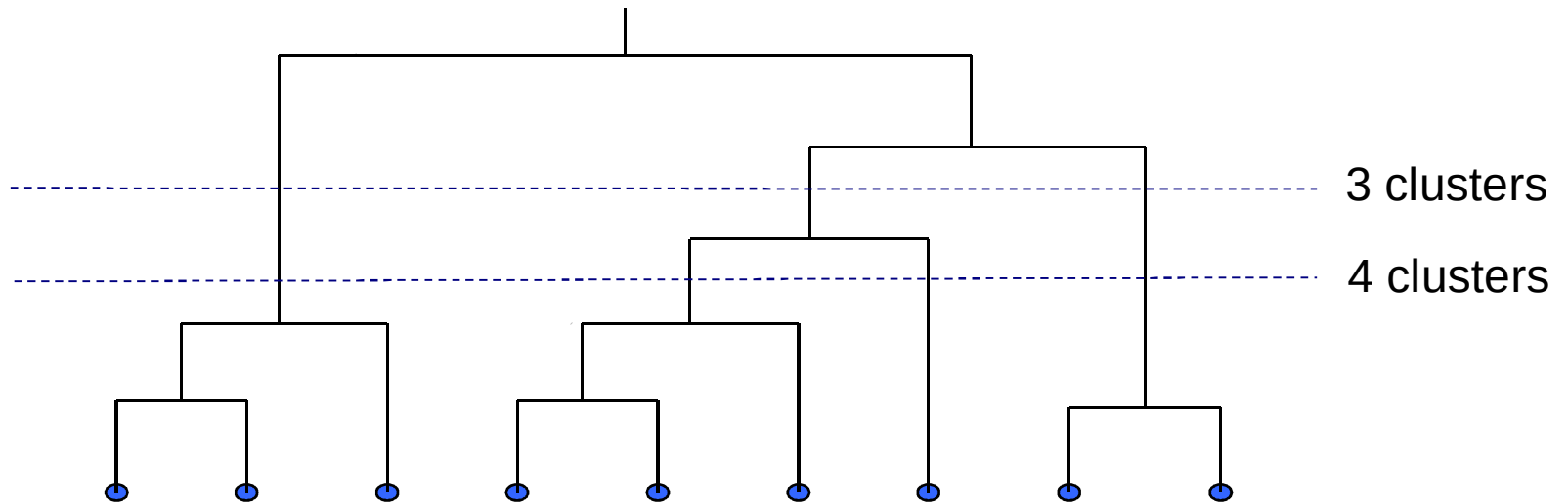


# Dendrogram

- ▶ A tree-like diagram that
  - ▶ Visually represents the hierarchical structure of clusters,
  - ▶ Shows how data points or clusters are merged or split at each step of the clustering process.
- ▶ The height of the branches indicates the distance between the merged or split clusters.
  - ▶ Distance should monotonically increase (or non-decrease) upwards



# Hierarchical Clustering

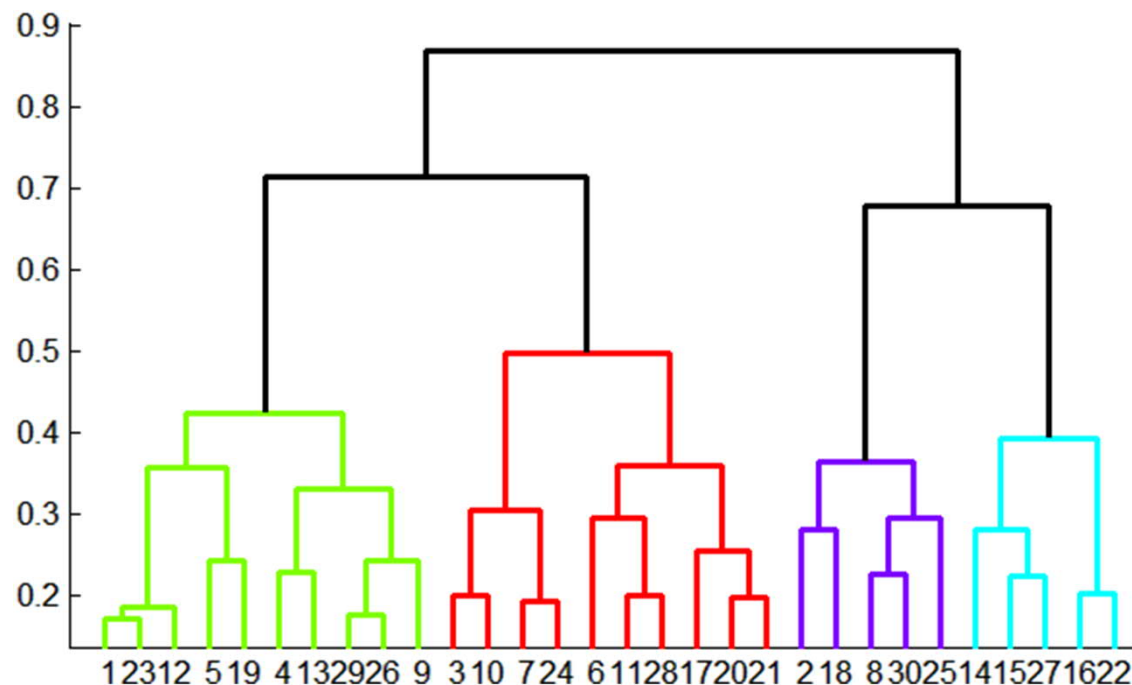


- ▶ No need to generate the whole dendrogram. Merging or splitting clusters can stop:
  - ▶ if only  $k$  clusters are needed, or
  - ▶ Another threshold is satisfied, such as:
    - ▶ The distance between two closest clusters exceeds a distance threshold



# Hierarchical Clustering

- ▶ **A Key Question:**
  - ▶ **How do we measure the distance between clusters?**



# Inter-cluster Distances in Hierarchical Clustering

- ▶ Several ways to define the distance between two clusters:

- ▶ **Single linkage** method (nearest neighbor):

$$d(C_i, C_j) = \min_{x \in C_i, y \in C_j} \{d(x, y)\}$$

$C_i$  and  $C_j$  are  
two clusters

- ▶ **Complete linkage** method (furthest neighbor):

$$d(C_i, C_j) = \max_{x \in C_i, y \in C_j} \{d(x, y)\}$$

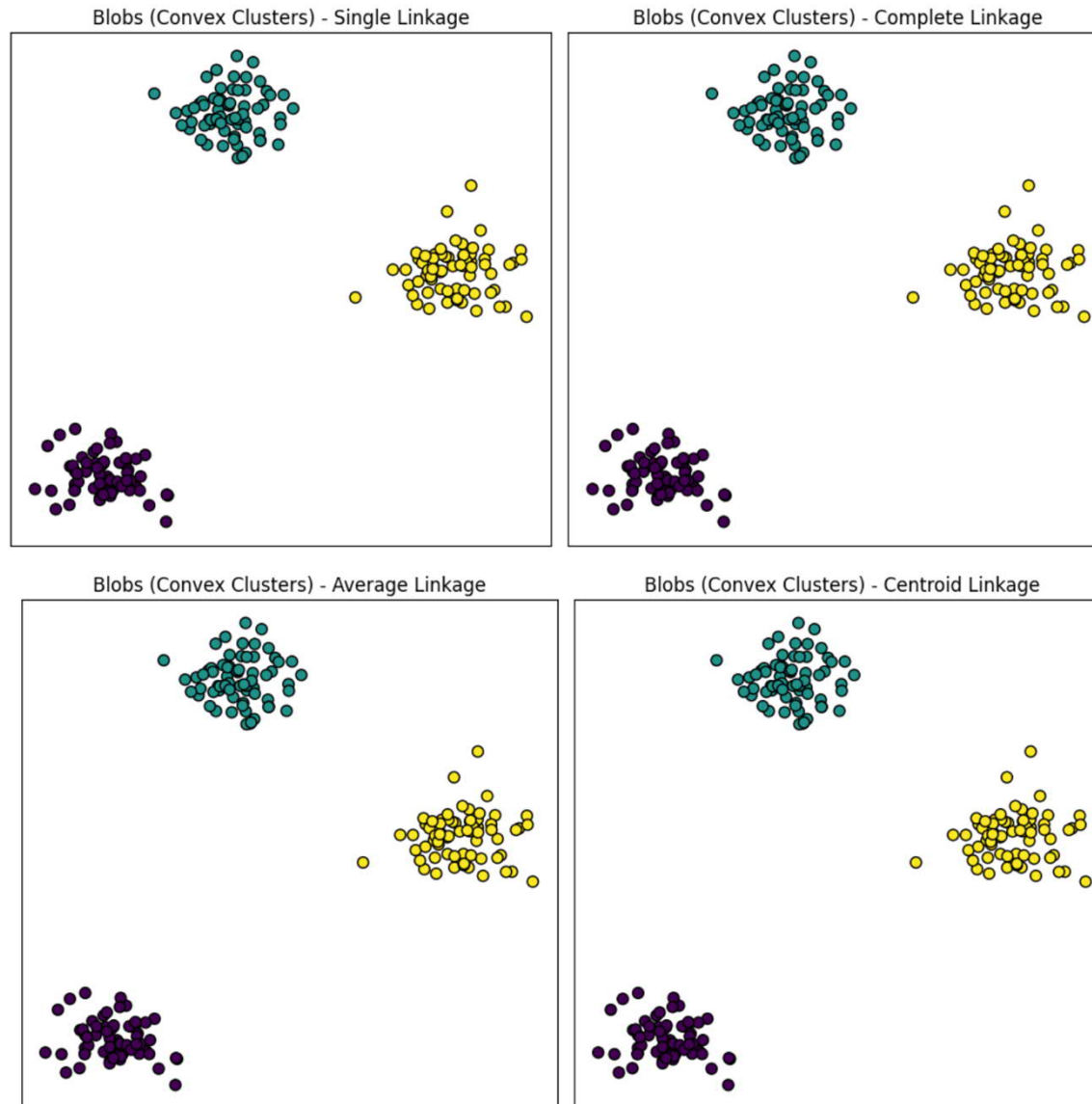
- ▶ **Average linkage** method (unweighted pair-group average):

$$d(C_i, C_j) = \text{avg}_{x \in C_i, y \in C_j} \{d(x, y)\}$$

- ▶ **Centroid linkage** method:

- ▶ Distance between the centroids (e.g., means) of two clusters.

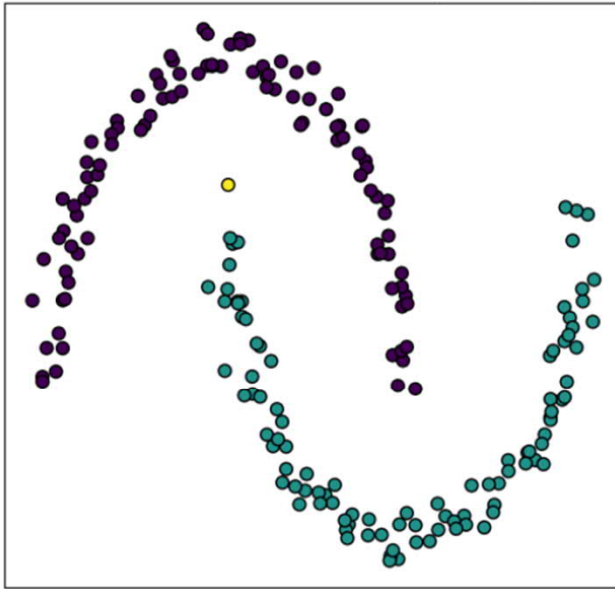
# Comparison of Linkage Methods



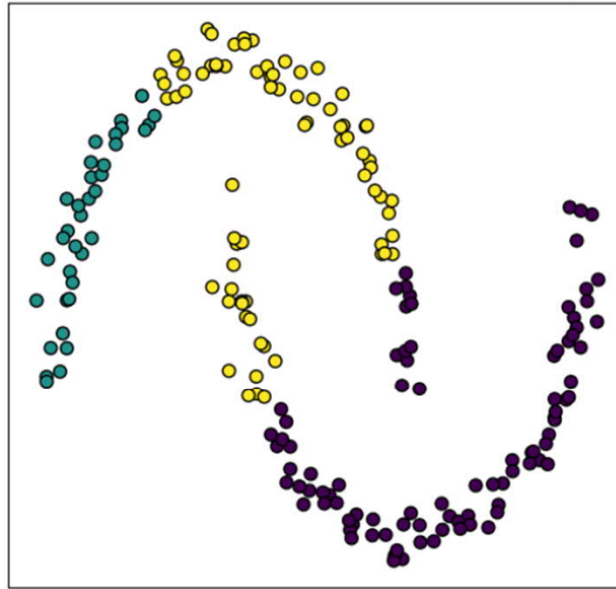
Well-separated, convex clusters

# Comparison of Linkage Methods

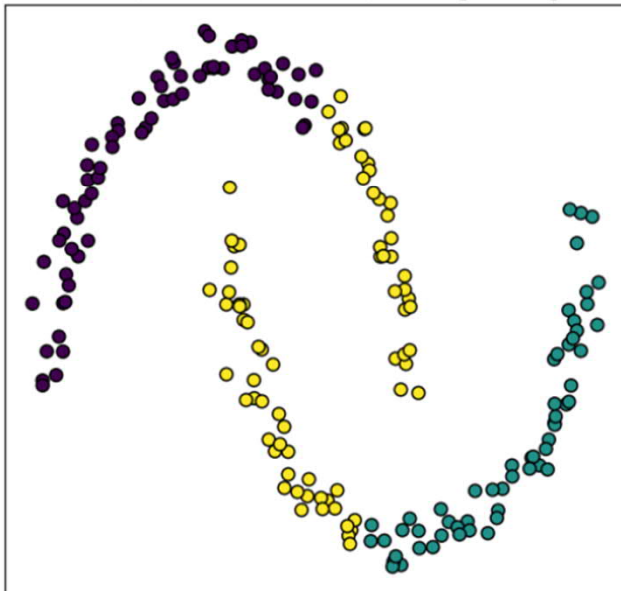
Moons (Non-Convex Clusters) - Single Linkage



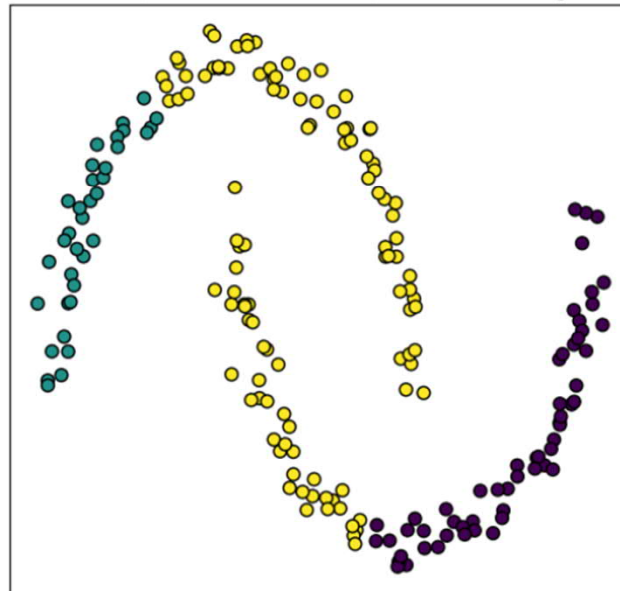
Moons (Non-Convex Clusters) - Complete Linkage



Moons (Non-Convex Clusters) - Average Linkage

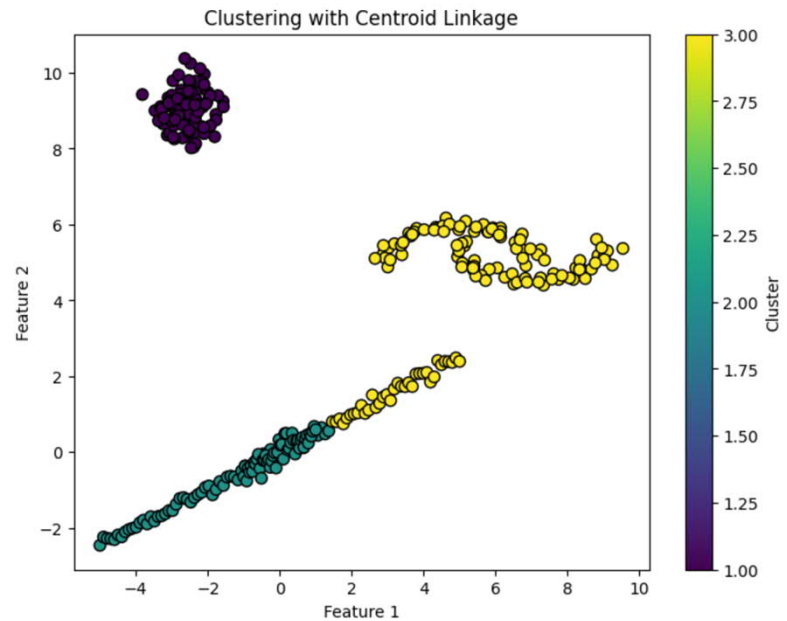
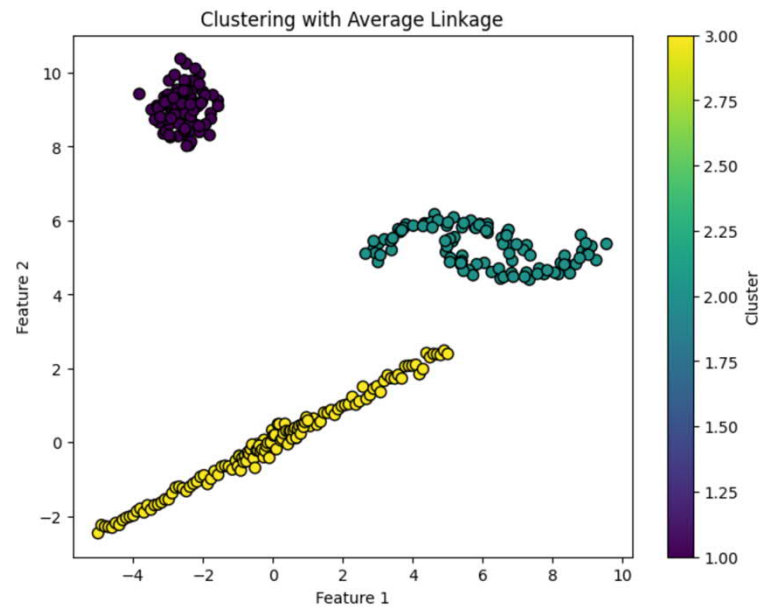
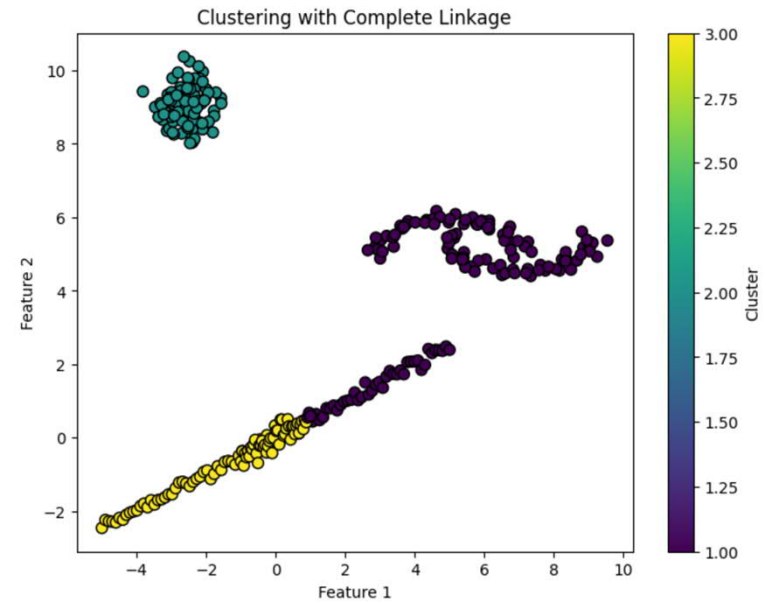
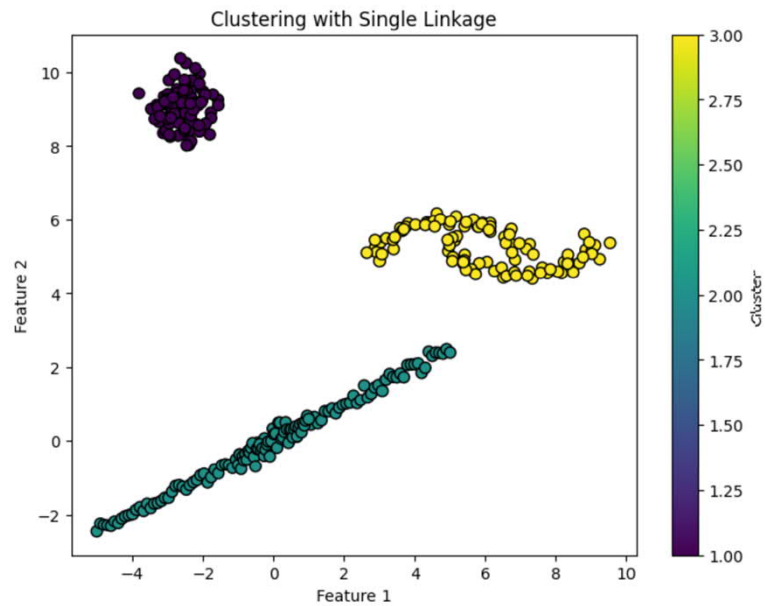


Moons (Non-Convex Clusters) - Centroid Linkage

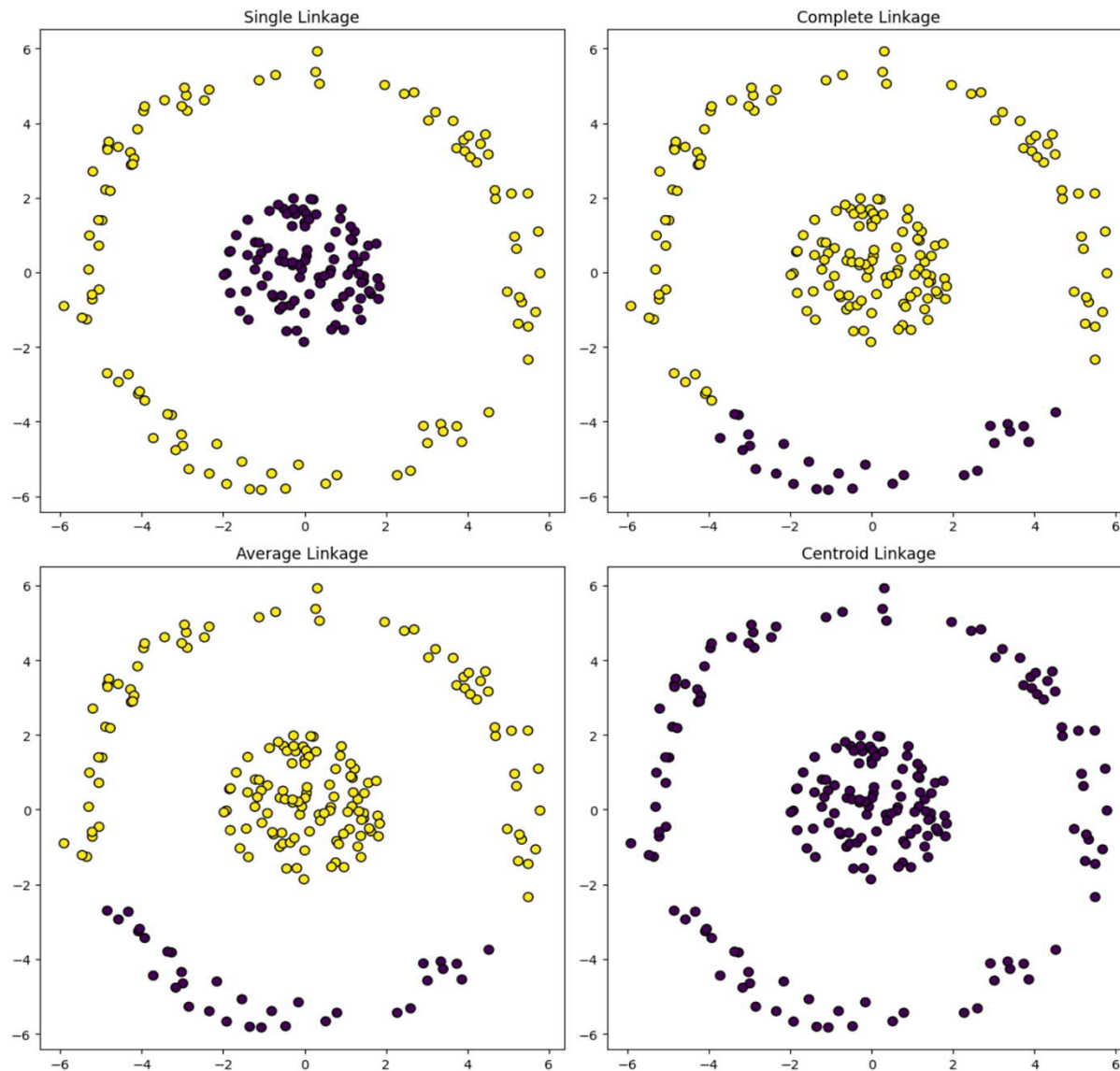


Elongated,  
chain-like  
clusters

# Comparison of Linkage Methods



# Comparison of Linkage Methods



Nested  
clusters

# Comparison of Linkage Methods

- ▶ Single linkage
  - ▶ Good at detecting elongated, chain-like clusters.
  - ▶ **Advantages:**
    - ▶ Can detect non-spherical and non-convex shapes.
  - ▶ **Disadvantages:**
    - ▶ Can suffer from the chaining effect where distant points get merges due to intermediate, noisy points
    - ▶ Not robust to noise between clusters
      - ▶ This type of noise is hard to detect during preprocessing.

# Comparison of Linkage Methods

- ▶ Complete linkage
  - ▶ Good at detecting compact, spherical clusters by minimizing the diameter of cluster
    - ▶ Diameter: the distance between the two farthest points in a cluster
  - ▶ **Advantages:**
    - ▶ Robust to between-cluster noise and outliers compared to single linkage.
    - ▶ Tends to produce more balanced clusters.
  - ▶ **Disadvantages:**
    - ▶ Sensitive to outliers within clusters that are far away; may fail to merge clusters that should be merged.
    - ▶ May break large clusters into smaller ones.
    - ▶ Struggles with detecting clusters of varying sizes or non-spherical shapes.



# Comparison of Linkage Methods

## ▶ Average linkage

- ▶ A compromise between single and complete linkage, suitable for detecting clusters of various shapes and sizes.
  - ▶ But not as flexible as single linkage in terms of cluster shapes
- ▶ **Advantages:**
  - ▶ Less sensitive to noise and outliers (due to averaging out the effect of outliers).
  - ▶ Can handle clusters of different sizes and shapes better than complete and centroid linkage methods.
- ▶ **Disadvantages:**
  - ▶ Computationally more expensive than single and complete linkage.
    - ▶ Need to recompute the average distances between the merged cluster and each of others after merging or splitting.
  - ▶ May still struggle with very irregular cluster shapes.

# Comparison of Linkage Methods

## ▶ Centroid linkage

- ▶ Detecting spherical clusters, similar to k-means

### ▶ **Advantages:**

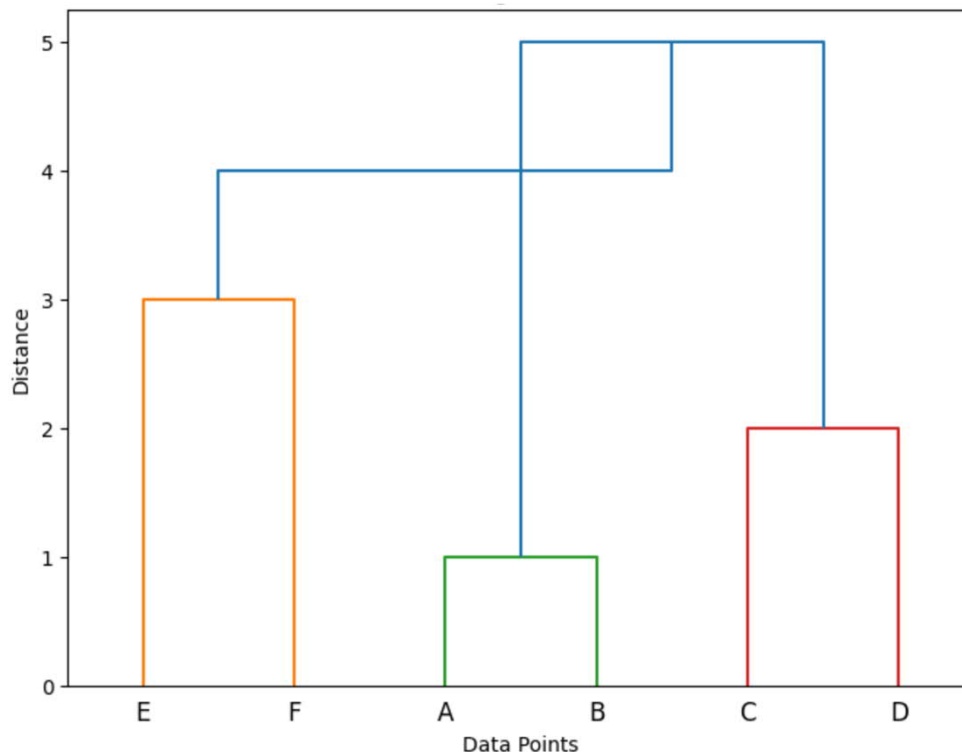
- ▶ Intuitive and easy to understand.
- ▶ Works well with clusters that are roughly spherical and of similar size.

### ▶ **Disadvantages:**

- ▶ Struggles with clusters that have irregular or non-convex shapes (e.g., crescent moons, rings, or chains).
  - ▶ Centroids can be out of non-convex shaped clusters (not a good cluster representation)
- ▶ Sensitive to outliers
  - ▶ Outliers or sparse regions in a cluster can pull the centroid away from the dense region, distorting the distance calculations
- ▶ Can produce "inversions" in the dendrogram, where the distance between merged clusters decreases instead of increasing.
  - ▶ This happens because the centroid of the merged cluster might be closer to another cluster than the original clusters were.

# Dendrogram with Inversions

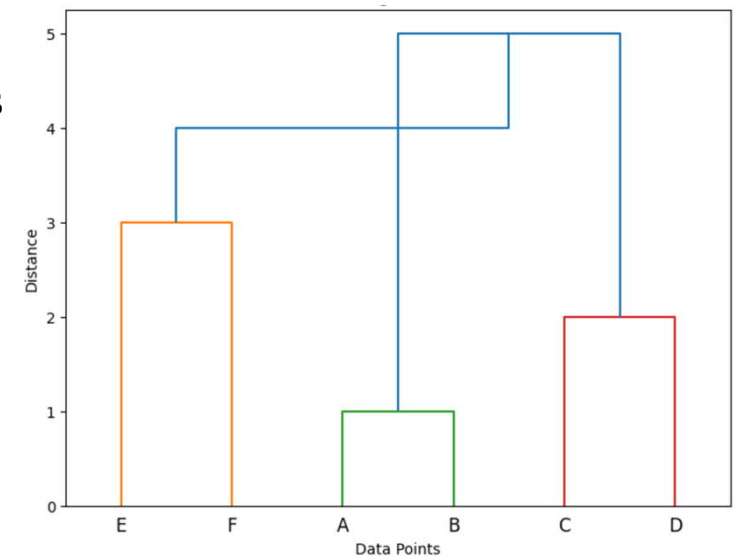
- ▶ Distance may decrease instead of increasing upwards.
- ▶ Example:
  - ▶ Suppose we have 6 data points: A, B, C, D, E, F. We perform hierarchical clustering using centroid linkage



- The second last merge of cluster AUB and cluster CUD is at distance  $d=5$
- The last merge of cluster (EUF) and cluster (AUBUCUD) is at distance  $d=4$
- Distance decreases

# Dendrogram with Inversions

- ▶ Cause of inversions:
  - ▶ The centroid of the merged cluster might be closer to another cluster than the original clusters were.
- ▶ Problems it may cause
  - ▶ Difficulty interpreting the dendrogram
    - ▶ Inversions make the dendrogram non-monotonic, as height increases.
    - ▶ Lead to confusion when trying to determine the relationships between clusters.
  - ▶ Challenges in obtaining the final clusters:
    - ▶ When cutting the dendrogram to obtain a specific number of clusters, inversions can make it unclear where to cut, as the distances may not reflect the true relationships between clusters.
- ▶ This is why centroid linkage is **not commonly used** in hierarchical clustering



---

# Summary of When to Use What

- ▶ Single Linkage:
  - ▶ Use when you expect elongated clusters and can handle noise.
- ▶ Complete Linkage:
  - ▶ Use when you expect compact, well-separated clusters.
- ▶ Average Linkage:
  - ▶ Use as a general-purpose method for clusters of varying shapes and sizes.
- ▶ Centroid Linkage:
  - ▶ Not commonly used
  - ▶ Can be used when clusters are expected to be spherical and of similar size, but be cautious of inversions.

# Strengths of Hierarchical Clustering

- ▶ Conceptually simple
- ▶ No need to specify the number of clusters
- ▶ Captures hierarchical relationships
- ▶ Flexibility in Cluster Shapes:
  - ▶ Depending on the linkage method, hierarchical clustering can handle clusters of varying shapes and sizes, though it is not as robust as density-based methods like DBSCAN.
- ▶ Deterministic
  - ▶ Produce the same result every time for the same dataset.
- ▶ Visualization:
  - ▶ The dendrogram provides an intuitive visualization of the clustering process, making it easier to interpret the results.

# Limitations of Hierarchical Clustering

- ▶ Do not scale well:
  - ▶ Time complexity is  $O(n^3)$  for naïve implementation and  $O(n^2 \log n)$  for optimized implementations, where  $n$  is the number of total data points
- ▶ Sensitivity to Noise and Outliers:
  - ▶ Some linkage methods (e.g., single linkage) are sensitive to noise and outliers, which can lead to poor clustering results.
- ▶ Irreversible Merges:
  - ▶ Once two clusters are merged/split, the decision cannot be reversed. This can lead to suboptimal clustering if early merges are incorrect.
- ▶ Choice of Linkage Method:
  - ▶ The results depend heavily on the choice of linkage method, and selecting the wrong method can lead to poor clustering.

---

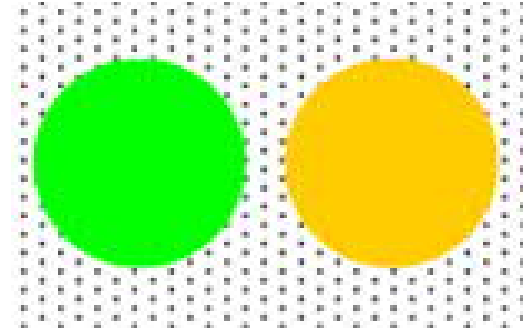
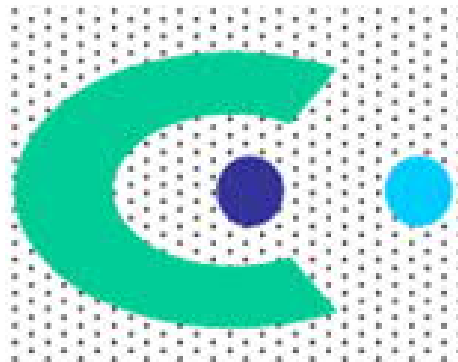
# Outline

- ▶ What is Clustering?
- ▶ Types of Data in Cluster Analysis and Similarity Measures
- ▶ Some clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN: a Density-based Algorithm
- ▶ Cluster Validity



# Density-based Clustering

- ▶ A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.
- ▶ Used when the clusters are **irregular** or intertwined, and when noise and outliers are present.



# Density-based Clustering

- ▶ Clustering based on density (local cluster criterion), such as density-connected points
- ▶ Major features:
  - ▶ Discover clusters of arbitrary shape
  - ▶ Handle noise
  - ▶ Do not need to specify  $k$ , but need density parameters
- ▶ Several interesting studies:
  - ▶ DBSCAN: Ester, et al. (KDD'96)
  - ▶ OPTICS: Ankerst, et al. (SIGMOD'99)
  - ▶ DENCLUE: Hinneburg & D. Kein (KDD'98)
  - ▶ CLIQUE: Agrawal, et al. (SIGMOD'98)

# DBSCAN: Basic Concepts

- ▶ ***Eps***-neighborhood of point  $p$  in data set  $D$ :

$$N_{Eps}(p) = \{q \in D \mid \text{dist}(p, q) \leq Eps\}$$

where ***Eps*** is called the radius of the neighborhood

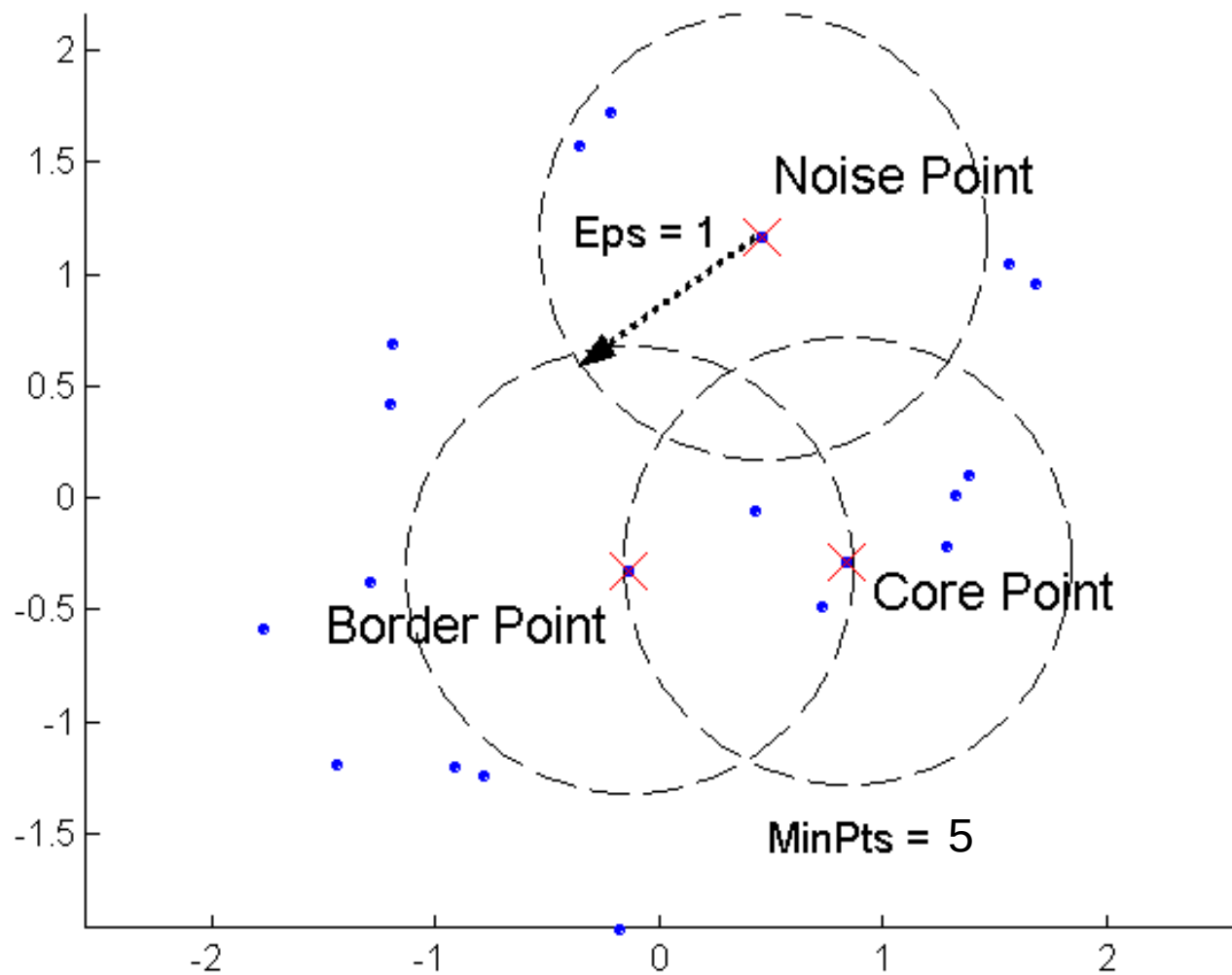
- ▶ Density of ***Eps***-neighborhood of  $p$ :

the number of points in  $N_{Eps}(p)$

## DBSCAN: Basic Concepts

- ▶ A point is a **core point** if there are at least *MinPts* number of points in its *Eps*-neighborhood.
  - ▶ These are points that are at the interior of a cluster
- ▶ A **border point** has fewer than *MinPts* points in its *Eps*-neighborhood, but is in the *Eps*-neighborhood of a core point.
  - ▶ These are points that are on or close to the border of a cluster
- ▶ A **noise point** is any point that is not a core point or a border point.
  - ▶ These are points that are outside any cluster

# DBSCAN: Core, Border and Noise Points



# DBSCAN: Input Parameters

- ▶ ***Eps:***

Maximum radius of the neighbourhood

- ▶ ***MinPts:***

Minimum number of points in an *Eps*-neighbourhood of a core point

# DBSCAN: Basic Concepts

- ▶ Directly density-reachable:

- ▶ A point  $p$  is **directly density-reachable** from a point  $q$  wrt.  $Eps$  and  $MinPts$  if

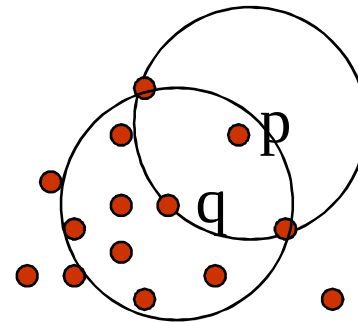
- 1)  $p$  belongs to  $N_{Eps}(q)$

- 2)  $q$  is a core point, that is:

$$|N_{Eps}(q)| \geq MinPts$$

- ▶ Asymmetric in general

- ▶ Symmetric only when both  $p$  and  $q$  are core points.



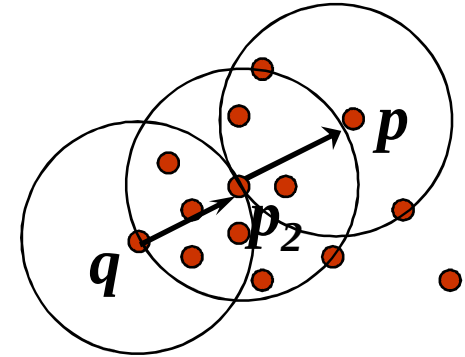
MinPts = 5

Eps = 1 cm

# DBSCAN: Basic Concepts

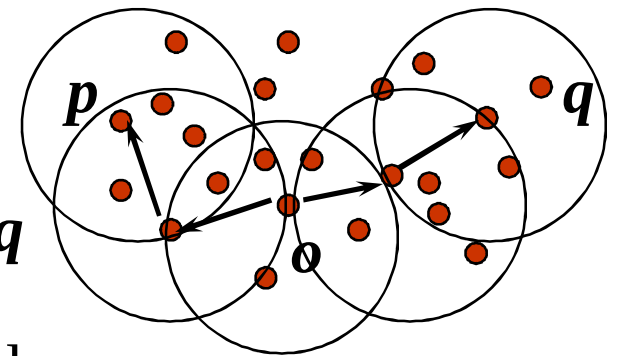
## ► Density-reachable:

- A point  $p$  is **density-reachable** from a point  $q$  wrt.  $Eps$  and  $MinPts$  if there is a chain of points  $p_1, p_2, \dots, p_n, p_1 = q, p_n = p$  such that  $p_{i+1}$  is directly density-reachable from  $p_i$
- Symmetric only when  $p$  and  $q$  are both core points



## ► Density-connected:

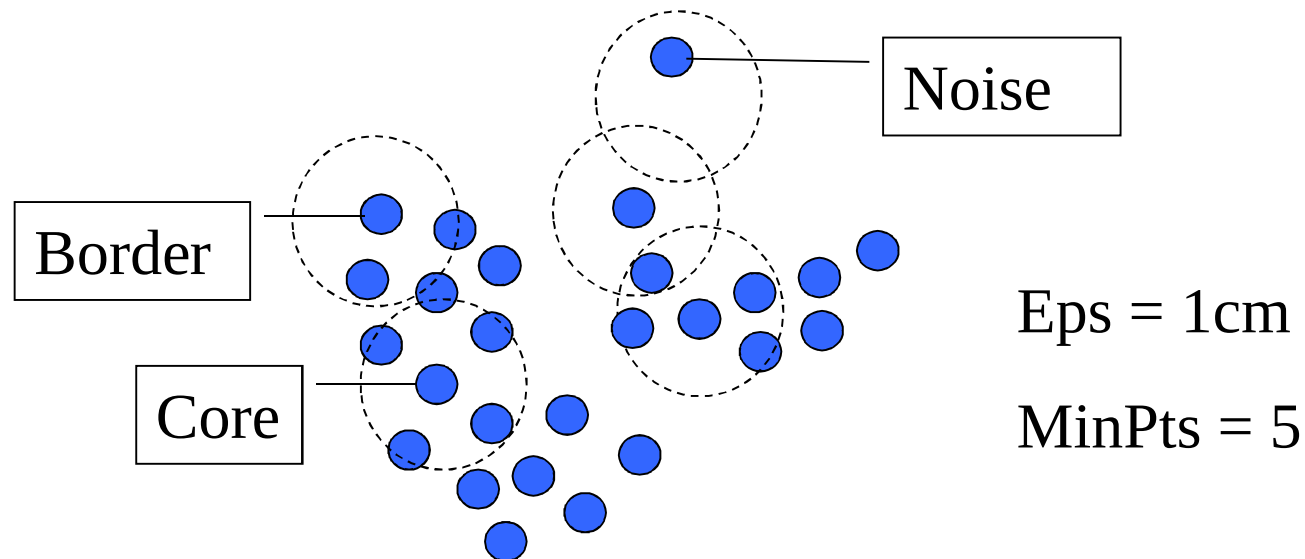
- A point  $p$  is **density-connected** to a point  $q$  wrt.  $Eps$  and  $MinPts$  if there is a point  $o$  such that both  $p$  and  $q$  are density-reachable from  $o$  wrt.  $Eps$  and  $MinPts$ .
- Symmetric





# DBSCAN: Cluster

- ▶ *Density-based notion of cluster*: A **cluster** is defined as a **maximal set of density-connected points**
  - ▶ Each pair of points in a cluster are density-connected to each other (Connectivity)
  - ▶ Core points in other clusters are not density-connected to any core points in this cluster (Maximality)
- ▶ Noise points are not in any cluster



# DBSCAN: The Algorithm

- ▶ Arbitrarily select an unprocessed point  $p$
- ▶ If  $p$  is a core point,
  - ▶ Retrieve all points density-reachable from  $p$  wrt  $Eps$  and  $MinPts$ .
  - ▶ A cluster is formed which includes all the points density-reachable from  $p$
  - ▶ Mark all the points in the cluster as “processed”
- ▶ If  $p$  is not a core point, no points are density-reachable from  $p$  and DBSCAN visits the next unprocessed point of the database.
- ▶ Continue the process until all of the core points have been processed.

These points and  $p$  are in a cluster because they are density-connected through  $p$ .

*Note: Membership of a border point depends on the order of the points being processed if it is density-connected to core points in two or more clusters.*

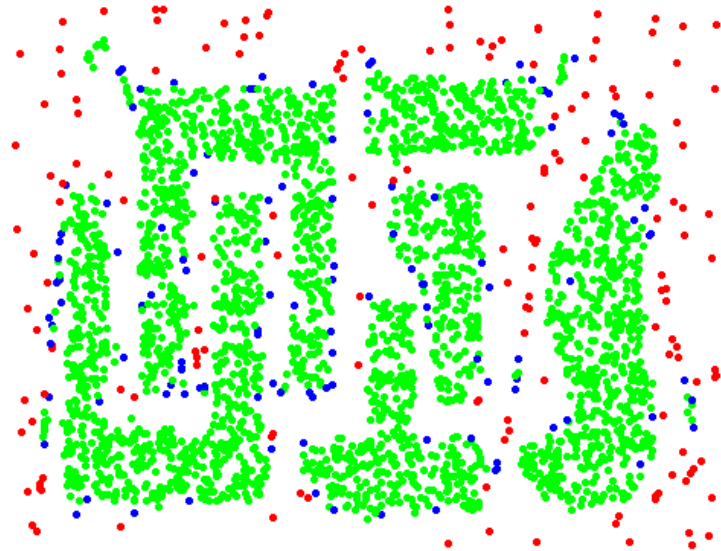
## Another Version of DBSCAN (for simplicity, not efficiency)

- ▶ Label all points as core, border or noise points
- ▶ Eliminate noise points
- ▶ Put an edge between all core points that are within *Eps* of each other
- ▶ Make each group of connected core points into a separate cluster
- ▶ Assign each border point to one of the clusters of its associated core points.

# DBSCAN: Core, Border and Noise Points



Original Points

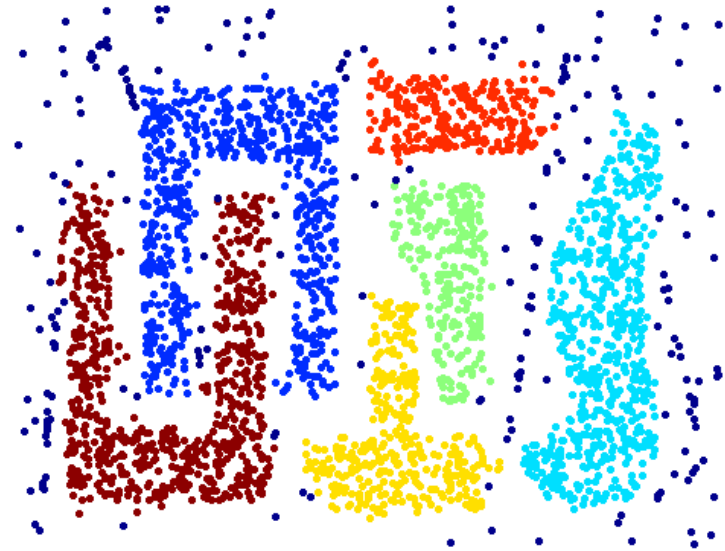


Point types: **core**,  
**border** and **noise**

# When DBSCAN Works Well



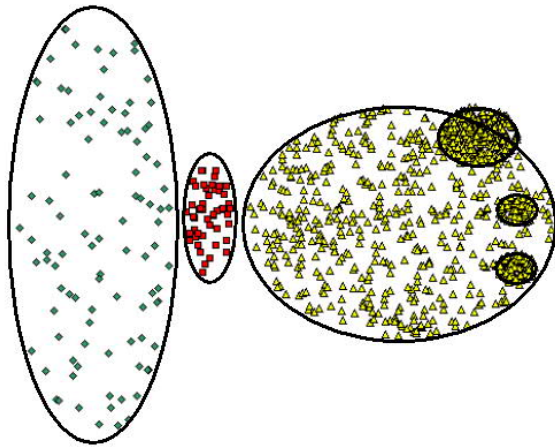
Original Points



Clusters

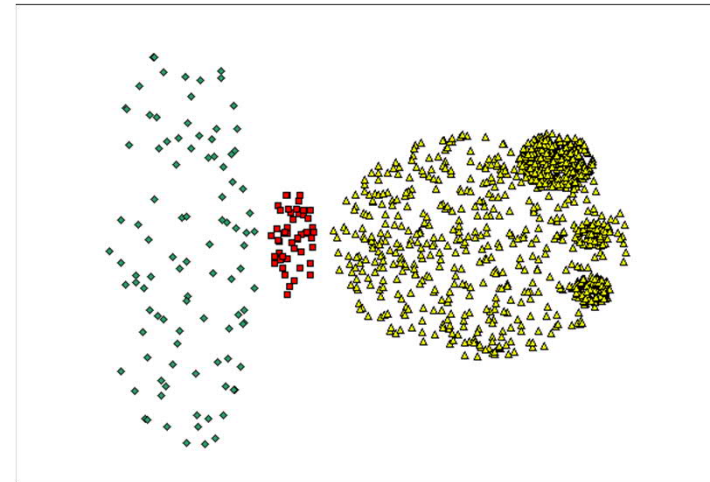
- Resistant to Noise
- Can handle clusters of different shapes and sizes

# When DBSCAN Does NOT Work Well

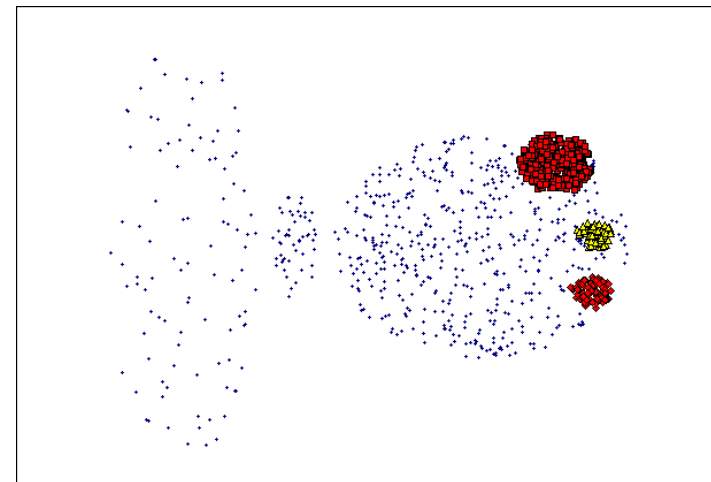


**Original Points**

- Varying densities



(MinPts=4, Eps=9.75).



(MinPts=4, Eps=9.92)

# DBSCAN: Sensitive to Parameters

Figure 8. DBScan results for DS1 with MinPts at 4 and Eps at (a) 0.5 and (b) 0.4.

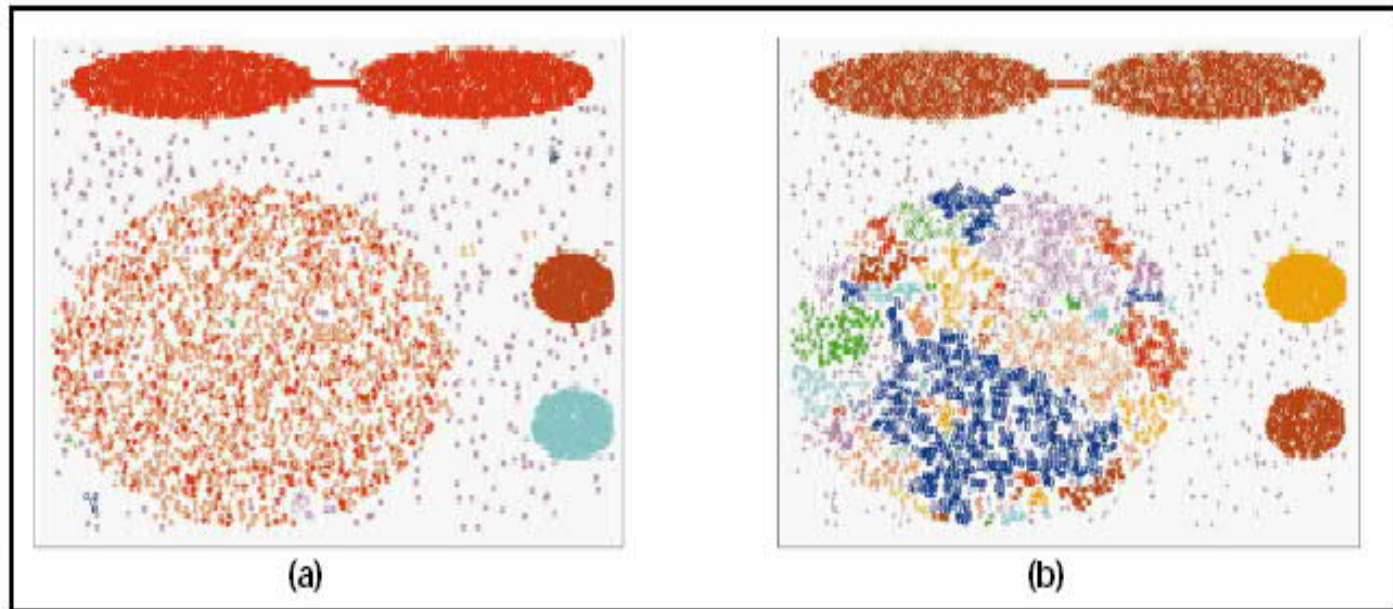
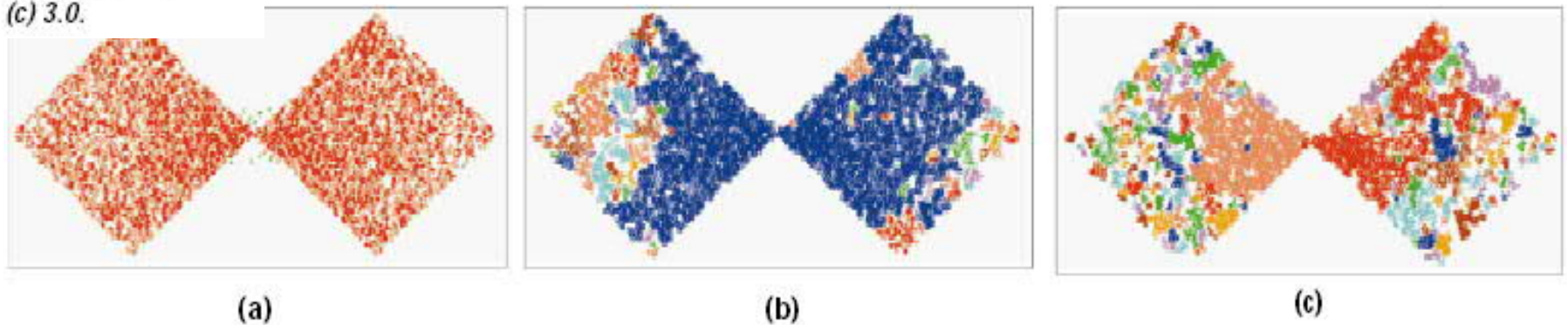


Figure 9. DBScan results for DS2 with MinPts at 4 and Eps at (a) 5.0, (b) 3.5, and (c) 3.0.



# Comparing DBSCAN and K-means

- ▶ Cluster shapes

- ▶ K-means: spheres
- ▶ DBSCAN: arbitrary shape



- ▶ Cluster sizes

- ▶ K-means may have a problem when clusters are of different sizes and are close to each other.
- ▶ DBSCAN can handle clusters of different sizes

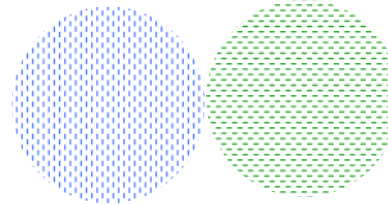
- ▶ Noise and outliers

- ▶ K-means is sensitive to noise or outliers
- ▶ DBSCAN is not strongly affected by noise or outliers



# Comparing DBSCAN and K-means (Cont'd)

- ▶ Not well-separated clusters

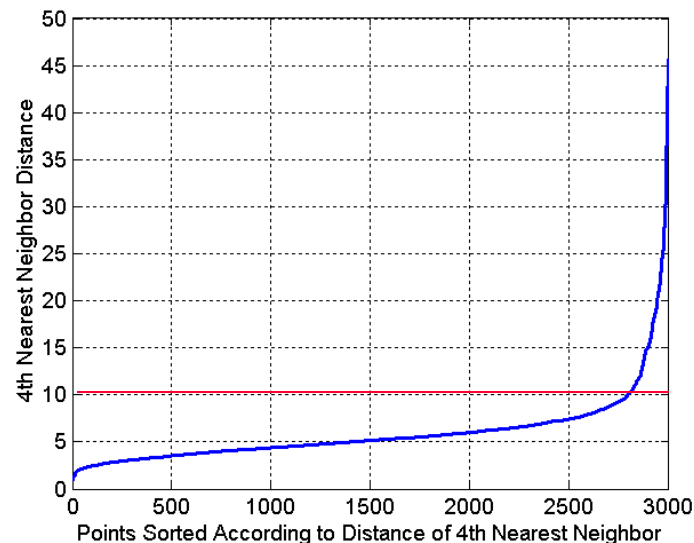


- ▶ K-means can find clusters that are not well separated, even if they overlap
  - ▶ DBSCAN merges clusters that overlap
- ▶ Stability
  - ▶ K-means' result depends on the random initialization of centroids
  - ▶ DBSCAN produces the same set of clusters from one run to another (except that the membership of some border points depends on the order of the points being processed)
- ▶ Number of clusters
  - ▶ For k-means, the number of clusters needs to be specified as a parameter
  - ▶ DBSCAN automatically determines the number of clusters. However, it has two other parameters: *Eps* and *MinPts*

# DBSCAN: Determining Eps Given MinPts

- ▶ Let  $\text{MinPts} = k$
- ▶ Idea is that for points in a cluster, their  $k^{\text{th}}$  nearest neighbors are at roughly the same distance
- ▶ Noise points have the  $k^{\text{th}}$  nearest neighbor at farther distance
- ▶ So, plot sorted distance of every point to its  $k^{\text{th}}$  nearest neighbor (e.g.,  $k=4$ )

Thus,  $\text{Eps}=10$  for  $\text{MinPts}=4$



# Comparing to Hierarchical Clustering

## ► Key differences

| Feature            | DBSCAN                                      | Hierarchical Method                         |
|--------------------|---------------------------------------------|---------------------------------------------|
| Cluster Shape      | Arbitrary (work well for non-convex shapes) | Depends on linkage                          |
| Noise Handling     | Explicitly identifies noise                 | No inherent noise handling                  |
| Scalability        | $O(n \log n)$ with spatial indexing         | $O(n^3)$ or $O(n^2 \log n)$                 |
| Parameters         | Eps, MinPts                                 | Number of clusters or distance threshold    |
| Hierarchy          | No hierarchy                                | Provides a dendrogram                       |
| Number of Clusters | Automatic                                   | Specified (if not using distance threshold) |

# Pros and Cons of DBSCAN

## ▶ **Pros:**

- ▶ Handles noise and outliers well
- ▶ Discovers clusters of arbitrary shapes
- ▶ No need to predefine the number of clusters

## ▶ **Cons:**

- ▶ Struggles with varying densities
- ▶ Sensitive to Eps and MinPts

# Outline

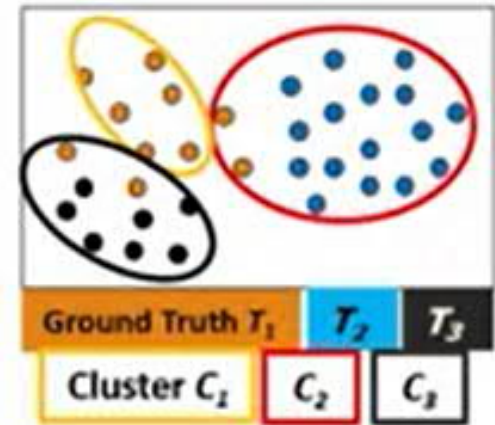
- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures

# Measures of Clustering Quality

- ▶ Two types of numerical measures:
  - ▶ **External Index:** Used to measure the extent to which cluster labels match externally supplied class labels.
    - ▶ The true class labels are *external* to the data set
    - ▶ E.g., Rand statistic, Jaccard, entropy
  - ▶ **Internal Index:** Used to measure the goodness of a clustering structure *without* external information.
    - ▶ E.g., Sum of Squared Error (SSE), Davies-Bouldin (DB) index
- ▶ These are also referred to as **criteria**
  - ▶ However, sometimes criterion is the general strategy and index is the numerical measure that implements the criterion.

# External Indexes

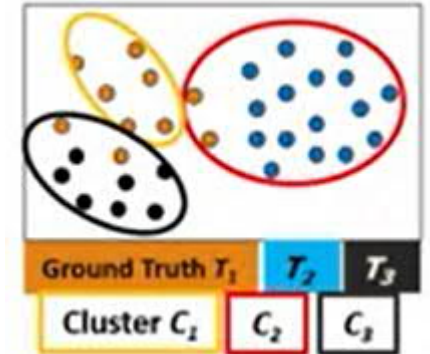
- ▶ Ground truth is given
  - ▶ *True partitions/classes of the data points are known.*
  - ▶ Clustering result is compared with the true partitions



# True Positives, True Negatives, False Positives and False Negatives

## ▶ True Positives

- ▶  $x_i$  and  $x_j$  are a *true positive pair* if they belong to the same partition and are also in the same cluster
- ▶ TP is the number of true positive pairs

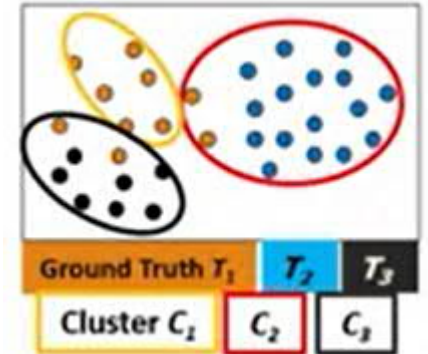


## ▶ False Positives

- ▶  $x_i$  and  $x_j$  are a *false positive pair* if they do not belong to the same partition, but are clustered into the same cluster
- ▶ FP is the number of false positive pairs



# True Positives, True Negatives, False Positives and False Negatives



## ▶ True Negative

- ▶  $x_i$  and  $x_j$  are a *true negative pair* if they do not belong to the same partition and are also not in the same cluster
- ▶ TN is the number of true negative pairs

## ▶ False Negative

- ▶  $x_i$  and  $x_j$  are a *false negative pair* if they belong to the same partition, but are clustered into different clusters
- ▶ FN is the number of false negative pairs

# Rand Statistic/Index

- ▶ Rand Statistic/Index (also called HA Index)
  - ▶ Measures the fraction of true positives and true negatives over all pairs

$$Rand = \frac{TP + TN}{TP + FP + TN + FN}$$

- ▶ Measures the fraction of point pairs where both clustering result and the ground truth agree.
  - ▶ A perfect clustering has a value of 1 for the statistic.

## Jaccard Coefficient

- ▶ Measures the fraction of true positive point pairs, but after ignoring the true negatives:

$$Jaccard = \frac{TP}{TP + FP + FN}$$

- ▶ For a perfect clustering, the coefficient is 1, that is, there are no false positives nor false negatives

---

# Internal Indexes

- ▶ Based on data only, without consulting *external* true partitions
- ▶ Normally consider intra-cluster similarity or **compactness** contrasted with the notations of inter-cluster **separation**
- ▶ Typically propose a trade-off to maximizing these two competing measures.

## SSE: an Internal Index

- ▶ Sum of Squared Error (SSE):

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \text{dist}(x, v_i)^2$$

$v_i$  is the center of the cluster  $C_i$

$k$  is the number of clusters

- ▶ Considers only the compactness (i.e., the intra-cluster distances) of the clusters

## DB: an Internal Index

- ▶ Davies-Bouldin index (DB) :

$$DB = \frac{1}{k} \sum_i \max_{j=1..k, j \neq i} (d_{ij}) \quad \text{where} \quad d_{ij} = \frac{\sigma_i + \sigma_j}{d(v_i, v_j)}$$

- ▶  $k$  is the number of clusters,
- ▶  $\sigma_i$  is the average distance between cluster points and the center in the  $i$ th cluster
- ▶  $d(v_i, v_j)$  is the distance between the  $i$ th and  $j$ th cluster centers.
- ▶  $d_{ij}$  decreases when the clusters are more compact and when the distance between them is larger.
- ▶ For each cluster  $C_i$ , the  $\max(d_{ij})$  identifies its “worst” neighbour. The DB index is the average of such value for all clusters.
- ▶ The DB index varies on the interval  $[0, \infty)$  and is *small* when the clusters are *compact and well separated*.

## Final Comment on Cluster Validity

“The validation of clustering structures is the most difficult and frustrating part of cluster analysis.

Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage.”

*Algorithms for Clustering Data*, Jain and Dubes

# Summary

- ▶ What is Clustering?
- ▶ Types of Data to Cluster
- ▶ Distance/ Similarity Measures
- ▶ Clustering Methods
  - ▶ K-means and K-means++
  - ▶ K-medoids
  - ▶ Hierarchical clustering method
  - ▶ DBSCAN
- ▶ Cluster Validity Measures